Patrones de diseño en Java

17 patrones del GoF, con diagramas UML, construyendo Emberdeep, un juego de mazmorras por turnos

Fábrica de Cursos
27 de octubre de 2026

Índice general

Cómo usar este libro **vii**

I Punto de partida y primer patrón **1**

1. Del dolor al patrón: el núcleo de Emberdeep **2**

1.1. Qué es un patrón (y qué no) . . . 2
1.2. Cómo se lee cada capítulo . . . 3
1.3. Leer un diagrama de clases UML . . . 3
1.4. Emberdeep y su estructura de carpetas . . . 5
1.4.1. Capas arriba, funcionalidad dentro del motor . . . 5
1.4.2. Composition root . . . 5
1.4.3. Un proyecto Gradle . . . 5
1.5. En el proyecto: H1, el núcleo sin patrones . . . 7
1.5.1. Las clases de datos y Fighter . . . 7
1.5.2. El resto del núcleo . . . 8
1.6. Seis olores que ya se ven en el núcleo . . . 10
1.7. Ejercicios . . . 11
1.8. Resumen . . . 14
Soluciones de los ejercicios . . . 15

2. Strategy: IA del enemigo y fórmula de daño **27**

2.1. El olor que pide Strategy: los `if` por nombre . . . 27
2.2. Piezas del patrón Strategy . . . 28
2.3. El ejemplo de Strategy, pieza a pieza: la IA del enemigo . . . 29
2.3.1. La estrategia: `EnemyAi` . . . 29
2.3.2. Una estrategia concreta: `FirstAliveAi` . . . 29
2.3.3. El portador y el contexto: `Enemy` y `Fighter` . . . 30
2.3.4. El resultado en Combat . . . 31
2.3.5. Aleatoriedad repetible: Random con semilla inyectada . . . 31
2.4. Strategy con lambdas: la fórmula de daño . . . 32
2.4.1. Una interfaz funcional . . . 32
2.4.2. Una versión de clase y otra de lambda . . . 32
2.4.3. Clase o lambda . . . 33

2.5. Strategy: cuándo sí y cuándo no . . . 34
2.6. En el proyecto: H2 con Strategy . . . 34
2.7. Ejercicios . . . 36
2.8. Resumen . . . 39
Soluciones de los ejercicios . . . 40

II Crear y compartir objetos 55

3. Factory Method y Abstract Factory: enemigos y objetos por bioma 56
3.1. El olor que pide Factory Method: `new` por todas partes . . . 56
3.2. Piezas del patrón Factory Method . . . 57
3.3. El ejemplo de Factory Method, pieza a pieza: los spawners . . . 58
3.3.1. El producto: `Enemy` gana un `kind` . . . 58
3.3.2. El creador: `EnemySpawner` . . . 58
3.3.3. Los creadores concretos . . . 59
3.3.4. El cliente: `GameSession` . . . 60
3.4. El olor que pide Abstract Factory: un esqueleto con una poción de caverna . . . 61
3.5. Piezas del patrón Abstract Factory . . . 61
3.6. El ejemplo de Abstract Factory, pieza a pieza: los biomas . . . 62
3.6.1. Los productos: `Item` e `ItemKind` . . . 62
3.6.2. La fábrica abstracta y las dos fábricas concretas . . . 63
3.6.3. El cliente con una sola fábrica . . . 65
3.6.4. Por qué los spawners se retiran . . . 65
3.7. Si ya lo conoces: lo nuevo en Factory Method y Abstract Factory . . . 66
3.8. Factory Method y Abstract Factory: cuándo sí y cuándo no . . . 67
3.9. En el proyecto: H3 con Factory Method y Abstract Factory . . . 68
3.9.1. El inventario: un record inmutable . . . 68
3.10. Ejercicios . . . 70
3.11. Resumen . . . 75
Soluciones de los ejercicios . . . 76

4. Builder y Singleton: héroes y mazmorra paso a paso 90
4.1. El olor que pide Builder: un constructor con ocho parámetros . . . 90
4.2. Piezas del patrón Builder . . . 91
4.3. El ejemplo de Builder, pieza a pieza: `HeroBuilder` . . . 91
4.4. Si ya lo conoces: Builder y `record` . . . 92
4.5. Builder anidado: `DungeonBuilder` y `FloorBuilder` . . . 93
4.6. El olor que pide Singleton: la configuración que todos leen . . . 94
4.7. Piezas del patrón Singleton . . . 95

4.8. El ejemplo de Singleton, pieza a pieza: holder y `enum` . . . 95
4.8.1. Primer intento: perezoso, y con un fallo . . . 95
4.8.2. Solución 1: el holder . . . 96
4.8.3. Solución 2: el `enum` . . . 97
4.9. Si ya lo conoces: lo nuevo en Singleton es medir su coste . . . 97
4.10. Singleton: cuándo sí y cuándo no; la alternativa de `Main` . . . 97
4.11. En el proyecto: H4 con Builder y Singleton . . . 98
4.12. Ejercicios . . . 99
4.13. Resumen . . . 105
Soluciones de los ejercicios . . . 106

5. Prototype y Flyweight: copiar o compartir 120
5.1. El olor que pide Prototype: montar la misma plantilla cien veces . . . 120
5.2. Piezas del patrón Prototype . . . 121
5.3. El ejemplo de Prototype, pieza a pieza: `EnemyTemplate.copy()` . . . 122
5.4. Prototype: el bug de la copia superficial . . . 125
5.5. Si ya lo conoces: lo nuevo en Prototype es la copia profunda y el registro . . . 126
5.6. El olor que pide Flyweight: un `TileType` por cada casilla . . . 126
5.7. Piezas del patrón Flyweight . . . 127
5.8. El ejemplo de Flyweight, pieza a pieza: `TileTypes` . . . 128
5.9. Prototype, Factory y Flyweight: copiar, construir o compartir . . . 130
5.10. Prototype y Flyweight: cuándo sí y cuándo no . . . 130
5.11. En el proyecto: H5 y H6 con Prototype y Flyweight . . . 131
5.12. Ejercicios . . . 132
5.13. Resumen . . . 138
Soluciones de los ejercicios . . . 139

III Ampliar el comportamiento 164

6. Decorator y Chain of Responsibility: apilar o encadenar 165
6.1. El olor que pide Decorator: una subclase por cada combinación . . . 165
6.2. Piezas del patrón Decorator . . . 166
6.3. El ejemplo de Decorator, pieza a pieza: `StatSource` . . . 167
6.4. El olor que pide Chain of Responsibility: un método de daño con seis `if` . . . 169
6.5. Piezas del patrón Chain of Responsibility . . . 170
6.6. El ejemplo de Chain of Responsibility, pieza a pieza . . . 171
6.7. Decorator, Chain of Responsibility y Strategy: envolver, encadenar o elegir . . . 174
6.8. Decorator y Chain of Responsibility: cuándo sí y cuándo no . . . 175
6.9. En el proyecto: H7 y H8 con Decorator y Chain of Responsibility . . . 175

6.10. Ejercicios . . . 178
6.11. Resumen . . . 186
Soluciones de los ejercicios . . . 186

7. Command: acciones como objetos, deshacer y replay 215
7.1. El olor que pide Command: acciones que no se pueden guardar . . . 215
7.2. Piezas del patrón Command . . . 217
7.3. El ejemplo de Command, pieza a pieza (1): ejecutar . . . 217
7.4. Command con deshacer (2): la pila . . . 220
7.5. Command y el modo guion (3): replay . . . 223
7.6. Command: cuándo sí y cuándo no . . . 225
7.7. En el proyecto: H9 con Command . . . 225
7.8. Ejercicios . . . 228
7.9. Resumen . . . 240
Soluciones de los ejercicios . . . 241

8. Memento: instantáneas del estado, con JUnit 282
8.1. El olor que pide Memento: deshacer cuando hay cien cambios . . . 282
8.2. Piezas del patrón Memento . . . 284
8.3. El ejemplo de Memento, pieza a pieza . . . 285
8.4. Memento: el memento que guarda referencias mutables . . . 288
8.5. Memento o Command: dos formas de deshacer . . . 289
8.6. Instalación y primeros pasos con JUnit . . . 289
8.7. Memento: cuándo sí y cuándo no . . . 293
8.8. En el proyecto: H15a con Memento . . . 294
8.9. Ejercicios . . . 297
8.10. Resumen . . . 304
Soluciones de los ejercicios . . . 305

IV Comunicar 334

9. Observer: un motor que emite eventos 335
9.1. El olor que pide Observer: el motor sabe demasiado de la consola . . . 335
9.2. Piezas del patrón Observer . . . 336
9.3. El ejemplo de Observer, pieza a pieza: `EventBus` . . . 338
9.4. Observer: eventos `sealed` y `switch` . . . 339
9.5. Observer: el oyente que se da de baja mientras se notifica . . . 342
9.6. Observer: cuándo sí y cuándo no . . . 343
9.7. En el proyecto: H10 con Observer . . . 344
9.8. Ejercicios . . . 346

9.9. Resumen . . . 356
Soluciones de los ejercicios . . . 357

10. Bridge: vistas y salidas que crecen por separado 394
10.1. El olor que pide Bridge: una clase por cada combinación . . . 394
10.2. Piezas del patrón Bridge . . . 396
10.3. El ejemplo de Bridge, pieza a pieza: `HudView` y `Output` . . . 397
10.4. Si ya lo conoces: lo nuevo en Bridge es separar vistas y salidas . . . 401
10.5. Bridge y los colores ANSI escritos a mano . . . 401
10.6. Bridge y Strategy: dos diagramas parecidos, dos problemas . . . 402
10.7. Bridge: cuándo sí y cuándo no . . . 403
10.8. En el proyecto: H11 con Bridge . . . 404
10.9. Ejercicios . . . 407
10.10Resumen . . . 412
Soluciones de los ejercicios . . . 413

11. Mediator: la arena que coordina el combate 444
11.1. El olor que pide Mediator: héroes y enemigos con referencias entre sí . . . 444
11.2. Piezas del patrón Mediator . . . 446
11.3. El ejemplo de Mediator, pieza a pieza: `taunt` y `healAlly` . . . 448
11.4. Mediator y Observer: quién decide . . . 452
11.5. Mediator: el riesgo del «dios» y cómo evitarlo . . . 453
11.6. Mediator: cuándo sí y cuándo no . . . 454
11.7. En el proyecto: H12 con Mediator . . . 454
11.8. Ejercicios . . . 457
11.9. Resumen . . . 465
Soluciones de los ejercicios . . . 466

V Estados, estructura y cierre 516

12. State: estados de los personajes y fases de la partida 517
12.1. El olor que pide State: banderas y combinaciones imposibles . . . 517
12.2. Piezas del patrón State . . . 519
12.3. El ejemplo de State, pieza a pieza: `Status` . . . 520
12.4. State para el ciclo de la partida: `GamePhase` . . . 522
12.5. State: cuándo sí y cuándo no . . . 524
12.6. En el proyecto: H13 con State . . . 525
12.7. Ejercicios . . . 527
12.8. Resumen . . . 534
Soluciones de los ejercicios . . . 535

13. Composite: oleadas e inventario anidados **591**
13.1. El olor que pide Composite: bolsas dentro de bolsas con bucles anidados 591
13.2. Piezas del patrón Composite 592
13.3. El ejemplo de Composite, pieza a pieza: `InventoryNode` 593
13.4. Composite para oleadas: `WaveNode` 597
13.5. Composite con sealed y switch frente al Visitor clásico 598
13.6. Composite: cuándo sí y cuándo no 599
13.7. En el proyecto: H14 con Composite 600
13.8. Ejercicios 602
13.9. Resumen 607
Soluciones de los ejercicios 608

14. Facade y el guardado en fichero: cerrar el juego **644**
14.1. El olor que pide Facade: `Main` conoce doce clases del motor 644
14.2. Piezas del patrón Facade 645
14.3. El ejemplo de Facade, pieza a pieza: `GameEngine` 647
14.4. Si ya lo conoces: lo nuevo en Facade es frente a Mediator y a las capas 649
14.5. El Memento llega al disco: guardar y cargar con `clave=valor` 650
14.6. Facade: cuándo sí y cuándo no 655
14.7. Elegir y defender el patrón 656
14.8. En el proyecto: H15b con Facade 657
14.9. Ejercicios 660
14.10Resumen 668
Soluciones de los ejercicios 669

A. Guía visual de consulta **705**
A.1. Los diecisiete patrones de un vistazo 705
A.2. Cuatro contrastes para no confundirlos 708
A.2.1. Decorator, Chain of Responsibility y Strategy (cap. 6) 708
A.2.2. Prototype, Factory y Flyweight (cap. 5) 708
A.2.3. Bridge y Strategy (cap. 10) 708
A.2.4. Mediator y Observer (cap. 11) 709
A.3. Del olor al patrón: la checklist 709
A.3.1. Señales de sobrediseño 710
A.3.2. Checklist de revisión de código 710
A.4. Mirada atrás: dónde vive cada patrón en Emberdeep 711

Referencias **713**

Cómo usar este libro

A quién va dirigido

Este libro es para quien ya programa en Java moderno y quiere aprender los patrones de diseño del catálogo GoF[9] para reconocerlos y aplicarlos en su trabajo, y poder defender la decisión en una *code review*.

Damos por sabido lo siguiente:

- Java 21: *records*, interfaces `sealed`, *pattern matching*, `switch` con patrones y lambdas. No hay repaso.
- Los principios SOLID. Tampoco se repasan.
- Singleton, Builder y Factory: seguramente los conoces. Aun así tienen su capítulo, con un enfoque nuevo, y una sección «Si ya lo conoces: lo nuevo aquí es...» que te dice qué repasas y qué es nuevo.

No necesitas conocer ningún framework. Este libro no usa Spring ni ningún otro, ni inyección de dependencias automática, ni Lombok: las dependencias se pasan a mano por el constructor.

Qué vas a aprender

Aprenderás 17 patrones del GoF con profundidad: cuándo aparecen (el «olor» en el código), qué piezas tienen, cómo se escriben y cuándo *no* conviene usarlos. Los capítulos se ordenan por el problema del juego, no por familia del catálogo, en cinco partes:

- **Parte I. Punto de partida y primer patrón** (caps. 1-2): el núcleo de Emberdeep sin patrones y Strategy.
- **Parte II. Crear y compartir objetos** (caps. 3-5): Factory Method, Abstract Factory, Builder, Singleton, Prototype y Flyweight.
- **Parte III. Ampliar el comportamiento** (caps. 6-8): Decorator, Chain of Responsibility, Command y Memento.
- **Parte IV. Comunicar** (caps. 9-11): Observer, Bridge y Mediator.
- **Parte V. Estados, estructura y cierre** (caps. 12-14): State, Composite y Facade, con guardado de la partida en fichero.

Cuenta con unas 25-28 horas, a razón de hora y media a dos horas por capítulo. Los capítulos 5 y 6 tratan dos patrones cada uno y son los más largos.

Al terminar sabrás reconocer el patrón adecuado, aplicarlo y explicar por qué es mejor que la alternativa más simple. El libro sirve también como guía de consulta: el apéndice final reúne una tabla con los 17 patrones para recordar de un vistazo qué hace cada uno y cómo se usa.

Cómo se lee cada patrón

Cada patrón sigue siempre la misma ficha, para que sepas dónde buscar:

1. **El olor que pide el patrón.** El problema, visto en el código de Emberdeep antes de aplicar nada.
2. **Piezas del patrón.** Una tabla y un diagrama UML.
3. **El ejemplo, pieza a pieza.** El código, explicado parte por parte.
4. **Cuándo sí y cuándo no.** Cuándo compensa y cuándo es sobrediseño.
5. **Contraste**, cuando el patrón se confunde con otro (por ejemplo, Decorator frente a Chain of Responsibility).
6. **En el proyecto.** Cómo avanza Emberdeep con ese patrón.

El nombre del patrón aparece en el título de cada sección que trata de él. Los patrones que quizá ya conoces (Factory, Builder, Singleton, Prototype, Bridge y Facade) llevan además una sección «Si ya lo conoces: lo nuevo aquí es...».

La tabla de piezas tiene cuatro columnas. Así se lee, con el patrón Strategy como ejemplo:

Rol	Clase en el ejemplo	Propósito	Paquete
Estrategia (interfaz)	`EnemyAi`	Elige objetivo	`engine.combat.ai`
Contexto (clase)	`Fighter`	Usa la estrategia	`engine.character`

- **Rol**: el nombre que le da el patrón a la pieza y su tipo en Java (interfaz, clase, *record*...).
- **Clase en el ejemplo**: cómo se llama esa pieza en el código de Emberdeep.
- **Propósito**: para qué sirve, en pocas palabras.
- **Paquete**: dónde vive en el proyecto.

Las piezas que construyes tú en un ejercicio de proyecto, y no el texto, se marcan con «(ejercicio)».

Cómo leer los diagramas UML

Cada patrón lleva un diagrama de clases UML para entenderlo de un vistazo. El capítulo 1 trae la leyenda completa: aquí tienes solo el mapa de símbolos, que usarás en todo el libro. Los diagramas muestran solo lo relevante para el patrón, no todos los atributos y métodos.

Símbolo	Significa
Caja con tres compartimentos	Clase: nombre, atributos y métodos.
Caja con «interface»	Interfaz: solo métodos.
Caja amarilla	Nota con una aclaración.
Línea con triángulo hueco	Herencia (`extends`).
Línea discontinua con triángulo hueco	Realización (`implements`).
Flecha abierta	Asociación: una clase guarda una referencia a otra.
Rombo hueco	Agregación: el todo agrupa partes que pueden vivir solas.
Rombo lleno	Composición: las partes no existen sin el todo.
Flecha discontinua	Dependencia: una clase usa a otra sin guardarla.

Cómo se dibujan los diagramas

Los diagramas están dibujados con TikZ[25], que ya forma parte del libro, y no con una herramienta externa. A ti no te hace falta saber TikZ: solo leerlos.

Los diagramas de contraste se colocan lado a lado, para comparar a la vez la forma de dos patrones parecidos.

El proyecto del curso

Construirás **Emberdeep**, un juego de mazmorras por turnos para la consola. Un grupo de unos tres héroes baja por una mazmorra de unas pocas plantas y lucha contra enemigos con cuatro acciones: atacar, defender, usar un objeto y lanzar una habilidad. Al terminar el curso tendrás un juego completo, con equipo y mejoras temporales, una tubería de daño, deshacer, repetición de partidas, logros, estados, oleadas de enemigos y guardado de la partida en un fichero.

El juego es el pretexto: cada patrón entra porque Emberdeep lo necesita, no porque toque. Las reglas son mínimas a propósito, para que lo difícil sean los patrones y no el dominio. En el capítulo 1 escribes un núcleo jugable sin ningún patrón; después, cada capítulo avanza el proyecto un hito y parte exactamente del estado que dejó el anterior. El orden de los hitos no es siempre el de su número: por ejemplo, los puntos de control de Memento (hito H15a) llegan en el capítulo 8, antes del hito H10.

Emberdeep vive en una carpeta `emberdeep/`, aparte del libro, y se organiza en tres paquetes (`app`, `engine` y `ui`). El motor no imprime nada; la capa `ui` pinta lo que el motor emite. Cada vez que la estructura de carpetas cambia, el libro te muestra el árbol actualizado.

Cómo ejecutar Emberdeep

El proyecto usa Gradle y no tiene dependencias en tiempo de ejecución; la única librería que aparecerá es JUnit, a partir del capítulo 8. El capítulo 1 crea los archivos de Gradle y genera el *wrapper*. Desde la carpeta `emberdeep/`, el juego se arranca así:

```
./gradlew run
```

Más adelante, el modo «guion» lee las acciones de un archivo en lugar del teclado, y se lanza con `./gradlew run -args="scripts/demo.txt"`. El capítulo donde aparece lo explica con detalle.

El código Java del libro no se compila al escribirlo: lo escribes tú y lo ejecutas en tu máquina. Por eso conviene que, al seguir cada capítulo, ejecutes el proyecto con frecuencia y compares tu resultado con el que se indica.

Los ejercicios

Cada capítulo termina con ejercicios. Todos se pueden resolver con lo que has visto hasta ese punto del libro: si un ejercicio necesita una herramienta o una dependencia, el libro te habrá explicado antes cómo instalarla y usarla. Cada ejercicio indica su dificultad:

Dificultad	Qué significa
★☆☆☆☆	Reproducir: aplicar lo que acabas de ver siguiendo los pasos que se indican.
★★☆☆☆	Aplicar con guía: usar un concepto nuevo en un caso parecido, con pistas.
★★★☆☆	Aplicar sin guía: resolver un problema nuevo combinando lo visto en el capítulo.
★★★★☆	Integrar: combinar conceptos de varios capítulos y tomar decisiones de diseño.
★★★★★	Diseñar: problema abierto con varias soluciones válidas; se da una de referencia y criterios para valorar la tuya.

y su tipo:

Tipo	En qué consiste
Guiado	Pasos numerados que te llevan de la mano.
Práctica	Resolver un problema concreto con lo aprendido.
Proyecto	Avanzar el proyecto del curso un hito más.
Diseño	Decidir cómo estructurar algo y justificarlo.
Depuración	Encontrar y corregir el fallo de un código dado.
Lectura de código	Leer código y responder preguntas sobre él.
Test	Escribir o completar pruebas automáticas.

Cada capítulo incluye un ejercicio de tipo proyecto, que avanza Emberdeep. Si tu código del capítulo anterior difiere del que usa el libro, compáralo con la solución del ejercicio de proyecto anterior antes de seguir.

Las soluciones completas están al final de cada capítulo, en la sección *Soluciones de los ejercicios*. Cada enunciado enlaza con su solución y cada solución con su enunciado. Intenta resolver el ejercicio antes de mirarla: la solución es más útil cuando la comparas con la tuya.

Qué necesitas instalar para empezar

Para empezar solo necesitas lo que ya tienes: JDK 21, Gradle, Git y un editor con soporte para Java (en este curso, NeoVim). El libro no explica esas instalaciones. Antes del capítulo 1 comprueba que funcionan:

```
java -version
gradle --version
git --version
```

`java -version` debe indicar la versión 21. Cualquier otra herramienta que hiciera falta se instala en el capítulo donde se usa por primera vez (la única prevista es JUnit, en el capítulo 8).

Referencias

El libro se apoya en libros, artículos y documentación que puedes consultar para profundizar. Aparecen citados entre corchetes, como [9], y la lista completa está al final, en *Referencias*.

Parte I

Punto de partida y primer patrón

CAPÍTULO 1

Del dolor al patrón: el núcleo de Emberdeep

Objetivos del capítulo

- Explicar qué es un patrón de diseño y qué no es, y cómo se lee la ficha de cada capítulo.
- Leer un diagrama de clases UML: clases, interfaces y las seis relaciones que usa este libro.
- Crear un proyecto Gradle con Java 21, wrapper y entrada por teclado.
- Organizar el código en tres paquetes (`app`, `engine`, `ui`) con un motor que no imprime.
- Dar el primer paso del proyecto: un combate por turnos jugable, sin patrones, y localizar sus olores.

Un patrón de diseño no se aprende como se aprende una librería: se aprende a reconocer el dolor que lo pide. Por eso este capítulo no enseña ningún patrón. Primero construyes Emberdeep a la manera más directa, con `if` y listas, y después miras con calma dónde empieza a doler. Cada capítulo siguiente cura uno de esos dolores con un patrón. Por el camino aprendes lo que necesitas para leer el resto del libro: la ficha de cada patrón, los diagramas UML y la estructura del proyecto.

1.1 Qué es un patrón (y qué no)

Un *patrón de diseño* es una solución con nombre a un problema que se repite al diseñar software orientado a objetos. Tiene tres partes: el problema (cuándo aparece), la solución (qué piezas colaboran y cómo) y las consecuencias (qué ganas y qué pagas). La idea viene de la arquitectura de edificios, y Beck y Cunningham la llevaron a la programación orientada a objetos en 1987 [2]. Los cuatro autores del catálogo que seguimos en este libro lo presentaron primero en un artículo sobre abstracción y reutilización del diseño [8] y después en el libro que lo hizo famoso [9], con 23 patrones. Aquí estudiamos 17, los que Emberdeep necesita de verdad.

El nombre es la parte más útil en el trabajo diario. En una *code review*, decir «esto pide un Strategy» comunica en tres palabras lo que de otro modo llevaría un párrafo, y obliga a defender la propuesta con los términos del propio patrón.

Un patrón **no** es:

- **Una librería ni código para copiar.** Cada patrón se escribe de forma distinta según el lenguaje y el problema. En Java 21 muchos son más cortos que hace veinte años gracias a los *records*, las interfaces `sealed` y las lambdas.
- **Un objetivo.** Un programa no es mejor por llevar más patrones. Cada uno añade clases e indirección, y esa indirección solo compensa si resuelve un problema que ya tienes.
- **Un sustituto de entender el problema.** El patrón es la respuesta; si no sabes cuál es la pregunta, lo aplicarás donde no toca.

Patrón sin dolor es sobrediseño

La regla de este libro: primero el olor (el síntoma en el código), después el patrón. Si no puedes señalar la línea que duele, todavía no necesitas el patrón.

Para una primera lectura más ligera

Si quieres una explicación muy visual y motivada por problemas, *Head First Design Patterns* [7] cubre buena parte del catálogo con otro estilo. Este libro es más breve por patrón y busca que puedas volver a él para consultar.

1.2 Cómo se lee cada capítulo

El capítulo «Cómo usar este libro» describe la ficha de cada patrón. Aquí la tienes como mapa de consulta: qué pregunta contesta cada apartado y cuándo lo usarás.

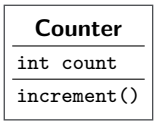
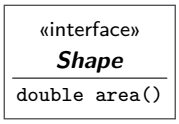
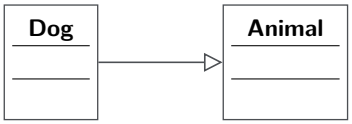
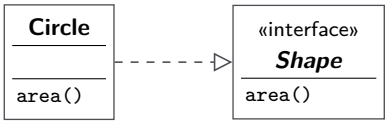
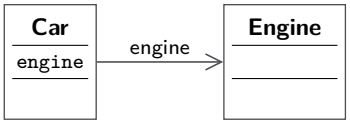
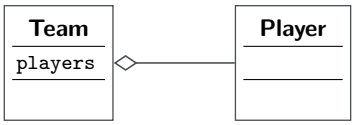
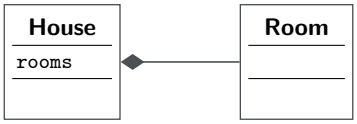
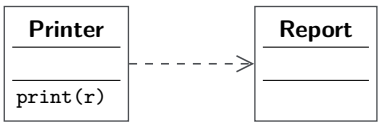
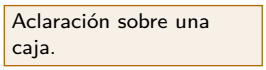
Apartado	Pregunta que contesta	Lo usas para
El olor que pide X	¿Qué síntoma del código lo pide?	Decidir si lo necesitas.
Piezas del patrón X	¿Qué roles hay y cómo se relacionan?	Recordarlo de un vistazo (tabla y UML).
El ejemplo de X, pieza a pieza	¿Cómo se escribe en Java 21?	Copiar la forma, no el código.
X: cuándo sí y cuándo no	¿Compensa aquí?	Defenderlo en una code review.
Contraste	¿En qué se diferencia del patrón parecido?	No confundirlos.
En el proyecto: Hn con X	¿Cómo avanza Emberdeep?	Practicar con el ejercicio de proyecto.

Este capítulo es la excepción: no tiene patrón, así que solo tiene el proyecto y el anticipo de los olores. Cada ejercicio indica con qué se parte. Los que no son de tipo *proyecto* usan clases propias y no tocan Emberdeep; solo el ejercicio de proyecto cambia el código del juego, y su solución es el punto de partida del capítulo siguiente.

1.3 Leer un diagrama de clases UML

Cada patrón lleva un diagrama de clases para verlo de un vistazo. La notación completa está en la especificación de UML [20]; para leer los diagramas de este libro basta una guía breve [6], y aquí tienes la tuya. Los diagramas se dibujan con TikZ [25] y tú solo los lees. Muestran lo relevante para el patrón, no todos los miembros de cada clase.

La tabla recoge todos los símbolos, con su equivalente en Java. Las cajas de ejemplo no son de Emberdeep: son clases mínimas para ver el símbolo solo.

Dibujo	Significa	En Java
	**Clase**: nombre, atributos y métodos. Un compartimento vacío no tiene miembros.	`class Counter { int count; void increment() }`
	**Interfaz**: «interface» y nombre en cursiva; solo métodos.	`interface Shape { double area(); }`
	**Herencia**: línea continua con triángulo hueco hacia la clase base.	`class Dog extends Animal`
	**Realización**: línea discontinua con triángulo hueco hacia la interfaz.	`class Circle implements Shape`
	**Asociación**: flecha abierta; la clase de origen guarda una referencia.	`private final Engine engine;`
	**Agregación**: rombo hueco en el todo; las partes vienen de fuera y viven solas.	`Team(List<Player> players)`: el equipo recibe jugadores ya creados.
	**Composición**: rombo lleno en el todo; las partes nacen y mueren con él.	`rooms.add(new Room())` dentro de `House`.
	**Dependencia**: flecha discontinua; se usa un tipo sin guardarlo.	`void print(Report r)`: `Report` solo es un parámetro.
	**Nota**: caja amarilla con un comentario.	Un comentario `//` en el código.

Cuatro reglas para leerlos:

1. Lo abstracto (interfaces y clases base) suele ir arriba; sus implementaciones, debajo.
2. La punta de la flecha señala lo que se conoce o se usa. Quien está en la cola *depende* de quien está en la punta: si cambia la punta, puede romperse la cola.
3. En agregación y composición el rombo está en el todo, y el todo conoce a sus partes.
4. Cuando una caja de interfaz aparece con varias flechas de realización, esas clases son intercambiables para quien solo conoce la interfaz. Esa es la razón de ser de casi todos los patrones.

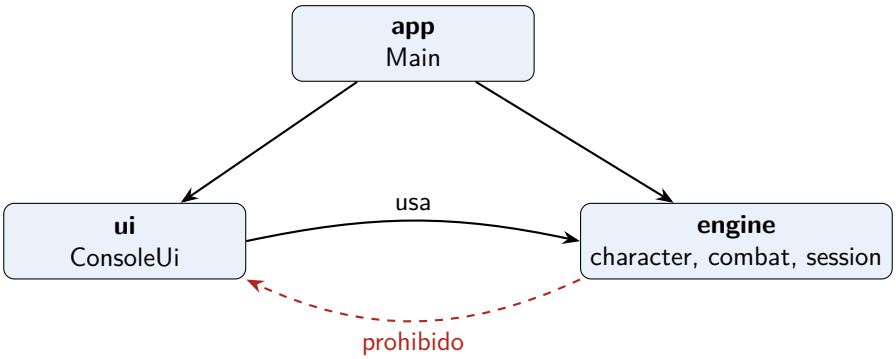
Lo practicarás con el diagrama del núcleo de Emberdeep (sección 1.5) y en el ejercicio 1.1.

1.4 Emberdeep y su estructura de carpetas

1.4.1 Capas arriba, funcionalidad dentro del motor

El código de Emberdeep se organiza con un único principio: **capas arriba, funcionalidad dentro del motor**. Hay tres paquetes de primer nivel:

- `engine`: las reglas del juego (héroes, combate, sesión). El motor no imprime, no lee de la consola y no importa nada de `ui`.
- `ui`: lo que el jugador ve y teclea. Depende del motor.
- `app`: el punto de arranque. Conoce a los dos y los une.



Dependencias entre paquetes: la flecha va de quien depende a quien es usado.

Dentro de `engine` se agrupa por funcionalidad del juego (`character`, `combat`, `session`...), nunca por patrón: el patrón vive en el paquete del problema que resuelve. Así se encuentra el código sabiendo qué hace, no cómo está hecho. La razón de que el motor no imprima es práctica: devuelve mensajes y datos, y quien quiera pintarlos (la consola hoy, otra salida mañana) lo hace desde `ui`.

1.4.2 Composition root

Emberdeep no usa ningún framework que cree objetos por ti. Las dependencias se pasan a mano por el constructor, y hay un único lugar donde se montan todas: el método `main`. Ese lugar se llama *composition root* (raíz de composición): crea los objetos, los conecta y arranca el juego. Ninguna otra clase hace `new` de sus colaboradores principales. Es una convención, no un patrón del catálogo, y te será útil en todos los capítulos.

1.4.3 Un proyecto Gradle

Emberdeep se construye con Gradle (DSL Groovy). Ya tienes Gradle instalado; lo que haces aquí es crear el proyecto: dos archivos de configuración a mano y el *wrapper*, un pequeño script que descarga y usa la versión de Gradle que fija el proyecto, para que cualquiera lo compile igual.

☞ Proyecto Gradle con Java 21 y wrapper

1. Carpetas. Crea la raíz del proyecto y los paquetes de la sección anterior:

```
$ mkdir -p emberdeep/src/main/java/emberdeep/{app,engine/character,engine/
    ↪ combat,engine/session,ui}
$ cd emberdeep
```

2. `settings.gradle`. Solo da nombre al proyecto:

settings.gradle

```
rootProject.name = 'emberdeep'
```

3. `build.gradle`. Aplica el plugin `application`, fija Java 21 y conecta el teclado:

build.gradle

```
plugins {
    id 'application'
}

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

application {
    mainClass = 'emberdeep.app.Main'
}

run {
    standardInput = System.in
}
```

4. Wrapper. Con el Gradle instalado, genera el wrapper fijando la versión:

```
$ gradle wrapper --gradle-version 9.8.2
$ ./gradlew --version
```

La segunda orden debe mostrar `Gradle 9.8.2`. La primera vez, `./gradlew` descarga esa versión (hace falta conexión). Se crean `gradlew`, `gradlew.bat` y la carpeta `gradle/wrapper/`.

Qué hace cada pieza:

- `id 'application'` añade la tarea `run` y el bloque `application`, donde `mainClass` indica la clase con el método `main`. Sin él, Gradle no sabe qué ejecutar.
- `java { toolchain { ...} }` pide compilar y ejecutar con Java 21 aunque el JDK por defecto de tu máquina sea otro. Gradle lo recomienda para evitar que el proyecto se compile con versiones distintas [11]. Con JDK 21 instalado, Gradle lo encuentra solo.
- `run { standardInput = System.in }`: por defecto Gradle ejecuta el programa con una entrada vacía (lo dice la referencia de la tarea `JavaExec` en su documentación). Sin esta línea, el programa no puede leer lo que escribes. Es imprescindible en un juego de consola.
- El wrapper (`gradlew`) es la forma recomendada de ejecutar Gradle en un proyecto [12].

Para generarlo necesitas un Gradle instalado que pueda ejecutarse con Java 21 (a partir de la 8.5, según el manual).

Carpetas que crea Gradle

Al ejecutar, Gradle crea `.gradle/` y `build/` dentro del proyecto. Son archivos generados: si usas Git, no los añadas al repositorio. No aparecen en los árboles de este libro.

Leer del teclado con Scanner. Para leer lo que escribe el jugador usarás `java.util.Scanner`, que ya viene en el JDK. `new Scanner(System.in)` crea un lector sobre la entrada estándar, y su método `nextLine()` devuelve la siguiente línea escrita, sin el salto final. Si no hay más entrada (por ejemplo, la entrada vacía de Gradle sin `standardInput`), lanza `NoSuchElementException`. Lo probarás en el ejercicio 1.2.

1.5 En el proyecto: H1, el núcleo sin patrones

El hito H1 es un combate por turnos jugable en consola, escrito a la manera más directa. Estas son las reglas, deliberadamente mínimas:

- Tres héroes contra los tres enemigos de la planta 1. Cada combatiente tiene vida máxima, ataque y defensa.
- Una ronda: cada héroe vivo actúa en orden (hoy solo puede *atacar* a un enemigo vivo que elige el jugador) y después actúan los enemigos vivos.
- Daño = ataque del atacante − defensa del defensor, con un mínimo de 1.
- Gana quien deje sin vida a todos los del otro bando.

1.5.1 Las clases de datos y Fighter

Las definiciones de héroe y enemigo son *records*: datos inmutables. Lo que cambia durante el combate (la vida actual) vive en `Fighter`, una clase mutable que envuelve a un héroe o a un enemigo. Los cuatro archivos van en `engine/character/`:

`Stats.java`

```
1 package emberdeep.engine.character;
2
3 public record Stats(int maxHp, int attack, int defense) {}
```

`Hero.java`

```
1 package emberdeep.engine.character;
2
3 public record Hero(String name, Stats stats) {}
```

`Enemy.java`

```
1 package emberdeep.engine.character;
2
3 public record Enemy(String name, Stats stats) {}
```

Fighter.java

```java
package emberdeep.engine.character;

public class Fighter {
    private final String name;
    private final Stats stats;
    private int hp;

    private Fighter(String name, Stats stats) {
        this.name = name;
        this.stats = stats;
        this.hp = stats.maxHp();
    }

    public static Fighter ofHero(Hero hero) {
        return new Fighter(hero.name(), hero.stats());
    }

    public static Fighter ofEnemy(Enemy enemy) {
        return new Fighter(enemy.name(), enemy.stats());
    }

    public String name() { return name; }
    public Stats stats() { return stats; }
    public int hp() { return hp; }
    public boolean isAlive() { return hp > 0; }

    public void takeDamage(int amount) {
        hp = Math.max(0, hp - amount);
    }
}
```

El constructor es privado y hay dos fábricas estáticas con nombre, `ofHero` y `ofEnemy`, para que se vea desde dónde se crea cada combatiente. La vida nunca baja de 0, y `isAlive()` es lo único que el resto del motor necesita preguntar.

1.5.2 El resto del núcleo

El diagrama muestra cómo se relacionan las siete clases del hito. Léelo con las reglas de la sección anterior: `ConsoleUi` usa a `GameSession` sin guardarla (dependencia); `GameSession` guarda su `Combat` (asociación); `Combat` agrupa a los `Fighter` que recibe ya creados (agregación), y cada uno de ellos, igual que `Hero` y `Enemy`, guarda unas `Stats` (asociación).

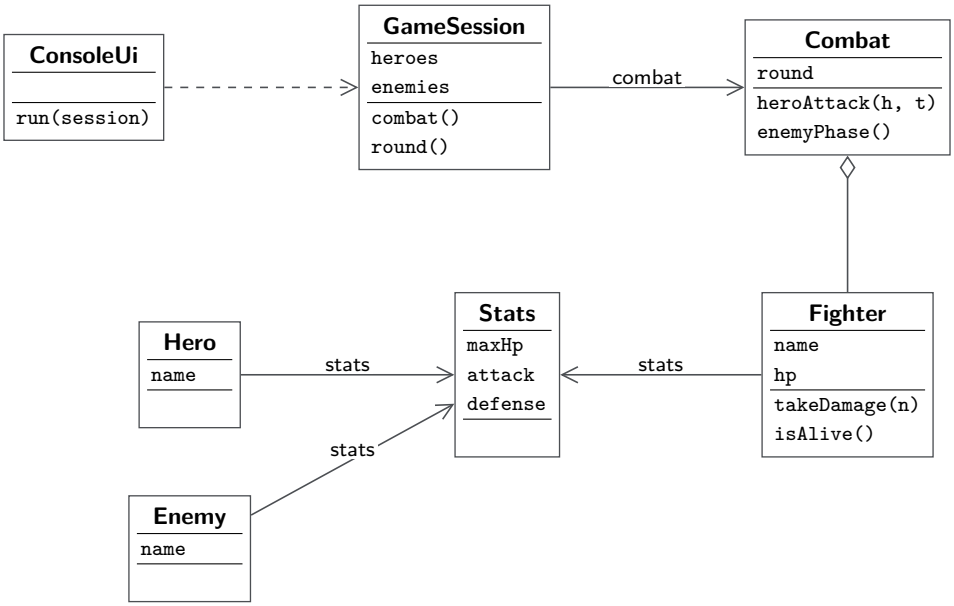


Diagrama UML del núcleo de Emberdeep tras el hito H1 (sin patrones).

Las clases que faltan las escribes tú en el ejercicio 1.5. Esta tabla dice qué hace cada una y con qué firma, para que encajen entre sí:

Clase	Paquete	Qué hace
`Combat`	`engine.combat`	Reglas del combate y contador de ronda `round()`. No imprime: devuelve `List<String>` con los mensajes.
`GameSession`	`engine.session`	Guarda los tres héroes, los enemigos de la planta 1 y su `Combat`; `round()` se delega en el combate.
`ConsoleUi`	`ui`	Pinta el estado, muestra el menú «[1] atacar», lee con `Scanner` y escribe los mensajes.
`Main`	`app`	Composition root: crea los datos, la sesión y la interfaz, y arranca.

La decisión de qué hace cada enemigo en su turno es la parte que más importa del hito, y la verás pronto como un problema. Cada enemigo actúa según su nombre:

`Combat.java (fragmento: el turno de los enemigos)`

```
1  public List<String> enemyPhase() {
2      List<String> messages = new ArrayList<>();
3      for (Fighter enemy : enemies) {
4          if (!enemy.isAlive()) {
5              continue;
6          }
7          if (heroesLost()) {
8              break;
9          }
10         Fighter target;
11         int damage;
12         if (enemy.name().equals("Goblin")) {
13             target = firstAlive(heroes);
14             damage = damageOf(enemy, target);
15         } else if (enemy.name().equals("Rata de ceniza")) {
```

```
            target = weakest(heroes);
            damage = damageOf(enemy, target);
        } else if (enemy.name().equals("Esqueleto")) {
            target = firstAlive(heroes);
            damage = 3;
        } else {
            target = firstAlive(heroes);
            damage = damageOf(enemy, target);
        }
        hit(enemy, target, damage, messages);
    }
    round++;
    return messages;
}
```

Cuatro auxiliares privados completan la clase: `damageOf(atacante, defensor)` calcula `max(1, ataque - defensa)`, `firstAlive(grupo)` devuelve el primer combatiente vivo, `weakest(grupo)` el vivo con menos vida y `hit(...)` aplica el daño y añade a la lista los mensajes del golpe. El goblin ataca al primer héroe vivo, la rata de ceniza al más débil y el esqueleto golpea con un daño fijo de 3 que ignora la defensa.

Hito 1: el núcleo sin patrones

Emberdeep nace aquí, desde cero. Al terminar el ejercicio 1.5 tienes un combate completo con `./gradlew run`: tres héroes contra Goblin, Rata de ceniza y Esqueleto, con un menú de una opción. Esta es la estructura:

Estructura de carpetas tras el capítulo 1

```
emberdeep/
|-- settings.gradle
|-- build.gradle
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
`-- src/main/java/emberdeep/
    |-- app/
    |   `-- Main.java              (composition root)
    |-- engine/
    |   |-- character/             Stats, Hero, Enemy, Fighter
    |   |-- combat/                Combat
    |   `-- session/               GameSession
    `-- ui/
        `-- ConsoleUi.java
```

1.6 Seis olores que ya se ven en el núcleo

El núcleo funciona, y ese es el punto: casi nada del código es «incorrecto». Duele cuando quieras cambiarlo. Estos seis síntomas ya están ahí, y cada uno tiene su capítulo. No hace falta entender los patrones todavía; basta con que reconozcas el síntoma.

Dónde se ve	Qué duele	Lo cura (capítulo)
La cadena de `if` por nombre en `enemyPhase`	Cada enemigo nuevo es otra rama en `Combat`, y el comportamiento depende de un texto.	Strategy (cap. 2)
`new Enemy(...)` escrito a mano en `Main`	Otra planta con otros enemigos obliga a repetir y cambiar `Main`.	Factory Method y Abstract Factory (cap. 3)
`new Hero("Aria", new Stats(30, 8, 3))`	Tres números sin nombre; con más opciones nadie recuerda el orden.	Builder (cap. 4)
`Stats` fijas durante el combate	El equipo y las mejoras temporales cambiarán el valor final del ataque o la defensa.	Decorator (cap. 6)
La acción «atacar» metida en `heroAttack` y en el menú	Deshacer o repetir una partida exige tratar cada acción como un objeto.	Command (cap. 7)
`Combat` devuelve `List<String>` ya redactada	Un HUD o un logro no pueden reaccionar a «hizo 7 de daño» sin leer texto.	Observer (cap. 9)

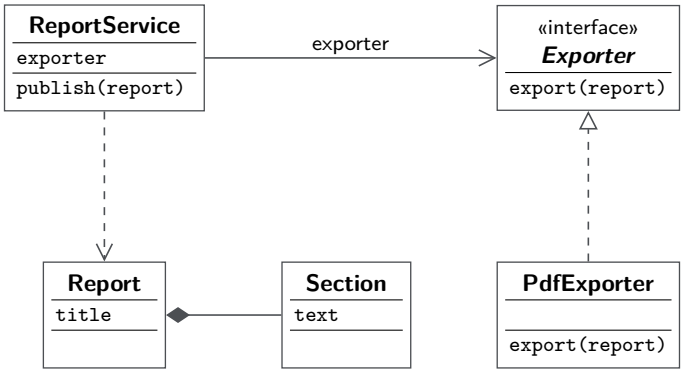
Qué no se cura

No todo en este núcleo es un olor. Que `Fighter` sea mutable y los *records* no lo sean, o que `Main` haga `new` de todo, es una decisión razonable por ahora. Aprender patrones también es aprender a dejar sin patrón lo que no duele.

1.7 Ejercicios

Ejercicio 1.1: Lee un diagrama de cinco clases ★☆☆☆☆

Este diagrama no es de Emberdeep. Contesta con una frase cada pregunta.



1. ¿Cuál de las cinco es una interfaz y cómo lo sabes?
2. ¿Qué relación hay entre `PdfExporter` y `Exporter`? ¿Qué palabra de Java usarías?
3. ¿Y entre `ReportService` y `Exporter`? ¿Qué tendría `ReportService` en su código?
4. ¿Y entre `ReportService` y `Report`? ¿En qué se diferencia de la anterior?
5. ¿Y entre `Report` y `Section`? Si descartas un `Report`, ¿qué pasa con sus `Section`?

6. ¿Quién depende de quién? Escribe las dependencias del diagrama y di si `ReportService` conoce a `PdfExporter`.

Está bien si cada respuesta nombra la relación y la regla de la leyenda en la que te apoyas.

Lectura de código ▪ solución en la pág. 15

Ejercicio 1.2: Un Gradle mínimo que lee del teclado ★☆☆☆☆

Crea un proyecto Gradle llamado `hola`, fuera de `emberdeep/`, que pregunte cómo te llamas y te salude.

1. Crea las carpetas con `mkdir -p hola/src/main/java/demo` y entra en `hola/`.
2. Crea `settings.gradle` con `rootProject.name = 'hola'`.
3. Crea `build.gradle` como el de la instalación de este capítulo, pero con `mainClass = 'demo.Main'` y con el bloque `run` que conecta `System.in`.
4. Crea `src/main/java/demo/Main.java` (paquete `demo`). Muestra «¿Cómo te llamas?», lee el nombre con `new Scanner(System.in).nextLine()` y muestra «Hola, » seguido del nombre.
5. Genera el wrapper y ejecuta el programa:

```
$ gradle wrapper --gradle-version 9.8.2
$ ./gradlew run
```

6. Borra el bloque `run` de `build.gradle`, ejecuta de nuevo y fíjate en el error.

Está bien si en el paso 5 escribes tu nombre y el programa te saluda, y si en el paso 6 el programa falla con una excepción en lugar de esperar tu texto.

Guiado ▪ solución en la pág. 15

Ejercicio 1.3: Tres olores en el turno de los enemigos ★★☆☆☆

Lee este fragmento de `Combat` (el turno de un enemigo, sección 1.5). Los auxiliares `firstAlive`, `weakest` y `damageOf` devuelven el primer héroe vivo, el vivo con menos vida y el daño máx$(1, \text{ataque} - \text{defensa})$.

```
Fighter target;
int damage;
if (enemy.name().equals("Goblin")) {
    target = firstAlive(heroes);
    damage = damageOf(enemy, target);
} else if (enemy.name().equals("Rata de ceniza")) {
    target = weakest(heroes);
    damage = damageOf(enemy, target);
} else if (enemy.name().equals("Esqueleto")) {
    target = firstAlive(heroes);
    damage = 3;
} else {
    target = firstAlive(heroes);
    damage = damageOf(enemy, target);
}
```

Señala tres olores concretos que harán sufrir al añadir enemigos nuevos. Para cada uno indica qué línea o qué expresión lo muestra y qué habría que cambiar si llega un «Murciélago» que ataca al héroe con menos defensa. Está bien si los tres olores son distintos entre sí y cada uno apunta a código del fragmento.

Lectura de código ▪ solución en la pág. 16

Ejercicio 1.4: Reparte las clases en paquetes ★★☆☆☆

Tienes diez clases de un juego como Emberdeep: `Main`, `Stats`, `Hero`, `Enemy`, `Fighter`, `Combat`, `GameSession`, `ConsoleUi`, `MenuReader` (lee y valida números del teclado) y `AnsiColors` (da color al texto en la consola).

1. Escribe una tabla con el paquete de cada clase: `app`, `engine.character`, `engine.combat`, `engine.session` o `ui`.
2. Esta versión de `Combat` rompe la regla «el motor no importa `ui`». Corrígela sin que el motor conozca la interfaz (supón que `ConsoleUi` tiene un método `print(String)`):

`Combat.java (versión con el defecto)`

```
package emberdeep.engine.combat;

import emberdeep.engine.character.Fighter;
import emberdeep.ui.ConsoleUi;

public class Combat {
    private final ConsoleUi ui;

    public Combat(ConsoleUi ui) {
        this.ui = ui;
    }

    public void attack(Fighter attacker, Fighter target) {
        int damage = Math.max(1, attacker.stats().attack() - target.stats().
    ↪ defense());
        target.takeDamage(damage);
        ui.print(attacker.name() + " ataca a " + target.name() + " (" +
    ↪ damage + ")");
    }
}
```

Está bien si ninguna clase de `engine` importa nada de `ui` y el mensaje sigue llegando a la pantalla.

Práctica ▪ solución en la pág. 17

Ejercicio 1.5: Hito 1: el núcleo jugable de Emberdeep ★★★☆☆

Implementa el hito H1 siguiendo la estructura de este capítulo.

- Parte de la carpeta `emberdeep/` con los archivos de Gradle de la instalación, el wrapper generado y los cuatro archivos de `engine/character/` de la sección 1.5.
- Escribe `Combat`, `GameSession`, `ConsoleUi` y `Main` con las firmas de la tabla de la sección. `Combat` tiene un constructor `Combat(List<Fighter> heroes, List<Fighter> enemies)`, `heroAttack(hero, target)` y el turno de los enemigos del fragmento, cuyos mensajes tienen el formato «Aria ataca a Goblin y le hace 7 de daño.» y, si el

defensor cae, «Goblin queda fuera de combate.».

- `ConsoleUi(Scanner in, PrintStream out)` tiene `run(GameSession)`: en cada turno de héroe muestra la vida de ambos bandos, el menú «[1] atacar» y la lista de enemigos vivos numerada, y vuelve a preguntar si la entrada no es un número válido.
- `Main` crea los héroes Aria (30 de vida, 8 de ataque, 3 de defensa), Borin (40, 6, 5) y Cael (24, 9, 2), y los enemigos Goblin (14, 6, 1), Rata de ceniza (8, 5, 0) y Esqueleto (18, 7, 2).

Está bien si:

1. `./gradlew run` compila y muestra el menú.
2. Tecleando siempre `1`, la primera ronda muestra estos mensajes y la partida acaba en la ronda 3 con «¡Victoria!»:

   Mensajes de la primera ronda

   ```
   Aria ataca a Goblin y le hace 7 de daño.
   Borin ataca a Goblin y le hace 5 de daño.
   Cael ataca a Goblin y le hace 8 de daño.
   Goblin queda fuera de combate.
   Rata de ceniza ataca a Cael y le hace 3 de daño.
   Esqueleto ataca a Aria y le hace 3 de daño.
   ```

3. Ninguna clase de `engine` importa `emberdeep.ui` ni usa `System.out`.

Pista: *el estado que cambia (la vida) está en `Fighter`; `Combat` y `ConsoleUi` no deben guardar nada que no necesiten.*

Proyecto ▪ solución en la pág. 18

1.8 Resumen

Resumen

- Un patrón es una solución con nombre a un problema recurrente de diseño, con sus consecuencias; no es una librería, ni un objetivo, ni un sustituto de entender el problema.
- Primero el olor y después el patrón: si no puedes señalar la línea que duele, aún no lo necesitas.
- En un diagrama UML la flecha apunta a lo que se conoce o se usa; los rombos están en el todo, y las interfaces con sus realizaciones marcan lo intercambiable.
- Emberdeep tiene tres paquetes: `engine` (reglas, sin imprimir ni importar `ui`), `ui` y `app`. `Main` es el composition root.
- El proyecto Gradle necesita `settings.gradle`, `build.gradle` con `application`, toolchain de Java 21 y `run { standardInput = System.in }`, y el wrapper (versión 9.8.2).
- H1 es un combate jugable sin patrones. Sus olores (el `if` por nombre y otros cinco) son el hilo de los capítulos siguientes.

Soluciones de los ejercicios

Solución del ejercicio 1.1

enunciado en la pág. 11

1. `Exporter`: su caja lleva «interface» y el nombre en cursiva, y solo tiene métodos.
2. Realización (línea discontinua con triángulo hueco hacia la interfaz): `class PdfExporter implements Exporter`.
3. Asociación (flecha abierta con la etiqueta `exporter`): `ReportService` guarda una referencia, un campo `private final Exporter exporter;`. Se asocia a la interfaz, no a una clase concreta.
4. Dependencia (flecha discontinua): `Report` solo aparece como parámetro de `publish(report)`, no se guarda. La anterior era un campo; esta, un uso pasajero.
5. Composición (rombo lleno en `Report`): las `Section` nacen y mueren con su `Report`; si descartas el informe, sus secciones dejan de existir con él.
6. Dependencias del diagrama: `ReportService` depende de `Exporter` y de `Report`; `PdfExporter` depende de `Exporter` (lo realiza); `Report` conoce a sus `Section`. `ReportService` *no* conoce a `PdfExporter`: no hay ninguna flecha entre ambos, y por eso se puede cambiar el exportador sin tocarlo.

Errores típicos: confundir asociación y dependencia (la primera guarda una referencia, la segunda solo usa el tipo) y leer la flecha de realización al revés (la punta apunta a la interfaz, a lo que se promete, no a quien lo cumple).

Solución del ejercicio 1.2

enunciado en la pág. 12

Estructura final:

Estructura de hola/

```
hola/
|-- settings.gradle
|-- build.gradle
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
`-- src/main/java/demo/
    `-- Main.java
```

settings.gradle

```
rootProject.name = 'hola'
```

build.gradle

```
plugins {
    id 'application'
}

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

application {
    mainClass = 'demo.Main'
}

run {
    standardInput = System.in
}
```

src/main/java/demo/Main.java

```
package demo;

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        System.out.println("¿Cómo te llamas?");
        String name = new Scanner(System.in).nextLine();
        System.out.println("Hola, " + name + ".");
    }
}
```

Comandos:

```
$ gradle wrapper --gradle-version 9.8.2
$ ./gradlew run
```

Al ejecutar, escribes tu nombre y ves «Hola, » seguido de él. Al quitar el bloque `run`, Gradle da al programa una entrada vacía y `nextLine()` lanza `NoSuchElementException: No line found`: el programa no espera tu texto.
Errores típicos: la clase está en el paquete `demo` pero `mainClass` dice otra cosa (Gradle no encuentra la clase principal); olvidar el `import java.util.Scanner`; ejecutar `./gradlew` desde fuera de la carpeta del proyecto.

Solución del ejercicio 1.3 enunciado en la pág. 12

1. **Cadena de `if` por nombre** (`enemy.name().equals("Goblin")...`). Cada enemigo nuevo es otra rama que obliga a editar `Combat`, una clase que no tendría por qué conocer a los enemigos. El «Murciélago» necesita una rama más. Además el comportamiento depende de un texto: renombrar «Goblin» a «Goblin chamán» cambia en silencio lo que hace, porque cae en el `else` final.

2. **Elección del objetivo repetida dentro de cada rama** (`target = firstAlive(heroes)` aparece tres veces). La política de objetivo está mezclada con la cadena de `if`, y para el «Murciélago» habría que escribir una función nueva (el héroe con menos defensa) y otra rama que la use.
3. **Daño fijo como número mágico** (`damage = 3` del esqueleto). Una regla del juego queda enterrada en un `if` de `Combat`, y la fórmula normal (`damageOf`) está duplicada en tres ramas. Si cambia la regla de daño, hay que recordar todas las líneas.

Otros olores válidos: el `else` final trata cualquier enemigo desconocido como un goblin sin avisar, y `enemyPhase` mezcla en un método elegir al enemigo, elegir el objetivo, calcular el daño y avanzar la ronda. Cualquier respuesta es correcta si apunta a una expresión concreta del fragmento y explica el coste de añadir un enemigo.
Error típico: señalar el `for` o los helpers; el dolor no está en recorrer la lista, sino en la decisión por nombre.

Solución del ejercicio 1.4 enunciado en la pág. 13

Clase	Paquete	Motivo
`Main`	`app`	Composition root: crea y conecta.
`Stats`, `Hero`, `Enemy`, `Fighter`	`engine.character`	Datos y estado de los combatientes.
`Combat`	`engine.combat`	Reglas del combate.
`GameSession`	`engine.session`	Estado de la partida.
`ConsoleUi`	`ui`	Pinta y lee de la consola.
`MenuReader`	`ui`	Lee y valida el teclado.
`AnsiColors`	`ui`	Presentación (colores).

La importación defectuosa se arregla haciendo que el motor *devuelva* los mensajes en lugar de imprimirlos; quien los pinta es la `ui`:

Combat.java (corregida)

```java
package emberdeep.engine.combat;

import emberdeep.engine.character.Fighter;
import java.util.List;

public class Combat {
    public List<String> attack(Fighter attacker, Fighter target) {
        int damage = Math.max(1, attacker.stats().attack() - target.stats().defense());
        target.takeDamage(damage);
        return List.of(attacker.name() + " ataca a " + target.name() + " (" + damage + ")");
    }
}
```

Y en `ConsoleUi`, la llamada pasa a ser `combat.attack(a, b).forEach(this::print);` (con `print(String)` como método de la propia interfaz de usuario).
Errores típicos: mover `ConsoleUi` a `engine` para que el import sea legal (esconde el

problema: el motor sigue atado a la consola); dejar el `import` sin usar; poner `Stats` en `ui` por «ser datos». Una alternativa válida es una interfaz de salida definida en `engine` que `ui` implementa, pero para H1 es más de lo que el problema pide.

Solución del ejercicio 1.5 enunciado en la pág. 13

Parte de la carpeta con los archivos de Gradle, el wrapper y los cuatro archivos de `engine/character/` del capítulo (`Stats`, `Hero`, `Enemy` y `Fighter`), que se repiten aquí para que la solución esté completa.

engine/character/Stats.java, Hero.java y Enemy.java

Stats.java

```java
package emberdeep.engine.character;

public record Stats(int maxHp, int attack, int defense) {}
```

Hero.java

```java
package emberdeep.engine.character;

public record Hero(String name, Stats stats) {}
```

Enemy.java

```java
package emberdeep.engine.character;

public record Enemy(String name, Stats stats) {}
```

engine/character/Fighter.java

Fighter.java

```java
package emberdeep.engine.character;

public class Fighter {
    private final String name;
    private final Stats stats;
    private int hp;

    private Fighter(String name, Stats stats) {
        this.name = name;
        this.stats = stats;
        this.hp = stats.maxHp();
    }

    public static Fighter ofHero(Hero hero) {
        return new Fighter(hero.name(), hero.stats());
    }

    public static Fighter ofEnemy(Enemy enemy) {
        return new Fighter(enemy.name(), enemy.stats());
    }

    public String name() { return name; }
    public Stats stats() { return stats; }
    public int hp() { return hp; }
    public boolean isAlive() { return hp > 0; }

    public void takeDamage(int amount) {
        hp = Math.max(0, hp - amount);
    }
}
```

engine/combat/Combat.java (en tres tramos seguidos, dentro de la misma clase)

Combat.java (1/3: estado y turno de los héroes)

```java
package emberdeep.engine.combat;

import emberdeep.engine.character.Fighter;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class Combat {
    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private int round = 1;

    public Combat(List<Fighter> heroes, List<Fighter> enemies) {
        this.heroes = heroes;
        this.enemies = enemies;
    }

    public int round() {
        return round;
    }

    public boolean heroesWon() {
        return enemies.stream().noneMatch(Fighter::isAlive);
    }

    public boolean heroesLost() {
        return heroes.stream().noneMatch(Fighter::isAlive);
    }

    public boolean isOver() {
        return heroesWon() || heroesLost();
    }

    public List<String> heroAttack(Fighter hero, Fighter target) {
        List<String> messages = new ArrayList<>();
        hit(hero, target, damageOf(hero, target), messages);
        return messages;
    }
```

Combat.java (2/3: turno de los enemigos)

```
    public List<String> enemyPhase() {
        List<String> messages = new ArrayList<>();
        for (Fighter enemy : enemies) {
            if (!enemy.isAlive()) {
                continue;
            }
            if (heroesLost()) {
                break;
            }
            Fighter target;
            int damage;
            if (enemy.name().equals("Goblin")) {
                target = firstAlive(heroes);
                damage = damageOf(enemy, target);
            } else if (enemy.name().equals("Rata de ceniza")) {
                target = weakest(heroes);
                damage = damageOf(enemy, target);
            } else if (enemy.name().equals("Esqueleto")) {
                target = firstAlive(heroes);
                damage = 3;
            } else {
                target = firstAlive(heroes);
                damage = damageOf(enemy, target);
            }
            hit(enemy, target, damage, messages);
        }
        round++;
        return messages;
    }
```

Combat.java (3/3: auxiliares privados y llave final)

```
    private void hit(Fighter attacker, Fighter target, int damage, List<String> messages) {
        target.takeDamage(damage);
        messages.add(attacker.name() + " ataca a " + target.name()
                + " y le hace " + damage + " de daño.");
        if (!target.isAlive()) {
            messages.add(target.name() + " queda fuera de combate.");
        }
    }

    private static int damageOf(Fighter attacker, Fighter target) {
        return Math.max(1, attacker.stats().attack() - target.stats().defense());
    }

    private static Fighter firstAlive(List<Fighter> group) {
        return group.stream().filter(Fighter::isAlive).findFirst().orElseThrow();
    }

    private static Fighter weakest(List<Fighter> group) {
        return group.stream().filter(Fighter::isAlive)
                .min(Comparator.comparingInt(Fighter::hp)).orElseThrow();
    }
}
```

engine/session/GameSession.java

GameSession.java

```java
package emberdeep.engine.session;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.combat.Combat;
import java.util.List;

public class GameSession {
    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final Combat combat;

    public GameSession(List<Hero> heroes, List<Enemy> floorOneEnemies) {
        this.heroes = heroes.stream().map(Fighter::ofHero).toList();
        this.enemies = floorOneEnemies.stream().map(Fighter::ofEnemy).
    ↪ toList();
        this.combat = new Combat(this.heroes, this.enemies);
    }

    public List<Fighter> heroes() { return heroes; }
    public List<Fighter> enemies() { return enemies; }
    public Combat combat() { return combat; }
    public int round() { return combat.round(); }
}
```

ui/ConsoleUi.java (en dos tramos seguidos, dentro de la misma clase)

ConsoleUi.java (1/2: partida)

```java
package emberdeep.ui;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.session.GameSession;
import java.io.PrintStream;
import java.util.List;
import java.util.Scanner;
import java.util.stream.Collectors;

public class ConsoleUi {
    private final Scanner in;
    private final PrintStream out;

    public ConsoleUi(Scanner in, PrintStream out) {
        this.in = in;
        this.out = out;
    }

    public void run(GameSession session) {
        Combat combat = session.combat();
        out.println("=== Emberdeep: planta 1 ===");
        while (!combat.isOver()) {
            out.println("--- Ronda " + session.round() + " ---");
            for (Fighter hero : session.heroes()) {
                if (!hero.isAlive() || combat.isOver()) {
                    continue;
                }
                out.println("Héroes:   " + describe(session.heroes()));
                out.println("Enemigos: " + describe(session.enemies()));
                out.println("Turno de " + hero.name() + ". [1] atacar");
                ask(1);
                Fighter target = askTarget(session.enemies());
                print(combat.heroAttack(hero, target));
            }
            if (!combat.isOver()) {
                print(combat.enemyPhase());
            }
        }
        out.println(combat.heroesWon() ? "¡Victoria!" : "Derrota...");
    }
```

ConsoleUi.java (2/2: auxiliares y llave final)

```
    private String describe(List<Fighter> group) {
        return group.stream()
                .map(f -> f.name() + " " + f.hp() + "/" + f.stats().maxHp())
                .collect(Collectors.joining(", "));
    }

    private void print(List<String> messages) {
        messages.forEach(out::println);
    }

    private int ask(int max) {
        while (true) {
            out.print("> ");
            out.flush();
            String line = in.nextLine().trim();
            try {
                int n = Integer.parseInt(line);
                if (n >= 1 && n <= max) {
                    return n;
                }
            } catch (NumberFormatException e) {
                // no es un número: se avisa más abajo
            }
            out.println("Escribe un número entre 1 y " + max + ".");
        }
    }

    private Fighter askTarget(List<Fighter> enemies) {
        List<Fighter> alive = enemies.stream().filter(Fighter::isAlive).
toList();
        StringBuilder menu = new StringBuilder("Objetivo:");
        for (int i = 0; i < alive.size(); i++) {
            menu.append("  [").append(i + 1).append("] ").append(alive.get(
i).name());
        }
        out.println(menu);
        return alive.get(ask(alive.size()) - 1);
    }
}
```

app/Main.java

Main.java

```java
package emberdeep.app;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.Stats;
import emberdeep.engine.session.GameSession;
import emberdeep.ui.ConsoleUi;
import java.util.List;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        List<Hero> heroes = List.of(
                new Hero("Aria", new Stats(30, 8, 3)),
                new Hero("Borin", new Stats(40, 6, 5)),
                new Hero("Cael", new Stats(24, 9, 2)));
        List<Enemy> floorOne = List.of(
                new Enemy("Goblin", new Stats(14, 6, 1)),
                new Enemy("Rata de ceniza", new Stats(8, 5, 0)),
                new Enemy("Esqueleto", new Stats(18, 7, 2)));
        GameSession session = new GameSession(heroes, floorOne);
        ConsoleUi ui = new ConsoleUi(new Scanner(System.in), System.out);
        ui.run(session);
    }
}
```

Por qué funciona. Los *records* describen; `Fighter` lleva la vida. `Combat` decide y devuelve texto; solo `ConsoleUi` usa `System.out`, y `Main` es el único sitio con `new` de los colaboradores. Con `1` en cada pregunta: ronda 1, los héroes dejan al goblin fuera ($7 + 5 + 8$ de daño sobre 14 de vida), la rata golpea a Cael (el de menos vida, 24) con $5 - 2 = 3$ y el esqueleto a Aria con su 3 fijo. En la ronda 2 Aria acaba con la rata (8 de daño, 8 de vida) y Borin y Cael dejan al esqueleto en 7; el esqueleto golpea a Aria otra vez. En la ronda 3 Aria (6) y Borin (4) lo derrotan y sale «¡Victoria!» sin que Cael llegue a actuar. En la ronda 3, en el turno de Borin, verás `Héroes: Aria 24/30, Borin 40/40, Cael 21/24`.
Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo siguiente.

Errores típicos:

- Olvidar el bloque `run` del `build.gradle`: el juego se cierra al primer `nextLine()` con `NoSuchElementException`.
- Que el esqueleto use `damageOf` en lugar del 3 fijo: el mensaje de la ronda 1 diría 4 en vez de 3 (7 de ataque menos 3 de defensa de Aria).
- Importar `emberdeep.ui` desde `engine`, o imprimir con `System.out` en `Combat`: rompe la regla de las capas.
- Dejar que `firstAlive` lance una excepción cuando ya no quedan héroes vivos: por eso `enemyPhase` comprueba `heroesLost()` antes de que actúe cada enemigo.

CAPÍTULO 2

Strategy: IA del enemigo y fórmula de daño

Objetivos del capítulo

- Reconocer el olor que pide Strategy: una cadena de `if` que elige un comportamiento.
- Describir las piezas de Strategy (estrategia, estrategias concretas y contexto) y dibujarlas en UML.
- Sustituir los `if` por nombre del combate por objetos intercambiables (`EnemyAi`).
- Expresar una estrategia como lambda cuando la interfaz tiene un solo método (`DamageFormula`).
- Inyectar un `java.util.Random` con semilla para que las partidas sean repetibles.

En el capítulo anterior terminaste con un combate que funciona y con una cadena de `if` que decide, según el nombre del enemigo, a quién ataca y con cuánta fuerza. Funciona, pero cada enemigo nuevo obliga a abrir `Combat`. En este capítulo curas ese primer dolor con tu primer patrón, Strategy, y aprendes a leer su ficha completa: el olor, las piezas, el ejemplo, el UML y cuándo no usarlo.

2.1 El olor que pide Strategy: los `if` por nombre

Este es el turno de los enemigos tal como lo dejaste en el capítulo 1:

Combat.java (H1, fragmento)

```
Fighter target;
int damage;
if (enemy.name().equals("Goblin")) {
    target = firstAlive(heroes);
    damage = damageOf(enemy, target);
} else if (enemy.name().equals("Rata de ceniza")) {
    target = weakest(heroes);
    damage = damageOf(enemy, target);
} else if (enemy.name().equals("Esqueleto")) {
    target = firstAlive(heroes);
    damage = 3;
} else {
    target = firstAlive(heroes);
    damage = damageOf(enemy, target);
}
```

Mira lo que mezcla esta cadena. Hay **dos decisiones** distintas: *a quién* ataca el enemigo (`firstAlive`, `weakest`) y *cuánto* daño hace (`damageOf`, o un 3 fijo). Las dos dependen de un

texto, el nombre. Y las dos viven dentro de `Combat`, una clase que no debería saber qué enemigos existen.

Si mañana llega un «Murciélago» que ataca al héroe con menos defensa, tienes que: escribir una función nueva, añadir otra rama al `if` y acordarte de que el `else` final tratará como un goblin a cualquier enemigo cuyo nombre escribas mal.

Ese es el olor: **una cadena de `if` o `switch` que elige entre variantes de un mismo comportamiento**. El patrón que lo cura se llama *Strategy*. El catálogo del GoF lo define como una familia de algoritmos, cada uno encapsulado y todos intercambiables, de forma que el algoritmo cambia sin que cambie quien lo usa [9]. Dicho con nuestro código: cada forma de elegir objetivo pasa a ser un objeto, y `Combat` deja de preguntar «¿quién eres?» para decir solo «elige».

Ese tipo de cambio, partir de un código que ya existe y llegar al patrón paso a paso, es justo el enfoque de la refactorización hacia patrones [17], y es el que sigue este libro: primero el dolor, luego la pieza.

2.2 Piezas del patrón Strategy

Strategy tiene tres piezas. La *estrategia* es la interfaz que fija qué se decide. Las *estrategias concretas* son las variantes. El *contexto* es quien guarda una estrategia y le delega la decisión. En Emberdeep hay además una segunda familia, la fórmula de daño, que sigue el mismo esquema.

Rol	Clase en el ejemplo	Propósito	Paquete
Estrategia (interfaz)	`EnemyAi`	Elige objetivo	`engine.combat.ai`
Estrategias concretas (clases)	`FirstAliveAi`; `LowestHpAi` y `RandomTargetAi` (ejercicio)	Un criterio cada una	`engine.combat.ai`
Contexto (clase)	`Fighter`	Consulta la IA de su `Enemy`	`engine.character`
Portador de la estrategia (record)	`Enemy(name, stats, ai)`	Solo guarda la IA	`engine.character`
Segunda estrategia (interfaz funcional)	`DamageFormula`	Calcula el daño	`engine.combat.damage`
Estrategias de la fórmula (clases o lambdas)	`FlatFormula`; `ScaledFormula` (ejercicio)	Una regla de daño cada una	`engine.combat.damage`
Cliente (clase)	`Main`	Elige y monta	`app`

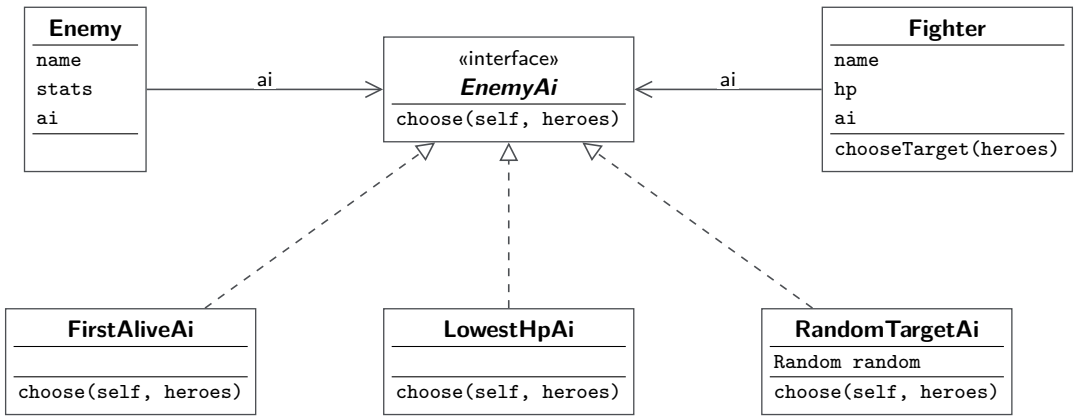


Diagrama UML del patrón Strategy: `Fighter` (contexto) conoce solo la interfaz `EnemyAi`; las tres IA son intercambiables.

Léelo con las reglas del capítulo anterior. La interfaz está arriba, con tres realizaciones debajo: esas tres clases son intercambiables para quien solo conoce `EnemyAi`. Nada del contexto señala a una clase concreta. Esa ausencia de flechas es la ganancia del patrón.

2.3 El ejemplo de Strategy, pieza a pieza: la IA del enemigo

2.3.1 La estrategia: `EnemyAi`

La interfaz dice qué se decide y con qué información. Recibe quien decide (`self`) y la lista de héroes, y devuelve el héroe elegido:

engine/combat/ai/EnemyAi.java

```
1  package emberdeep.engine.combat.ai;
2
3  import emberdeep.engine.character.Fighter;
4  import java.util.List;
5
6  public interface EnemyAi {
7      Fighter choose(Fighter self, List<Fighter> heroes);
8  }
```

Por qué recibe `self`: algunas IA necesitan saber quién las ejecuta (por ejemplo, para decidir según la vida del propio enemigo). Aunque las tres de este capítulo no lo usen, la firma no tendrá que cambiar cuando alguna lo necesite.

2.3.2 Una estrategia concreta: `FirstAliveAi`

Es la lógica del goblin del capítulo anterior, ahora en su propia clase:

engine/combat/ai/FirstAliveAi.java

```
1  package emberdeep.engine.combat.ai;
2
3  import emberdeep.engine.character.Fighter;
4  import java.util.List;
5
6  public class FirstAliveAi implements EnemyAi {
```

```
    @Override
    public Fighter choose(Fighter self, List<Fighter> heroes) {
        return heroes.stream().filter(Fighter::isAlive).findFirst().orElseThrow
    ↪ ();
    }
}
```

No tiene campos: no guarda nada entre llamadas. Una estrategia sin estado se puede compartir entre muchos enemigos con una sola instancia. `orElseThrow()` no es un descuido: `Combat` comprueba `heroesLost()` antes de dar el turno a cada enemigo, así que siempre queda algún héroe vivo.

2.3.3 El portador y el contexto: `Enemy` y `Fighter`

`Enemy` sigue siendo un record de datos y gana un campo, la IA. Cada enemigo *lleva* su estrategia:

engine/character/Enemy.java

```
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;

public record Enemy(String name, Stats stats, EnemyAi ai) {}
```

El contexto es `Fighter`. Guarda la IA que recibe de su `Enemy` y ofrece un método que se la consulta. Aquí solo van las partes que cambian respecto del capítulo anterior (la solución del ejercicio 2.4 trae la clase entera):

Fighter.java (fragmento: lo que cambia)

```
private final EnemyAi ai;   // null en los héroes: los elige el jugador

private Fighter(String name, Stats stats, EnemyAi ai) {
    this.name = name;
    this.stats = stats;
    this.ai = ai;
    this.hp = stats.maxHp();
}

public static Fighter ofHero(Hero hero) {
    return new Fighter(hero.name(), hero.stats(), null);
}

public static Fighter ofEnemy(Enemy enemy) {
    return new Fighter(enemy.name(), enemy.stats(), enemy.ai());
}

public Fighter chooseTarget(List<Fighter> heroes) {
    if (ai == null) {
        throw new IllegalStateException(name + " no tiene IA: es un héroe");
    }
    return ai.choose(this, heroes);
}
```

Es la delegación que define el patrón: `Fighter` no sabe *cómo* se elige, solo a quién preguntar. El `null` de los héroes es provisional y se nota: un método que solo vale para enemigos. Es un olor menor que dejamos pasar, porque los héroes no tienen IA y de momento no hay dónde ponerla mejor.

2.3.4 El resultado en Combat

Con la estrategia en su sitio, el turno de los enemigos pierde toda la cadena:

Combat.java (fragmento: el turno de los enemigos)

```
public List<String> enemyPhase() {
    List<String> messages = new ArrayList<>();
    for (Fighter enemy : enemies) {
        if (!enemy.isAlive()) {
            continue;
        }
        if (heroesLost()) {
            break;
        }
        Fighter target = enemy.chooseTarget(heroes);
        hit(enemy, target, damageOf(enemy, target), messages);
    }
    round++;
    return messages;
}
```

Añadir el Murciélago es ahora escribir *una* clase (`LowestDefenseAi`) y una línea en `Main`. `Combat` no se toca.

2.3.5 Aleatoriedad repetible: Random con semilla inyectada

Una IA que elige un héroe al azar necesita números aleatorios, y el azar complica dos cosas: probar el juego y repetir una partida. La solución es tratar la fuente de azar como una dependencia más.

`java.util.Random` es un generador de números pseudoaleatorios. Dos cosas importan:

- `new Random(42)` crea un generador con *semilla* 42. La semilla fija toda la secuencia: dos generadores con la misma semilla producen los mismos números en el mismo orden. Sin semilla (`new Random()`), cada ejecución sale distinta.
- `nextInt(n)` devuelve un entero de 0 a $n-1$, ambos incluidos. Para elegir entre tres héroes, `random.nextInt(3)` da el índice 0, 1 o 2.

Prueba de la semilla (no es parte de Emberdeep)

```
import java.util.Random;

public class SeedDemo {
    public static void main(String[] args) {
        Random a = new Random(42);
        Random b = new Random(42);
        for (int i = 0; i < 6; i++) {
            System.out.print(a.nextInt(3) + " ");
```

```
        }
        System.out.println();
        for (int i = 0; i < 6; i++) {
            System.out.print(b.nextInt(3) + " ");
        }
        System.out.println();
    }
}
```

Las dos líneas imprimen lo mismo: `2 0 0 2 0 1`. La regla de este libro es **inyectar** el `Random` por constructor y no hacer `new Random()` dentro de la clase que lo usa. Así `Main`, el composition root, decide la semilla: con una fija las partidas son repetibles (útil para depurar y para comparar dos ejecuciones); sin semilla serían distintas cada vez. Es la misma idea que ya conoces: una dependencia se pasa de fuera.

Un solo Random, no uno por clase

Si cada IA creara su propio `new Random(42)`, todas repetirían la misma secuencia. Se crea *un* `Random` en `Main` y se comparte con quien lo necesite.

2.4 Strategy con lambdas: la fórmula de daño

La segunda decisión del combate es *cuánto* daño se hace. En el capítulo anterior era `max(1, ataque - defensa)` escrito dentro de `Combat`, y el esqueleto lo ignoraba con un 3 fijo. Ahora será una estrategia más. Pero esta tiene un rasgo que la distingue de `EnemyAi`: su interfaz tiene un único método y las variantes no guardan estado.

2.4.1 Una interfaz funcional

Una *interfaz funcional* es una interfaz con exactamente un método abstracto. Java permite crear sus instancias con una *lambda*, una función anónima con la forma `(parámetros) -> expresión`. La anotación `@FunctionalInterface` es opcional, pero hace que el compilador proteja esa condición.

engine/combat/damage/DamageFormula.java

```
package emberdeep.engine.combat.damage;

@FunctionalInterface
public interface DamageFormula {
    int damage(int attack, int defense);
}
```

Recibe números y no `Fighter`: la fórmula solo necesita ataque y defensa, y así sirve también cuando esas cifras salgan de otro sitio.

2.4.2 Una versión de clase y otra de lambda

La regla de H1 como clase es la forma que ya conoces:

engine/combat/damage/FlatFormula.java

```java
package emberdeep.engine.combat.damage;

public class FlatFormula implements DamageFormula {
    @Override
    public int damage(int attack, int defense) {
        return Math.max(1, attack - defense);
    }
}
```

La misma regla como lambda ocupa una línea. La segunda fórmula del hito, la escalada, reduce el daño en proporción a la defensa en lugar de restarla: $\max(1, \lfloor \text{ataque} \times 100/(100 + \text{defensa}) \rfloor)$. Aquí la tienes como lambda:

Dos formas de escribir la misma estrategia

```java
DamageFormula flat = new FlatFormula();                          // clase
DamageFormula flatLambda = (attack, defense) -> Math.max(1, attack - defense);
DamageFormula scaled = (attack, defense) ->
        Math.max(1, attack * 100 / (100 + defense));

Combat combat = new Combat(heroes, enemies, scaled);         // la que quieras
```

Con ataque 7 y defensa 2, la plana da 5 y la escalada $700/102 = 6$ (la división entera descarta decimales). Con defensa 0 las dos dan el ataque completo.

2.4.3 Clase o lambda

Las dos formas son Strategy; cambia el coste de escribirlas.

Mejor una clase cuando...	Mejor una lambda cuando...
La estrategia guarda estado o dependencias (`RandomTargetAi` guarda un `Random`).	Es una expresión corta y sin estado.
Tiene un nombre del dominio que merece aparecer en el código y en las pruebas.	Se usa en un solo sitio y el nombre aportaría poco.
Se reutiliza en varios puntos o tendrá más de un método.	La interfaz tiene un único método abstracto.

El JDK trae interfaces funcionales genéricas en `java.util.function` [22]: `Function<T,R>` (recibe un `T` y devuelve un `R`), `Predicate<T>` (devuelve `boolean`) o `IntBinaryOperator` (dos `int` y devuelve otro). Esta última tiene exactamente la forma de nuestra fórmula, con el método `applyAsInt`. Se podría usar, pero `DamageFormula` gana en dos cosas: su nombre cuenta qué es, y puedes añadirle métodos con valor por defecto más adelante sin tocar a quien la usa. Las lambdas y las referencias a método (`Fighter::isAlive`) son la herramienta del lenguaje que lo hace posible [26].

Funciones de primera clase y patrones

Que Strategy se reduzca a pasar una función no es casualidad. Norvig defendió en 1996 que en los lenguajes con funciones como valores algunos patrones se vuelven mucho más simples

o desaparecen [18]. Java 21 está a medio camino: tiene lambdas, pero sigue pidiendo una interfaz que les dé tipo. Por eso aquí el patrón sigue teniendo su interfaz, y la lambda es solo una forma corta de escribir una implementación.

Si añades un segundo método, las lambdas dejan de compilar

Una lambda solo puede implementar una interfaz con un único método abstracto. Si algún día `DamageFormula` necesita otro método, o lo haces `default` (con cuerpo) o las lambdas del proyecto se rompen. Con `@FunctionalInterface` el error aparece en la propia interfaz y no en cada lambda.

Por qué `EnemyAi` no lleva la anotación aunque hoy también tenga un único método: porque es una pieza del juego con clases que guardan estado, y esperamos que crezca. Una interfaz marcada como funcional es una promesa de que seguirá teniendo un solo método.

2.5 Strategy: cuándo sí y cuándo no

Sí compensa cuando:

- Tienes varias formas de hacer lo mismo y el código que las elige crece con cada variante nueva (el olor de la primera sección).
- Quieres cambiar la variante sin tocar a quien la usa: por configuración, por elección del jugador o en mitad de la ejecución.
- Quieres probar cada variante por separado, o reutilizarla en otro contexto.

No compensa cuando:

- Hay dos variantes y no vas a tener más: un `boolean` o un `if` se lee mejor que una interfaz y dos clases.
- La variante es un *dato*, no un comportamiento. Si lo único que cambia entre dos enemigos es un número (el 3 fijo del esqueleto), basta un campo.
- Las variantes se parecen tanto que se resuelven con un parámetro.

Qué pagas. Más clases y un paso de indirección al leer el código. Y la elección no desaparece: se mueve a quien monta los objetos (`Main`), que debe conocer todas las variantes. Strategy no elimina el «cuál usar»; lo lleva a un único sitio razonable.

Una pista del contraste con Bridge

Dos jerarquías que varían de forma independiente (por ejemplo, qué se muestra y dónde se escribe) se parecen a Strategy, pero se resuelven con otro patrón, Bridge. Lo contrastamos en el capítulo 10, cuando el proyecto lo pida. Por ahora, `EnemyAi` y `DamageFormula` son dos estrategias independientes que no se conocen entre sí.

2.6 En el proyecto: H2 con Strategy

El hito H2 parte del estado del capítulo anterior y aplica Strategy a las dos decisiones del combate. Estos son los cambios:

- **Nuevo** `engine/combat/ai/`: `EnemyAi`, `FirstAliveAi`, `LowestHpAi` (el `weakest` del capítulo anterior) y `RandomTargetAi(Random)`.
- **Nuevo** `engine/combat/damage/`: `DamageFormula`, `FlatFormula` (ataque − defensa, mínimo 1) y `ScaledFormula` (ataque × 100 / (100 + defensa), mínimo 1).
- `Enemy` pasa a `Enemy(name, stats, ai)`; `Fighter` guarda la IA y gana `chooseTarget(heroes)`.
- `Combat` recibe la fórmula por constructor, `Combat(heroes, enemies, formula)`. Desaparecen los `if` por nombre y los auxiliares `firstAlive` y `weakest`.
- `GameSession` recibe la fórmula y se la pasa al combate.
- `ConsoleUi` gana `askFormula()`, que pregunta qué fórmula usar antes de empezar. `Main` fija la semilla del `Random` (42) y monta cada enemigo con su IA.

Cambia una regla del juego, y es deliberado. El esqueleto ya no hace un daño fijo de 3: la fórmula se aplica a todos. Con la plana, el esqueleto (7 de ataque) hace $7 - 2 = 5$ a Cael. Recuerda que el 3 fijo era uno de los olores del capítulo anterior.

Hito 2: IA y fórmula intercambiables

Tras este hito, cada enemigo lleva su propia IA (Goblin: `FirstAliveAi`; Rata de ceniza: `LowestHpAi`; Esqueleto: `RandomTargetAi`) y el jugador elige la fórmula de daño al empezar. Con la misma semilla, dos ejecuciones dan el mismo combate. Esta es la estructura:

Estructura de carpetas tras el capítulo 2

```
emberdeep/
|-- settings.gradle
|-- build.gradle
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
`-- src/main/java/emberdeep/
    |-- app/
    |   `-- Main.java              (composition root)
    |-- engine/
    |   |-- character/             Stats, Hero, Enemy, Fighter
    |   |-- combat/
    |   |   |-- Combat.java
    |   |   |-- ai/                EnemyAi, FirstAliveAi, LowestHpAi,
    ↪ RandomTargetAi
    |   |   `-- damage/            DamageFormula, FlatFormula,
    ↪ ScaledFormula
    |   `-- session/               GameSession
    `-- ui/
        `-- ConsoleUi.java
```

Lo practicas en el ejercicio 2.4. Antes tienes tres ejercicios más pequeños, en otros dominios, para dominar el patrón.

2.7 Ejercicios

Ejercicio 2.1: De un switch a una interfaz de puntuación ★☆☆☆☆

Este código calcula la puntuación de una partida según el modo. No es de Emberdeep: son dos archivos sueltos, `Mode.java` y `ScoreBoard.java`, sin paquete.

`ScoreBoard.java` y `Mode.java`

```
public enum Mode { EASY, HARD }

public class ScoreBoard {
    public int score(Mode mode, int kills, int secondsLeft) {
        return switch (mode) {
            case EASY -> kills * 10;
            case HARD -> kills * 25 + secondsLeft;
        };
    }
}
```

Conviértelo a Strategy con estos pasos (cada clase en su archivo):

1. Crea la interfaz `ScoreRule` con un método `int score(int kills, int secondsLeft)`.
2. Crea `EasyScore` y `HardScore`, que implementan `ScoreRule` con las dos fórmulas del `switch`.
3. Cambia `ScoreBoard`: recibe una `ScoreRule` en el constructor, la guarda y su método `score(int kills, int secondsLeft)` delega en ella. No debe quedar ningún `if` ni `switch`.
4. Escribe una clase `Main` que elija la regla según el primer argumento (`"hard"` elige `HardScore`; cualquier otra cosa, `EasyScore`) e imprima la puntuación de 4 muertes con 30 segundos restantes.
5. Borra `Mode`.

Está bien si, tras compilar con `javac *.java`, `java Main` imprime 40, `java Main hard` imprime 130, y ni `ScoreBoard` ni ningún archivo mencionan `Mode`.

Guiado ▪ solución en la pág. 40

Ejercicio 2.2: Lee un Strategy: envíos de una tienda ★★☆☆☆

Lee este código (no es de Emberdeep). Los precios son enteros en euros.

Shipping.java

```java
interface ShippingPolicy { int cost(int weightKg); }

class StandardShipping implements ShippingPolicy {
    public int cost(int weightKg) { return 3 + 2 * weightKg; }
}

class ExpressShipping implements ShippingPolicy {
    public int cost(int weightKg) { return 8 + 4 * weightKg; }
}

class FreeShipping implements ShippingPolicy {
    public int cost(int weightKg) { return 0; }
}

class Order {
    private final int weightKg;
    private ShippingPolicy policy;

    Order(int weightKg, ShippingPolicy policy) {
        this.weightKg = weightKg;
        this.policy = policy;
    }

    void changePolicy(ShippingPolicy policy) { this.policy = policy; }

    int total(int itemsPrice) { return itemsPrice + policy.cost(weightKg); }
}

public class Shipping {
    public static void main(String[] args) {
        Order order = new Order(2, new StandardShipping());
        System.out.println(order.total(20));
        order.changePolicy(new ExpressShipping());
        System.out.println(order.total(20));
    }
}
```

1. Haz una tabla con el rol de Strategy que cumple cada clase o interfaz (estrategia, estrategia concreta, contexto, cliente).
2. ¿Qué imprime el programa?
3. Quieres añadir `PickupShipping` (recogida en tienda, coste 1 euro). ¿Qué archivo o clase hay que tocar? ¿Cuál no?
4. ¿Dónde está ahora la decisión de qué política usar?

Está bien si cada rol tiene su clase, la salida es correcta y la respuesta a la pregunta 3 distingue lo que se añade de lo que se deja intacto.

Lectura de código ▪ solución en la pág. 41

Ejercicio 2.3: Un guardián que se pone a la defensiva ★★★☆☆

Partes de esta interfaz y de un guardián con vida (no es de Emberdeep; dos archivos, `Behavior.java` y `Guardian.java`):

`Behavior.java` y `Guardian.java`

```
@FunctionalInterface
public interface Behavior {
    String act(String actor);
}

public class Guardian {
    private final String name;
    private final int maxHp;
    private int hp;

    public Guardian(String name, int maxHp) {
        this.name = name;
        this.maxHp = maxHp;
        this.hp = maxHp;
    }

    public void takeDamage(int amount) {
        hp = Math.max(0, hp - amount);
    }
}
```

Haz que el guardián cambie de comportamiento en mitad del combate:

1. `Guardian` recibe en el constructor dos comportamientos, `Guardian(String name, int maxHp, Behavior opening, Behavior defensive)`. Empieza con `opening`. Añade `setBehavior(Behavior)` y `String act()`, que delega en el comportamiento actual pasándole su nombre.
2. En `takeDamage`, cuando tras el daño le queda *menos del 30 %* de su vida máxima, el guardián pasa a `defensive`. Calcula el umbral solo con enteros. La decisión de cambiar vive en `Guardian`, no en los comportamientos.
3. En un `Main`, define los dos comportamientos como **lambdas**: el primero devuelve `actor + " ataca con furia."` y el segundo `actor + " se cubre con el escudo."`. Crea un `Guardian("Golem", 20, ...)`, y llama en este orden a `act()`, `takeDamage(5)`, `act()`, `takeDamage(10)`, `act()`, imprimiendo lo que devuelve cada `act()`.

Está bien si la salida es exactamente:

```
Golem ataca con furia.
Golem ataca con furia.
Golem se cubre con el escudo.
```

y si, con 20 de vida máxima, un guardián al que le quedan justo 6 (el 30 %) *no* cambia.

Pista: *Menos del 30 % de `maxHp` equivale a `hp * 100 < maxHp * 30`, sin decimales.*

Práctica ▪ solución en la pág. 42

Ejercicio 2.4: Hito 2: IA y fórmula de daño en Emberdeep ★★★☆☆

Implementa el hito H2 partiendo del estado del capítulo anterior (solución del ejercicio 1.5).

- Crea `engine/combat/ai/` con `EnemyAi` (`Fighter choose(Fighter self, List<Fighter> heroes)`), `FirstAliveAi`, `LowestHpAi` (el héroe vivo con menos vida; `Comparator.comparingInt(Fighter::hp)` y `min`) y `RandomTargetAi`, que recibe un `Random` por constructor y elige con `nextInt` entre los héroes vivos.
- Crea `engine/combat/damage/` con `DamageFormula` (`int damage(int attack, int defense)`, funcional), `FlatFormula` y `ScaledFormula`, ambas con mínimo 1.
- `Enemy` pasa a `Enemy(name, stats, ai)`. `Fighter` guarda la IA y tiene `chooseTarget(heroes)`. `Combat` recibe la fórmula, no tiene ningún `if` por nombre y ya no tiene `firstAlive` ni `weakest`. `GameSession` recibe la fórmula. `ConsoleUi` gana `int askFormula()`, que muestra las dos opciones y devuelve 1 o 2.
- `Main` crea `ConsoleUi` primero, elige la fórmula con `askFormula()`, crea *un* `Random(42)` y monta a Goblin con `FirstAliveAi`, a la Rata de ceniza con `LowestHpAi` y al Esqueleto con `RandomTargetAi`.

Está bien si:

1. `./gradlew run` compila y pregunta la fórmula antes de empezar.
2. Eligiendo `1` y tecleando siempre `1`, la primera ronda muestra estos mensajes y la partida acaba en la ronda 3 con «¡Victoria!» (en el turno de Borin de la ronda 3 se ve `Héroes: Aria 26/30, Borin 40/40, Cael 16/24`):

   Mensajes de la primera ronda con la fórmula plana

   ```
   Aria ataca a Goblin y le hace 7 de daño.
   Borin ataca a Goblin y le hace 5 de daño.
   Cael ataca a Goblin y le hace 8 de daño.
   Goblin queda fuera de combate.
   Rata de ceniza ataca a Cael y le hace 3 de daño.
   Esqueleto ataca a Cael y le hace 5 de daño.
   ```

3. Eligiendo `2`, esos dos últimos mensajes dicen 4 y 6 de daño.
4. Repetir la ejecución con la misma elección da exactamente los mismos mensajes.

Pista: *`Combat` calcula el daño con `formula.damage(atacante.stats().attack(), defensor.stats().defense())`; la IA solo decide el objetivo.*

Proyecto ▪ solución en la pág. 44

2.8 Resumen

Resumen

- El olor de Strategy es una cadena de `if` o `switch` que elige entre variantes de un comportamiento. El patrón saca cada variante a un objeto con la misma interfaz.
- Piezas: la estrategia (interfaz), las estrategias concretas y el contexto, que guarda una y le delega. En Emberdeep: `EnemyAi`, `FirstAliveAi`… y `Fighter`.
- Añadir una variante es añadir una clase; el contexto no se toca. La elección de cuál usar

se mueve al composition root.

- Si la interfaz tiene un único método y la variante no guarda estado, una lambda basta (`DamageFormula`). Una clase es mejor con estado o con un nombre del dominio.
- `Random` se inyecta con semilla fija desde `Main`: las partidas son repetibles y todas las clases comparten el mismo generador.
- No uses Strategy para dos variantes que no crecerán ni para simples datos: un `boolean` o un campo se lee mejor.
- H2: los `if` por nombre desaparecen; cada enemigo lleva su IA y el jugador elige la fórmula de daño al empezar. Cambia una regla (el esqueleto ya no hace daño fijo).

Soluciones de los ejercicios

Solución del ejercicio 2.1 enunciado en la pág. 36

Los cinco archivos finales (sin `Mode.java`):

`ScoreRule.java`

```
public interface ScoreRule {
    int score(int kills, int secondsLeft);
}
```

`EasyScore.java`

```
public class EasyScore implements ScoreRule {
    @Override
    public int score(int kills, int secondsLeft) {
        return kills * 10;
    }
}
```

`HardScore.java`

```
public class HardScore implements ScoreRule {
    @Override
    public int score(int kills, int secondsLeft) {
        return kills * 25 + secondsLeft;
    }
}
```

ScoreBoard.java

```java
public class ScoreBoard {
    private final ScoreRule rule;

    public ScoreBoard(ScoreRule rule) {
        this.rule = rule;
    }

    public int score(int kills, int secondsLeft) {
        return rule.score(kills, secondsLeft);
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        ScoreRule rule = args.length > 0 && args[0].equals("hard")
                ? new HardScore()
                : new EasyScore();
        ScoreBoard board = new ScoreBoard(rule);
        System.out.println(board.score(4, 30));
    }
}
```

Ejecución:

```
$ javac *.java
$ java Main
40
$ java Main hard
130
```

Por qué funciona. `ScoreBoard` (el contexto) solo conoce `ScoreRule`; cada modo es una clase y el `switch` desapareció. El `?:` de `Main` es la elección, que se ha movido al único sitio que debe conocer las variantes. Easy: $4 \cdot 10 = 40$. Hard: $4 \cdot 25 + 30 = 130$.
Errores típicos: dejar el `switch` dentro de `ScoreBoard` y llamar a las clases desde ahí (se mueve el problema, no se cura); no borrar `Mode` y mantener dos formas de elegir; olvidar `@Override`, que sin él un error en la firma crea un método nuevo en vez de implementar el de la interfaz.

Solución del ejercicio 2.2

enunciado en la pág. 36

1. Roles:

Clase	Rol
`ShippingPolicy`	Estrategia (interfaz)
`StandardShipping`, `ExpressShipping`, `FreeShipping`	Estrategias concretas
`Order`	Contexto: guarda una política y le delega `cost`
`Shipping` (su `main`)	Cliente: crea las políticas y elige

2. Imprime `27` y `36`. Estándar: $20 + (3 + 2 \cdot 2) = 27$. Exprés, tras `changePolicy`: $20 + (8 + 4 \cdot 2) = 36$.
3. Solo se añade una clase nueva, `PickupShipping implements ShippingPolicy`, con `cost` devolviendo 1. `Order`, `ShippingPolicy` y las tres clases existentes no se tocan. Solo el cliente (`Shipping.main`) cambia, y únicamente si quiere usar la política nueva: una línea, `new PickupShipping()`.
4. En el cliente: es quien escribe `new StandardShipping()` y `new ExpressShipping()`. `Order` no sabe cuál tiene.

Errores típicos: tomar a `Order` por una estrategia (es la que *usa* la estrategia, no la que varía) y decir que hay que modificar `Order` para soportar la nueva política.

Solución del ejercicio 2.3

enunciado en la pág. 38

`Behavior.java`

```java
@FunctionalInterface
public interface Behavior {
    String act(String actor);
}
```

Guardian.java

```java
public class Guardian {
    private final String name;
    private final int maxHp;
    private final Behavior defensive;
    private Behavior behavior;
    private int hp;

    public Guardian(String name, int maxHp, Behavior opening, Behavior
    ↪ defensive) {
        this.name = name;
        this.maxHp = maxHp;
        this.behavior = opening;
        this.defensive = defensive;
        this.hp = maxHp;
    }

    public void setBehavior(Behavior behavior) {
        this.behavior = behavior;
    }

    public String act() {
        return behavior.act(name);
    }

    public void takeDamage(int amount) {
        hp = Math.max(0, hp - amount);
        if (hp * 100 < maxHp * 30) {
            setBehavior(defensive);
        }
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Behavior aggressive = actor -> actor + " ataca con furia.";
        Behavior defensive = actor -> actor + " se cubre con el escudo.";

        Guardian golem = new Guardian("Golem", 20, aggressive, defensive);
        System.out.println(golem.act());
        golem.takeDamage(5);
        System.out.println(golem.act());
        golem.takeDamage(10);
        System.out.println(golem.act());
    }
}
```

Por qué funciona. Tras 5 de daño le quedan 15: $15 \cdot 100 = 1500$ no es menor que $20 \cdot 30 = 600$, sigue agresivo. Tras 10 más le quedan 5: $500 < 600$, pasa a defensivo. Con 6 de vida, $600 < 600$ es falso: el umbral es «menos de» el 30 %, no «como mucho». Las lambdas implementan `Behavior` porque tiene un solo método; `Guardian` (el contexto)

toma la decisión de cambiar, y las estrategias ni se enteran. Una variante válida tipa los comportamientos como `Function<String, String>` del JDK y llama a `apply` en lugar de `act`, con el mismo resultado.
Errores típicos: poner el `if` del 30 % dentro de la lambda (la estrategia no conoce la vida); calcular con `double` y comparar con `0.3` (funciona, pero arrastra decimales que aquí no hacen falta); olvidar que `setBehavior` existe para que quien lo necesite cambie la estrategia desde fuera.

Solución del ejercicio 2.4 enunciado en la pág. 39

Parte del estado del capítulo 1. `Stats.java` y `Hero.java` no cambian. Cambian `Enemy`, `Fighter`, `Combat`, `GameSession`, `ConsoleUi` y `Main`, y hay siete archivos nuevos.
engine/combat/ai/

EnemyAi.java

```java
package emberdeep.engine.combat.ai;

import emberdeep.engine.character.Fighter;
import java.util.List;

public interface EnemyAi {
    Fighter choose(Fighter self, List<Fighter> heroes);
}
```

FirstAliveAi.java

```java
package emberdeep.engine.combat.ai;

import emberdeep.engine.character.Fighter;
import java.util.List;

public class FirstAliveAi implements EnemyAi {
    @Override
    public Fighter choose(Fighter self, List<Fighter> heroes) {
        return heroes.stream().filter(Fighter::isAlive).findFirst().
    ↪ orElseThrow();
    }
}
```

LowestHpAi.java

```java
package emberdeep.engine.combat.ai;

import emberdeep.engine.character.Fighter;
import java.util.Comparator;
import java.util.List;

public class LowestHpAi implements EnemyAi {
    @Override
    public Fighter choose(Fighter self, List<Fighter> heroes) {
        return heroes.stream()
                .filter(Fighter::isAlive)
                .min(Comparator.comparingInt(Fighter::hp))
                .orElseThrow();
    }
}
```

RandomTargetAi.java

```java
package emberdeep.engine.combat.ai;

import emberdeep.engine.character.Fighter;
import java.util.List;
import java.util.Random;

public class RandomTargetAi implements EnemyAi {
    private final Random random;

    public RandomTargetAi(Random random) {
        this.random = random;
    }

    @Override
    public Fighter choose(Fighter self, List<Fighter> heroes) {
        List<Fighter> alive = heroes.stream().filter(Fighter::isAlive).
    ↪ toList();
        return alive.get(random.nextInt(alive.size()));
    }
}
```

engine/combat/damage/

DamageFormula.java

```java
package emberdeep.engine.combat.damage;

@FunctionalInterface
public interface DamageFormula {
    int damage(int attack, int defense);
}
```

FlatFormula.java

```java
package emberdeep.engine.combat.damage;

public class FlatFormula implements DamageFormula {
    @Override
    public int damage(int attack, int defense) {
        return Math.max(1, attack - defense);
    }
}
```

ScaledFormula.java

```java
package emberdeep.engine.combat.damage;

public class ScaledFormula implements DamageFormula {
    @Override
    public int damage(int attack, int defense) {
        return Math.max(1, attack * 100 / (100 + defense));
    }
}
```

engine/character/Enemy.java y Fighter.java

Enemy.java

```java
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;

public record Enemy(String name, Stats stats, EnemyAi ai) {}
```

Fighter.java

```java
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;
import java.util.List;

public class Fighter {
    private final String name;
    private final Stats stats;
    private final EnemyAi ai;   // null en los héroes: los elige el jugador
    private int hp;

    private Fighter(String name, Stats stats, EnemyAi ai) {
        this.name = name;
        this.stats = stats;
        this.ai = ai;
        this.hp = stats.maxHp();
    }

    public static Fighter ofHero(Hero hero) {
        return new Fighter(hero.name(), hero.stats(), null);
    }

    public static Fighter ofEnemy(Enemy enemy) {
        return new Fighter(enemy.name(), enemy.stats(), enemy.ai());
    }

    public String name() { return name; }
    public Stats stats() { return stats; }
    public int hp() { return hp; }
    public boolean isAlive() { return hp > 0; }

    public void takeDamage(int amount) {
        hp = Math.max(0, hp - amount);
    }

    public Fighter chooseTarget(List<Fighter> heroes) {
        if (ai == null) {
            throw new IllegalStateException(name + " no tiene IA: es un hé
    ↪ roe");
        }
        return ai.choose(this, heroes);
    }
}
```

engine/combat/Combat.java (en dos tramos seguidos, dentro de la misma clase)

Combat.java (1/2: estado y turno de los héroes)

```java
package emberdeep.engine.combat;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.damage.DamageFormula;
import java.util.ArrayList;
import java.util.List;

public class Combat {
    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final DamageFormula formula;
    private int round = 1;

    public Combat(List<Fighter> heroes, List<Fighter> enemies,
    ↪ DamageFormula formula) {
        this.heroes = heroes;
        this.enemies = enemies;
        this.formula = formula;
    }

    public int round() {
        return round;
    }

    public boolean heroesWon() {
        return enemies.stream().noneMatch(Fighter::isAlive);
    }

    public boolean heroesLost() {
        return heroes.stream().noneMatch(Fighter::isAlive);
    }

    public boolean isOver() {
        return heroesWon() || heroesLost();
    }

    public List<String> heroAttack(Fighter hero, Fighter target) {
        List<String> messages = new ArrayList<>();
        hit(hero, target, damageOf(hero, target), messages);
        return messages;
    }
```

Combat.java (2/2: turno de los enemigos y auxiliares)

```

    public List<String> enemyPhase() {
        List<String> messages = new ArrayList<>();
        for (Fighter enemy : enemies) {
            if (!enemy.isAlive()) {
                continue;
            }
            if (heroesLost()) {
                break;
            }
            Fighter target = enemy.chooseTarget(heroes);
            hit(enemy, target, damageOf(enemy, target), messages);
        }
        round++;
        return messages;
    }

    private void hit(Fighter attacker, Fighter target, int damage, List<
    ↪ String> messages) {
        target.takeDamage(damage);
        messages.add(attacker.name() + " ataca a " + target.name()
                + " y le hace " + damage + " de daño.");
        if (!target.isAlive()) {
            messages.add(target.name() + " queda fuera de combate.");
        }
    }

    private int damageOf(Fighter attacker, Fighter target) {
        return formula.damage(attacker.stats().attack(), target.stats().
    ↪ defense());
    }
}
```

engine/session/GameSession.java

GameSession.java

```java
package emberdeep.engine.session;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.DamageFormula;
import java.util.List;

public class GameSession {
    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final Combat combat;

    public GameSession(List<Hero> heroes, List<Enemy> floorOneEnemies,
    ↪ DamageFormula formula) {
        this.heroes = heroes.stream().map(Fighter::ofHero).toList();
        this.enemies = floorOneEnemies.stream().map(Fighter::ofEnemy).
    ↪ toList();
        this.combat = new Combat(this.heroes, this.enemies, formula);
    }

    public List<Fighter> heroes() { return heroes; }
    public List<Fighter> enemies() { return enemies; }
    public Combat combat() { return combat; }
    public int round() { return combat.round(); }
}
```

ui/ConsoleUi.java (en dos tramos seguidos, dentro de la misma clase)

ConsoleUi.java (1/2: fórmula y partida)

```java
package emberdeep.ui;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.session.GameSession;
import java.io.PrintStream;
import java.util.List;
import java.util.Scanner;
import java.util.stream.Collectors;

public class ConsoleUi {
    private final Scanner in;
    private final PrintStream out;

    public ConsoleUi(Scanner in, PrintStream out) {
        this.in = in;
        this.out = out;
    }

    public int askFormula() {
        out.println("Fórmula de daño:");
        out.println("  [1] plana: ataque - defensa");
        out.println("  [2] escalada: ataque x 100 / (100 + defensa)");
        return ask(2);
    }

    public void run(GameSession session) {
        Combat combat = session.combat();
        out.println("=== Emberdeep: planta 1 ===");
        while (!combat.isOver()) {
            out.println("--- Ronda " + session.round() + " ---");
            for (Fighter hero : session.heroes()) {
                if (!hero.isAlive() || combat.isOver()) {
                    continue;
                }
                out.println("Héroes:   " + describe(session.heroes()));
                out.println("Enemigos: " + describe(session.enemies()));
                out.println("Turno de " + hero.name() + ". [1] atacar");
                ask(1);
                Fighter target = askTarget(session.enemies());
                print(combat.heroAttack(hero, target));
            }
            if (!combat.isOver()) {
                print(combat.enemyPhase());
            }
        }
        out.println(combat.heroesWon() ? "¡Victoria!" : "Derrota...");
    }
```

ConsoleUi.java (2/2: auxiliares y llave final)

```
    private String describe(List<Fighter> group) {
        return group.stream()
                .map(f -> f.name() + " " + f.hp() + "/" + f.stats().maxHp())
                .collect(Collectors.joining(", "));
    }

    private void print(List<String> messages) {
        messages.forEach(out::println);
    }

    private int ask(int max) {
        while (true) {
            out.print("> ");
            out.flush();
            String line = in.nextLine().trim();
            try {
                int n = Integer.parseInt(line);
                if (n >= 1 && n <= max) {
                    return n;
                }
            } catch (NumberFormatException e) {
                // no es un número: se avisa más abajo
            }
            out.println("Escribe un número entre 1 y " + max + ".");
        }
    }

    private Fighter askTarget(List<Fighter> enemies) {
        List<Fighter> alive = enemies.stream().filter(Fighter::isAlive).
toList();
        StringBuilder menu = new StringBuilder("Objetivo:");
        for (int i = 0; i < alive.size(); i++) {
            menu.append("  [").append(i + 1).append("] ").append(alive.get(
i).name());
        }
        out.println(menu);
        return alive.get(ask(alive.size()) - 1);
    }
}
```

app/Main.java

Main.java

```java
package emberdeep.app;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.session.GameSession;
import emberdeep.ui.ConsoleUi;
import java.util.List;
import java.util.Random;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        ConsoleUi ui = new ConsoleUi(new Scanner(System.in), System.out);
        DamageFormula formula = ui.askFormula() == 1
                ? new FlatFormula()
                : new ScaledFormula();
        Random random = new Random(42);

        List<Hero> heroes = List.of(
                new Hero("Aria", new Stats(30, 8, 3)),
                new Hero("Borin", new Stats(40, 6, 5)),
                new Hero("Cael", new Stats(24, 9, 2)));
        List<Enemy> floorOne = List.of(
                new Enemy("Goblin", new Stats(14, 6, 1), new FirstAliveAi())
    ↪ ,
                new Enemy("Rata de ceniza", new Stats(8, 5, 0), new
    ↪ LowestHpAi()),
                new Enemy("Esqueleto", new Stats(18, 7, 2), new
    ↪ RandomTargetAi(random)));
        GameSession session = new GameSession(heroes, floorOne, formula);
        ui.run(session);
    }
}
```

Por qué funciona. Cada decisión del combate es una estrategia que `Main` monta: `Fighter` consulta la IA de su `Enemy`, y `Combat` pide el daño a la fórmula. En `Combat` ya no hay nombres de enemigos. `Main` crea un único `Random(42)` y lo comparte con la IA aleatoria. Con la fórmula plana y siempre `1`: en la ronda 1 el goblin cae (7+5+8 sobre 14), la rata ataca a Cael (el de menos vida: 24) con $5 - 2 = 3$, y el esqueleto, con el primer `nextInt(3)` de la semilla 42, que es 2, elige al tercer héroe y le hace $7 - 2 = 5$. En la ronda 2 la rata cae a manos de Aria y el esqueleto golpea a Aria (4); en la ronda 3 Aria y Borin lo derrotan. Con la escalada, el daño a Cael es 4 y 6 (500/102 y 700/102).

Errores típicos:

- Crear un `new Random(42)` por enemigo o dentro de `RandomTargetAi`: la IA deja de ser repetible o repite la misma secuencia.

- Olvidar quitar `firstAlive` y `weakest` de `Combat`, o dejar un resto de `if` por nombre.
- Llamar a `askFormula()` con la sesión ya creada: `GameSession` necesita la fórmula en el constructor, por eso `Main` crea primero la `ConsoleUi`.
- Usar `chooseTarget` con un héroe: lanza `IllegalStateException`.

Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo siguiente.

Parte II

Crear y compartir objetos

CAPÍTULO 3

Factory Method y Abstract Factory: enemigos y objetos por bioma

Objetivos del capítulo

- Reconocer el olor que pide una fábrica: `new` de clases concretas repartido por el código.
- Aplicar Factory Method: un creador abstracto fija el esqueleto y cada subclase decide qué producto crea.
- Aplicar Abstract Factory: una interfaz que crea una familia de productos que van juntos (enemigos y objetos de un bioma).
- Distinguir la fábrica simple con `enum` y `switch` exhaustivo de los dos patrones del GoF, y saber qué es nuevo si ya los conocías.
- Dar el hito H3: Emberdeep con dos biomas, un `Inventory` inmutable y los spawners retirados.

Hasta ahora el grupo se enfrentaba siempre a los mismos tres enemigos, montados a mano en `Main`. Emberdeep va a tener plantas distintas: una cripta con esqueletos y una caverna con goblins, cada una con sus propios objetos. Si cada sitio que necesita un enemigo escribe su `new Enemy(...)`, cualquier cambio en cómo se construye un enemigo toca medio proyecto, y nada impide que una cripta reparta una poción de caverna. En este capítulo repartes la creación entre fábricas, con dos patrones que se parecen pero resuelven problemas distintos.

3.1 El olor que pide Factory Method: `new` por todas partes

Así monta `Main` los enemigos de una planta con lo que tienes tras el capítulo anterior, y así se vería si añadieras una segunda planta:

Main.java (fragmento): dos plantas a mano

```
List<Enemy> floorOne = List.of(
        new Enemy("Esqueleto", new Stats(18, 7, 2), new RandomTargetAi(random))
    ↪ ,
        new Enemy("Rata de ceniza", new Stats(8, 5, 0), new LowestHpAi()),
        new Enemy("Zombi", new Stats(22, 5, 1), new FirstAliveAi()));
List<Enemy> floorTwo = List.of(
        new Enemy("Goblin", new Stats(14, 6, 1), new FirstAliveAi()),
        new Enemy("Murciélago", new Stats(10, 6, 0), new LowestHpAi()),
        new Enemy("Araña", new Stats(16, 7, 2), new RandomTargetAi(random)));
```

Tres cosas duelen aquí:

- **Quien crea conoce las clases concretas.** `Main` importa `FirstAliveAi`, `LowestHpAi` y `RandomTargetAi`, y sabe qué IA y qué estadísticas lleva cada enemigo. Esos datos de

diseño del juego no son cosa del composition root.

- **El `new` se repite.** En este capítulo `Enemy` va a ganar un campo (`kind`) y su constructor cambiará. Cada `new Enemy(...)` del proyecto dejará de compilar a la vez. Con diez sitios que creen enemigos, son diez ediciones.
- **Nada centraliza las reglas de creación.** Si un día todo enemigo debe nacer con IA (un `null` es un error), tendrás que acordarte de comprobarlo en cada `new`.

Ese es el olor: **el cliente decide la clase concreta que crea, y lo hace en muchos sitios**. Ya conoces una primera cura, la fábrica simple (un método estático con un `switch`); la comparamos con los patrones del GoF en la sección 3.7. Aquí empezamos por Factory Method, cuya intención en el catálogo es definir una interfaz para crear un objeto y dejar que las subclases decidan qué clase instanciar [9]. Dicho con nuestro código: una clase abstracta sabe *cuándo* se crea un enemigo y qué reglas cumple siempre; cada subclase sabe *cuál*.

3.2 Piezas del patrón Factory Method

Factory Method tiene cinco piezas. El *producto* es lo que se crea. El *creador* es una clase abstracta que usa el producto y declara el *método fábrica*, un método abstracto que devuelve el producto. Los *creadores concretos* implementan ese método y eligen la clase concreta. El *cliente* trabaja solo con el creador.

Rol	Clase en el ejemplo	Propósito	Paquete
Producto (record)	`Enemy`	El enemigo que se crea	`engine.character`
Creador (clase abstracta)	`EnemySpawner`	Fija reglas; `spawn` usa `create`	`engine.world.biome`
Creadores concretos (clases)	`CryptSpawner`, `CavernSpawner`	Crean los enemigos de su bioma	`engine.world.biome`
Método fábrica (método abstracto)	`create(EnemyKind)`	Elige la clase concreta	`engine.world.biome`
Cliente (clase)	`GameSession`	Pide enemigos al creador	`engine.session`

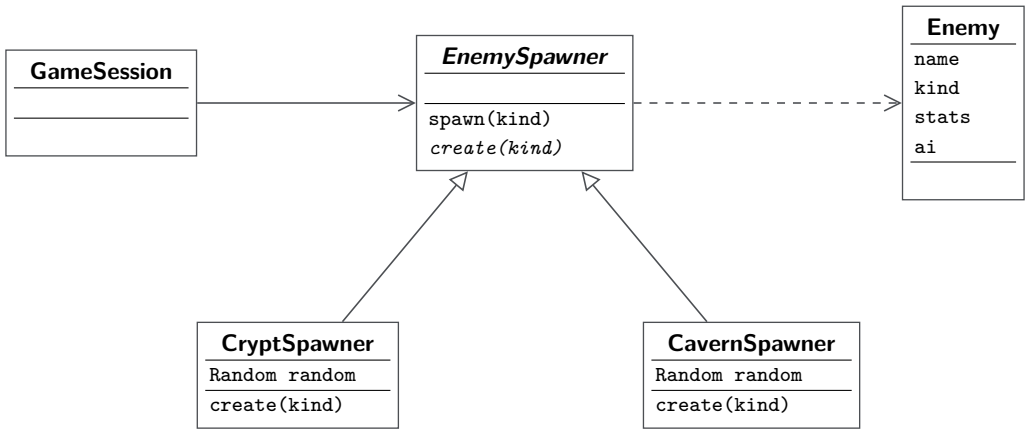


Diagrama UML del patrón Factory Method: `GameSession` conoce solo a `EnemySpawner`; cada subclase decide qué `Enemy` crea.

Léelo con la leyenda del capítulo 1. Los nombres en cursiva son abstractos. La flecha discontinua de `EnemySpawner` a `Enemy` dice que el creador depende del producto; las flechas de herencia, que las dos subclases rellenan el hueco. `GameSession` no apunta a ninguna clase concreta.

3.3 El ejemplo de Factory Method, pieza a pieza: los spawners

3.3.1 El producto: `Enemy` gana un `kind`

Para que un método fábrica sepa qué crear necesita una petición. Usamos un `enum`, el tipo de Java para un conjunto cerrado de valores con nombre. `EnemyKind` tiene seis tipos, tres por bioma:

engine/character/EnemyKind.java

```
package emberdeep.engine.character;

public enum EnemyKind {
    SKELETON, ASH_RAT, ZOMBIE,    // cripta
    GOBLIN, BAT, SPIDER           // caverna
}
```

El record `Enemy` guarda el tipo como un campo más. Es el cambio de forma que anunciaba el olor: desde ahora, `Enemy(name, kind, stats, ai)`.

engine/character/Enemy.java

```
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;

public record Enemy(String name, EnemyKind kind, Stats stats, EnemyAi ai) {}
```

`Fighter` no cambia: sigue leyendo nombre, estadísticas e IA del `Enemy`. Guardar el tipo no es un capricho. Más adelante hará falta saber de qué clase de enemigo se trata (sus resistencias, su plantilla) sin mirar su nombre, el mismo `if` por texto que curaste en el capítulo anterior.

3.3.2 El creador: `EnemySpawner`

`EnemySpawner` es una clase abstracta con dos métodos. `create` es el método fábrica: abstracto, lo implementan las subclases. `spawn` es el que usa el cliente y añade lo que todos los creadores comparten, una regla que ninguna subclase tiene que repetir:

engine/world/biome/EnemySpawner.java

```
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;

public abstract class EnemySpawner {

    public final Enemy spawn(EnemyKind kind) {
```

```
        Enemy enemy = create(kind);
        if (enemy.ai() == null) {
            throw new IllegalStateException(enemy.name() + " nace sin IA");
        }
        return enemy;
    }

    protected abstract Enemy create(EnemyKind kind);
}
```

`spawn` es `final`: el esqueleto no se puede saltar. `create` es `protected`: solo la jerarquía lo llama, el cliente pasa por `spawn`. Esa estructura, un método que fija los pasos y deja uno abierto a las subclases, es una idea que reencontrarás en otros patrones; aquí lo que importa es que el *cuándo* y las reglas viven en un único sitio.

3.3.3 Los creadores concretos

Cada bioma sabe qué enemigos tiene, con qué estadísticas y con qué IA. La cripta:

engine/world/biome/CryptSpawner.java

```
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import java.util.Random;

public class CryptSpawner extends EnemySpawner {
    private final Random random;

    public CryptSpawner(Random random) {
        this.random = random;
    }

    @Override
    protected Enemy create(EnemyKind kind) {
        return switch (kind) {
            case SKELETON -> new Enemy("Esqueleto", kind,
                    new Stats(18, 7, 2), new RandomTargetAi(random));
            case ASH_RAT -> new Enemy("Rata de ceniza", kind,
                    new Stats(8, 5, 0), new LowestHpAi());
            case ZOMBIE -> new Enemy("Zombi", kind,
                    new Stats(22, 5, 1), new FirstAliveAi());
            case GOBLIN, BAT, SPIDER -> throw new IllegalArgumentException(
                    kind + " no vive en la cripta");
        };
    }
}
```

El `Random` llega por constructor, como en el capítulo anterior: `Main` crea uno solo y lo reparte. Fíjate en el `switch`. Es una *expresión* (devuelve un valor) sobre un `enum` y no tiene `default`:

nombramos los seis valores. Si mañana añades un séptimo a `EnemyKind`, el compilador marcará este `switch` como incompleto y te obligará a decidir qué hace la cripta con él. La caverna es la misma estructura con otros datos:

engine/world/biome/CavernSpawner.java

```
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import java.util.Random;

public class CavernSpawner extends EnemySpawner {
    private final Random random;

    public CavernSpawner(Random random) {
        this.random = random;
    }

    @Override
    protected Enemy create(EnemyKind kind) {
        return switch (kind) {
            case GOBLIN -> new Enemy("Goblin", kind,
                    new Stats(14, 6, 1), new FirstAliveAi());
            case BAT -> new Enemy("Murciélago", kind,
                    new Stats(10, 6, 0), new LowestHpAi());
            case SPIDER -> new Enemy("Araña", kind,
                    new Stats(16, 7, 2), new RandomTargetAi(random));
            case SKELETON, ASH_RAT, ZOMBIE -> throw new
    ↪ IllegalArgumentException(
                    kind + " no vive en la caverna");
        };
    }
}
```

3.3.4 El cliente: `GameSession`

El cliente recibe un `EnemySpawner` y la lista de tipos que quiere. No sabe si detrás hay una cripta o una caverna:

GameSession.java (fragmento: el constructor con spawner)

```
public GameSession(List<Hero> heroes, EnemySpawner spawner,
                   List<EnemyKind> kinds, DamageFormula formula) {
    this.heroes = heroes.stream().map(Fighter::ofHero).toList();
    this.enemies = kinds.stream()
            .map(spawner::spawn)
            .map(Fighter::ofEnemy)
            .toList();
    this.combat = new Combat(this.heroes, this.enemies, formula);
}
```

`spawner::spawn` es una referencia a método, como las del capítulo anterior. Añadir un tercer bioma es escribir una subclase de `EnemySpawner` y cambiar una línea de `Main`. Nadie más se entera.

Una sola creación por llamada

Cada `spawn` construye un `Enemy` y una IA nuevos. `RandomTargetAi` guarda un `Random`, y lo que se comparte entre todas las IA es ese único `Random` de `Main`, la fuente de azar de la partida, nunca uno por IA.

3.4 El olor que pide Abstract Factory: un esqueleto con una poción de caverna

Los enemigos ya salen de la fábrica correcta. Ahora Emberdeep necesita *objetos*. Un `Item` tiene nombre, tipo (`ItemKind`: `HEALING` o `BOMB`) y potencia; lo defines en la sección siguiente. Una poción cura y una bomba hace daño. Cada bioma tiene los suyos (la cripta, con la poción de cripta y bombas de ceniza; la caverna, con la poción de caverna y bombas de roca). La tentación es repetir el truco:

Main.java (fragmento): dos decisiones por separado

```
EnemySpawner spawner = floor == 1
        ? new CryptSpawner(random)
        : new CavernSpawner(random);
Item bomb = floor == 2
        ? new Item("Bomba de ceniza", ItemKind.BOMB, 10)
        : new Item("Bomba de roca", ItemKind.BOMB, 14);
```

Lee la segunda condición con atención: dice `floor == 2`, la contraria de la primera. Un descuido y la cripta reparte una bomba de roca. El compilador no avisa, porque ambas cosas son un `Item` y un `EnemySpawner` válidos. Este es el olor: **objetos que deben ir juntos (los de un bioma) se eligen con decisiones independientes**. Cada vez que añadas un tipo de objeto, hay otra condición que mantener sincronizada con las demás.

Abstract Factory lo cura dando una sola decisión: se elige *una* fábrica de bioma y de ella salen enemigos y objetos. Su intención en el catálogo es proporcionar una interfaz para crear familias de objetos relacionados sin especificar sus clases concretas [9]. La mezcla pasa de ser un descuido posible a un caso en el que ni siquiera hay dónde equivocarse: el cliente sostiene una sola fábrica.

3.5 Piezas del patrón Abstract Factory

Abstract Factory tiene cinco piezas. La *fábrica abstracta* es una interfaz con un método por cada producto de la familia. Las *fábricas concretas* la implementan, una por familia. Los *productos abstractos* son los tipos que devuelve la interfaz. Los *productos concretos* son lo que cada fábrica crea de verdad. El *cliente* usa solo las interfaces.

Rol	Clase en el ejemplo	Propósito	Paquete
Fábrica abstracta (interfaz)	`BiomeFactory`	Crea enemigos y objetos	`engine.world.biome`
Fábricas concretas (clases)	`CryptBiome`, `CavernBiome`	Una familia coherente cada una	`engine.world.biome`
Productos abstractos (records)	`Enemy`; `Item` (con `ItemKind`, un `enum` aparte)	Lo que devuelve la fábrica	`engine.character`; `engine.item`
Productos concretos	Los enemigos y objetos de cada bioma	Datos propios del bioma	`engine.world.biome`
Cliente (clase)	`GameSession`	Usa una sola fábrica	`engine.session`

Los productos «abstractos» de Emberdeep son records: no hay una jerarquía de clases `Goblin extends Enemy`. Lo que distingue a un goblin de un esqueleto son datos (nombre, tipo, estadísticas, IA), y en Java moderno los datos se modelan con un record. El patrón no exige herencia en los productos: exige que el cliente reciba productos de una misma familia sin nombrar su clase.

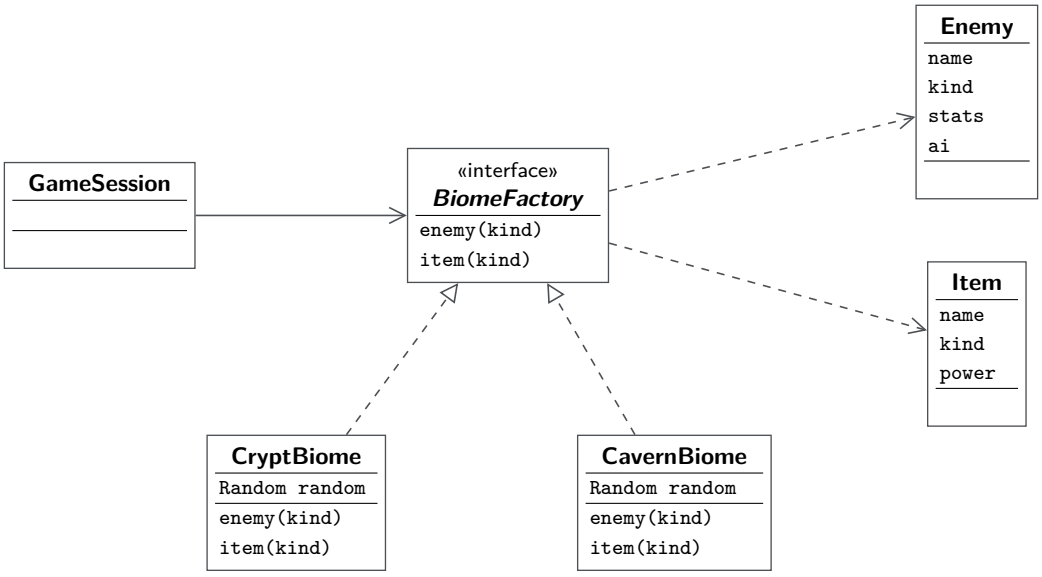


Diagrama UML del patrón Abstract Factory: `GameSession` conoce solo la interfaz `BiomeFactory`; cada bioma entrega su familia de `Enemy` e `Item`.

3.6 El ejemplo de Abstract Factory, pieza a pieza: los biomas

3.6.1 Los productos: `Item` e `ItemKind`

El segundo producto es el objeto. Un `Item` tiene nombre, un tipo y una potencia (los puntos de vida que cura una poción o el daño de una bomba). Como con los enemigos, el tipo es un `enum` aparte:

engine/item/ItemKind.java

```
1 package emberdeep.engine.item;
2
```

```
3 public enum ItemKind { HEALING, BOMB }
```

engine/item/Item.java

```
1 package emberdeep.engine.item;
2
3 public record Item(String name, ItemKind kind, int power) {}
```

Se crean en el paquete nuevo `engine/item/`, que también acogerá el inventario.

3.6.2 La fábrica abstracta y las dos fábricas concretas

La interfaz tiene un método por producto. Por convención de este libro, nombres cortos y en minúscula, como en el diagrama:

engine/world/biome/BiomeFactory.java

```
1  package emberdeep.engine.world.biome;
2
3  import emberdeep.engine.character.Enemy;
4  import emberdeep.engine.character.EnemyKind;
5  import emberdeep.engine.item.Item;
6  import emberdeep.engine.item.ItemKind;
7
8  public interface BiomeFactory {
9      Enemy enemy(EnemyKind kind);
10
11     Item item(ItemKind kind);
12 }
```

`CryptBiome` reúne lo que hacía `CryptSpawner` y añade los objetos de la cripta. Es la misma estructura de `switch` sin `default`:

engine/world/biome/CryptBiome.java

```
1  package emberdeep.engine.world.biome;
2
3  import emberdeep.engine.character.Enemy;
4  import emberdeep.engine.character.EnemyKind;
5  import emberdeep.engine.character.Stats;
6  import emberdeep.engine.combat.ai.FirstAliveAi;
7  import emberdeep.engine.combat.ai.LowestHpAi;
8  import emberdeep.engine.combat.ai.RandomTargetAi;
9  import emberdeep.engine.item.Item;
10 import emberdeep.engine.item.ItemKind;
11 import java.util.Random;
12
13 public class CryptBiome implements BiomeFactory {
14     private final Random random;
15
16     public CryptBiome(Random random) {
17         this.random = random;
18     }
19
```

```
    @Override
    public Enemy enemy(EnemyKind kind) {
        return switch (kind) {
            case SKELETON -> new Enemy("Esqueleto", kind,
                    new Stats(18, 7, 2), new RandomTargetAi(random));
            case ASH_RAT -> new Enemy("Rata de ceniza", kind,
                    new Stats(8, 5, 0), new LowestHpAi());
            case ZOMBIE -> new Enemy("Zombi", kind,
                    new Stats(22, 5, 1), new FirstAliveAi());
            case GOBLIN, BAT, SPIDER -> throw new IllegalArgumentException(
                    kind + " no vive en la cripta");
        };
    }

    @Override
    public Item item(ItemKind kind) {
        return switch (kind) {
            case HEALING -> new Item("Poción de cripta", kind, 12);
            case BOMB -> new Item("Bomba de ceniza", kind, 10);
        };
    }
}
```

engine/world/biome/CavernBiome.java

```
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import java.util.Random;

public class CavernBiome implements BiomeFactory {
    private final Random random;

    public CavernBiome(Random random) {
        this.random = random;
    }

    @Override
    public Enemy enemy(EnemyKind kind) {
        return switch (kind) {
            case GOBLIN -> new Enemy("Goblin", kind,
                    new Stats(14, 6, 1), new FirstAliveAi());
            case BAT -> new Enemy("Murciélago", kind,
                    new Stats(10, 6, 0), new LowestHpAi());
            case SPIDER -> new Enemy("Araña", kind,
                    new Stats(16, 7, 2), new RandomTargetAi(random));
            case SKELETON, ASH_RAT, ZOMBIE -> throw new
    ↪ IllegalArgumentException(
                    kind + " no vive en la caverna");
```

```
        };
    }

    @Override
    public Item item(ItemKind kind) {
        return switch (kind) {
            case HEALING -> new Item("Poción de caverna", kind, 10);
            case BOMB -> new Item("Bomba de roca", kind, 14);
        };
    }
}
```

3.6.3 El cliente con una sola fábrica

`GameSession` cambia el `EnemySpawner` por un `BiomeFactory`. De la misma fábrica salen los enemigos de la planta y los objetos con los que empieza el grupo, así que no hay una segunda decisión que mantener sincronizada:

GameSession.java (fragmento: el constructor con la fábrica de bioma)

```
public GameSession(List<Hero> heroes, BiomeFactory biome,
                   List<EnemyKind> kinds, DamageFormula formula) {
    this.heroes = heroes.stream().map(Fighter::ofHero).toList();
    this.enemies = kinds.stream()
            .map(biome::enemy)
            .map(Fighter::ofEnemy)
            .toList();
    this.combat = new Combat(this.heroes, this.enemies, formula);
    this.inventory = new Inventory(List.of())
            .with(biome.item(ItemKind.HEALING))
            .with(biome.item(ItemKind.HEALING))
            .with(biome.item(ItemKind.BOMB));
}
```

`Inventory` se presenta en la sección 3.9. De momento fíjate en el patrón: una planta de cripta solo puede dar objetos de cripta porque `biome` es la única fuente.

La garantía tiene un límite

Abstract Factory garantiza la familia *si el cliente la usa como única fuente*. No impide que alguien escriba `new Item("Bomba de roca", ...)` en otro rincón del código, ni que se llame a `CryptBiome.enemy(EnemyKind.GOBLIN)`: por eso lanza una excepción. Con un `enum` compartido por los dos biomas el tipo no puede evitarlo. Se podría separar `EnemyKind` en un `enum` por bioma, con el coste de otra interfaz. Aquí decidimos que la excepción basta y que el `switch` sin `default` nos avisa al crecer.

3.6.4 Por qué los spawners se retiran

Mira lo que hace ahora `CryptBiome`: crea los mismos enemigos que `CryptSpawner` y además los objetos. `CryptSpawner` se ha quedado sin nada que aportar, y mantener las dos clases significa mantener dos copias de las estadísticas del esqueleto. Los tres spawners cumplieron su función en este capítulo (mostrar Factory Method con un solo producto) y se **borran al final del hito**: `EnemySpawner`, `CryptSpawner` y `CavernSpawner` no aparecen en el árbol final. Esto no significa

que Factory Method sobrara. Significa que, cuando hay una familia de productos, la fábrica abstracta es la forma natural de crecer: aquí empezamos con un creador y evolucionamos a una familia cuando aparece el segundo producto.

Abstract Factory se implementa con métodos fábrica

Cada método de `BiomeFactory` (`enemy`, `item`) es un método fábrica, y cada bioma decide qué clase concreta crea. En el catálogo, de hecho, las fábricas abstractas se suelen implementar así [9]. La diferencia está en la unidad: Factory Method se ocupa de *un producto* y de heredar el esqueleto de su creación; Abstract Factory, de *una familia* y de que se elija junta. Fíjate también en que Factory Method usa herencia (la subclase decide) y Abstract Factory usa composición: `GameSession` recibe una fábrica como objeto.

3.7 Si ya lo conoces: lo nuevo en Factory Method y Abstract Factory

Si ya usabas «Factory», casi seguro era la *fábrica simple*: una clase o un método estático que devuelve objetos según un parámetro. Es un idioma útil de Java más que un patrón del GoF; los libros de introducción la presentan como paso previo a los dos patrones [7], y [5] recomienda considerar fábricas estáticas en lugar de constructores. En Emberdeep sería esto:

`EnemyFactory.java (fábrica simple, no forma parte del proyecto)`

```
public final class EnemyFactory {
    private EnemyFactory() {}

    public static Enemy create(EnemyKind kind, Random random) {
        return switch (kind) {
            case SKELETON -> new Enemy("Esqueleto", kind,
                    new Stats(18, 7, 2), new RandomTargetAi(random));
            case ASH_RAT -> new Enemy("Rata de ceniza", kind,
                    new Stats(8, 5, 0), new LowestHpAi());
            case ZOMBIE -> new Enemy("Zombi", kind,
                    new Stats(22, 5, 1), new FirstAliveAi());
            case GOBLIN -> new Enemy("Goblin", kind,
                    new Stats(14, 6, 1), new FirstAliveAi());
            case BAT -> new Enemy("Murciélago", kind,
                    new Stats(10, 6, 0), new LowestHpAi());
            case SPIDER -> new Enemy("Araña", kind,
                    new Stats(16, 7, 2), new RandomTargetAi(random));
        };
    }
}
```

Un `switch` sobre un `enum` que devuelve un valor es exhaustivo cuando cubre todos los valores: el compilador lo exige [21]. Si añades un valor y olvidas su caso, el programa no compila, que es mejor que descubrirlo en ejecución. Lo mismo vale para una interfaz `sealed` con *pattern matching*, que cubre el ejercicio 3.4 [4]. Esta fábrica funciona y es suficiente en muchos proyectos. Comparada con los dos patrones:

	Fábrica simple	**Factory Method**	**Abstract Factory**
Qué es	Un método (estático) con un `switch`	Una clase abstracta con un método fábrica	Una interfaz con un método por producto
Quién decide la clase concreta	El `switch`, dentro de la fábrica	La subclase elegida	La fábrica concreta elegida
Productos	Uno (o varios sin relación)	Uno por creador	Una familia que va junta
Añadir una variante	Editar el `switch`	Escribir una subclase	Escribir una fábrica
Añadir un producto nuevo	Otro método o `switch`	Otro método fábrica en el creador	Cambiar la interfaz y todas las fábricas
Se apoya en	Un parámetro	Herencia	Composición
En Emberdeep	`EnemyFactory` (solo ejemplo)	`EnemySpawner` (se retira)	`BiomeFactory`

Lo nuevo aquí, si ya conocías la fábrica simple, es:

- **El creador con esqueleto.** En Factory Method el cliente no llama al método fábrica, llama a un método del creador que lo usa (`spawn`), y ahí viven las reglas comunes. La fábrica simple no tiene dónde ponerlas salvo repetirlas.
- **Variar sin editar.** Añadir un bioma es una clase nueva. En la fábrica simple es una rama más de un `switch` que ya mezcla seis enemigos de dos biomas.
- **La familia.** Abstract Factory no es «una fábrica con más métodos»: lo que aporta es que una sola elección determina todos los productos. Eso no lo da ninguna fábrica simple.
- **El criterio de elección.** Antes de añadir clases, pregúntate si la fábrica simple te basta: a menudo sí.

3.8 Factory Method y Abstract Factory: cuándo sí y cuándo no

Factory Method compensa cuando:

- El cliente no debe conocer la clase concreta que crea, y varias formas de crear comparten un esqueleto o unas reglas (en el ejemplo, «nunca un enemigo sin IA»).
- Hay una jerarquía que ya existe por otro motivo y cada subclase aporta su producto.
- Esperas más variantes con el tiempo y quieres añadirlas sin tocar al cliente.

No compensa cuando hay una sola clase concreta (un `new` es más claro), cuando lo único que cambia es un dato (basta pasar el dato), o cuando la variante cabe en una lambda: un `Supplier<Enemy>`, del paquete `java.util.function` [22], es una función sin parámetros que devuelve un objeto, y un mapa de `Supplier` sustituye muchas jerarquías de creadores. Lo pagas en una clase creadora más una subclase por variante.

Abstract Factory compensa cuando:

- Varios productos deben ser coherentes entre sí y el error de mezclarlos es real (la cripta y la caverna).
- El resto del sistema debe funcionar con cualquier familia sin saber cuál es.

No compensa con un solo producto (Factory Method o la fábrica simple bastan) ni con una única familia que nunca se va a repetir. Y tiene un coste que conviene decir claro: añadir una *familia* nueva es fácil, pero añadir un *tipo de producto* nuevo (por ejemplo, trampas) obliga a cambiar la interfaz y todas las fábricas concretas.

Qué pagas. Más clases e interfaces, una indirección al leer y, como siempre, la elección no desaparece: se mueve al composition root (`Main`), que sigue decidiendo qué fábrica monta. El catálogo web de Refactoring.Guru recoge esta misma distinción con diagramas [24].

3.9 En el proyecto: H3 con Factory Method y Abstract Factory

El hito H3 hace tres cosas: crea enemigos y objetos con las fábricas del bioma de cada planta, añade un inventario del grupo y retira los spawners. Estos son los cambios:

- **Nuevo** `EnemyKind` (en `engine/character/`); `Enemy` pasa a `Enemy(name, kind, stats, ai)`.
- **Nuevo** `engine/item/`: `Item(name, kind, power)`, `ItemKind` (`HEALING`, `BOMB`) e `Inventory`.
- **Nuevo** `engine/world/biome/`: `BiomeFactory`, `CryptBiome` y `CavernBiome`. Durante el hito pasan por aquí `EnemySpawner`, `CryptSpawner` y `CavernSpawner`, que se borran al final.
- `GameSession` recibe la fábrica del bioma y la lista de tipos de enemigo; guarda un `Inventory` y ofrece `inventory()` y `replaceInventory(Inventory)`.
- `ConsoleUi` muestra una línea «Objetos» en cada turno y ya no pone «planta 1» en la cabecera, porque ahora hay dos plantas. `Main` elige la fábrica según un argumento: sin argumentos, planta 1 (cripta); con 2, planta 2 (caverna).

3.9.1 El inventario: un record inmutable

El grupo empieza con dos pociones y una bomba. Un `Inventory` es un record con la lista de objetos. Será **inmutable**: `with(item)` y `without(item)` no cambian el inventario, devuelven uno *nuevo*. Quien lo usa lo sustituye por el que recibe:

engine/item/Inventory.java

```
package emberdeep.engine.item;

import java.util.ArrayList;
import java.util.List;

public record Inventory(List<Item> items) {

    public Inventory {
        items = List.copyOf(items);
    }

    public Inventory with(Item item) {
        List<Item> next = new ArrayList<>(items);
        next.add(item);
        return new Inventory(next);
    }
```

```
17
18      public Inventory without(Item item) {
19          List<Item> next = new ArrayList<>(items);
20          next.remove(item);
21          return new Inventory(next);
22      }
23  }
```

El constructor compacto (sin paréntesis de parámetros) se ejecuta antes de asignar los campos. `List.copyOf` hace una copia que no se puede modificar, así que ni el código de fuera ni una lista que se pasara por referencia pueden alterar el inventario. `without` quita la primera aparición del objeto (los records comparan por valor, así que dos pociones iguales son «el mismo» objeto para `remove`). `GameSession` guarda el inventario actual y lo *sustituye* al cambiar: en este hito aún nada gasta objetos, pero el método `replaceInventory` queda preparado para quien lo haga.

Hito 3: dos biomas, objetos y un inventario inmutable

Tras este hito, la planta 1 solo da enemigos y objetos de cripta; la planta 2, de caverna. El grupo empieza con dos pociones y una bomba de su bioma. Ningún archivo del motor, salvo las fábricas, importa `CryptBiome` ni `CavernBiome`. Los spawners ya no existen. Esta es la estructura:

Estructura de carpetas tras el capítulo 3

```
emberdeep/
|-- settings.gradle
|-- build.gradle
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
`-- src/main/java/emberdeep/
    |-- app/
    |   `-- Main.java              (composition root)
    |-- engine/
    |   |-- character/             Stats, Hero, Enemy, EnemyKind, Fighter
    |   |-- combat/
    |   |   |-- Combat.java
    |   |   |-- ai/                EnemyAi, FirstAliveAi, LowestHpAi,
    ↪ RandomTargetAi
    |   |   `-- damage/            DamageFormula, FlatFormula,
    ↪ ScaledFormula
    |   |-- item/                  Item, ItemKind, Inventory
    |   |-- session/               GameSession
    |   `-- world/
    |       `-- biome/             BiomeFactory, CryptBiome, CavernBiome
    `-- ui/
        `-- ConsoleUi.java
```

Lo practicas en el ejercicio 3.5. Antes tienes cuatro ejercicios más pequeños, en otros dominios.

3.10 Ejercicios

Ejercicio 3.1: De tres new a un Factory Method ★☆☆☆☆

Este código no es de Emberdeep: son cuatro archivos sueltos sin paquete, y tres clases repiten el mismo patrón, un `new` y el mismo mensaje.

`Potion.java, Chest.java, Vendor.java y Altar.java`

```java
public record Potion(String name, int heal) {}

public class Chest {
    public String open() {
        Potion potion = new Potion("Poción menor", 10);
        return "Obtienes " + potion.name() + " (+" + potion.heal() + ")";
    }
}

public class Vendor {
    public String open() {
        Potion potion = new Potion("Poción mayor", 25);
        return "Obtienes " + potion.name() + " (+" + potion.heal() + ")";
    }
}

public class Altar {
    public String open() {
        Potion potion = new Potion("Poción sagrada", 40);
        return "Obtienes " + potion.name() + " (+" + potion.heal() + ")";
    }
}
```

Aplica Factory Method con estos pasos (cada clase en su archivo):

1. Crea la clase abstracta `PotionSource` con un método público y `final String open()`, que llama a `create()` y construye el mensaje «Obtienes … (+N)» una sola vez, y con el método fábrica `protected abstract Potion create()`.
2. Haz que `Chest`, `Vendor` y `Altar` extiendan `PotionSource` y conserven solo `create()`, con su `new Potion(...)` de antes.
3. Escribe una clase `Main` con una `List<PotionSource>` de los tres e imprime el resultado de `open()` de cada uno.

Está bien si, tras compilar con `javac *.java`, `java Main` imprime

```
Obtienes Poción menor (+10)
Obtienes Poción mayor (+25)
Obtienes Poción sagrada (+40)
```

y si el texto «Obtienes» aparece en un único archivo.

Guiado ▪ solución en la pág. 76

Ejercicio 3.2: Lee un código donde la cripta suelta un hongo de caverna ★★☆☆☆

Lee este código (no es de Emberdeep; son ocho archivos sin paquete, listados juntos).

`Encounter.java y sus fuentes`

```
public interface EnemySource { String enemy(); }
public interface LootSource { String drop(); }

public class CryptEnemies implements EnemySource {
    public String enemy() { return "Esqueleto"; }
}
public class CavernEnemies implements EnemySource {
    public String enemy() { return "Goblin"; }
}
public class CryptLoot implements LootSource {
    public String drop() { return "Agua bendita"; }
}
public class CavernLoot implements LootSource {
    public String drop() { return "Hongo curativo"; }
}

public class Encounter {
    private final EnemySource enemies;
    private final LootSource loot;

    public Encounter(EnemySource enemies, LootSource loot) {
        this.enemies = enemies;
        this.loot = loot;
    }

    public String describe() {
        return enemies.enemy() + " suelta " + loot.drop();
    }
}

public class Main {
    public static void main(String[] args) {
        Encounter ok = new Encounter(new CryptEnemies(), new CryptLoot());
        Encounter bad = new Encounter(new CryptEnemies(), new CavernLoot());
        System.out.println(ok.describe());
        System.out.println(bad.describe());
    }
}
```

1. ¿Qué imprime el programa?
2. ¿Cuál de las dos líneas es incoherente con el juego y por qué compila sin queja?
3. ¿Qué patrón evita el fallo? Haz una tabla con el rol de cada clase o interfaz (fábrica abstracta, fábricas concretas, productos, cliente) tras aplicarlo.
4. Tras el cambio, ¿qué recibe el constructor de `Encounter`? ¿Puede construirse ahora un encuentro con el enemigo de la cripta y el botín de la caverna?

Está bien si identificas la causa (dos decisiones independientes que el tipo no relaciona), nombras Abstract Factory con sus roles y señalas que la garantía depende de que el cliente

reciba una sola fábrica.

Lectura de código ▪ solución en la pág. 77

Ejercicio 3.3: Añade el bioma Volcano sin tocar al cliente ★★☆☆☆

Partes de este programa (no es de Emberdeep; siete archivos sin paquete, listados juntos).

Un BiomeFactory sencillo

```
public record Monster(String name, int hp) {}
public record Loot(String name, int value) {}

public interface BiomeFactory {
    Monster monster();
    Loot loot();
}

public class CryptBiome implements BiomeFactory {
    public Monster monster() { return new Monster("Esqueleto", 18); }
    public Loot loot() { return new Loot("Agua bendita", 12); }
}

public class CavernBiome implements BiomeFactory {
    public Monster monster() { return new Monster("Goblin", 14); }
    public Loot loot() { return new Loot("Hongo curativo", 10); }
}

public class Encounter {
    private final BiomeFactory biome;

    public Encounter(BiomeFactory biome) {
        this.biome = biome;
    }

    public String describe() {
        Monster monster = biome.monster();
        Loot loot = biome.loot();
        return monster.name() + " (" + monster.hp() + " PV) guarda "
                + loot.name() + " (valor " + loot.value() + ")";
    }
}

public class Main {
    public static void main(String[] args) {
        String name = args.length > 0 ? args[0] : "crypt";
        BiomeFactory biome = switch (name) {
            case "cavern" -> new CavernBiome();
            default -> new CryptBiome();
        };
        System.out.println(new Encounter(biome).describe());
    }
}
```

Añade el bioma `Volcano`: un `VolcanoBiome` cuyo monstruo es un «Elemental de lava» con 30 PV y cuyo botín es «Ceniza ardiente» de valor 15, y haz que `java Main volcano` lo use.

No cambies `Encounter` ni `BiomeFactory`.
Está bien si `java Main volcano` imprime

```
Elemental de lava (30 PV) guarda Ceniza ardiente (valor 15)
```

y `java Main` y `java Main cavern` siguen imprimiendo lo de antes (esqueleto con agua bendita; goblin con hongo curativo), y solo has creado un archivo y editado otro.

Pista: *El único sitio que nombra clases concretas es el composition root.*

Práctica ▪ solución en la pág. 78

Ejercicio 3.4: Una fábrica simple que el compilador vigila ★★★☆☆

Partes de estos archivos (no son de Emberdeep; sin paquete). Un `Drop` describe qué suelta un enemigo, y `ItemFactory` lo convierte en un `Item`:

`Drop.java, Item.java e ItemFactory.java`

```java
public sealed interface Drop permits GoldDrop, PotionDrop, KeyDrop {}
public record GoldDrop(int amount) implements Drop {}
public record PotionDrop(String size) implements Drop {}
public record KeyDrop(int door) implements Drop {}

public record Item(String name, int value) {}

public class ItemFactory {
    public static Item create(Drop drop) {
        if (drop instanceof GoldDrop g) {
            return new Item("Oro", g.amount());
        } else if (drop instanceof PotionDrop p) {
            return new Item("Poción " + p.size(), 10);
        } else {
            return new Item("Llave", 1);
        }
    }
}
```

Una interfaz `sealed` limita qué clases pueden implementarla (las de su `permits`). Un `switch` con *patrones de tipo* (`case GoldDrop g ->`) sobre una interfaz sellada es exhaustivo si nombra todos los tipos permitidos [4].

1. Reescribe `create` con un `switch` que devuelva el `Item`, sin `default` y sin `instanceof`. La llave debe llamarse «Llave» seguida de la puerta (`"Llave 3"`) y valer 1.
2. Añade un cuarto tipo, `ScrollDrop(String spell)`, a `Drop` y compila *sin* tocar `create`: copia el primer mensaje de error que da el compilador.
3. Cubre el nuevo caso: un pergamino se llama «Pergamino de» seguido del hechizo y vale 5.
4. Escribe un `Main` que cree un `GoldDrop(25)`, un `PotionDrop("grande")`, un `KeyDrop(3)` y un `ScrollDrop("fuego")` e imprima el `Item` de cada uno.

Está bien si el paso 2 no compila, si el resultado final imprime

```
Item[name=Oro, value=25]
Item[name=Poción grande, value=10]
Item[name=Llave 3, value=1]
Item[name=Pergamino de fuego, value=5]
```

y si en `ItemFactory` no queda ningún `default` ni `instanceof`.

Pista: *Con un `switch` como expresión, cada rama es `case Tipo variable -> expresión;`.*

Práctica ▪ solución en la pág. 78

Ejercicio 3.5: Hito 3: biomas y objetos en Emberdeep ★★★☆☆

Implementa el hito H3 partiendo del estado del capítulo anterior (solución del ejercicio 2.4), en dos pasos.

1. **Factory Method.** Crea `EnemyKind` y pasa `Enemy` a `Enemy(name, kind, stats, ai)`. Crea `EnemySpawner`, `CryptSpawner` y `CavernSpawner` en `engine/world/biome/` con los seis enemigos del capítulo (cripta: Esqueleto con `RandomTargetAi`, Rata de ceniza con `LowestHpAi`, Zombi con `FirstAliveAi`; caverna: Goblin con `FirstAliveAi`, Murciélago con `LowestHpAi`, Araña con `RandomTargetAi`). Haz que `GameSession` reciba un `EnemySpawner` y una `List<EnemyKind>`. Comprueba que el juego arranca.
2. **Abstract Factory.** Crea `engine/item/` con `ItemKind`, `Item` e `Inventory` (inmutable, con `with`, `without` e `items`). Crea `BiomeFactory`, `CryptBiome` y `CavernBiome`. `GameSession` recibe ahora un `BiomeFactory`, crea los enemigos con él y empieza con dos pociones y una bomba del bioma en su `Inventory`; añade `inventory()` y `replaceInventory(Inventory)`. `ConsoleUi` muestra la línea «Objetos: . . . » antes del turno de cada héroe. `Main` usa `CavernBiome` si el primer argumento es `2` y `CryptBiome` en cualquier otro caso. **Borra** los tres spawners.

Las estadísticas son: Esqueleto (18, 7, 2), Rata de ceniza (8, 5, 0), Zombi (22, 5, 1), Goblin (14, 6, 1), Murciélago (10, 6, 0) y Araña (16, 7, 2). Objetos: cripta, «Poción de cripta» (`HEALING`, 12) y «Bomba de ceniza» (`BOMB`, 10); caverna, «Poción de caverna» (10) y «Bomba de roca» (14).
Está bien si:

1. `./gradlew run` muestra en cada turno de héroe una línea de objetos como esta:

   Línea de objetos en la cripta

   ```
   Objetos:  Poción de cripta, Poción de cripta, Bomba de ceniza
   ```

   Eligiendo `1` (plana) y tecleando siempre `1`, la primera ronda termina con estos mensajes y la partida acaba en la ronda 4 con «¡Victoria!»:

   Ronda 1 en la cripta, fórmula plana

   ```
   Aria ataca a Esqueleto y le hace 6 de daño.
   Borin ataca a Esqueleto y le hace 4 de daño.
   Cael ataca a Esqueleto y le hace 7 de daño.
   Esqueleto ataca a Cael y le hace 5 de daño.
   Rata de ceniza ataca a Cael y le hace 3 de daño.
   Zombi ataca a Aria y le hace 2 de daño.
   ```

2. Para la caverna se arranca con el argumento 2:

   Planta 2

   ```
   ./gradlew run --args="2"
   ```

   La línea de objetos es `Objetos:` seguida de «Poción de caverna, Poción de caverna, Bomba de roca». Con la misma elección, la ronda 1 acaba con «Murciélago ataca a Cael y le hace 4 de daño.» y «Araña ataca a Cael y le hace 5 de daño.», y la partida termina en la ronda 3 con «¡Victoria!».

3. Ningún archivo de `src` contiene la palabra «Spawner», y ningún archivo fuera de `engine/world/biome/` y `app/Main.java` menciona `CryptBiome` ni `CavernBiome`.

4. El inventario es inmutable: si añades temporalmente a `Main` estas líneas, la salida es «1 0». Bórralas después.

   Comprobación temporal en Main.java

   ```
   Inventory a = new Inventory(List.of(new Item("X", ItemKind.BOMB, 1)));
   Inventory b = a.without(a.items().get(0));
   System.out.println(a.items().size() + " " + b.items().size());
   ```

Pista: *El orden de los enemigos en la lista de* `kinds` *es el orden en que atacan: en la cripta,* `SKELETON`*,* `ASH_RAT`*,* `ZOMBIE`*.*

Proyecto ▪ solución en la pág. 79

3.11 Resumen

Resumen

- El olor de las fábricas es un `new` de clases concretas repartido por el código, y objetos que deben ir juntos elegidos con decisiones independientes.
- Factory Method: un creador abstracto fija el esqueleto (`spawn`) y cada subclase implementa el método fábrica (`create`). Añadir una variante es añadir una subclase.
- Abstract Factory: una interfaz con un método por producto (`BiomeFactory`) y una fábrica concreta por familia. Una sola elección determina enemigos y objetos.
- La fábrica simple con `enum` y `switch` exhaustivo ya funciona y el compilador avisa si falta un caso. Los patrones aportan reglas comunes y familias, no un `switch` mejor.
- Añadir una familia es fácil; añadir un producto a la familia toca la interfaz y todas las fábricas. La garantía vale mientras el cliente use solo la fábrica.
- H3: `Enemy(name, kind, stats, ai)`, `Item`, `ItemKind`, `Inventory` inmutable y dos biomas. Los spawners sirvieron de ejemplo y se borran.

Soluciones de los ejercicios

Solución del ejercicio 3.1

enunciado en la pág. 70

Los cinco archivos finales:

PotionSource.java

```java
public abstract class PotionSource {

    public final String open() {
        Potion potion = create();
        return "Obtienes " + potion.name() + " (+" + potion.heal() + ")";
    }

    protected abstract Potion create();
}
```

Chest.java, Vendor.java y Altar.java (tres archivos)

```java
public class Chest extends PotionSource {
    @Override
    protected Potion create() {
        return new Potion("Poción menor", 10);
    }
}

public class Vendor extends PotionSource {
    @Override
    protected Potion create() {
        return new Potion("Poción mayor", 25);
    }
}

public class Altar extends PotionSource {
    @Override
    protected Potion create() {
        return new Potion("Poción sagrada", 40);
    }
}
```

Main.java (Potion.java no cambia)

```java
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<PotionSource> sources = List.of(new Chest(), new Vendor(), new
    ↪ Altar());
        for (PotionSource source : sources) {
            System.out.println(source.open());
        }
    }
}
```

Por qué funciona. `open()` es el método del creador que el cliente llama; dentro usa `create()`, el método fábrica, que cada subclase implementa. El mensaje se construye una vez en `PotionSource`, y el `new Potion` vive solo en el método fábrica de cada subclase. `Main` trabaja con `PotionSource` y no conoce `Potion` en concreto.
Errores típicos:

- Dejar `open()` abstracto o sin `final` y reescribirlo en cada subclase: el mensaje vuelve a repetirse.
- Hacer `create()` `private`: las subclases no pueden sobrescribirlo (con `protected` o paquete sí).
- Olvidar `import java.util.List;` en `Main`.

Solución del ejercicio 3.2 enunciado en la pág. 71

1. Imprime

```
Esqueleto suelta Agua bendita
Esqueleto suelta Hongo curativo
```

2. La segunda línea es incoherente: un esqueleto de cripta no suelta un hongo de caverna. Compila porque `Encounter` pide un `EnemySource` y un `LootSource`, y `CryptEnemies` y `CavernLoot` lo son. Son dos decisiones independientes y nada en los tipos las relaciona.
3. Es Abstract Factory.

Rol	Clase	Nota
Fábrica abstracta (interfaz)	Nueva, por ejemplo `BiomeFactory` con `enemy()` y `drop()`	Sustituye a `EnemySource` y `LootSource`
Fábricas concretas	`CryptBiome`, `CavernBiome`	Cada una devuelve su enemigo y su botín
Productos	El enemigo y el botín (aquí, textos)	Siempre del mismo bioma
Cliente	`Encounter`	Recibe una sola fábrica

4. `Encounter(BiomeFactory biome)` y `describe()` devuelve `biome.enemy() + " suelta " + biome.drop()`. Ya no se puede construir ese encuentro mezclado: hay un único argumento y todo sale de él.

Matiz. La garantía depende de que el cliente use la fábrica como única fuente. Si `Encounter` siguiera pidiendo un `LootSource` aparte, el problema volvería.
Errores típicos:

- Responder «Factory Method»: crea un solo producto y no relaciona dos productos.
- Mantener las dos interfaces antiguas y solo añadir una tercera: sigue habiendo dos puertas para mezclar.

Solución del ejercicio 3.3

enunciado en la pág. 72

Un archivo nuevo y una línea en `Main`:

`VolcanoBiome.java`

```
public class VolcanoBiome implements BiomeFactory {
    public Monster monster() { return new Monster("Elemental de lava", 30);
    ↪ }
    public Loot loot() { return new Loot("Ceniza ardiente", 15); }
}
```

`Main.java (única edición: el caso volcano)`

```
public class Main {
    public static void main(String[] args) {
        String name = args.length > 0 ? args[0] : "crypt";
        BiomeFactory biome = switch (name) {
            case "cavern" -> new CavernBiome();
            case "volcano" -> new VolcanoBiome();
            default -> new CryptBiome();
        };
        System.out.println(new Encounter(biome).describe());
    }
}
```

Por qué funciona. `Encounter` solo conoce la interfaz, así que no se entera del bioma nuevo. Quien nombra clases concretas es el composition root, y es el único que cambia. Con `java Main volcano` se imprime `Elemental de lava (30 PV) guarda Ceniza ardiente (valor 15)`.

Errores típicos:

- Meter un `if (biome instanceof VolcanoBiome)` o un `switch` en `Encounter`: el cliente vuelve a conocer las clases concretas.
- Olvidar `implements BiomeFactory`: el `case` no compilaría, porque el `switch` devuelve un `BiomeFactory`.
- Tocar la interfaz para añadir un método: obligaría a cambiar las otras dos fábricas.

Solución del ejercicio 3.4

enunciado en la pág. 73

Estado final (el paso 2 se hace con `ScrollDrop` añadido y `create` sin tocar):

`Drop.java (cambia el permits; GoldDrop, PotionDrop y KeyDrop no cambian)`

```
public sealed interface Drop permits GoldDrop, PotionDrop, KeyDrop,
    ↪ ScrollDrop {}
```

`ScrollDrop.java (nuevo)`

```
public record ScrollDrop(String spell) implements Drop {}
```

ItemFactory.java

```java
public class ItemFactory {
    public static Item create(Drop drop) {
        return switch (drop) {
            case GoldDrop g -> new Item("Oro", g.amount());
            case PotionDrop p -> new Item("Poción " + p.size(), 10);
            case KeyDrop k -> new Item("Llave " + k.door(), 1);
            case ScrollDrop s -> new Item("Pergamino de " + s.spell(), 5);
        };
    }
}
```

Main.java

```java
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Drop> drops = List.of(
                new GoldDrop(25),
                new PotionDrop("grande"),
                new KeyDrop(3),
                new ScrollDrop("fuego"));
        for (Drop drop : drops) {
            System.out.println(ItemFactory.create(drop));
        }
    }
}
```

Por qué funciona. El `switch` es una expresión sobre una interfaz sellada y cubre sus cuatro tipos permitidos, así que es exhaustivo y no necesita `default`. En el paso 2, con `ScrollDrop` ya permitido pero sin su `case`, `javac` se queja en el `switch` con el mensaje «the switch expression does not cover all possible input values». El compilador te obliga a decidir qué hace la fábrica con cada tipo nuevo. La versión con `if/else` habría tratado cualquier cosa nueva como una llave.

Errores típicos:

- Añadir un `default -> new Item("Llave", 1)` «para que compile»: se pierde la protección, que era el objetivo.
- Olvidar la palabra `ScrollDrop` en el `permits`: sin eso, la clase no puede implementar `Drop`.

Solución del ejercicio 3.5

enunciado en la pág. 74

Parte del estado del capítulo 2. `Stats`, `Hero`, `Fighter`, `Combat`, el paquete `ai` y el paquete `damage` no cambian. Esta solución muestra el estado *final*; el paso 1 son los tres archivos del texto del capítulo (`EnemySpawner`, `CryptSpawner`, `CavernSpawner`) y el constructor de `GameSession` con `EnemySpawner`, que se sustituyen en el paso 2.

engine/character/

EnemyKind.java

```
package emberdeep.engine.character;

public enum EnemyKind {
    SKELETON, ASH_RAT, ZOMBIE,    // cripta
    GOBLIN, BAT, SPIDER           // caverna
}
```

Enemy.java

```
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;

public record Enemy(String name, EnemyKind kind, Stats stats, EnemyAi ai) {}
```

engine/item/

ItemKind.java

```
package emberdeep.engine.item;

public enum ItemKind { HEALING, BOMB }
```

Item.java

```
package emberdeep.engine.item;

public record Item(String name, ItemKind kind, int power) {}
```

Inventory.java

```java
package emberdeep.engine.item;

import java.util.ArrayList;
import java.util.List;

public record Inventory(List<Item> items) {

    public Inventory {
        items = List.copyOf(items);
    }

    public Inventory with(Item item) {
        List<Item> next = new ArrayList<>(items);
        next.add(item);
        return new Inventory(next);
    }

    public Inventory without(Item item) {
        List<Item> next = new ArrayList<>(items);
        next.remove(item);
        return new Inventory(next);
    }
}
```

engine/world/biome/

BiomeFactory.java

```java
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;

public interface BiomeFactory {
    Enemy enemy(EnemyKind kind);

    Item item(ItemKind kind);
}
```

CryptBiome.java

```java
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import java.util.Random;

public class CryptBiome implements BiomeFactory {
    private final Random random;

    public CryptBiome(Random random) {
        this.random = random;
    }

    @Override
    public Enemy enemy(EnemyKind kind) {
        return switch (kind) {
            case SKELETON -> new Enemy("Esqueleto", kind,
                    new Stats(18, 7, 2), new RandomTargetAi(random));
            case ASH_RAT -> new Enemy("Rata de ceniza", kind,
                    new Stats(8, 5, 0), new LowestHpAi());
            case ZOMBIE -> new Enemy("Zombi", kind,
                    new Stats(22, 5, 1), new FirstAliveAi());
            case GOBLIN, BAT, SPIDER -> throw new IllegalArgumentException(
                    kind + " no vive en la cripta");
        };
    }

    @Override
    public Item item(ItemKind kind) {
        return switch (kind) {
            case HEALING -> new Item("Poción de cripta", kind, 12);
            case BOMB -> new Item("Bomba de ceniza", kind, 10);
        };
    }
}
```

CavernBiome.java

```java
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import java.util.Random;

public class CavernBiome implements BiomeFactory {
    private final Random random;

    public CavernBiome(Random random) {
        this.random = random;
    }

    @Override
    public Enemy enemy(EnemyKind kind) {
        return switch (kind) {
            case GOBLIN -> new Enemy("Goblin", kind,
                    new Stats(14, 6, 1), new FirstAliveAi());
            case BAT -> new Enemy("Murciélago", kind,
                    new Stats(10, 6, 0), new LowestHpAi());
            case SPIDER -> new Enemy("Araña", kind,
                    new Stats(16, 7, 2), new RandomTargetAi(random));
            case SKELETON, ASH_RAT, ZOMBIE -> throw new
    ↪ IllegalArgumentException(
                    kind + " no vive en la caverna");
        };
    }

    @Override
    public Item item(ItemKind kind) {
        return switch (kind) {
            case HEALING -> new Item("Poción de caverna", kind, 10);
            case BOMB -> new Item("Bomba de roca", kind, 14);
        };
    }
}
```

engine/session/GameSession.java

GameSession.java

```java
package emberdeep.engine.session;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.List;

public class GameSession {
    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final Combat combat;
    private Inventory inventory;

    public GameSession(List<Hero> heroes, BiomeFactory biome,
                       List<EnemyKind> kinds, DamageFormula formula) {
        this.heroes = heroes.stream().map(Fighter::ofHero).toList();
        this.enemies = kinds.stream()
                .map(biome::enemy)
                .map(Fighter::ofEnemy)
                .toList();
        this.combat = new Combat(this.heroes, this.enemies, formula);
        this.inventory = new Inventory(List.of())
                .with(biome.item(ItemKind.HEALING))
                .with(biome.item(ItemKind.HEALING))
                .with(biome.item(ItemKind.BOMB));
    }

    public List<Fighter> heroes() { return heroes; }
    public List<Fighter> enemies() { return enemies; }
    public Combat combat() { return combat; }
    public int round() { return combat.round(); }
    public Inventory inventory() { return inventory; }

    public void replaceInventory(Inventory next) {
        this.inventory = next;
    }
}
```

ui/ConsoleUi.java (en tres tramos seguidos, dentro de la misma clase)

ConsoleUi.java (1/3: imports y fórmula)

```java
package emberdeep.ui;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.Item;
import emberdeep.engine.session.GameSession;
import java.io.PrintStream;
import java.util.List;
import java.util.Scanner;
import java.util.stream.Collectors;

public class ConsoleUi {
    private final Scanner in;
    private final PrintStream out;

    public ConsoleUi(Scanner in, PrintStream out) {
        this.in = in;
        this.out = out;
    }

    public int askFormula() {
        out.println("Fórmula de daño:");
        out.println("  [1] plana: ataque - defensa");
        out.println("  [2] escalada: ataque x 100 / (100 + defensa)");
        return ask(2);
    }
```

ConsoleUi.java (2/3: la partida)

```
    public void run(GameSession session) {
        Combat combat = session.combat();
        out.println("=== Emberdeep ===");
        while (!combat.isOver()) {
            out.println("--- Ronda " + session.round() + " ---");
            for (Fighter hero : session.heroes()) {
                if (!hero.isAlive() || combat.isOver()) {
                    continue;
                }
                out.println("Héroes:   " + describe(session.heroes()));
                out.println("Enemigos: " + describe(session.enemies()));
                out.println("Objetos:  " + describeItems(session.inventory()
    ↪ ));
                out.println("Turno de " + hero.name() + ". [1] atacar");
                ask(1);
                Fighter target = askTarget(session.enemies());
                print(combat.heroAttack(hero, target));
            }
            if (!combat.isOver()) {
                print(combat.enemyPhase());
            }
        }
        out.println(combat.heroesWon() ? "¡Victoria!" : "Derrota...");
    }
```

ConsoleUi.java (3/3: auxiliares y llave final)

```
    private String describe(List<Fighter> group) {
        return group.stream()
                .map(f -> f.name() + " " + f.hp() + "/" + f.stats().maxHp())
                .collect(Collectors.joining(", "));
    }

    private String describeItems(Inventory inventory) {
        if (inventory.items().isEmpty()) {
            return "(ninguno)";
        }
        return inventory.items().stream()
                .map(Item::name)
                .collect(Collectors.joining(", "));
    }

    private void print(List<String> messages) {
        messages.forEach(out::println);
    }

    private int ask(int max) {
        while (true) {
            out.print("> ");
            out.flush();
            String line = in.nextLine().trim();
            try {
                int n = Integer.parseInt(line);
                if (n >= 1 && n <= max) {
                    return n;
                }
            } catch (NumberFormatException e) {
                // no es un número: se avisa más abajo
            }
            out.println("Escribe un número entre 1 y " + max + ".");
        }
    }

    private Fighter askTarget(List<Fighter> enemies) {
        List<Fighter> alive = enemies.stream().filter(Fighter::isAlive).toList();
        StringBuilder menu = new StringBuilder("Objetivo:");
        for (int i = 0; i < alive.size(); i++) {
            menu.append("  [").append(i + 1).append("] ").append(alive.get(i).name());
        }
        out.println(menu);
        return alive.get(ask(alive.size()) - 1);
    }
}
```

app/Main.java

Main.java

```java
package emberdeep.app;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.biome.BiomeFactory;
import emberdeep.engine.world.biome.CavernBiome;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.ui.ConsoleUi;
import java.util.List;
import java.util.Random;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        ConsoleUi ui = new ConsoleUi(new Scanner(System.in), System.out);
        DamageFormula formula = ui.askFormula() == 1
                ? new FlatFormula()
                : new ScaledFormula();
        Random random = new Random(42);

        boolean cavern = args.length > 0 && args[0].equals("2");
        BiomeFactory biome = cavern
                ? new CavernBiome(random)
                : new CryptBiome(random);
        List<EnemyKind> kinds = cavern
                ? List.of(EnemyKind.GOBLIN, EnemyKind.BAT, EnemyKind.SPIDER)
                : List.of(EnemyKind.SKELETON, EnemyKind.ASH_RAT, EnemyKind.ZOMBIE);

        List<Hero> heroes = List.of(
                new Hero("Aria", new Stats(30, 8, 3)),
                new Hero("Borin", new Stats(40, 6, 5)),
                new Hero("Cael", new Stats(24, 9, 2)));
        GameSession session = new GameSession(heroes, biome, kinds, formula);
        ui.run(session);
    }
}
```

Por qué funciona. Cada bioma es una fábrica completa: `GameSession` pide enemigos y objetos a la misma `BiomeFactory`, así que la cripta no puede repartir objetos de caverna. Solo `Main` y el paquete `biome` nombran las clases concretas. Los spawners fueron el escalón intermedio y desaparecen; `grep -r Spawner src/` no encuentra nada. `Inventory` copia la lista en el constructor compacto, y `with` y `without` crean otro, así que el original no cambia. En la cripta con la fórmula plana y siempre `1`: en la ronda 1 el esqueleto, con el primer `nextInt(3)` de la semilla 42 (que es 2), elige a Cael; la rata, el de menos vida, también ataca a Cael; el zombi, al primero vivo, Aria. El esqueleto cae en la ronda 2, la rata también,

y el zombi en la ronda 4. En la caverna, el goblin cae en la ronda 1 y la partida termina en la ronda 3.

Errores típicos:

- Dejar los spawners «por si acaso»: duplican estadísticas y el criterio de «hecho» pide su ausencia.
- Crear un `new Random(42)` por bioma: cambia las trazas. Solo se crea uno, en `Main`.
- Mutar la lista de `Inventory` con `items().add(...)`: lanza `UnsupportedOperationException` porque la copia es inmutable. Para añadir, usa `with` y sustituye el inventario.
- Cambiar el orden de la lista de tipos: cambia quién ataca primero y la traza.
- Ejecutar la caverna sin el argumento: sin `2`, `Main` monta la cripta.

Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo siguiente.

CAPÍTULO 4

Builder y Singleton: héroes y mazmorra paso a paso

Objetivos del capítulo

- Reconocer el olor que pide un Builder: un constructor con muchos parámetros o un objeto que solo es válido si se monta entero.
- Aplicar Builder: pasos que devuelven `this` y una validación única en `build()`, incluido un builder anidado.
- Decidir cuándo basta un `record` con constructor compacto y cuándo hace falta un Builder.
- Aplicar Singleton (holder y `enum`), medir su coste y conocer la alternativa: crear la instancia una vez en `Main`.
- Dar el hito H4: `HeroBuilder`, mazmorra de tres plantas con `DungeonBuilder` y `GameConfig`.

Conoces Builder y Singleton, así que este capítulo es de repaso con otro enfoque: aquí medimos cuánto cuesta cada uno y en qué momento dejan de compensar. Emberdeep ya tiene héroes y dos plantas, pero los héroes se montan con tres números sueltos y las plantas con listas escritas a mano en `Main`. Además, pronto habrá datos (la semilla del azar, el número de plantas) que muchas clases querrán leer, y la tentación de hacerlos globales es fuerte. Primero, Builder.

4.1 El olor que pide Builder: un constructor con ocho parámetros

Hoy un héroe es `new Hero("Aria", new Stats(30, 8, 3))`: dos parámetros y tres números dentro de `Stats`. Ya cuesta recordar el orden de los números. Imagina el héroe que verás en capítulos posteriores, con equipo y habilidades, si siguiera el mismo camino:

Hipotético, no forma parte del proyecto

```
Hero hero = new Hero("Aria", 30, 8, 3, 5, "espada", "cota", true);
```

¿Qué es el `5`? ¿Y el `true`? El compilador no protege nada: dos `int` intercambiados compilan igual. Este es el olor: **un constructor con muchos parámetros, algunos opcionales, que se llama con valores sin nombre**. Hay un segundo síntoma, más importante: el objeto solo tiene sentido si cumple reglas (nombre no vacío, vida positiva) y esas reglas se comprueban, o no, en cada sitio que lo crea.

Builder separa el *montaje* del objeto del objeto final. Su intención en el catálogo es separar la construcción de un objeto complejo de su representación, de modo que el mismo proceso pueda crear representaciones distintas [9]. En Java, la forma habitual es un objeto auxiliar con un método por cada dato, que se encadenan, y un `build()` que valida todo de golpe [5].

4.2 Piezas del patrón Builder

Rol	Clase en el ejemplo	Propósito	Paquete
Producto (records)	`Hero`; `Dungeon` y `Floor`	Lo que se monta	`engine.character`; `engine.world`
Builder (clase)	`HeroBuilder`	Guarda los datos y valida	`engine.character`
Builder anidado (clase interna)	`DungeonBuilder.FloorBuilder`	Monta una planta dentro del builder de la mazmorra	`engine.world`
Pasos (métodos)	`name`, `stats`, `floor`, `biome`, `enemies`	Dan un dato y devuelven el builder	`engine.character`; `engine.world`
Validación (método)	`build()`	Comprueba y crea el producto	`engine.character`; `engine.world`
Cliente (clase)	`Main`	Encadena los pasos	`app`

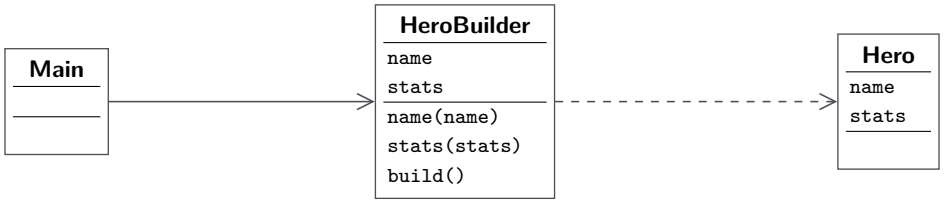


Diagrama UML del patrón Builder: `Main` da los pasos a `HeroBuilder`; `build()` valida y crea el `Hero`.

El «director» del GoF no se usa

El catálogo original incluye un *director*, una clase que sabe en qué orden dar los pasos. En Java casi nunca existe: el propio cliente encadena los pasos, como hará `Main`. Por eso la tabla no lo lista.

4.3 El ejemplo de Builder, pieza a pieza: `HeroBuilder`

El producto no cambia: sigue siendo `Hero(name, stats)` del capítulo 1. El builder guarda los datos en campos y cada paso devuelve `this`, que es lo que permite encadenar las llamadas:

engine/character/HeroBuilder.java

```
1  package emberdeep.engine.character;
2
3  public class HeroBuilder {
4      private String name;
5      private Stats stats;
6
7      public HeroBuilder name(String name) {
8          this.name = name;
9          return this;
10     }
11
12     public HeroBuilder stats(Stats stats) {
13         this.stats = stats;
```

```
        return this;
    }

    public Hero build() {
        if (name == null || name.isBlank()) {
            throw new IllegalStateException("El héroe necesita un nombre");
        }
        if (stats == null) {
            throw new IllegalStateException("El héroe necesita estadísticas");
        }
        if (stats.maxHp() <= 0 || stats.attack() <= 0 || stats.defense() < 0) {
            throw new IllegalStateException("Estadísticas no válidas: " + stats
    ↪ );
        }
        return new Hero(name, stats);
    }
}
```

Tres decisiones a notar. Los pasos **no validan**: aceptan cualquier cosa, porque algunas reglas dependen de varios datos y solo se pueden comprobar cuando están todos. Toda la validación vive en `build()`, un único sitio. La excepción es `IllegalStateException`: el builder está en un estado en el que no puede construir. Y la defensa admite 0 (hay héroes sin armadura); la vida y el ataque deben ser positivos. Se usa así:

Uso (fragmento)

```
Hero aria = new HeroBuilder()
        .name("Aria")
        .stats(new Stats(30, 8, 3))
        .build();
```

Un builder se usa una vez

El builder guarda estado mutable. Si lo reutilizas para otro héroe sin volver a dar todos los pasos, el segundo héroe hereda datos del primero. Crea uno nuevo (`new HeroBuilder()`) por cada objeto.

4.4 Si ya lo conoces: Builder y `record`

Un `record` ya trae mucho de lo que se pedía a un Builder: es inmutable y su *constructor compacto* valida antes de asignar los campos [21]:

Hero con constructor compacto (alternativa, no es el proyecto)

```
public record Hero(String name, Stats stats) {
    public Hero {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("El héroe necesita un nombre");
        }
    }
}
```

Para dos campos es suficiente, y es lo que deberías usar si ese fuera todo. Aquí el Builder se

justifica por lo que viene: el héroe ganará campos opcionales (equipo, habilidades) y entonces la llamada del principio del capítulo dejará de ser legible. Qué elegir:

	Constructor compacto de un `record`	**Builder**
Parámetros	Pocos, casi todos obligatorios	Muchos, varios opcionales con valor por defecto
Validación	Reglas sobre los campos del propio objeto	Reglas entre datos que llegan por pasos, o que dependen de ellos
Legibilidad de la llamada	Por posición	Cada dato lleva el nombre de su paso
Montaje en pasos o anidado	No	Sí (el builder de la mazmorra)
Coste	Ninguno	Una clase más y estado mutable

Hay una tercera vía para cambiar *un* dato de un objeto ya creado: un método `with` que devuelve una copia modificada, como el `with` de `Inventory` del capítulo anterior. Lo nuevo aquí, si ya conocías Builder, es:

- **Cuándo no usarlo.** Con pocos parámetros un `record` basta (ejercicio 4.3).
- **Validar en un solo sitio**, en `build()`, y no en cada paso.
- **Builders anidados**, que montan un árbol de objetos (siguiente sección).
- **La refactorización hacia el Builder**: llegar a él desde código que ya repite el montaje, como recoge [17] con *Encapsulate Composite with Builder*.

4.5 Builder anidado: `DungeonBuilder` y `FloorBuilder`

Emberdeep tendrá una mazmorra de tres plantas: cripta, caverna y de nuevo cripta (un bioma puede repetirse). Cada planta es un bioma y una lista de tipos de enemigo; más adelante tendrá también un mapa. Dos records modelan el resultado:

`engine/world/Floor.java y Dungeon.java (dos archivos)`

```
package emberdeep.engine.world;

public record Floor(BiomeFactory biome, List<EnemyKind> enemies) {
    public Floor {
        enemies = List.copyOf(enemies);
    }
}

public record Dungeon(List<Floor> floors) {
    public Dungeon {
        floors = List.copyOf(floors);
    }
}
```

(Sin imports para abreviar; la solución del hito trae los archivos completos.) Montar una mazmorra a mano exige construir cada `Floor` y luego la lista, y nada impide una mazmorra

vacía o una planta sin enemigos. Un builder *anidado* lo resuelve: el builder de la mazmorra ofrece `floor()`, que devuelve un builder de planta; al terminar la planta, `done()` te devuelve al builder de la mazmorra:

Uso (fragmento de Main)

```
Dungeon dungeon = new DungeonBuilder()
        .floor().biome(crypt).enemies(cryptKinds).done()
        .floor().biome(cavern).enemies(cavernKinds).done()
        .floor().biome(crypt).enemies(cryptKinds).done()
        .build();
```

`FloorBuilder` es una clase *interna* (no `static`) de `DungeonBuilder`: puede llamar a la lista `floors` de su builder padre. Su `done()` añade la planta y devuelve el padre con `DungeonBuilder.this`. La parte clave, en `build()`:

DungeonBuilder.java (fragmento: la validación)

```
public Dungeon build() {
    if (floors.isEmpty()) {
        throw new IllegalStateException("La mazmorra necesita al menos una
    ↪ planta");
    }
    for (int i = 0; i < floors.size(); i++) {
        if (floors.get(i).enemies().isEmpty()) {
            throw new IllegalStateException("La planta " + (i + 1) + " no tiene
    ↪  enemigos");
        }
    }
    return new Dungeon(floors);
}
```

El mapa de cada planta llega en el capítulo 5 como un paso más de `FloorBuilder`; no hace falta tocar nada de lo anterior. Los archivos completos están en la solución del ejercicio de proyecto.

4.6 El olor que pide Singleton: la configuración que todos leen

`Main` ya reparte una semilla de azar y una fórmula de daño. Pronto querrás más datos comunes: el número de plantas, la semilla. ¿Cómo llegan a quien los necesita? La salida fácil es un acceso global. Así lo harían tres clases del juego:

Hipotético: tres clases leen la misma configuración

```
// en ConsoleUi:    if (floor > GlobalSettings.getInstance().floors()) { ... }
// en GameSession:  new Random(GlobalSettings.getInstance().seed())
// en Combat:       GlobalSettings.getInstance().seed()
```

Funciona, y el código es corto. Pero mira lo que ha pasado: `ConsoleUi`, `GameSession` y `Combat` dependen de `GlobalSettings` y **nada en sus constructores lo dice**. Para saber qué necesita una clase tienes que leer su cuerpo entero. Ese es el olor de fondo: estado global accesible desde cualquier sitio. Nystrom, en el capítulo que dedica a Singleton en su libro sobre patrones para juegos, lo resume con un título: por qué nos arrepentimos de usarlo, empezando por que es una variable global con otra cara [19].

El patrón, sin embargo, responde a una necesidad real: garantizar que de una clase hay una sola instancia y dar acceso a ella [9]. Lo veremos primero y luego juzgaremos si compensa.

4.7 Piezas del patrón Singleton

Rol	Clase en el ejemplo	Propósito	Paquete
Singleton (clase final)	`GlobalSettings`	Tiene una sola instancia, constructor privado	`app` (ejemplo)
Variante (enum)	`SettingsEnum` con `INSTANCE`	La misma garantía sin código extra	`app` (ejemplo)
Acceso (método estático)	`getInstance()`	Punto único de acceso	`app` (ejemplo)
Alternativa en Emberdeep (record)	`GameConfig`	Creada en `Main` y pasada por constructor	`app`
Clientes (clases)	Quienes leen la configuración	Piden el dato	varios

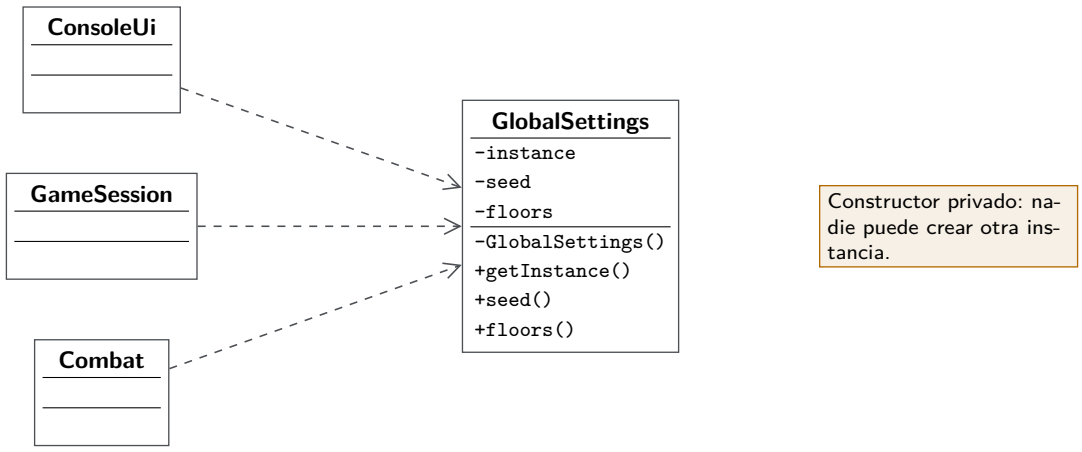


Diagrama UML del patrón Singleton: los clientes dependen de una instancia global que ninguno recibe por constructor.

4.8 El ejemplo de Singleton, pieza a pieza: holder y `enum`

4.8.1 Primer intento: perezoso, y con un fallo

La versión que primero se le ocurre a casi todo el mundo crea la instancia la primera vez que se pide («inicialización perezosa»):

GlobalSettings.java (versión ingenua, con fallo)

```
1 public final class GlobalSettings {
2     private static GlobalSettings instance;
3
4     private GlobalSettings() {}
5
```

```
    public static GlobalSettings getInstance() {
        if (instance == null) {
            instance = new GlobalSettings();
        }
        return instance;
    }
}
```

Con un solo hilo funciona. Con dos tiene un fallo, que explica el siguiente recuadro.

Dos hilos y una carrera de datos

Un *hilo* es una secuencia de instrucciones que Java ejecuta a la vez que otras. Un programa normal tiene uno solo, el de `main`. Para crear otro se pasa lo que debe hacer a un `Thread`; `start()` lo arranca y `join()` espera a que termine (puede lanzar `InterruptedException`, por eso `main` declara `throws`):

Dos hilos (fragmento)

```
Thread t = new Thread(() -> System.out.println("desde otro hilo"));
t.start();   // el hilo empieza a ejecutarse en paralelo
t.join();    // esperamos a que termine antes de seguir
```

Una *carrera de datos* ocurre cuando dos hilos leen y escriben lo mismo sin coordinarse, y el resultado depende de quién llega antes. En `getInstance` el hilo A comprueba `instance == null` y es cierto. Antes de que asigne, el hilo B hace la misma comprobación, también cierta. Ambos ejecutan `new` y hay **dos instancias**. No ocurre siempre: depende de la sincronización de los hilos, y por eso es difícil de reproducir. Se arregla haciendo que la creación no sea una comprobación en dos pasos.

4.8.2 Solución 1: el holder

Java inicializa una clase una sola vez y de forma segura entre hilos, la primera vez que se usa [5]. El truco, llamado *holder*, es poner la instancia en una clase interna que no se carga hasta que alguien la necesita:

app/GlobalSettings.java (ejemplo, no está en el proyecto)

```
package emberdeep.app;

public final class GlobalSettings {
    private final long seed = 42L;
    private final int floors = 3;

    private GlobalSettings() {}

    private static final class Holder {
        private static final GlobalSettings INSTANCE = new GlobalSettings();
    }

    public static GlobalSettings getInstance() {
        return Holder.INSTANCE;
    }
```

```
16
17      public long seed() { return seed; }
18      public int floors() { return floors; }
19  }
```

`Holder` no se inicializa hasta la primera llamada a `getInstance()`, así que es perezoso, y no hay ninguna comprobación que pueda cruzarse. No usa `synchronized`.

4.8.3 Solución 2: el `enum`

Un `enum` con un solo valor da la misma garantía con menos código, y además resiste la reflexión y la serialización, que pueden romper un constructor privado. Es la opción que Bloch prefiere [5]:

app/SettingsEnum.java (ejemplo, no está en el proyecto)

```
1  package emberdeep.app;
2
3  public enum SettingsEnum {
4      INSTANCE;
5
6      public long seed() { return 42L; }
7      public int floors() { return 3; }
8  }
```

Se usa como `SettingsEnum.INSTANCE.seed()`. Su límite: un `enum` no puede extender otra clase (sí implementar interfaces).

4.9 Si ya lo conoces: lo nuevo en Singleton es medir su coste

Escribir un Singleton ya lo sabías. Lo nuevo es ponerle precio:

- **Estado global.** Cualquier clase lo toca sin que su constructor lo diga. No ves las dependencias, y un cambio en la instancia afecta a clases que no esperabas.
- **Pruebas.** No puedes darle a una clase una configuración distinta para probarla, porque ella misma llama a `getInstance()`. Con una instancia por constructor sí. Lo practicarás en el ejercicio 4.4.
- **Hilos.** El Singleton ingenuo tiene una carrera de datos; el holder y el `enum` la evitan. Si el Singleton además guarda estado mutable, sus datos siguen siendo compartidos entre hilos.
- **Salida.** Quitar un Singleton de un proyecto es una refactorización conocida, *Inline Singleton* [17].

4.10 Singleton: cuándo sí y cuándo no; la alternativa de `Main`

La pregunta no es «¿hay una sola instancia?», sino «¿necesito además que se pueda llegar a ella desde cualquier sitio?». Casi siempre lo primero sí y lo segundo no. La alternativa: crear la instancia *una vez* en `Main`, el composition root del capítulo 1, y pasarla por constructor a quien la use. Sigue habiendo una sola, pero cada clase declara que la necesita.

Singleton compensa cuando el recurso es único de verdad y no se puede construir varias veces (un registro del sistema), el acceso desde muchos sitios es inevitable y no vas a necesitar sustituirlo en pruebas. **No compensa** para configuración, listas de datos o cualquier cosa que quieras probar con otro valor: ahí, una instancia pasada por constructor. Lo pagas en que algunas clases reciban un parámetro más, un coste pequeño y visible.

Emberdeep aplica la alternativa con un `record`:

app/GameConfig.java

```java
package emberdeep.app;

import emberdeep.engine.combat.damage.DamageFormula;
import java.util.Objects;

public record GameConfig(long seed, int floors, DamageFormula defaultFormula) {
    public GameConfig {
        if (floors < 1) {
            throw new IllegalArgumentException("floors debe ser al menos 1");
        }
        Objects.requireNonNull(defaultFormula, "defaultFormula");
    }
}
```

`Main` crea un `GameConfig` una sola vez y lo recibe por su constructor; nadie más puede alcanzarlo sin que se lo pasen. En una prueba puedes crear otra configuración distinta sin que ninguna global se entere. Para una primera lectura del patrón y sus variantes, tienes [7] y [24]; lo importante es que usar un Singleton sea una decisión y no un hábito.

4.11 En el proyecto: H4 con Builder y Singleton

El hito H4 hace tres cosas: los héroes salen de un `HeroBuilder`, la mazmorra de tres plantas sale de un `DungeonBuilder` con `FloorBuilder` anidado, y los datos comunes viven en un `GameConfig` creado una sola vez en `Main`. Cambios:

- **Nuevo** `HeroBuilder` (en `engine/character/`); `Hero` no cambia.
- **Nuevo** `Floor`, `Dungeon` y `DungeonBuilder` en `engine/world/`.
- **Nuevo** `GameConfig` en `app/`: semilla 42, tres plantas y la fórmula plana por defecto.
- `Main` deja de ser solo un método estático: guarda el `GameConfig` que le pasa su propio `main`. El primer argumento es ahora el número de planta (1 a 3; la 3 es otra cripta). Con un segundo argumento `default` no se pregunta la fórmula: se usa la de la configuración.
- `GameSession`, `Combat` y el resto del motor no cambian.

Hito 4: héroes y mazmorra construidos paso a paso

Tras este hito, el grupo sale de tres `HeroBuilder`; la mazmorra tiene tres plantas (cripta, caverna, cripta) construidas con `DungeonBuilder`; y un `build()` inválido lanza `IllegalStateException`. La estructura:

Estructura de carpetas tras el capítulo 4

```
emberdeep/
|-- settings.gradle
|-- build.gradle
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
`-- src/main/java/emberdeep/
    |-- app/
    |   |-- Main.java              (composition root)
    |   `-- GameConfig.java
    |-- engine/
    |   |-- character/             Stats, Hero, HeroBuilder, Enemy,
    ↪ EnemyKind, Fighter
    |   |-- combat/
    |   |   |-- Combat.java
    |   |   |-- ai/                EnemyAi, FirstAliveAi, LowestHpAi,
    ↪ RandomTargetAi
    |   |   `-- damage/            DamageFormula, FlatFormula,
    ↪ ScaledFormula
    |   |-- item/                  Item, ItemKind, Inventory
    |   |-- session/               GameSession
    |   `-- world/
    |       |-- Dungeon.java, Floor.java, DungeonBuilder.java
    |       `-- biome/             BiomeFactory, CryptBiome, CavernBiome
    `-- ui/
        `-- ConsoleUi.java
```

Lo practicas en el ejercicio 4.5. Antes, cuatro ejercicios más pequeños, en otros dominios.

4.12 Ejercicios

Ejercicio 4.1: Un QuestBuilder que valida en build ★★☆☆☆

Vas a crear una misión (`Quest`) con dos datos obligatorios y dos opcionales. Son archivos nuevos, sin paquete, no de Emberdeep. Pasos:

1. Crea el `record Quest(String title, int reward, int minLevel, String giver)`.
2. Dentro del record, declara la clase `public static class Builder`, con los campos `title` y `reward` (obligatorios) y `minLevel` (valor por defecto 1) y `giver` (por defecto `"Anónimo"`).
3. Añade un método por campo, con el mismo nombre del campo, que guarde el valor y devuelva `this`.
4. Escribe `Quest build()`: lanza `IllegalStateException` con el mensaje «Falta el título» si `title` es `null` o está en blanco, y con «La recompensa debe ser positiva» si `reward` no es mayor que 0. Si todo está bien, devuelve el `Quest`.
5. En una clase `Main`, crea una misión «Limpiar la cripta» con recompensa 50 y solo esos datos, otra «Matar al rey goblin» con recompensa 200, nivel 3 y dador «Mara»,

y un intento con título «Sin premio» y sin recompensa, capturando la excepción e imprimiendo «Error: » y su mensaje.

Está bien si `java Main` imprime

```
Quest[title=Limpiar la cripta, reward=50, minLevel=1, giver=Anónimo]
Quest[title=Matar al rey goblin, reward=200, minLevel=3, giver=Mara]
Error: La recompensa debe ser positiva
```

y si los dos opcionales pueden omitirse sin pasar `null`.

Guiado ▪ solución en la pág. 106

Ejercicio 4.2: Dos hilos, dos instancias ★★☆☆☆

Este programa (no es de Emberdeep; dos archivos sin paquete) debería usar un único `MapRegistry`. El constructor tarda un poco en construirse: `Thread.sleep(100)` pausa el hilo 100 ms, y su `try/catch` es obligatorio porque `sleep` puede lanzar `InterruptedException`. Se simula así una creación lenta.

MapRegistry.java y Main.java

```java
public class MapRegistry {
    private static MapRegistry instance;
    private static int created = 0;

    private MapRegistry() {
        created++;
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            throw new IllegalStateException(e);
        }
    }

    public static MapRegistry getInstance() {
        if (instance == null) {
            instance = new MapRegistry();
        }
        return instance;
    }

    public static int created() { return created; }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        MapRegistry[] seen = new MapRegistry[2];
        Thread a = new Thread(() -> seen[0] = MapRegistry.getInstance());
        Thread b = new Thread(() -> seen[1] = MapRegistry.getInstance());
        a.start();
        b.start();
        a.join();
        b.join();
        System.out.println((seen[0] == seen[1]) + " " + MapRegistry.created
    ↪ ());
    }
}
```

1. Ejecútalo varias veces. ¿Qué imprime en casi todas y qué debería imprimir?
2. Explica qué ocurre, paso a paso, entre los dos hilos.
3. Corrígelo con el holder, sin tocar `Main`.

Está bien si, tras corregirlo, imprime siempre `true 1`, y si explicas la carrera con las palabras «comprobación» y «asignación».

Pista: *Ahora el hilo que llega segundo ve* `instance == null` *mientras el primero aún está dentro del constructor.*

Depuración ▪ solución en la pág. 107

Ejercicio 4.3: Record, Builder o with ★★★☆☆

Para cada clase de la tabla (ninguna es de Emberdeep), decide si basta un `record` con constructor compacto, hace falta un Builder, o conviene un método `with`.

Clase	Descripción
`Coordinate`	Dos enteros `x` e `y`, ambos no negativos.
`Spell`	Obligatorios: `name` y `manaCost` (mayor que 0). Opcionales: `damage`, `range`, `cooldown`, `element` y `area`. Regla: si `area` es mayor que 0, `cooldown` debe ser al menos 1.
`Stats`	`maxHp`, `attack`, `defense`, todos obligatorios. Al subir de nivel un héroe, solo cambia el ataque.

Entrega: (1) una tabla con la decisión de cada clase y dos criterios que la justifican (usa los de la sección 4.4); (2) el código que decidiste para cada una: el constructor compacto de `Coordinate`, el `Spell` con su Builder completo, y el método `withAttack` de `Stats`.
Está bien si cada decisión se apoya en al menos dos criterios (número de opcionales, reglas entre campos, legibilidad, un solo cambio), si el Builder valida la regla entre `area` y `cooldown` en `build()` y no en los pasos, y si el código de las tres clases es completo.

Diseño ▪ solución en la pág. 108

Ejercicio 4.4: De getInstance a una instancia pasada por constructor ★★★☆☆

Partes de estos archivos (no son de Emberdeep; sin paquete).

Settings, Welcome, Reward, Report y Main

```java
public final class Settings {
    private static final Settings INSTANCE = new Settings();
    private final String playerName = "Aria";
    private final int difficulty = 2;

    private Settings() {}

    public static Settings getInstance() { return INSTANCE; }
    public String playerName() { return playerName; }
    public int difficulty() { return difficulty; }
}

public class Welcome {
    public String text() {
        return "Hola, " + Settings.getInstance().playerName();
    }
}

public class Reward {
    public int gold() {
        return 10 * Settings.getInstance().difficulty();
    }
}

public class Report {
    public String text() {
        Settings s = Settings.getInstance();
        return s.playerName() + " juega en dificultad " + s.difficulty();
    }
}

public class Main {
    public static void main(String[] args) {
        System.out.println(new Welcome().text());
        System.out.println("Recompensa: " + new Reward().gold());
        System.out.println(new Report().text());
    }
}
```

Refactoriza para que no exista `getInstance()`: `Settings` pasa a ser un `record Settings(String playerName, int difficulty)`, y `Welcome`, `Reward` y `Report` reciben el `Settings` por constructor. Después, en `main`, usa cada clase con el `Settings("Aria", 2)` y con otro, `Settings("Test", 5)`.
Está bien si `java Main` imprime

```
Hola, Aria
Recompensa: 20
Aria juega en dificultad 2
Hola, Test
Recompensa: 50
Test juega en dificultad 5
```

y si ninguna clase del programa contiene la palabra `getInstance`.

Pista: *Cada una de las tres clases guarda el* `Settings` *en un campo* `private final`*.*

Práctica ▪ solución en la pág. 111

Ejercicio 4.5: Hito 4: héroes y mazmorra en Emberdeep ★★★☆☆

Implementa el hito H4 partiendo del estado del capítulo anterior (solución del ejercicio 3.5), en tres pasos.

1. **`HeroBuilder`.** En `engine/character/`, con los pasos `name` y `stats` y un `build()` que lanza `IllegalStateException` si el nombre es `null` o está en blanco, si faltan las estadísticas, o si la vida o el ataque no son positivos o la defensa es negativa. `Main` crea con él a Aria (30, 8, 3), Borin (40, 6, 5) y Cael (24, 9, 2).
2. **Mazmorra.** En `engine/world/`, los records `Floor(biome, enemies)` y `Dungeon(floors)` (con copia inmutable de las listas) y `DungeonBuilder` con `FloorBuilder` interno (`biome`, `enemies(List<EnemyKind>)`, `done()`). `build()` falla si no hay plantas o si alguna no tiene enemigos. `Main` monta cripta, caverna y cripta, reutilizando el mismo `CryptBiome`.
3. **`GameConfig`.** En `app/`, un `record` con semilla, número de plantas y fórmula por defecto, que `Main` crea una sola vez (semilla 42, 3 plantas, `FlatFormula`) y recibe por constructor. El primer argumento es el número de planta (por defecto 1; fuera de rango, `IllegalArgumentException`); con un segundo argumento `default` no se pregunta la fórmula.

Está bien si:

1. Sin argumentos y con el argumento `2`, el juego da exactamente las mismas trazas que al final del capítulo anterior (cripta y caverna), y con el argumento `3` las de la cripta, con los mismos enemigos. Por ejemplo, para la planta 3:

   Planta 3

   ```
   ./gradlew run --args="3"
   ```

2. Con dos argumentos, el segundo `default`, no aparece la pregunta de la fórmula y se juega con la fórmula plana:

   Sin pregunta de fórmula

   ```
   ./gradlew run --args="1 default"
   ```

3. Con el argumento `4` el programa termina con `IllegalArgumentException`.
4. Si añades temporalmente estas líneas al principio de `play`, la salida son tres mensajes; bórralas después:

Comprobación temporal en Main.java

```
List<Runnable> checks = List.of(
        () -> new DungeonBuilder().build(),
        () -> new DungeonBuilder().floor()
                .biome(new CryptBiome(new Random(1))).done().build(),
        () -> new HeroBuilder().name(" ").build());
for (Runnable check : checks) {
    try {
        check.run();
    } catch (IllegalStateException e) {
        System.out.println(e.getMessage());
    }
}
```

```
La mazmorra necesita al menos una planta
La planta 1 no tiene enemigos
El héroe necesita un nombre
```

Pista: *Un `Runnable` es una función sin parámetros ni resultado; `check.run()` la ejecuta.*

Proyecto ▪ solución en la pág. 112

4.13 Resumen

Resumen

- El olor de Builder es un constructor con muchos parámetros sin nombre, o un objeto que solo vale si se monta entero. Builder da un paso por dato y valida todo en `build()`.
- Un `record` con constructor compacto basta con pocos parámetros y reglas sobre sus propios campos; el Builder compensa con muchos opcionales, reglas entre datos o montaje anidado. Para cambiar un dato, un `with`.
- Un builder anidado (`FloorBuilder` dentro de `DungeonBuilder`) monta un árbol; `done()` vuelve al padre. No se usa el «director» del GoF.
- El Singleton ingenuo tiene una carrera de datos con dos hilos; el holder y el `enum` la evitan.
- El coste de Singleton es el estado global: dependencias ocultas y pruebas difíciles. La alternativa es crear la instancia una vez en `Main` y pasarla por constructor.
- H4: `HeroBuilder`, `Dungeon`, `Floor`, `DungeonBuilder` con `FloorBuilder` y `GameConfig`; mazmorra de tres plantas.

Soluciones de los ejercicios

Solución del ejercicio 4.1

enunciado en la pág. 99

Quest.java

```
public record Quest(String title, int reward, int minLevel, String giver) {

    public static class Builder {
        private String title;
        private int reward;
        private int minLevel = 1;
        private String giver = "Anónimo";

        public Builder title(String title) {
            this.title = title;
            return this;
        }

        public Builder reward(int reward) {
            this.reward = reward;
            return this;
        }

        public Builder minLevel(int minLevel) {
            this.minLevel = minLevel;
            return this;
        }

        public Builder giver(String giver) {
            this.giver = giver;
            return this;
        }

        public Quest build() {
            if (title == null || title.isBlank()) {
                throw new IllegalStateException("Falta el título");
            }
            if (reward <= 0) {
                throw new IllegalStateException("La recompensa debe ser
    ↪ positiva");
            }
            return new Quest(title, reward, minLevel, giver);
        }
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Quest simple = new Quest.Builder()
                .title("Limpiar la cripta")
                .reward(50)
                .build();
        Quest full = new Quest.Builder()
                .title("Matar al rey goblin")
                .reward(200)
                .minLevel(3)
                .giver("Mara")
                .build();
        System.out.println(simple);
        System.out.println(full);
        try {
            new Quest.Builder().title("Sin premio").build();
        } catch (IllegalStateException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Por qué funciona. Los obligatorios no tienen valor por defecto útil (`null` y 0) y `build()` los comprueba; los opcionales nacen con su valor por defecto, así que se pueden omitir. Cada paso devuelve `this`, y por eso se encadenan. Desde fuera, la clase se nombra `Quest.Builder` porque es una clase anidada `static`.

Errores típicos:

- Olvidar `static`: sin él, `new Quest.Builder()` necesitaría un `Quest` previo y no compila.
- Validar en cada paso en lugar de en `build()`: el orden de los pasos pasa a importar.
- Devolver `void` en un paso: la cadena se rompe.

Solución del ejercicio 4.2

enunciado en la pág. 100

1. En casi todas las ejecuciones imprime `false 2`; debería imprimir `true 1` (la misma instancia y una sola creación).
2. El hilo A llama a `getInstance()`, comprueba `instance == null` (cierto) y entra en el constructor, que tarda 100 ms. `instance` aún no está asignada, porque la asignación ocurre cuando el constructor termina. Mientras tanto el hilo B hace la misma comprobación, que sigue siendo cierta, y también construye. Cada hilo asigna la suya: dos instancias. La comprobación y la asignación son dos pasos separados, y nada impide que otro hilo se cuele entre ellos.
3. Con el holder (`Main` no cambia):

MapRegistry.java (corregido)

```java
public class MapRegistry {
    private static int created = 0;

    private MapRegistry() {
        created++;
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            throw new IllegalStateException(e);
        }
    }

    private static class Holder {
        private static final MapRegistry INSTANCE = new MapRegistry();
    }

    public static MapRegistry getInstance() {
        return Holder.INSTANCE;
    }

    public static int created() { return created; }
}
```

Por qué funciona. Java inicializa `Holder` una sola vez; si dos hilos lo piden a la vez, uno inicializa y el otro espera. No hay comprobación en dos pasos. Ahora imprime siempre `true` 1.

Alternativa. Marcar `getInstance` como `synchronized` también lo arregla, pero bloquea cada llamada aunque la instancia ya exista.

Errores típicos:

- Quitar el `sleep` y concluir que «ya funciona»: la carrera sigue ahí, solo se vuelve rara. No funcionar a veces es peor que no funcionar nunca.
- Arreglar `Main` (por ejemplo, serializando los hilos con `join` antes de arrancar el segundo): el fallo está en la clase, no en quien la usa.

Solución del ejercicio 4.3 enunciado en la pág. 101

Clase	Decisión	Criterios
`Coordinate`	`record` con constructor compacto	Dos parámetros obligatorios; la regla (no negativos) es sobre sus propios campos. Un Builder sería más código que el propio record.
`Spell`	Builder	Cinco opcionales (la llamada por posición sería ilegible) y una regla entre campos (`area` y `cooldown`) que solo se puede comprobar con todos los datos.
`Stats`	`record` más `withAttack`	Tres campos obligatorios, pero el uso real es copiar y cambiar uno solo; un método `with` lo expresa sin builder.

Coordinate.java y Stats.java

```java
public record Coordinate(int x, int y) {
    public Coordinate {
        if (x < 0 || y < 0) {
            throw new IllegalArgumentException("Coordenada negativa");
        }
    }
}

public record Stats(int maxHp, int attack, int defense) {
    public Stats withAttack(int newAttack) {
        return new Stats(maxHp, newAttack, defense);
    }
}
```

Spell.java

```java
public record Spell(String name, int manaCost, int damage, int range,
                    int cooldown, String element, int area) {

    public static class Builder {
        private String name;
        private int manaCost;
        private int damage;
        private int range = 1;
        private int cooldown;
        private String element = "neutro";
        private int area;

        public Builder name(String name) { this.name = name; return this; }
        public Builder manaCost(int v) { this.manaCost = v; return this; }
        public Builder damage(int v) { this.damage = v; return this; }
        public Builder range(int v) { this.range = v; return this; }
        public Builder cooldown(int v) { this.cooldown = v; return this; }
        public Builder element(String v) { this.element = v; return this; }
        public Builder area(int v) { this.area = v; return this; }

        public Spell build() {
            if (name == null || name.isBlank()) {
                throw new IllegalStateException("Falta el nombre");
            }
            if (manaCost <= 0) {
                throw new IllegalStateException("El coste debe ser positivo
    ↪ ");
            }
            if (area > 0 && cooldown < 1) {
                throw new IllegalStateException("Un hechizo de área
    ↪ necesita enfriamiento");
            }
            return new Spell(name, manaCost, damage, range, cooldown,
    ↪ element, area);
        }
    }
}
```

Criterios para valorar otras soluciones. Vale otra decisión si la justificas con los criterios: por ejemplo, `Coordinate` con Builder solo se defiende si prevés muchos campos más. Para `Stats`, un Builder sería razonable si el héroe ganara muchos campos opcionales; si mañana `Stats` tuviera diez campos, la decisión cambiaría.

Errores típicos:

- Poner la regla de `area` y `cooldown` en los pasos `area` o `cooldown`: depende del orden en que los llames.
- Dar un Builder a `Coordinate` «por si acaso»: sobrediseño.

Solución del ejercicio 4.4

enunciado en la pág. 102

`Settings.java`, `Welcome.java`, `Reward.java` y `Report.java` (cuatro archivos)

```java
public record Settings(String playerName, int difficulty) {}

public class Welcome {
    private final Settings settings;

    public Welcome(Settings settings) {
        this.settings = settings;
    }

    public String text() {
        return "Hola, " + settings.playerName();
    }
}

public class Reward {
    private final Settings settings;

    public Reward(Settings settings) {
        this.settings = settings;
    }

    public int gold() {
        return 10 * settings.difficulty();
    }
}

public class Report {
    private final Settings settings;

    public Report(Settings settings) {
        this.settings = settings;
    }

    public String text() {
        return settings.playerName() + " juega en dificultad "
                + settings.difficulty();
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        show(new Settings("Aria", 2));
        show(new Settings("Test", 5));
    }

    private static void show(Settings settings) {
        System.out.println(new Welcome(settings).text());
        System.out.println("Recompensa: " + new Reward(settings).gold());
        System.out.println(new Report(settings).text());
    }
}
```

Por qué funciona. Cada clase declara en su constructor lo que necesita, y `main`, el composition root, decide qué `Settings` le da. Como ya no hay acceso global, se puede usar cada clase con una configuración distinta en la misma ejecución, que con `getInstance()` era imposible. Sigue habiendo un solo `Settings` *por uso*, creado en un único sitio.

Errores típicos:

- Dejar un constructor sin argumentos que siga llamando a `getInstance()`.
- Conservar `Settings` como clase con constructor privado: no se puede crear uno distinto.
- Olvidar pasar el `Settings` a una de las tres clases: no compila, y eso es lo que se quiere.

Solución del ejercicio 4.5

enunciado en la pág. 104

Los archivos nuevos o cambiados son seis; `GameSession`, `Combat`, `ConsoleUi`, las fábricas de bioma y el resto no cambian.

engine/character/HeroBuilder.java

HeroBuilder.java

```java
package emberdeep.engine.character;

public class HeroBuilder {
    private String name;
    private Stats stats;

    public HeroBuilder name(String name) {
        this.name = name;
        return this;
    }

    public HeroBuilder stats(Stats stats) {
        this.stats = stats;
        return this;
    }

    public Hero build() {
        if (name == null || name.isBlank()) {
            throw new IllegalStateException("El héroe necesita un nombre");
        }
        if (stats == null) {
            throw new IllegalStateException("El héroe necesita estadísticas
    ↪ ");
        }
        if (stats.maxHp() <= 0 || stats.attack() <= 0 || stats.defense() <
    ↪ 0) {
            throw new IllegalStateException("Estadísticas no válidas: " +
    ↪ stats);
        }
        return new Hero(name, stats);
    }
}
```

engine/world/Floor.java y Dungeon.java

Floor.java

```java
package emberdeep.engine.world;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.List;

public record Floor(BiomeFactory biome, List<EnemyKind> enemies) {
    public Floor {
        enemies = List.copyOf(enemies);
    }
}
```

Dungeon.java

```java
package emberdeep.engine.world;

import java.util.List;

public record Dungeon(List<Floor> floors) {
    public Dungeon {
        floors = List.copyOf(floors);
    }
}
```

engine/world/DungeonBuilder.java

DungeonBuilder.java

```java
package emberdeep.engine.world;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.ArrayList;
import java.util.List;

public class DungeonBuilder {
    private final List<Floor> floors = new ArrayList<>();

    public FloorBuilder floor() {
        return new FloorBuilder();
    }

    public Dungeon build() {
        if (floors.isEmpty()) {
            throw new IllegalStateException("La mazmorra necesita al menos una planta");
        }
        for (int i = 0; i < floors.size(); i++) {
            Floor floor = floors.get(i);
            if (floor.biome() == null) {
                throw new IllegalStateException("La planta " + (i + 1) + " no tiene bioma");
            }
            if (floor.enemies().isEmpty()) {
                throw new IllegalStateException("La planta " + (i + 1) + " no tiene enemigos");
            }
        }
        return new Dungeon(floors);
    }

    public class FloorBuilder {
        private BiomeFactory biome;
        private List<EnemyKind> enemies = List.of();

        public FloorBuilder biome(BiomeFactory biome) {
            this.biome = biome;
            return this;
        }

        public FloorBuilder enemies(List<EnemyKind> enemies) {
            this.enemies = enemies;
            return this;
        }

        public DungeonBuilder done() {
            floors.add(new Floor(biome, enemies));
            return DungeonBuilder.this;
        }
    }
}
```

app/GameConfig.java

GameConfig.java

```java
package emberdeep.app;

import emberdeep.engine.combat.damage.DamageFormula;
import java.util.Objects;

public record GameConfig(long seed, int floors, DamageFormula
    ↪ defaultFormula) {
    public GameConfig {
        if (floors < 1) {
            throw new IllegalArgumentException("floors debe ser al menos 1")
    ↪ ;
        }
        Objects.requireNonNull(defaultFormula, "defaultFormula");
    }
}
```

app/Main.java (en tres tramos seguidos, dentro de la misma clase)

Main.java (1/3: imports y arranque)

```java
package emberdeep.app;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.Dungeon;
import emberdeep.engine.world.DungeonBuilder;
import emberdeep.engine.world.Floor;
import emberdeep.engine.world.biome.BiomeFactory;
import emberdeep.engine.world.biome.CavernBiome;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.ui.ConsoleUi;
import java.util.List;
import java.util.Random;
import java.util.Scanner;

public class Main {
    private final GameConfig config;

    private Main(GameConfig config) {
        this.config = config;
    }

    public static void main(String[] args) {
        GameConfig config = new GameConfig(42L, 3, new FlatFormula());
        new Main(config).play(args);
    }
```

Main.java (2/3: la partida)

```
    private void play(String[] args) {
        int floorNumber = chooseFloor(args);
        ConsoleUi ui = new ConsoleUi(new Scanner(System.in), System.out);
        DamageFormula formula = chooseFormula(ui, args);
        Random random = new Random(config.seed());

        BiomeFactory crypt = new CryptBiome(random);
        BiomeFactory cavern = new CavernBiome(random);
        List<EnemyKind> cryptKinds = List.of(
                EnemyKind.SKELETON, EnemyKind.ASH_RAT, EnemyKind.ZOMBIE);
        List<EnemyKind> cavernKinds = List.of(
                EnemyKind.GOBLIN, EnemyKind.BAT, EnemyKind.SPIDER);

        Dungeon dungeon = new DungeonBuilder()
                .floor().biome(crypt).enemies(cryptKinds).done()
                .floor().biome(cavern).enemies(cavernKinds).done()
                .floor().biome(crypt).enemies(cryptKinds).done()
                .build();
        Floor floor = dungeon.floors().get(floorNumber - 1);

        GameSession session = new GameSession(
                heroes(), floor.biome(), floor.enemies(), formula);
        ui.run(session);
    }
```

Main.java (3/3: auxiliares y llave final)

```
    private DamageFormula chooseFormula(ConsoleUi ui, String[] args) {
        if (args.length > 1 && args[1].equals("default")) {
            return config.defaultFormula();
        }
        return ui.askFormula() == 1 ? new FlatFormula() : new ScaledFormula
    ↪ ();
    }

    private int chooseFloor(String[] args) {
        int floor = args.length > 0 ? Integer.parseInt(args[0]) : 1;
        if (floor < 1 || floor > config.floors()) {
            throw new IllegalArgumentException(
                    "La planta debe estar entre 1 y " + config.floors());
        }
        return floor;
    }

    private List<Hero> heroes() {
        return List.of(
                new HeroBuilder().name("Aria")
                        .stats(new Stats(30, 8, 3)).build(),
                new HeroBuilder().name("Borin")
                        .stats(new Stats(40, 6, 5)).build(),
                new HeroBuilder().name("Cael")
                        .stats(new Stats(24, 9, 2)).build());
    }
}
```

Por qué funciona. Los héroes salen de tres builders, con los mismos números que antes, y la mazmorra, de un builder anidado que valida al final. El mismo `CryptBiome` sirve a las plantas 1 y 3: un bioma es una fábrica sin estado propio salvo el `Random` compartido. Como cada ejecución juega una sola planta con `Random(42)` recién creado y el builder no consume números aleatorios, las trazas de las plantas 1 y 2 son las del capítulo anterior, y la 3 repite la de la 1. `GameConfig` se crea en `main` y se pasa al constructor de `Main`; nadie más lo ve. Para comprobar los mensajes de validación, pega temporalmente en `play` el fragmento del enunciado: da los tres textos esperados.

Errores típicos:

- Olvidar `done()`: la planta no se añade, y `build()` dice que no hay plantas.
- Hacer `FloorBuilder` `static`: no puede ver `floors` ni escribir `DungeonBuilder.this`.
- Crear un `new Random(...)` por bioma o usar la semilla en otro sitio: cambia las trazas. Se crea uno, con `config.seed()`.
- Pasar un número de planta fuera de 1 a 3 y esperar la cripta por defecto: ahora es un error.

Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo siguiente.

CAPÍTULO 5

Prototype y Flyweight: copiar o compartir

Objetivos del capítulo

- Reconocer el olor que pide un Prototype: un objeto ya montado del que quieres variantes con un retoque.
- Aplicar Prototype con una plantilla mutable, un `copy()` profundo y un registro, y demostrar por qué falla la copia superficial.
- Reconocer el olor que pide un Flyweight y separar el estado intrínseco del extrínseco.
- Aplicar Flyweight con una fábrica de flyweights, `Map.computeIfAbsent`, la identidad con `==` y un contador de instancias.
- Elegir entre Factory, Prototype y Flyweight, y dar los hitos H5 y H6: plantillas de enemigo con élites y un mapa ASCII con casillas compartidas.

Los enemigos de Emberdeep se construyen desde cero cada vez que una fábrica de bioma lo pide, y no tienen habilidades. Pronto querrás un «élite»: un zombi igual que el normal, pero con más vida y un golpe extra. Y querrás un mapa, donde cientos de casillas repiten los mismos tres datos. Son dos formas de pensar un objeto repetido. Con la primera (Prototype) *copias* algo ya montado y lo retocas. Con la segunda (Flyweight) *compartes* lo que nunca cambia, en lugar de copiarlo. Empiezas por copiar.

5.1 El olor que pide Prototype: montar la misma plantilla cien veces

Hoy una fábrica de bioma monta cada enemigo con un `new` y una lista de datos. Supón que los enemigos ya tuvieran habilidades (en este capítulo las tendrán) y que quieres el zombi normal y, además, el zombi élite:

Hipotético: dos zombis montados a mano

```
Enemy zombie = new Enemy("Zombi", EnemyKind.ZOMBIE, new Stats(22, 5, 1),
        new FirstAliveAi(), List.of(new Skill("Abrazo pútrido", 7)));

Enemy eliteZombie = new Enemy("Élite Zombi", EnemyKind.ZOMBIE, new Stats(33, 7,
    ↪ 2),
        new FirstAliveAi(), List.of(new Skill("Abrazo pútrido", 7),
                                    new Skill("Furia", 12)));
```

El élite repite todo el zombi y cambia tres cosas. Si mañana el abrazo pasa de 7 a 8, tienes que acordarte de cambiarlo en los dos sitios, y en los seis tipos de enemigo, y en todas las variantes que se añadan. Este es el olor: **varios objetos casi iguales que se montan repitiendo los mismos datos**. Lo que quieres decir es «dame otro como este, pero con esto distinto».

Prototype hace justo eso. Su intención en el catálogo es indicar qué objetos crear mediante una instancia prototípica y crear los nuevos *copiándola* [9]. No se parte de una clase y un `new`, sino de un objeto ya montado. En Emberdeep, ese objeto será una *plantilla* de enemigo, y la copia se convertirá en un enemigo de verdad. Nystrom dedica un capítulo a este patrón, con monstruos que se crean con «spawners» como ejemplo [19].

5.2 Piezas del patrón Prototype

Rol	Clase en el ejemplo	Propósito	Paquete
Prototipo (clase mutable)	`EnemyTemplate`	Objeto montado que sabe copiarse con `copy()`	`engine.character`
Parte mutable copiada (clase y record)	`List<Skill>` y `Skill` (inmutable)	La lista se duplica; los `Skill` se comparten	`engine.character`
Registro de prototipos (clase)	`TemplateRegistry`	Guarda una plantilla por `EnemyKind`	`engine.character`
Variante (método)	`asElite()`	Copia más ajustes de estadísticas y una habilidad	`engine.character`
Alternativa evitada (interfaz del JDK)	`Cloneable`	No se usa: se explica por qué	`java.lang`
Cliente (clase)	`CryptBiome`, `CavernBiome`	Piden una copia y la convierten en `Enemy`	`engine.world.biome`

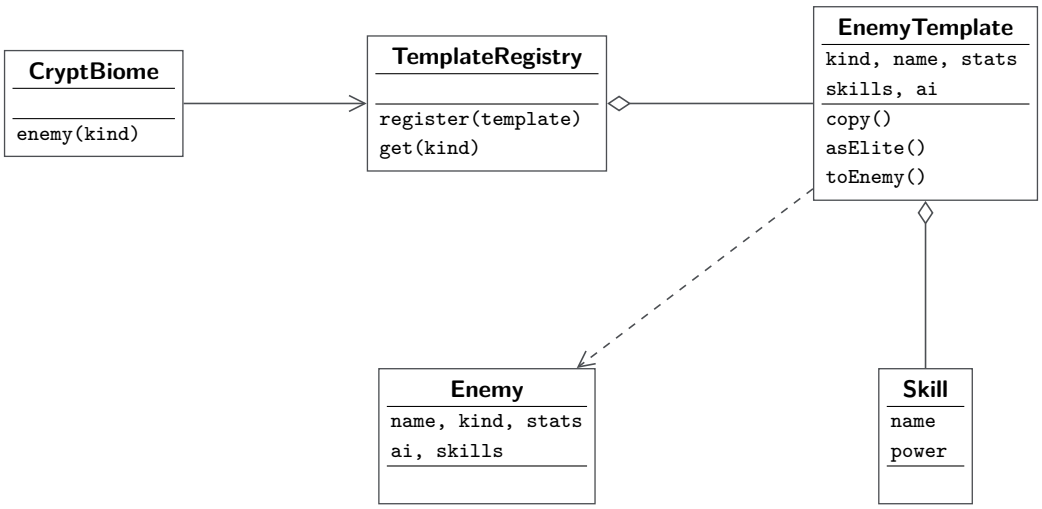


Diagrama UML del patrón Prototype: el cliente pide al registro una plantilla, la copia y la convierte en `Enemy`.

Sin interfaz Prototype

El catálogo original define una interfaz `Prototype` con un método de clonado, porque el cliente debe copiar objetos de clases que no conoce. Aquí solo hay una clase prototípica, `EnemyTemplate`, así que no hace falta la interfaz y el diagrama no la dibuja.

5.3 El ejemplo de Prototype, pieza a pieza: `EnemyTemplate.copy()`

La habilidad. Una `Skill` es un dato: un nombre y una potencia. Hasta el capítulo 7 solo se guarda, no se usa. Es un `record`, y por eso inmutable:

`engine/character/Skill.java`

```
package emberdeep.engine.character;

public record Skill(String name, int power) {}
```

El enemigo. El `Enemy` gana un campo, la lista de habilidades. Sigue siendo un record, y conserva el `kind` del capítulo 3:

`engine/character/Enemy.java`

```
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;
import java.util.List;

public record Enemy(String name, EnemyKind kind, Stats stats, EnemyAi ai,
                    List<Skill> skills) {}
```

La plantilla. `EnemyTemplate` es una clase *mutable*: puedes cambiarle el nombre, las estadísticas y las habilidades. Con un record inmutable no tendría sentido copiar (compartir el original sería seguro), así que el patrón pide un objeto con estado que cambia. Los campos y los dos constructores:

`engine/character/EnemyTemplate.java (1/2: datos y copia)`

```
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class EnemyTemplate {
    private final EnemyKind kind;
    private String name;
    private Stats stats;
    private final List<Skill> skills;
    private final EnemyAi ai;

    public EnemyTemplate(EnemyKind kind, String name, Stats stats, EnemyAi ai)
    ↪ {
        this(kind, name, stats, new ArrayList<>(), ai);
    }

    private EnemyTemplate(EnemyKind kind, String name, Stats stats,
                          List<Skill> skills, EnemyAi ai) {
        this.kind = kind;
        this.name = name;
        this.stats = stats;
```

```
        this.skills = skills;
        this.ai = ai;
    }

    public EnemyTemplate copy() {
        return new EnemyTemplate(kind, name, stats, new ArrayList<>(skills), ai
    ↪ );
    }

    public Enemy toEnemy() {
        return new Enemy(name, kind, stats, ai, List.copyOf(skills));
    }
```

El constructor privado recibe la lista tal cual; el público crea una vacía. `copy()` es el corazón del patrón. Distingue tres casos de lo que contiene la plantilla:

- **Inmutable: se comparte.** `kind`, `name` (un `String`), `stats` (un record) no pueden cambiar por dentro; si cambias el nombre de la copia, asignas un `String` nuevo y el original no se entera.
- **Mutable: se duplica.** La lista `skills` sí cambia por dentro (`add`). Por eso la copia recibe `new ArrayList<>(skills)`, una lista nueva con los mismos elementos. Como cada `Skill` es inmutable, basta copiar la lista, no sus elementos. Esto es una **copia profunda** de lo que hace falta.
- **Compartido a propósito.** La IA (`ai`) se comparte entre la plantilla y sus copias: es una estrategia del capítulo 2, y la de la cripta lleva dentro el único `Random` del juego. Duplicarla rompería la traza con semilla.

`toEnemy()` cierra el trabajo: crea el `Enemy` con el mismo `kind` y una copia inmutable de las habilidades (`List.copyOf`), para que el enemigo ya no comparta nada con la plantilla. La segunda mitad de la clase añade la variante élite, los modificadores y los accesos:

engine/character/EnemyTemplate.java (2/2: élite y accesos)

```

    public EnemyTemplate asElite() {
        EnemyTemplate elite = copy();
        elite.setName("Élite " + name);
        elite.setStats(new Stats(stats.maxHp() * 3 / 2,
                stats.attack() + 2, stats.defense() + 1));
        elite.addSkill(new Skill("Furia", 12));
        return elite;
    }

    public EnemyTemplate addSkill(Skill skill) {
        skills.add(skill);
        return this;
    }

    public void setName(String name) { this.name = name; }
    public void setStats(Stats stats) { this.stats = stats; }

    public EnemyKind kind() { return kind; }
    public String name() { return name; }
    public Stats stats() { return stats; }
```

```
    public EnemyAi ai() { return ai; }

    public List<Skill> skills() {
        return Collections.unmodifiableList(skills);
    }
}
```

`asElite()` es la «variante» del patrón: una copia a la que se le aplican ajustes (vida por uno y medio, un punto más de defensa, dos de ataque y una habilidad nueva). Lo más importante es que *primero* copia y *después* retoca la copia; la plantilla de partida no cambia. `skills()` devuelve una vista que no se puede modificar desde fuera: el único camino para añadir una habilidad es `addSkill`.

El registro. Una plantilla por cada tipo de enemigo, en un `EnumMap` (mapa especializado para claves `enum`, más rápido y ordenado según el orden del `enum`):

engine/character/TemplateRegistry.java

```
package emberdeep.engine.character;

import java.util.EnumMap;
import java.util.Map;

public class TemplateRegistry {
    private final Map<EnemyKind, EnemyTemplate> templates =
            new EnumMap<>(EnemyKind.class);

    public void register(EnemyTemplate template) {
        templates.put(template.kind(), template);
    }

    public EnemyTemplate get(EnemyKind kind) {
        EnemyTemplate template = templates.get(kind);
        if (template == null) {
            throw new IllegalArgumentException("No hay plantilla para " + kind)
    ↪ ;
        }
        return template;
    }
}
```

`get` devuelve el prototipo registrado, no una copia. Regla de uso: **nunca lo modifiques ni lo conviertas directamente**; pide siempre `copy()`. Si lo retocases, cambiarías todos los enemigos futuros de ese tipo.

El cliente. Cada bioma tiene su propio registro y lo rellena en el constructor. Fíjate en que el constructor sigue recibiendo el `Random` (y por eso `Main` no cambia), y en que el `switch` sobre los seis tipos desaparece: el registro solo conoce los tipos de su bioma, y pedir uno ajeno lanza `IllegalArgumentException`, como antes.

engine/world/biome/CryptBiome.java (fragmento: sin imports ni objetos)

```
public class CryptBiome implements BiomeFactory {
    private final TemplateRegistry registry = new TemplateRegistry();

```

```
    public CryptBiome(Random random) {
        registry.register(new EnemyTemplate(EnemyKind.SKELETON, "Esqueleto",
                new Stats(18, 7, 2), new RandomTargetAi(random))
                .addSkill(new Skill("Golpe de hueso", 9)));
        registry.register(new EnemyTemplate(EnemyKind.ASH_RAT, "Rata de ceniza"
    ↪ ,
                new Stats(8, 5, 0), new LowestHpAi())
                .addSkill(new Skill("Mordisco sucio", 6)));
        registry.register(new EnemyTemplate(EnemyKind.ZOMBIE, "Zombi",
                new Stats(22, 5, 1), new FirstAliveAi())
                .addSkill(new Skill("Abrazo pútrido", 7)));
    }

    @Override
    public Enemy enemy(EnemyKind kind) {
        return registry.get(kind).copy().toEnemy();
    }
    // item(ItemKind) no cambia
}
```

`registry.get(kind).copy().toEnemy()` resume el patrón: del prototipo, una copia; de la copia, el producto. Esto también significa que `CryptBiome` sigue siendo una fábrica del capítulo 3: Prototype no la sustituye, trabaja *por dentro*. El mismo registro puede dar una copia por llamada, y cada `Enemy` resultante tiene su propia lista de habilidades.

5.4 Prototype: el bug de la copia superficial

Una copia *superficial* duplica el objeto pero no lo que este tiene dentro: los campos que son referencias apuntan a los mismos objetos que el original. Si esos objetos cambian, cambian para los dos. Es el fallo clásico de Prototype, y se ve con una sola línea mal escrita en `copy()`:

Roto: copy() que comparte la lista

```
public EnemyTemplate copy() {
    return new EnemyTemplate(kind, name, stats, skills, ai);   // skills, sin
    ↪ copiar
}
```

Compila, y la copia parece correcta. Mira qué pasa al crear un élite y mirar el original:

Comprobación (fragmento)

```
EnemyTemplate zombie = new EnemyTemplate(EnemyKind.ZOMBIE, "Zombi",
        new Stats(22, 5, 1), new FirstAliveAi())
        .addSkill(new Skill("Abrazo pútrido", 7));
EnemyTemplate elite = zombie.asElite();
System.out.println(zombie.name() + " " + zombie.skills().size());
System.out.println(elite.name() + " " + elite.skills().size());
```

Con la copia superficial

```
Zombi 2
Élite Zombi 2
```

El zombi normal ya tiene la *Furia* del élite. `asElite()` añadió la habilidad a «su» lista, que era la lista del original. El nombre sí se separó, porque asignar un `String` nuevo no toca el anterior; la lista no, porque `add` cambia el objeto compartido. Con `new ArrayList<>(skills)` en `copy()` la salida es `Zombi 1` y `Élite Zombi 2`. Se arregla igual en cualquier clase: por cada campo, pregunta si es inmutable (se comparte) o mutable (se duplica). Lo practicas en el ejercicio 5.1.

5.5 Si ya lo conoces: lo nuevo en Prototype es la copia profunda y el registro

Si ya conocías Prototype, lo nuevo aquí es:

- **Qué se copia y qué se comparte.** La copia profunda no es «copiar todo»: se duplica lo mutable (la lista), se comparte lo inmutable (los records) y lo compartido a propósito (la IA con su `Random`).
- **El registro de prototipos.** En lugar de que cada cliente guarde su plantilla, un `TemplateRegistry` la entrega por tipo; es la pieza que lo conecta con las fábricas.
- **Por qué no `Cloneable`.** Es una interfaz sin métodos: marca a una clase como clonable, pero el método `Object.clone()` es protegido, lanza una excepción comprobada y hace una copia superficial. Para hacerlo bien hay que sobrescribirlo, llamar a `super.clone()` y arreglar a mano cada campo mutable, y todo choca con los campos `final`. Bloch recomienda un constructor o un método de copia en su lugar [5], que es lo que hace `copy()`.
- **Convive con Factory.** `CryptBiome` es una fábrica que, por dentro, copia plantillas. Copiar y construir no son rivales.

5.6 El olor que pide Flyweight: un `TileType` por cada casilla

El segundo problema es el mapa. Una planta es una rejilla de casillas: muro, suelo, puerta, escalera y trampa. La forma directa de representarlas es un objeto por casilla:

Hipotético: un objeto por casilla

```
public record Tile(char glyph, String color, boolean walkable) {}

// al leer el mapa, para cada carácter:
grid[y][x] = new Tile('#', "gray", false);
```

Cada muro repite los mismos tres valores, y cada suelo también. En el mapa pequeño de este capítulo son unas pocas decenas de objetos y no importa; en un mapa de cien por cien serían diez mil objetos idénticos, y para cambiar el color de los muros tendrías que recorrerlos todos. El olor es **muchos objetos pequeños con datos idénticos que nunca cambian**.

La idea de Flyweight, en el catálogo, es usar la compartición para manejar de forma eficiente un gran número de objetos de grano fino [9]: si cien casillas son «muro», que sean cien referencias al *mismo* objeto muro. Nystrom lo ilustra también con un terreno hecho de casillas [19]. Para que compartir sea seguro hay que separar dos clases de datos:

- **Estado intrínseco**: lo que es igual para todos los usos y no cambia (el glifo, el color, si se puede pisar). Vive dentro del flyweight, que es inmutable.

- **Estado extrínseco**: lo que depende de cada uso (en qué posición está la casilla, dónde está el grupo). *No* vive en el flyweight: lo aporta quien lo usa.

Si metieras la posición dentro del flyweight, dos casillas distintas ya no podrían ser el mismo objeto.

Aquí el ahorro es de demostración

Con cinco tipos de casilla y mapas de 84 casillas, compartir apenas ahorra nada, y no lo medimos: este curso no hace pruebas de rendimiento. Lo que demuestra el ejemplo es la *estructura* (una fábrica que reparte las mismas instancias) y que lo compartido es el mismo objeto. Con cinco tipos fijos, un `enum` también serviría (Nystrom parte de él antes de llegar al flyweight [19]); usamos un `record` y una fábrica porque así se ve el patrón.

5.7 Piezas del patrón Flyweight

Rol	Clase en el ejemplo	Propósito	Paquete
Flyweight (record)	`TileType`	Estado intrínseco inmutable: glifo, color, transitable	`engine.world.map`
Fábrica de flyweights (clase)	`TileTypes`	Entrega siempre la misma instancia por glifo y cuenta cuántas creó	`engine.world.map`
Estado extrínseco (dato, no clase)	Posición `(x, y)`	Dónde está cada casilla y dónde está el grupo	`engine.world.map`
Contexto (clase)	`GameMap`	Rejilla de referencias a `TileType` y posición del grupo	`engine.world.map`
Cliente (clase)	`MapParser`	Convierte texto en `GameMap` pidiendo los tipos a la fábrica	`engine.world.map`

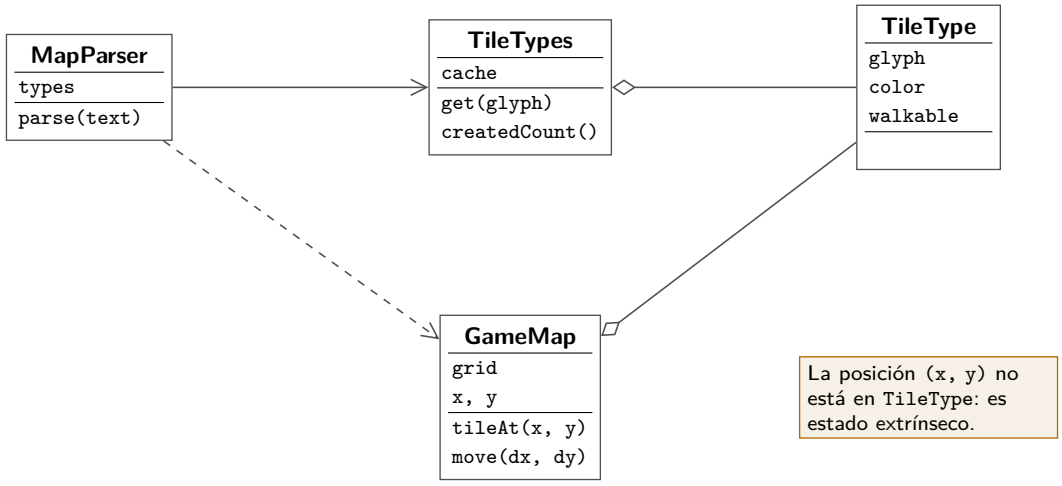


Diagrama UML del patrón Flyweight: la fábrica reparte instancias de `TileType`, y el mapa las referencia muchas veces.

5.8 El ejemplo de Flyweight, pieza a pieza: `TileTypes`

El flyweight. Es un record con los tres datos intrínsecos. El color es un nombre simbólico (`"gray"`, `"red"`): el capítulo 10 decidirá cómo se pinta, y mientras tanto no se usa.

engine/world/map/TileType.java

```
package emberdeep.engine.world.map;

public record TileType(char glyph, String color, boolean walkable) {}
```

Los cinco tipos, con su glifo en el texto del mapa: `#` muro (no transitable), `.` suelo, `+` puerta, `>` escalera y `^` trampa (todas transitables; la trampa aún no hace daño).

La fábrica. Guarda un mapa de glifo a `TileType` y un contador:

engine/world/map/TileTypes.java

```
package emberdeep.engine.world.map;

import java.util.HashMap;
import java.util.Map;

public class TileTypes {
    private final Map<Character, TileType> cache = new HashMap<>();
    private int created = 0;

    public TileType get(char glyph) {
        return cache.computeIfAbsent(glyph, this::create);
    }

    public int createdCount() {
        return created;
    }

    private TileType create(char glyph) {
        TileType type = switch (glyph) {
            case '#' -> new TileType('#', "gray", false);
            case '.' -> new TileType('.', "white", true);
            case '+' -> new TileType('+', "brown", true);
            case '>' -> new TileType('>', "cyan", true);
            case '^' -> new TileType('^', "red", true);
            default -> throw new IllegalArgumentException(
                    "Casilla desconocida: " + glyph);
        };
        created++;
        return type;
    }
}
```

`Map.computeIfAbsent`. Es el método de `Map` que implementa «dame lo que hay; si no hay nada, créalo, guárdalo y dámelo». Recibe la clave y una función que se llama *solo si la clave no está*. Escrito a mano, sería:

A mano: get, comprobar null y put

```
TileType type = cache.get(glyph);
if (type == null) {
    type = create(glyph);
    cache.put(glyph, type);
}
return type;
```

Con `computeIfAbsent(glyph, this::create)` son tres pasos en uno. Si la clave existe, devuelve el valor guardado y *no* llama a `create`; si no, llama una vez, guarda el resultado y lo devuelve. Dos detalles: la función no debe modificar el propio mapa (la nuestra solo incrementa un contador), y si lanza una excepción no se guarda nada, por eso `created++` va después del `switch`: un glifo desconocido no cuenta.

Demostración con == y con el contador. Dos llamadas con el mismo glifo devuelven el mismo objeto; una con otro glifo, otro objeto:

Comprobación (fragmento)

```
TileTypes types = new TileTypes();
TileType a = types.get('#');
TileType b = types.get('#');
TileType c = types.get('.');
System.out.println(a == b);                                  // true
System.out.println(a == c);                                  // false
System.out.println(a == new TileType('#', "gray", false)); // false
System.out.println(a.equals(new TileType('#', "gray", false))); // true
System.out.println(types.createdCount());                    // 2
```

La tercera línea muestra la diferencia: `==` compara *identidad* (¿es el mismo objeto?) y `equals`, en un record, compara *valores*. Flyweight garantiza lo primero. Pedir cien veces `'#'` deja `createdCount()` en 1.

== solo es fiable si la fábrica es la única vía

Con un flyweight, `==` funciona porque nadie más construye `TileType`: si alguien escribe `new TileType('#', ...)`, tendrás dos «muros» distintos. Un caso conocido del propio JDK: `Integer.valueOf(100) == Integer.valueOf(100)` es `true` porque `valueOf` reutiliza los enteros de -128 a 127, pero fuera de ese rango no puedes contar con ello. Moraleja: en código normal, compara con `equals`; usa `==` solo para demostrar la identidad de un flyweight, como aquí.

El contexto y el estado extrínseco. `GameMap` guarda una rejilla de *referencias* a `TileType` y la posición del grupo. Ninguna casilla sabe en qué posición está: la posición es el índice dentro de la rejilla, y es el estado extrínseco. Un fragmento (el archivo entero está en la solución del ejercicio de proyecto):

engine/world/map/GameMap.java (fragmento)

```
public class GameMap {
    private final TileType[][] grid;   // grid[fila][columna]
    private int x;                     // posición del grupo: estado extrínseco
    private int y;

```

```
    public TileType tileAt(int x, int y) {
        return grid[y][x];
    }

    public boolean move(int dx, int dy) {
        int nx = x + dx;
        int ny = y + dy;
        if (nx < 0 || ny < 0 || nx >= width() || ny >= height()
                || !grid[ny][nx].walkable()) {
            return false;
        }
        x = nx;
        y = ny;
        return true;
    }
}
```

El cliente. `MapParser` lee un texto y, para cada carácter, pide el tipo a la fábrica en lugar de crearlo: `grid[y][x] = types.get(c)`. Si el mapa tiene 84 casillas y cinco glifos distintos, termina con 84 referencias y `createdCount()` igual a 5.

5.9 Prototype, Factory y Flyweight: copiar, construir o compartir

Los tres patrones entregan objetos, y se confunden. Mira lo que hace cada uno:

	Prototype	**Factory**	**Flyweight**
Pregunta	¿Cómo obtengo otro como este, con un retoque?	¿Cómo construyo un objeto de este tipo desde cero?	¿Cómo evito tener mil copias de lo mismo?
Objeto nuevo	Sí, una copia por llamada	Sí, uno por llamada	Casi nunca: devuelve el mismo
Estado mutable	Sí, y cada copia tiene el suyo	Sí	No: el flyweight es inmutable
`==` entre dos resultados	`false`	`false`	`true` (mismo tipo)
En Emberdeep	`EnemyTemplate.copy()`	`CryptBiome.enemy(kind)`	`TileTypes.get(glyph)`

Criterio rápido: si lo que tienes es un *objeto ya montado* que quieres duplicar, es Prototype; si lo que tienes es un *tipo* y una receta, es Factory; si lo que tienes es *un dato que se repite y nunca cambia*, es Flyweight. Se combinan sin problema: `CryptBiome` es una Factory que usa Prototype por dentro. Practicas la decisión en el ejercicio 5.4.

5.10 Prototype y Flyweight: cuándo sí y cuándo no

Prototype compensa cuando el objeto es costoso o tedioso de montar y quieres variantes, cuando las variantes se definen como «lo mismo que X pero con esto distinto» (los élites) o cuando el tipo se decide en tiempo de ejecución con plantillas registradas. **No compensa** si el

objeto es un record inmutable (compartir el original ya es seguro, y para cambiar un dato basta un `with`), si tiene pocos campos (un constructor es más claro) o si no puedes decir qué campos son mutables: una copia mal hecha da errores silenciosos, como el de la sección anterior.

Flyweight compensa cuando hay muchísimos objetos con datos idénticos e inmutables y puedes separar el estado extrínseco sin esfuerzo. **No compensa** con pocos objetos, cuando el estado intrínseco cambia (no puedes compartir lo que muta), o cuando no has comprobado que el problema existe: sin medirlo, lo único seguro es que añades una fábrica, un mapa y la regla de que nadie use `new`. En Emberdeep, ya lo has visto, es una demostración. Para elegir con más calma entre los tres, el catálogo de [24] tiene una ficha de cada uno.

5.11 En el proyecto: H5 y H6 con Prototype y Flyweight

Este capítulo avanza dos hitos, cada uno con su patrón. Cambios del **H5** (Prototype):

- **Nuevos** `Skill`, `EnemyTemplate` y `TemplateRegistry` en `engine/character/`.
- **Cambia** `Enemy`: pasa a `Enemy(name, kind, stats, ai, skills)`. `Fighter`, `Combat` y `GameSession` no cambian: de momento nadie lee las habilidades.
- **Cambian** `CryptBiome` y `CavernBiome`: cada una tiene su `TemplateRegistry` y fabrica con `registry.get(kind).copy().toEnemy()`. La partida es la misma: las estadísticas, las IA y el `Random` no se tocan, así que las trazas de los capítulos anteriores se repiten. Los élites existen, pero ninguna planta los usa todavía.

Habilidades de cada tipo (datos del juego, un solo `Skill` por plantilla):

Tipo	Habilidad	Potencia	Bioma
`SKELETON`	Golpe de hueso	9	Cripta
`ASH_RAT`	Mordisco sucio	6	Cripta
`ZOMBIE`	Abrazo pútrido	7	Cripta
`GOBLIN`	Puñalada	8	Caverna
`BAT`	Chillido	5	Caverna
`SPIDER`	Picadura	9	Caverna

Cambios del **H6** (Flyweight):

- **Nuevo paquete** `engine/world/map/` con `TileType`, `TileTypes`, `GameMap`, `MapParser` y `MapData`. Hay un mapa fijo de 12 por 7 casillas por planta, escrito como bloque de texto en `MapData`, con `@` como posición inicial del grupo.
- **Cambia** `Floor`, que gana el texto del mapa: `Floor(biome, enemies, map)`. Es el paso que anunció el capítulo anterior: `FloorBuilder` gana `map(String)`, y `build()` avisa si una planta no tiene mapa («La planta N no tiene mapa»; se comprueba después del bioma y de los enemigos).
- **Cambia** `ConsoleUi`: nuevo método `explore`, que mueve al grupo con `w a s d`, avisa si choca con un muro, baja por la escalera y, con la tecla `t`, muestra cuántos `TileType` se han creado y comprueba con `==` que dos esquinas iguales son el mismo objeto.
- **Cambia** `Main`: con el segundo argumento `map` se explora el mapa desde la planta elegida en lugar de combatir. La lucha sigue empezando por planta, como antes.

Hitos 5 y 6: plantillas de enemigo y mapa con casillas compartidas

Tras estos hitos, cada enemigo sale de una plantilla copiada, existe el élite, y puedes recorrer un mapa de casillas compartidas. La estructura:

Estructura de carpetas tras el capítulo 5

```
emberdeep/
|-- settings.gradle
|-- build.gradle
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
`-- src/main/java/emberdeep/
    |-- app/
    |   |-- Main.java              (composition root)
    |   `-- GameConfig.java
    |-- engine/
    |   |-- character/             Stats, Hero, HeroBuilder, Enemy,
    ↪ EnemyKind, Fighter,
    |   |                          Skill, EnemyTemplate, TemplateRegistry
    |   |-- combat/
    |   |   |-- Combat.java
    |   |   |-- ai/                EnemyAi, FirstAliveAi, LowestHpAi,
    ↪ RandomTargetAi
    |   |   `-- damage/            DamageFormula, FlatFormula,
    ↪ ScaledFormula
    |   |-- item/                  Item, ItemKind, Inventory
    |   |-- session/               GameSession
    |   `-- world/
    |       |-- Dungeon.java, Floor.java, DungeonBuilder.java
    |       |-- biome/             BiomeFactory, CryptBiome, CavernBiome
    |       `-- map/               TileType, TileTypes, GameMap, MapParser,
    ↪  MapData
    `-- ui/
        `-- ConsoleUi.java
```

Lo practicas en el ejercicio 5.5. Antes, cuatro ejercicios más pequeños, en otros dominios.

5.12 Ejercicios

Ejercicio 5.1: El élite que contagia al original ★★☆☆☆

Este programa (no es de Emberdeep; dos archivos sin paquete) crea un lobo y su variante élite. Cada `MonsterTemplate` guarda un nombre, una vida y una lista de habilidades.

MonsterTemplate.java y Main.java

```java
import java.util.ArrayList;
import java.util.List;

public class MonsterTemplate {
    private String name;
    private int hp;
    private final List<String> abilities;

    public MonsterTemplate(String name, int hp, List<String> abilities) {
        this.name = name;
        this.hp = hp;
        this.abilities = abilities;
    }

    public MonsterTemplate copy() {
        return new MonsterTemplate(name, hp, abilities);
    }

    public MonsterTemplate asElite() {
        MonsterTemplate elite = copy();
        elite.name = "Élite " + name;
        elite.hp = hp * 2;
        elite.abilities.add("Furia");
        return elite;
    }

    public String name() { return name; }
    public List<String> abilities() { return abilities; }
}

public class Main {
    public static void main(String[] args) {
        MonsterTemplate wolf = new MonsterTemplate(
                "Lobo", 12, new ArrayList<>(List.of("Mordisco")));
        MonsterTemplate elite = wolf.asElite();
        System.out.println(wolf.name() + " " + wolf.abilities());
        System.out.println(elite.name() + " " + elite.abilities());
        System.out.println(wolf.abilities() == elite.abilities());
    }
}
```

1. Ejecútalo. ¿Qué imprime y qué debería imprimir?
2. Explica por qué el nombre del lobo no se contagió y la lista sí.
3. Corrige solo `MonsterTemplate`.

Está bien si, tras corregirlo, imprime

```
Lobo [Mordisco]
Élite Lobo [Mordisco, Furia]
false
```

y si en tu explicación aparecen las palabras «referencia» y «lista».

Pista: *Mira qué recibe el constructor en* `copy()`*: ¿la lista del original o una lista nueva?*

Depuración ▪ solución en la pág. 139

Ejercicio 5.2: Un SpellBook con copia profunda ★★☆☆☆

Vas a escribir un libro de hechizos que se copia sin compartir su lista. Archivos nuevos, sin paquete, no de Emberdeep. Pasos:

1. Crea el `record Spell(String name, int cost)`.
2. Crea la clase `SpellBook` con un campo `owner` (`String`, `final`) y una lista `spells` (`private final List<Spell>`, `ArrayList` vacía al crearse). Constructor público `SpellBook(String owner)`.
3. Añade `void add(Spell spell)`, `String owner()` y `List<Spell> spells()`, que devuelve `List.copyOf(spells)`.
4. Añade un constructor *privado* `SpellBook(SpellBook other)` que copie el dueño y *añada* cada hechizo de `other.spells` a la lista propia, y un método `SpellBook copy()` que lo use.
5. En `Main`, crea el libro de «Aria» con el hechizo «Bola de fuego» (coste 5), haz `copy()`, añade «Rayo» (coste 8) *solo a la copia* e imprime los tamaños y el dueño de la copia.

Está bien si imprime

```
Original: 1
Copia: 2
Dueña de la copia: Aria
```

y si los hechizos no se copian uno por uno (son un record: se comparten con seguridad).

Guiado ▪ solución en la pág. 140

Ejercicio 5.3: Un estilo de texto compartido ★★★☆☆

Un editor de texto guarda cada carácter con su estilo. Vas a compartir los estilos. Archivos nuevos, sin paquete, no de Emberdeep:

1. `record GlyphStyle(String font, int size, boolean bold)`: el flyweight.
2. `StyleFactory`, con `GlyphStyle get(String font, int size, boolean bold)` que usa `computeIfAbsent` y `int createdCount()`, que dice cuántos estilos creó.
3. `record PlacedChar(char value, int x, GlyphStyle style)`: el valor y la posición `x` son estado extrínseco; el estilo, intrínseco.
4. En `Main`, genera 10 000 `PlacedChar`. El de índice `i` tiene `value` igual a `(char) ('a' + i % 26)`, `x` igual a `i` y el estilo número `i % 4` de esta tabla: 0 = `mono`, 12, sin negrita; 1 = `mono`, 12, negrita; 2 = `serif`, 14, sin negrita; 3 = `serif`, 14, negrita.
5. Imprime el número de caracteres, `createdCount()` y el resultado de comparar con `==` los estilos de los índices 0 y 4, y de los índices 0 y 1.

Está bien si imprime

```
Caracteres: 10000
Estilos creados: 4
Índices 0 y 4, mismo objeto: true
Índices 0 y 1, mismo objeto: false
```

y si `GlyphStyle` no contiene la posición.

Pista: *Como clave del mapa de la fábrica puedes usar un* `String` *que una los tres datos, por ejemplo* `font + ":" + size + ":" + bold`*.*

Práctica ▪ solución en la pág. 142

Ejercicio 5.4: Copiar, construir o compartir ★★★☆☆

Para cada situación del juego, decide si usarías Factory, Prototype o Flyweight y justifícalo en una frase:

1. Crear el enemigo de un tipo dado, que en la cripta y en la caverna tiene datos distintos.
2. Un jefe montado con equipo y seis habilidades, del que quieres tres variantes que cambian solo la vida y una habilidad.
3. Las 10 000 casillas de suelo de un mapa grande, que comparten glifo y color.
4. El icono de cada tipo de objeto (un dato que no cambia), mostrado en cientos de huecos del inventario.
5. Una poción nueva cuya potencia se calcula a partir del número de planta.

Entrega una tabla con situación, patrón y la frase. Está bien si cada frase usa el verbo adecuado (construir desde cero, copiar y retocar, compartir lo que no cambia), si aparece al menos una situación de cada patrón y si ninguna justificación se reduce a «es más eficiente» sin decir en qué.

Diseño ▪ solución en la pág. 144

Ejercicio 5.5: Hitos 5 y 6: plantillas, élites y mapa ★★★★☆

Implementa los hitos H5 y H6 partiendo del estado del capítulo anterior (solución del ejercicio 4.5), en dos partes. Primero H5, y compruébalo antes de pasar a H6.

Parte 1 (H5): plantillas clonables.

1. En `engine/character/`: el record `Skill(String name, int power)`, y `Enemy(name, kind, stats, ai, skills)` con `List<Skill> skills` como último componente.
2. `EnemyTemplate`, clase mutable, con el constructor `(EnemyKind, String, Stats, EnemyAi)`, `addSkill(Skill)` (devuelve `this`), `setName`, `setStats`, los accesos `kind()`, `name()`, `stats()`, `ai()` y `skills()` (vista no modificable), y tres métodos: `copy()` (profunda: lista nueva; la IA se comparte), `toEnemy()` (con `List.copyOf`) y `asElite()` (nombre «Élite » más el nombre; vida por 3 y entre 2, *división entera*; ataque +2, defensa +1 y la habilidad `Skill("Furia", 12)` extra).
3. `TemplateRegistry` con `register(EnemyTemplate)` y `get(EnemyKind)`, que lanza `IllegalArgumentException` si no hay plantilla.
4. `CryptBiome` y `CavernBiome` montan su registro en el constructor (que sigue recibiendo el `Random`), con las habilidades de la tabla de la sección anterior, y fabrican con `registry.get(kind).copy().toEnemy()`. `item` no cambia.

Está bien si:

1. Sin argumentos y con el argumento `2`, el juego da las mismas trazas que al final del capítulo anterior.
2. Si añades temporalmente este fragmento al principio de `play` (con los imports de `Enemy`, `EnemyTemplate`, `Skill` y `FirstAliveAi`), la salida son cuatro líneas; bórralo después:

Comprobación temporal en Main.java

```
EnemyTemplate base = new EnemyTemplate(EnemyKind.ZOMBIE, "Zombi",
        new Stats(22, 5, 1), new FirstAliveAi())
        .addSkill(new Skill("Abrazo pútrido", 7));
EnemyTemplate elite = base.asElite();
System.out.println(base.skills().size() + " " + elite.skills().size())
    ↪ ;
System.out.println(elite.name() + " " + elite.stats());
Enemy enemy = elite.toEnemy();
System.out.println(enemy.kind() + " " + enemy.skills().size());
CryptBiome crypt = new CryptBiome(new Random(1));
Enemy a = crypt.enemy(EnemyKind.ZOMBIE);
Enemy b = crypt.enemy(EnemyKind.ZOMBIE);
System.out.println((a == b) + " " + (a.skills() == b.skills())
        + " " + a.skills().equals(b.skills()));
```

```
1 2
Élite Zombi Stats[maxHp=33, attack=7, defense=2]
ZOMBIE 2
false false true
```

Parte 2 (H6): mapa con casillas compartidas.

1. En `engine/world/map/`: `TileType` y `TileTypes` como en la sección del Flyweight (los cinco glifos de la sección).
2. `GameMap(TileType[][] grid, int startX, int startY)` con `width()`, `height()`, `tileAt(x, y)`, `partyX()`, `partyY()`, `partyTile()`, `onStairs()` (el glifo de la casilla del grupo es `>`), `move(dx, dy)` y `render()`. `move` devuelve `false` y no mueve si la casilla destino está fuera del mapa o no es transitable. `render()` devuelve las filas, cada una terminada en salto de línea, con `@` en la posición del grupo.
3. `MapParser(TileTypes)` con `parse(String)`: separa por líneas, trata `@` como la posición inicial (la casilla es suelo, `.`) y lanza `IllegalArgumentException` si una fila tiene otra longitud que la primera («Fila N con otra longitud») o si falta la `@` («Falta la posición inicial '@'»).
4. `MapData.floor(int)` devuelve el texto de cada una de las tres plantas, de 12 columnas por 7 filas, rodeado de muros, con una sola `@`, una sola `>` y los cinco glifos en cada mapa. El de la planta 1 es este (diseña tú los otros dos con las mismas reglas):

Mapa de la planta 1

```
############
#@...#....>#
#.##.#.##..#
#.#..+.#.^.#
#.#.##.#...#
#...#..+...#
############
```

5. `Floor` gana el componente `String map`; `FloorBuilder` gana `map(String)` (valor por defecto: vacío) y `DungeonBuilder.build()` lanza `IllegalStateException` con «La planta N no tiene mapa» si está vacío, después de comprobar el bioma y los enemigos. `Main` pasa `MapData.floor(n)` a cada planta.
6. `ConsoleUi.explore(GameMap, TileTypes)` devuelve `boolean`. En cada vuelta imprime el mapa, la línea «[w a s d] mover, [t] casillas, [q] salir» y el prompt `> `. Con `w`, `a`, `s` y `d` mueve (si no puede, imprime «No puedes pasar.»); con `t` imprime «Casillas: N, TileType creados: M» (N = ancho por alto del mapa) y «Esquinas opuestas, mismo objeto: B», donde B es el resultado de comparar con `==` las casillas `(0, 0)` y `(ancho-1, alto-1)`; con `q` devuelve `false`. Si tras una tecla el grupo está sobre la escalera, imprime «Bajas por la escalera.» y devuelve `true`.
7. En `Main`, con el segundo argumento `map`, se explora desde la planta del primer argumento: cada planta se parsea con el mismo `TileTypes`; si `explore` devuelve `true` y quedan plantas, se pasa a la siguiente; si no, termina. En ese modo no se pregunta la fórmula ni se combate.

Está bien si:

1. Los combates dan las mismas trazas que antes con los argumentos `1`, `2`, `3` y `1 default`.
2. Esta orden pulsa `a` (choca con el muro), `t` y `q`:

   Explorar la planta 1

   ```
   printf 'a\nt\nq\n' | ./gradlew -q run --args="1 map"
   ```

   y la salida contiene, en este orden (entre ellas se repite el mapa), estas líneas:

   Líneas esperadas

   ```
   > No puedes pasar.
   > Casillas: 84, TileType creados: 5
   Esquinas opuestas, mismo objeto: true
   ```

3. Esta orden recorre el camino hasta la escalera y sale en la planta siguiente:

   Bajar a la planta 2

   ```
   printf '%s\n' d d d s s d d w w d d d d q | ./gradlew -q run --args="1
       ↪ map"
   ```

   Aparece «Bajas por la escalera.» y justo después el mapa de la planta 2.

4. Si añades temporalmente este fragmento al principio de `play` (con los imports de las clases de `engine.world.map`), imprime `252 casillas, 5 TileType`:

Comprobación temporal en Main.java

```
TileTypes types = new TileTypes();
MapParser parser = new MapParser(types);
int cells = 0;
for (int number = 1; number <= 3; number++) {
    GameMap map = parser.parse(MapData.floor(number));
    cells += map.width() * map.height();
}
System.out.println(cells + " casillas, " + types.createdCount() + "
    ↪ TileType");
```

5. Un texto sin `@` y un texto con una fila más corta lanzan `IllegalArgumentException`.

Pista: *Para trocear el texto del mapa:* `text.strip().split("\n")`*. Un* `TileType[][]` *se indexa* `grid[fila][columna]`*, es decir,* `grid[y][x]`*.*

Proyecto ▪ solución en la pág. 144

5.13 Resumen

Resumen

- El olor de Prototype son objetos casi iguales montados repitiendo datos. Prototype copia un objeto ya montado y retoca la copia.
- Una copia profunda no copia todo: duplica lo mutable (la lista de habilidades), comparte lo inmutable (records, `String`) y lo que debe ser compartido (la IA con su `Random`). Una copia superficial comparte la lista y el élite contagia al original.
- `Cloneable` no se usa: `Object.clone()` es superficial y engorroso. Se prefiere un constructor o un método de copia (`copy()`).
- Un registro de prototipos entrega una plantilla por tipo; el cliente copia y nunca modifica el prototipo registrado. Prototype convive con Factory: `CryptBiome` copia por dentro.
- Flyweight comparte los objetos idénticos e inmutables: el estado intrínseco (glifo, color, transitable) vive en el flyweight; el extrínseco (la posición) lo aporta el contexto.
- Una fábrica de flyweights con `Map.computeIfAbsent` devuelve siempre la misma instancia por clave; se demuestra con `==` y con un contador (`createdCount()`), no con rendimiento.
- Copiar (Prototype), construir (Factory) o compartir (Flyweight): objeto montado, tipo y receta, o dato que se repite.
- H5 y H6: `Skill`, `EnemyTemplate`, `TemplateRegistry`, `Enemy` con `skills`, `TileType`, `TileTypes`, `GameMap`, `MapParser`, `MapData` y el modo `map`.

Soluciones de los ejercicios

Solución del ejercicio 5.1

enunciado en la pág. 132

1. Imprime `Lobo [Mordisco, Furia]`, `Élite Lobo [Mordisco, Furia]` y `true`. Debería imprimir `Lobo [Mordisco]`, `Élite Lobo [Mordisco, Furia]` y `false`.
2. En `copy()`, el constructor recibe `abilities`, es decir, la *referencia* a la lista del original. La copia y el original apuntan a la misma lista, y `elite.abilities.add("Furia")` la cambia para los dos; la última línea (`==` entre las dos listas) lo delata con `true`. El nombre, en cambio, es un `String` inmutable: `elite.name = "Élite " + name` asigna otro objeto a la copia y el original conserva el suyo.
3. La corrección es crear una lista nueva en `copy()`:

MonsterTemplate.java (corregido)

```
import java.util.ArrayList;
import java.util.List;

public class MonsterTemplate {
    private String name;
    private int hp;
    private final List<String> abilities;

    public MonsterTemplate(String name, int hp, List<String> abilities) {
        this.name = name;
        this.hp = hp;
        this.abilities = abilities;
    }

    public MonsterTemplate copy() {
        return new MonsterTemplate(name, hp, new ArrayList<>(abilities));
    }

    public MonsterTemplate asElite() {
        MonsterTemplate elite = copy();
        elite.name = "Élite " + name;
        elite.hp = hp * 2;
        elite.abilities.add("Furia");
        return elite;
    }

    public String name() { return name; }
    public List<String> abilities() { return abilities; }
}
```

Main.java (el mismo del enunciado)

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        MonsterTemplate wolf = new MonsterTemplate(
                "Lobo", 12, new ArrayList<>(List.of("Mordisco")));
        MonsterTemplate elite = wolf.asElite();
        System.out.println(wolf.name() + " " + wolf.abilities());
        System.out.println(elite.name() + " " + elite.abilities());
        System.out.println(wolf.abilities() == elite.abilities());
    }
}
```

Por qué funciona. `new ArrayList<>(abilities)` crea una lista distinta con los mismos elementos. Como los elementos son `String` (inmutables), no hace falta copiarlos: compartirlos es seguro. La copia profunda es «copiar lo mutable»; aquí lo único mutable es la lista.
Errores típicos:

- Arreglarlo en `asElite()` (por ejemplo, creando allí otra lista): el fallo está en `copy()`, y cualquier otro cliente de `copy()` seguiría contagiando.
- Pensar que el nombre también estaba compartido: lo que se comparte son las referencias, pero un `String` no se puede cambiar por dentro.
- Comprobar el arreglo solo con el tamaño de la lista: la prueba de identidad (`==`) es más directa.

Solución del ejercicio 5.2

enunciado en la pág. 134

Spell.java

```
public record Spell(String name, int cost) {}
```

SpellBook.java

```java
import java.util.ArrayList;
import java.util.List;

public class SpellBook {
    private final String owner;
    private final List<Spell> spells = new ArrayList<>();

    public SpellBook(String owner) {
        this.owner = owner;
    }

    private SpellBook(SpellBook other) {
        this.owner = other.owner;
        this.spells.addAll(other.spells);
    }

    public SpellBook copy() {
        return new SpellBook(this);
    }

    public void add(Spell spell) {
        spells.add(spell);
    }

    public String owner() { return owner; }

    public List<Spell> spells() {
        return List.copyOf(spells);
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        SpellBook book = new SpellBook("Aria");
        book.add(new Spell("Bola de fuego", 5));
        SpellBook copy = book.copy();
        copy.add(new Spell("Rayo", 8));
        System.out.println("Original: " + book.spells().size());
        System.out.println("Copia: " + copy.spells().size());
        System.out.println("Dueña de la copia: " + copy.owner());
    }
}
```

Por qué funciona. El constructor de copia crea su propia lista (por la inicialización del campo) y le añade los hechizos del otro libro: dos listas, los mismos elementos. Añadir «Rayo» a la copia no toca la lista del original. `Spell` es un record, inmutable, así que compartir sus instancias es seguro, y `owner` es un `String`, también inmutable.

Errores típicos:

- Escribir `this.spells = other.spells`: no compila si el campo es `final`; si quitas el `final` para que compile, vuelves a compartir la lista (el bug de la sección del capítulo).

- Devolver la lista interna en `spells()`: quien la reciba podría modificar el libro sin pasar por `add`. Por eso devuelve una copia inmutable.
- Copiar cada `Spell` con `new Spell(...)`: es innecesario con un record.

Solución del ejercicio 5.3

enunciado en la pág. 134

GlyphStyle.java

```java
public record GlyphStyle(String font, int size, boolean bold) {}
```

PlacedChar.java

```java
public record PlacedChar(char value, int x, GlyphStyle style) {}
```

StyleFactory.java

```java
import java.util.HashMap;
import java.util.Map;

public class StyleFactory {
    private final Map<String, GlyphStyle> cache = new HashMap<>();
    private int created = 0;

    public GlyphStyle get(String font, int size, boolean bold) {
        String key = font + ":" + size + ":" + bold;
        return cache.computeIfAbsent(key, k -> {
            created++;
            return new GlyphStyle(font, size, bold);
        });
    }

    public int createdCount() {
        return created;
    }
}
```

Main.java

```java
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        StyleFactory styles = new StyleFactory();
        String[] fonts = {"mono", "mono", "serif", "serif"};
        int[] sizes = {12, 12, 14, 14};
        boolean[] bolds = {false, true, false, true};

        List<PlacedChar> chars = new ArrayList<>();
        for (int i = 0; i < 10_000; i++) {
            int s = i % 4;
            GlyphStyle style = styles.get(fonts[s], sizes[s], bolds[s]);
            chars.add(new PlacedChar((char) ('a' + i % 26), i, style));
        }
        System.out.println("Caracteres: " + chars.size());
        System.out.println("Estilos creados: " + styles.createdCount());
        System.out.println("Índices 0 y 4, mismo objeto: "
                + (chars.get(0).style() == chars.get(4).style()));
        System.out.println("Índices 0 y 1, mismo objeto: "
                + (chars.get(0).style() == chars.get(1).style()));
    }
}
```

Por qué funciona. Los 10 000 caracteres son 10 000 objetos `PlacedChar` distintos (cada uno tiene su valor y su posición), pero el estilo es una referencia a uno de cuatro `GlyphStyle`. `computeIfAbsent` solo llama a la función cuando la clave es nueva, así que `created` sube cuatro veces. Los índices 0 y 4 piden el mismo estilo (`mono`, 12, sin negrita) y comparten objeto; el 0 y el 1 difieren en la negrita. La lambda modifica `created`, que es un campo, y por eso puede hacerlo (una variable local no se podría modificar desde una lambda).

Errores típicos:

- Poner la posición `x` dentro de `GlyphStyle`: ya no habría cuatro estilos, sino diez mil.
- Dejar fuera la negrita de la clave: los estilos 0 y 1 se fusionan y ya no son cuatro.
- Crear estilos con `new GlyphStyle(...)` en `Main`: se salta la fábrica y el `==` deja de ser fiable.

Solución del ejercicio 5.4 enunciado en la pág. 135

N.º	Patrón	Justificación
1	Factory	Se *construye desde cero* a partir de un tipo y de una receta que depende del bioma: eso es una fábrica (Abstract Factory, de hecho).
2	Prototype	Ya hay un objeto *montado*; las variantes son «lo mismo con esto distinto», y se resuelven *copiando y retocando* en lugar de repetir los datos.
3	Flyweight	Hay muchísimas casillas con datos idénticos e inmutables, y se *comparte* una sola instancia; la posición queda fuera, como estado extrínseco.
4	Flyweight	Es un dato que no cambia y se repite en cientos de usos: se *comparte* el icono de cada tipo y cada hueco solo guarda la referencia.
5	Factory	No hay nada montado que copiar: la poción se *construye desde cero* con un valor calculado (la planta), lo que es una fábrica o un método fábrica.

Hay dos matices que se valoran. La situación 2 podría resolverse con una fábrica si las variantes fueran pocas y los datos pocos; lo que inclina hacia Prototype es que el jefe es costoso de montar y las variantes son retoques. La situación 4 solo es Flyweight si el icono es inmutable; si cada hueco pudiera cambiar el suyo, habría que copiarlo o separar lo que cambia. Una respuesta es válida aunque difiera en un caso dudoso si lo justifica con el criterio de montar, copiar o compartir.

Errores típicos:

- Justificar con «es más eficiente» sin decir en qué: no se ha medido nada.
- Elegir Prototype para la poción de la situación 5 porque «hay muchas»: sin un objeto montado que copiar, es una fábrica.
- Elegir Flyweight cuando el dato cambia por uso (por ejemplo, la vida de cada enemigo): lo mutable no se comparte.

Solución del ejercicio 5.5 enunciado en la pág. 135

La solución sigue las dos partes. `Stats`, `Hero`, `HeroBuilder`, `Fighter`, `Combat`, `GameSession`, `GameConfig`, el paquete `ai` y el paquete `damage` no cambian. Aquí aparecen enteros todos los archivos que cambian o se crean. La parte 1 trae seis archivos (`Skill`, `Enemy`, `EnemyTemplate`, `TemplateRegistry`, `CryptBiome` y `CavernBiome`) y la parte 2, nueve (`Floor`, `DungeonBuilder`, `TileType`, `TileTypes`, `GameMap`, `MapParser`, `MapData`, `ConsoleUi` y `Main`).

Parte 1 (H5). engine/character/

Skill.java

```
1 package emberdeep.engine.character;
2
3 public record Skill(String name, int power) {}
```

Enemy.java

```java
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;
import java.util.List;

public record Enemy(String name, EnemyKind kind, Stats stats, EnemyAi ai,
                    List<Skill> skills) {}
```

EnemyTemplate.java (1/2: datos y copia)

```java
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class EnemyTemplate {
    private final EnemyKind kind;
    private String name;
    private Stats stats;
    private final List<Skill> skills;
    private final EnemyAi ai;

    public EnemyTemplate(EnemyKind kind, String name, Stats stats, EnemyAi
    ↪ ai) {
        this(kind, name, stats, new ArrayList<>(), ai);
    }

    private EnemyTemplate(EnemyKind kind, String name, Stats stats,
                          List<Skill> skills, EnemyAi ai) {
        this.kind = kind;
        this.name = name;
        this.stats = stats;
        this.skills = skills;
        this.ai = ai;
    }

    public EnemyTemplate copy() {
        return new EnemyTemplate(kind, name, stats, new ArrayList<>(skills),
    ↪  ai);
    }

    public Enemy toEnemy() {
        return new Enemy(name, kind, stats, ai, List.copyOf(skills));
    }
```

EnemyTemplate.java (2/2: élite y accesos)

```

    public EnemyTemplate asElite() {
        EnemyTemplate elite = copy();
        elite.setName("Élite " + name);
        elite.setStats(new Stats(stats.maxHp() * 3 / 2,
                stats.attack() + 2, stats.defense() + 1));
        elite.addSkill(new Skill("Furia", 12));
        return elite;
    }

    public EnemyTemplate addSkill(Skill skill) {
        skills.add(skill);
        return this;
    }

    public void setName(String name) { this.name = name; }
    public void setStats(Stats stats) { this.stats = stats; }

    public EnemyKind kind() { return kind; }
    public String name() { return name; }
    public Stats stats() { return stats; }
    public EnemyAi ai() { return ai; }

    public List<Skill> skills() {
        return Collections.unmodifiableList(skills);
    }
}
```

TemplateRegistry.java

```
package emberdeep.engine.character;

import java.util.EnumMap;
import java.util.Map;

public class TemplateRegistry {
    private final Map<EnemyKind, EnemyTemplate> templates =
            new EnumMap<>(EnemyKind.class);

    public void register(EnemyTemplate template) {
        templates.put(template.kind(), template);
    }

    public EnemyTemplate get(EnemyKind kind) {
        EnemyTemplate template = templates.get(kind);
        if (template == null) {
            throw new IllegalArgumentException("No hay plantilla para " +
    ↪ kind);
        }
        return template;
    }
}
```

engine/world/biome/CryptBiome.java (1/2: registro)

```java
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.EnemyTemplate;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Stats;
import emberdeep.engine.character.TemplateRegistry;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import java.util.Random;

public class CryptBiome implements BiomeFactory {
    private final TemplateRegistry registry = new TemplateRegistry();

    public CryptBiome(Random random) {
        registry.register(new EnemyTemplate(EnemyKind.SKELETON, "Esqueleto",
    ↪
                new Stats(18, 7, 2), new RandomTargetAi(random))
                .addSkill(new Skill("Golpe de hueso", 9)));
        registry.register(new EnemyTemplate(EnemyKind.ASH_RAT, "Rata de
    ↪ ceniza",
                new Stats(8, 5, 0), new LowestHpAi())
                .addSkill(new Skill("Mordisco sucio", 6)));
        registry.register(new EnemyTemplate(EnemyKind.ZOMBIE, "Zombi",
                new Stats(22, 5, 1), new FirstAliveAi())
                .addSkill(new Skill("Abrazo pútrido", 7)));
    }
```

CryptBiome.java (2/2: enemy e item)

```java

    @Override
    public Enemy enemy(EnemyKind kind) {
        return registry.get(kind).copy().toEnemy();
    }

    @Override
    public Item item(ItemKind kind) {
        return switch (kind) {
            case HEALING -> new Item("Poción de cripta", kind, 12);
            case BOMB -> new Item("Bomba de ceniza", kind, 10);
        };
    }
}
```

engine/world/biome/CavernBiome.java (1/2: registro)

```
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.EnemyTemplate;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Stats;
import emberdeep.engine.character.TemplateRegistry;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import java.util.Random;

public class CavernBiome implements BiomeFactory {
    private final TemplateRegistry registry = new TemplateRegistry();

    public CavernBiome(Random random) {
        registry.register(new EnemyTemplate(EnemyKind.GOBLIN, "Goblin",
                new Stats(14, 6, 1), new FirstAliveAi())
                .addSkill(new Skill("Puñalada", 8)));
        registry.register(new EnemyTemplate(EnemyKind.BAT, "Murciélago",
                new Stats(10, 6, 0), new LowestHpAi())
                .addSkill(new Skill("Chillido", 5)));
        registry.register(new EnemyTemplate(EnemyKind.SPIDER, "Araña",
                new Stats(16, 7, 2), new RandomTargetAi(random))
                .addSkill(new Skill("Picadura", 9)));
    }
```

CavernBiome.java (2/2: enemy e item)

```

    @Override
    public Enemy enemy(EnemyKind kind) {
        return registry.get(kind).copy().toEnemy();
    }

    @Override
    public Item item(ItemKind kind) {
        return switch (kind) {
            case HEALING -> new Item("Poción de caverna", kind, 10);
            case BOMB -> new Item("Bomba de roca", kind, 14);
        };
    }
}
```

Por qué funciona (parte 1). Cada bioma guarda sus plantillas y entrega, por cada `enemy(kind)`, un `Enemy` nuevo con su propia lista de habilidades. El `switch` sobre los seis tipos ha desaparecido: pedir un tipo ajeno a la cripta cae en `registry.get`, que lanza `IllegalArgumentException` (el mensaje cambia, el tipo de excepción no). Las estadísticas, las IA y el `Random` son los de siempre, y el registro se monta en el constructor sin consumir

números aleatorios, así que las trazas son idénticas a las del capítulo anterior. En el fragmento de comprobación, el élite tiene 2 habilidades y el zombi base sigue teniendo 1; los dos enemigos del mismo tipo no comparten lista (`false`) pero sus listas son iguales (`true`).

Errores típicos (parte 1):

- Convertir `registry.get(kind)` en enemigo sin `copy()`: dos enemigos del mismo tipo compartirían la lista de la plantilla.
- Llamar a `addSkill` sobre el resultado de `get`: cambias el prototipo y todos los enemigos futuros de ese tipo ganan la habilidad.
- Duplicar la IA en `copy()` con `new RandomTargetAi(...)`: rompe la semilla compartida.
- Calcular la vida del élite con `maxHp() * 1.5`: no compila (resultado `double`); se usa `* 3 / 2`.

Parte 2 (H6). engine/world/ (cambios de H4)

Floor.java

```
package emberdeep.engine.world;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.List;

public record Floor(BiomeFactory biome, List<EnemyKind> enemies, String map)
    ↪ {
    public Floor {
        enemies = List.copyOf(enemies);
    }
}
```

DungeonBuilder.java (1/2: la mazmorra)

```java
package emberdeep.engine.world;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.ArrayList;
import java.util.List;

public class DungeonBuilder {
    private final List<Floor> floors = new ArrayList<>();

    public FloorBuilder floor() {
        return new FloorBuilder();
    }

    public Dungeon build() {
        if (floors.isEmpty()) {
            throw new IllegalStateException("La mazmorra necesita al menos una planta");
        }
        for (int i = 0; i < floors.size(); i++) {
            Floor floor = floors.get(i);
            String label = "La planta " + (i + 1);
            if (floor.biome() == null) {
                throw new IllegalStateException(label + " no tiene bioma");
            }
            if (floor.enemies().isEmpty()) {
                throw new IllegalStateException(label + " no tiene enemigos");
            }
            if (floor.map() == null || floor.map().isBlank()) {
                throw new IllegalStateException(label + " no tiene mapa");
            }
        }
        return new Dungeon(floors);
    }
```

DungeonBuilder.java (2/2: FloorBuilder)

```

    public class FloorBuilder {
        private BiomeFactory biome;
        private List<EnemyKind> enemies = List.of();
        private String map = "";

        public FloorBuilder biome(BiomeFactory biome) {
            this.biome = biome;
            return this;
        }

        public FloorBuilder enemies(List<EnemyKind> enemies) {
            this.enemies = enemies;
            return this;
        }

        public FloorBuilder map(String map) {
            this.map = map;
            return this;
        }

        public DungeonBuilder done() {
            floors.add(new Floor(biome, enemies, map));
            return DungeonBuilder.this;
        }
    }
}
```

engine/world/map/

TileType.java

```
package emberdeep.engine.world.map;

public record TileType(char glyph, String color, boolean walkable) {}
```

TileTypes.java

```java
package emberdeep.engine.world.map;

import java.util.HashMap;
import java.util.Map;

public class TileTypes {
    private final Map<Character, TileType> cache = new HashMap<>();
    private int created = 0;

    public TileType get(char glyph) {
        return cache.computeIfAbsent(glyph, this::create);
    }

    public int createdCount() {
        return created;
    }

    private TileType create(char glyph) {
        TileType type = switch (glyph) {
            case '#' -> new TileType('#', "gray", false);
            case '.' -> new TileType('.', "white", true);
            case '+' -> new TileType('+', "brown", true);
            case '>' -> new TileType('>', "cyan", true);
            case '^' -> new TileType('^', "red", true);
            default -> throw new IllegalArgumentException(
                    "Casilla desconocida: " + glyph);
        };
        created++;
        return type;
    }
}
```

GameMap.java

```
package emberdeep.engine.world.map;

public class GameMap {
    private final TileType[][] grid;
    private int x;
    private int y;

    public GameMap(TileType[][] grid, int startX, int startY) {
        this.grid = grid;
        this.x = startX;
        this.y = startY;
    }

    public int width() { return grid[0].length; }
    public int height() { return grid.length; }
    public TileType tileAt(int x, int y) { return grid[y][x]; }
    public int partyX() { return x; }
    public int partyY() { return y; }
    public TileType partyTile() { return grid[y][x]; }
    public boolean onStairs() { return partyTile().glyph() == '>'; }

    public boolean move(int dx, int dy) {
        int nx = x + dx;
        int ny = y + dy;
        if (nx < 0 || ny < 0 || nx >= width() || ny >= height()
                || !grid[ny][nx].walkable()) {
            return false;
        }
        x = nx;
        y = ny;
        return true;
    }

    public String render() {
        StringBuilder text = new StringBuilder();
        for (int row = 0; row < height(); row++) {
            for (int col = 0; col < width(); col++) {
                boolean party = row == y && col == x;
                text.append(party ? '@' : grid[row][col].glyph());
            }
            text.append('\n');
        }
        return text.toString();
    }
}
```

MapParser.java

```java
package emberdeep.engine.world.map;

public class MapParser {
    private final TileTypes types;

    public MapParser(TileTypes types) {
        this.types = types;
    }

    public GameMap parse(String text) {
        String[] rows = text.strip().split("\n");
        int width = rows[0].length();
        TileType[][] grid = new TileType[rows.length][width];
        int startX = -1;
        int startY = -1;
        for (int y = 0; y < rows.length; y++) {
            if (rows[y].length() != width) {
                throw new IllegalArgumentException(
                        "Fila " + (y + 1) + " con otra longitud");
            }
            for (int x = 0; x < width; x++) {
                char glyph = rows[y].charAt(x);
                if (glyph == '@') {
                    startX = x;
                    startY = y;
                    glyph = '.';
                }
                grid[y][x] = types.get(glyph);
            }
        }
        if (startX < 0) {
            throw new IllegalArgumentException("Falta la posición inicial '
    ↪ @'");
        }
        return new GameMap(grid, startX, startY);
    }
}
```

MapData.java

```
package emberdeep.engine.world.map;

public final class MapData {
    private static final String FLOOR_1 = """
            #############
            #@...#....>#
            #.##.#.##..#
            #.#..+.#.^.#
            #.#.##.#...#
            #...#..+...#
            #############
            """;
    private static final String FLOOR_2 = """
            #############
            #>..#......#
            #.#.#.####.#
            ###...#..#.#
            #.####.^.+.#
            #@.......#.#
            #############
            """;
    private static final String FLOOR_3 = """
            #############
            #@.#.....>.#
            #..#.###.#.#
            #.##.#^..#.#
            #....#.+##.#
            #.####...^.#
            #############
            """;

    private MapData() {}

    public static String floor(int number) {
        return switch (number) {
            case 1 -> FLOOR_1;
            case 2 -> FLOOR_2;
            case 3 -> FLOOR_3;
            default -> throw new IllegalArgumentException(
                    "No hay mapa para la planta " + number);
        };
    }
}
```

ui/ConsoleUi.java (1/4: imports y fórmula)

```java
package emberdeep.ui;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.Item;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.TileType;
import emberdeep.engine.world.map.TileTypes;
import java.io.PrintStream;
import java.util.List;
import java.util.Scanner;
import java.util.stream.Collectors;

public class ConsoleUi {
    private final Scanner in;
    private final PrintStream out;

    public ConsoleUi(Scanner in, PrintStream out) {
        this.in = in;
        this.out = out;
    }

    public int askFormula() {
        out.println("Fórmula de daño:");
        out.println("  [1] plana: ataque - defensa");
        out.println("  [2] escalada: ataque x 100 / (100 + defensa)");
        return ask(2);
    }
```

ConsoleUi.java (2/4: la partida)

```
    public void run(GameSession session) {
        Combat combat = session.combat();
        out.println("=== Emberdeep ===");
        while (!combat.isOver()) {
            out.println("--- Ronda " + session.round() + " ---");
            for (Fighter hero : session.heroes()) {
                if (!hero.isAlive() || combat.isOver()) {
                    continue;
                }
                out.println("Héroes:   " + describe(session.heroes()));
                out.println("Enemigos: " + describe(session.enemies()));
                out.println("Objetos:  " + describeItems(session.inventory()
    ↪ ));
                out.println("Turno de " + hero.name() + ". [1] atacar");
                ask(1);
                Fighter target = askTarget(session.enemies());
                print(combat.heroAttack(hero, target));
            }
            if (!combat.isOver()) {
                print(combat.enemyPhase());
            }
        }
        out.println(combat.heroesWon() ? "¡Victoria!" : "Derrota...");
    }
```

ConsoleUi.java (3/4: modo mapa)

```
    public boolean explore(GameMap map, TileTypes types) {
        while (true) {
            out.print(map.render());
            out.println("[w a s d] mover, [t] casillas, [q] salir");
            out.print("> ");
            out.flush();
            String key = in.nextLine().trim();
            switch (key) {
                case "w" -> step(map, 0, -1);
                case "a" -> step(map, -1, 0);
                case "s" -> step(map, 0, 1);
                case "d" -> step(map, 1, 0);
                case "t" -> showTiles(map, types);
                case "q" -> { return false; }
                default -> out.println("Tecla no válida.");
            }
            if (map.onStairs()) {
                out.println("Bajas por la escalera.");
                return true;
            }
        }
    }

    private void step(GameMap map, int dx, int dy) {
        if (!map.move(dx, dy)) {
            out.println("No puedes pasar.");
        }
    }

    private void showTiles(GameMap map, TileTypes types) {
        TileType corner = map.tileAt(0, 0);
        TileType opposite = map.tileAt(map.width() - 1, map.height() - 1);
        out.println("Casillas: " + map.width() * map.height()
                + ", TileType creados: " + types.createdCount());
        out.println("Esquinas opuestas, mismo objeto: " + (corner ==
    ↪ opposite));
    }
```

ConsoleUi.java (4/4: auxiliares y llave final)

```
    private String describe(List<Fighter> group) {
        return group.stream()
                .map(f -> f.name() + " " + f.hp() + "/" + f.stats().maxHp())
                .collect(Collectors.joining(", "));
    }

    private String describeItems(Inventory inventory) {
        if (inventory.items().isEmpty()) {
            return "(ninguno)";
        }
        return inventory.items().stream()
                .map(Item::name)
                .collect(Collectors.joining(", "));
    }

    private void print(List<String> messages) {
        messages.forEach(out::println);
    }

    private int ask(int max) {
        while (true) {
            out.print("> ");
            out.flush();
            String line = in.nextLine().trim();
            try {
                int n = Integer.parseInt(line);
                if (n >= 1 && n <= max) {
                    return n;
                }
            } catch (NumberFormatException e) {
                // no es un número: se avisa más abajo
            }
            out.println("Escribe un número entre 1 y " + max + ".");
        }
    }

    private Fighter askTarget(List<Fighter> enemies) {
        List<Fighter> alive = enemies.stream().filter(Fighter::isAlive).toList();
        StringBuilder menu = new StringBuilder("Objetivo:");
        for (int i = 0; i < alive.size(); i++) {
            menu.append("  [").append(i + 1).append("] ").append(alive.get(i).name());
        }
        out.println(menu);
        return alive.get(ask(alive.size()) - 1);
    }
}
```

app/Main.java (1/3: imports y arranque)

```java
package emberdeep.app;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.Dungeon;
import emberdeep.engine.world.DungeonBuilder;
import emberdeep.engine.world.Floor;
import emberdeep.engine.world.biome.BiomeFactory;
import emberdeep.engine.world.biome.CavernBiome;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.engine.world.map.MapData;
import emberdeep.engine.world.map.MapParser;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.ConsoleUi;
import java.util.List;
import java.util.Random;
import java.util.Scanner;

public class Main {
    private final GameConfig config;

    private Main(GameConfig config) {
        this.config = config;
    }

    public static void main(String[] args) {
        GameConfig config = new GameConfig(42L, 3, new FlatFormula());
        new Main(config).play(args);
    }
```

Main.java (2/3: la partida y el modo mapa)

```

    private void play(String[] args) {
        int floorNumber = chooseFloor(args);
        ConsoleUi ui = new ConsoleUi(new Scanner(System.in), System.out);
        Random random = new Random(config.seed());

        BiomeFactory crypt = new CryptBiome(random);
        BiomeFactory cavern = new CavernBiome(random);
        List<EnemyKind> cryptKinds = List.of(
                EnemyKind.SKELETON, EnemyKind.ASH_RAT, EnemyKind.ZOMBIE);
        List<EnemyKind> cavernKinds = List.of(
                EnemyKind.GOBLIN, EnemyKind.BAT, EnemyKind.SPIDER);

        Dungeon dungeon = new DungeonBuilder()
                .floor().biome(crypt).enemies(cryptKinds)
                        .map(MapData.floor(1)).done()
                .floor().biome(cavern).enemies(cavernKinds)
                        .map(MapData.floor(2)).done()
                .floor().biome(crypt).enemies(cryptKinds)
                        .map(MapData.floor(3)).done()
                .build();

        if (args.length > 1 && args[1].equals("map")) {
            explore(ui, dungeon, floorNumber);
            return;
        }
        DamageFormula formula = chooseFormula(ui, args);
        Floor floor = dungeon.floors().get(floorNumber - 1);
        GameSession session = new GameSession(
                heroes(), floor.biome(), floor.enemies(), formula);
        ui.run(session);
    }

    private void explore(ConsoleUi ui, Dungeon dungeon, int firstFloor) {
        TileTypes types = new TileTypes();
        MapParser parser = new MapParser(types);
        int number = firstFloor;
        while (true) {
            Floor floor = dungeon.floors().get(number - 1);
            boolean descended = ui.explore(parser.parse(floor.map()), types)
    ↪ ;
            if (!descended || number == dungeon.floors().size()) {
                return;
            }
            number++;
        }
    }
```

Main.java (3/3: auxiliares y llave final)

```
    private DamageFormula chooseFormula(ConsoleUi ui, String[] args) {
        if (args.length > 1 && args[1].equals("default")) {
            return config.defaultFormula();
        }
        return ui.askFormula() == 1 ? new FlatFormula() : new ScaledFormula
    ↪ ();
    }

    private int chooseFloor(String[] args) {
        int floor = args.length > 0 ? Integer.parseInt(args[0]) : 1;
        if (floor < 1 || floor > config.floors()) {
            throw new IllegalArgumentException(
                    "La planta debe estar entre 1 y " + config.floors());
        }
        return floor;
    }

    private List<Hero> heroes() {
        return List.of(
                new HeroBuilder().name("Aria")
                        .stats(new Stats(30, 8, 3)).build(),
                new HeroBuilder().name("Borin")
                        .stats(new Stats(40, 6, 5)).build(),
                new HeroBuilder().name("Cael")
                        .stats(new Stats(24, 9, 2)).build());
    }
}
```

Por qué funciona (parte 2). `TileTypes` reparte una sola instancia por glifo: el mapa de la planta 1 tiene 84 casillas y sus cinco glifos distintos (el `@` se sustituye por suelo) crean 5 `TileType`; las otras plantas piden los mismos y no crean más, por eso el fragmento de comprobación da `252 casillas, 5 TileType`. Las dos esquinas del mapa son muros, así que `tileAt(0, 0) == tileAt(ancho-1, alto-1)` es `true`: es el mismo objeto, no dos iguales. La posición del grupo está en `GameMap` (estado extrínseco); ninguna casilla sabe dónde está. `render()` devuelve el texto y es `ConsoleUi` quien lo imprime, así que el motor sigue sin usar `System.out`. `Main` construye la mazmorra antes de decidir si combate o explora; esto no consume números aleatorios ni imprime nada, y la pregunta de la fórmula (que sí imprime) sigue yendo antes de la primera ronda, por lo que las trazas de combate no cambian. Como `DungeonBuilder` comprueba el mapa *después* del bioma y de los enemigos, la comprobación del capítulo anterior (planta sin enemigos) sigue dando «La planta 1 no tiene enemigos».

Errores típicos (parte 2):

- Poner la posición del grupo dentro de `TileType` o crear un `TileType` por carácter: pierdes la compartición y `createdCount()` sube con el tamaño del mapa.
- Dejar `@` como un sexto glifo: `TileTypes` lanzaría `IllegalArgumentException`. El parser lo trata como posición inicial sobre suelo.
- Hacer `created++` antes del `switch` en `create`: un glifo desconocido contaría aunque no se guarde nada.

- Modificar `cache` dentro de la función de `computeIfAbsent`: el método no lo permite.
- Escribir el mapa con una fila de otra longitud: el parser avisa con «Fila N con otra longitud».
- Olvidar la `q` al final de la orden con `printf`: al acabarse la entrada, `nextLine()` lanza `NoSuchElementException`.

Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo siguiente.

Parte III

Ampliar el comportamiento

CAPÍTULO 6

Decorator y Chain of Responsibility: apilar o encadenar

Objetivos del capítulo

- Reconocer el olor que pide un Decorator: una subclase por cada combinación de equipo y de efectos.
- Aplicar Decorator con una interfaz común, un decorador abstracto y decoradores inmutables que se apilan, y separar el equipo permanente (`gear`) de la cadena actual (`stats`).
- Reconocer el olor que pide una Chain of Responsibility: un método con una fila de `if` que modifican o cortan un valor.
- Aplicar Chain of Responsibility con eslabones que modifican o cortan, y razonar el orden en que se montan.
- Distinguir Decorator, Chain of Responsibility y Strategy, y dar los hitos H7 y H8: equipo y buffs apilables, y una tubería de daño de cuatro eslabones.

Hasta ahora, un héroe de Emberdeep tiene las estadísticas con las que nace y el daño que recibe es una resta. Pronto querrás que lleve una espada, una armadura y un buff de furia que dure dos turnos; que pueda esquivar un golpe, absorberlo con un escudo o resistir el fuego. Dos problemas parecidos, con dos soluciones que se confunden. En el primero *añades capas* a un objeto sin cambiar cómo se le pregunta (Decorator). En el segundo *pasas una petición por una fila de pasos*, y cualquiera de ellos puede cambiarla o detenerla (Chain of Responsibility). Empiezas por apilar.

6.1 El olor que pide Decorator: una subclase por cada combinación

Un héroe parte con ataque 8. Con espada, 10. Con furia, 12. Con espada y furia, 14. Si modelas cada caso con herencia, el código se parece a esto (es hipotético; `Hero` es un `record` y no admite subclases):

Hipotético: una clase por combinación

```
class Warrior                  { int attack() { return 8; } }
class WarriorWithSword         extends Warrior { int attack() { return 10; } }
class WarriorWithRage          extends Warrior { int attack() { return 12; } }
class WarriorWithSwordAndRage  extends Warrior { int attack() { return 14; } }
// ... y otras cuatro si añades una armadura
```

Con tres cosas que se pueden llevar o no (espada, armadura, furia) hay $2^3 = 8$ clases; con cinco, 32. Además, la furia dura dos turnos y la herencia se decide al crear el objeto: no puedes «quitarle» una subclase a un objeto que ya existe. Este es el olor: **una explosión de subclases**

para combinar características que deberían ser independientes.

Decorator ataca justo eso. Su intención en el catálogo es añadir responsabilidades a un objeto de forma dinámica, como alternativa flexible a la herencia [9]. La idea: cada capa (espada, armadura, furia) es un objeto que *implementa la misma interfaz* que lo que envuelve y que *guarda* lo envuelto. Bloch llega a la misma conclusión por otro camino cuando defiende preferir la composición a la herencia y presenta las clases envoltorio [5]; en refactorización, el paso se llama *Move Embellishment to Decorator* [17].

6.2 Piezas del patrón Decorator

Rol	Clase en el ejemplo	Propósito	Paquete
Componente (interfaz)	`StatSource`	Lo que se pregunta: ataque, defensa, vida, armadura	`engine.stats`
Componente concreto (clase)	`BaseStats`	Estadísticas desnudas, sin capas	`engine.stats`
Decorador abstracto (clase)	`StatDecorator`	Guarda lo envuelto y delega	`engine.stats`
Decoradores concretos (clases)	`WeaponBonus`, `ArmorBonus` (ejercicio), `TempBuff` (ejercicio)	Cada uno cambia una cifra y delega el resto	`engine.stats`
Cliente (clase)	`Fighter`	Pide sus estadísticas a la capa más externa	`engine.character`

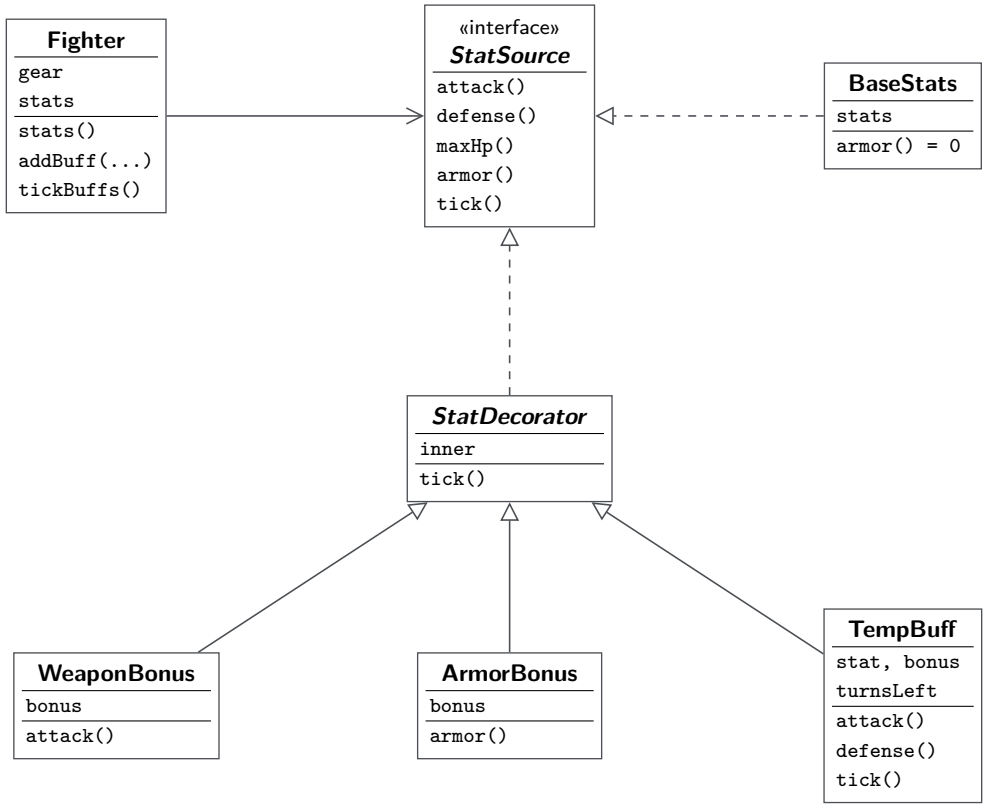


Diagrama UML del patrón Decorator: el decorador es un `StatSource` y guarda otro `StatSource` en `inner`.

6.3 El ejemplo de Decorator, pieza a pieza: `StatSource`

La interfaz. Quien necesite las estadísticas de un combatiente solo conocerá este contrato. Sus cuatro cifras se llaman igual que las del `record Stats`, y por eso el resto del motor (`stats().attack()`, `stats().maxHp()`) seguirá compilando. El método `tick()` lo explico enseguida:

engine/stats/StatSource.java

```
package emberdeep.engine.stats;

public interface StatSource {
    int attack();
    int defense();
    int maxHp();
    int armor();          // lo que resta la armadura del equipo
    StatSource tick();    // la cadena después de pasar un turno
}
```

El componente base. Envuelve el `Stats` de siempre. No tiene armadura (la armadura es de los equipos, no de las estadísticas base) y `tick()` no cambia nada: devuelve `this`.

engine/stats/BaseStats.java

```
package emberdeep.engine.stats;

import emberdeep.engine.character.Stats;

public record BaseStats(Stats stats) implements StatSource {
    @Override public int attack() { return stats.attack(); }
    @Override public int defense() { return stats.defense(); }
    @Override public int maxHp() { return stats.maxHp(); }
    @Override public int armor() { return 0; }
    @Override public StatSource tick() { return this; }
}
```

El decorador abstracto. Es una clase que *es* un `StatSource` y *tiene* otro: el que envuelve. Delega las cuatro preguntas en él; los decoradores concretos redefinen solo lo que cambian. `tick()` también delega: avanza lo envuelto y, si algo cambió, reconstruye la capa sobre el resultado con `rewrap`. Si nada cambió, devuelve la misma capa (no se crean objetos de más):

engine/stats/StatDecorator.java

```
package emberdeep.engine.stats;

public abstract class StatDecorator implements StatSource {
    protected final StatSource inner;

    protected StatDecorator(StatSource inner) {
        this.inner = inner;
    }

    @Override public int attack() { return inner.attack(); }
    @Override public int defense() { return inner.defense(); }
    @Override public int maxHp() { return inner.maxHp(); }
```

```
    @Override public int armor() { return inner.armor(); }

    @Override
    public StatSource tick() {
        StatSource next = inner.tick();
        return next == inner ? this : rewrap(next);
    }

    // La misma capa, pero sobre otra cadena
    protected abstract StatSource rewrap(StatSource newInner);
}
```

Un decorador concreto. La espada suma ataque y delega lo demás. Es inmutable: dos campos `final` y ningún *setter*. Para «cambiar» algo se construye otra capa, nunca se modifica la existente; es lo que permitirá a un capítulo posterior guardar la cadena entera sin miedo a que cambie después.

engine/stats/WeaponBonus.java

```
package emberdeep.engine.stats;

public class WeaponBonus extends StatDecorator {
    private final int bonus;

    public WeaponBonus(StatSource inner, int bonus) {
        super(inner);
        this.bonus = bonus;
    }

    @Override
    public int attack() {
        return inner.attack() + bonus;
    }

    @Override
    protected StatSource rewrap(StatSource newInner) {
        return new WeaponBonus(newInner, bonus);
    }
}
```

Apilar. Cada capa envuelve a la anterior, y desde fuera sigue pareciendo un `StatSource`:

Fragmento: dos capas sobre BaseStats

```
StatSource aria = new BaseStats(new Stats(30, 8, 3));
StatSource armed = new WeaponBonus(aria, 2);
StatSource twice = new WeaponBonus(armed, 1);
System.out.println(aria.attack() + " " + armed.attack() + " " + twice.attack())
    ↪ ;
```

Salida

```
8 10 11
```

Qué es `tick()`. Una espada es permanente, pero una furia dura unos turnos. Como las capas

son inmutables, no se puede «restarle un turno» a una furia: `tick()` *devuelve una cadena nueva* en la que cada buff tiene un turno menos y los que llegan a cero ya no están. Piensa en una pila de tres capas, de fuera adentro: furia con 2 turnos, espada, estadísticas base. Tras un `tick()` queda furia con 1 turno, espada, base. Tras otro, queda espada, base: la furia se ha ido. La espada y la base devuelven la misma capa. Construirás ese buff (`TempBuff`) en el ejercicio de proyecto.

El cliente: equipo y cadena actual. Un combatiente necesita dos cosas distintas: su *equipo permanente* (la base, la espada y la armadura con las que sale a la partida) y su *cadena actual*, que es ese equipo más los buffs que se le apilen durante el combate. `Fighter` guarda las dos: `gear` y `stats`. Al nacer son la misma referencia; `addBuff` solo cambia `stats`. Guardarlas separadas parece un lujo ahora, pero es lo que permitirá, más adelante, volver al equipo permanente sin reconstruirlo.

Fragmento de Fighter: gear frente a stats

```
private final StatSource gear;   // equipo permanente: nunca cambia
private StatSource stats;        // gear + buffs de este combate

public void addBuff(StatKind stat, int bonus, int turns) {
    stats = new TempBuff(stats, stat, bonus, turns);   // envuelve la cadena
    ↪ actual
}

public void tickBuffs() {
    stats = stats.tick();
}
```

6.4 El olor que pide Chain of Responsibility: un método de daño con seis `if`

Ahora el daño recibido. Cuando el motor sepa de esquivas, escudos, armaduras y resistencias, la tentación es meterlo todo en un método (hipotético):

Hipotético: el daño recibido en un solo método

```
int receive(Fighter target, int amount, DamageType type) {
    if (random.nextDouble() < dodgeChance) {            // 1. esquiva
        return 0;
    }
    if (target.shield() > 0) {                          // 2. escudo
        int absorbed = Math.min(target.shield(), amount);
        target.loseShield(absorbed);
        amount -= absorbed;
        if (amount == 0) {                              // 3. todo absorbido
            return 0;
        }
    }
    if (target.stats().armor() > 0) {                   // 4. armadura
        amount = Math.max(1, amount - target.stats().armor());
    }
    if (target.resists(type)) {                         // 5. resistencia
        amount = amount / 2;
```

```
18      }
19      if (amount < 1) {                                   // 6. mínimo
20          amount = 1;
21      }
22      return amount;
23  }
```

Es corto y funciona, pero tiene tres problemas. El **orden está escondido**: esquivar tiene que ir primero y nada lo dice. Cada regla nueva (una bendición, un veneno) exige **abrir el método** y decidir entre qué `if` se mete. Y no puedes **reutilizar ni probar** una regla sola: para saber si la armadura funciona hay que pasar por la esquiva. Este es el olor: **una fila de pasos, cada uno con su condición, que o modifican un valor o lo cortan**. La esquiva y el escudo completo cortan; la armadura y la resistencia modifican.

Chain of Responsibility evita acoplar el emisor de una petición con su receptor, dando a más de un objeto la oportunidad de manejarla: se encadenan los objetos receptores y la petición pasa por ellos hasta que alguno la maneja [9]. En Emberdeep adaptamos la idea: la petición es un golpe, cada eslabón hace *una* cosa, y pasa el golpe (tal cual o cambiado) al siguiente, o lo corta y devuelve su resultado.

6.5 Piezas del patrón Chain of Responsibility

Rol	Clase en el ejemplo	Propósito	Paquete
Manejador (interfaz)	`DamageHandler`	Contrato: `handle(target, damage)`; cada eslabón guarda su `next`	`engine.combat.defense`
Manejadores concretos (clases)	`DodgeHandler`, `ShieldHandler`, `ArmorHandler` (ejercicio), `ResistanceHandler` (ejercicio)	Cada uno modifica el daño o lo corta	`engine.combat.defense`
Petición (record)	`Damage`	Cantidad y tipo del golpe	`engine.combat.damage`
Montaje de la cadena (clase)	`DamagePipeline`	Une los eslabones en un orden fijo	`engine.combat.defense`
Cliente (clase)	`Combat`	Pasa cada golpe por la tubería	`engine.combat`

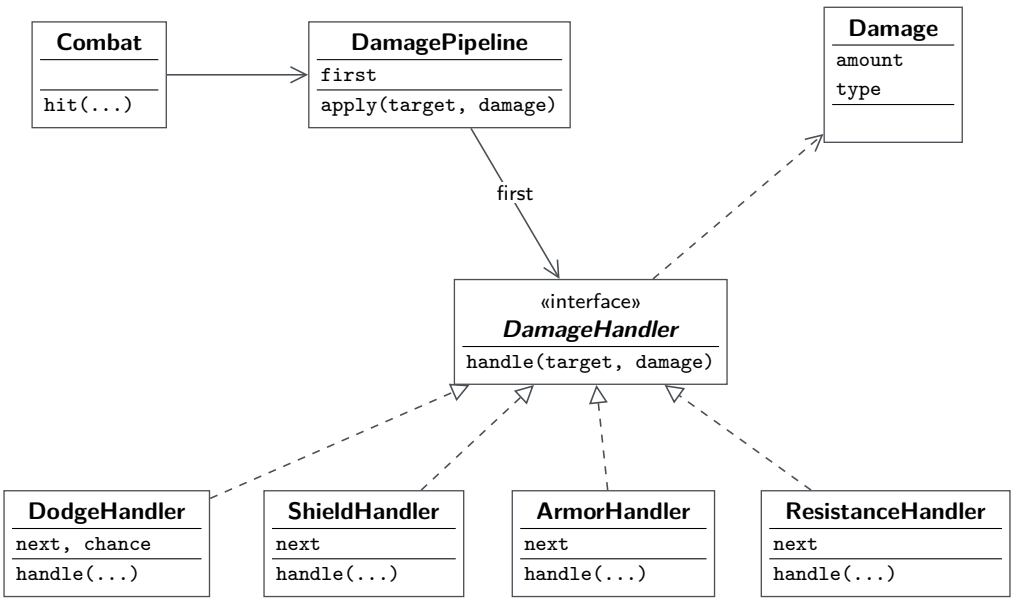


Diagrama UML del patrón Chain of Responsibility: la tubería guarda el primer eslabón y cada eslabón guarda el siguiente en `next`.

6.6 El ejemplo de Chain of Responsibility, pieza a pieza

La petición. Un golpe es una cantidad y un tipo. Es un `record`: inmutable, así que cada eslabón que quiera cambiarlo crea uno nuevo. Hay dos tipos de daño por ahora:

engine/combat/damage/Damage.java

```java
package emberdeep.engine.combat.damage;

public record Damage(int amount, DamageType type) {}
```

engine/combat/damage/DamageType.java

```java
package emberdeep.engine.combat.damage;

public enum DamageType { PHYSICAL, FIRE }
```

Qué necesita el defensor. Los eslabones preguntan cosas al combatiente que recibe el golpe. Para el escudo, `Fighter` gana un contador y tres métodos: cuánto tiene (`shield()`), cómo se le da (`addShield`) y cómo absorbe (`absorbShield`), que devuelve lo que *no* pudo absorber. En este capítulo nadie en el juego llama a `addShield`: la usan los ejemplos y los ejercicios, y el capítulo 7 la conecta a una acción de defensa.

Fragmento de Fighter: el escudo

```java
private int shield;   // puntos de escudo; empieza en 0

public int shield() { return shield; }

public void addShield(int amount) {
    shield += amount;
}
```

```
8
9  // Gasta escudo y devuelve el daño que sobra
10 public int absorbShield(int damage) {
11     int absorbed = Math.min(shield, damage);
12     shield -= absorbed;
13     return damage - absorbed;
14 }
```

El manejador. Un contrato de un solo método: recibe al defensor y el golpe y devuelve el daño final que se aplicará. Cada eslabón decide si lo *cambia y pasa* al siguiente (`next.handle(...)`) o lo *corta* devolviendo un resultado sin llamarlo.

engine/combat/defense/DamageHandler.java

```
1  package emberdeep.engine.combat.defense;
2
3  import emberdeep.engine.character.Fighter;
4  import emberdeep.engine.combat.damage.Damage;
5
6  @FunctionalInterface
7  public interface DamageHandler {
8      int handle(Fighter target, Damage damage);
9  }
```

Un eslabón que corta. La esquiva tira un número aleatorio. Si esquiva, devuelve 0 y *no llama a nadie más*: es el corte. Si no, pasa el golpe tal cual. Con `chance` igual a 0 ni siquiera gasta un número del `Random`, de modo que una partida sin esquiva sigue siendo reproducible con la misma semilla.

engine/combat/defense/DodgeHandler.java

```
1  package emberdeep.engine.combat.defense;
2
3  import emberdeep.engine.character.Fighter;
4  import emberdeep.engine.combat.damage.Damage;
5  import java.util.Random;
6
7  public class DodgeHandler implements DamageHandler {
8      private final Random random;
9      private final double chance;
10     private final DamageHandler next;
11
12     public DodgeHandler(Random random, double chance, DamageHandler next) {
13         this.random = random;
14         this.chance = chance;
15         this.next = next;
16     }
17
18     @Override
19     public int handle(Fighter target, Damage damage) {
20         if (chance > 0 && random.nextDouble() < chance) {
21             return 0;
22         }
23         return next.handle(target, damage);
24     }
```

```
25 }
```

Un eslabón que modifica, y a veces corta. El escudo absorbe puntos del daño. Si queda daño, lo pasa reducido; si el escudo se lo ha comido todo, corta con 0. Fíjate en que este eslabón tiene un efecto lateral: gasta el escudo del defensor. Una cadena de Responsabilidad no tiene por qué ser pura, pero entonces el orden importa todavía más.

engine/combat/defense/ShieldHandler.java

```
1  package emberdeep.engine.combat.defense;
2
3  import emberdeep.engine.character.Fighter;
4  import emberdeep.engine.combat.damage.Damage;
5
6  public class ShieldHandler implements DamageHandler {
7      private final DamageHandler next;
8
9      public ShieldHandler(DamageHandler next) {
10         this.next = next;
11     }
12
13     @Override
14     public int handle(Fighter target, Damage damage) {
15         int left = target.absorbShield(damage.amount());
16         if (left == 0) {
17             return 0;
18         }
19         return next.handle(target, new Damage(left, damage.type()));
20     }
21 }
```

Montar la cadena. Los eslabones se construyen de dentro afuera: el último guarda un final que devuelve el daño que le llega (una lambda, porque `DamageHandler` es una interfaz funcional), y cada uno guarda el siguiente. `DamagePipeline` esconde ese montaje y fija el orden en un solo sitio. Esta es la versión del texto, con los dos eslabones que acabas de ver; en el ejercicio de proyecto añades los otros dos.

engine/combat/defense/DamagePipeline.java (versión del texto)

```
1  package emberdeep.engine.combat.defense;
2
3  import emberdeep.engine.character.Fighter;
4  import emberdeep.engine.combat.damage.Damage;
5  import java.util.Random;
6
7  public class DamagePipeline {
8      private final DamageHandler first;
9
10     public DamagePipeline(Random random, double dodgeChance) {
11         DamageHandler end = (target, damage) -> damage.amount();
12         this.first = new DodgeHandler(random, dodgeChance,
13                 new ShieldHandler(end));
14     }
15
16     public int apply(Fighter target, Damage damage) {
```

```
17          return first.handle(target, damage);
18      }
19  }
```

Un golpe de ejemplo. Un defensor con 3 puntos de escudo recibe un golpe de 10. Con esquiva al 100 % el golpe se corta en el primer eslabón y el escudo no se toca. Con esquiva al 0 % llega al escudo, que absorbe 3 y deja pasar 7:

Fragmento: un golpe de 10 contra 3 de escudo

```
Damage hit = new Damage(10, DamageType.PHYSICAL);

target.addShield(3);
int dodged = new DamagePipeline(new Random(1), 1.0).apply(target, hit);
System.out.println(dodged + " " + target.shield());

int hurt = new DamagePipeline(new Random(1), 0.0).apply(target, hit);
System.out.println(hurt + " " + target.shield());
```

Salida

```
0 3
7 0
```

Por qué ese orden. La esquiva va primero porque si el golpe falla, *nada más debe pasar*: ni escudo gastado ni armadura aplicada. Si la pusieras la última, el escudo ya se habría consumido para un golpe que no te iba a hacer daño. En general, los eslabones que cortan van delante y los que solo modifican detrás. El escudo va antes que la armadura por una regla de diseño: el escudo absorbe puntos del golpe «en bruto» y la armadura reduce solo lo que de verdad llega. El ejercicio 6.4 te enseña lo que ocurre si te equivocas.

6.7 Decorator, Chain of Responsibility y Strategy: envolver, encadenar o elegir

Los tres patrones sacan lógica de una clase hacia objetos intercambiables, y se confunden. Lo que los separa es qué hacen con el siguiente objeto:

	Decorator	**Chain of Responsibility**	**Strategy**
Idea	Envolver para añadir	Pasar y poder cortar	Elegir una de varias
Objetos implicados	Una pila de capas	Una fila de eslabones	Uno solo, el elegido
¿Llama al siguiente?	Siempre (delega y añade)	A veces: puede cortar	No hay siguiente
¿Cambia el resultado?	Sí, sumando a lo envuelto	Sí, o lo sustituye	Lo calcula la estrategia
En Emberdeep	`WeaponBonus` sobre `BaseStats`	`DodgeHandler` antes que `ShieldHandler`	`FlatFormula` o `ScaledFormula`

Criterio rápido: si todas las capas se aplican siempre y la pregunta es «qué cifra sale tras sumar todo», es Decorator. Si cada paso puede decir «aquí acaba», es Chain of Responsibility. Si solo hay un algoritmo y lo que importa es *cuál*, es Strategy. En el código se ven parecidos: los dos primeros guardan un objeto de su mismo tipo y lo llaman. La diferencia está en una línea: el `return` que no llama a `next`.

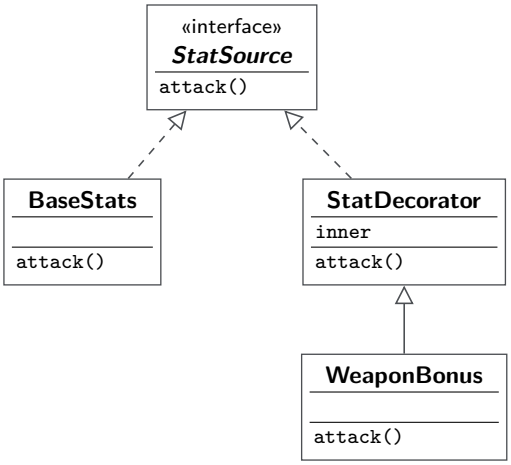


Diagrama UML de Decorator: envuelve y siempre delega.

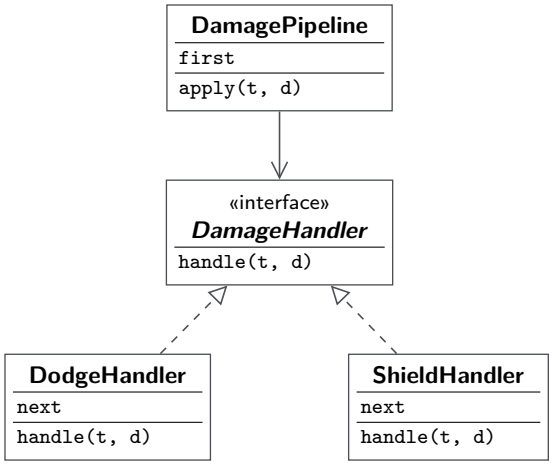


Diagrama UML de Chain of Responsibility: encadena y puede cortar.

Aprenderás más viendo cada uno en un caso pequeño: el ejercicio 6.3 te da dos fragmentos casi idénticos y te pide separarlos.

6.8 Decorator y Chain of Responsibility: cuándo sí y cuándo no

Decorator compensa cuando las características se combinan libremente, cuando se añaden y se quitan mientras el programa corre (los buffs) y cuando cada capa puede cambiar *comportamiento* y no solo sumar un número. **No compensa** si lo único que apilas son números: una lista de bonificaciones y un bucle que las suma sería más simple y más fácil de depurar. En Emberdeep el Decorator se justifica por el patrón, no por la dificultad: aquí un `List<Bonus>` también serviría, y es honesto reconocerlo. Otros costes: muchos objetos pequeños, una pila que se lee de fuera adentro (depurar «¿qué capa sumó esto?» exige recorrerla) y la identidad: una capa no es el objeto que envuelve, así que `==` y `instanceof` dejan de decir lo que parecen. Hannemann y Kiczales implementaron los 23 patrones del catálogo en Java y AspectJ para comparar su modularidad [13]: es una lectura útil si quieres ver qué patrones cuestan más de escribir en Java.

Chain of Responsibility compensa cuando hay una secuencia de pasos independientes, cada uno con su condición, y quieres poder reordenarlos, añadirlos o probarlos uno a uno. **No compensa** si los pasos son dos o tres y no cambian: el método con `if` del principio sería más claro. Riesgos propios: el orden es un contrato que vive en el montaje y no en el tipo (como acabas de ver con la esquiva y el escudo), una petición que ningún eslabón corta ni termina se pierde si olvidas el final de la cadena (por eso el último eslabón devuelve el daño), y los efectos laterales, como gastar escudo, hacen que el orden cambie el estado del juego y no solo un número. Para una ficha de cada patrón con diagramas, mira [24].

6.9 En el proyecto: H7 y H8 con Decorator y Chain of Responsibility

Este capítulo avanza dos hitos. Cambios del **H7** (Decorator):

- **Nuevo paquete** `engine/stats/` con `StatSource`, `StatKind`, `BaseStats`, `StatDecorator`

y exactamente tres decoradores: `WeaponBonus`, `ArmorBonus` y `TempBuff`.

- **Cambia** `Fighter`: guarda `gear` y `stats` (ambos `StatSource`) en lugar de un `Stats`. `stats()` devuelve ahora un `StatSource`, pero `attack()`, `defense()` y `maxHp()` se llaman igual: `Combat` y `ConsoleUi` no se enteran. Gana `gear()`, `addBuff` y `tickBuffs`, y su fábrica de héroes pasa a `ofHero(hero, gear)`.
- **Cambia** `GameSession`: monta el equipo de cada héroe al empezar la partida: estadísticas base, una espada de +2 de ataque y una armadura de 1. No hay acción de equipar. Los enemigos no llevan equipo: su cadena es solo `BaseStats` y su `armor()` vale 0.

Cambios del **H8** (Chain of Responsibility):

- **Nuevo** `engine/combat/defense/` con `DamageHandler`, `DamagePipeline` y los cuatro eslabones (`DodgeHandler`, `ShieldHandler`, `ArmorHandler`, `ResistanceHandler`), y en `engine/combat/damage/` `Damage` y `DamageType` (`PHYSICAL` y `FIRE`). El orden es ese.
- **Cambia** `Fighter`: gana `shield`, `shield()`, `addShield`, `absorbShield` y `resists`, que recibe un `DamageType`. Un enemigo resiste lo que dice `enemy.kind().resistances()`; el héroe no resiste nada.
- **Cambia** `EnemyKind`: gana `resistances()`. Solo `ASH_RAT` resiste el fuego; los demás tipos no resisten nada.
- **Cambia** `Combat`: recibe un `DamagePipeline`, pasa por él el daño físico que calcula la `DamageFormula` del capítulo 2 y avanza los buffs (`tickBuffs`) al final de cada ronda. Si el golpe no hace daño, el mensaje es «X ataca a Y, pero no le hace daño.».
- **Cambian** `GameSession` (recibe el pipeline como último parámetro) y `Main` (crea `new DamagePipeline(random, 0.0)`: la esquiva queda al 0 % en la partida real y, al no gastar números aleatorios, las trazas dependen solo del equipo).

Una decisión consciente: dos mitigaciones. Puede parecer un error que el daño se reduzca dos veces. No lo es. La `DamageFormula` del capítulo 2 resta la *defensa* (una estadística base del personaje); la cadena resta además la *armadura* (algo que llevas puesto). Son dos conceptos con dos fuentes distintas y se calculan en sitios distintos. Un golpe de ejemplo: el Esqueleto (ataque 7) golpea a Cael (defensa 2, con su armadura de 1). La fórmula plana da $7 - 2 = 5$ y la armadura lo deja en $\text{máx}(1,\ 5 - 1) = 4$. Antes de este capítulo, Cael recibía 5. Si prefieres una sola mitigación, es un buen ejercicio de diseño; aquí se mantienen las dos para que `ArmorHandler` tenga trabajo real.

Un golpe de fuego. Ninguna acción del juego produce todavía daño de fuego (las bombas y las habilidades de fuego llegan en el capítulo 7), así que el ejercicio lo construye a mano con `new Damage(10, DamageType.FIRE)`, sin pasar por la `DamageFormula`: solo por la cadena. Contra una Rata de ceniza (resiste el fuego, sin armadura, sin escudo y con esquiva 0) baja la mitad: 5. Con armadura o con escudo la cifra cambia, y el ejercicio te pide predecirla.

Qué cambia en la partida. Con la espada, los héroes pegan 2 más; con la armadura y la tubería (parte 2 del ejercicio), reciben 1 menos por golpe. Las trazas de los capítulos anteriores ya no se repiten, y es lo esperado: el equipo es un cambio de reglas. Con la fórmula plana y siempre el primer enemigo vivo como objetivo, la cripta acaba en la ronda 3 (antes, en la 4), la ronda 1 termina con el Esqueleto fuera de combate (8, 6 y 9 de daño) y la caverna acaba en la ronda 2.

Hitos 7 y 8: equipo y buffs apilables, y tubería de daño

Tras estos hitos, cada héroe lleva una espada y una armadura que se apilan como capas sobre sus estadísticas, el motor sabe avanzar buffs temporales, y todo golpe pasa por una tubería de cuatro eslabones antes de quitar vida. La estructura:

Estructura de carpetas tras el capítulo 6

```
emberdeep/
|-- settings.gradle
|-- build.gradle
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
`-- src/main/java/emberdeep/
    |-- app/
    |   |-- Main.java              (composition root)
    |   `-- GameConfig.java
    |-- engine/
    |   |-- character/             Stats, Hero, HeroBuilder, Enemy,
    ↪ EnemyKind, Fighter,
    |   |                          Skill, EnemyTemplate, TemplateRegistry
    |   |-- stats/                 StatSource, StatKind, BaseStats,
    ↪ StatDecorator,
    |   |                          WeaponBonus, ArmorBonus, TempBuff
    |   |-- combat/
    |   |   |-- Combat.java
    |   |   |-- ai/                EnemyAi, FirstAliveAi, LowestHpAi,
    ↪ RandomTargetAi
    |   |   |-- damage/            DamageFormula, FlatFormula,
    ↪ ScaledFormula,
    |   |   |                      Damage, DamageType
    |   |   `-- defense/           DamageHandler, DamagePipeline,
    ↪ DodgeHandler,
    |   |                          ShieldHandler, ArmorHandler,
    ↪ ResistanceHandler
    |   |-- item/                  Item, ItemKind, Inventory
    |   |-- session/               GameSession
    |   `-- world/
    |       |-- Dungeon.java, Floor.java, DungeonBuilder.java
    |       |-- biome/             BiomeFactory, CryptBiome, CavernBiome
    |       `-- map/               TileType, TileTypes, GameMap, MapParser,
    ↪  MapData
    `-- ui/
        `-- ConsoleUi.java
```

Lo practicas en el ejercicio 6.5. Antes, cuatro ejercicios más pequeños, en otros dominios.

6.10 Ejercicios

Ejercicio 6.1: Un texto con decoradores ★★☆☆☆

No es de Emberdeep. Crea una carpeta vacía y, dentro, archivos sin paquete. Vas a decorar un texto con tres capas: recortar espacios, pasar a mayúsculas y añadir un prefijo.

1. `TextSource.java`: interfaz con un método `String text()`.
2. `PlainText.java`: implementa `TextSource`, recibe un `String` en el constructor y lo devuelve en `text()`.
3. `TextDecorator.java`: clase abstracta que implementa `TextSource`. Guarda el texto envuelto en un campo `protected final TextSource inner`, que recibe en su constructor.
4. `Trim.java`, `Upper.java` y `Prefix.java`: tres decoradores que extienden `TextDecorator`. `Trim` devuelve `inner.text().trim()`, `Upper` devuelve `inner.text().toUpperCase()` y `Prefix` recibe también un `String prefix` y devuelve el prefijo seguido de `inner.text()`.
5. `Main.java`: copia este código y ejecútalo:

Main.java

```
public class Main {
    public static void main(String[] args) {
        TextSource raw = new PlainText("  hola  ");
        TextSource a = new Prefix(new Upper(new Trim(raw)), ">> ");
        TextSource b = new Trim(new Upper(new Prefix(raw, ">> ")));
        System.out.println("[" + a.text() + "]");
        System.out.println("[" + b.text() + "]");
    }
}
```

Está bien si imprime las dos líneas siguientes, y sabes decir por qué la segunda lleva espacios entre el prefijo y el texto:

Salida esperada

```
[>> HOLA]
[>>   HOLA]
```

Pista: *Las capas se evalúan de dentro afuera: en* `b`*, el prefijo se añade* antes *de recortar, y* `trim()` *solo quita los espacios de los extremos.*

Guiado ▪ solución en la pág. 186

Ejercicio 6.2: Una cadena de validadores ★★☆☆☆

No es de Emberdeep. Quieres validar el texto de un comando con tres reglas, en este orden, y quedarte con el *primer* fallo: (1) no está vacío ni son solo espacios, (2) no pasa de 8 caracteres, (3) contiene solo letras. Usa este contrato; `validate` devuelve `null` si el comando es válido y, si no, el mensaje de error:

CommandValidator.java

```java
public interface CommandValidator {
    String validate(String command);   // null = válido
}
```

1. Crea `NotEmptyValidator`, `MaxLengthValidator` (recibe el máximo en el constructor) y `OnlyLettersValidator`. Cada uno recibe en el constructor el siguiente `CommandValidator` (`next`). Si su regla falla, devuelve el mensaje sin llamar al siguiente; si no, devuelve `next.validate(command)`. Los mensajes son «El comando está vacío», «El comando supera 8 caracteres» y «Solo se admiten letras».
2. En un `Main`, monta la cadena en el orden indicado. El final de la cadena es la lambda `command -> null`.
3. Valida, en este orden, `""`, `"atacar"`, `"ataquemuyfuerte"`, `"ata2"` y `"ataque2muy"`, e imprime cada resultado como `'comando' -> ok` o `'comando' -> mensaje`.

Está bien si la salida es la siguiente (fíjate en la última línea: el comando falla dos reglas y solo se informa de la primera en el orden de la cadena):

Salida esperada

```
'' -> El comando está vacío
'atacar' -> ok
'ataquemuyfuerte' -> El comando supera 8 caracteres
'ata2' -> Solo se admiten letras
'ataque2muy' -> El comando supera 8 caracteres
```

Pista: *`Character.isLetter(char)` dice si un carácter es una letra. Para recorrer los caracteres de un `String`, `for (char c : command.toCharArray())`.*

Práctica ▪ solución en la pág. 189

Ejercicio 6.3: Decorator o Chain of Responsibility ★★★☆☆

Estas dos clases se parecen: las dos implementan `Notifier`, las dos guardan otro `Notifier` y las dos lo llaman.

Notifier.java

```java
public interface Notifier { void send(String message); }
```

LoggedNotifier.java

```java
public class LoggedNotifier implements Notifier {
    private final Notifier inner;
    public LoggedNotifier(Notifier inner) { this.inner = inner; }

    @Override
    public void send(String message) {
        System.out.println("Enviando: " + message);
        inner.send(message);
    }
}
```

FilteredNotifier.java

```java
public class FilteredNotifier implements Notifier {
    private final Notifier next;
    public FilteredNotifier(Notifier next) { this.next = next; }

    @Override
    public void send(String message) {
        if (message.contains("spoiler")) {
            return;
        }
        next.send(message);
    }
}
```

1. Clasifica cada clase como Decorator o Chain of Responsibility.
2. Para cada una, cita la línea que lo demuestra y explica la diferencia con la otra en una frase usando «siempre delega» y «puede cortar».
3. Se monta `new LoggedNotifier(new FilteredNotifier(console))`, donde `console` imprime el mensaje. Se envía «hola» y luego «spoiler: muere». ¿Qué sale por pantalla? ¿Y si el montaje es `new FilteredNotifier(new LoggedNotifier(console))`?

Está bien si tu clasificación se apoya en el `return` que evita llamar al siguiente y si las dos salidas del apartado 3 son distintas y correctas.

Lectura de código ▪ solución en la pág. 191

Ejercicio 6.4: Esquivo, pero el escudo se gasta ★★★☆☆

Este programa (no es de Emberdeep; un solo archivo sin paquete) monta una cadena de tres eslabones con lambdas: armadura, escudo y esquiva, en ese orden. Un `Defender` tiene puntos de escudo y de armadura; el golpe es un entero.

ChainBug.java (1/2: eslabones)

```java
import java.util.Random;

public class ChainBug {
    interface Handler { int handle(Defender d, int damage); }

    static class Defender {
        int shield;
        final int armor;
        Defender(int shield, int armor) { this.shield = shield; this.armor = armor; }
    }

    static Handler dodge(Random random, double chance, Handler next) {
        return (d, dmg) -> random.nextDouble() < chance ? 0 : next.handle(d, dmg);
    }

    static Handler shield(Handler next) {
        return (d, dmg) -> {
            int absorbed = Math.min(d.shield, dmg);
            d.shield -= absorbed;
            return next.handle(d, dmg - absorbed);
        };
    }

    static Handler armor(Handler next) {
        return (d, dmg) -> next.handle(d, Math.max(1, dmg - d.armor));
    }

    static Handler pipeline(Random random, double chance) {
        Handler end = (d, dmg) -> dmg;
        return armor(shield(dodge(random, chance, end)));
    }
```

ChainBug.java (2/2: main)

```java
    public static void main(String[] args) {
        Defender a = new Defender(4, 2);
        System.out.println(pipeline(new Random(1), 1.0).handle(a, 10) + " " + a.shield);
        Defender b = new Defender(3, 2);
        System.out.println(pipeline(new Random(1), 0.0).handle(b, 3) + " " + b.shield);
        Defender c = new Defender(4, 2);
        System.out.println(pipeline(new Random(1), 0.0).handle(c, 10) + " " + c.shield);
    }
}
```

Cada línea imprime el daño final y el escudo que queda. Con esquiva al 100 % (primera línea) el golpe debería hacer 0 de daño y dejar el escudo en 4; la salida real es `0 0`. Haz esto:

1. Ejecútalo y explica por qué se gasta el escudo si el golpe se esquiva.

2. Reordena los eslabones en `pipeline` (sin cambiar su código) para que un golpe esquivado no toque nada. Usa este criterio: lo que corta va delante de lo que modifica, y el escudo absorbe puntos en bruto *antes* de que la armadura reduzca lo que pasa.
3. Vuelve a ejecutar. La segunda línea (golpe de 3 contra escudo 3 y armadura 2) cambia a algo incorrecto. Explica por qué y arréglalo en el eslabón del escudo.

Está bien si al final la salida es la siguiente, y puedes explicar por qué el orden importa aquí con las tres líneas:

Salida esperada

```
0 4
0 0
4 0
```

Pista: *En el apartado 3, mira qué recibe la armadura cuando el escudo se lo ha comido todo, y qué hace* `Math.max(1, ...)` *con ese valor.*

Depuración ▪ solución en la pág. 191

Ejercicio 6.5: Hitos 7 y 8: equipo, buffs y tubería de daño ★★★★☆

Implementa los hitos H7 y H8 partiendo del estado del capítulo anterior (solución del ejercicio 5.5), en dos partes. Primero H7, y compruébalo antes de pasar a H8.

Parte 1 (H7): equipo y buffs que se apilan.

1. En el paquete `emberdeep.engine.stats`: `StatSource` (`attack()`, `defense()`, `maxHp()`, `armor()`, `tick()`), el enum `StatKind` (`ATTACK`, `DEFENSE`), `BaseStats`, `StatDecorator` y `WeaponBonus` tal como están en el texto del capítulo, y los dos que faltan: `ArmorBonus(inner, bonus)`, que suma a `armor()`, y `TempBuff(inner, stat, bonus, turnsLeft)`, que suma `bonus` a `attack()` o a `defense()` según `stat`. Su `tick()` devuelve la cadena interior ya avanzada si `turnsLeft <= 1` (el buff se acaba) y, si no, un `TempBuff` nuevo con un turno menos.
2. `Fighter`: guarda `gear` (final) y `stats`, los dos `StatSource`, en lugar del `Stats` anterior. Nace con `stats = gear` y la vida inicial es `gear.maxHp()`. `ofHero(Hero, StatSource gear)` sustituye a `ofHero(Hero)`; `ofEnemy(Enemy)` usa `new BaseStats(enemy.stats())`. Nuevos: `stats()` devuelve un `StatSource`, `gear()`, `addBuff(StatKind, int bonus, int turns)` (envuelve `stats` en un `TempBuff`) y `tickBuffs()` (`stats = stats.tick()`).
3. `GameSession`: monta el equipo de cada héroe con un método privado `gearOf(Hero)`: `BaseStats` del héroe, con `WeaponBonus` de +2 y encima `ArmorBonus` de +1 (constantes privadas).

Está bien si:

1. Este archivo temporal, `app/Check06.java`, imprime las cuatro líneas siguientes (la última tras dos `tick()`: el buff de 2 turnos se ha acabado):

app/Check06.java (parte 1; bórralo al terminar)

```java
package emberdeep.app;

import emberdeep.engine.character.Stats;
import emberdeep.engine.stats.ArmorBonus;
import emberdeep.engine.stats.BaseStats;
import emberdeep.engine.stats.StatKind;
import emberdeep.engine.stats.StatSource;
import emberdeep.engine.stats.TempBuff;
import emberdeep.engine.stats.WeaponBonus;

public class Check06 {
    public static void main(String[] args) {
        StatSource hero = new BaseStats(new Stats(30, 8, 3));
        hero = new WeaponBonus(hero, 2);
        hero = new ArmorBonus(hero, 1);
        System.out.println(hero.attack() + " " + hero.defense() + " "
    ↪ + hero.armor());
        hero = new TempBuff(hero, StatKind.ATTACK, 4, 2);
        System.out.println(hero.attack());
        hero = hero.tick();
        System.out.println(hero.attack());
        hero = hero.tick();
        System.out.println(hero.attack() + " " + hero.armor());
    }
}
```

Salida esperada

```
10 3 1
14
14
10 1
```

2. Con `yes 1 | head -n 20 | ./gradlew -q run -args="1 default"` (la cripta, fórmula plana, siempre el primer enemigo), la ronda 1 empieza con «Aria ataca a Esqueleto y le hace 8 de daño.», «Borin. . . 6» y «Cael. . . 9», el Esqueleto queda fuera de combate, la Rata de ceniza golpea a Cael con 3 y el Zombi a Aria con 2 (la armadura aún no actúa: no hay tubería). La partida acaba en la ronda 3 con «¡Victoria!».

Parte 2 (H8): tubería de daño.

1. En `engine/combat/damage/`: `Damage(int amount, DamageType type)` y el enum `DamageType` (`PHYSICAL`, `FIRE`).
2. `EnemyKind` gana `Set<DamageType> resistances()`: vacío para todos salvo `ASH_RAT`, que resiste `FIRE`.
3. `Fighter` gana el campo `kind` (el `EnemyKind` del enemigo; `null` en los héroes), `shield`, `shield()`, `addShield(int)`, `absorbShield(int damage)` (gasta escudo y devuelve el daño que sobra) y `resists(DamageType)` (verdadero solo si es un enemigo cuyo tipo resiste ese daño).
4. En `engine/combat/defense/`: `DamageHandler`, `DodgeHandler`, `ShieldHandler` y

`DamagePipeline` como en el texto, más `ArmorHandler` (devuelve `next.handle` con `Math.max(1, amount - target.stats().armor())`) y `ResistanceHandler` (si `target.resists(damage.type())`, el daño pasa a la mitad, con mínimo 1). El orden de montaje es esquiva, escudo, armadura, resistencia. Cada eslabón recibe el siguiente en el constructor y el último guarda el final.

5. `Combat` recibe un `DamagePipeline` como cuarto parámetro. En `hit`, el daño que calcula la `DamageFormula` se envuelve en un `Damage` físico, se pasa por la tubería y lo que sale se aplica a `takeDamage`; si sale 0, el mensaje es «X ataca a Y, pero no le hace daño.». Al final de `enemyPhase`, tras subir la ronda, todos los combatientes llaman a `tickBuffs()`.

6. `GameSession` recibe el pipeline como último parámetro y se lo pasa a `Combat`; `Main` lo crea con `new DamagePipeline(random, 0.0)` y lo pasa al construir la sesión.

Está bien si:

1. Este archivo temporal sustituye al anterior e imprime las cuatro líneas siguientes (esquiva al 100 % con el escudo intacto; después, escudo 3 y armadura 1 sobre un golpe de 10; un golpe de fuego contra la Rata de ceniza, que la resiste, sin armadura ni escudo; y un golpe físico de 10 contra la misma Rata):

app/Check06.java (parte 2; bórralo al terminar)

```java
package emberdeep.app;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.damage.Damage;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.stats.ArmorBonus;
import emberdeep.engine.stats.BaseStats;
import java.util.List;
import java.util.Random;

public class Check06 {
    public static void main(String[] args) {
        Stats stats = new Stats(30, 8, 3);
        Fighter aria = Fighter.ofHero(new Hero("Aria", stats),
                new ArmorBonus(new BaseStats(stats), 1));
        aria.addShield(3);
        Damage hit = new Damage(10, DamageType.PHYSICAL);
        System.out.println(new DamagePipeline(new Random(1), 1.0).
    ↪ apply(aria, hit)
                + " " + aria.shield());
        DamagePipeline plain = new DamagePipeline(new Random(1), 0.0);
        System.out.println(plain.apply(aria, hit) + " " + aria.shield
    ↪ ());

        Fighter rat = Fighter.ofEnemy(new Enemy("Rata de ceniza",
    ↪ EnemyKind.ASH_RAT,
                new Stats(8, 5, 0), new FirstAliveAi(), List.of()));
        System.out.println(plain.apply(rat, new Damage(10, DamageType.
    ↪ FIRE)));
        System.out.println(plain.apply(rat, hit));
    }
}
```

Salida esperada

```
0 3
6 0
5
10
```

2. Predice antes de ejecutar, y luego comprueba. (a) Un `Damage(10, FIRE)` contra Aria (armadura 1) con 3 de escudo y esquiva 0. (b) El mismo golpe contra la Rata de ceniza con 4 de escudo.

3. Con los mismos comandos del apartado anterior, la cripta acaba en la ronda 3 con la ronda 1 así: Aria 8, Borin 6 y Cael 9 al Esqueleto (que cae), la Rata de ceniza a

Cael con 2 y el Zombi a Aria con 1. Con `-args="2 default"` (caverna), la ronda 1 termina con el Goblin y el Murciélago fuera de combate y la Araña golpeando a Cael con 4; la partida acaba en la ronda 2.

Pista: *Las constantes del equipo y los pasos de montaje van en `GameSession`; no hace falta ninguna clase nueva para el equipo. Para (a) y (b), sigue la tubería eslabón a eslabón con lápiz y papel, como en el golpe de ejemplo del texto.*

Proyecto ▪ solución en la pág. 193

6.11 Resumen

Resumen

- El olor de Decorator es una explosión de subclases para combinar características independientes. Decorator envuelve un objeto en capas que implementan su misma interfaz y guardan lo envuelto; las capas se apilan y se pueden quitar.
- Un decorador es un `StatSource` y tiene un `StatSource`. Cada capa cambia una cifra y delega el resto. Son inmutables: «cambiar» una cadena es construir otra, y `tick()` devuelve la cadena con un turno menos.
- `Fighter` guarda el equipo permanente (`gear`) y la cadena actual (`stats` = `gear` más buffs) por separado.
- El olor de Chain of Responsibility es un método con una fila de `if` que modifican o cortan un valor, con el orden escondido. La cadena da un eslabón a cada regla.
- Un eslabón o bien modifica y pasa al siguiente, o bien corta y devuelve su resultado sin llamarlo. El último guarda un final que devuelve el daño.
- El orden es parte del diseño: lo que corta va delante (la esquiva, para no gastar escudo en un golpe fallado) y el escudo absorbe puntos en bruto antes de la armadura.
- Decorator siempre delega; Chain puede cortar; Strategy elige uno. La pista en el código es el `return` que no llama al siguiente.
- La `DamageFormula` resta la defensa (estadística) y `ArmorHandler` resta la armadura (equipo): son dos mitigaciones distintas, declaradas a propósito.
- H7 y H8: `engine/stats/`, `engine/combat/defense/`, `Damage`, `DamageType`, `Fighter` con `gear`, `stats`, `shield` y `resists`, `EnemyKind.resistances()` y `Combat` con un `DamagePipeline`.

Soluciones de los ejercicios

Solución del ejercicio 6.1 enunciado en la pág. 178

Siete archivos sin paquete, en la misma carpeta.

TextSource.java

```java
public interface TextSource {
    String text();
}
```

PlainText.java

```java
public class PlainText implements TextSource {
    private final String value;

    public PlainText(String value) {
        this.value = value;
    }

    @Override
    public String text() {
        return value;
    }
}
```

TextDecorator.java

```java
public abstract class TextDecorator implements TextSource {
    protected final TextSource inner;

    protected TextDecorator(TextSource inner) {
        this.inner = inner;
    }
}
```

Trim.java

```java
public class Trim extends TextDecorator {
    public Trim(TextSource inner) {
        super(inner);
    }

    @Override
    public String text() {
        return inner.text().trim();
    }
}
```

Upper.java

```java
public class Upper extends TextDecorator {
    public Upper(TextSource inner) {
        super(inner);
    }

    @Override
    public String text() {
        return inner.text().toUpperCase();
    }
}
```

Prefix.java

```java
public class Prefix extends TextDecorator {
    private final String prefix;

    public Prefix(TextSource inner, String prefix) {
        super(inner);
        this.prefix = prefix;
    }

    @Override
    public String text() {
        return prefix + inner.text();
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        TextSource raw = new PlainText("  hola  ");
        TextSource a = new Prefix(new Upper(new Trim(raw)), ">> ");
        TextSource b = new Trim(new Upper(new Prefix(raw, ">> ")));
        System.out.println("[" + a.text() + "]");
        System.out.println("[" + b.text() + "]");
    }
}
```

Por qué funciona. Cada decorador es un `TextSource` y guarda otro, así que se pueden apilar en cualquier orden sin que el resto lo sepa. En `a`, el texto se recorta primero (`hola`), se pasa a mayúsculas (`HOLA`) y por último se le pone el prefijo: `>> HOLA`. En `b`, el prefijo se añade el primero: `>>  hola  ` (el prefijo, un espacio, y los dos espacios del texto original), después se pasa a mayúsculas y, al final, `trim()` solo quita los espacios del extremo derecho (el izquierdo ya es un `>`): queda `>>  HOLA`, con tres espacios entre el prefijo y el texto. El orden de apilado cambia el resultado.

Errores típicos:

- Olvidar el `super(inner)` en un decorador concreto: no compila, porque `TextDecorator` no tiene constructor sin argumentos.
- Hacer que `Upper` modifique el `PlainText` en lugar de envolverlo: pierdes la posibilidad

de combinar y de cambiar el orden.

- Esperar que `trim()` quite los espacios «de dentro»: solo limpia los extremos.

Solución del ejercicio 6.2 enunciado en la pág. 178

Cinco archivos sin paquete.

CommandValidator.java

```java
public interface CommandValidator {
    String validate(String command);   // null = válido
}
```

NotEmptyValidator.java

```java
public class NotEmptyValidator implements CommandValidator {
    private final CommandValidator next;

    public NotEmptyValidator(CommandValidator next) {
        this.next = next;
    }

    @Override
    public String validate(String command) {
        if (command == null || command.isBlank()) {
            return "El comando está vacío";
        }
        return next.validate(command);
    }
}
```

MaxLengthValidator.java

```java
public class MaxLengthValidator implements CommandValidator {
    private final int max;
    private final CommandValidator next;

    public MaxLengthValidator(int max, CommandValidator next) {
        this.max = max;
        this.next = next;
    }

    @Override
    public String validate(String command) {
        if (command.length() > max) {
            return "El comando supera " + max + " caracteres";
        }
        return next.validate(command);
    }
}
```

OnlyLettersValidator.java

```java
public class OnlyLettersValidator implements CommandValidator {
    private final CommandValidator next;

    public OnlyLettersValidator(CommandValidator next) {
        this.next = next;
    }

    @Override
    public String validate(String command) {
        for (char c : command.toCharArray()) {
            if (!Character.isLetter(c)) {
                return "Solo se admiten letras";
            }
        }
        return next.validate(command);
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        CommandValidator chain = new NotEmptyValidator(
                new MaxLengthValidator(8,
                        new OnlyLettersValidator(command -> null)));
        String[] commands = {"", "atacar", "ataquemuyfuerte", "ata2", "
    ↪ ataque2muy"};
        for (String command : commands) {
            String error = chain.validate(command);
            System.out.println("'" + command + "' -> " + (error == null ? "
    ↪ ok" : error));
        }
    }
}
```

Por qué funciona. Cada eslabón mira una sola regla. Si falla, devuelve su mensaje *sin llamar* al siguiente: ese es el corte, y por eso el resultado es siempre el primer error. Si pasa, delega. El final de la cadena (la lambda `command -> null`) hace que un comando que supera todas las reglas devuelva `null`, es decir, «válido». `"ataque2muy"` falla la longitud (10 caracteres) y también las letras (el 2), pero la longitud va antes en la cadena, y las letras nunca se comprueban.

Errores típicos:

- Llamar siempre a `next.validate` y quedarte con el último mensaje: ya no es una cadena que corta, y el resultado sería el de la última regla que falla.
- Poner `NotEmptyValidator` detrás de `MaxLengthValidator`: con `null`, `command.length()` lanzaría una `NullPointerException`. El orden también protege a los eslabones siguientes.
- Olvidar el final de la cadena: el último eslabón haría `next.validate` sobre `null`.

Solución del ejercicio 6.3 enunciado en la pág. 179

1. `LoggedNotifier` es un **Decorator**; `FilteredNotifier` es un eslabón de una **Chain of Responsibility**.
2. En `LoggedNotifier`, la línea `inner.send(message);` se ejecuta *siempre*: el objeto añade comportamiento (el registro) y delega; no hay ningún camino que no llame a `inner`. En `FilteredNotifier`, la línea `return;` dentro del `if` *puede cortar*: si el mensaje contiene «spoiler», nunca llega a `next.send`. «Siempre delega» es Decorator; «puede cortar» es Chain.
3. Con `new LoggedNotifier(new FilteredNotifier(console))`:

   Registro por fuera

   ```
   Enviando: hola
   hola
   Enviando: spoiler: muere
   ```

   El registro está por fuera, así que anota los dos mensajes, incluso el que el filtro descarta después. Con `new FilteredNotifier(new LoggedNotifier(console))`:

   Filtro por fuera

   ```
   Enviando: hola
   hola
   ```

   El filtro corta el segundo mensaje antes de que llegue al registro, así que ni se anota ni se imprime. Es el mismo efecto que en la tubería de daño: lo que corta, delante, evita que los demás eslabones trabajen para nada.

Errores típicos: clasificar por el nombre del campo (`inner` frente a `next`): es solo una convención, la diferencia está en si `send` puede acabar sin llamar al otro objeto. Y creer que, como `LoggedNotifier` termina llamando a otro objeto, es una cadena: en una cadena, quien recibe la petición decide si seguir.

Solución del ejercicio 6.4 enunciado en la pág. 180

1. La salida original es `0 0`, `0 2` y `4 0`. El primer golpe (10, con esquiva al 100 %) primero pasa por la armadura (queda en 8) y luego por el escudo, que absorbe 4 y se gasta; *después* la esquiva lo corta y devuelve 0. El daño es 0, pero el escudo ya se ha perdido: la esquiva está al final, así que los eslabones anteriores ya han actuado y su efecto lateral (gastar escudo) no se deshace. La esquiva debe ir primero.
2. El montaje correcto es esquiva, escudo, armadura:

ChainBug.java (1/2: eslabones, ya corregido)

```
import java.util.Random;

public class ChainBug {
    interface Handler { int handle(Defender d, int damage); }

    static class Defender {
        int shield;
        final int armor;
        Defender(int shield, int armor) { this.shield = shield; this.armor
    ↪ = armor; }
    }

    static Handler dodge(Random random, double chance, Handler next) {
        return (d, dmg) -> random.nextDouble() < chance ? 0 : next.handle(d,
    ↪  dmg);
    }

    static Handler shield(Handler next) {
        return (d, dmg) -> {
            int absorbed = Math.min(d.shield, dmg);
            d.shield -= absorbed;
            int left = dmg - absorbed;
            return left == 0 ? 0 : next.handle(d, left);
        };
    }

    static Handler armor(Handler next) {
        return (d, dmg) -> next.handle(d, Math.max(1, dmg - d.armor));
    }

    static Handler pipeline(Random random, double chance) {
        Handler end = (d, dmg) -> dmg;
        return dodge(random, chance, shield(armor(end)));
    }
```

ChainBug.java (2/2: main)

```
    public static void main(String[] args) {
        Defender a = new Defender(4, 2);
        System.out.println(pipeline(new Random(1), 1.0).handle(a, 10) + " "
    ↪  + a.shield);
        Defender b = new Defender(3, 2);
        System.out.println(pipeline(new Random(1), 0.0).handle(b, 3) + " " +
    ↪  b.shield);
        Defender c = new Defender(4, 2);
        System.out.println(pipeline(new Random(1), 0.0).handle(c, 10) + " "
    ↪  + c.shield);
    }
}
```

3. Solo con el reordenamiento, la segunda línea pasa a `1 0`: el escudo (3) absorbe los 3 puntos del golpe, queda 0 y ese 0 llega a la armadura, que hace `Math.max(1, 0 -`

`2)` = 1. Un golpe que el escudo absorbió por completo acaba haciendo 1 de daño. El eslabón del escudo debe cortar cuando no queda nada que pasar: `left == 0 ? 0 : next.handle(d, left)`. Con eso, las tres líneas son `0 4`, `0 0` y `4 0`. En la tercera, 10 menos 4 de escudo deja 6, y 6 menos 2 de armadura deja 4: el orden de escudo y armadura no se nota ahí, pero sí en el segundo golpe, donde la armadura primero habría dejado el golpe en 1 y gastado solo 1 punto de escudo.

Errores típicos: dejar la esquiva al final y arreglar el síntoma devolviendo el escudo gastado (parche que repite la lógica); no ver que el `Math.max(1, ...)` de la armadura convierte un 0 en 1; y creer que reordenar basta sin volver a probar todos los casos: el orden nuevo destapó un fallo que el orden viejo escondía.

Solución del ejercicio 6.5

enunciado en la pág. 182

Parte del estado del capítulo 5. No cambian `Stats`, `Hero`, `HeroBuilder`, `Enemy`, `Skill`, `EnemyTemplate`, `TemplateRegistry`, el paquete `ai`, el paquete `damage` salvo los dos archivos nuevos, `Inventory`, `CryptBiome`, `CavernBiome`, el mundo, el mapa ni `ConsoleUi` (usa `f.stats().maxHp()`, que sigue existiendo en `StatSource`).

Parte 1 (H7). Paquete `engine/stats/`:

StatSource.java

```java
package emberdeep.engine.stats;

public interface StatSource {
    int attack();
    int defense();
    int maxHp();
    int armor();
    StatSource tick();
}
```

StatKind.java

```java
package emberdeep.engine.stats;

public enum StatKind { ATTACK, DEFENSE }
```

BaseStats.java

```java
package emberdeep.engine.stats;

import emberdeep.engine.character.Stats;

public record BaseStats(Stats stats) implements StatSource {
    @Override public int attack() { return stats.attack(); }
    @Override public int defense() { return stats.defense(); }
    @Override public int maxHp() { return stats.maxHp(); }
    @Override public int armor() { return 0; }
    @Override public StatSource tick() { return this; }
}
```

StatDecorator.java

```java
package emberdeep.engine.stats;

public abstract class StatDecorator implements StatSource {
    protected final StatSource inner;

    protected StatDecorator(StatSource inner) {
        this.inner = inner;
    }

    @Override public int attack() { return inner.attack(); }
    @Override public int defense() { return inner.defense(); }
    @Override public int maxHp() { return inner.maxHp(); }
    @Override public int armor() { return inner.armor(); }

    @Override
    public StatSource tick() {
        StatSource next = inner.tick();
        return next == inner ? this : rewrap(next);
    }

    protected abstract StatSource rewrap(StatSource newInner);
}
```

WeaponBonus.java

```java
package emberdeep.engine.stats;

public class WeaponBonus extends StatDecorator {
    private final int bonus;

    public WeaponBonus(StatSource inner, int bonus) {
        super(inner);
        this.bonus = bonus;
    }

    @Override
    public int attack() {
        return inner.attack() + bonus;
    }

    @Override
    protected StatSource rewrap(StatSource newInner) {
        return new WeaponBonus(newInner, bonus);
    }
}
```

ArmorBonus.java

```java
package emberdeep.engine.stats;

public class ArmorBonus extends StatDecorator {
    private final int bonus;

    public ArmorBonus(StatSource inner, int bonus) {
        super(inner);
        this.bonus = bonus;
    }

    @Override
    public int armor() {
        return inner.armor() + bonus;
    }

    @Override
    protected StatSource rewrap(StatSource newInner) {
        return new ArmorBonus(newInner, bonus);
    }
}
```

TempBuff.java

```
package emberdeep.engine.stats;

public class TempBuff extends StatDecorator {
    private final StatKind stat;
    private final int bonus;
    private final int turnsLeft;

    public TempBuff(StatSource inner, StatKind stat, int bonus, int
    ↪ turnsLeft) {
        super(inner);
        this.stat = stat;
        this.bonus = bonus;
        this.turnsLeft = turnsLeft;
    }

    @Override
    public int attack() {
        return inner.attack() + (stat == StatKind.ATTACK ? bonus : 0);
    }

    @Override
    public int defense() {
        return inner.defense() + (stat == StatKind.DEFENSE ? bonus : 0);
    }

    @Override
    public StatSource tick() {
        StatSource next = inner.tick();
        if (turnsLeft <= 1) {
            return next;
        }
        return new TempBuff(next, stat, bonus, turnsLeft - 1);
    }

    @Override
    protected StatSource rewrap(StatSource newInner) {
        return new TempBuff(newInner, stat, bonus, turnsLeft);
    }
}
```

engine/character/Fighter.java (después de la parte 1), en dos tramos seguidos dentro de la misma clase:

Fighter.java (1/2: datos y fábricas)

```java
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;
import emberdeep.engine.stats.BaseStats;
import emberdeep.engine.stats.StatKind;
import emberdeep.engine.stats.StatSource;
import emberdeep.engine.stats.TempBuff;
import java.util.List;

public class Fighter {
    private final String name;
    private final StatSource gear;     // equipo permanente
    private StatSource stats;          // gear más los buffs del combate
    private final EnemyAi ai;          // null en los héroes
    private int hp;

    private Fighter(String name, StatSource gear, EnemyAi ai) {
        this.name = name;
        this.gear = gear;
        this.stats = gear;
        this.ai = ai;
        this.hp = gear.maxHp();
    }

    public static Fighter ofHero(Hero hero, StatSource gear) {
        return new Fighter(hero.name(), gear, null);
    }

    public static Fighter ofEnemy(Enemy enemy) {
        return new Fighter(enemy.name(), new BaseStats(enemy.stats()),
    ↪ enemy.ai());
    }

    public String name() { return name; }
    public StatSource stats() { return stats; }
    public StatSource gear() { return gear; }
    public int hp() { return hp; }
    public boolean isAlive() { return hp > 0; }
```

Fighter.java (2/2: daño, buffs y objetivo)

```

    public void takeDamage(int amount) {
        hp = Math.max(0, hp - amount);
    }

    public void addBuff(StatKind stat, int bonus, int turns) {
        stats = new TempBuff(stats, stat, bonus, turns);
    }

    public void tickBuffs() {
        stats = stats.tick();
    }

    public Fighter chooseTarget(List<Fighter> heroes) {
        if (ai == null) {
            throw new IllegalStateException(name + " no tiene IA: es un hé
↪ roe");
        }
        return ai.choose(this, heroes);
    }
}
```

engine/session/GameSession.java (después de la parte 1), en dos tramos:

GameSession.java (1/2: equipo y constructor)

```
package emberdeep.engine.session;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.stats.ArmorBonus;
import emberdeep.engine.stats.BaseStats;
import emberdeep.engine.stats.StatSource;
import emberdeep.engine.stats.WeaponBonus;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.List;

public class GameSession {
    private static final int SWORD_ATTACK = 2;
    private static final int ARMOR_VALUE = 1;

    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final Combat combat;
    private Inventory inventory;

    public GameSession(List<Hero> heroes, BiomeFactory biome,
                       List<EnemyKind> kinds, DamageFormula formula) {
        this.heroes = heroes.stream()
                .map(hero -> Fighter.ofHero(hero, gearOf(hero)))
                .toList();
        this.enemies = kinds.stream()
                .map(biome::enemy)
                .map(Fighter::ofEnemy)
                .toList();
        this.combat = new Combat(this.heroes, this.enemies, formula);
```

GameSession.java (2/2: inventario y accesos)

```
        this.inventory = new Inventory(List.of())
                .with(biome.item(ItemKind.HEALING))
                .with(biome.item(ItemKind.HEALING))
                .with(biome.item(ItemKind.BOMB));
    }

    private static StatSource gearOf(Hero hero) {
        return new ArmorBonus(
                new WeaponBonus(new BaseStats(hero.stats()), SWORD_ATTACK),
                ARMOR_VALUE);
    }

    public List<Fighter> heroes() { return heroes; }
    public List<Fighter> enemies() { return enemies; }
    public Combat combat() { return combat; }
    public int round() { return combat.round(); }
    public Inventory inventory() { return inventory; }

    public void replaceInventory(Inventory next) {
        this.inventory = next;
    }
}
```

Por qué funciona (parte 1). `BaseStats` devuelve `this` en `tick()`, `StatDecorator` devuelve la misma capa si lo de dentro no cambió, y solo `TempBuff` cambia la cadena: con 2 turnos queda una cadena con un `TempBuff` de 1 turno y, con 1, devuelve la cadena interior ya avanzada, sin el buff. Por eso `Check06` imprime `14`, `14` y, tras el segundo `tick()`, `10` `1`. En `gearOf`, el orden de las capas no importa para la suma (la espada toca el ataque y la armadura la armadura), pero se escribe de dentro afuera: base, espada, armadura. `Fighter.stats()` devuelve un `StatSource` cuyos métodos se llaman como los de `Stats`, así que `Combat` (`attacker.stats().attack()`) y `ConsoleUi` (`f.stats().maxHp()`) compilan sin tocarlos. En esta parte `armor()` aún no lo lee nadie: por eso la Rata de ceniza hace 3 a Cael y el Zombi 2 a Aria.

Parte 2 (H8). Daño y resistencias:

engine/combat/damage/Damage.java

```
package emberdeep.engine.combat.damage;

public record Damage(int amount, DamageType type) {}
```

engine/combat/damage/DamageType.java

```
package emberdeep.engine.combat.damage;

public enum DamageType { PHYSICAL, FIRE }
```

engine/character/EnemyKind.java

```java
package emberdeep.engine.character;

import emberdeep.engine.combat.damage.DamageType;
import java.util.Set;

public enum EnemyKind {
    SKELETON, ASH_RAT(Set.of(DamageType.FIRE)), ZOMBIE,    // cripta
    GOBLIN, BAT, SPIDER;                                   // caverna

    private final Set<DamageType> resistances;

    EnemyKind() {
        this(Set.of());
    }

    EnemyKind(Set<DamageType> resistances) {
        this.resistances = resistances;
    }

    public Set<DamageType> resistances() {
        return resistances;
    }
}
```

engine/character/Fighter.java (final), en dos tramos:

Fighter.java (1/2: datos y fábricas)

```
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.stats.BaseStats;
import emberdeep.engine.stats.StatKind;
import emberdeep.engine.stats.StatSource;
import emberdeep.engine.stats.TempBuff;
import java.util.List;

public class Fighter {
    private final String name;
    private final EnemyKind kind;      // null en los héroes
    private final StatSource gear;     // equipo permanente
    private StatSource stats;          // gear más los buffs del combate
    private final EnemyAi ai;          // null en los héroes
    private int hp;
    private int shield;

    private Fighter(String name, EnemyKind kind, StatSource gear, EnemyAi
    ↪ ai) {
        this.name = name;
        this.kind = kind;
        this.gear = gear;
        this.stats = gear;
        this.ai = ai;
        this.hp = gear.maxHp();
    }

    public static Fighter ofHero(Hero hero, StatSource gear) {
        return new Fighter(hero.name(), null, gear, null);
    }

    public static Fighter ofEnemy(Enemy enemy) {
        return new Fighter(enemy.name(), enemy.kind(),
                new BaseStats(enemy.stats()), enemy.ai());
    }

    public String name() { return name; }
    public StatSource stats() { return stats; }
    public StatSource gear() { return gear; }
    public int hp() { return hp; }
    public boolean isAlive() { return hp > 0; }
```

Fighter.java (2/2: escudo, resistencias y buffs)

```
    public int shield() { return shield; }

    public void addShield(int amount) {
        shield += amount;
    }

    public int absorbShield(int damage) {
        int absorbed = Math.min(shield, damage);
        shield -= absorbed;
        return damage - absorbed;
    }

    public boolean resists(DamageType type) {
        return kind != null && kind.resistances().contains(type);
    }

    public void takeDamage(int amount) {
        hp = Math.max(0, hp - amount);
    }

    public void addBuff(StatKind stat, int bonus, int turns) {
        stats = new TempBuff(stats, stat, bonus, turns);
    }

    public void tickBuffs() {
        stats = stats.tick();
    }

    public Fighter chooseTarget(List<Fighter> heroes) {
        if (ai == null) {
            throw new IllegalStateException(name + " no tiene IA: es un hé
    ↪ roe");
        }
        return ai.choose(this, heroes);
    }
}
```

Los eslabones, en `engine/combat/defense/`:

DamageHandler.java

```
package emberdeep.engine.combat.defense;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.damage.Damage;

@FunctionalInterface
public interface DamageHandler {
    int handle(Fighter target, Damage damage);
}
```

DodgeHandler.java

```java
package emberdeep.engine.combat.defense;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.damage.Damage;
import java.util.Random;

public class DodgeHandler implements DamageHandler {
    private final Random random;
    private final double chance;
    private final DamageHandler next;

    public DodgeHandler(Random random, double chance, DamageHandler next) {
        this.random = random;
        this.chance = chance;
        this.next = next;
    }

    @Override
    public int handle(Fighter target, Damage damage) {
        if (chance > 0 && random.nextDouble() < chance) {
            return 0;
        }
        return next.handle(target, damage);
    }
}
```

ShieldHandler.java

```java
package emberdeep.engine.combat.defense;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.damage.Damage;

public class ShieldHandler implements DamageHandler {
    private final DamageHandler next;

    public ShieldHandler(DamageHandler next) {
        this.next = next;
    }

    @Override
    public int handle(Fighter target, Damage damage) {
        int left = target.absorbShield(damage.amount());
        if (left == 0) {
            return 0;
        }
        return next.handle(target, new Damage(left, damage.type()));
    }
}
```

ArmorHandler.java

```java
package emberdeep.engine.combat.defense;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.damage.Damage;

public class ArmorHandler implements DamageHandler {
    private final DamageHandler next;

    public ArmorHandler(DamageHandler next) {
        this.next = next;
    }

    @Override
    public int handle(Fighter target, Damage damage) {
        int reduced = Math.max(1, damage.amount() - target.stats().armor());
        return next.handle(target, new Damage(reduced, damage.type()));
    }
}
```

ResistanceHandler.java

```java
package emberdeep.engine.combat.defense;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.damage.Damage;

public class ResistanceHandler implements DamageHandler {
    private final DamageHandler next;

    public ResistanceHandler(DamageHandler next) {
        this.next = next;
    }

    @Override
    public int handle(Fighter target, Damage damage) {
        if (!target.resists(damage.type())) {
            return next.handle(target, damage);
        }
        int halved = Math.max(1, damage.amount() / 2);
        return next.handle(target, new Damage(halved, damage.type()));
    }
}
```

DamagePipeline.java

```java
package emberdeep.engine.combat.defense;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.damage.Damage;
import java.util.Random;

public class DamagePipeline {
    private final DamageHandler first;

    public DamagePipeline(Random random, double dodgeChance) {
        DamageHandler end = (target, damage) -> damage.amount();
        this.first = new DodgeHandler(random, dodgeChance,
                new ShieldHandler(
                        new ArmorHandler(
                                new ResistanceHandler(end))));
    }

    public int apply(Fighter target, Damage damage) {
        return first.handle(target, damage);
    }
}
```

engine/combat/Combat.java, en dos tramos seguidos dentro de la misma clase:

Combat.java (1/2: estado y turno de los héroes)

```
package emberdeep.engine.combat;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.damage.Damage;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.combat.defense.DamagePipeline;
import java.util.ArrayList;
import java.util.List;

public class Combat {
    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final DamageFormula formula;
    private final DamagePipeline pipeline;
    private int round = 1;

    public Combat(List<Fighter> heroes, List<Fighter> enemies,
                  DamageFormula formula, DamagePipeline pipeline) {
        this.heroes = heroes;
        this.enemies = enemies;
        this.formula = formula;
        this.pipeline = pipeline;
    }

    public int round() {
        return round;
    }

    public boolean heroesWon() {
        return enemies.stream().noneMatch(Fighter::isAlive);
    }

    public boolean heroesLost() {
        return heroes.stream().noneMatch(Fighter::isAlive);
    }

    public boolean isOver() {
        return heroesWon() || heroesLost();
    }

    public List<String> heroAttack(Fighter hero, Fighter target) {
        List<String> messages = new ArrayList<>();
        hit(hero, target, damageOf(hero, target), messages);
        return messages;
    }
```

Combat.java (2/2: enemigos, golpe y auxiliares)

```
    public List<String> enemyPhase() {
        List<String> messages = new ArrayList<>();
        for (Fighter enemy : enemies) {
            if (!enemy.isAlive()) {
                continue;
            }
            if (heroesLost()) {
                break;
            }
            Fighter target = enemy.chooseTarget(heroes);
            hit(enemy, target, damageOf(enemy, target), messages);
        }
        round++;
        heroes.forEach(Fighter::tickBuffs);
        enemies.forEach(Fighter::tickBuffs);
        return messages;
    }

    private void hit(Fighter attacker, Fighter target, int raw, List<String>
    ↪  messages) {
        int damage = pipeline.apply(target, new Damage(raw, DamageType.
    ↪ PHYSICAL));
        target.takeDamage(damage);
        if (damage == 0) {
            messages.add(attacker.name() + " ataca a " + target.name()
                    + ", pero no le hace daño.");
        } else {
            messages.add(attacker.name() + " ataca a " + target.name()
                    + " y le hace " + damage + " de daño.");
        }
        if (!target.isAlive()) {
            messages.add(target.name() + " queda fuera de combate.");
        }
    }

    private int damageOf(Fighter attacker, Fighter target) {
        return formula.damage(attacker.stats().attack(), target.stats().
    ↪ defense());
    }
}
```

engine/session/GameSession.java (final), en dos tramos:

GameSession.java (1/2: equipo y constructor)

```
package emberdeep.engine.session;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.stats.ArmorBonus;
import emberdeep.engine.stats.BaseStats;
import emberdeep.engine.stats.StatSource;
import emberdeep.engine.stats.WeaponBonus;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.List;

public class GameSession {
    private static final int SWORD_ATTACK = 2;
    private static final int ARMOR_VALUE = 1;

    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final Combat combat;
    private Inventory inventory;

    public GameSession(List<Hero> heroes, BiomeFactory biome,
                       List<EnemyKind> kinds, DamageFormula formula,
                       DamagePipeline pipeline) {
        this.heroes = heroes.stream()
                .map(hero -> Fighter.ofHero(hero, gearOf(hero)))
                .toList();
        this.enemies = kinds.stream()
                .map(biome::enemy)
                .map(Fighter::ofEnemy)
                .toList();
        this.combat = new Combat(this.heroes, this.enemies, formula,
    ↪ pipeline);
```

GameSession.java (2/2: inventario y accesos)

```
        this.inventory = new Inventory(List.of())
                .with(biome.item(ItemKind.HEALING))
                .with(biome.item(ItemKind.HEALING))
                .with(biome.item(ItemKind.BOMB));
    }

    private static StatSource gearOf(Hero hero) {
        return new ArmorBonus(
                new WeaponBonus(new BaseStats(hero.stats()), SWORD_ATTACK),
                ARMOR_VALUE);
    }

    public List<Fighter> heroes() { return heroes; }
    public List<Fighter> enemies() { return enemies; }
    public Combat combat() { return combat; }
    public int round() { return combat.round(); }
    public Inventory inventory() { return inventory; }

    public void replaceInventory(Inventory next) {
        this.inventory = next;
    }
}
```

app/Main.java, en tres tramos seguidos dentro de la misma clase:

Main.java (1/3: imports y arranque)

```java
package emberdeep.app;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.Dungeon;
import emberdeep.engine.world.DungeonBuilder;
import emberdeep.engine.world.Floor;
import emberdeep.engine.world.biome.BiomeFactory;
import emberdeep.engine.world.biome.CavernBiome;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.engine.world.map.MapData;
import emberdeep.engine.world.map.MapParser;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.ConsoleUi;
import java.util.List;
import java.util.Random;
import java.util.Scanner;

public class Main {
    private final GameConfig config;

    private Main(GameConfig config) {
        this.config = config;
    }

    public static void main(String[] args) {
        GameConfig config = new GameConfig(42L, 3, new FlatFormula());
        new Main(config).play(args);
    }
```

Main.java (2/3: la partida y el modo mapa)

```
    private void play(String[] args) {
        int floorNumber = chooseFloor(args);
        ConsoleUi ui = new ConsoleUi(new Scanner(System.in), System.out);
        Random random = new Random(config.seed());

        BiomeFactory crypt = new CryptBiome(random);
        BiomeFactory cavern = new CavernBiome(random);
        List<EnemyKind> cryptKinds = List.of(
                EnemyKind.SKELETON, EnemyKind.ASH_RAT, EnemyKind.ZOMBIE);
        List<EnemyKind> cavernKinds = List.of(
                EnemyKind.GOBLIN, EnemyKind.BAT, EnemyKind.SPIDER);

        Dungeon dungeon = new DungeonBuilder()
                .floor().biome(crypt).enemies(cryptKinds)
                        .map(MapData.floor(1)).done()
                .floor().biome(cavern).enemies(cavernKinds)
                        .map(MapData.floor(2)).done()
                .floor().biome(crypt).enemies(cryptKinds)
                        .map(MapData.floor(3)).done()
                .build();

        if (args.length > 1 && args[1].equals("map")) {
            explore(ui, dungeon, floorNumber);
            return;
        }
        DamageFormula formula = chooseFormula(ui, args);
        Floor floor = dungeon.floors().get(floorNumber - 1);
        GameSession session = new GameSession(heroes(), floor.biome(),
                floor.enemies(), formula, new DamagePipeline(random, 0.0));
        ui.run(session);
    }

    private void explore(ConsoleUi ui, Dungeon dungeon, int firstFloor) {
        TileTypes types = new TileTypes();
        MapParser parser = new MapParser(types);
        int number = firstFloor;
        while (true) {
            Floor floor = dungeon.floors().get(number - 1);
            boolean descended = ui.explore(parser.parse(floor.map()), types)
    ↪ ;
            if (!descended || number == dungeon.floors().size()) {
                return;
            }
            number++;
        }
    }
```

Main.java (3/3: auxiliares y llave final)

```
1
2       private DamageFormula chooseFormula(ConsoleUi ui, String[] args) {
3           if (args.length > 1 && args[1].equals("default")) {
4               return config.defaultFormula();
5           }
6           return ui.askFormula() == 1 ? new FlatFormula() : new ScaledFormula
        ↪ ();
7       }
8
9       private int chooseFloor(String[] args) {
10          int floor = args.length > 0 ? Integer.parseInt(args[0]) : 1;
11          if (floor < 1 || floor > config.floors()) {
12              throw new IllegalArgumentException(
13                      "La planta debe estar entre 1 y " + config.floors());
14          }
15          return floor;
16      }
17
18      private List<Hero> heroes() {
19          return List.of(
20                  new HeroBuilder().name("Aria")
21                          .stats(new Stats(30, 8, 3)).build(),
22                  new HeroBuilder().name("Borin")
23                          .stats(new Stats(40, 6, 5)).build(),
24                  new HeroBuilder().name("Cael")
25                          .stats(new Stats(24, 9, 2)).build());
26      }
27  }
```

Por qué funciona (parte 2). Un golpe sigue el mismo camino siempre: `Combat` calcula el daño de la `DamageFormula` (que resta la *defensa* del defensor, ya con sus buffs), lo envuelve en un `Damage` físico y lo pasa por la tubería esquiva, escudo, armadura, resistencia. Cada eslabón o corta o modifica y pasa. Con esquiva 0 no se gasta ningún número aleatorio, por lo que, salvo el equipo, la partida es idéntica a la del capítulo anterior. Los enemigos tienen `armor()` igual a 0 (su cadena es solo `BaseStats`) y escudo 0, así que la tubería no les cambia el daño: de ahí que los héroes sigan haciendo a los enemigos lo que la fórmula dice. Los cuatro valores de `Check06` salen así: con esquiva 1.0 el primer eslabón corta y el escudo queda en 3 (`0 3`); con esquiva 0, el golpe de 10 pierde 3 en el escudo (7), 1 en la armadura (6) y no se resiste: `6 0`; el fuego de 10 contra la Rata, que lo resiste y no tiene armadura ni escudo, se parte a la mitad: 5; y un golpe físico de 10 contra la misma Rata no se resiste: 10. Las predicciones: (a) Aria con 3 de escudo recibiendo fuego 10: $10 - 3 = 7$, $7 - 1 = 6$, no resiste: **6**. (b) La Rata con 4 de escudo: $10 - 4 = 6$, armadura 0 deja 6, y la resistencia lo parte: **3**. Las trazas: en la cripta con fórmula plana, la Rata de ceniza hace 3 a Cael (5 menos su defensa 2) y la armadura lo deja en 2; el Zombi hace 2 a Aria (5 menos 3) y queda en 1. Termina en la ronda 3 con la vida de los héroes en 28, 40 y 22. En la caverna, la Araña hace 4 a Cael ($7 - 2 = 5$, menos la armadura) y todo acaba en la ronda 2. El mensaje «pero no le hace daño» solo sale cuando un eslabón deja el daño en 0; hoy, con la esquiva a 0, no ocurre en una partida real.

Dos decisiones que conviene tener claras: (1) `Fighter` guarda `gear` y `stats`, y nadie lee `gear` salvo `gear()`: es el equipo al que volverá `stats` cuando acabe un combate (capítulo 12). (2) `Combat` avanza los buffs al final de cada ronda para todos los combatientes; hasta el

capítulo 7 ninguna acción añade buffs, pero la maquinaria ya funciona.

Errores típicos:

- Poner la esquiva detrás del escudo (o de la armadura): el escudo se gasta en golpes que no hacían daño (ejercicio 6.4).
- Hacer que `DodgeHandler` llame a `random.nextDouble()` aunque `chance` sea 0: la secuencia de números del `Random` compartido con `RandomTargetAi` cambiaría y las trazas de partidas sin esquiva dejarían de repetirse.
- Modificar una capa existente (por ejemplo, restar a `turnsLeft` en el sitio) en lugar de devolver una nueva: pierdes la inmutabilidad en la que se apoyará el guardado del estado.
- Olvidar que el final de la cadena (`end`) devuelve `damage.amount()`: sin él, el último eslabón no tendría a quién llamar.
- Dejar `ofHero(hero)` con un solo parámetro: `Fighter` necesita el equipo para saber su vida máxima y sus estadísticas desde el primer momento.
- Usar `Math.max(1, ...)` sin que el escudo corte primero: un golpe totalmente absorbido subiría a 1.

Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo 7: `Fighter` con `gear`, `stats`, `shield` y `addShield`, `engine/stats/`, `engine/combat/defense/` y `Combat` con tubería. Borra `app/Check06.java`.

CAPÍTULO 7

Command: acciones como objetos, deshacer y replay

Objetivos del capítulo

- Reconocer el olor que pide un Command: acciones repartidas en un `switch` por tecla, que no se pueden guardar, deshacer ni repetir.
- Aplicar Command con una interfaz, comandos concretos que llevan sus datos, un receptor que cambia de verdad y un invocador que ejecuta.
- Deshacer con una pila (`Deque`): cada comando guarda, antes de ejecutarse, lo que va a tocar.
- Repetir una partida desde un guion de teclas y explicar por qué la semilla hace el replay exacto.
- Dar el hito H9: acciones de héroes y enemigos como comandos, deshacer con `u` y modo guion.

Hasta ahora, un héroe de Emberdeep solo sabe hacer una cosa: atacar. En este capítulo le das cuatro (atacar, defenderse, usar un objeto, lanzar una habilidad) y haces que los enemigos usen las mismas. Después querrás tres cosas más: que el jugador pueda deshacer su última acción, que una partida entera se pueda repetir tecla a tecla y que la IA de un enemigo elija entre acciones sin que `Combat` sepa cuáles hay. Las tres salen del mismo cambio: convertir cada acción en un objeto. Lo harás en tres pasos pequeños, uno por cada cosa: primero *ejecutar*, luego *deshacer* y por último *repetir*.

7.1 El olor que pide Command: acciones que no se pueden guardar

Imagina el menú del héroe con las cuatro acciones y cada tecla haciendo su trabajo en el sitio (el código es hipotético; no existe en el proyecto):

Hipotético: cada tecla hace su trabajo en el menú

```
switch (key) {
    case "1" -> {                                       // atacar
        int raw = formula.damage(hero.stats().attack(),
                                 target.stats().defense());
        target.takeDamage(pipeline.apply(target,
                new Damage(raw, DamageType.PHYSICAL)));
    }
    case "2" -> {                                       // defender
        hero.addShield(5);
        hero.addBuff(StatKind.DEFENSE, 2, 1);
    }
    case "3" -> {                                       // usar un objeto
```

```
            session.replaceInventory(session.inventory().without(item));
            // curar o hacer daño de fuego, según el objeto
        }
        case "4" -> { /* habilidad: otra rama con su propio daño */ }
    }
```

Y la fase de enemigos tendría que repetir las mismas ideas con otras condiciones:

Hipotético: el enemigo, otra vez

```
if (enemy.name().equals("Zombi") && round % 3 == 0) {
    // su habilidad: el mismo cálculo de daño, copiado
} else {
    // su ataque: el mismo cálculo de daño, copiado otra vez
}
```

Funciona, y es corto. Pero tiene tres problemas que crecen juntos:

1. **Una acción no es un valor.** Existe mientras el `switch` la ejecuta y desaparece. No puedes guardarla en una lista, pasarla a otro método ni preguntarle qué ha hecho.
2. **Jugador y enemigo duplican el trabajo.** Atacar es lo mismo para un héroe que para un esqueleto, pero el código vive en dos sitios, y cada acción nueva (un veneno, una invocación) hay que escribirla dos veces.
3. **Deshacer y repetir no tienen dónde agarrarse.** Para deshacer un ataque necesitarías saber qué rama se ejecutó y con qué valores; para repetir una partida, que alguien anotara cada decisión. Con el `switch` no queda rastro de ninguna.

Este es el olor: **acciones que son código disperso y no objetos**. Command lo cura convirtiendo cada acción en un objeto. Su intención en el catálogo es encapsular una petición como un objeto, de modo que puedas parametrizar a los clientes con peticiones distintas, ponerlas en cola o registrarlas, y soportar operaciones que se pueden deshacer [9]. Nystrom lo resume de forma más corta: un comando es una llamada a método convertida en objeto, algo que puedes guardar en una variable o pasar a una función [19]. Head First lo explica con un mando a distancia cuyos botones guardan comandos, y le dedica un capítulo [7]. En Emberdeep ganas tres cosas: las acciones de héroes y enemigos pasan por el mismo cauce, una pila de comandos permite deshacer, y la lista de teclas de una partida basta para repetirla.

7.2 Piezas del patrón Command

Rol	Clase en el ejemplo	Propósito	Paquete
Comando (interfaz)	`CombatCommand`	Contrato: `execute()` hace la acción y devuelve los mensajes; `undo()` la revierte	`engine.combat.command`
Comandos concretos (clases)	`Attack`, `Defend`, `UseItem`, `UseSkill`	Una acción con sus datos y lo que guarda para deshacerla	`engine.combat.command`
Receptor (clases)	`Fighter` (vida, escudo, buffs) y `GameSession` (inventario)	Quienes cambian de verdad; el comando solo les pide cosas	`engine.character`, `engine.session`
Invocador (clases)	`Combat` con su `CommandHistory`	Ejecuta el comando sin saber cuál es y lo apila	`engine.combat`
Cliente (clases)	`ConsoleUi` (menú del jugador) y `EnemyAi` (`act`)	Deciden qué comando crear, con qué datos	`ui`, `engine.combat.ai`

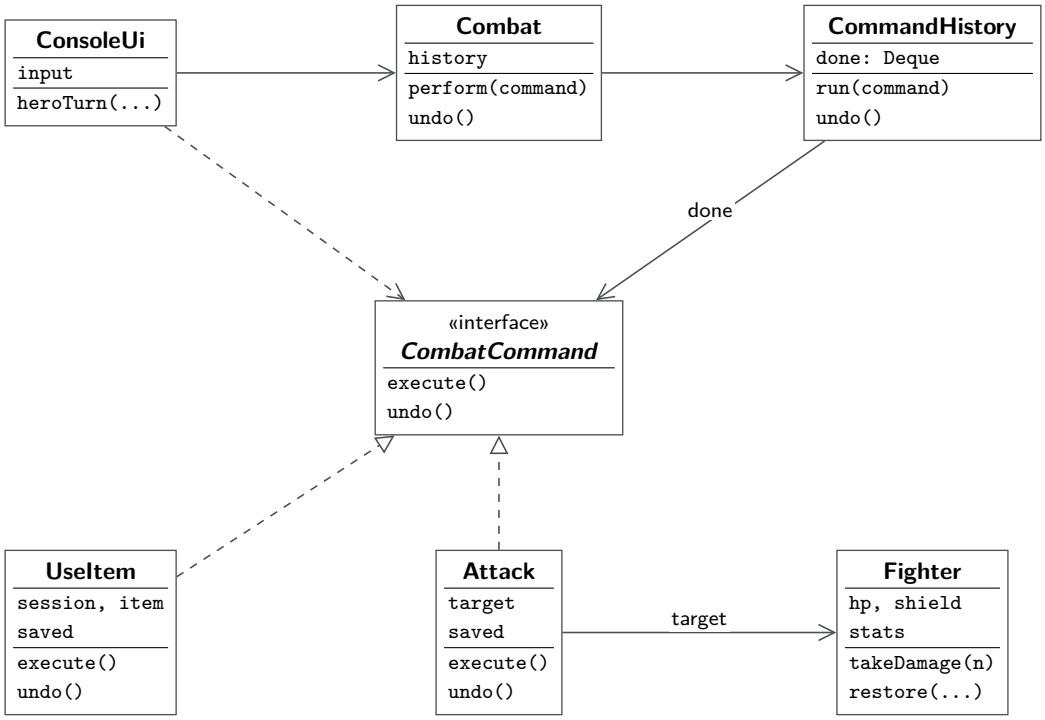


Diagrama UML del patrón Command: el cliente crea comandos, el invocador los ejecuta y apila, y cada comando actúa sobre su receptor.

7.3 El ejemplo de Command, pieza a pieza (1): ejecutar

El contrato. Por ahora solo hace falta una operación. `execute()` devuelve los mensajes de lo ocurrido, porque el motor sigue sin imprimir nada: quien pinta es la interfaz.

engine/combat/command/CombatCommand.java (versión 1)

```
package emberdeep.engine.combat.command;

import java.util.List;

public interface CombatCommand {
    List<String> execute();
}
```

Qué necesita el receptor. Para atacar, un comando tiene que calcular el daño con la `DamageFormula` y pasarlo por la tubería del capítulo 6. Ambas viven en `Combat`. Dos métodos que hasta ahora eran privados pasan a ser públicos: uno para la fórmula (`damageOf`) y otro para golpear (`deal`), que pasa el golpe por la tubería y aplica lo que sale:

Fragmento de Combat: lo que usan los comandos

```
public int damageOf(Fighter attacker, Fighter target) {
    return formula.damage(attacker.stats().attack(),
                          target.stats().defense());
}

public int deal(Fighter target, Damage damage) {
    int dealt = pipeline.apply(target, damage);
    target.takeDamage(dealt);
    return dealt;
}
```

Un comando concreto. `Attack` guarda quién ataca, a quién y el `Combat` en el que ocurre. Sus mensajes son los de siempre, los que antes escribía `Combat`: la partida se ve igual. Fíjate en que el objeto *ya contiene* todo lo necesario para ejecutarse; quien lo ejecuta no tiene que saber nada más.

engine/combat/command/Attack.java (versión 1)

```
package emberdeep.engine.combat.command;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.Damage;
import emberdeep.engine.combat.damage.DamageType;
import java.util.ArrayList;
import java.util.List;

public class Attack implements CombatCommand {
    private final Combat combat;
    private final Fighter attacker;
    private final Fighter target;

    public Attack(Combat combat, Fighter attacker, Fighter target) {
        this.combat = combat;
        this.attacker = attacker;
        this.target = target;
    }

```

```
    @Override
    public List<String> execute() {
        int raw = combat.damageOf(attacker, target);
        int dealt = combat.deal(target,
                new Damage(raw, DamageType.PHYSICAL));
        String who = attacker.name() + " ataca a " + target.name();
        List<String> messages = new ArrayList<>();
        messages.add(dealt == 0 ? who + ", pero no le hace daño."
                : who + " y le hace " + dealt + " de daño.");
        if (!target.isAlive()) {
            messages.add(target.name() + " queda fuera de combate.");
        }
        return messages;
    }
}
```

Defender. Aquí el capítulo 6 da fruto. `Defend` usa el `TempBuff` de defensa que ya existe y la fuente de escudo que ese capítulo dejó sin conectar a ninguna acción: suma 5 puntos de escudo con `addShield` y 2 de defensa durante un turno. Ese turno es la ronda en curso: `Combat` avanza los buffs al cerrar la ronda, después de que actúen los enemigos.

engine/combat/command/Defend.java (versión 1)

```
package emberdeep.engine.combat.command;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.stats.StatKind;
import java.util.List;

public class Defend implements CombatCommand {
    private static final int SHIELD = 5;
    private static final int DEFENSE_BONUS = 2;

    private final Fighter user;

    public Defend(Fighter user) {
        this.user = user;
    }

    @Override
    public List<String> execute() {
        user.addShield(SHIELD);
        user.addBuff(StatKind.DEFENSE, DEFENSE_BONUS, 1);
        return List.of(user.name() + " se cubre: +" + SHIELD
                + " de escudo y +" + DEFENSE_BONUS
                + " de defensa este turno.");
    }
}
```

Habilidades y objetos. `UseSkill(combat, user, skill, target)` se escribe como `Attack`, con una diferencia: el daño no sale de la fórmula, sale de la propia habilidad, que desde este capítulo es un `record Skill(String name, int power, DamageType type)`. Su `execute()` llama a `combat.deal` con `new Damage(skill.power(), skill.type())`. Es lo que permite una llamarada de fuego (`FIRE`) contra una Rata de ceniza que lo resiste. `UseItem(session, user, target, item)` cura con una poción o hace daño de fuego con una bomba, y además

gasta el objeto; lo verás entero en la sección siguiente, porque ahí está lo interesante.

El cliente. El menú del héroe no ejecuta nada por su cuenta: *construye* el comando que corresponde a la tecla y se lo entrega a quien lo ejecute. Desaparece el trabajo de cada rama y queda el trabajo de elegir:

Fragmento de ConsoleUi: el menú crea comandos

```
CombatCommand command = switch (key) {
    case "1" -> new Attack(combat, hero,
            askTarget("Objetivo", session.enemies()));
    case "2" -> new Defend(hero);
    case "3" -> chooseItem(session, hero);
    case "4" -> chooseSkill(session, hero);
    default -> null;
};
```

Quién elige la acción del enemigo. Hasta ahora la IA de un enemigo solo respondía a una pregunta: ¿a quién ataco? (`choose`). Ahora responde a otra más amplia: ¿qué hago? `EnemyAi` gana un método `act` que *devuelve un comando*. Su implementación por defecto es atacar al objetivo que elige `choose`, así que `FirstAliveAi`, `LowestHpAi` y `RandomTargetAi` no cambian ni una línea. `SkillfulAi` es la novedad: envuelve otra IA (la que elige el objetivo) y, cada tercera ronda, usa la primera habilidad del enemigo en lugar de atacar. La IA *decide*; `Combat` solo ejecuta lo que le dan, igual que con el jugador. Este es el uso que Nystrom señala para la IA: que emita comandos, igual que el jugador, y que el actor los cumpla sin saber quién los eligió [19].

engine/combat/ai/EnemyAi.java y SkillfulAi.java (fragmentos)

```
// EnemyAi: por defecto, atacar al objetivo que elige choose
default CombatCommand act(Combat combat, Fighter self,
                          List<Fighter> heroes) {
    return new Attack(combat, self, choose(self, heroes));
}

// SkillfulAi: objetivo con otra IA; cada tercera ronda, su habilidad
@Override
public CombatCommand act(Combat combat, Fighter self,
                         List<Fighter> heroes) {
    Fighter target = choose(self, heroes);
    if (combat.round() % 3 == 0 && !self.skills().isEmpty()) {
        return new UseSkill(combat, self, self.skills().get(0), target);
    }
    return new Attack(combat, self, target);
}
```

7.4 Command con deshacer (2): la pila

Si una acción es un objeto, deshacerla es pedirle que se deshaga. La interfaz gana una segunda operación, y cada comando se hace responsable de guardar lo que necesita:

engine/combat/command/CombatCommand.java (versión 2)

```java
public interface CombatCommand {
    List<String> execute();
    void undo();
}
```

Guardar antes de tocar. La regla es una sola: *al principio de `execute()`, el comando apunta el estado que va a modificar; `undo()` lo devuelve*. Un ataque puede cambiar la vida del objetivo y gastar su escudo; una defensa cambia escudo y buffs. Para devolverlo todo, `Fighter` gana un método que fija sus tres números a la vez:

Fragmento de Fighter: restore y heal

```java
public void restore(int hp, int shield, StatSource stats) {
    this.hp = hp;
    this.shield = shield;
    this.stats = stats;
}

// Cura hasta el máximo y devuelve lo que ha curado de verdad
public int heal(int amount) {
    int healed = Math.min(amount, stats.maxHp() - hp);
    hp += healed;
    return healed;
}
```

Así queda `Attack` con deshacer. Las líneas nuevas son los tres campos que guardan el estado de antes, su captura al principio y el `undo()`:

Attack.java (versión 2: solo lo que cambia)

```java
private int hpBefore;
private int shieldBefore;
private StatSource statsBefore;

@Override
public List<String> execute() {
    hpBefore = target.hp();                // se apunta ANTES de tocar nada
    shieldBefore = target.shield();
    statsBefore = target.stats();
    // ... el cálculo y los mensajes de antes ...
}

@Override
public void undo() {
    target.restore(hpBefore, shieldBefore, statsBefore);
}
```

Guardar `statsBefore` cuesta casi nada: la cadena de estadísticas del capítulo 6 está hecha de decoradores inmutables, así que apuntar la referencia *es* congelar los buffs; nadie la modificará después. Es la técnica que Nystrom llama estructuras de datos persistentes: cada comando guarda una referencia al objeto de antes y deshacer es volver a ponerlo [19]. El inventario del capítulo 3 también es inmutable, y `UseItem` lo aprovecha:

engine/combat/command/UseItem.java (execute y undo)

```
@Override
public List<String> execute() {
    inventoryBefore = session.inventory();      // la referencia, sin copiar
    hpBefore = target.hp();
    shieldBefore = target.shield();
    statsBefore = target.stats();
    session.replaceInventory(inventoryBefore.without(item));
    List<String> messages = new ArrayList<>();
    switch (item.kind()) {
        case HEALING -> messages.add(user.name() + " usa " + item.name()
                + " sobre " + target.name() + ": recupera "
                + target.heal(item.power()) + " de vida.");
        case BOMB -> { /* daño de fuego por la tubería, como UseSkill */ }
    }
    return messages;
}

@Override
public void undo() {
    session.replaceInventory(inventoryBefore);  // la poción vuelve
    target.restore(hpBefore, shieldBefore, statsBefore);
}
```

La pila. Deshacer lo último que se hizo, luego lo anterior, y así: es una pila, la estructura *último en entrar, primero en salir*. En Java sirve `Deque`, que tiene la implementación `ArrayDeque`. Cuatro métodos bastan: `push` añade por arriba; `pop` quita lo de arriba y lo devuelve; `peek` lo devuelve sin quitarlo; `isEmpty` comprueba si queda algo.

Fragmento: una pila con Deque

```
Deque<String> stack = new ArrayDeque<>();
stack.push("a");
stack.push("b");
stack.push("c");
System.out.println(stack.peek());   // c, sigue en la pila
System.out.println(stack.pop());    // c, ya fuera
System.out.println(stack.pop());    // b
System.out.println(stack.size());   // 1
```

Un error típico: llamar a `pop()` con la pila vacía, que lanza `NoSuchElementException`. Se comprueba antes con `isEmpty()`. `CommandHistory` es esa pila con dos operaciones y el propio invocador del patrón:

engine/combat/command/CommandHistory.java

```
package emberdeep.engine.combat.command;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.List;

public class CommandHistory {
    private final Deque<CombatCommand> done = new ArrayDeque<>();
```

```
9
10      // Ejecuta y, solo si no falla, apila
11      public List<String> run(CombatCommand command) {
12          List<String> messages = command.execute();
13          done.push(command);
14          return messages;
15      }
16
17      // false si no había nada que deshacer
18      public boolean undo() {
19          if (done.isEmpty()) {
20              return false;
21          }
22          done.pop().undo();
23          return true;
24      }
25
26      public int size() { return done.size(); }
27      public void clear() { done.clear(); }
28  }
```

`Combat` tiene un `CommandHistory` y dos métodos que lo delegan: `perform(command)` y `undo()`. Los héroes pasan por `perform`. Un caso concreto de la pila, con la acción de cada héroe en una ronda:

Se pulsa	Pila (arriba a la izquierda)	Qué ocurre
Aria: bomba	`[UseItem]`	Se gasta la bomba y baja la vida del enemigo
Borin: `u`	`[]`	`pop` y `undo()`: la bomba vuelve al inventario
Aria: atacar	`[Attack]`	Aria vuelve a decidir; ahora ataca
Borin: defender	`[Defend, Attack]`	Escudo y buff de Borin

Dónde termina el deshacer. Hay una decisión que conviene declarar. Los enemigos también actúan con comandos, pero `Combat` los ejecuta sin apilarlos, y cuando acaba la fase de enemigos, `CommandHistory` se vacía (`clear()`). La razón: al cerrar la ronda, `Combat` suma uno al contador y hace avanzar los buffs de todos, y eso *no es un comando*; ningún `undo()` sabría devolver el contador ni los turnos de buff de cada combatiente. Por eso `u` deshace las acciones de los héroes de la ronda en curso, y solo esas. Es un límite real de Command, y lo retomo en la sección 6.

7.5 Command y el modo guion (3): replay

Con las acciones hechas objetos, una partida es una lista de decisiones. Si registras las teclas que pulsa el jugador y mantienes todo lo demás igual, puedes volver a jugarla entera. Nystrom cuenta la idea sin rodeos: en vez de grabar el estado del juego en cada instante, que ocuparía demasiado, muchos juegos graban los comandos que cada entidad ejecutó y reproducen la partida ejecutándolos otra vez [19].

En Emberdeep el replay es más sencillo todavía, porque la entrada ya es una secuencia de teclas. Solo hay que decidir *de dónde* salen. Hoy salen del teclado y las lee `ConsoleUi` con un `Scanner`. Se separa esa lectura en una interfaz de un solo método, `InputSource`, con dos implementaciones:

ui/input/InputSource.java

```
package emberdeep.ui.input;

public interface InputSource {
    String next();   // la siguiente tecla que el jugador pulsa
}
```

`ConsoleInput` es lo que `ConsoleUi` hacía con el teclado: muestra el prompt `>` y devuelve la línea que se teclea. `ScriptInput` lee las teclas de un archivo de texto, una por línea, e ignora las líneas vacías y las que empiezan por `#` (comentarios del guion). Como nadie teclea, imprime cada tecla que consume, para que la salida se lea como una sesión real.

ui/input/ScriptInput.java

```
package emberdeep.ui.input;

import java.io.IOException;
import java.io.PrintStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

public class ScriptInput implements InputSource {
    private final List<String> keys;
    private final PrintStream out;
    private int position = 0;

    public ScriptInput(Path file, PrintStream out) throws IOException {
        this.keys = Files.readAllLines(file).stream()
                .map(String::trim)
                .filter(line -> !line.isEmpty() && !line.startsWith("#"))
                .toList();
        this.out = out;
    }

    @Override
    public String next() {
        if (position == keys.size()) {
            throw new IllegalStateException("El guion se ha acabado");
        }
        String key = keys.get(position++);
        out.println("> " + key);
        return key;
    }
}
```

Leer un archivo, lo mínimo

`Files.readAllLines(path)` lee todo un archivo de texto y devuelve una `List<String>` con sus líneas. `Path.of("scripts/demo.txt")` construye la ruta. Leer un archivo puede fallar, y Java obliga a declararlo: el constructor lleva `throws IOException`, y quien lo llama (`Main`) también. El capítulo 14 estudia `java.nio.file` con calma; aquí solo necesitas esa línea.

El guion es un archivo normal. Las primeras líneas de uno, con comentarios:

scripts/demo.txt (el principio)

```
# Ronda 1: Aria lanza la bomba al Zombi y Borin lo deshace
3
3
3
u
```

Por qué el replay es exacto. Dos ejecuciones con el mismo guion dan la misma partida porque no queda ninguna fuente de azar fuera de control: la entrada son las mismas teclas, la semilla del `Random` está fijada en `GameConfig` (42) y todas las IA comparten ese único generador, y los comandos son deterministas. Lo rompería cualquier cosa que no se repita: un `new Random()` sin semilla, `Math.random()`, la hora del sistema o recorrer un `HashSet` cuyo orden no esté garantizado. El ejercicio 7.2 te hace calcular a mano una partida con semilla y ver por qué la tercera ejecución, con otra semilla, es distinta.

7.6 Command: cuándo sí y cuándo no

Compensa cuando necesitas alguna de estas cosas: guardar, encolar o registrar acciones; deshacerlas; repetirlas; o que varios emisores (jugador, IA, un guion) usen el mismo cauce. En Emberdeep, la IA y el jugador ya comparten comandos, y el guion es un tercer emisor que no ha costado tocar el motor. Los comandos son piezas pequeñas que se prueban una a una.

No compensa si la acción se ejecuta y se olvida: ahí un objeto con `execute()` sobra. Con una interfaz de un solo método y sin deshacer, una lambda o un `Runnable` cumple el mismo papel [26]. El coste de Command es una clase por acción y, si deshaces, la disciplina de que *todo* cambio de estado pase por un comando: basta una modificación hecha por fuera para que `undo()` devuelva un estado que ya no es el de antes.

Acciones que no se pueden invertir. Tirar una moneda para curar, escribir un archivo, enviar un mensaje por red: no hay forma de hacer "lo contrario". Hay dos salidas honestas: guardar el valor anterior para volver a ponerlo (el ejercicio 7.4 las compara) o declarar la acción no deshacible y vaciar la pila al ejecutarla. La segunda es lo que hace `Combat` con el cierre de la ronda.

Avance del capítulo 8. Cada comando guarda *solo lo que va a tocar*. Eso es barato cuando las acciones son pequeñas, pero cuando cambia mucho a la vez, listar lo que se toca es un riesgo. La alternativa es guardar una instantánea del estado entero: es el patrón Memento. Nystrom prefiere guardar a mano solo los datos que cambian, porque una instantánea completa gasta memoria de más cuando los cambios son pequeños [19]; el capítulo siguiente compara las dos formas con tus propios datos. Para una ficha resumida con diagramas: [24].

7.7 En el proyecto: H9 con Command

Este capítulo avanza el hito H9. Son bastantes piezas, así que se hace en **tres pasos** que se comprueban uno a uno. Cambios del hito:

- **Nuevo paquete** `engine/combat/command/`: `CombatCommand`, `Attack`, `Defend`, `UseItem`, `UseSkill` y `CommandHistory`.
- **Nuevo paquete** `ui/input/` con `InputSource`, `ConsoleInput` y `ScriptInput`, y una carpeta `scripts/` con `demo.txt`. `ConsoleUi` ya no tiene `Scanner`: pide las teclas a un

`InputSource` y se queda con el menú y el pintado de mensajes.

- **Nuevo** `SkillfulAi` en `engine/combat/ai/`, y `EnemyAi` gana `act`.
- **Cambian** `Skill` (gana `type`), `Hero` (gana `skills`), `HeroBuilder` (paso `skill`), `Fighter` (`skills()`, `heal`, `restore` y `act`, que sustituye a `chooseTarget`), `Combat` (`perform`, `undo`, y `damageOf` y `deal` públicos; `heroAttack` y el privado `hit` desaparecen: los hace `Attack`), `EnemyTemplate` (la habilidad del élite lleva tipo), `CryptBiome` y `CavernBiome` (los datos de habilidad llevan tipo; en la cripta, el Zombi usa `SkillfulAi`), `ConsoleUi` y `Main`. `TemplateRegistry` no cambia: las plantillas con sus habilidades se registran en los dos biomas, y son ellos los que se actualizan.
- **No cambian** `GameSession` (ya tenía el inventario y su sustitución), `engine/stats/`, `engine/combat/defense/` ni el mapa.

Tres decisiones que el patrón te pide. (1) Los comandos reciben `Combat` por el constructor, porque necesitan la fórmula y la tubería; por eso `act` también lo recibe: la IA crea el comando y necesita saber en qué combate ocurrirá. (2) Los héroes tienen dos habilidades, iguales para los tres y definidas en `Main`, el composition root, que ya es donde se dan sus nombres y estadísticas: «Golpe poderoso» (12, físico) y «Llamarada» (10, fuego). (3) La historia de comandos se vacía al cerrar la ronda, como viste en la sección 4.

Qué cambia en la partida. Mucho menos de lo que parece, a propósito. Con `yes 1` (siempre atacar al primer enemigo) los mensajes son los del capítulo 6: `Attack` escribe lo mismo que antes escribía `Combat`, y las IA existentes siguen llamando a `choose` una vez por acción, sin gastar números del `Random` de más. El Zombi usa ahora `SkillfulAi`, pero esa partida acaba en la ronda 3, en la fase de los héroes, antes de que le toque usar su habilidad.

Los tres pasos y qué debes ver en cada uno.

- **Paso 1, ejecutar.** Las cuatro acciones existen como comandos y se ejecutan desde el menú; los enemigos también emiten comandos. Resultado: un programa de comprobación imprime los mensajes de un ataque, una habilidad de fuego (que contra la Rata de ceniza baja a la mitad), una defensa y una bomba, y la partida con `yes 1` es la de siempre.
- **Paso 2, deshacer.** Cada comando se deshace y `u` funciona en el menú. Resultado: tras deshacer una bomba, el inventario vuelve a tener tres objetos; deshacer sin acciones previas avisa; tras cerrar la ronda, ya no se puede deshacer.
- **Paso 3, replay.** Un guion de teclas juega la partida entera. Resultado: dos ejecuciones del mismo guion dan 108 líneas idénticas.

Hito 9: acciones como comandos, deshacer y modo guion

Tras este hito, cada acción del juego es un comando que se ejecuta, se apila y se puede deshacer, el jugador la elige en un menú de cuatro opciones, los enemigos emiten comandos con su IA y una partida se puede repetir desde un archivo de teclas. La estructura:

Estructura de carpetas tras el capítulo 7

```
emberdeep/
|-- settings.gradle
|-- build.gradle
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
|-- scripts/
|   `-- demo.txt
`-- src/main/java/emberdeep/
    |-- app/
    |   |-- Main.java              (composition root)
    |   `-- GameConfig.java
    |-- engine/
    |   |-- character/             Stats, Hero, HeroBuilder, Enemy,
    ↪ EnemyKind, Fighter,
    |   |                          Skill, EnemyTemplate, TemplateRegistry
    |   |-- stats/                 StatSource, StatKind, BaseStats,
    ↪ StatDecorator,
    |   |                          WeaponBonus, ArmorBonus, TempBuff
    |   |-- combat/
    |   |   |-- Combat.java
    |   |   |-- ai/                EnemyAi, FirstAliveAi, LowestHpAi,
    ↪ RandomTargetAi,
    |   |   |                      SkillfulAi
    |   |   |-- command/           CombatCommand, Attack, Defend, UseItem,
    ↪ UseSkill,
    |   |   |                      CommandHistory
    |   |   |-- damage/            DamageFormula, FlatFormula,
    ↪ ScaledFormula,
    |   |   |                      Damage, DamageType
    |   |   `-- defense/           DamageHandler, DamagePipeline,
    ↪ DodgeHandler,
    |   |                          ShieldHandler, ArmorHandler,
    ↪ ResistanceHandler
    |   |-- item/                  Item, ItemKind, Inventory
    |   |-- session/               GameSession
    |   `-- world/
    |       |-- Dungeon.java, Floor.java, DungeonBuilder.java
    |       |-- biome/             BiomeFactory, CryptBiome, CavernBiome
    |       `-- map/               TileType, TileTypes, GameMap, MapParser,
    ↪  MapData
    `-- ui/
        |-- ConsoleUi.java
        `-- input/                 InputSource, ConsoleInput, ScriptInput
```

Lo practicas en el ejercicio 7.5. Antes, cuatro ejercicios más pequeños, en otros dominios.

7.8 Ejercicios

Ejercicio 7.1: Insertar y borrar con comandos ★★☆☆☆

No es de Emberdeep. Crea una carpeta vacía y, dentro, archivos sin paquete. Vas a editar un texto con dos comandos que, por ahora, solo se ejecutan.

1. `Command.java`: interfaz con un método `void execute()`.
2. `Document.java`: el receptor. Guarda el texto en un `StringBuilder text` y ofrece `void insert(int position, String s)` (`text.insert(position, s)`), `void delete(int position, int length)` (`text.delete(position, position + length)`) y `String text()` (`text.toString()`).
3. `Insert.java`: implementa `Command`. Recibe en el constructor el `Document`, la posición y el `String` a insertar; `execute()` inserta.
4. `Delete.java`: implementa `Command`. Recibe el `Document`, la posición y la longitud; `execute()` borra.
5. `Main.java`: copia este código y ejecútalo:

Main.java

```java
import java.util.List;

public class Main {
    public static void main(String[] args) {
        Document doc = new Document();
        List<Command> commands = List.of(
                new Insert(doc, 0, "hola mundo"),
                new Insert(doc, 4, ","),
                new Delete(doc, 5, 1),
                new Insert(doc, 5, " querido "));
        for (Command command : commands) {
            command.execute();
            System.out.println("[" + doc.text() + "]");
        }
    }
}
```

Está bien si imprime las cuatro líneas siguientes y sabes decir qué papel hace cada clase: quién es el receptor, quiénes son los comandos y qué parte de `Main` es el invocador (el bucle, que ejecuta sin saber qué comando es cada uno).

Salida esperada

```
[hola mundo]
[hola, mundo]
[hola,mundo]
[hola, querido mundo]
```

Pista: *En el tercer comando, la posición 5 es el espacio que sigue a la coma: tras el segundo comando el texto es* `hola, mundo`.

Guiado ▪ solución en la pág. 241

Ejercicio 7.2: Una partida que se repite ★★☆☆☆

No es de Emberdeep. Lee este programa. Cada tecla de un guion se convierte en una acción (un comando escrito como lambda) que se ejecuta sobre el estado de una partida.

Replay.java

```java
import java.util.Random;

public class Replay {
    interface Action { void run(State state); }

    static class State {
        int hp = 20;
        int gold = 0;
        final Random random;
        State(long seed) { random = new Random(seed); }
    }

    static Action of(char key) {
        return switch (key) {
            case 'g' -> state -> state.gold += state.random.nextInt(6) + 1;
            case 'h' -> state -> state.hp -= state.random.nextInt(6) + 1;
            case 'r' -> state -> state.hp = Math.min(20, state.hp + 4);
            default -> state -> { };
        };
    }

    static String play(long seed, String script) {
        State state = new State(seed);
        for (char key : script.toCharArray()) {
            of(key).run(state);
        }
        return state.hp + " " + state.gold;
    }

    public static void main(String[] args) {
        System.out.println(play(42, "ghgrhgg"));
        System.out.println(play(42, "ghgrhgg"));
        System.out.println(play(7, "ghgrhgg"));
    }
}
```

Un dato que necesitas, porque no se puede calcular a mano: las seis primeras llamadas a `nextInt(6)` de `new Random(42)` devuelven 2, 3, 0, 2, 0 y 1, y las de `new Random(7)` devuelven 4, 2, 3, 4, 4 y 4.

1. Recorre el guion `ghgrhgg` con la semilla 42 y anota, tecla a tecla, el valor de `hp` y de `gold`. Ten en cuenta qué teclas gastan una tirada y cuáles no. ¿Qué imprime la primera línea?
2. ¿Por qué la segunda línea es igual que la primera? ¿Qué imprime la tercera?
3. ¿Qué dos cambios al programa harían que la segunda línea *dejara* de ser igual que la primera? Escríbelos en una frase cada uno.

Está bien si tus tres líneas son `17 7`, `17 7` y `15 19`, y si explicas que el replay es exacto porque la entrada (el guion), la semilla y las acciones (deterministas) son siempre las mismas.

Lectura de código ▪ solución en la pág. 243

Ejercicio 7.3: Deshacer varias veces ★★★☆☆

No es de Emberdeep. Parte de los archivos del ejercicio 7.1 (`Command`, `Document`, `Insert`, `Delete`; vale la solución del libro). Vas a hacer que se pueda deshacer.

1. `Command` gana un segundo método: `void undo()`.
2. `Insert.undo()` borra exactamente lo que insertó.
3. `Delete` apunta, al ejecutarse, el texto que borra (con `doc.text()` y `substring`) y `undo()` lo vuelve a insertar donde estaba.
4. `Editor.java`: guarda una `Deque<Command>` y ofrece `void run(Command command)`, que ejecuta y apila, y `boolean undo()`, que deshace el último y devuelve `true`, o devuelve `false` si no hay nada que deshacer.
5. Ejecuta este `Main`:

Main.java

```
public class Main {
    public static void main(String[] args) {
        Document doc = new Document();
        Editor editor = new Editor();
        editor.run(new Insert(doc, 0, "hola mundo"));
        editor.run(new Insert(doc, 4, ","));
        editor.run(new Delete(doc, 5, 1));
        System.out.println("[" + doc.text() + "]");
        for (int i = 0; i < 3; i++) {
            editor.undo();
            System.out.println("[" + doc.text() + "]");
        }
        System.out.println(editor.undo());
    }
}
```

Está bien si la salida es la siguiente, y un `Delete` deshecho devuelve el texto exacto que había borrado, aunque el borrado fuera en mitad del documento:

Salida esperada

```
[hola,mundo]
[hola, mundo]
[hola mundo]
[]
false
```

Pista: *`Deque` se importa de `java.util`, igual que `ArrayDeque`. Con `push` y `pop` se apila y se saca por arriba; comprueba `isEmpty()` antes de sacar.*

Práctica ▪ solución en la pág. 243

Ejercicio 7.4: Curar con probabilidad no se resta ★★★★☆

No es de Emberdeep. Una unidad tiene vida máxima 18 y empieza con 14. El comando `HealRandom` la cura `random.nextInt(6) + 1` puntos, sin pasar del máximo. Este es el código de partida, con un `undo()` que parece razonable:

Unit.java y Command.java

```
public class Unit {
    private final int maxHp;
    private int hp;
    public Unit(int maxHp, int hp) { this.maxHp = maxHp; this.hp = hp; }
    public int hp() { return hp; }
    public int heal(int amount) {          // devuelve lo curado de verdad
        int healed = Math.min(amount, maxHp - hp);
        hp += healed;
        return healed;
    }
    public void hurt(int amount) { hp = Math.max(0, hp - amount); }
    public void setHp(int hp) { this.hp = hp; }
}

public interface Command {
    void execute();
    void undo();
}
```

HealNaive.java y Main.java

```
import java.util.Random;

public class HealNaive implements Command {
    private final Unit unit;
    private final Random random;
    private int roll;

    public HealNaive(Unit unit, Random random) {
        this.unit = unit;
        this.random = random;
    }

    @Override public void execute() {
        roll = random.nextInt(6) + 1;
        unit.heal(roll);
    }

    @Override public void undo() { unit.hurt(roll); }
}

// Main.java: tres curas con Random(42) y tres deshacer,
// imprimiendo la vida cada vez
```

Con `Random(42)` las tres tiradas valen 3, 4 y 1. La unidad sube a 17, 18 y 18; pero al deshacer las tres, el programa llega a 10 y no a 14.

1. Explica por qué falla restar la tirada.

2. Propón tres formas distintas de que `undo()` devuelva siempre la vida exacta. Para cada una, di qué guarda el comando, qué cuesta y cuándo dejaría de funcionar.
3. Elige una, justifícala e impleméntala en una clase, con un `Main` que muestre la vida de la unidad tras cada cura y tras cada deshacer.

Está bien si las tres formas son realmente distintas, si cada una tiene un caso en que falla o sale cara, y si tu implementación imprime `cura -> 17`, `cura -> 18`, `cura -> 18`, `deshace -> 18`, `deshace -> 17` y `deshace -> 14`. Criterios: corrección con acciones que topan con el máximo, memoria por comando, claridad.

Diseño ▪ solución en la pág. 246

Ejercicio 7.5: Hito 9: acciones como comandos, deshacer y modo guion ★★★★☆

Implementa el hito H9 partiendo del estado del capítulo anterior (solución del ejercicio 6.5), en tres partes. Comprueba cada una antes de pasar a la siguiente.

Parte 1 (ejecutar): las acciones como comandos.

1. `Skill` pasa a `Skill(String name, int power, DamageType type)`. Actualiza la habilidad extra del élite en `EnemyTemplate.asElite()` (“Furia”, 12, `PHYSICAL`) y las seis habilidades de `CryptBiome` y `CavernBiome`, todas `PHYSICAL`.
2. `Hero(String name, Stats stats, List<Skill> skills)`, con copia inmutable de la lista en el constructor compacto. `HeroBuilder` gana `HeroBuilder skill(Skill)` y `build()` las pasa al héroe. En `Main.heroes()`, cada héroe recibe dos habilidades: “Golpe poderoso” (12, `PHYSICAL`) y “Llamarada” (10, `FIRE`).
3. `Fighter`: `List<Skill> skills()` (las del héroe o las del enemigo), `int heal(int amount)` (cura hasta `stats.maxHp()` y devuelve lo curado) y `CombatCommand act(Combat combat, List<Fighter> heroes)`, que sustituye a `chooseTarget` (misma `IllegalStateException` si no tiene IA).
4. `Combat`: `damageOf` y `deal(Fighter, Damage)` públicos, como en el texto; sin `heroAttack` ni `hit`. En `enemyPhase()`, cada enemigo vivo ejecuta `enemy.act(this, heroes).execute()` y se recogen sus mensajes; lo demás (subir la ronda, avanzar los buffs) queda igual.
5. Paquete `engine/combat/command/`: `CombatCommand` (solo `execute()` por ahora) y los cuatro comandos. `Attack(combat, attacker, target)` y los mensajes de siempre. `Defend(user)`: `addShield(5)` y un `TempBuff` de defensa de +2 durante 1 turno con `addBuff`; mensaje «X se cubre: +5 de escudo y +2 de defensa este turno.». `UseSkill(combat, user, skill, target)`: `Damage(power, type)` por `combat.deal`; mensaje «X usa H sobre Y y le hace N de daño.» (o «X usa H sobre Y, pero no le hace daño.») y, si cae, «Y queda fuera de combate.». `UseItem(session, user, target, item)`: quita el objeto con `session.replaceInventory(session.inventory().without(item))`; una poción cura con `target.heal(item.power())` («X usa P sobre Y: recupera N de vida.», con N lo curado de verdad) y una bomba hace `Damage(item.power(), FIRE)` por `session.combat().deal` («X lanza B sobre Y y le hace N de daño.», más el aviso de caída).
6. `EnemyAi.act(combat, self, heroes)` como método por defecto (ataca al objetivo de `choose`) y `SkillfulAi(EnemyAi targeting)` en `engine/combat/ai/`, con `choose` delegado y `act` como en el texto. En `CryptBiome`, el Zombi usa `new SkillfulAi(new FirstAliveAi())`.

7. Paquete `ui/input/`: `InputSource` (`String next()`) y `ConsoleInput(Scanner, PrintStream)`, que imprime `> `, vacía el buffer y devuelve la línea sin espacios en los extremos. `ConsoleUi(InputSource, PrintStream)` sustituye al `Scanner`. En el turno de cada héroe muestra «Héroes:», «Enemigos:» y «Objetos:» como antes y «Turno de X. [1] atacar [2] defender [3] objeto [4] habilidad»; las listas usan el formato «Título: [1] a [2] b» (dos espacios entre opciones). Tecla `1`: «Objetivo:» con los enemigos vivos. `2`: defender. `3`: «Objeto:», y luego «Aliado:» (héroes vivos) si es una poción u «Objetivo:» si es una bomba; sin objetos, «No te quedan objetos.» y se vuelve al menú. `4`: «Habilidad:» y «Objetivo:». Otra tecla: «Tecla no válida.». `Main` crea `new ConsoleUi(new ConsoleInput(new Scanner(System.in), System.out), System.out)`.

Está bien si:

1. Este archivo temporal, `app/Check07.java`, imprime las seis líneas siguientes (un ataque, una llamarada contra la Rata de ceniza, que resiste el fuego, una defensa, una bomba que derriba al Esqueleto y, por último, escudo, defensa y objetos que quedan):

app/Check07.java (parte 1; bórralo al terminar)

```java
package emberdeep.app;

import emberdeep.engine.character.*;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.*;
import emberdeep.engine.combat.damage.*;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.item.Item;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.biome.CryptBiome;
import java.util.List;
import java.util.Random;

public class Check07 {
    public static void main(String[] args) {
        Random random = new Random(42);
        Skill flare = new Skill("Llamarada", 10, DamageType.FIRE);
        Hero aria = new HeroBuilder().name("Aria")
                .stats(new Stats(30, 8, 3)).skill(flare).build();
        GameSession session = new GameSession(List.of(aria),
                new CryptBiome(random),
                List.of(EnemyKind.SKELETON, EnemyKind.ASH_RAT),
                new FlatFormula(), new DamagePipeline(random, 0.0));
        Combat combat = session.combat();
        Fighter hero = session.heroes().get(0);
        Fighter skeleton = session.enemies().get(0);
        Fighter rat = session.enemies().get(1);
        Item bomb = session.inventory().items().get(2);

        List<CombatCommand> commands = List.of(
                new Attack(combat, hero, skeleton),
                new UseSkill(combat, hero, flare, rat),
                new Defend(hero),
                new UseItem(session, hero, skeleton, bomb));
        for (CombatCommand command : commands) {
            command.execute().forEach(System.out::println);
        }
        System.out.println(hero.shield() + " " + hero.stats().defense
    ↪ ()
                + " " + session.inventory().items().size());
    }
}
```

Salida esperada

```
Aria ataca a Esqueleto y le hace 8 de daño.
Aria usa Llamarada sobre Rata de ceniza y le hace 5 de daño.
Aria se cubre: +5 de escudo y +2 de defensa este turno.
Aria lanza Bomba de ceniza sobre Esqueleto y le hace 10 de daño.
Esqueleto queda fuera de combate.
5 5 2
```

2. Esta orden juega la cripta con la fórmula plana, siempre la acción `1` y el primer

enemigo:

Partida de prueba

```
yes 1 | head -n 20 | ./gradlew -q run --args="1 default"
```

La partida es la del capítulo 6: la ronda 1 empieza con «Aria ataca a Esqueleto y le hace 8 de daño.», «Borin. . . 6» y «Cael. . . 9», el Esqueleto cae, la Rata de ceniza golpea a Cael con 2 y el Zombi a Aria con 1. Acaba en la ronda 3 con «¡Victoria!».

Parte 2 (deshacer): la pila.

1. `CombatCommand` gana `void undo()`.
2. `Fighter.restore(int hp, int shield, StatSource stats)`. Cada comando apunta, al principio de `execute()`, la vida, el escudo y la cadena de estadísticas del combatiente que cambia (el objetivo; el propio usuario en `Defend`); `UseItem` apunta además el `Inventory` de la sesión. `undo()` los devuelve.
3. `CommandHistory` (una `Deque<CombatCommand>` con `run`, `undo`, `size` y `clear`, como en el texto). `Combat` la guarda y ofrece `List<String> perform(CombatCommand)` y `boolean undo()`; `enemyPhase()` la vacía al final.
4. `ConsoleUi`: los héroes ejecutan sus comandos con `combat.perform`. El menú acaba en «[u] deshacer». Con `u`, si `combat.undo()` devuelve `true`, muestra «Deshaces la última acción.» y la acción deshecha vuelve a ser el turno del héroe anterior; si no, «No hay nada que deshacer en esta ronda.» y el mismo héroe sigue.

Está bien si:

1. Este `Check07` conserva la preparación del anterior y sustituye todo lo que sigue a la línea `Item bomb`. Imprime las ocho líneas siguientes: vida del Esqueleto, escudo de Aria, objetos y defensa de Aria después de las tres acciones y tras cada `undo`; `false` cuando ya no hay nada; una poción que vuelve al inventario; y `false` tras cerrar la ronda:

app/Check07.java (parte 2; el final, desde la línea Item bomb)

```
        Item bomb = session.inventory().items().get(2);

        combat.perform(new Attack(combat, hero, skeleton));
        combat.perform(new Defend(hero));
        combat.perform(new UseItem(session, hero, skeleton, bomb));
        show(skeleton, hero, session);
        for (int i = 0; i < 3; i++) {
            combat.undo();
            show(skeleton, hero, session);
        }
        System.out.println(combat.undo());

        hero.takeDamage(16);
        combat.perform(new UseItem(session, hero, hero,
                session.inventory().items().get(0)));
        show(hero, session);
        combat.undo();
        show(hero, session);

        combat.perform(new Defend(hero));
        combat.enemyPhase();
        System.out.println(combat.undo());
    }

    private static void show(Fighter hero, GameSession session) {
        System.out.println(hero.hp() + " "
                + session.inventory().items().size());
    }

    private static void show(Fighter skeleton, Fighter hero,
                             GameSession s) {
        System.out.println(skeleton.hp() + " " + hero.shield() + " "
                + s.inventory().items().size() + " "
                + hero.stats().defense());
    }
}
```

Salida esperada

```
0 5 2 5
10 5 3 5
10 0 3 3
18 0 3 3
false
26 2
14 3
false
```

2. Esta prueba por tubería pulsa `1 1` (Aria ataca al Esqueleto), `u` (Borin deshace), `u` (Aria, sin nada que deshacer) y `1 1` otra vez. Se queda sin teclas al llegar a Borin, y por eso se oculta el error con `2>/dev/null`:

Prueba de deshacer

```
printf '1\n1\nu\nu\n1\n1\n' \
  | ./gradlew -q run --args="1 default" 2>/dev/null \
  | grep -E 'Enemigos|Deshaces|No hay'
```

Debe imprimir siete líneas (las de «Enemigos:» muestran la vida del Esqueleto: 18, 10, 18, 18 y 10; los avisos van precedidos del prompt `> `, porque el prompt no termina en salto de línea):

Salida esperada

```
Enemigos: Esqueleto 18/18, Rata de ceniza 8/8, Zombi 22/22
Enemigos: Esqueleto 10/18, Rata de ceniza 8/8, Zombi 22/22
> Deshaces la última acción.
Enemigos: Esqueleto 18/18, Rata de ceniza 8/8, Zombi 22/22
> No hay nada que deshacer en esta ronda.
Enemigos: Esqueleto 18/18, Rata de ceniza 8/8, Zombi 22/22
Enemigos: Esqueleto 10/18, Rata de ceniza 8/8, Zombi 22/22
```

Parte 3 (replay): el modo guion.

1. `ScriptInput(Path file, PrintStream out)` como en el texto (lee con `Files.readAllLines`, ignora líneas vacías y las que empiezan por `#`, imprime `> ` y la tecla que consume y lanza `IllegalStateException` si se acaba).
2. Crea `scripts/demo.txt` con este contenido (23 teclas):

scripts/demo.txt

```
# Ronda 1: Aria lanza la bomba al Zombi y Borin lo deshace
3
3
3
u
# Aria ataca al Esqueleto; Borin lanza Llamarada a la Rata; Cael se
    ↪ cubre
1
1
4
2
2
2
# Ronda 2: Aria remata a la Rata, Borin y Cael rematan al Esqueleto
1
2
1
1
1
1
# Ronda 3: Aria se cubre; Borin y Cael hieren al Zombi sin matarlo
2
1
1
1
1
# Ronda 4: Aria remata al Zombi
1
1
```

3. `Main`: un argumento que acaba en `.txt` es un guion. Los demás argumentos (planta, `default`, `map`) siguen como antes, sin contar el guion. Con guion, la entrada es un `ScriptInput` y no se pregunta la fórmula (se usa `config.defaultFormula()`). `main` y `play` declaran `throws IOException`.

Está bien si:

1. La orden `./gradlew -q run` con el argumento `scripts/demo.txt` imprime 108 líneas. Las doce primeras y las trece últimas son estas (entre medias, la ronda 1 sigue con la Llamarada de Borin, que baja a la Rata 5 puntos, y con Cael cubriéndose):

Principio de la salida

```
=== Emberdeep ===
--- Ronda 1 ---
Héroes:   Aria 30/30, Borin 40/40, Cael 24/24
Enemigos: Esqueleto 18/18, Rata de ceniza 8/8, Zombi 22/22
Objetos:  Poción de cripta, Poción de cripta, Bomba de ceniza
Turno de Aria. [1] atacar  [2] defender  [3] objeto  [4] habilidad  [u]
    ↪ deshacer
> 3
Objeto:  [1] Poción de cripta  [2] Poción de cripta  [3] Bomba de ceniza
> 3
Objetivo:  [1] Esqueleto  [2] Rata de ceniza  [3] Zombi
> 3
Aria lanza Bomba de ceniza sobre Zombi y le hace 10 de daño.
```

Final de la salida

```
Cael ataca a Zombi y le hace 10 de daño.
Zombi usa Abrazo pútrido sobre Aria y le hace 1 de daño.
--- Ronda 4 ---
Héroes:   Aria 27/30, Borin 40/40, Cael 24/24
Enemigos: Esqueleto 0/18, Rata de ceniza 0/8, Zombi 5/22
Objetos:  Poción de cripta, Poción de cripta, Bomba de ceniza
Turno de Aria. [1] atacar  [2] defender  [3] objeto  [4] habilidad  [u]
    ↪ deshacer
> 1
Objetivo:  [1] Zombi
> 1
Aria ataca a Zombi y le hace 9 de daño.
Zombi queda fuera de combate.
¡Victoria!
```

2. El replay es exacto: dos ejecuciones dan archivos iguales, y la vida final de los héroes es 27, 40 y 24:

Dos partidas, una sola

```
./gradlew -q run --args="scripts/demo.txt" > partida1.txt
./gradlew -q run --args="scripts/demo.txt" > partida2.txt
diff partida1.txt partida2.txt && echo "Las dos partidas son idénticas"
wc -l partida1.txt
```

Salida esperada

```
Las dos partidas son idénticas
108 partida1.txt
```

3. Predice antes de ejecutar, y luego comprueba. En la ronda 3, el Zombi usa «Abrazo pútrido» (poder 7, físico) contra Aria, que se ha cubierto (escudo 5, armadura 1). ¿Cuánto daño le hace? ¿Por qué no es 0, como en un ataque normal del Zombi (ataque 5 contra la defensa 3 de Aria más 2 del buff)?

4. El modo mapa sigue funcionando como en el capítulo 5:

Modo mapa

```
printf 'a\nt\nq\n' | ./gradlew -q run --args="1 map"
```

Pista: *Cada héroe vivo aporta exactamente un comando por ronda, así que el número de comandos apilados es el número de héroes que ya han actuado. Eso permite llevar un índice sobre los héroes vivos que sube al actuar y baja al deshacer, sin que* `Combat` *sepa nada de la interfaz. Para la predicción de la parte 3, sigue la tubería eslabón a eslabón, como en el capítulo 6.*

Proyecto ▪ solución en la pág. 250

7.9 Resumen

Resumen

- El olor de Command es un menú o un bucle que hace el trabajo de cada acción en el sitio: la acción no es un valor, jugador y enemigo duplican código y no queda rastro para deshacer ni repetir. Command convierte cada acción en un objeto.
- Piezas: la interfaz `CombatCommand`, comandos concretos con sus datos, un receptor que cambia de verdad (`Fighter`, `GameSession`), un invocador que ejecuta sin saber qué comando es (`Combat` y `CommandHistory`) y los clientes que los crean (el menú y `EnemyAi.act`).
- Deshacer: al principio de `execute()`, el comando apunta lo que va a tocar; la pila (`Deque`: `push`, `pop`, `peek`) devuelve la última acción. Guardar la referencia a un objeto inmutable (el `Inventory`, la cadena de estadísticas) es barato y seguro.
- Todo lo que cambia el estado y no es un comando queda fuera de la pila: el cierre de la ronda vacía `CommandHistory`, y por eso `u` solo deshace las acciones de los héroes de la ronda en curso.
- Replay: si la entrada es una lista de teclas (`InputSource`), una partida es un archivo. Es exacta porque las teclas, la semilla del `Random` y las acciones deterministas son siempre las mismas.
- Una acción que se ejecuta y se olvida no necesita Command: basta una lambda. Las que no se pueden invertir se resuelven guardando el valor anterior o declarándolas no deshacibles. Si el cambio es grande, el patrón del capítulo 8 (Memento) guarda el estado entero.
- H9: `engine/combat/command/`, `SkillfulAi`, `Skill` con tipo, héroes con habilidades, `ui/input/`, `scripts/demo.txt` y una historia que se vacía en cada ronda.

Soluciones de los ejercicios

Solución del ejercicio 7.1

enunciado en la pág. 228

Cinco archivos sin paquete, en la misma carpeta.

Command.java

```
public interface Command {
    void execute();
}
```

Document.java

```
public class Document {
    private final StringBuilder text = new StringBuilder();

    public void insert(int position, String s) {
        text.insert(position, s);
    }

    public void delete(int position, int length) {
        text.delete(position, position + length);
    }

    public String text() {
        return text.toString();
    }
}
```

Insert.java

```
public class Insert implements Command {
    private final Document doc;
    private final int position;
    private final String s;

    public Insert(Document doc, int position, String s) {
        this.doc = doc;
        this.position = position;
        this.s = s;
    }

    @Override
    public void execute() {
        doc.insert(position, s);
    }
}
```

Delete.java

```java
public class Delete implements Command {
    private final Document doc;
    private final int position;
    private final int length;

    public Delete(Document doc, int position, int length) {
        this.doc = doc;
        this.position = position;
        this.length = length;
    }

    @Override
    public void execute() {
        doc.delete(position, length);
    }
}
```

Main.java

```java
import java.util.List;

public class Main {
    public static void main(String[] args) {
        Document doc = new Document();
        List<Command> commands = List.of(
                new Insert(doc, 0, "hola mundo"),
                new Insert(doc, 4, ","),
                new Delete(doc, 5, 1),
                new Insert(doc, 5, " querido "));
        for (Command command : commands) {
            command.execute();
            System.out.println("[" + doc.text() + "]");
        }
    }
}
```

Por qué funciona. `Document` es el receptor: sabe insertar y borrar y nada más. `Insert` y `Delete` son los comandos: cada uno lleva en sus campos todo lo que necesita (el documento, la posición, el texto o la longitud), así que se pueden construir primero y ejecutar después. El bucle de `Main` es el invocador: llama a `execute()` sin saber si es un `Insert` o un `Delete`. Y `Main`, que construye cada comando con sus datos, hace de cliente. Traza: `hola mundo`; se inserta una coma en la posición 4, `hola, mundo`; se borra el espacio de la posición 5, `hola,mundo`; se inserta `querido` en la posición 5, `hola, querido mundo`.

Errores típicos:

- Ejecutar la acción en el constructor del comando: entonces no es un objeto que se pueda guardar y ejecutar más tarde, solo una llamada con otra sintaxis.
- Contar mal las posiciones al borrar: el índice empieza en 0, y `delete` recibe posición y longitud, no posición inicial y final.

Solución del ejercicio 7.2

enunciado en la pág. 229

Traza con la semilla 42 (tiradas 2, 3, 0, 2, 0, 1). La tecla `r` no gasta tirada; las demás, una cada una:

Tecla	Tirada	Efecto	hp	gold
(inicio)			20	0
`g`	2	+3 de oro	20	3
`h`	3	−4 de vida	16	3
`g`	0	+1 de oro	16	4
`r`	—	+4 de vida, tope 20	20	4
`h`	2	−3 de vida	17	4
`g`	0	+1 de oro	17	5
`g`	1	+2 de oro	17	7

Primera línea: `17 7`. Segunda línea: `17 7`, porque `play` crea un `State` nuevo y, con él, un `Random` nuevo con la misma semilla: la secuencia de tiradas empieza otra vez desde el principio, el guion es el mismo y las acciones solo dependen del estado y de esas tiradas. Tercera línea, con la semilla 7 (tiradas 4, 2, 3, 4, 4, 4): oro +5 (5), vida −3 (17), oro +4 (9), vida 20 por `r`, vida −5 (15), oro +5 (14) y oro +5 (19): `15 19`.

Dos cambios que romperían el replay: (1) usar `new Random()` sin semilla (o `Math.random()`) en `State`, de modo que cada ejecución tiraría números distintos; (2) hacer el `Random` un campo `static` compartido entre partidas, de modo que la segunda partida seguiría la secuencia donde acabó la primera en lugar de reiniciarla. Otros, menos obvios: leer la hora del sistema o depender del orden de iteración de un `HashSet`.

Errores típicos:

- Gastar una tirada también en `r`: desplaza todas las siguientes y el resultado no coincide.
- Olvidar el tope de 20 de la vida en `r`: dejaría `hp` en 20 de todas formas, pero en otros guiones sí importa.

Solución del ejercicio 7.3

enunciado en la pág. 230

Seis archivos sin paquete. `Document` es el mismo del ejercicio anterior.

`Command.java`

```
public interface Command {
    void execute();
    void undo();
}
```

Document.java

```
public class Document {
    private final StringBuilder text = new StringBuilder();

    public void insert(int position, String s) {
        text.insert(position, s);
    }

    public void delete(int position, int length) {
        text.delete(position, position + length);
    }

    public String text() {
        return text.toString();
    }
}
```

Insert.java

```
public class Insert implements Command {
    private final Document doc;
    private final int position;
    private final String s;

    public Insert(Document doc, int position, String s) {
        this.doc = doc;
        this.position = position;
        this.s = s;
    }

    @Override
    public void execute() {
        doc.insert(position, s);
    }

    @Override
    public void undo() {
        doc.delete(position, s.length());
    }
}
```

Delete.java

```
public class Delete implements Command {
    private final Document doc;
    private final int position;
    private final int length;
    private String removed = "";

    public Delete(Document doc, int position, int length) {
        this.doc = doc;
        this.position = position;
        this.length = length;
    }

    @Override
    public void execute() {
        removed = doc.text().substring(position, position + length);
        doc.delete(position, length);
    }

    @Override
    public void undo() {
        doc.insert(position, removed);
    }
}
```

Editor.java

```
import java.util.ArrayDeque;
import java.util.Deque;

public class Editor {
    private final Deque<Command> done = new ArrayDeque<>();

    public void run(Command command) {
        command.execute();
        done.push(command);
    }

    public boolean undo() {
        if (done.isEmpty()) {
            return false;
        }
        done.pop().undo();
        return true;
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Document doc = new Document();
        Editor editor = new Editor();
        editor.run(new Insert(doc, 0, "hola mundo"));
        editor.run(new Insert(doc, 4, ","));
        editor.run(new Delete(doc, 5, 1));
        System.out.println("[" + doc.text() + "]");
        for (int i = 0; i < 3; i++) {
            editor.undo();
            System.out.println("[" + doc.text() + "]");
        }
        System.out.println(editor.undo());
    }
}
```

Por qué funciona. Cada comando guarda lo mínimo para revertirse. `Insert` ya tiene lo que necesita (posición y texto) y deshace borrando esa longitud. `Delete` no lo tiene: el texto borrado desaparece del documento, así que lo apunta en `execute()`, *antes* de borrar. La `Deque` es una pila: `push` apila el último y `pop` devuelve el último en entrar, que es el que hay que deshacer primero. Traza: `hola,mundo` se deshace a `hola, mundo` (se reinserta el espacio en la posición 5), luego a `hola mundo` (se quita la coma) y luego a vacío; con la pila vacía, `undo()` devuelve `false`.

Errores típicos:

- Guardar el texto borrado *después* de borrar: `removed` quedaría vacío y el `undo()` no restauraría nada.
- Llamar a `pop()` sin comprobar `isEmpty()`: lanza `NoSuchElementException` cuando no hay nada que deshacer.
- Usar `add` y `remove` en lugar de `push` y `pop` sin darse cuenta de que cambian el extremo de la cola: se deshacerían las acciones en el orden equivocado.

Solución del ejercicio 7.4

enunciado en la pág. 231

1. Por qué falla restar la tirada. La cura que se aplica no es la tirada, es `min(tirada, máximo - vida)`. En la segunda cura la tirada vale 4 pero solo se curó 1 (de 17 a 18); en la tercera la tirada vale 1 y se curó 0. Restar la tirada quita puntos que nunca se pusieron, y el programa acaba en 10 en lugar de 14.

2. Tres formas.

Forma	Qué guarda	Qué cuesta	Cuándo falla
Valor anterior	La vida de antes (`before`); `undo` la fija	Un entero. Simple y robusta	Si deshaces fuera de orden y otro cambio posterior se pierde; si el comando toca varias cifras, hay que guardarlas todas
Resultado aplicado	Lo curado de verdad (`healed`); `undo` lo resta con `hurt`	Un entero; hay que devolver el valor real desde `heal`	Si la operación inversa también topa con un límite, o si la acción no es una suma ("poner al máximo")
Comando inverso explícito	Un objeto `Hurt(unit, healed)` creado en `execute`; `undo` lo ejecuta	Una clase y un objeto más; permite rehacer y registrar el inverso	Igual que el resultado aplicado si el inverso no es exacto; más código para lo mismo

Las tres son correctas si se deshace en orden estricto (último en entrar, primero en salir), que es lo que garantiza una pila. Una cuarta, fuera de Command, es guardar una instantánea del estado entero antes de ejecutar: el capítulo 8.

3. Implementación de referencia: el resultado aplicado.

Unit.java

```java
public class Unit {
    private final int maxHp;
    private int hp;

    public Unit(int maxHp, int hp) {
        this.maxHp = maxHp;
        this.hp = hp;
    }

    public int hp() { return hp; }

    // Devuelve lo que se ha curado de verdad
    public int heal(int amount) {
        int healed = Math.min(amount, maxHp - hp);
        hp += healed;
        return healed;
    }

    public void hurt(int amount) { hp = Math.max(0, hp - amount); }
    public void setHp(int hp) { this.hp = hp; }
}
```

Command.java

```java
public interface Command {
    void execute();
    void undo();
}
```

HealDelta.java

```java
import java.util.Random;

public class HealDelta implements Command {
    private final Unit unit;
    private final Random random;
    private int healed;

    public HealDelta(Unit unit, Random random) {
        this.unit = unit;
        this.random = random;
    }

    @Override
    public void execute() {
        healed = unit.heal(random.nextInt(6) + 1);   // lo curado de verdad
    }

    @Override
    public void undo() {
        unit.hurt(healed);
    }
}
```

Main.java

```java
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Random;

public class Main {
    public static void main(String[] args) {
        Deque<Command> history = new ArrayDeque<>();
        Unit unit = new Unit(18, 14);
        Random random = new Random(42);
        for (int i = 0; i < 3; i++) {
            Command heal = new HealDelta(unit, random);
            heal.execute();
            history.push(heal);
            System.out.println("cura -> " + unit.hp());
        }
        while (!history.isEmpty()) {
            history.pop().undo();
            System.out.println("deshace -> " + unit.hp());
        }
    }
}
```

Salida: `cura -> 17`, `cura -> 18`, `cura -> 18`, `deshace -> 18`, `deshace -> 17`, `deshace -> 14`. La segunda cura curó 1 y la tercera 0, y eso es justo lo que se deshace.

Alternativa válida: guardar el valor anterior. Cambia solo la clase del comando:

HealBefore.java

```java
import java.util.Random;

public class HealBefore implements Command {
    private final Unit unit;
    private final Random random;
    private int before;

    public HealBefore(Unit unit, Random random) {
        this.unit = unit;
        this.random = random;
    }

    @Override
    public void execute() {
        before = unit.hp();                    // se apunta antes de curar
        unit.heal(random.nextInt(6) + 1);
    }

    @Override
    public void undo() {
        unit.setHp(before);
    }
}
```

Criterios para valorar otras soluciones. Está bien si: (a) el `undo()` no usa la tirada

sino la cura real o el valor anterior; (b) la salida coincide, incluidas las dos curas que topan con el máximo; (c) el diseño dice en qué orden deshace y por qué la pila lo garantiza; (d) explica qué guarda y qué cuesta. Es mejor la que guarda lo mínimo y se explica con una frase; "valor anterior" gana en simplicidad si el comando toca una sola cifra, y "comando inverso" gana si además quieres rehacer o registrar la acción.

Errores típicos:

- Guardar la tirada y llamarlo "resultado": es el dato, no lo aplicado.
- Apuntar el valor anterior *después* de curar: `undo` no cambiaría nada.
- Guardar el valor anterior en un campo de la clase `Unit` compartido por todos los comandos: dos curas seguidas se pisarían.

Solución del ejercicio 7.5

enunciado en la pág. 232

Parte del estado del capítulo 6. `GameSession`, `Stats`, `Enemy`, `TemplateRegistry`, `engine/stats/`, `engine/combat/defense/`, las tres IA que ya existían, el mapa y `GameConfig` no cambian. Aquí aparecen enteros todos los archivos que cambian o se crean, por partes. Lo marcado con `// parte 2` se añade al deshacer: en la parte 1, los comandos no tienen `undo()` ni los campos `...Before`, y `CombatCommand` solo declara `execute()`.

Parte 1 (ejecutar). engine/character/

Skill.java

```
package emberdeep.engine.character;

import emberdeep.engine.combat.damage.DamageType;

public record Skill(String name, int power, DamageType type) {}
```

Hero.java

```
package emberdeep.engine.character;

import java.util.List;

public record Hero(String name, Stats stats, List<Skill> skills) {
    public Hero {
        skills = List.copyOf(skills);
    }
}
```

HeroBuilder.java

```java
package emberdeep.engine.character;

import java.util.ArrayList;
import java.util.List;

public class HeroBuilder {
    private String name;
    private Stats stats;
    private final List<Skill> skills = new ArrayList<>();

    public HeroBuilder name(String name) {
        this.name = name;
        return this;
    }

    public HeroBuilder stats(Stats stats) {
        this.stats = stats;
        return this;
    }

    public HeroBuilder skill(Skill skill) {
        skills.add(skill);
        return this;
    }

    public Hero build() {
        if (name == null || name.isBlank()) {
            throw new IllegalStateException("El héroe necesita un nombre");
        }
        if (stats == null) {
            throw new IllegalStateException("El héroe necesita estadísticas
    ↪ ");
        }
        if (stats.maxHp() <= 0 || stats.attack() <= 0 || stats.defense() <
    ↪ 0) {
            throw new IllegalStateException("Estadísticas no válidas: " +
    ↪ stats);
        }
        return new Hero(name, stats, skills);
    }
}
```

EnemyTemplate.java (1/2: datos y copia)

```
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;
import emberdeep.engine.combat.damage.DamageType;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class EnemyTemplate {
    private final EnemyKind kind;
    private String name;
    private Stats stats;
    private final List<Skill> skills;
    private final EnemyAi ai;

    public EnemyTemplate(EnemyKind kind, String name, Stats stats, EnemyAi
    ↪ ai) {
        this(kind, name, stats, new ArrayList<>(), ai);
    }

    private EnemyTemplate(EnemyKind kind, String name, Stats stats,
                          List<Skill> skills, EnemyAi ai) {
        this.kind = kind;
        this.name = name;
        this.stats = stats;
        this.skills = skills;
        this.ai = ai;
    }

    public EnemyTemplate copy() {
        return new EnemyTemplate(kind, name, stats, new ArrayList<>(skills),
    ↪  ai);
    }

    public Enemy toEnemy() {
        return new Enemy(name, kind, stats, ai, List.copyOf(skills));
    }
```

EnemyTemplate.java (2/2: élite y accesos)

```
    public EnemyTemplate asElite() {
        EnemyTemplate elite = copy();
        elite.setName("Élite " + name);
        elite.setStats(new Stats(stats.maxHp() * 3 / 2,
                stats.attack() + 2, stats.defense() + 1));
        elite.addSkill(new Skill("Furia", 12, DamageType.PHYSICAL));
        return elite;
    }

    public EnemyTemplate addSkill(Skill skill) {
        skills.add(skill);
        return this;
    }

    public void setName(String name) { this.name = name; }
    public void setStats(Stats stats) { this.stats = stats; }

    public EnemyKind kind() { return kind; }
    public String name() { return name; }
    public Stats stats() { return stats; }
    public EnemyAi ai() { return ai; }

    public List<Skill> skills() {
        return Collections.unmodifiableList(skills);
    }
}
```

engine/character/Fighter.java, en tres tramos seguidos dentro de la misma clase:

Fighter.java (1/3: datos y fábricas)

```
package emberdeep.engine.character;

import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.ai.EnemyAi;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.stats.BaseStats;
import emberdeep.engine.stats.StatKind;
import emberdeep.engine.stats.StatSource;
import emberdeep.engine.stats.TempBuff;
import java.util.List;

public class Fighter {
    private final String name;
    private final EnemyKind kind;      // null en los héroes
    private final StatSource gear;     // equipo permanente
    private StatSource stats;          // gear más los buffs del combate
    private final EnemyAi ai;          // null en los héroes
    private final List<Skill> skills;
    private int hp;
    private int shield;

    private Fighter(String name, EnemyKind kind, StatSource gear,
                    EnemyAi ai, List<Skill> skills) {
        this.name = name;
        this.kind = kind;
        this.gear = gear;
        this.stats = gear;
        this.ai = ai;
        this.skills = skills;
        this.hp = gear.maxHp();
    }

    public static Fighter ofHero(Hero hero, StatSource gear) {
        return new Fighter(hero.name(), null, gear, null, hero.skills());
    }

    public static Fighter ofEnemy(Enemy enemy) {
        return new Fighter(enemy.name(), enemy.kind(),
                new BaseStats(enemy.stats()), enemy.ai(), enemy.skills());
    }
```

Fighter.java (2/3: accesos, escudo y daño)

```
    public String name() { return name; }
    public StatSource stats() { return stats; }
    public StatSource gear() { return gear; }
    public List<Skill> skills() { return skills; }
    public int hp() { return hp; }
    public boolean isAlive() { return hp > 0; }
    public int shield() { return shield; }

    public void addShield(int amount) {
        shield += amount;
    }

    public int absorbShield(int damage) {
        int absorbed = Math.min(shield, damage);
        shield -= absorbed;
        return damage - absorbed;
    }

    public boolean resists(DamageType type) {
        return kind != null && kind.resistances().contains(type);
    }

    public void takeDamage(int amount) {
        hp = Math.max(0, hp - amount);
    }
```

Fighter.java (3/3: curar, restaurar, buffs y acción)

```java

    public int heal(int amount) {
        int healed = Math.min(amount, stats.maxHp() - hp);
        hp += healed;
        return healed;
    }

    // parte 2: devuelve los tres números que los comandos apuntan
    public void restore(int hp, int shield, StatSource stats) {
        this.hp = hp;
        this.shield = shield;
        this.stats = stats;
    }

    public void addBuff(StatKind stat, int bonus, int turns) {
        stats = new TempBuff(stats, stat, bonus, turns);
    }

    public void tickBuffs() {
        stats = stats.tick();
    }

    public CombatCommand act(Combat combat, List<Fighter> heroes) {
        if (ai == null) {
            throw new IllegalStateException(name + " no tiene IA: es un hé
    ↪ roe");
        }
        return ai.act(combat, this, heroes);
    }
}
```

engine/world/biome/: solo cambian los datos de habilidad (con tipo) y, en la cripta, la IA del Zombi.

CryptBiome.java (1/2: registro)

```java
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.EnemyTemplate;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Stats;
import emberdeep.engine.character.TemplateRegistry;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import emberdeep.engine.combat.ai.SkillfulAi;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import java.util.Random;

public class CryptBiome implements BiomeFactory {
    private final TemplateRegistry registry = new TemplateRegistry();

    public CryptBiome(Random random) {
        registry.register(new EnemyTemplate(EnemyKind.SKELETON, "Esqueleto",
    ↪
                new Stats(18, 7, 2), new RandomTargetAi(random))
                .addSkill(new Skill("Golpe de hueso", 9,
                        DamageType.PHYSICAL)));
        registry.register(new EnemyTemplate(EnemyKind.ASH_RAT,
                "Rata de ceniza", new Stats(8, 5, 0), new LowestHpAi())
                .addSkill(new Skill("Mordisco sucio", 6,
                        DamageType.PHYSICAL)));
        registry.register(new EnemyTemplate(EnemyKind.ZOMBIE, "Zombi",
                new Stats(22, 5, 1), new SkillfulAi(new FirstAliveAi()))
                .addSkill(new Skill("Abrazo pútrido", 7,
                        DamageType.PHYSICAL)));
    }
```

CryptBiome.java (2/2: enemy e item)

```java

    @Override
    public Enemy enemy(EnemyKind kind) {
        return registry.get(kind).copy().toEnemy();
    }

    @Override
    public Item item(ItemKind kind) {
        return switch (kind) {
            case HEALING -> new Item("Poción de cripta", kind, 12);
            case BOMB -> new Item("Bomba de ceniza", kind, 10);
        };
    }
}
```

CavernBiome.java (1/2: registro)

```java
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.EnemyTemplate;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Stats;
import emberdeep.engine.character.TemplateRegistry;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import java.util.Random;

public class CavernBiome implements BiomeFactory {
    private final TemplateRegistry registry = new TemplateRegistry();

    public CavernBiome(Random random) {
        registry.register(new EnemyTemplate(EnemyKind.GOBLIN, "Goblin",
                new Stats(14, 6, 1), new FirstAliveAi())
                .addSkill(new Skill("Puñalada", 8, DamageType.PHYSICAL)));
        registry.register(new EnemyTemplate(EnemyKind.BAT, "Murciélago",
                new Stats(10, 6, 0), new LowestHpAi())
                .addSkill(new Skill("Chillido", 5, DamageType.PHYSICAL)));
        registry.register(new EnemyTemplate(EnemyKind.SPIDER, "Araña",
                new Stats(16, 7, 2), new RandomTargetAi(random))
                .addSkill(new Skill("Picadura", 9, DamageType.PHYSICAL)));
    }
```

CavernBiome.java (2/2: enemy e item)

```java

    @Override
    public Enemy enemy(EnemyKind kind) {
        return registry.get(kind).copy().toEnemy();
    }

    @Override
    public Item item(ItemKind kind) {
        return switch (kind) {
            case HEALING -> new Item("Poción de caverna", kind, 10);
            case BOMB -> new Item("Bomba de roca", kind, 14);
        };
    }
}
```

engine/combat/ai/: `EnemyAi` gana `act` y nace `SkillfulAi`.

EnemyAi.java

```
package emberdeep.engine.combat.ai;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.Attack;
import emberdeep.engine.combat.command.CombatCommand;
import java.util.List;

public interface EnemyAi {
    Fighter choose(Fighter self, List<Fighter> heroes);

    default CombatCommand act(Combat combat, Fighter self,
                              List<Fighter> heroes) {
        return new Attack(combat, self, choose(self, heroes));
    }
}
```

SkillfulAi.java

```
package emberdeep.engine.combat.ai;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.Attack;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.command.UseSkill;
import java.util.List;

public class SkillfulAi implements EnemyAi {
    private final EnemyAi targeting;

    public SkillfulAi(EnemyAi targeting) {
        this.targeting = targeting;
    }

    @Override
    public Fighter choose(Fighter self, List<Fighter> heroes) {
        return targeting.choose(self, heroes);
    }

    @Override
    public CombatCommand act(Combat combat, Fighter self,
                             List<Fighter> heroes) {
        Fighter target = choose(self, heroes);
        if (combat.round() % 3 == 0 && !self.skills().isEmpty()) {
            return new UseSkill(combat, self, self.skills().get(0), target);
        }
        return new Attack(combat, self, target);
    }
}
```

engine/combat/Combat.java, en dos tramos seguidos dentro de la misma clase:

Combat.java (1/2: estado, historia y daño)

```java
package emberdeep.engine.combat;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.command.CommandHistory;
import emberdeep.engine.combat.damage.Damage;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import java.util.ArrayList;
import java.util.List;

public class Combat {
    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final DamageFormula formula;
    private final DamagePipeline pipeline;
    private final CommandHistory history = new CommandHistory();  // parte
    ↪ 2
    private int round = 1;

    public Combat(List<Fighter> heroes, List<Fighter> enemies,
                  DamageFormula formula, DamagePipeline pipeline) {
        this.heroes = heroes;
        this.enemies = enemies;
        this.formula = formula;
        this.pipeline = pipeline;
    }

    public int round() {
        return round;
    }

    public boolean heroesWon() {
        return enemies.stream().noneMatch(Fighter::isAlive);
    }

    public boolean heroesLost() {
        return heroes.stream().noneMatch(Fighter::isAlive);
    }

    public boolean isOver() {
        return heroesWon() || heroesLost();
    }
```

Combat.java (2/2: ejecutar, deshacer y enemigos)

```
    // parte 2: los héroes ejecutan sus comandos por aquí
    public List<String> perform(CombatCommand command) {
        return history.run(command);
    }

    public boolean undo() {
        return history.undo();
    }

    public int damageOf(Fighter attacker, Fighter target) {
        return formula.damage(attacker.stats().attack(),
                              target.stats().defense());
    }

    public int deal(Fighter target, Damage damage) {
        int dealt = pipeline.apply(target, damage);
        target.takeDamage(dealt);
        return dealt;
    }

    public List<String> enemyPhase() {
        List<String> messages = new ArrayList<>();
        for (Fighter enemy : enemies) {
            if (!enemy.isAlive()) {
                continue;
            }
            if (heroesLost()) {
                break;
            }
            messages.addAll(enemy.act(this, heroes).execute());
        }
        round++;
        heroes.forEach(Fighter::tickBuffs);
        enemies.forEach(Fighter::tickBuffs);
        history.clear();                                          // parte
    ↪ 2
        return messages;
    }
}
```

engine/combat/command/ (versión final, partes 1 y 2):

CombatCommand.java

```
package emberdeep.engine.combat.command;

import java.util.List;

public interface CombatCommand {
    List<String> execute();
    void undo();                // parte 2
}
```

Attack.java

```java
package emberdeep.engine.combat.command;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.Damage;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.stats.StatSource;
import java.util.ArrayList;
import java.util.List;

public class Attack implements CombatCommand {
    private final Combat combat;
    private final Fighter attacker;
    private final Fighter target;
    private int hpBefore;                    // parte 2
    private int shieldBefore;
    private StatSource statsBefore;

    public Attack(Combat combat, Fighter attacker, Fighter target) {
        this.combat = combat;
        this.attacker = attacker;
        this.target = target;
    }

    @Override
    public List<String> execute() {
        hpBefore = target.hp();              // parte 2: se apunta antes
        shieldBefore = target.shield();
        statsBefore = target.stats();
        int raw = combat.damageOf(attacker, target);
        int dealt = combat.deal(target,
                new Damage(raw, DamageType.PHYSICAL));
        String who = attacker.name() + " ataca a " + target.name();
        List<String> messages = new ArrayList<>();
        messages.add(dealt == 0 ? who + ", pero no le hace daño."
                : who + " y le hace " + dealt + " de daño.");
        if (!target.isAlive()) {
            messages.add(target.name() + " queda fuera de combate.");
        }
        return messages;
    }

    @Override
    public void undo() {                     // parte 2
        target.restore(hpBefore, shieldBefore, statsBefore);
    }
}
```

Defend.java

```java
package emberdeep.engine.combat.command;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.stats.StatKind;
import emberdeep.engine.stats.StatSource;
import java.util.List;

public class Defend implements CombatCommand {
    private static final int SHIELD = 5;
    private static final int DEFENSE_BONUS = 2;

    private final Fighter user;
    private int hpBefore;                    // parte 2
    private int shieldBefore;
    private StatSource statsBefore;

    public Defend(Fighter user) {
        this.user = user;
    }

    @Override
    public List<String> execute() {
        hpBefore = user.hp();                // parte 2: se apunta antes
        shieldBefore = user.shield();
        statsBefore = user.stats();
        user.addShield(SHIELD);
        user.addBuff(StatKind.DEFENSE, DEFENSE_BONUS, 1);
        return List.of(user.name() + " se cubre: +" + SHIELD
                + " de escudo y +" + DEFENSE_BONUS
                + " de defensa este turno.");
    }

    @Override
    public void undo() {                     // parte 2
        user.restore(hpBefore, shieldBefore, statsBefore);
    }
}
```

UseSkill.java

```java
package emberdeep.engine.combat.command;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Skill;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.Damage;
import emberdeep.engine.stats.StatSource;
import java.util.ArrayList;
import java.util.List;

public class UseSkill implements CombatCommand {
    private final Combat combat;
    private final Fighter user;
    private final Skill skill;
    private final Fighter target;
    private int hpBefore;                    // parte 2
    private int shieldBefore;
    private StatSource statsBefore;

    public UseSkill(Combat combat, Fighter user,
                    Skill skill, Fighter target) {
        this.combat = combat;
        this.user = user;
        this.skill = skill;
        this.target = target;
    }

    @Override
    public List<String> execute() {
        hpBefore = target.hp();              // parte 2: se apunta antes
        shieldBefore = target.shield();
        statsBefore = target.stats();
        int dealt = combat.deal(target,
                new Damage(skill.power(), skill.type()));
        String who = user.name() + " usa " + skill.name()
                + " sobre " + target.name();
        List<String> messages = new ArrayList<>();
        messages.add(dealt == 0 ? who + ", pero no le hace daño."
                : who + " y le hace " + dealt + " de daño.");
        if (!target.isAlive()) {
            messages.add(target.name() + " queda fuera de combate.");
        }
        return messages;
    }

    @Override
    public void undo() {                     // parte 2
        target.restore(hpBefore, shieldBefore, statsBefore);
    }
}
```

UseItem.java (1/2: datos y execute)

```java
package emberdeep.engine.combat.command;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.damage.Damage;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.Item;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.stats.StatSource;
import java.util.ArrayList;
import java.util.List;

public class UseItem implements CombatCommand {
    private final GameSession session;
    private final Fighter user;
    private final Fighter target;
    private final Item item;
    private Inventory inventoryBefore;      // parte 2
    private int hpBefore;
    private int shieldBefore;
    private StatSource statsBefore;

    public UseItem(GameSession session, Fighter user,
                   Fighter target, Item item) {
        this.session = session;
        this.user = user;
        this.target = target;
        this.item = item;
    }

    @Override
    public List<String> execute() {
        inventoryBefore = session.inventory();   // parte 2: la referencia
        hpBefore = target.hp();
        shieldBefore = target.shield();
        statsBefore = target.stats();
        session.replaceInventory(inventoryBefore.without(item));
```

UseItem.java (2/2: efecto y undo)

```
        List<String> messages = new ArrayList<>();
        String who = user.name() + " usa " + item.name()
                + " sobre " + target.name();
        switch (item.kind()) {
            case HEALING -> {
                int healed = target.heal(item.power());
                messages.add(who + ": recupera " + healed + " de vida.");
            }
            case BOMB -> {
                int dealt = session.combat().deal(target,
                        new Damage(item.power(), DamageType.FIRE));
                messages.add(user.name() + " lanza " + item.name()
                        + " sobre " + target.name()
                        + " y le hace " + dealt + " de daño.");
                if (!target.isAlive()) {
                    messages.add(target.name() + " queda fuera de combate.")
    ↪ ;
                }
            }
        }
        return messages;
    }

    @Override
    public void undo() {                    // parte 2
        session.replaceInventory(inventoryBefore);
        target.restore(hpBefore, shieldBefore, statsBefore);
    }
}
```

CommandHistory.java

```java
package emberdeep.engine.combat.command;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.List;

public class CommandHistory {
    private final Deque<CombatCommand> done = new ArrayDeque<>();

    public List<String> run(CombatCommand command) {
        List<String> messages = command.execute();
        done.push(command);
        return messages;
    }

    public boolean undo() {
        if (done.isEmpty()) {
            return false;
        }
        done.pop().undo();
        return true;
    }

    public int size() { return done.size(); }
    public void clear() { done.clear(); }
}
```

ui/input/

InputSource.java

```java
package emberdeep.ui.input;

public interface InputSource {
    String next();   // la siguiente tecla que el jugador pulsa
}
```

ConsoleInput.java

```java
package emberdeep.ui.input;

import java.io.PrintStream;
import java.util.Scanner;

public class ConsoleInput implements InputSource {
    private final Scanner in;
    private final PrintStream out;

    public ConsoleInput(Scanner in, PrintStream out) {
        this.in = in;
        this.out = out;
    }

    @Override
    public String next() {
        out.print("> ");
        out.flush();
        return in.nextLine().trim();
    }
}
```

ScriptInput.java (parte 3)

```
package emberdeep.ui.input;

import java.io.IOException;
import java.io.PrintStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

public class ScriptInput implements InputSource {
    private final List<String> keys;
    private final PrintStream out;
    private int position = 0;

    public ScriptInput(Path file, PrintStream out) throws IOException {
        this.keys = Files.readAllLines(file).stream()
                .map(String::trim)
                .filter(line -> !line.isEmpty() && !line.startsWith("#"))
                .toList();
        this.out = out;
    }

    @Override
    public String next() {
        if (position == keys.size()) {
            throw new IllegalStateException("El guion se ha acabado");
        }
        String key = keys.get(position++);
        out.println("> " + key);
        return key;
    }
}
```

ui/ConsoleUi.java (versión final, partes 1 y 2), en seis tramos seguidos dentro de la misma clase:

ConsoleUi.java (1/6: imports y fórmula)

```java
package emberdeep.ui;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Skill;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.Attack;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.command.Defend;
import emberdeep.engine.combat.command.UseItem;
import emberdeep.engine.combat.command.UseSkill;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.TileType;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.input.InputSource;
import java.io.PrintStream;
import java.util.List;
import java.util.stream.Collectors;

public class ConsoleUi {
    private final InputSource input;
    private final PrintStream out;

    public ConsoleUi(InputSource input, PrintStream out) {
        this.input = input;
        this.out = out;
    }

    public int askFormula() {
        out.println("Fórmula de daño:");
        out.println("  [1] plana: ataque - defensa");
        out.println("  [2] escalada: ataque x 100 / (100 + defensa)");
        return ask(2);
    }
```

ConsoleUi.java (2/6: la partida)

```
    public void run(GameSession session) {
        Combat combat = session.combat();
        out.println("=== Emberdeep ===");
        while (!combat.isOver()) {
            out.println("--- Ronda " + session.round() + " ---");
            List<Fighter> acting = session.heroes().stream()
                    .filter(Fighter::isAlive).toList();
            int i = 0;
            while (i < acting.size() && !combat.isOver()) {
                boolean acted = heroTurn(session, acting.get(i));
                i += acted ? 1 : -1;     // parte 2: deshacer retrocede
            }
            if (!combat.isOver()) {
                print(combat.enemyPhase());
            }
        }
        out.println(combat.heroesWon() ? "¡Victoria!" : "Derrota...");
    }
```

ConsoleUi.java (3/6: el turno de un héroe)

```
1
2       // true si el héroe actúa; false si se deshace la acción anterior
3       private boolean heroTurn(GameSession session, Fighter hero) {
4           Combat combat = session.combat();
5           while (true) {
6               out.println("Héroes:   " + describe(session.heroes()));
7               out.println("Enemigos: " + describe(session.enemies()));
8               out.println("Objetos:  " + describeItems(session.inventory()));
9               out.println("Turno de " + hero.name()
10                      + ". [1] atacar  [2] defender  [3] objeto"
11                      + "  [4] habilidad  [u] deshacer");
12              String key = input.next();
13              if (key.equals("u")) {           // parte 2
14                  if (combat.undo()) {
15                      out.println("Deshaces la última acción.");
16                      return false;
17                  }
18                  out.println("No hay nada que deshacer en esta ronda.");
19                  continue;
20              }
21              CombatCommand command = switch (key) {
22                  case "1" -> new Attack(combat, hero,
23                          askTarget("Objetivo", session.enemies()));
24                  case "2" -> new Defend(hero);
25                  case "3" -> chooseItem(session, hero);
26                  case "4" -> chooseSkill(session, hero);
27                  default -> {
28                      out.println("Tecla no válida.");
29                      yield null;
30                  }
31              };
32              if (command != null) {
33                  print(combat.perform(command));
34                  return true;
35              }
36          }
37      }
```

ConsoleUi.java (4/6: objetos, habilidades y objetivos)

```
    private CombatCommand chooseItem(GameSession session, Fighter hero) {
        List<Item> items = session.inventory().items();
        if (items.isEmpty()) {
            out.println("No te quedan objetos.");
            return null;
        }
        Item item = items.get(choose("Objeto",
                items.stream().map(Item::name).toList()));
        Fighter target = item.kind() == ItemKind.HEALING
                ? askTarget("Aliado", session.heroes())
                : askTarget("Objetivo", session.enemies());
        return new UseItem(session, hero, target, item);
    }

    private CombatCommand chooseSkill(GameSession session, Fighter hero) {
        List<Skill> skills = hero.skills();
        if (skills.isEmpty()) {
            out.println("No tienes habilidades.");
            return null;
        }
        Skill skill = skills.get(choose("Habilidad",
                skills.stream().map(Skill::name).toList()));
        Fighter target = askTarget("Objetivo", session.enemies());
        return new UseSkill(session.combat(), hero, skill, target);
    }

    private Fighter askTarget(String title, List<Fighter> group) {
        List<Fighter> alive = group.stream()
                .filter(Fighter::isAlive).toList();
        return alive.get(choose(title,
                alive.stream().map(Fighter::name).toList()));
    }
```

ConsoleUi.java (5/6: menús, entrada y auxiliares)

```
    // Muestra "Título:  [1] a  [2] b" y devuelve la posición (desde 0)
    private int choose(String title, List<String> labels) {
        StringBuilder menu = new StringBuilder(title + ":");
        for (int i = 0; i < labels.size(); i++) {
            menu.append("  [").append(i + 1).append("] ")
                    .append(labels.get(i));
        }
        out.println(menu);
        return ask(labels.size()) - 1;
    }

    private int ask(int max) {
        while (true) {
            String line = input.next();
            try {
                int n = Integer.parseInt(line);
                if (n >= 1 && n <= max) {
                    return n;
                }
            } catch (NumberFormatException e) {
                // no es un número: se avisa más abajo
            }
            out.println("Escribe un número entre 1 y " + max + ".");
        }
    }

    private String describe(List<Fighter> group) {
        return group.stream()
                .map(f -> f.name() + " " + f.hp() + "/" + f.stats().maxHp())
                .collect(Collectors.joining(", "));
    }

    private String describeItems(Inventory inventory) {
        if (inventory.items().isEmpty()) {
            return "(ninguno)";
        }
        return inventory.items().stream()
                .map(Item::name)
                .collect(Collectors.joining(", "));
    }

    private void print(List<String> messages) {
        messages.forEach(out::println);
    }
```

ConsoleUi.java (6/6: modo mapa y llave final)

```
    public boolean explore(GameMap map, TileTypes types) {
        while (true) {
            out.print(map.render());
            out.println("[w a s d] mover, [t] casillas, [q] salir");
            String key = input.next();
            switch (key) {
                case "w" -> step(map, 0, -1);
                case "a" -> step(map, -1, 0);
                case "s" -> step(map, 0, 1);
                case "d" -> step(map, 1, 0);
                case "t" -> showTiles(map, types);
                case "q" -> { return false; }
                default -> out.println("Tecla no válida.");
            }
            if (map.onStairs()) {
                out.println("Bajas por la escalera.");
                return true;
            }
        }
    }

    private void step(GameMap map, int dx, int dy) {
        if (!map.move(dx, dy)) {
            out.println("No puedes pasar.");
        }
    }

    private void showTiles(GameMap map, TileTypes types) {
        TileType corner = map.tileAt(0, 0);
        TileType opposite = map.tileAt(map.width() - 1, map.height() - 1);
        out.println("Casillas: " + map.width() * map.height()
                + ", TileType creados: " + types.createdCount());
        out.println("Esquinas opuestas, mismo objeto: "
                + (corner == opposite));
    }
}
```

app/Main.java, en cuatro tramos seguidos dentro de la misma clase (el modo guion es de la parte 3):

Main.java (1/4: imports)

```
package emberdeep.app;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.Dungeon;
import emberdeep.engine.world.DungeonBuilder;
import emberdeep.engine.world.Floor;
import emberdeep.engine.world.biome.BiomeFactory;
import emberdeep.engine.world.biome.CavernBiome;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.engine.world.map.MapData;
import emberdeep.engine.world.map.MapParser;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.ConsoleUi;
import emberdeep.ui.input.ConsoleInput;
import emberdeep.ui.input.InputSource;
import emberdeep.ui.input.ScriptInput;
import java.io.IOException;
import java.nio.file.Path;
import java.util.Arrays;
import java.util.List;
import java.util.Random;
import java.util.Scanner;
```

Main.java (2/4: arranque y entrada)

```
1
2  public class Main {
3      private static final Skill POWER_STRIKE =
4              new Skill("Golpe poderoso", 12, DamageType.PHYSICAL);
5      private static final Skill FLARE =
6              new Skill("Llamarada", 10, DamageType.FIRE);
7
8      private final GameConfig config;
9
10     private Main(GameConfig config) {
11         this.config = config;
12     }
13
14     public static void main(String[] args) throws IOException {
15         GameConfig config = new GameConfig(42L, 3, new FlatFormula());
16         new Main(config).play(args);
17     }
18
19     private void play(String[] args) throws IOException {
20         String script = scriptOf(args);
21         String[] options = optionsOf(args);
22         int floorNumber = chooseFloor(options);
23         InputSource input = script == null
24                 ? new ConsoleInput(new Scanner(System.in), System.out)
25                 : new ScriptInput(Path.of(script), System.out);
26         ConsoleUi ui = new ConsoleUi(input, System.out);
27         Random random = new Random(config.seed());
28
29         BiomeFactory crypt = new CryptBiome(random);
30         BiomeFactory cavern = new CavernBiome(random);
31         List<EnemyKind> cryptKinds = List.of(
32                 EnemyKind.SKELETON, EnemyKind.ASH_RAT, EnemyKind.ZOMBIE);
33         List<EnemyKind> cavernKinds = List.of(
34                 EnemyKind.GOBLIN, EnemyKind.BAT, EnemyKind.SPIDER);
```

Main.java (3/4: mazmorra, partida y modo mapa)

```
        Dungeon dungeon = new DungeonBuilder()
                .floor().biome(crypt).enemies(cryptKinds)
                        .map(MapData.floor(1)).done()
                .floor().biome(cavern).enemies(cavernKinds)
                        .map(MapData.floor(2)).done()
                .floor().biome(crypt).enemies(cryptKinds)
                        .map(MapData.floor(3)).done()
                .build();

        if (options.length > 1 && options[1].equals("map")) {
            explore(ui, dungeon, floorNumber);
            return;
        }
        DamageFormula formula = chooseFormula(ui, options, script != null);
        Floor floor = dungeon.floors().get(floorNumber - 1);
        GameSession session = new GameSession(heroes(), floor.biome(),
                floor.enemies(), formula, new DamagePipeline(random, 0.0));
        ui.run(session);
    }

    private void explore(ConsoleUi ui, Dungeon dungeon, int firstFloor) {
        TileTypes types = new TileTypes();
        MapParser parser = new MapParser(types);
        int number = firstFloor;
        while (true) {
            Floor floor = dungeon.floors().get(number - 1);
            boolean descended =
                    ui.explore(parser.parse(floor.map()), types);
            if (!descended || number == dungeon.floors().size()) {
                return;
            }
            number++;
        }
    }
```

Main.java (4/4: argumentos, héroes y llave final)

```
    // El guion es el argumento que acaba en .txt (parte 3)
    private static String scriptOf(String[] args) {
        return Arrays.stream(args)
                .filter(arg -> arg.endsWith(".txt"))
                .findFirst().orElse(null);
    }

    private static String[] optionsOf(String[] args) {
        return Arrays.stream(args)
                .filter(arg -> !arg.endsWith(".txt"))
                .toArray(String[]::new);
    }

    private DamageFormula chooseFormula(ConsoleUi ui, String[] options,
                                        boolean scripted) {
        boolean useDefault = scripted
                || (options.length > 1 && options[1].equals("default"));
        if (useDefault) {
            return config.defaultFormula();
        }
        return ui.askFormula() == 1
                ? new FlatFormula() : new ScaledFormula();
    }

    private int chooseFloor(String[] options) {
        int floor = options.length > 0 ? Integer.parseInt(options[0]) : 1;
        if (floor < 1 || floor > config.floors()) {
            throw new IllegalArgumentException(
                    "La planta debe estar entre 1 y " + config.floors());
        }
        return floor;
    }

    private List<Hero> heroes() {
        return List.of(
                new HeroBuilder().name("Aria").stats(new Stats(30, 8, 3))
                        .skill(POWER_STRIKE).skill(FLARE).build(),
                new HeroBuilder().name("Borin").stats(new Stats(40, 6, 5))
                        .skill(POWER_STRIKE).skill(FLARE).build(),
                new HeroBuilder().name("Cael").stats(new Stats(24, 9, 2))
                        .skill(POWER_STRIKE).skill(FLARE).build());
    }
}
```

scripts/demo.txt: el de la parte 3 (se repite entero para que la solución esté completa).

scripts/demo.txt

```
# Ronda 1: Aria lanza la bomba al Zombi y Borin lo deshace
3
3
3
u
# Aria ataca al Esqueleto; Borin lanza Llamarada a la Rata; Cael se cubre
1
1
4
2
2
2
# Ronda 2: Aria remata a la Rata, Borin y Cael rematan al Esqueleto
1
2
1
1
1
1
# Ronda 3: Aria se cubre; Borin y Cael hieren al Zombi sin matarlo
2
1
1
1
1
# Ronda 4: Aria remata al Zombi
1
1
```

Por qué funciona (parte 1). Los cuatro comandos tienen todo en sus campos y hablan con los receptores; `Combat` deja de saber qué acciones existen. `Attack` escribe los mismos mensajes que antes, y las IA anteriores usan el `act` por defecto, que llama a `choose` exactamente una vez por acción, de modo que `RandomTargetAi` gasta los mismos números del `Random`: por eso la partida con `yes 1` es la del capítulo 6. Los seis valores de `Check07` salen así: el ataque de Aria (10 con la espada) contra el Esqueleto (defensa 2) hace 8; la llamarada hace 10 de fuego a la Rata de ceniza, que sin armadura ni escudo lo resiste y recibe 5; `Defend` deja el escudo en 5 y la defensa en $3 + 2 = 5$; la bomba (10 de fuego) deja al Esqueleto (que tenía 10) en 0; y queda un inventario de 2 objetos. Las IA y los comandos se reparten el trabajo así: la IA *decide* (qué acción y contra quién), el comando *hace* y `Combat` solo *ejecuta*.

Por qué funciona (parte 2). Cada comando apunta, antes de tocar nada, la vida, el escudo y la cadena de estadísticas del combatiente que cambia; `UseItem` apunta además la referencia al `Inventory` anterior (no hace falta copiarlo, es inmutable). La pila devuelve primero lo último: en `Check07`, la bomba, luego la defensa y luego el ataque, y las cuatro primeras líneas muestran cada estado recuperado. La poción de la sexta y séptima línea cura 12 (Aria estaba en 14 de 30) y su `undo` devuelve la vida y la poción. `enemyPhase()` vacía la historia al terminar, y por eso la última línea es `false`. En `ConsoleUi`, cada héroe vivo aporta un comando por ronda, así que el índice `i` coincide con el número de comandos apilados: `u` deshace el del héroe anterior (`i` baja y ese héroe vuelve a decidir). Con `i = 0` la pila está vacía y `combat.undo()` devuelve `false`.

Por qué funciona (parte 3). `ConsoleUi` no sabe de dónde vienen las teclas, solo llama a

`input.next()`: con `ConsoleInput` salen del teclado, con `ScriptInput`, del archivo. Todo lo demás es el mismo código, y la semilla 42 es la de siempre, así que dos ejecuciones son idénticas. La predicción: el Zombi usa «Abrazo pútrido» en la ronda 3 (`3 % 3 == 0`). Su poder (7) no pasa por la fórmula: va directo a la tubería, donde el escudo de Aria (5) absorbe 5 y deja 2, y la armadura (1) lo deja en 1. Un ataque normal habría hecho $5 - (3 + 2) = 0$ y el escudo lo habría absorbido entero: "pero no le hace daño". Por eso sale 1 y no 0. La partida acaba en la ronda 4 con la vida de los héroes en 27, 40 y 24.

Errores típicos:

- Apuntar el estado *después* de tocarlo (o no apuntar la cadena de estadísticas en `Defend`): el `undo()` no cambia nada, o deja el buff y el escudo puestos.
- Restaurar el inventario en `UseItem.undo()` y olvidar la vida del objetivo (o al revés): la poción vuelve, pero la vida no.
- Apilar el comando antes de ejecutarlo: si `execute()` falla, queda en la pila una acción que nunca ocurrió.
- No vaciar la historia al cerrar la ronda: `u` podría deshacer acciones de una ronda ya cerrada, con el contador de rondas y los buffs ya avanzados.
- Que una IA use `Math.random()` o un `new Random()` propio: el replay deja de repetirse.
- Pasar el guion como primer argumento a `chooseFloor`: lanza `NumberFormatException`. Se separa antes con `scriptOf` y `optionsOf`.
- Dejar los comentarios `#` del guion sin filtrar: `ScriptInput` los tomaría como teclas, y el menú respondería «Tecla no válida.».

Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo 8: `engine/combat/command/` con la pila `CommandHistory`, `Fighter` con `restore`, `heal` y `skills()`, héroes con habilidades, `ui/input/` y `scripts/demo.txt`. Borra `app/Check07.java`.

CAPÍTULO 8

Memento: instantáneas del estado, con JUnit

Objetivos del capítulo

- Reconocer el olor que pide un Memento: volver a un punto anterior de la partida cuando cambian muchas cosas a la vez y no todas son acciones que se puedan invertir.
- Aplicar Memento con un originador que crea y restaura su estado, una instantánea inmutable y un cuidador que la guarda sin mirar dentro.
- Explicar por qué una instantánea solo es inmutable si lo que guarda también lo es, y qué pasa si guarda referencias mutables.
- Elegir entre Command y Memento para deshacer, y declarar los límites de Memento (historia obsoleta, azar no guardado).
- Añadir JUnit al proyecto, escribir una prueba con `@Test` y `assertEquals` y ejecutarla con `./gradlew test`.
- Dar el hito H15a: puntos de control en memoria con las teclas `c` y `r`.

En el capítulo anterior aprendiste a deshacer una acción. Es útil, pero corto de miras: `u` solo llega hasta el principio de la ronda en curso, y deshace de una en una. Un juego de mazmorras pide algo más generoso: guardar la partida antes de entrar en una sala peligrosa y volver a ese punto si sale mal, con todo como estaba, aunque entre medias hayan pasado tres rondas, se hayan gastado objetos, hayan caído enemigos y se hayan acabado los efectos temporales. Eso es una *instantánea* del estado, y el patrón que la organiza es Memento. Hay además una segunda razón para que este capítulo exista: Memento es el primer lugar del curso donde una prueba automática demuestra algo que de otro modo tendrías que comprobar a mano (que el estado *vuelve*). Así que también instalarás JUnit, la librería de pruebas de Java, y escribirás tu primera prueba.

8.1 El olor que pide Memento: deshacer cuando hay cien cambios

Piensa en lo que ocurre en Emberdeep durante tres rondas: cada héroe actúa, cada enemigo actúa, los escudos se gastan, los efectos temporales avanzan un turno y se apagan, el contador de ronda sube, el inventario cambia. Quieres que la tecla `c` guarde ese momento y que la tecla `r` lo recupere. Con lo que ya sabes, hay dos caminos y los dos huelen mal.

Camino 1: usar los comandos del capítulo 7. No sirve. `CommandHistory` se vacía al cerrar cada ronda, porque el contador de rondas y el avance de los efectos no son comandos. Y aunque la ampliaras, tendrías que escribir el inverso de *cada* cambio que ocurre entre el punto guardado y el actual, incluidos los de la IA de los enemigos y los del azar. Cuantos más cambios, más código de deshacer, y todo él tiene que estar de acuerdo con el código de hacer.

Camino 2: guardar los datos a mano desde fuera. Podrías escribir, por ejemplo en la interfaz, algo así (el código es hipotético; no existe en el proyecto):

Hipotético: guardar el estado a mano desde la interfaz

```
// al pulsar "c"
int[] hps = new int[all.size()];
int[] shields = new int[all.size()];
StatSource[] chains = new StatSource[all.size()];
for (int i = 0; i < all.size(); i++) {
    hps[i] = all.get(i).hp();
    shields[i] = all.get(i).shield();
    chains[i] = all.get(i).stats();
}
Inventory inventory = session.inventory();
int round = session.round();
// al pulsar "r": un bucle que repone cada cosa, y alguien que
// pone la ronda y el inventario otra vez ...
```

Funciona hoy, y tiene tres problemas que crecen juntos:

1. **Quien guarda tiene que conocer todos los campos.** Si mañana un combatiente gana un estado (el capítulo 12 añade veneno y aturdimiento), alguien tiene que acordarse de añadirlo aquí. Si se olvida, `r` devuelve una partida a medias, sin ningún error: es de los fallos más difíciles de ver.
2. **Hay que abrir los objetos.** Para poner la ronda y el inventario donde estaban hace falta que `Combat` y `GameSession` tengan métodos que cualquiera pueda llamar para cambiarles los campos a la fuerza. Cada uno de esos métodos es una puerta abierta a dejar el estado incoherente.
3. **El guardado está pegado a la interfaz.** Cuando el capítulo 14 quiera guardar en un fichero, ese código habría que copiarlo a otro sitio.

Este es el olor: **volver a un punto anterior exige copiar y reponer muchos datos, y esa tarea acaba repartida por fuera de quien sabe lo que tiene dentro**. Memento lo cura dándole la tarea al propio objeto. Su intención en el catálogo es capturar y sacar fuera el estado interno de un objeto, sin violar su encapsulación, para poder devolverlo a ese estado más tarde [9]. Head First lo incluye entre los patrones de su apéndice [7], y el catálogo de Refactoring.Guru lo resume como guardar y restaurar el estado anterior de un objeto sin revelar los detalles de su implementación [24]. La idea cabe en una frase: el objeto *entrega* una instantánea cerrada de sí mismo, otro la guarda sin saber qué hay dentro, y más tarde el objeto *se repone* desde ella.

8.2 Piezas del patrón Memento

Rol	Clase en el ejemplo	Propósito	Paquete
Originador (clase)	`GameSession`	Sabe qué forma el estado de la partida: crea la instantánea (`createSnapshot`) y se repone desde ella (`restore`)	`engine.session`
Memento (records)	`Snapshot` y `FighterState`	Copia inmutable del estado: ronda, posición, inventario y, por combatiente, vida, escudo y estadísticas	`engine.save`
Cuidador (clase)	`SnapshotStore`	Guarda las instantáneas en una pila y devuelve la última; no mira dentro	`engine.save`
Cliente (clase)	`ConsoleUi`	Decide cuándo guardar y cuándo volver (teclas `c` y `r`)	`ui`

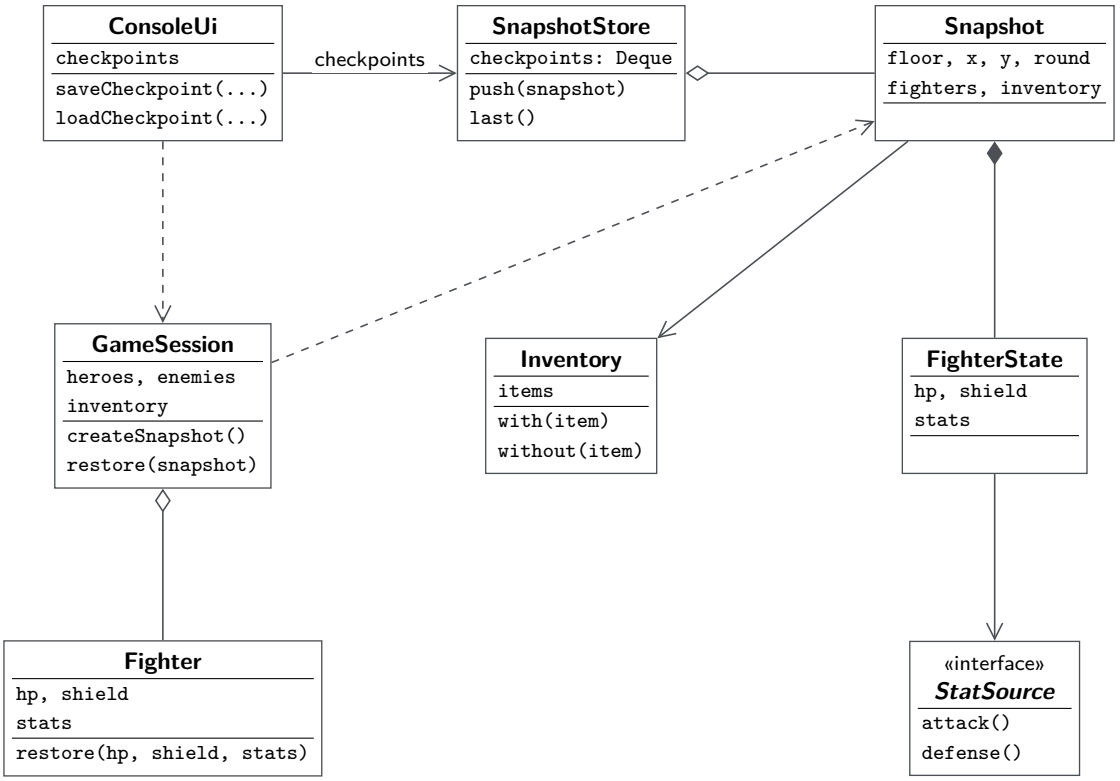


Diagrama UML del patrón Memento: el cliente pide al originador que se guarde, el cuidador apila la instantánea y el originador se repone desde ella.

Una nota sobre el *encapsulado*, que es la palabra clave del patrón. En el diseño original, el cuidador solo ve una interfaz «estrecha» de la instantánea (no puede leer ni cambiar nada) y el

originador ve una «ancha». Java no tiene esa distinción entre clases sin trucos, y un `record` expone todo lo que guarda. Aquí no es un problema, y por una razón concreta: `Snapshot` es *inmutable*. El cuidador puede mirarla, pero no puede estropearla, así que la protección que se busca (que nadie cambie lo guardado) se consigue por otro camino.

8.3 El ejemplo de Memento, pieza a pieza

La instantánea de un combatiente. Por cada combatiente hace falta lo que cambia durante la partida: su vida, su escudo y su cadena de estadísticas (el equipo más los efectos temporales, capítulo 6). Un `record` de tres campos lo dice todo:

engine/save/FighterState.java

```
package emberdeep.engine.save;

import emberdeep.engine.stats.StatSource;

public record FighterState(int hp, int shield, StatSource stats) {}
```

Fíjate en lo que *no* lleva: ni el nombre, ni el tipo de enemigo, ni la IA, ni el equipo permanente. Eso no cambia durante una partida, así que no hace falta guardarlo. Lo que sí cambia es la cadena `stats`, y se guarda *por referencia*, sin copiarla. Ya viste por qué se puede: los decoradores del capítulo 6 son inmutables, y `tick()` no modifica la cadena, devuelve otra. Quien tenga hoy una referencia a la cadena la seguirá viendo igual dentro de diez rondas. El equipo y los efectos temporales vuelven solos, sin ningún catálogo de equipo ni identificador.

La instantánea de la partida.

engine/save/Snapshot.java

```
package emberdeep.engine.save;

import emberdeep.engine.item.Inventory;
import java.util.List;

public record Snapshot(int floor, int x, int y, int round,
                       List<FighterState> fighters, Inventory inventory) {
    public Snapshot {
        fighters = List.copyOf(fighters);
    }
}
```

Cuatro números (planta, posición del grupo y ronda), la lista de estados de los combatientes y el inventario. El constructor compacto copia la lista con `List.copyOf`, como ya hicieron `Hero` y `Inventory`: si quien la creó cambia después su lista, la instantánea no se entera. El `Inventory` también se guarda por referencia, sin copiarlo, porque desde el capítulo 3 es inmutable (`with` y `without` devuelven uno nuevo y `GameSession` sustituye el suyo). Lo viste en acción en `UseItem`: guardar «el inventario de antes» costaba una referencia.

El originador. `GameSession` es quien sabe de qué se compone el estado. Gana tres cosas. La primera, dónde está el grupo: una planta y una casilla (en este capítulo el combate no mueve al grupo, así que son los de la entrada de la planta). Las otras dos son la propia pareja de métodos del patrón, `createSnapshot` y `restore`:

engine/session/GameSession.java (fragmento: lo que añade el cap. 8)

```
private int floor = 1;
private int x;
private int y;

public int floor() { return floor; }
public int x() { return x; }
public int y() { return y; }

public void setLocation(int floor, int x, int y) {
    this.floor = floor;
    this.x = x;
    this.y = y;
}

private List<Fighter> allFighters() {
    return Stream.concat(heroes.stream(), enemies.stream()).toList();
}

public Snapshot createSnapshot() {
    List<FighterState> states = allFighters().stream()
            .map(f -> new FighterState(f.hp(), f.shield(), f.stats()))
            .toList();
    return new Snapshot(floor, x, y, combat.round(), states, inventory);
}
```

(Los `import` de este fragmento son `java.util.stream.Stream`, `Snapshot` y `FighterState`; el archivo completo está en la solución del hito.)

Hay un detalle de orden: los estados salen *en el orden de la lista de combatientes* (primero los héroes, luego los enemigos). La instantánea no guarda el nombre de nadie; sabe que el primer estado es del primer combatiente porque nace y se repone con el mismo orden. Es el tipo de acuerdo silencioso que hay que proteger con una comprobación:

engine/session/GameSession.java (fragmento: restore)

```
public void restore(Snapshot snapshot) {
    List<Fighter> all = allFighters();
    if (snapshot.fighters().size() != all.size()) {
        throw new IllegalArgumentException(
                "La instantánea es de otra partida");
    }
    for (int i = 0; i < all.size(); i++) {
        FighterState state = snapshot.fighters().get(i);
        all.get(i).restore(state.hp(), state.shield(), state.stats());
    }
    combat.restoreRound(snapshot.round());
    inventory = snapshot.inventory();
    floor = snapshot.floor();
    x = snapshot.x();
    y = snapshot.y();
}
```

Dos decisiones que merecen una explicación. La primera: `restore` *no crea combatientes nuevos*, repone en su sitio los que ya existen. Tiene que ser así, porque `Combat` guarda las mismas listas y

los mismos objetos `Fighter`; si `GameSession` los sustituyera, `Combat` seguiría mirando los viejos. Para reponer cada uno se reutiliza `Fighter.restore(hp, shield, stats)`, que escribiste en el capítulo anterior para los `undo()`. La segunda: la ronda vive en `Combat`, que no tenía forma de cambiarla, así que gana un método pequeño y explícito, `restoreRound(int round)`, que rechaza rondas menores que 1.

El cuidador. Guardar instantáneas y devolver la última es una pila, la misma estructura `Deque` del capítulo anterior:

`engine/save/SnapshotStore.java`

```
package emberdeep.engine.save;

import java.util.ArrayDeque;
import java.util.Deque;

public class SnapshotStore {
    private final Deque<Snapshot> checkpoints = new ArrayDeque<>();

    public void push(Snapshot snapshot) {
        checkpoints.push(snapshot);
    }

    public Snapshot last() {
        if (checkpoints.isEmpty()) {
            throw new IllegalStateException("No hay puntos de control");
        }
        return checkpoints.peek();
    }

    public boolean isEmpty() { return checkpoints.isEmpty(); }
    public int size() { return checkpoints.size(); }
}
```

`last()` usa `peek`, no `pop`: volver a un punto de control *no lo gasta*, y puedes volver a él las veces que quieras (en un juego, es lo que esperas de una partida guardada). El cuidador no sabe qué hay dentro de un `Snapshot`: no llama a ninguno de sus métodos. Y en el uso, el cliente es tan corto como esto:

`Uso de las tres piezas`

```
Snapshot saved = session.createSnapshot();   // c: el originador se entrega
store.push(saved);                           //    el cuidador la guarda
// ... pasan cosas ...
session.restore(store.last());               // r: el originador se repone
```

Un ejemplo con números. Aria tiene 30 de vida. Guardas. Un golpe la deja en 20. Restauras. Aria vuelve a tener 30, *es el mismo objeto* `Fighter` de antes (la instantánea no lo sustituye, lo repone), y el escudo, las estadísticas, el inventario y la ronda vuelven con ella. Es exactamente lo que comprobará la prueba de la sección 8.6.

8.4 Memento: el memento que guarda referencias mutables

Una instantánea es solo tan inmutable como lo que guarda. Es la misma trampa de la copia superficial del capítulo 5: `List.copyOf` duplica la *lista*, no los elementos que hay dentro. Si la lista contiene objetos mutables, el original y la «copia» comparten esos objetos. Mira una instantánea mal diseñada, que guarda los propios combatientes (hipotético; no existe en el proyecto):

Hipotético: una instantánea que guarda los Fighter

```
record BadSnapshot(List<Fighter> fighters) {
    BadSnapshot {
        fighters = List.copyOf(fighters);     // copia la lista, no a los
    ↪ Fighter
    }
}

// en un programa de prueba:
BadSnapshot bad = new BadSnapshot(session.heroes());
Fighter aria = session.heroes().get(0);
aria.takeDamage(10);
System.out.println(bad.fighters().get(0).hp());
```

Aria tenía 30 de vida y recibe 10 de daño. Lo que imprime el programa es `20`, no `30`: la «instantánea» apunta al mismo `Fighter` que acaba de cambiar. Guardó una referencia, no el estado. Restaurar desde ella no restaura nada.

La regla es: **lo que entra en un memento tiene que ser un valor o algo inmutable**. Repasa lo que guarda `Snapshot` y por qué es seguro:

Dato	Cómo se guarda	Por qué es seguro
`floor`, `x`, `y`, `round`, `hp`, `shield`	Por valor (`int`)	Un `int` se copia al guardarlo; nadie puede cambiar la copia.
`stats` (cadena de `StatSource`)	Por referencia	Los decoradores son inmutables (capítulo 6): `tick()` devuelve otra cadena y no toca la que guardaste.
`inventory`	Por referencia	`Inventory` es un `record` que copia su lista con `List.copyOf`, y sus `Item` son records (capítulo 3).
`fighters` (lista de `FighterState`)	Copia de la lista	`List.copyOf` basta porque los elementos son records de valores o inmutables.

Y la lección vale al revés. Si algún día un decorador cambiara un campo suyo en lugar de devolver otro (un `TempBuff` con un contador `turnsLeft--`), la cadena guardada iría cambiando por debajo de la instantánea y volvería «a medias». Que los decoradores sean inmutables no es un capricho del capítulo 6: es lo que permite que el Memento de este capítulo sea barato. Cuando no puedas garantizar la inmutabilidad, la alternativa es copiar de verdad (copia profunda, como el `copy()` de `EnemyTemplate`) o guardar solo valores y reconstruir el objeto al volver.

8.5 Memento o Command: dos formas de deshacer

Los dos patrones sirven para volver atrás, pero resuelven problemas distintos. Esta tabla los compara con lo que ya tienes en el proyecto:

	Command (capítulo 7)	**Memento (este capítulo)**
Qué guarda	Solo lo que toca *esa* acción (`hpBefore`, `inventoryBefore`).	El estado entero (o la parte que decidas) en un instante.
Qué deshace	Una acción cada vez, en orden inverso.	Cualquier número de cambios de golpe: vuelve a un punto.
Coste de memoria	Pequeño por acción.	Una instantánea completa, aunque entre medias haya cambiado poco.
Coste de escribir	Hace falta un `undo()` correcto por cada comando.	Un par `createSnapshot`/`restore`, pero hay que mantenerlo al día con cada campo nuevo.
Acciones que no se pueden invertir	Hay que guardar el valor anterior o no permitir deshacer.	No importa: se repone el valor, no se invierte la acción.
Riesgo típico	Un cambio hecho por fuera de un comando deja el `undo()` en un estado que ya no es el de antes.	Olvidar un campo en la instantánea: `restore` devuelve una partida a medias.
En Emberdeep	`u`: deshacer lo último de la ronda en curso.	`c` y `r`: volver a un punto aunque hayan pasado rondas.

No son rivales. Un juego puede tener los dos, y Emberdeep los tiene: `u` para el error de un segundo y `r` para la sala que salió mal. Sí tienen un punto de fricción que verás en la sección 8.7: no se enteran el uno del otro. Además, hay un híbrido útil: un comando cuyo `undo()` usa una instantánea. Lo explorarás en el ejercicio 8.5.

8.6 Instalación y primeros pasos con JUnit

Hasta ahora has comprobado tu código ejecutándolo y mirando la salida, o con programas temporales como `Check07`. Funciona, pero cada comprobación la haces tú, a mano, y la pierdes cuando borras el archivo. Una **prueba automática** es un trozo de código que comprueba otro y dice «bien» o «mal» sin que mires nada. Se queda en el proyecto y se vuelve a ejecutar cada vez que cambias algo. En Java, la librería estándar de hecho para escribirlas es JUnit.

Qué versión. La guía oficial es la de JUnit 6.1.3 y dice que JUnit necesita Java 17 o superior para ejecutarse [15]. Con el Java 21 del proyecto va sobrada, así que **este curso usa JUnit 6.1.3**. Para Gradle, la guía indica dos cosas: que desde la versión 4.6 sabe ejecutar pruebas de la plataforma JUnit si lo activas con `useJUnitPlatform()` en la tarea `test`, y que se declare también la dependencia `junit-platform-launcher` (el componente que Gradle usa para arrancar las pruebas), que en algunos casos es obligatoria [15]; declárala siempre. Siguiendo ambas, y con el wrapper 9.8.2 de tu proyecto, las pruebas funcionan. El resto, de dónde salen las dependencias, qué es una configuración como `testImplementation` y dónde van los fuentes de prueba (`src/test/java`), es el funcionamiento normal del plugin de Java de Gradle [11].

☞ JUnit 6.1.3 con Gradle

1. `build.gradle`. Cambia tres cosas: añade el repositorio de donde se descargan las librerías, añade las dos dependencias de prueba y activa la plataforma JUnit en la tarea `test`. Este es el archivo completo:

build.gradle

```
plugins {
    id 'application'
}

repositories {
    mavenCentral()
}

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

application {
    mainClass = 'emberdeep.app.Main'
}

run {
    standardInput = System.in
}

dependencies {
    testImplementation 'org.junit.jupiter:junit-jupiter:6.1.3'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

test {
    useJUnitPlatform()
}
```

2. La carpeta de pruebas. Gradle busca las pruebas en `src/test/java`, con los mismos paquetes que el código que prueban:

```
$ mkdir -p src/test/java/emberdeep/engine/save
```

3. Ejecutar. Con la primera prueba escrita (la tienes más abajo), lanza:

```
$ ./gradlew test
```

La primera vez Gradle descarga las librerías de JUnit desde Maven Central, así que hace falta conexión. Si todo va bien, la salida acaba en `BUILD SUCCESSFUL`. El informe detallado queda en `build/reports/tests/test/index.html`.

Qué hace cada pieza nueva de `build.gradle`:

- `repositories { mavenCentral() }` dice dónde buscar las librerías. Hasta ahora el pro-

yecto no usaba ninguna, por eso no hacía falta.

- `testImplementation` añade una librería *solo* al código de prueba: `junit-jupiter` trae lo necesario para escribir pruebas (la anotación `@Test` y las aserciones) y ejecutarlas. El código del juego no depende de ella.
- `testRuntimeOnly` la añade solo al ejecutar las pruebas, sin que el código las pueda usar al compilar. El `launcher` entra aquí.
- `test { useJUnitPlatform() }` le dice a Gradle que las pruebas son de JUnit Platform. Si lo olvidas, `./gradlew test` no se queja y no ejecuta ninguna: es un error silencioso, de los peores.

Anatomía de una prueba. Una prueba de JUnit es un método con la anotación `@Test` dentro de una clase normal. Si el método termina sin lanzar nada, la prueba pasa. Si una *aserción* (`assertEquals` y compañía) no se cumple, lanza un error y la prueba falla. La forma habitual de organizarla son tres pasos: preparar el escenario, hacer lo que se prueba y comprobar el resultado. Esta es la prueba más pequeña posible para Memento, la que dice «restaurar devuelve la vida guardada»:

Fragmento: la prueba mínima

```
@Test
void restoreBringsBackSavedHp() {
    GameSession session = newSession();              // preparar
    Fighter aria = session.heroes().get(0);
    Snapshot saved = session.createSnapshot();

    aria.takeDamage(10);                             // hacer
    session.restore(saved);

    assertEquals(30, aria.hp());                     // comprobar
}
```

`assertEquals(esperado, obtenido)` compara dos valores y falla si no son iguales. El orden importa solo para el mensaje: con el orden invertido la prueba sigue funcionando, pero el error dirá «esperado 20» cuando lo esperado era 30, y perderás tiempo. Si la prueba falla, JUnit dice algo como `expected: <30> but was: <20>` y Gradle marca la prueba con `FAILED`.

La primera prueba del proyecto. Va en el mismo paquete que lo que prueba, y es la única prueba que tendrá Emberdeep: comprueba, a la vez, la vida, el escudo y el inventario. Necesita las piezas de la sección anterior en tu proyecto (`Snapshot`, `FighterState`, `SnapshotStore` y los dos métodos de `GameSession`); si estás leyendo sin escribir código, la ejecutarás en el paso 3 del hito.

src/test/java/emberdeep/engine/save/SnapshotTest.java

```
package emberdeep.engine.save;

import static org.junit.jupiter.api.Assertions.assertEquals;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Stats;
```

```
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.biome.CryptBiome;
import java.util.List;
import java.util.Random;
import org.junit.jupiter.api.Test;

class SnapshotTest {

    private static GameSession newSession() {
        Random random = new Random(42);
        List<Hero> heroes = List.of(
                new HeroBuilder().name("Aria")
                        .stats(new Stats(30, 8, 3)).build());
        return new GameSession(heroes, new CryptBiome(random),
                List.of(EnemyKind.SKELETON),
                new FlatFormula(), new DamagePipeline(random, 0.0));
    }

    @Test
    void restoreBringsBackSavedState() {
        GameSession session = newSession();
        Fighter aria = session.heroes().get(0);
        aria.addShield(5);
        Snapshot saved = session.createSnapshot();

        aria.takeDamage(10);
        aria.absorbShield(5);
        session.replaceInventory(new Inventory(List.of()));
        session.restore(saved);

        assertEquals(30, aria.hp());
        assertEquals(5, aria.shield());
        assertEquals(3, session.inventory().items().size());
    }
}
```

Lee la prueba despacio. `newSession()` monta una partida pequeña (un héroe y un esqueleto) con la semilla de siempre; cada prueba crea la suya, para que no dependa de ninguna otra. Se guarda con 30 de vida, 5 de escudo y 3 objetos. Después se estropea todo: Aria recibe 10 de daño, gasta el escudo y se vacía el inventario. Tras `restore`, las tres aserciones comprueban que cada cosa ha vuelto. Los nombres de los métodos de prueba dicen *qué comportamiento* se verifica (`restoreBringsBackSavedState`), no cómo. Además, la clase y el método no son `public`: JUnit no lo necesita.

Otras aserciones que usarás en el curso: `assertTrue` y `assertFalse` (para condiciones), `assertSame` (dos referencias al mismo objeto) y `assertThrows` (comprueba que algo lanza una excepción). Esta última recibe la clase de la excepción y una lambda (capítulo 2) con el código que debe fallar:

Fragmento: assertThrows (no forma parte de SnapshotTest)

```
import static org.junit.jupiter.api.Assertions.assertThrows;
```

```
2
3 @Test
4 void emptyStoreHasNoLastCheckpoint() {
5     SnapshotStore store = new SnapshotStore();
6     assertThrows(IllegalStateException.class, () -> store.last());
7 }
```

Como `assertEquals`, hay que importarla de forma estática, como se ve en la primera línea.

Errores típicos con JUnit:

- Olvidar `useJUnitPlatform()`: `./gradlew test` termina bien sin ejecutar nada. Si alguna vez dudas, rompe una aserción a propósito y comprueba que falla.
- Poner el fuente de prueba en `src/main/java`: Gradle lo compila como código del juego y falla porque no encuentra `org.junit`.
- Olvidar el `@Test`: el método se ignora sin avisar.
- Escribir pruebas que dependen unas de otras (una usa lo que dejó la anterior): cada prueba prepara su propio escenario.
- Un error de resolución como `Could not resolve org.junit.jupiter:junit-jupiter`: falta el bloque `repositories` o no hay conexión.

Más adelante, esta misma prueba te servirá de alarma: cada vez que una instantánea gane un campo nuevo (los capítulos 12 y 13 lo harán), la prueba tendrá que crecer, y ejecutarla con `./gradlew test` te avisará si te olvidas de guardar o de reponer algo. Eso es, justo, el coste de Memento del que habla la tabla anterior.

8.7 Memento: cuándo sí y cuándo no

Compensa cuando varias cosas cambian a la vez y quieres volver a un punto entero: puntos de control, «guardar partida», deshacer una operación grande o que se pueda hacer sobre una copia («probar» algo y descartarlo). También cuando algunas acciones no se pueden invertir: no importa, porque no deshaces la acción, repones el valor. Y cuando el estado es pequeño y está hecho de valores o de objetos inmutables, como en Emberdeep: entonces la instantánea es barata y segura.

No compensa si el estado es muy grande y casi todo es igual de una instantánea a la siguiente (guardas lo mismo mil veces; ahí guardar diferencias, o usar Command, gasta menos), si el estado vive fuera de tu programa (un archivo, la red: no lo puedes reponer) o si basta deshacer la última acción y tienes ya un `CommandHistory`. Su coste más constante es el mantenimiento: cada campo nuevo del originador tiene que acordarse de entrar en la instantánea y en `restore`. En Emberdeep lo verás dos veces más (los capítulos 12 y 13), y la prueba de la sección anterior estará ahí para avisarte.

Y hay dos límites que **esta versión de Memento no resuelve** y que debes conocer:

1. **El `CommandHistory` queda obsoleto tras `restore`.** Cada comando de la pila guarda los valores de antes *de su ejecución*. Imagina que Aria ataca al Esqueleto (comando A), guardas con `c`, Borin lanza la bomba (B) y Cael ataca (C). La pila contiene A, B y C. Pulsas `r`: el estado vuelve al de antes de B. Pero la pila sigue teniendo A, B y C. Si ahora pulsas `u`, se deshace C, que repone al Esqueleto con la vida que tenía *después* de la bomba: el efecto de B, que habías cancelado, reaparece. La interfaz tampoco sabe a quién

le tocaba. Dos arreglos posibles son vaciar la historia en `restore` o guardar también la pila en la instantánea. Ninguno se aplica en este curso: se declara el límite y el hito evita el peor efecto, que `u` retroceda más allá del primer héroe.

2. **El estado del `Random` no se guarda.** `RandomTargetAi` consume números del mismo `Random` que el resto de la partida. Al restaurar, los combatientes vuelven a su estado, pero el generador sigue donde estaba: la partida restaurada *no repite exactamente lo que ocurrió después*. Es inofensivo para un jugador (de hecho, es lo que se espera de volver a intentarlo), pero rompe una propiedad que viste en el capítulo 7: con un guion de teclas, la partida ya no es reproducible si usas `r`. Guardarlo no es posible con `java.util.Random`, que no ofrece una forma de leer su estado interno.

Declarar un límite no es ocultar un fallo. Está aquí, escrito, para que el siguiente que toque el código sepa dónde pisa.

8.8 En el proyecto: H15a con Memento

Este capítulo avanza el hito H15a, y se hace en **tres pasos** que se comprueban uno a uno. El orden de los hitos del libro no es monótono: H15a va aquí, antes de H10, porque Memento se apoya en Command (capítulo 7) y la prueba que lo acompaña es el mejor sitio para enseñar JUnit. El hito H15b, en el capítulo 14, llevará este mismo `Snapshot` a un fichero. Cambios del hito:

- **Nuevo paquete** `engine/save/`: `Snapshot`, `FighterState` y `SnapshotStore`.
- **Nueva carpeta** `src/test/java/`, con `SnapshotTest` (la única prueba del proyecto, sección anterior), y `build.gradle` con JUnit.
- **Cambian** `GameSession` (`floor`, `x`, `y`, `setLocation`, `createSnapshot` y `restore`), `Combat` (`restoreRound`), `ConsoleUi` (teclas `c` y `r`) y `Main` (da a la sesión su planta y su casilla de entrada).
- **No cambian** `Fighter` (reutiliza el `restore` del capítulo 7), `engine/stats/`, `engine/combat/command/` ni el mapa.

Qué cambia en la partida. Muy poco, a propósito: la línea «Turno de ...» del menú gana al final «`[c] guardar`» y «`[r] volver`», y por lo demás la partida es la de siempre. Las 108 líneas de salida de `scripts/demo.txt` siguen siendo 108, y los resultados de las trazas son los mismos. Lo nuevo aparece solo si pulsas `c` o `r`.

Los tres pasos y qué debes ver en cada uno.

- **Paso 1, las piezas.** `Snapshot`, `FighterState`, `SnapshotStore`, y en `GameSession` y `Combat` el par `createSnapshot`/`restore` y `restoreRound`. Resultado: un programa de comprobación guarda tras un `Defend`, estropea todo (daño que gasta el escudo, un efecto que se acaba, una bomba) y restaura: la vida, el escudo, la defensa, la vida del enemigo y el inventario vuelven a lo guardado.
- **Paso 2, las teclas.** `c` guarda un punto de control y `r` vuelve al último. Resultado: un guion de teclas guarda, lanza una bomba, vuelve y muestra al enemigo y los objetos como estaban.
- **Paso 3, la prueba.** `build.gradle` con JUnit y `SnapshotTest`. Resultado: `./gradlew test` termina con `BUILD SUCCESSFUL`, y falla si rompes a propósito el `restore`.

Hito 15a: puntos de control en memoria con Memento

Tras este hito, el jugador puede guardar un punto de control con `c` y volver al último con `r`, tantas veces como quiera, aunque hayan pasado rondas. La partida se guarda en una instantánea inmutable y el proyecto tiene su primera prueba automática. La estructura:

Estructura de carpetas tras el capítulo 8

```
emberdeep/
|-- settings.gradle
|-- build.gradle                    (con JUnit 6.1.3)
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
|-- scripts/
|   `-- demo.txt
`-- src/
    |-- main/java/emberdeep/
    |   |-- app/
    |   |   |-- Main.java              (composition root)
    |   |   `-- GameConfig.java
    |   |-- engine/
    |   |   |-- character/             Stats, Hero, HeroBuilder, Enemy,
    ↪ EnemyKind,
    |   |   |                          Fighter, Skill, EnemyTemplate,
    ↪ TemplateRegistry
    |   |   |-- stats/                 StatSource, StatKind, BaseStats,
    ↪ StatDecorator,
    |   |   |                          WeaponBonus, ArmorBonus, TempBuff
    |   |   |-- combat/
    |   |   |   |-- Combat.java
    |   |   |   |-- ai/                EnemyAi, FirstAliveAi, LowestHpAi,
    |   |   |   |                      RandomTargetAi, SkillfulAi
    |   |   |   |-- command/           CombatCommand, Attack, Defend,
    ↪ UseItem,
    |   |   |   |                      UseSkill, CommandHistory
    |   |   |   |-- damage/            DamageFormula, FlatFormula,
    ↪ ScaledFormula,
    |   |   |   |                      Damage, DamageType
    |   |   |   `-- defense/           DamageHandler, DamagePipeline,
    ↪ DodgeHandler,
    |   |   |                          ShieldHandler, ArmorHandler,
    ↪ ResistanceHandler
    |   |   |-- item/                  Item, ItemKind, Inventory
    |   |   |-- save/                  Snapshot, FighterState,
    ↪ SnapshotStore
    |   |   |-- session/               GameSession
    |   |   `-- world/
    |   |       |-- Dungeon.java, Floor.java, DungeonBuilder.java
    |   |       |-- biome/             BiomeFactory, CryptBiome,
    ↪ CavernBiome
    |   |       `-- map/               TileType, TileTypes, GameMap,
    ↪ MapParser,
    |   |                              MapData
    |   `-- ui/
    |       |-- ConsoleUi.java
    |       `-- input/                 InputSource, ConsoleInput,
    ↪ ScriptInput
    `-- test/java/emberdeep/
        `-- engine/save/               SnapshotTest
```

Lo practicas en el ejercicio 8.4. Antes, tres ejercicios más pequeños, en otros dominios.

8.9 Ejercicios

Ejercicio 8.1: Un editor con instantáneas ★★☆☆☆

No es de Emberdeep. Crea una carpeta vacía y, dentro, archivos sin paquete. Vas a añadirle instantáneas a un editor de texto con cursor. Este es el editor de partida; guárdalo como `Editor.java`:

`Editor.java` (de partida)

```
public class Editor {
    private String text = "";
    private int cursor;

    public void type(String s) {            // inserta s donde está el
    ↪ cursor
        text = text.substring(0, cursor) + s + text.substring(cursor);
        cursor += s.length();
    }

    public void moveTo(int position) {      // el cursor no sale del texto
        cursor = Math.max(0, Math.min(position, text.length()));
    }

    public String text() { return text; }
    public int cursor() { return cursor; }
}
```

1. `EditorState.java`: un `record` inmutable `EditorState(String text, int cursor)`. Será el memento.
2. En `Editor`, añade `EditorState snapshot()`, que devuelve un `EditorState` con el texto y el cursor actuales, y `void restore(EditorState state)`, que pone ambos campos con lo que trae el estado. El editor es el originador.
3. `History.java`: el cuidador. Guarda estados en un `Deque<EditorState>` (`ArrayDeque`) con `void save(EditorState state)` (`push`), `EditorState pop()` (`pop`) y `boolean isEmpty()`.
4. `Main.java`: copia este código y ejecútalo con `javac *.java` y `java Main`:

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Editor editor = new Editor();
        History history = new History();
        editor.type("hola");
        history.save(editor.snapshot());
        editor.type(" mundo");
        history.save(editor.snapshot());
        editor.moveTo(0);
        editor.type(">> ");
        show(editor);
        editor.restore(history.pop());
        show(editor);
        editor.restore(history.pop());
        show(editor);
    }

    private static void show(Editor editor) {
        System.out.println("[" + editor.text() + "] cursor=" + editor.
    ↪ cursor());
    }
}
```

Está bien si imprime las tres líneas siguientes y sabes decir qué papel hace cada clase (originador, memento, cuidador) y por qué `EditorState` no necesita copiar nada más que dos valores:

Salida esperada

```
[>> hola mundo] cursor=3
[hola mundo] cursor=10
[hola] cursor=4
```

Pista: *El segundo `pop()` devuelve la instantánea más antigua, la primera que se guardó: una pila devuelve primero lo último que entró. Un `String` es inmutable, así que guardar la referencia al texto es seguro.*

Guiado ▪ solución en la pág. 305

Ejercicio 8.2: Dos pruebas para el editor ★★☆☆☆

No es de Emberdeep, pero usa el `build.gradle` con JUnit de la sección 8.6. Estas son las dos clases que vas a probar, ya completas (son las del ejercicio anterior con un `package`). Guárdalas en `src/main/java/scratch/`:

src/main/java/scratch/EditorState.java

```java
package scratch;

public record EditorState(String text, int cursor) {}
```

src/main/java/scratch/Editor.java

```java
package scratch;

public class Editor {
    private String text = "";
    private int cursor;

    public void type(String s) {
        text = text.substring(0, cursor) + s + text.substring(cursor);
        cursor += s.length();
    }

    public void moveTo(int position) {
        cursor = Math.max(0, Math.min(position, text.length()));
    }

    public EditorState snapshot() { return new EditorState(text, cursor); }

    public void restore(EditorState state) {
        text = state.text();
        cursor = state.cursor();
    }

    public String text() { return text; }
    public int cursor() { return cursor; }
}
```

Crea `src/test/java/scratch/EditorTest.java` con **dos** pruebas:

1. Restaurar devuelve el estado guardado: escribe «hola», guarda, escribe « mundo», mueve el cursor al principio, restaura y comprueba el texto y el cursor.
2. Cambiar el editor después de guardar no altera la instantánea: escribe «hola», guarda, escribe « mundo» y comprueba que la instantánea guardada sigue valiendo «hola» y cursor 4.

Está bien si `./gradlew test` termina con `BUILD SUCCESSFUL` y, además, si quitas a propósito la línea `cursor += s.length();` de `type` la primera prueba falla (y la compruebas antes de volver a ponerla). Cuando termines, borra las carpetas `scratch`. Si tu `SnapshotTest` todavía no compila porque no has hecho el hito, no lo guardes aún en el proyecto.

Pista: *Usa `assertEquals(esperado, obtenido);` para el estado guardado hay dos aserciones en la primera prueba. Cada prueba crea su propio `Editor`.*

Test ▪ solución en la pág. 307

Ejercicio 8.3: La mochila que no vuelve ★★★☆☆

No es de Emberdeep. Esta mochila guarda una instantánea y, al restaurarla, no recupera lo que tenía. Guarda las tres clases en una carpeta sin paquete:

Backpack.java

```java
import java.util.ArrayList;
import java.util.List;

public class Backpack {
    private List<String> items = new ArrayList<>();

    public void add(String item) { items.add(item); }
    public List<String> items() { return items; }

    public BackpackMemento save() {
        return new BackpackMemento(items);
    }

    public void restore(BackpackMemento memento) {
        items = memento.items();
    }
}

record BackpackMemento(List<String> items) {}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Backpack backpack = new Backpack();
        backpack.add("espada");
        BackpackMemento saved = backpack.save();
        backpack.add("poción");
        backpack.restore(saved);
        System.out.println(backpack.items());
        backpack.add("bomba");
        backpack.restore(saved);
        System.out.println(backpack.items());
    }
}
```

Debería imprimir `[espada]` dos veces. Imprime `[espada, poción]` y `[espada, poción, bomba]`.

1. Ejecútalo y explica, con tus palabras, por qué `restore` no restaura nada. Hay *dos* sitios donde la mochila y el memento comparten la misma lista: encuéntralos.
2. Corrige solo lo necesario. Tras tu primer arreglo, vuelve a ejecutar: si ahora aparece una excepción, es una pista de que el arreglo está a medias.
3. Ejecuta de nuevo el `Main` y comprueba que imprime `[espada]` dos veces.

Está bien si el memento guarda una copia propia y la mochila repone con otra copia distinta, y si sabes explicar por qué hacen falta las dos.

Pista: *Piensa en quién tiene una referencia a la lista en cada momento: tras `save()`, tras `restore()` y tras la siguiente llamada a `add`. Y recuerda que `List.copyOf` devuelve una lista que no se puede modificar.*

Depuración ▪ solución en la pág. 310

Ejercicio 8.4: Hito 15a: puntos de control con Memento ★★★☆☆

Implementa el hito H15a partiendo del estado del capítulo anterior (solución del ejercicio 7.5), en tres partes. Comprueba cada una antes de pasar a la siguiente.

Parte 1 (las piezas): la instantánea y el originador.

1. Paquete `engine/save/`: `FighterState(int hp, int shield, StatSource stats)` y `Snapshot(int floor, int x, int y, int round, List<FighterState> fighters, Inventory inventory)`, ambos `record`; `Snapshot` copia la lista con `List.copyOf` en el constructor compacto. `SnapshotStore`, con un `Deque<Snapshot>`: `push(Snapshot)`, `Snapshot last()` (el último, sin quitarlo; `IllegalStateException` con «No hay puntos de control» si está vacío), `boolean isEmpty()` e `int size()`.
2. `Combat`: `void restoreRound(int round)`, que lanza `IllegalArgumentException` si `round < 1`.
3. `GameSession`: campos `floor` (1 al empezar), `x` e `y`, con `floor()`, `x()`, `y()` y `setLocation(int floor, int x, int y)`; `createSnapshot()`, con los estados en el orden héroes y luego enemigos y el inventario por referencia; y `restore(Snapshot)`, que repone cada combatiente con `Fighter.restore(hp, shield, stats)`, la ronda, el inventario y la posición. Lanza `IllegalArgumentException` («La instantánea es de otra partida») si el número de estados no coincide con el de combatientes.
4. `Main`: tras crear la sesión, parsea el mapa de la planta elegida con `new MapParser(new TileTypes()).parse(floor.map())` y llama a `session.setLocation(floorNumber, mapa.partyX(), mapa.partyY())`: es la casilla de entrada de esa planta.

Está bien si este archivo temporal, `app/Check08.java`, imprime las tres líneas siguientes (el estado tras un `Defend`, el estado tras un golpe del Zombi, el final del efecto de la defensa y una bomba al Esqueleto, y el estado restaurado):

app/Check08.java (parte 1; bórralo al terminar)

```java
package emberdeep.app;

import emberdeep.engine.character.*;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.*;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.save.Snapshot;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.biome.CryptBiome;
import java.util.List;
import java.util.Random;

public class Check08 {
    public static void main(String[] args) {
        Random random = new Random(42);
        Hero aria = new HeroBuilder().name("Aria")
                .stats(new Stats(30, 8, 3)).build();
        GameSession session = new GameSession(List.of(aria),
                new CryptBiome(random),
                List.of(EnemyKind.SKELETON, EnemyKind.ZOMBIE),
                new FlatFormula(), new DamagePipeline(random, 0.0));
        Combat combat = session.combat();
        Fighter hero = session.heroes().get(0);
        Fighter skeleton = session.enemies().get(0);
        Fighter zombie = session.enemies().get(1);
        combat.perform(new Defend(hero));
        Snapshot saved = session.createSnapshot();
        show("guardado", session);

        combat.perform(new UseSkill(combat, zombie,
                zombie.skills().get(0), hero));
        hero.tickBuffs();
        combat.perform(new UseItem(session, hero, skeleton,
                session.inventory().items().get(2)));
        show("después", session);
        session.restore(saved);
        show("restaurado", session);
    }

    private static void show(String label, GameSession session) {
        Fighter hero = session.heroes().get(0);
        System.out.println(label + ": Aria hp=" + hero.hp()
                + " escudo=" + hero.shield()
                + " defensa=" + hero.stats().defense()
                + " | Esqueleto hp=" + session.enemies().get(0).hp()
                + " | objetos=" + session.inventory().items().size());
    }
}
```

Salida esperada

```
guardado: Aria hp=30 escudo=5 defensa=5 | Esqueleto hp=18 | objetos=3
después: Aria hp=29 escudo=0 defensa=3 | Esqueleto hp=8 | objetos=2
restaurado: Aria hp=30 escudo=5 defensa=5 | Esqueleto hp=18 | objetos=3
```

Parte 2 (las teclas): `c` y `r` en el menú.

1. `ConsoleUi` tiene un `SnapshotStore` propio (el cuidador). El menú de cada héroe termina ahora con «`[c] guardar`» y «`[r] volver`», separadas de las anteriores por dos espacios, como el resto.
2. Tecla `c`: guarda `session.createSnapshot()` y escribe «Punto de control guardado (N).», con N el número de puntos guardados; el menú se vuelve a pintar.
3. Tecla `r`: sin puntos de control, escribe «No hay puntos de control.» y vuelve a pintar el menú. Con alguno, repone el *último* sin quitarlo del almacén y escribe «Vuelves al último punto de control (ronda R).», con R la ronda restaurada.
4. Tras restaurar, la fase de héroes de esa ronda debe continuar con el mismo héroe al que le tocaba al guardar, y con la lista de héroes vivos de ese momento. La instantánea no guarda ese número: apúntalo en la interfaz, junto al almacén. Hazlo con un tipo enumerado de tres valores (la acción se hizo, se deshizo o se restauró) que devuelve el turno de cada héroe.
5. `u` no debe bajar nunca del primer héroe, aunque después de un `r` la pila de comandos tenga más acciones de las que han hecho los héroes (límite de la sección 8.7).

Está bien si este guion, `scripts/check08.txt` (bórralo al terminar), jugado con `./gradlew -q run -args="scripts/check08.txt"` produce lo que se indica debajo.

`scripts/check08.txt (parte 2; bórralo al terminar)`

```
# Ronda 1: Aria se cubre
2
# Borin guarda y lanza la bomba al Esqueleto
c
3
3
1
# Cael vuelve al punto de control
r
```

Verás, en este orden: el mensaje de la defensa de Aria; en el turno de Borin, «`> c`» y «Punto de control guardado (1).»; la bomba, con «Borin lanza Bomba de ceniza sobre Esqueleto y le hace 10 de daño.»; en el turno de Cael, la línea «Enemigos:» con «Esqueleto 8/18» y «Objetos:» con solo dos pociones; luego «`> r`», «Vuelves al último punto de control (ronda 1).» y el menú de *Borin* (no el de Aria ni el de Cael), con «Esqueleto 18/18» y «Objetos:» con las tres cosas. Después, el guion se acaba y el programa termina con `IllegalStateException: El guion se ha acabado`: es lo esperado.

Parte 3 (la prueba): JUnit y `SnapshotTest`.

1. `build.gradle` como el de la sección 8.6 (repositorio, las dos dependencias y `useJUnitPlatform()`).
2. `SnapshotTest` tal como está en esa sección, en

`src/test/java/emberdeep/engine/save/`.

Está bien si `./gradlew test` termina con `BUILD SUCCESSFUL`, y si al comentar la línea que repone el inventario en `restore`, la prueba falla con `expected: <3> but was: <0>`. Vuelve a ponerla. Borra `Check08.java` y `scripts/check08.txt`.

Pista: *En `GameSession`, `allFighters()` devuelve héroes y luego enemigos con `Stream.concat`. En `ConsoleUi`, el bucle de rondas reasigna la lista de héroes vivos y el índice cuando el turno devuelve «restaurado».*

Proyecto ▪ solución en la pág. 311

Ejercicio 8.5: Comandos o instantáneas ★★★★☆

Emberdeep quiere ofrecer deshacer en tres operaciones nuevas. Para cada una decide si el deshacer va con **comandos inversos** (capítulo 7) o con **instantáneas** (este capítulo), y justifícalo:

1. **Beber una poción**: cura a un héroe y quita el objeto del inventario.
2. **Bajar a la planta siguiente**: cambia la planta, el mapa, la casilla del grupo, los enemigos y el bioma, y descarta el combate anterior.
3. **Terremoto**: una habilidad del jefe final que, de una vez, baja la vida de los seis combatientes, gasta el escudo de los que lo tengan y acaba con los efectos temporales de todos.

Para cada operación indica: qué guardarías, cuánta memoria cuesta, qué riesgo corres (qué se te puede olvidar) y qué pasaría si eligieras el otro patrón. Después, explica en tres o cuatro líneas qué harías si el jugador pudiera deshacer *cien* operaciones seguidas. No hace falta escribir código, salvo si decides un híbrido: en ese caso, esboza la clase.
Está bien si: (1) cada decisión se apoya en una razón concreta del problema (cuánto cambia, si se puede invertir, cuánta memoria cuesta) y no en «Memento es más moderno»; (2) mencionas, al menos, un riesgo real de cada opción (un inverso mal escrito; un campo olvidado en la instantánea); (3) tratas el caso de las cien operaciones (límite de instantáneas guardadas o guardar solo diferencias); (4) tu respuesta no usa el mismo patrón para las tres operaciones sin discutir si eso es sensato. Criterios de valoración: ajuste de la elección al problema, conciencia de costes y riesgos, y claridad.

Diseño ▪ solución en la pág. 331

8.10 Resumen

Resumen

- El olor de Memento es «volver a un punto» cuando cambian muchas cosas a la vez: copiar el estado a mano desde fuera obliga a conocer todos los campos y a abrirlos, y el `CommandHistory` no cruza rondas. Memento da la tarea al propio objeto.
- Piezas: el originador (`GameSession`) crea y se repone desde la instantánea; el memento (`Snapshot`, `FighterState`) es inmutable; el cuidador (`SnapshotStore`) la guarda sin mirar dentro; el cliente (`ConsoleUi`) decide cuándo.
- Una instantánea es tan inmutable como lo que guarda. Los `int` se copian; la cadena de `StatSource` y el `Inventory` se guardan por referencia porque son inmutables; una lista se copia con `List.copyOf` solo si sus elementos también lo son. Guardar los propios

`Fighter` es el error clásico.

- `restore` repone los objetos que ya existen, no los sustituye, porque otros los comparten (`Combat`).
- Command guarda lo que toca cada acción y deshace de una en una; Memento guarda el estado entero y vuelve a un punto de golpe. Se pueden combinar.
- Límites declarados: tras `restore` el `CommandHistory` queda obsoleto y el estado del `Random` no se guarda. Cada campo nuevo del estado exige actualizar la instantánea y su prueba.
- JUnit 6.1.3 necesita Java 17 o superior; en Gradle: `repositories`, las dependencias `testImplementation` y `testRuntimeOnly`, y `useJUnitPlatform()`. Una prueba es un método con `@Test` y aserciones como `assertEquals(esperado, obtenido)`; se ejecuta con `./gradlew test`.
- H15a: `engine/save/`, `GameSession.createSnapshot` y `restore`, `Combat.restoreRound`, teclas `c` y `r` y `SnapshotTest` como primera prueba del proyecto.

Soluciones de los ejercicios

Solución del ejercicio 8.1

enunciado en la pág. 297

Cuatro archivos sin paquete, en la misma carpeta.

`EditorState.java`

```
1 public record EditorState(String text, int cursor) {}
```

Editor.java

```
public class Editor {
    private String text = "";
    private int cursor;

    public void type(String s) {
        text = text.substring(0, cursor) + s + text.substring(cursor);
        cursor += s.length();
    }

    public void moveTo(int position) {
        cursor = Math.max(0, Math.min(position, text.length()));
    }

    public EditorState snapshot() {
        return new EditorState(text, cursor);
    }

    public void restore(EditorState state) {
        text = state.text();
        cursor = state.cursor();
    }

    public String text() { return text; }
    public int cursor() { return cursor; }
}
```

History.java

```
import java.util.ArrayDeque;
import java.util.Deque;

public class History {
    private final Deque<EditorState> states = new ArrayDeque<>();

    public void save(EditorState state) {
        states.push(state);
    }

    public EditorState pop() {
        return states.pop();
    }

    public boolean isEmpty() {
        return states.isEmpty();
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Editor editor = new Editor();
        History history = new History();
        editor.type("hola");
        history.save(editor.snapshot());
        editor.type(" mundo");
        history.save(editor.snapshot());
        editor.moveTo(0);
        editor.type(">> ");
        show(editor);
        editor.restore(history.pop());
        show(editor);
        editor.restore(history.pop());
        show(editor);
    }

    private static void show(Editor editor) {
        System.out.println("[" + editor.text() + "] cursor=" + editor.
    ↪ cursor());
    }
}
```

Por qué funciona. `Editor` es el originador: es el único que sabe que su estado son dos campos, y por eso crea y repone la instantánea. `EditorState` es el memento: un `record` con un `String` (inmutable) y un `int` (un valor), así que la instantánea no cambia aunque el editor siga escribiendo. `History` es el cuidador: guarda y devuelve, sin mirar dentro. Recorre la salida: tras escribir «hola» y guardar (cursor 4), escribir « mundo» y guardar (cursor 10), el editor va al principio y escribe «» », y queda `[» hola mundo]` con el cursor en 3. El primer `pop()` devuelve la segunda instantánea (la última en entrar) y el editor vuelve a `[hola mundo]` con el cursor en 10. El segundo devuelve la primera: `[hola]` y cursor 4.

Errores típicos:

- Guardar solo el texto y olvidar el cursor: el texto vuelve, pero el cursor queda donde estaba (en el ejemplo, en 3 y no en 10).
- Usar un `StringBuilder` en el estado: el memento y el editor compartirían el mismo objeto mutable, y la instantánea cambiaría al escribir (sección 8.4).
- Esperar la instantánea más antigua con `pop()`: una pila devuelve primero la última que entró.

Solución del ejercicio 8.2

enunciado en la pág. 298

Tres archivos: los dos que se prueban, en `src/main/java/scratch/`, y la prueba, en `src/test/java/scratch/`.

src/main/java/scratch/EditorState.java

```java
package scratch;

public record EditorState(String text, int cursor) {}
```

src/main/java/scratch/Editor.java

```
package scratch;

public class Editor {
    private String text = "";
    private int cursor;

    public void type(String s) {
        text = text.substring(0, cursor) + s + text.substring(cursor);
        cursor += s.length();
    }

    public void moveTo(int position) {
        cursor = Math.max(0, Math.min(position, text.length()));
    }

    public EditorState snapshot() { return new EditorState(text, cursor); }

    public void restore(EditorState state) {
        text = state.text();
        cursor = state.cursor();
    }

    public String text() { return text; }
    public int cursor() { return cursor; }
}
```

src/test/java/scratch/EditorTest.java

```
package scratch;

import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.Test;

class EditorTest {

    @Test
    void restoreBringsBackTextAndCursor() {
        Editor editor = new Editor();
        editor.type("hola");
        EditorState saved = editor.snapshot();
        editor.type(" mundo");
        editor.moveTo(0);

        editor.restore(saved);

        assertEquals("hola", editor.text());
        assertEquals(4, editor.cursor());
    }

    @Test
    void changingTheEditorDoesNotChangeTheSnapshot() {
        Editor editor = new Editor();
        editor.type("hola");
        EditorState saved = editor.snapshot();

        editor.type(" mundo");

        assertEquals("hola", saved.text());
        assertEquals(4, saved.cursor());
    }
}
```

Por qué funciona. Cada prueba crea su propio `Editor`, así que ninguna depende de la otra. La primera sigue los tres pasos (preparar, hacer, comprobar): estropea el editor después de guardar y verifica que `restore` devuelve el texto y el cursor. La segunda comprueba la propiedad que hace seguro al memento: la instantánea no cambia cuando cambia el original. Si quitas `cursor += s.length();` de `type`, tras escribir «hola» el cursor valdría 0, y la primera prueba falla con `expected: <4> but was: <0>` (lo esperado va primero).

Errores típicos:

- Poner `EditorTest` en `src/main/java`: Gradle lo compila como código del juego, donde no existe JUnit, y falla.
- Compartir un `Editor` entre las dos pruebas como atributo de la clase: cada prueba debería empezar con un editor limpio.
- Invertir los argumentos de `assertEquals`: la prueba pasa o falla igual, pero el mensaje de error confunde lo esperado con lo obtenido.
- Olvidar el `@Test`: el método existe pero JUnit no lo ejecuta.

Al terminar, borra `src/main/java/scratch/` y `src/test/java/scratch/`.

Solución del ejercicio 8.3 enunciado en la pág. 299

La mochila corregida, con el mismo `Main`:

Backpack.java

```java
import java.util.ArrayList;
import java.util.List;

public class Backpack {
    private List<String> items = new ArrayList<>();

    public void add(String item) { items.add(item); }
    public List<String> items() { return items; }

    public BackpackMemento save() {
        return new BackpackMemento(items);
    }

    public void restore(BackpackMemento memento) {
        items = new ArrayList<>(memento.items());
    }
}

record BackpackMemento(List<String> items) {
    BackpackMemento {
        items = List.copyOf(items);
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Backpack backpack = new Backpack();
        backpack.add("espada");
        BackpackMemento saved = backpack.save();
        backpack.add("poción");
        backpack.restore(saved);
        System.out.println(backpack.items());
        backpack.add("bomba");
        backpack.restore(saved);
        System.out.println(backpack.items());
    }
}
```

La salida es ahora `[espada]` y `[espada]`.

Por qué fallaba. Había dos sitios donde mochila y memento compartían la misma lista. El primero, en `save()`: `new BackpackMemento(items)` guarda la referencia a la lista de la mochila, no una copia, así que al añadir la poción el «memento» también la tiene (es el caso de `BadSnapshot` de la sección 8.4). El segundo, en `restore()`: `items = memento.items()` hace que la mochila use *la lista del memento*; a partir de ahí, cada `add` cambia también lo guardado, y una segunda restauración devuelve la bomba que ya estaba. Se arregla en los dos sitios: el memento copia la lista al crearse (`List.copyOf` en el constructor compacto) y la mochila repone con una copia suya (`new ArrayList<>(...)`).

Qué pasa si arreglas solo uno. Si copias en el memento pero dejas `items = memento.items()`, la primera restauración funciona, pero la mochila pasa a usar una lista inmutable (`List.copyOf` las devuelve así) y el `backpack.add("bomba")` lanza `UnsupportedOperationException`: ese es el síntoma de «arreglo a medias» que indicaba la pista. Si copias solo en `restore`, el memento sigue compartiendo la lista en `save()` y la primera impresión ya sale mal.

Errores típicos:

- Copiar con `new ArrayList<>(items)` la lista de *objetos mutables* y creer que ya es una copia profunda: aquí los elementos son `String` (inmutables) y basta. Con objetos mutables habría que copiar también cada elemento (capítulo 5).
- Corregir el síntoma (imprimir una copia) y no la causa (el memento comparte lista).

Solución del ejercicio 8.4

enunciado en la pág. 301

Partes 1 a 3 en orden. Se repiten los archivos enteros (también los que solo cambian un poco), para que la solución esté completa.

Parte 1: engine/save/

engine/save/FighterState.java

```
package emberdeep.engine.save;

import emberdeep.engine.stats.StatSource;

public record FighterState(int hp, int shield, StatSource stats) {}
```

engine/save/Snapshot.java

```
package emberdeep.engine.save;

import emberdeep.engine.item.Inventory;
import java.util.List;

public record Snapshot(int floor, int x, int y, int round,
                       List<FighterState> fighters, Inventory inventory) {
    public Snapshot {
        fighters = List.copyOf(fighters);
    }
}
```

engine/save/SnapshotStore.java

```
package emberdeep.engine.save;

import java.util.ArrayDeque;
import java.util.Deque;

public class SnapshotStore {
    private final Deque<Snapshot> checkpoints = new ArrayDeque<>();

    public void push(Snapshot snapshot) {
        checkpoints.push(snapshot);
    }

    public Snapshot last() {
        if (checkpoints.isEmpty()) {
            throw new IllegalStateException("No hay puntos de control");
        }
        return checkpoints.peek();
    }

    public boolean isEmpty() { return checkpoints.isEmpty(); }
    public int size() { return checkpoints.size(); }
}
```

engine/combat/Combat.java (solo gana `restoreRound`; el resto, sin cambios):

Combat.java (1/2: estado, ronda y daño)

```
package emberdeep.engine.combat;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.command.CommandHistory;
import emberdeep.engine.combat.damage.Damage;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import java.util.ArrayList;
import java.util.List;

public class Combat {
    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final DamageFormula formula;
    private final DamagePipeline pipeline;
    private final CommandHistory history = new CommandHistory();
    private int round = 1;

    public Combat(List<Fighter> heroes, List<Fighter> enemies,
                  DamageFormula formula, DamagePipeline pipeline) {
        this.heroes = heroes;
        this.enemies = enemies;
        this.formula = formula;
        this.pipeline = pipeline;
    }

    public int round() {
        return round;
    }

    public boolean heroesWon() {
        return enemies.stream().noneMatch(Fighter::isAlive);
    }

    public boolean heroesLost() {
        return heroes.stream().noneMatch(Fighter::isAlive);
    }

    public boolean isOver() {
        return heroesWon() || heroesLost();
    }

    // cap. 8: la usa GameSession.restore
    public void restoreRound(int round) {
        if (round < 1) {
            throw new IllegalArgumentException("La ronda empieza en 1");
        }
        this.round = round;
    }
```

Combat.java (2/2: ejecutar, deshacer y enemigos)

```
1
2       // los héroes ejecutan sus comandos por aquí
3       public List<String> perform(CombatCommand command) {
4           return history.run(command);
5       }
6
7       public boolean undo() {
8           return history.undo();
9       }
10
11      public int damageOf(Fighter attacker, Fighter target) {
12          return formula.damage(attacker.stats().attack(),
13                                target.stats().defense());
14      }
15
16      public int deal(Fighter target, Damage damage) {
17          int dealt = pipeline.apply(target, damage);
18          target.takeDamage(dealt);
19          return dealt;
20      }
21
22      public List<String> enemyPhase() {
23          List<String> messages = new ArrayList<>();
24          for (Fighter enemy : enemies) {
25              if (!enemy.isAlive()) {
26                  continue;
27              }
28              if (heroesLost()) {
29                  break;
30              }
31              messages.addAll(enemy.act(this, heroes).execute());
32          }
33          round++;
34          heroes.forEach(Fighter::tickBuffs);
35          enemies.forEach(Fighter::tickBuffs);
36          history.clear();
37          return messages;
38      }
39  }
```

engine/session/GameSession.java, en cuatro tramos seguidos dentro de la misma clase:

GameSession.java (1/4: imports y campos)

```
package emberdeep.engine.session;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.save.FighterState;
import emberdeep.engine.save.Snapshot;
import emberdeep.engine.stats.ArmorBonus;
import emberdeep.engine.stats.BaseStats;
import emberdeep.engine.stats.StatSource;
import emberdeep.engine.stats.WeaponBonus;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.List;
import java.util.stream.Stream;

public class GameSession {
    private static final int SWORD_ATTACK = 2;
    private static final int ARMOR_VALUE = 1;

    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final Combat combat;
    private Inventory inventory;
    private int floor = 1;
    private int x;
    private int y;
```

GameSession.java (2/4: constructor y equipo)

```

    public GameSession(List<Hero> heroes, BiomeFactory biome,
                       List<EnemyKind> kinds, DamageFormula formula,
                       DamagePipeline pipeline) {
        this.heroes = heroes.stream()
                .map(hero -> Fighter.ofHero(hero, gearOf(hero)))
                .toList();
        this.enemies = kinds.stream()
                .map(biome::enemy)
                .map(Fighter::ofEnemy)
                .toList();
        this.combat = new Combat(this.heroes, this.enemies, formula,
    ↪ pipeline);
        this.inventory = new Inventory(List.of())
                .with(biome.item(ItemKind.HEALING))
                .with(biome.item(ItemKind.HEALING))
                .with(biome.item(ItemKind.BOMB));
    }

    private static StatSource gearOf(Hero hero) {
        return new ArmorBonus(
                new WeaponBonus(new BaseStats(hero.stats()), SWORD_ATTACK),
                ARMOR_VALUE);
    }
```

GameSession.java (3/4: accesos y posición)

```

    public List<Fighter> heroes() { return heroes; }
    public List<Fighter> enemies() { return enemies; }
    public Combat combat() { return combat; }
    public int round() { return combat.round(); }
    public Inventory inventory() { return inventory; }
    public int floor() { return floor; }
    public int x() { return x; }
    public int y() { return y; }

    public void replaceInventory(Inventory next) {
        this.inventory = next;
    }

    public void setLocation(int floor, int x, int y) {
        this.floor = floor;
        this.x = x;
        this.y = y;
    }
```

GameSession.java (4/4: instantánea y restauración)

```
    private List<Fighter> allFighters() {
        return Stream.concat(heroes.stream(), enemies.stream()).toList();
    }

    public Snapshot createSnapshot() {
        List<FighterState> states = allFighters().stream()
                .map(f -> new FighterState(f.hp(), f.shield(), f.stats()))
                .toList();
        return new Snapshot(floor, x, y, combat.round(), states, inventory);
    }

    public void restore(Snapshot snapshot) {
        List<Fighter> all = allFighters();
        if (snapshot.fighters().size() != all.size()) {
            throw new IllegalArgumentException(
                    "La instantánea es de otra partida");
        }
        for (int i = 0; i < all.size(); i++) {
            FighterState state = snapshot.fighters().get(i);
            all.get(i).restore(state.hp(), state.shield(), state.stats());
        }
        combat.restoreRound(snapshot.round());
        inventory = snapshot.inventory();
        floor = snapshot.floor();
        x = snapshot.x();
        y = snapshot.y();
    }
}
```

app/Main.java, en cuatro tramos seguidos dentro de la misma clase (solo cambian un `import` y las líneas que dan a la sesión su casilla de entrada):

Main.java (1/4: imports)

```
package emberdeep.app;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.Dungeon;
import emberdeep.engine.world.DungeonBuilder;
import emberdeep.engine.world.Floor;
import emberdeep.engine.world.biome.BiomeFactory;
import emberdeep.engine.world.biome.CavernBiome;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.MapData;
import emberdeep.engine.world.map.MapParser;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.ConsoleUi;
import emberdeep.ui.input.ConsoleInput;
import emberdeep.ui.input.InputSource;
import emberdeep.ui.input.ScriptInput;
import java.io.IOException;
import java.nio.file.Path;
import java.util.Arrays;
import java.util.List;
import java.util.Random;
import java.util.Scanner;
```

Main.java (2/4: arranque y entrada)

```
1
2  public class Main {
3      private static final Skill POWER_STRIKE =
4              new Skill("Golpe poderoso", 12, DamageType.PHYSICAL);
5      private static final Skill FLARE =
6              new Skill("Llamarada", 10, DamageType.FIRE);
7
8      private final GameConfig config;
9
10     private Main(GameConfig config) {
11         this.config = config;
12     }
13
14     public static void main(String[] args) throws IOException {
15         GameConfig config = new GameConfig(42L, 3, new FlatFormula());
16         new Main(config).play(args);
17     }
18
19     private void play(String[] args) throws IOException {
20         String script = scriptOf(args);
21         String[] options = optionsOf(args);
22         int floorNumber = chooseFloor(options);
23         InputSource input = script == null
24                 ? new ConsoleInput(new Scanner(System.in), System.out)
25                 : new ScriptInput(Path.of(script), System.out);
26         ConsoleUi ui = new ConsoleUi(input, System.out);
27         Random random = new Random(config.seed());
28
29         BiomeFactory crypt = new CryptBiome(random);
30         BiomeFactory cavern = new CavernBiome(random);
31         List<EnemyKind> cryptKinds = List.of(
32                 EnemyKind.SKELETON, EnemyKind.ASH_RAT, EnemyKind.ZOMBIE);
33         List<EnemyKind> cavernKinds = List.of(
34                 EnemyKind.GOBLIN, EnemyKind.BAT, EnemyKind.SPIDER);
```

Main.java (3/4: mazmorra, partida y modo mapa)

```
1
2           Dungeon dungeon = new DungeonBuilder()
3                   .floor().biome(crypt).enemies(cryptKinds)
4                           .map(MapData.floor(1)).done()
5                   .floor().biome(cavern).enemies(cavernKinds)
6                           .map(MapData.floor(2)).done()
7                   .floor().biome(crypt).enemies(cryptKinds)
8                           .map(MapData.floor(3)).done()
9                   .build();
10
11          if (options.length > 1 && options[1].equals("map")) {
12              explore(ui, dungeon, floorNumber);
13              return;
14          }
15          DamageFormula formula = chooseFormula(ui, options, script != null);
16          Floor floor = dungeon.floors().get(floorNumber - 1);
17          GameSession session = new GameSession(heroes(), floor.biome(),
18                  floor.enemies(), formula, new DamagePipeline(random, 0.0));
19          GameMap entrance = new MapParser(new TileTypes())
20                  .parse(floor.map());
21          session.setLocation(floorNumber,
22                  entrance.partyX(), entrance.partyY());
23          ui.run(session);
24      }
25
26      private void explore(ConsoleUi ui, Dungeon dungeon, int firstFloor) {
27          TileTypes types = new TileTypes();
28          MapParser parser = new MapParser(types);
29          int number = firstFloor;
30          while (true) {
31              Floor floor = dungeon.floors().get(number - 1);
32              boolean descended =
33                      ui.explore(parser.parse(floor.map()), types);
34              if (!descended || number == dungeon.floors().size()) {
35                  return;
36              }
37              number++;
38          }
39      }
```

Main.java (4/4: argumentos, héroes y llave final)

```
    // El guion es el argumento que acaba en .txt (parte 3)
    private static String scriptOf(String[] args) {
        return Arrays.stream(args)
                .filter(arg -> arg.endsWith(".txt"))
                .findFirst().orElse(null);
    }

    private static String[] optionsOf(String[] args) {
        return Arrays.stream(args)
                .filter(arg -> !arg.endsWith(".txt"))
                .toArray(String[]::new);
    }

    private DamageFormula chooseFormula(ConsoleUi ui, String[] options,
                                        boolean scripted) {
        boolean useDefault = scripted
                || (options.length > 1 && options[1].equals("default"));
        if (useDefault) {
            return config.defaultFormula();
        }
        return ui.askFormula() == 1
                ? new FlatFormula() : new ScaledFormula();
    }

    private int chooseFloor(String[] options) {
        int floor = options.length > 0 ? Integer.parseInt(options[0]) : 1;
        if (floor < 1 || floor > config.floors()) {
            throw new IllegalArgumentException(
                    "La planta debe estar entre 1 y " + config.floors());
        }
        return floor;
    }

    private List<Hero> heroes() {
        return List.of(
                new HeroBuilder().name("Aria").stats(new Stats(30, 8, 3))
                        .skill(POWER_STRIKE).skill(FLARE).build(),
                new HeroBuilder().name("Borin").stats(new Stats(40, 6, 5))
                        .skill(POWER_STRIKE).skill(FLARE).build(),
                new HeroBuilder().name("Cael").stats(new Stats(24, 9, 2))
                        .skill(POWER_STRIKE).skill(FLARE).build());
    }
}
```

Parte 2: ui/ConsoleUi.java, en siete tramos seguidos dentro de la misma clase:

ConsoleUi.java (1/7: imports, campos y fórmula)

```
package emberdeep.ui;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Skill;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.Attack;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.command.Defend;
import emberdeep.engine.combat.command.UseItem;
import emberdeep.engine.combat.command.UseSkill;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.save.SnapshotStore;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.TileType;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.input.InputSource;
import java.io.PrintStream;
import java.util.List;
import java.util.stream.Collectors;

public class ConsoleUi {
    private enum Turn { ACTED, UNDONE, RESTORED }

    private final InputSource input;
    private final PrintStream out;
    private final SnapshotStore checkpoints = new SnapshotStore();
    private int checkpointHero;     // a qué héroe le tocaba al guardar

    public ConsoleUi(InputSource input, PrintStream out) {
        this.input = input;
        this.out = out;
    }

    public int askFormula() {
        out.println("Fórmula de daño:");
        out.println("  [1] plana: ataque - defensa");
        out.println("  [2] escalada: ataque x 100 / (100 + defensa)");
        return ask(2);
    }
```

ConsoleUi.java (2/7: la partida)

```
    public void run(GameSession session) {
        Combat combat = session.combat();
        out.println("=== Emberdeep ===");
        while (!combat.isOver()) {
            out.println("--- Ronda " + session.round() + " ---");
            List<Fighter> acting = aliveHeroes(session);
            int i = 0;
            while (i < acting.size() && !combat.isOver()) {
                Turn turn = heroTurn(session, acting.get(i), i);
                switch (turn) {
                    case ACTED -> i++;
                    case UNDONE -> i = Math.max(0, i - 1);
                    case RESTORED -> {
                        acting = aliveHeroes(session);
                        i = checkpointHero;
                    }
                }
            }
            if (!combat.isOver()) {
                print(combat.enemyPhase());
            }
        }
        out.println(combat.heroesWon() ? "¡Victoria!" : "Derrota...");
    }

    private List<Fighter> aliveHeroes(GameSession session) {
        return session.heroes().stream().filter(Fighter::isAlive).toList();
    }
```

ConsoleUi.java (3/7: el turno de un héroe)

```
    // ACTED: el héroe actúa; UNDONE: se deshace la acción anterior;
    // RESTORED: se vuelve a un punto de control
    private Turn heroTurn(GameSession session, Fighter hero, int index) {
        Combat combat = session.combat();
        while (true) {
            out.println("Héroes:   " + describe(session.heroes()));
            out.println("Enemigos: " + describe(session.enemies()));
            out.println("Objetos:  " + describeItems(session.inventory()));
            out.println("Turno de " + hero.name()
                    + ". [1] atacar  [2] defender  [3] objeto"
                    + "  [4] habilidad  [u] deshacer"
                    + "  [c] guardar  [r] volver");
            String key = input.next();
            if (key.equals("u")) {
                if (combat.undo()) {
                    out.println("Deshaces la última acción.");
                    return Turn.UNDONE;
                }
                out.println("No hay nada que deshacer en esta ronda.");
                continue;
            }
            if (key.equals("c")) {
                saveCheckpoint(session, index);
                continue;
            }
            if (key.equals("r")) {
                if (loadCheckpoint(session)) {
                    return Turn.RESTORED;
                }
                continue;
            }
            CombatCommand command = switch (key) {
                case "1" -> new Attack(combat, hero,
                        askTarget("Objetivo", session.enemies()));
                case "2" -> new Defend(hero);
                case "3" -> chooseItem(session, hero);
                case "4" -> chooseSkill(session, hero);
                default -> {
                    out.println("Tecla no válida.");
                    yield null;
                }
            };
            if (command != null) {
                print(combat.perform(command));
                return Turn.ACTED;
            }
        }
    }
```

ConsoleUi.java (4/7: guardar y volver)

```
    private void saveCheckpoint(GameSession session, int index) {
        checkpoints.push(session.createSnapshot());
        checkpointHero = index;
        out.println("Punto de control guardado ("
                + checkpoints.size() + ").");
    }

    // false si todavía no hay ninguno
    private boolean loadCheckpoint(GameSession session) {
        if (checkpoints.isEmpty()) {
            out.println("No hay puntos de control.");
            return false;
        }
        session.restore(checkpoints.last());
        out.println("Vuelves al último punto de control (ronda "
                + session.round() + ").");
        return true;
    }
```

ConsoleUi.java (5/7: objetos, habilidades y objetivos)

```
    private CombatCommand chooseItem(GameSession session, Fighter hero) {
        List<Item> items = session.inventory().items();
        if (items.isEmpty()) {
            out.println("No te quedan objetos.");
            return null;
        }
        Item item = items.get(choose("Objeto",
                items.stream().map(Item::name).toList()));
        Fighter target = item.kind() == ItemKind.HEALING
                ? askTarget("Aliado", session.heroes())
                : askTarget("Objetivo", session.enemies());
        return new UseItem(session, hero, target, item);
    }

    private CombatCommand chooseSkill(GameSession session, Fighter hero) {
        List<Skill> skills = hero.skills();
        if (skills.isEmpty()) {
            out.println("No tienes habilidades.");
            return null;
        }
        Skill skill = skills.get(choose("Habilidad",
                skills.stream().map(Skill::name).toList()));
        Fighter target = askTarget("Objetivo", session.enemies());
        return new UseSkill(session.combat(), hero, skill, target);
    }

    private Fighter askTarget(String title, List<Fighter> group) {
        List<Fighter> alive = group.stream()
                .filter(Fighter::isAlive).toList();
        return alive.get(choose(title,
                alive.stream().map(Fighter::name).toList()));
    }
```

ConsoleUi.java (6/7: menús, entrada y auxiliares)

```
1
2       // Muestra "Título:  [1] a  [2] b" y devuelve la posición (desde 0)
3       private int choose(String title, List<String> labels) {
4           StringBuilder menu = new StringBuilder(title + ":");
5           for (int i = 0; i < labels.size(); i++) {
6               menu.append("  [").append(i + 1).append("] ")
7                       .append(labels.get(i));
8           }
9           out.println(menu);
10          return ask(labels.size()) - 1;
11      }
12
13      private int ask(int max) {
14          while (true) {
15              String line = input.next();
16              try {
17                  int n = Integer.parseInt(line);
18                  if (n >= 1 && n <= max) {
19                      return n;
20                  }
21              } catch (NumberFormatException e) {
22                  // no es un número: se avisa más abajo
23              }
24              out.println("Escribe un número entre 1 y " + max + ".");
25          }
26      }
27
28      private String describe(List<Fighter> group) {
29          return group.stream()
30                  .map(f -> f.name() + " " + f.hp() + "/" + f.stats().maxHp())
    ↪
31                  .collect(Collectors.joining(", "));
32      }
33
34      private String describeItems(Inventory inventory) {
35          if (inventory.items().isEmpty()) {
36              return "(ninguno)";
37          }
38          return inventory.items().stream()
39                  .map(Item::name)
40                  .collect(Collectors.joining(", "));
41      }
42
43      private void print(List<String> messages) {
44          messages.forEach(out::println);
45      }
```

ConsoleUi.java (7/7: modo mapa y llave final)

```

    public boolean explore(GameMap map, TileTypes types) {
        while (true) {
            out.print(map.render());
            out.println("[w a s d] mover, [t] casillas, [q] salir");
            String key = input.next();
            switch (key) {
                case "w" -> step(map, 0, -1);
                case "a" -> step(map, -1, 0);
                case "s" -> step(map, 0, 1);
                case "d" -> step(map, 1, 0);
                case "t" -> showTiles(map, types);
                case "q" -> { return false; }
                default -> out.println("Tecla no válida.");
            }
            if (map.onStairs()) {
                out.println("Bajas por la escalera.");
                return true;
            }
        }
    }

    private void step(GameMap map, int dx, int dy) {
        if (!map.move(dx, dy)) {
            out.println("No puedes pasar.");
        }
    }

    private void showTiles(GameMap map, TileTypes types) {
        TileType corner = map.tileAt(0, 0);
        TileType opposite = map.tileAt(map.width() - 1, map.height() - 1);
        out.println("Casillas: " + map.width() * map.height()
                + ", TileType creados: " + types.createdCount());
        out.println("Esquinas opuestas, mismo objeto: "
                + (corner == opposite));
    }
}
```

Parte 3: build.gradle y la prueba

build.gradle

```
plugins {
    id 'application'
}

repositories {
    mavenCentral()
}

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

application {
    mainClass = 'emberdeep.app.Main'
}

run {
    standardInput = System.in
}

dependencies {
    testImplementation 'org.junit.jupiter:junit-jupiter:6.1.3'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

test {
    useJUnitPlatform()
}
```

src/test/java/emberdeep/engine/save/SnapshotTest.java

```
package emberdeep.engine.save;

import static org.junit.jupiter.api.Assertions.assertEquals;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.biome.CryptBiome;
import java.util.List;
import java.util.Random;
import org.junit.jupiter.api.Test;

class SnapshotTest {

    private static GameSession newSession() {
        Random random = new Random(42);
        List<Hero> heroes = List.of(
                new HeroBuilder().name("Aria")
                        .stats(new Stats(30, 8, 3)).build());
        return new GameSession(heroes, new CryptBiome(random),
                List.of(EnemyKind.SKELETON),
                new FlatFormula(), new DamagePipeline(random, 0.0));
    }

    @Test
    void restoreBringsBackSavedState() {
        GameSession session = newSession();
        Fighter aria = session.heroes().get(0);
        aria.addShield(5);
        Snapshot saved = session.createSnapshot();

        aria.takeDamage(10);
        aria.absorbShield(5);
        session.replaceInventory(new Inventory(List.of()));
        session.restore(saved);

        assertEquals(30, aria.hp());
        assertEquals(5, aria.shield());
        assertEquals(3, session.inventory().items().size());
    }
}
```

Por qué funciona (parte 1). Los números de `Check08` salen así. Tras el `Defend`, Aria tiene 5 de escudo y la defensa sube a $3 + 2 = 5$ por el `TempBuff`; el Esqueleto tiene sus 18 y el inventario sus 3 objetos. La habilidad del Zombi («Abrazo pútrido», 7) no pasa por la fórmula, va directa a la tubería: el escudo absorbe 5 y quedan 2, la armadura de Aria (1) los deja en 1, y Aria baja a 29 con el escudo gastado. `tickBuffs()` apaga el `TempBuff` de 1

turno y la defensa vuelve a 3. La bomba (10 de fuego) golpea al Esqueleto, que no tiene armadura ni resiste el fuego, y lo deja en 8; el inventario baja a 2. Con `restore(saved)` vuelve todo: la vida y el escudo son enteros copiados; la cadena guardada es la que tenía el `TempBuff` (defensa 5), porque los decoradores son inmutables; el Esqueleto sube a 18; y el inventario es la referencia al `Inventory` de tres objetos, que nadie pudo modificar. Y como `restore` repone los `Fighter` que ya existen, `hero` sigue siendo el mismo objeto.

Por qué funciona (parte 2). `heroTurn` devuelve ahora un valor de un enumerado: `ACTED` avanza al siguiente héroe, `UNDONE` retrocede (sin bajar de 0, por el límite de la sección 8.7) y `RESTORED` recalcula los héroes vivos y salta al índice anotado al guardar. En el guion, Borin pulsa `c` cuando es el segundo héroe (índice 1) y `checkpointHero` vale 1. Cael ya ve al Esqueleto en 8 y dos pociones; al pulsar `r`, la sesión se repone, `run` pone `i = 1` y el menú que aparece es el de Borin, con el Esqueleto en 18 y las tres cosas en el inventario. Aria no vuelve a actuar, porque su acción (la defensa) estaba ya incluida en el punto de control. Después `ScriptInput` se queda sin teclas y lanza `IllegalStateException`, como en el capítulo 7.

Por qué funciona (parte 3). `testImplementation` añade JUnit solo al código de prueba y `useJUnitPlatform()` hace que Gradle lo ejecute. La prueba guarda con 30 de vida, 5 de escudo y 3 objetos, lo estropea todo y comprueba que `restore` lo devuelve. Si comentas `inventory = snapshot.inventory();`, el inventario queda vacío y la última aserción falla con `expected: <3> but was: <0>`.

Errores típicos:

- Guardar los propios `Fighter` en la instantánea: parece funcionar hasta que algo cambia (sección 8.4).
- Reponer creando `Fighter` nuevos: `Combat` sigue con los viejos y la restauración no se ve.
- Crear los estados en un orden y reponerlos en otro: el escudo de un héroe acaba en un enemigo. Por eso `allFighters()` se usa en las dos direcciones.
- Usar `pop()` en `last()`: el punto de control se gasta al volver y la segunda `r` falla.
- No recalcular la lista de héroes vivos ni el índice tras `r`: Aria volvería a actuar, o el índice quedaría fuera de la lista.
- Dejar que `u` baje de 0 después de un `r`: la pila de comandos puede tener más acciones que héroes han actuado, y `acting.get(-1)` lanza una excepción.
- Declarar JUnit con `implementation`: el código del juego pasaría a depender de la librería de pruebas. O olvidar `useJUnitPlatform()`: la prueba no se ejecuta.

Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo 9: `engine/save/` con `Snapshot`, `FighterState` y `SnapshotStore`; `GameSession` con `createSnapshot` y `restore`; `Combat` con `restoreRound`; teclas `c` y `r`; `build.gradle` con JUnit 6.1.3 y `SnapshotTest`. Borra `app/Check08.java` y `scripts/check08.txt`.

Solución del ejercicio 8.5 enunciado en la pág. 304

Solución de referencia.

Operación	Elección	Qué guarda y cuánto cuesta	Riesgo	Con el otro patrón
Beber una poción	Command	Dos o tres datos (vida, inventario anterior): `UseItem` ya lo hace. Memoria mínima.	Mutar el estado fuera del comando.	Una instantánea completa para curar a uno: mucho más de lo necesario.
Bajar de planta	Memento	Planta, casilla, enemigos, inventario: una instantánea, con referencias a lo inmutable.	Olvidar un campo en la instantánea.	Un inverso que regenere los enemigos y el combate anterior sería frágil y largo.
Terremoto	Memento (o híbrido)	Seis estados de combatiente: unos cientos de bytes.	Campo olvidado (por ejemplo, un estado nuevo en el capítulo 12).	Los valores de antes de seis combatientes, apuntados a mano en un comando.

Razonamiento. La poción es una acción pequeña, invertible y que ya existe como comando. Bajar de planta cambia casi todo lo que hay en pantalla, y lo que habría que escribir para invertirlo es del tamaño de hacerlo: es el caso del patrón Memento. El terremoto toca a muchos combatientes a la vez, y la instantánea evita escribir un `undo()` con seis veces lo mismo; el coste es memoria, pero el estado es pequeño y casi todo se guarda por referencia.
Las cien operaciones. Con comandos, la pila crece poco por acción y puede crecer mucho. Con instantáneas, guardar cien copias del estado entero sale más caro; se limita el historial (por ejemplo, las últimas veinte, descartando la más antigua con un `Deque`) o se guardan solo las diferencias respecto de la instantánea anterior, que es en el fondo un comando. Lo razonable es mezclar: comandos para lo corriente, instantánea para lo masivo.
Alternativa válida: el híbrido. Un comando que usa una instantánea para su `undo()`. Se integra con el `u` existente y no obliga a escribir los seis conjuntos de valores. Código completo:

engine/combat/command/Earthquake.java

```
package emberdeep.engine.combat.command;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.save.Snapshot;
import emberdeep.engine.session.GameSession;
import java.util.ArrayList;
import java.util.List;

public class Earthquake implements CombatCommand {
    private static final int DAMAGE = 3;

    private final GameSession session;
    private Snapshot before;

    public Earthquake(GameSession session) {
        this.session = session;
    }

    @Override
    public List<String> execute() {
        before = session.createSnapshot();
        List<Fighter> all = new ArrayList<>(session.heroes());
        all.addAll(session.enemies());
        for (Fighter fighter : all) {
            fighter.takeDamage(DAMAGE);
            fighter.restore(fighter.hp(), 0, fighter.gear());
        }
        return List.of("La tierra tiembla: todos pierden "
                + DAMAGE + " de vida.");
    }

    @Override
    public void undo() {
        session.restore(before);
    }
}
```

`execute()` guarda la instantánea antes de tocar nada, baja 3 de vida a cada combatiente y le quita el escudo y los efectos (deja `gear()`, que es la cadena sin efectos temporales); `undo()` repone la instantánea. Solo es correcto si el terremoto es lo último que se hizo (la disciplina de pila de Command): `restore` también repone el inventario y la ronda, y no serían los de antes si hubiera acciones posteriores.

Criterios para valorar otras soluciones. Una respuesta es buena si elige por razones del problema (cuánto cambia, si se invierte, qué cuesta), da un riesgo real de cada opción, trata las cien operaciones con un límite o con diferencias, y no usa el mismo patrón para las tres sin discutirlo. Es válido elegir Memento para las tres si se justifica (código único, coste de memoria despreciable) o Command para el terremoto si se acepta escribir y mantener los seis inversos. No es válido decir que Memento sirve «siempre» ni que Command no puede deshacer cambios grandes.

Parte IV

Comunicar

CAPÍTULO 9

Observer: un motor que emite eventos

Objetivos del capítulo

- Reconocer el olor que pide un Observer: el motor conoce la consola y cada función nueva (barras de vida, logros) obliga a tocarlo.
- Aplicar Observer con un bus de eventos: oyentes (una interfaz funcional), un sujeto que los guarda y los avisa, y eventos que son datos inmutables.
- Modelar los eventos con una interfaz `sealed` y `record`, y tratarlos con un `switch` exhaustivo sin `default`.
- Explicar por qué darse de baja mientras se notifica lanza `ConcurrentModificationException` y cómo evitarlo.
- Decidir cuándo compensa Observer, qué riesgos trae y por qué Mediator (capítulo 11) no es lo mismo.
- Dar el hito H10: un motor que publica eventos en lugar de devolver cadenas, un log y un HUD que los escuchan y tres logros.

Hasta ahora el motor de Emberdeep *habla*: cada comando devuelve una lista de frases («Aria ataca a Esqueleto y le hace 8 de daño.») y la interfaz las imprime. Funciona mientras solo haya una forma de enterarse de lo que pasa. Pero un juego quiere más: una barra de vida que se redibuje, un mensaje cuando consigues un logro, más adelante un sonido o un registro de la partida. Cada una de esas cosas necesita saber que ha ocurrido un golpe, y ninguna debería obligarte a abrir el motor para añadirla. El patrón que lo resuelve es Observer: el motor *anuncia* lo que ocurre y los interesados *escuchan*, sin que el motor sepa quiénes son ni cuántos. En este capítulo construyes ese anuncio (un bus de eventos), lo usas para sustituir los mensajes de texto y añades un sistema de logros que nace casi gratis.

9.1 El olor que pide Observer: el motor sabe demasiado de la consola

Mira cómo termina un golpe hoy. `Attack.execute()` calcula el daño, lo aplica y *redacta la frase* que se mostrará. El motor decide el texto, y la interfaz lo recibe ya hecho. Ahora te piden tres cosas más:

1. Una barra de vida del golpeado: `Esqueleto [#####.....] 8/18`.
2. Un logro, «primera sangre», al primer golpe que recibe un enemigo.
3. Otro logro, «intocable», si cinco golpes enemigos seguidos no hacen daño a los héroes.

La salida más directa es dar al motor referencias a quien lo necesita. Sería algo así (el código es hipotético; no existe en el proyecto):

Hipotético: el motor avisa a cada novedad

```
public Combat(List<Fighter> heroes, List<Fighter> enemies,
              DamageFormula formula, DamagePipeline pipeline,
              ConsoleUi ui, HudRenderer hud, Achievements achievements) {
    // ...
}

// dentro de Attack.execute(), tras aplicar el golpe:
hud.drawBar(target.name(), target.hp(), target.stats().maxHp());
achievements.onHit(attacker, target, dealt);
```

Funciona, y tiene cuatro problemas que crecen a la vez:

1. **El motor importa la interfaz.** `Combat` recibe un `ConsoleUi`: rompe la regla de capas del proyecto (`engine` no importa `ui`). Un motor que no puede compilarse sin la consola no se puede probar ni reutilizar.
2. **Cada novedad toca el motor.** Un sonido, un contador de estadísticas o un registro de la partida pedirían otro parámetro en el constructor y otra línea en cada comando. El motor crece por cosas que no son combate.
3. **Hay que acordarse en cada sitio.** Si `UseSkill` se olvida de llamar a `achievements`, un golpe de habilidad no cuenta para «primera sangre» y nadie recibe un error.
4. **Un texto no es un dato.** Para dibujar una barra de vida hacen falta números (vida actual y vida máxima), no la frase «le hace 8 de daño». Sacar números de una frase es el peor camino posible.

Este es el olor: **el emisor de un hecho conoce, uno por uno, a todos los que quieren enterarse**. Observer lo cura invirtiendo la relación. Su intención en el catálogo es definir una dependencia de uno a muchos entre objetos, de modo que cuando uno cambia, los que dependen de él reciben el aviso [9]. Head First lo explica con una estación meteorológica que reparte sus medidas a varias pantallas [7], y el catálogo de Refactoring.Guru lo resume como un mecanismo de suscripción para avisar a varios objetos de lo que le ocurre al que observan [24]. Nystrom abre su capítulo de Observer con el mismo caso que vas a construir, los logros de un juego, y dedica un apartado a la baja de los observadores [19]. La idea cabe en una frase: quien emite *publica* un hecho; quien se interesa se *suscribe*; y ninguno de los dos conoce al otro.

9.2 Piezas del patrón Observer

En el libro clásico el sujeto es cada objeto observado. En Emberdeep hay muchos emisores (`Combat`, los comandos) y muchas cosas que se quieren escuchar, así que el sujeto es un *bus de eventos*: un objeto único al que todos publican y en el que todos se suscriben. Es la misma idea, con una pieza compartida en medio.

Rol	Clase en el ejemplo	Propósito	Paquete
Observador (interfaz funcional)	`GameListener`	Un único método, `onEvent`, que recibe el evento	`engine.event`
Sujeto (clase)	`EventBus`	Guarda a los oyentes (`subscribe`, `unsubscribe`) y los avisa de cada evento (`publish`)	`engine.event`
Evento (interfaz sellada y records)	`GameEvent`, `DamageDealt`, `DamageDodged`, `CombatEnded` (y otros cuatro records)	Un hecho ya ocurrido, con los datos simples que hacen falta para contarlo	`engine.event`
Observadores concretos (clases)	`CombatLog`, `HudListener`, `AchievementTracker`	Pintan mensajes, pintan barras de vida y cuentan logros	`ui.listener` y `engine.achievement`
Emisor (clase)	`Combat` (y los comandos, a través de él)	Publica lo que ocurre en el combate	`engine.combat`

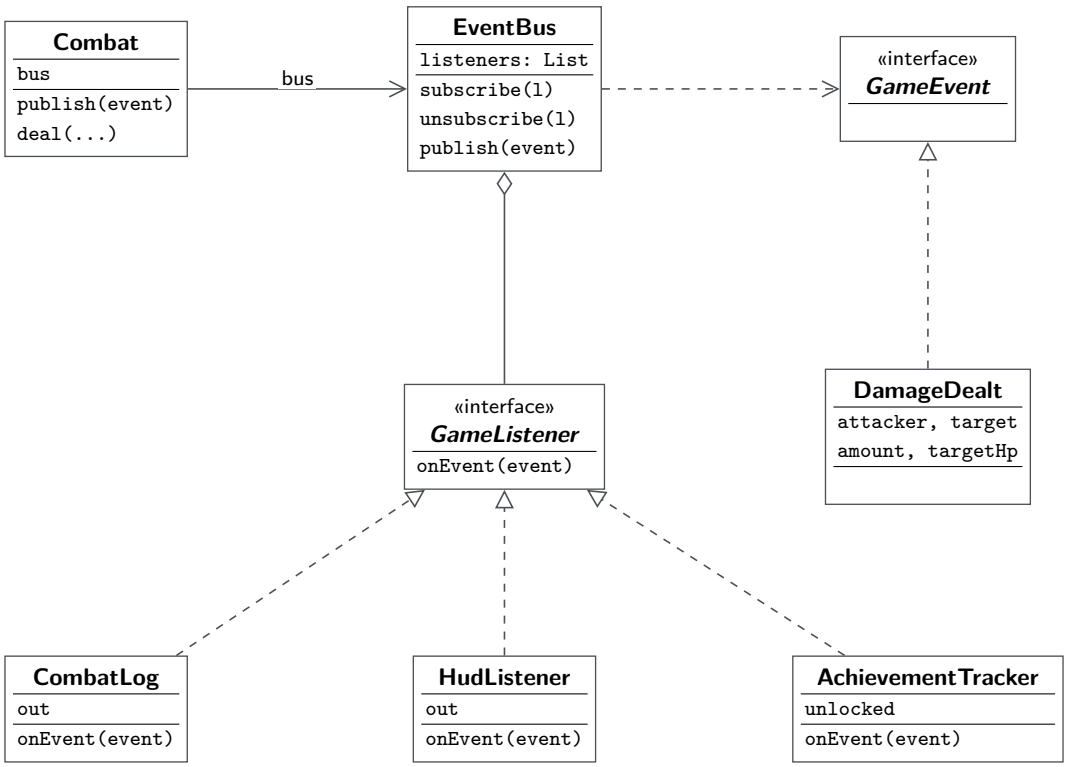


Diagrama UML del patrón Observer: el emisor publica un evento en el bus y el bus avisa a cada oyente suscrito.

Una observación sobre el *modelo de aviso*. Hay dos formas de avisar: decir «algo cambió» y dejar que el oyente pregunte al emisor (*pull*), o entregar en el aviso los datos del cambio (*push*). Aquí es *push*: el evento lleva el nombre del que ataca, el del golpeado, el daño y la vida que le queda. El oyente no necesita una referencia a `Combat` ni a ningún `Fighter`, y por eso no puede cambiarlos por error.

9.3 El ejemplo de Observer, pieza a pieza: `EventBus`

El observador. Un oyente solo tiene que poder recibir un evento. Una interfaz con un único método es una interfaz funcional (capítulo 2), así que cualquier lambda sirve:

`engine/event/GameListener.java`

```
package emberdeep.engine.event;

@FunctionalInterface
public interface GameListener {
    void onEvent(GameEvent event);
}
```

El JDK ya trae interfaces funcionales genéricas en `java.util.function` [22]: un `Consumer` de `GameEvent` serviría. Un tipo propio no añade comportamiento, pero dice qué es: en la firma de `subscribe` se lee «oyente de eventos del juego», no «alguien que consume algo».

El sujeto. El bus guarda una lista de oyentes y recorre esa lista en cada publicación. Esta es la *versión 1*; en la sección 9.5 verás un fallo real y la versión 2 que lo corrige.

`engine/event/EventBus.java (versión 1)`

```
package emberdeep.engine.event;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

public class EventBus {
    private final List<GameListener> listeners = new ArrayList<>();

    public void subscribe(GameListener listener) {
        listeners.add(Objects.requireNonNull(listener));
    }

    public void unsubscribe(GameListener listener) {
        listeners.remove(listener);
    }

    public void publish(GameEvent event) {
        for (GameListener listener : listeners) {
            listener.onEvent(event);
        }
    }
}
```

Tres detalles. Los oyentes reciben el evento *en el orden en que se suscribieron* y `publish` no vuelve hasta que el último ha terminado: es síncrono, y un oyente lento frena al emisor. `subscribe` rechaza `null` para que el fallo salte al suscribir, no más tarde, en mitad de una publicación. Y `unsubscribe` usa `remove`, que busca por igualdad: para darte de baja necesitas *la misma referencia* con la que te suscribiste. Una lambda escrita dos veces son dos objetos distintos; si quieres poder darla de baja, guárdala en una variable.

El emisor. `Combat` recibe el bus por constructor y lo usa para publicar. El punto central es `deal`, por donde pasa todo golpe (capítulo 7). Antes devolvía el daño y dejaba la frase a otros;

ahora, además, anuncia lo que ha pasado:

engine/combat/Combat.java (fragmento: deal)

```
public int deal(Fighter attacker, Fighter target, Damage damage) {
    int dealt = pipeline.apply(target, damage);
    target.takeDamage(dealt);
    boolean isHero = heroes.contains(target);
    if (dealt == 0) {
        publish(new DamageDodged(attacker.name(), target.name(),
                isHero));
    } else {
        publish(new DamageDealt(attacker.name(), target.name(), isHero,
                dealt, damage.type(),
                target.hp(), target.stats().maxHp()));
    }
    if (!target.isAlive()) {
        publish(new FighterDefeated(target.name(), isHero));
    }
    return dealt;
}
```

`deal` no sabe si hay uno o diez oyentes, ni qué hacen. Fíjate también en una decisión de vocabulario: un golpe que acaba en cero daño (porque se esquiva o porque el escudo se lo come entero) se publica como `DamageDodged`. Para el motor es un golpe que «no llegó»; el nombre del evento cuenta la historia de los dos casos. (En la partida real la esquiva está al 0 %, así que lo que verás será casi siempre el escudo.)

Los observadores. Un observador es una clase que implementa `GameListener`, o una lambda. Se suscribe una vez, normalmente donde se monta el programa (el `Main`):

Suscripciones (ejemplo)

```
EventBus events = session.events();
events.subscribe(new CombatLog(System.out));                // clase
events.subscribe(new HudListener(System.out));              // clase
events.subscribe(e -> System.out.println("evento: " + e)); // lambda
```

Un ejemplo con números. Aria lanza la Bomba de ceniza (10 de fuego) al Zombi, que tiene 22 de vida. `UseItem` llama a `deal`; el golpe baja al Zombi a 12 y se publica un `DamageDealt` con `amount` 10, `type` `FIRE`, `targetHp` 12 y `targetMaxHp` 22. El bus avisa por orden: el log escribe «Aria hace 10 de daño de fuego a Zombi.» y el HUD escribe una barra con 12 de 22. Ninguno de los dos sabe que el otro existe, y `deal` tampoco los conoce.

9.4 Observer: eventos `sealed` y `switch`

Un evento es un hecho ya ocurrido, y un hecho no cambia. Eso lo hace candidato a `record`: inmutable, con los datos en el nombre de los campos. Todos los eventos del juego son records que implementan una interfaz `sealed`, y viven en el mismo archivo:

engine/event/GameEvent.java

```
package emberdeep.engine.event;
```

```
2
3  import emberdeep.engine.combat.damage.DamageType;
4  import java.util.List;
5
6  public sealed interface GameEvent {
7      record DamageDealt(String attacker, String target,
8                         boolean targetIsHero, int amount, DamageType type,
9                         int targetHp, int targetMaxHp)
10             implements GameEvent {}
11
12     record DamageDodged(String attacker, String target,
13                         boolean targetIsHero) implements GameEvent {}
14
15     record FighterDefeated(String name, boolean isHero)
16             implements GameEvent {}
17
18     record Defended(String name, int shield, int defenseBonus)
19             implements GameEvent {}
20
21     record ItemUsed(String user, String target, String itemName,
22                     int targetHp, int targetMaxHp) implements GameEvent {}
23
24     record CombatEnded(boolean heroesWon, List<Integer> heroesHp)
25             implements GameEvent {
26         public CombatEnded {
27             heroesHp = List.copyOf(heroesHp);
28         }
29     }
30
31     record AchievementUnlocked(String name) implements GameEvent {}
32 }
```

Una interfaz `sealed` sin `permits` permite exactamente las clases que se declaran en el mismo archivo; aquí, los siete records. Tres decisiones de diseño:

- **Datos simples, no objetos del juego.** Los eventos llevan nombres (`String`), números y booleanos, nunca un `Fighter`. Es la lección del capítulo 8: un objeto mutable dentro de un dato lo deja de ser (el oyente vería el `Fighter` *de ahora*, no el de cuando ocurrió el golpe) y además permite al oyente cambiarlo. Por eso `DamageDealt` lleva `targetHp` y `targetMaxHp`: el HUD dibuja la barra sin preguntar a nadie.
- **Una lista dentro de un record se copia.** `CombatEnded` lleva la vida de cada héroe al terminar; `List.copyOf` garantiza que el evento sigue siendo inmutable (otra vez la copia superficial del capítulo 5, ahora resuelta de entrada).
- **Siete eventos.** Los hechos evidentes del combate son seis: golpe, golpe sin daño, derrota, objeto usado, fin del combate y logro. Falta uno, `Defended`: sin él la tecla «defender» dejaría de tener respuesta en el log. Es el coste de pasar de frases a eventos: cada frase que quieras seguir viendo necesita un hecho que la cuente.

Un `switch` que el compilador vigila. El oyente recibe un `GameEvent` y tiene que saber de qué tipo es. Con un `switch` de *patrones de tipo* sobre la interfaz sellada (capítulo 3) cada rama recibe el record ya convertido, y si nombras todos los tipos permitidos no hace falta `default` [4]. Este es el núcleo de `CombatLog`:

ui/listener/CombatLog.java (fragmento: onEvent)

```
@Override
public void onEvent(GameEvent event) {
    switch (event) {
        case DamageDealt e -> out.println(e.attacker() + " hace "
                + e.amount() + " de daño " + kind(e.type())
                + " a " + e.target() + ".");
        case DamageDodged e -> out.println("El golpe de " + e.attacker()
                + " a " + e.target() + " no hace daño.");
        case FighterDefeated e -> out.println(e.name()
                + " queda fuera de combate.");
        case Defended e -> out.println(e.name() + " se cubre: +" + e.shield()
                + " de escudo y +" + e.defenseBonus()
                + " de defensa este turno.");
        case ItemUsed e -> out.println(e.user() + " usa " + e.itemName()
                + " sobre " + e.target() + ".");
        case CombatEnded e -> out.println(e.heroesWon()
                ? "¡Victoria!" : "Derrota...");
        case AchievementUnlocked e -> out.println(
                "¡Logro desbloqueado: " + e.name() + "!");
    }
}
```

La variable `e` existe solo dentro de su rama, y por eso puede llamarse igual en todas. La ventaja se ve el día que alguien añade un evento. Si mañana el capítulo 11 añade `FighterHealed` y no actualizas este `switch`, el compilador se niega:

Error del compilador al añadir un evento

```
CombatLog.java:24: error: the switch statement does not cover all possible input
    ↪ values
        switch (event) {
        ^
```

Con un `if` en cadena, o con un `default`, ese olvido se convertiría en un evento ignorado en silencio. Es un coste a favor: los tipos sellados te señalan todos los oyentes que tienes que mirar. Por eso, en este proyecto, ni `CombatLog` ni `HudListener` ni `AchievementTracker` usan `default`: los eventos que no les interesan llevan una rama vacía (`case FighterDefeated e -> { }`), que dice «lo he visto y no me importa».

Un oyente que también publica. `AchievementTracker` escucha el bus y, cuando se cumple un logro, publica un `AchievementUnlocked` en el mismo bus. Su lógica es del juego (qué cuenta como logro), no de la pantalla, por eso vive en `engine` y no imprime nada; el log es quien pinta el aviso. Este es su núcleo:

engine/achievement/AchievementTracker.java (fragmento)

```
case DamageDealt e -> {
    if (!e.targetIsHero()) {
        unlock("Primera sangre");
    }
}
case DamageDodged e -> {
    if (e.targetIsHero()) {
```

```
            heroDodges++;
            if (heroDodges == DODGES_NEEDED) {
                unlock("Intocable");
            }
        }
    }
    case CombatEnded e -> {
        if (e.heroesWon() && e.heroesHp().contains(1)) {
            unlock("Último aliento");
        }
    }
```

(El archivo completo está en la solución del hito.) Hay dos hechos que merece la pena mirar de cerca. Primero, el **orden de suscripción**: si el tracker se suscribe después del log, al publicarse un golpe el log escribe el golpe, y *luego* el tracker publica el logro, que el log escribe a continuación: el aviso sale detrás de la frase que lo provocó. Si se suscribiera antes, el logro saldría antes que el golpe. Nada en el patrón lo impide; es una convención que se vigila. Segundo, **los eventos son hechos**: el logro se desbloquea cuando ocurre el golpe, aunque el jugador pulse `u` después y lo deshaga (capítulo 7). Deshacer cambia el estado del juego; no «despublica» lo que ya se anunció.

9.5 Observer: el oyente que se da de baja mientras se notifica

Hay un fallo clásico en la versión 1 del bus. Imagina un consejo para novatos que se muestra una sola vez y, después, se da de baja (el código es hipotético; no existe en el proyecto):

Hipotético: un oyente que se da de baja solo

```
public class FirstHint implements GameListener {
    private final EventBus bus;

    public FirstHint(EventBus bus) {
        this.bus = bus;
    }

    @Override
    public void onEvent(GameEvent event) {
        System.out.println("Consejo: defenderte absorbe golpes.");
        bus.unsubscribe(this);          // me doy de baja a mitad de publish
    }
}
```

Si el bus tiene este oyente y otros dos, al publicar el primer evento ocurre esto:

Salida

```
Consejo: defenderte absorbe golpes.
Exception in thread "main" java.util.ConcurrentModificationException
        at java.base/java.util.ArrayList$Itr.checkForComodification(...)
        at emberdeep.engine.event.EventBus.publish(...)
```

Qué es `ConcurrentModificationException`. El `for` de `publish` usa por debajo un *iterador* de la lista. Ese iterador apunta la «versión» de la lista cuando empieza a recorrerla y, en cada

paso, comprueba que nadie la ha cambiado. Si alguien añade o quita elementos por otro camino (aquí, `unsubscribe`, que llama a `remove` de la lista), el siguiente paso lanza la excepción. Es una protección: el iterador prefiere fallar a recorrer una lista que ya no es la que empezó. Se llama «concurrent» aunque no haya ningún hilo: basta con modificar la lista mientras se recorre.

Hay una trampa dentro de la trampa. La comprobación se hace al pedir el *siguiente* elemento. Si el oyente que se da de baja es el *penúltimo*, el recorrido ya cree que ha terminado, no pide ningún elemento más y no lanza nada, pero se salta al último oyente en silencio. Es peor que la excepción, y lo vivirás en el ejercicio 9.2.

El arreglo. Recorrer una *copia* de la lista. El bus puede cambiar su lista cuanto quiera, porque la copia no cambia durante el recorrido. Esta es la *versión 2*:

`engine/event/EventBus.java (versión 2: publish)`

```
public void publish(GameEvent event) {
    for (GameListener listener : List.copyOf(listeners)) {
        listener.onEvent(event);
    }
}
```

Dos consecuencias a conocer, porque son la definición del comportamiento:

- El oyente que se da de baja *sigue recibiendo el evento en curso* (estaba en la copia), y ya no recibirá los siguientes.
- Un oyente que se suscribe durante la publicación *no recibe el evento en curso*, solo los siguientes.

La copia cuesta una lista pequeña por evento; en Emberdeep es irrelevante. Existen estructuras pensadas para este caso (por ejemplo `CopyOnWriteArrayList`, que copia al modificar en lugar de al recorrer), pero para empezar la copia explícita se entiende de un vistazo. Con la versión 2, además, el tracker puede publicar `AchievementUnlocked` dentro de `onEvent`: es una publicación anidada, y cada una recorre su propia copia.

9.6 Observer: cuándo sí y cuándo no

Compensa cuando varias partes distintas reaccionan al mismo hecho y quien lo provoca no debería conocerlas: pantalla, sonido, estadísticas, logros, registro de la partida. También cuando el conjunto de interesados cambia en tiempo de ejecución (un oyente se apunta o se baja). Y cuando quieres probar el motor sin pantalla: el motor publica, y una prueba se suscribe con una lambda y comprueba lo recibido.

No compensa si hay un solo interesado y siempre el mismo (una llamada directa es más clara) o si el emisor necesita una *respuesta* del que escucha. Observer es de ida: el oyente no devuelve nada al emisor. Y tiene costes reales:

- **El flujo se vuelve invisible.** Al leer `deal` no ves qué ocurre después del golpe. Para saberlo hay que buscar quién se suscribe. Es el precio del desacoplamiento.
- **El orden importa y nadie lo garantiza.** Aquí es el de suscripción, y lo decide el `Main`; si dos oyentes dependen de ir en cierto orden (log antes que tracker), esa dependencia es silenciosa.
- **Hay que darse de baja.** Un oyente que ya no se usa pero sigue en el bus sigue recibiendo eventos y no puede ser recogido por el recolector de basura. Con oyentes que viven toda

la partida no es un problema; con los de corta vida, sí.

- **Es síncrono.** Un oyente que tarda frena al emisor, y una excepción en un oyente interrumpe a los que quedaban detrás.

Mediator no es Observer

En el capítulo 11 verás Mediator, que a primera vista se parece: también hay un objeto en medio y las partes no se conocen entre sí. La diferencia es de dirección y de poder. En Observer el emisor *anuncia* y no espera respuesta; los oyentes no se coordinan entre ellos. En Mediator las partes *piden* cosas al mediador, que decide y puede cambiar el estado de otras (por ejemplo, la arena que fuerza a un enemigo a atacar a quien lo provocó). Si lo que tienes es «cuando pase X, que se enteren», es Observer; si es «para hacer X, hay que consultar y mover a varios», es Mediator.

9.7 En el proyecto: H10 con Observer

Este capítulo avanza el hito H10, y el trabajo se reparte en **tres pasos** que se comprueban uno a uno. Cambios del hito:

- **Nuevo paquete** `engine/event/`: `GameEvent`, `GameListener` y `EventBus`.
- **Nuevo paquete** `engine/achievement/`, con `AchievementTracker`.
- **Nuevo paquete** `ui/listener/`, con `CombatLog` y `HudListener`. Es provisional: en el capítulo 10 se convierte en las vistas.
- **Cambian** `Combat`, los cuatro comandos y `CommandHistory` (publican en lugar de devolver cadenas), `GameSession` (crea el bus y lo expone con `events()`) y `Main` (conecta los oyentes).
- **Desaparece** `ConsoleUi`. Se reparte: lo que *pintaba* lo hacen `CombatLog` y `HudListener`; lo que *preguntaba* (el bucle de rondas, el menú de cada héroe y el modo mapa) pasa a `ConsoleMenu`, que ya no imprime mensajes del motor.

Qué cambia en la partida. Las reglas no cambian, y los números de las trazas tampoco (los mismos golpes, la misma vida final). Cambia la redacción: «hace N de daño físico», «de fuego», el golpe sin daño se dice «no hace daño», y bajo cada golpe aparece la barra de vida del golpeado. Los menús y las teclas son los de siempre.

Los tres pasos y qué debes ver en cada uno.

- **Paso 1, las piezas.** `GameEvent`, `GameListener` y `EventBus`. Resultado: un programa de comprobación suscribe una lambda, publica un evento, la da de baja y publica otro; el primero llega y el segundo no.
- **Paso 2, el motor publica.** `Combat` y los comandos publican; `CombatLog`, `HudListener` y `ConsoleMenu` sustituyen a `ConsoleUi`. Resultado: un guion de teclas juega una ronda y el texto sale del log, no del motor (y `grep System.out` en `engine/` no encuentra nada).
- **Paso 3, los logros.** `AchievementTracker`. Resultado: una partida de comprobación, con héroe frágil, desbloquea los tres logros.

Hito 10: un motor que emite eventos con Observer

Tras este hito, el motor no devuelve cadenas ni imprime: publica eventos en un bus, y la pantalla, el HUD y los logros son oyentes. La estructura:

Estructura de carpetas tras el capítulo 9

```
emberdeep/
|-- settings.gradle
|-- build.gradle                    (con JUnit 6.1.3)
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
|-- scripts/
|   `-- demo.txt
`-- src/
    |-- main/java/emberdeep/
    |   |-- app/
    |   |   |-- Main.java              (composition root)
    |   |   `-- GameConfig.java
    |   |-- engine/
    |   |   |-- character/             Stats, Hero, HeroBuilder, Enemy,
    |   |   |                          EnemyKind, Fighter, Skill,
    |   |   |                          EnemyTemplate, TemplateRegistry
    |   |   |-- stats/                 StatSource, StatKind, BaseStats,
    |   |   |                          StatDecorator, WeaponBonus,
    |   |   |                          ArmorBonus, TempBuff
    |   |   |-- combat/
    |   |   |   |-- Combat.java
    |   |   |   |-- ai/                EnemyAi, FirstAliveAi, LowestHpAi,
    |   |   |   |                      RandomTargetAi, SkillfulAi
    |   |   |   |-- command/           CombatCommand, Attack, Defend,
    |   |   |   |                      UseItem, UseSkill, CommandHistory
    |   |   |   |-- damage/            DamageFormula, FlatFormula,
    |   |   |   |                      ScaledFormula, Damage, DamageType
    |   |   |   `-- defense/           DamageHandler, DamagePipeline,
    |   |   |                          DodgeHandler, ShieldHandler,
    |   |   |                          ArmorHandler, ResistanceHandler
    |   |   |-- item/                  Item, ItemKind, Inventory
    |   |   |-- event/                 GameEvent, GameListener, EventBus
    |   |   |-- achievement/           AchievementTracker
    |   |   |-- save/                  Snapshot, FighterState,
    |   |   |                          SnapshotStore
    |   |   |-- session/               GameSession
    |   |   `-- world/
    |   |       |-- Dungeon.java, Floor.java,
    |   |       |   DungeonBuilder.java
    |   |       |-- biome/             BiomeFactory, CryptBiome,
    |   |       |                      CavernBiome
    |   |       `-- map/               TileType, TileTypes, GameMap,
    |   |                              MapParser, MapData
    |   `-- ui/
    |       |-- ConsoleMenu.java
    |       |-- input/                 InputSource, ConsoleInput,
    |       |                          ScriptInput
    |       `-- listener/              CombatLog, HudListener
    `-- test/java/emberdeep/
        `-- engine/save/               SnapshotTest
```

Lo practicas en el ejercicio 9.4. Antes, tres ejercicios más pequeños, en otros dominios.

9.8 Ejercicios

Ejercicio 9.1: Un termómetro que avisa ★★☆☆☆

No es de Emberdeep. Crea una carpeta vacía y, dentro, archivos sin paquete. Vas a construir un bus mínimo para un termómetro que avisa de las temperaturas altas sin saber quién escucha. Sigue los pasos en orden.

1. `TemperatureListener.java`: una interfaz con un único método, `void onHigh(int celsius)`.
2. `TemperatureBus.java`: una clase con una lista de `TemperatureListener`, un método `subscribe(TemperatureListener listener)` que lo añade y un método `publish(int celsius)` que recorre la lista y llama a `onHigh(celsius)` en cada oyente.
3. `Thermometer.java`: recibe en el constructor un `TemperatureBus` y un límite (`int`). Su método `measure(int celsius)` publica la temperatura en el bus *solo si es mayor que el límite*. El termómetro no debe tener ninguna referencia a un oyente.
4. `HighCounter.java`: un oyente que implementa `TemperatureListener` y cuenta cuántos avisos recibe (un método `count()` devuelve el total).
5. `Main.java`: crea el bus, un `Thermometer` con límite 30 y un `HighCounter`; suscribe el contador y también una lambda que escriba `ALARMA: 31 grados` (con la temperatura recibida); mide 25, 31, 28 y 35; y al final imprime cuántos avisos contó el contador.

Compila con `javac *.java` y ejecuta con `java Main`. Está bien si la salida es exactamente:

Salida esperada

```
ALARMA: 31 grados
ALARMA: 35 grados
Avisos recibidos por el contador: 2
```

y si puedes añadir un tercer oyente en `Main` sin tocar `Thermometer` ni `TemperatureBus`.

Pista: *La lambda tiene la forma* `c -> System.out.println("ALARMA: " + c + " grados")`. *Para que una lambda pueda ser un* `TemperatureListener`, *la interfaz debe tener un único método.*

Guiado ▪ solución en la pág. 357

Ejercicio 9.2: El aviso que se da de baja ★★★☆☆

No es de Emberdeep. Este tablón avisa a sus oyentes, y uno de ellos (la bienvenida) se da de baja tras el primer mensaje. Guarda estos cuatro archivos en una carpeta sin paquete:

Listener.java

```
1 public interface Listener {
2     void onMessage(String message);
3 }
```

Board.java

```java
import java.util.ArrayList;
import java.util.List;

public class Board {
    private final List<Listener> listeners = new ArrayList<>();

    public void subscribe(Listener listener) {
        listeners.add(listener);
    }

    public void unsubscribe(Listener listener) {
        listeners.remove(listener);
    }

    public void post(String message) {
        for (Listener listener : listeners) {
            listener.onMessage(message);
        }
    }
}
```

Welcome.java

```java
public class Welcome implements Listener {
    private final Board board;

    public Welcome(Board board) {
        this.board = board;
    }

    @Override
    public void onMessage(String message) {
        System.out.println("Bienvenida: " + message);
        board.unsubscribe(this);
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Board board = new Board();
        board.subscribe(new Welcome(board));
        board.subscribe(m -> System.out.println("Pantalla: " + m));
        board.subscribe(m -> System.out.println("Archivo: " + m));
        board.post("hola");
        board.post("adiós");
    }
}
```

Debería escribir cinco líneas: la bienvenida y los dos oyentes para «hola», y los dos oyentes para «adiós». Pero no lo hace.

1. Ejecútalo (`javac *.java` y `java Main`). Copia el nombre de la excepción y explica con tus palabras qué ha pasado y en qué línea de `Board` ocurre.
2. Cambia solo el orden de las dos primeras líneas de suscripción de `Main` para que la bienvenida sea la *segunda* de las tres, y vuelve a ejecutarlo *sin arreglar nada*. Ahora no hay excepción. Compara la salida con la esperada y di qué oyente se ha quedado sin recibir qué.
3. Corrige `Board` con un cambio de una sola línea, de modo que las dos ordenaciones den la salida correcta. Después de «hola», la bienvenida no debe volver a recibir mensajes.

Está bien si, con la bienvenida en cualquier posición, la salida son cinco líneas y la bienvenida solo aparece una vez, y si sabes explicar por qué en el paso 2 no saltó la excepción.

Pista: *Piensa en qué lista recorre el* `for` *mientras* `unsubscribe` *la cambia. Una copia de la lista no se ve afectada por los cambios de la original.*

Depuración ▪ solución en la pág. 359

Ejercicio 9.3: Un logro de cinco aciertos seguidos ★★★☆☆

No es de Emberdeep. Un arquero dispara y anuncia cada disparo en un bus; tú vas a añadir un logro sin modificar al arquero ni al bus. Guarda estos archivos, ya hechos, en una carpeta sin paquete (los usarás tal cual):

`ShotEvent.java`

```
public sealed interface ShotEvent permits Hit, Miss {}

record Hit(int damage) implements ShotEvent {}

record Miss() implements ShotEvent {}
```

`ShotListener.java`

```
public interface ShotListener {
    void onEvent(ShotEvent event);
}
```

ShotBus.java

```java
import java.util.ArrayList;
import java.util.List;

public class ShotBus {
    private final List<ShotListener> listeners = new ArrayList<>();

    public void subscribe(ShotListener listener) {
        listeners.add(listener);
    }

    public void publish(ShotEvent event) {
        for (ShotListener listener : List.copyOf(listeners)) {
            listener.onEvent(event);
        }
    }
}
```

Archer.java

```java
public class Archer {
    private final ShotBus bus;

    public Archer(ShotBus bus) {
        this.bus = bus;
    }

    public void shoot(boolean hits) {
        bus.publish(hits ? new Hit(4) : new Miss());
    }
}
```

Y este `Main`, al que solo le falta la línea que suscribe tu logro (donde pone el comentario):

Main.java

```java
public class Main {
    public static void main(String[] args) {
        ShotBus bus = new ShotBus();
        Archer archer = new Archer(bus);
        bus.subscribe(event -> {
            if (event instanceof Miss) {
                System.out.println("Fallo");
            }
        });
        // aquí: suscribe tu StreakAchievement

        boolean[] shots = {true, true, false, true, true, true, true,
                true, false, true, true, true, true, true};
        for (boolean hits : shots) {
            archer.shoot(hits);
        }
    }
}
```

Escribe `StreakAchievement`, un `ShotListener` que escriba `Logro: cinco aciertos seguidos` en el momento en que se cumple. Los fallos reinician la racha, y el logro se desbloquea *una sola vez* en toda la partida. Usa un `switch` sobre el evento, sin `default`. Está bien si, tras añadir la línea de suscripción a `Main` y compilar con `javac *.java`, `java Main` escribe exactamente:

Salida esperada

```
Fallo
Logro: cinco aciertos seguidos
Fallo
```

y si no has cambiado ni `Archer` ni `ShotBus`. (En los 14 disparos hay dos rachas de cinco aciertos; solo la primera debe dar el logro.)

Pista: *Necesitas dos campos: un contador de la racha actual y un indicador de si el logro ya se desbloqueó. El caso del fallo pone el contador a cero.*

Práctica ▪ solución en la pág. 360

Ejercicio 9.4: Hito 10: eventos, log, HUD y logros con Observer ★★★★☆

Implementa el hito H10 partiendo del estado del capítulo anterior (solución del ejercicio 8.4), en tres partes. Comprueba cada una antes de pasar a la siguiente.

Parte 1 (las piezas): el bus.

1. Paquete `engine/event/`: `GameEvent` (interfaz `sealed`, con siete records anidados), `GameListener` (interfaz funcional con `void onEvent(GameEvent event)`) y `EventBus` con `subscribe`, `unsubscribe` y `publish`, ya con la copia de la lista de la sección 9.5. Los records:
   - `DamageDealt(String attacker, String target, boolean targetIsHero, int amount, DamageType type, int targetHp, int targetMaxHp)`
   - `DamageDodged(String attacker, String target, boolean`

`targetIsHero)`

- `FighterDefeated(String name, boolean isHero)`
- `Defended(String name, int shield, int defenseBonus)`
- `ItemUsed(String user, String target, String itemName, int targetHp, int targetMaxHp)`
- `CombatEnded(boolean heroesWon, List<Integer> heroesHp)`, que copia la lista con `List.copyOf` en su constructor compacto
- `AchievementUnlocked(String name)`

Está bien si este archivo temporal, `app/Check09a.java`, escribe las dos líneas de debajo (el evento llega, y tras la baja ya no llega el siguiente):

`app/Check09a.java (parte 1; bórralo al terminar)`

```java
package emberdeep.app;

import emberdeep.engine.event.EventBus;
import emberdeep.engine.event.GameEvent.DamageDodged;
import emberdeep.engine.event.GameEvent.FighterDefeated;
import emberdeep.engine.event.GameListener;

public class Check09a {
    public static void main(String[] args) {
        EventBus bus = new EventBus();
        GameListener printer = e -> System.out.println("oído: " + e);
        bus.subscribe(printer);
        bus.publish(new DamageDodged("Esqueleto", "Aria", true));
        bus.unsubscribe(printer);
        bus.publish(new FighterDefeated("Esqueleto", false));
        System.out.println("fin");
    }
}
```

Salida esperada

```
oído: DamageDodged[attacker=Esqueleto, target=Aria, targetIsHero=true]
fin
```

Parte 2 (el motor publica): fuera las cadenas, fuera `ConsoleUi`.

1. `CombatCommand.execute()` y `CommandHistory.run(...)` pasan a ser `void`. `Combat.perform(...)` y `Combat.enemyPhase()` también.
2. `Combat` recibe un `EventBus` como quinto parámetro del constructor y gana `void publish(GameEvent event)`. El método `deal` pasa a ser `deal(Fighter attacker, Fighter target, Damage damage)` y publica: un `DamageDealt` si el daño final es mayor que 0 (con la vida y la vida máxima del golpeado *ya aplicado el golpe*), un `DamageDodged` si es 0 y, si el golpeado cae, un `FighterDefeated`. Un método privado `checkEnd()` publica un `CombatEnded` (con la vida de cada héroe, en orden) la primera vez que el combate termina; se llama al final de `perform` y al final del bucle de `enemyPhase`.
3. `Attack` y `UseSkill` llaman a `combat.deal(attacker/user, target, damage)` y ya

no devuelven mensajes. `Defend` recibe ahora el `Combat` (`Defend(Combat combat, Fighter user)`) y publica un `Defended`. `UseItem` publica un `ItemUsed` solo cuando el objeto cura (con la vida y vida máxima del objetivo tras curar); la bomba hace su daño con `deal` y no publica `ItemUsed`. El `undo()` de los cuatro no cambia.

4. `GameSession` crea el `EventBus` en su constructor, se lo pasa a `Combat` y lo expone con `EventBus events()`.

5. `ui/ConsoleUi` se sustituye por `ui/ConsoleMenu`: el mismo bucle, menú y modo mapa, pero sin imprimir los mensajes del motor ni el «¡Victoria!» final.

6. Paquete `ui/listener/`: `CombatLog(PrintStream out)` y `HudListener(PrintStream out)`, ambos `GameListener`, con un `switch` exhaustivo sin `default`. Los textos exactos de `CombatLog`:

Evento	Línea que escribe
`DamageDealt`	«Aria hace 8 de daño físico a Esqueleto.» (o «de daño de fuego» si el tipo es `FIRE`)
`DamageDodged`	«El golpe de Esqueleto a Cael no hace daño.»
`FighterDefeated`	«Esqueleto queda fuera de combate.»
`Defended`	«Cael se cubre: +5 de escudo y +2 de defensa este turno.»
`ItemUsed`	«Aria usa Poción de cripta sobre Borin.»
`CombatEnded`	«¡Victoria!» o «Derrota...»
`Achievement Unlocked`	«¡Logro desbloqueado: Primera sangre!»

   `HudListener` escribe, tras un `DamageDealt` o un `ItemUsed`, una línea con dos espacios, el nombre del afectado, una barra de 10 caracteres entre corchetes y «vida/máxima», por ejemplo `Zombi [######....] 12/22` (con dos espacios delante). Las casillas llenas, con el carácter `#`, son $\lceil 10 \cdot hp/maxHp \rceil$ y el resto son puntos.

7. `Main`: usa `ConsoleMenu` y, tras crear la sesión, suscribe en este orden `CombatLog` y `HudListener` (de momento, sin logros).

Está bien si este guion, `scripts/check09.txt` (bórralo al terminar), jugado con `./gradlew -q run -args="scripts/check09.txt"`, produce lo que se indica debajo:

scripts/check09.txt (parte 2; bórralo al terminar)

```
# Ronda 1: Aria lanza la bomba al Zombi
3
3
3
# Borin ataca al Esqueleto
1
1
# Cael se cubre
2
```

Salida esperada (el guion se acaba y el programa termina con IllegalStateException)

```
=== Emberdeep ===
--- Ronda 1 ---
Héroes:   Aria 30/30, Borin 40/40, Cael 24/24
Enemigos: Esqueleto 18/18, Rata de ceniza 8/8, Zombi 22/22
Objetos:  Poción de cripta, Poción de cripta, Bomba de ceniza
Turno de Aria. [1] atacar  [2] defender  [3] objeto  [4] habilidad  [u]
    ↪ deshacer  [c] guardar  [r] volver
> 3
Objeto:  [1] Poción de cripta  [2] Poción de cripta  [3] Bomba de ceniza
> 3
Objetivo:  [1] Esqueleto  [2] Rata de ceniza  [3] Zombi
> 3
Aria hace 10 de daño de fuego a Zombi.
  Zombi [######....] 12/22
Héroes:   Aria 30/30, Borin 40/40, Cael 24/24
Enemigos: Esqueleto 18/18, Rata de ceniza 8/8, Zombi 12/22
Objetos:  Poción de cripta, Poción de cripta
Turno de Borin. [1] atacar  [2] defender  [3] objeto  [4] habilidad  [u]
    ↪ deshacer  [c] guardar  [r] volver
> 1
Objetivo:  [1] Esqueleto  [2] Rata de ceniza  [3] Zombi
> 1
Borin hace 6 de daño físico a Esqueleto.
  Esqueleto [#######...] 12/18
Héroes:   Aria 30/30, Borin 40/40, Cael 24/24
Enemigos: Esqueleto 12/18, Rata de ceniza 8/8, Zombi 12/22
Objetos:  Poción de cripta, Poción de cripta
Turno de Cael. [1] atacar  [2] defender  [3] objeto  [4] habilidad  [u]
    ↪ deshacer  [c] guardar  [r] volver
> 2
Cael se cubre: +5 de escudo y +2 de defensa este turno.
El golpe de Esqueleto a Cael no hace daño.
El golpe de Rata de ceniza a Cael no hace daño.
Zombi hace 1 de daño físico a Aria.
  Aria [##########] 29/30
--- Ronda 2 ---
Héroes:   Aria 29/30, Borin 40/40, Cael 24/24
Enemigos: Esqueleto 12/18, Rata de ceniza 8/8, Zombi 12/22
Objetos:  Poción de cripta, Poción de cripta
Turno de Aria. [1] atacar  [2] defender  [3] objeto  [4] habilidad  [u]
    ↪ deshacer  [c] guardar  [r] volver
```

Comprueba además que el motor ya no imprime ni devuelve texto: desde la carpeta `emberdeep/`, `grep -rn "System.out" src/main/java/emberdeep/engine` no debe encontrar nada.

Parte 3 (los logros): `AchievementTracker`.

1. `engine/achievement/AchievementTracker`: un `GameListener` que recibe el bus en el constructor y publica `AchievementUnlocked` en él. Tres logros, cada uno como mucho una vez: «Primera sangre» (el primer `DamageDealt` con `targetIsHero` falso), «Intocable» (el quinto `DamageDodged` con `targetIsHero` verdadero) y «Último aliento» (un `CombatEnded` con victoria y algún héroe con exactamente 1 de vida). Sin `default` en el `switch`.

2. `Main`: suscribe el tracker *después* del log y del HUD.

Está bien si este programa temporal, `app/Check09c.java`, que juega a un héroe frágil (7 de vida y defensa 0) contra el Esqueleto, escribe lo de debajo. Cinco rondas de defensa dejan a todos los golpes en cero daño (el escudo de 5 absorbe justo los 5 que pasan la fórmula); en la sexta ronda el héroe deja de defenderse y recibe un golpe de 6 que lo deja en 1 de vida, y en la séptima remata:

app/Check09c.java (parte 3; bórralo al terminar)

```java
package emberdeep.app;

import emberdeep.engine.achievement.AchievementTracker;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.Attack;
import emberdeep.engine.combat.command.Defend;
import emberdeep.engine.combat.command.UseSkill;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.event.EventBus;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.ui.listener.CombatLog;
import emberdeep.ui.listener.HudListener;
import java.util.List;
import java.util.Random;

public class Check09c {
    public static void main(String[] args) {
        Random random = new Random(42);
        Hero aria = new HeroBuilder().name("Aria")
                .stats(new Stats(7, 8, 0))
                .skill(new Skill("Golpe poderoso", 12, DamageType.PHYSICAL))
                .build();
        GameSession session = new GameSession(List.of(aria),
                new CryptBiome(random), List.of(EnemyKind.SKELETON),
                new FlatFormula(), new DamagePipeline(random, 0.0));
        EventBus events = session.events();
        events.subscribe(new CombatLog(System.out));
        events.subscribe(new HudListener(System.out));
        events.subscribe(new AchievementTracker(events));

        Combat combat = session.combat();
        Fighter hero = session.heroes().get(0);
        Fighter skeleton = session.enemies().get(0);
        for (int round = 1; round <= 5; round++) {
            combat.perform(new Defend(combat, hero));
            combat.enemyPhase();
        }
        combat.perform(new UseSkill(combat, hero,
                hero.skills().get(0), skeleton));
        combat.enemyPhase();
        combat.perform(new Attack(combat, hero, skeleton));
    }
}
```

Salida esperada

```
Aria se cubre: +5 de escudo y +2 de defensa este turno.
El golpe de Esqueleto a Aria no hace daño.
Aria se cubre: +5 de escudo y +2 de defensa este turno.
El golpe de Esqueleto a Aria no hace daño.
Aria se cubre: +5 de escudo y +2 de defensa este turno.
El golpe de Esqueleto a Aria no hace daño.
Aria se cubre: +5 de escudo y +2 de defensa este turno.
El golpe de Esqueleto a Aria no hace daño.
Aria se cubre: +5 de escudo y +2 de defensa este turno.
El golpe de Esqueleto a Aria no hace daño.
¡Logro desbloqueado: Intocable!
Aria hace 12 de daño físico a Esqueleto.
  Esqueleto [####......] 6/18
¡Logro desbloqueado: Primera sangre!
Esqueleto hace 6 de daño físico a Aria.
  Aria [##........] 1/7
Aria hace 8 de daño físico a Esqueleto.
  Esqueleto [..........] 0/18
Esqueleto queda fuera de combate.
¡Victoria!
¡Logro desbloqueado: Último aliento!
```

Con el tracker ya suscrito, el guion de la parte 2 añade además, justo después de la barra del Zombi, la línea «¡Logro desbloqueado: Primera sangre!». Borra `Check09a.java`, `Check09c.java` y `scripts/check09.txt`.

Pista: *En `Combat`, `heroes.contains(target)` te dice si el golpeado es un héroe (los `Fighter` se comparan por identidad). Para la barra: $\lceil a/b \rceil$ con enteros es `(a + b - 1) / b`. En `AchievementTracker`, un `Set<String>` de logros ya publicados evita repetirlos.*

Proyecto ▪ solución en la pág. 362

9.9 Resumen

Resumen

- El olor de Observer es un emisor que conoce, uno por uno, a todos los que quieren enterarse: el motor importa la interfaz, crece con cada novedad, exige acordarse en cada sitio y entrega frases donde hacían falta datos. Observer invierte la relación: el emisor publica y los interesados se suscriben.
- Piezas: el observador (`GameListener`, interfaz funcional), el sujeto (`EventBus`, con `subscribe`, `unsubscribe` y `publish`), los eventos (`GameEvent` y sus records), los observadores concretos (`CombatLog`, `HudListener`, `AchievementTracker`) y el emisor (`Combat`).
- Un evento es un hecho ya ocurrido: un `record` inmutable con datos simples (nombres, números), nunca objetos mutables del juego; las listas se copian. La interfaz `sealed` y el `switch` sin `default` hacen que el compilador señale cada oyente cuando se añade un evento.
- Los oyentes se avisan en el orden de suscripción y de forma síncrona. Un oyente puede publicar mientras escucha (el tracker publica el logro), y el orden entre log y tracker decide qué línea sale antes.

- Darse de baja mientras se recorre la lista lanza `ConcurrentModificationException` (o, si es el penúltimo, salta a un oyente en silencio). El arreglo es recorrer una copia con `List.copyOf`: el que se da de baja recibe el evento en curso y el que se suscribe no.
- Observer compensa cuando varios reaccionan al mismo hecho sin que el emisor los conozca; tiene costes (flujo invisible, orden sin garantía, bajas olvidadas, síncrono). Los eventos son hechos: deshacer una acción no los retira. Mediator (capítulo 11) no es Observer: allí las partes piden y el mediador decide.
- H10: `engine/event/`, `engine/achievement/` y `ui/listener/`; el motor publica en lugar de devolver cadenas; `ConsoleUi` se reparte entre `CombatLog`, `HudListener` y `ConsoleMenu`; `Defended` es el séptimo evento.

Soluciones de los ejercicios

Solución del ejercicio 9.1

enunciado en la pág. 346

Cinco archivos sin paquete, en la misma carpeta.

`TemperatureListener.java`

```java
public interface TemperatureListener {
    void onHigh(int celsius);
}
```

`TemperatureBus.java`

```java
import java.util.ArrayList;
import java.util.List;

public class TemperatureBus {
    private final List<TemperatureListener> listeners = new ArrayList<>();

    public void subscribe(TemperatureListener listener) {
        listeners.add(listener);
    }

    public void publish(int celsius) {
        for (TemperatureListener listener : listeners) {
            listener.onHigh(celsius);
        }
    }
}
```

Thermometer.java

```java
public class Thermometer {
    private final TemperatureBus bus;
    private final int limit;

    public Thermometer(TemperatureBus bus, int limit) {
        this.bus = bus;
        this.limit = limit;
    }

    public void measure(int celsius) {
        if (celsius > limit) {
            bus.publish(celsius);
        }
    }
}
```

HighCounter.java

```java
public class HighCounter implements TemperatureListener {
    private int count;

    @Override
    public void onHigh(int celsius) {
        count++;
    }

    public int count() {
        return count;
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        TemperatureBus bus = new TemperatureBus();
        Thermometer thermometer = new Thermometer(bus, 30);
        HighCounter counter = new HighCounter();
        bus.subscribe(counter);
        bus.subscribe(c -> System.out.println("ALARMA: " + c + " grados"));

        thermometer.measure(25);
        thermometer.measure(31);
        thermometer.measure(28);
        thermometer.measure(35);
        System.out.println("Avisos recibidos por el contador: "
                + counter.count());
    }
}
```

Por qué funciona. `Thermometer` solo conoce al bus: mide y, si la temperatura supera el límite, la publica. No sabe cuántos oyentes hay ni qué hacen. Con 25 y 28 no publica nada

(no superan 30); con 31 y 35 el bus avisa a los dos oyentes por orden de suscripción: primero el contador suma, luego la lambda escribe. Por eso salen dos líneas de alarma y el contador termina en 2. Un tercer oyente es una línea más en `Main` (`bus.subscribe(...)`), sin tocar ni el termómetro ni el bus: ese es el desacoplamiento que busca el patrón.

Errores típicos:

- Dar al termómetro una lista de oyentes propia: entonces cada novedad lo obliga a cambiarlo, que es justo el olor del capítulo.
- Comparar con `>=` en lugar de `>`: el límite exacto (30) también avisaría.
- Publicar todas las medidas y filtrar en el oyente: funciona, pero obliga a cada oyente a repetir el filtro.

Solución del ejercicio 9.2

enunciado en la pág. 346

Solo cambia `Board`; los otros tres archivos se quedan como estaban.

Board.java

```
import java.util.ArrayList;
import java.util.List;

public class Board {
    private final List<Listener> listeners = new ArrayList<>();

    public void subscribe(Listener listener) {
        listeners.add(listener);
    }

    public void unsubscribe(Listener listener) {
        listeners.remove(listener);
    }

    public void post(String message) {
        for (Listener listener : List.copyOf(listeners)) {
            listener.onMessage(message);
        }
    }
}
```

Paso 1: la excepción. Con la bienvenida en primer lugar, el programa escribe «Bienvenida: hola» y se detiene con `ConcurrentModificationException`. Ocurre en el `for` de `post`, al pedir el segundo oyente: la bienvenida se dio de baja (`remove` cambió la lista) a mitad del recorrido, y el iterador del `for`, que vigila que nadie toque la lista mientras la recorre, lo detecta en su siguiente paso y falla.

Paso 2: el fallo silencioso. Con este `Main` (la bienvenida, segunda de tres):

Main.java (orden del paso 2)

```java
public class Main {
    public static void main(String[] args) {
        Board board = new Board();
        board.subscribe(m -> System.out.println("Pantalla: " + m));
        board.subscribe(new Welcome(board));
        board.subscribe(m -> System.out.println("Archivo: " + m));
        board.post("hola");
        board.post("adiós");
    }
}
```

y el `Board` sin arreglar, la salida es:

Salida sin arreglar (sin excepción)

```
Pantalla: hola
Bienvenida: hola
Pantalla: adiós
Archivo: adiós
```

El oyente «Archivo» no recibió «hola». La lista tenía tres elementos; al darse de baja la bienvenida (la segunda), quedaron dos, y el iterador, que ya había pasado por dos, cree que ha terminado: no pide un tercero, así que no hay excepción, y el último oyente se queda sin el mensaje. Es peor que el caso del paso 1, porque no avisa de nada.

Paso 3: la corrección. Recorrer `List.copyOf(listeners)` en lugar de la lista misma. La copia no cambia aunque `unsubscribe` cambie la original, así que el recorrido visita a los tres oyentes, y la bienvenida ya no está en la lista para el siguiente mensaje. Con la bienvenida primera la salida es «Bienvenida: hola», «Pantalla: hola», «Archivo: hola», «Pantalla: adiós», «Archivo: adiós». Con la bienvenida segunda es «Pantalla: hola», «Bienvenida: hola», «Archivo: hola», «Pantalla: adiós», «Archivo: adiós». Cinco líneas en los dos casos.

Errores típicos:

- Atrapar la excepción con un `try/catch`: tapa el síntoma y deja el fallo silencioso del paso 2 sin arreglar.
- Copiar la lista *fuera* del bucle, una vez, y reutilizarla: la copia queda desfasada y los oyentes nuevos no la ven. Hay que copiar en cada `post`.
- Quitar la baja de `Welcome`: elimina el síntoma, pero cualquier oyente que se dé de baja volverá a provocarlo.

Solución del ejercicio 9.3 enunciado en la pág. 348

Un archivo nuevo, y una línea en `Main`.

StreakAchievement.java

```java
public class StreakAchievement implements ShotListener {
    private static final int GOAL = 5;

    private int streak;
    private boolean unlocked;

    @Override
    public void onEvent(ShotEvent event) {
        switch (event) {
            case Hit h -> {
                streak++;
                if (streak == GOAL && !unlocked) {
                    unlocked = true;
                    System.out.println("Logro: cinco aciertos seguidos");
                }
            }
            case Miss m -> streak = 0;
        }
    }
}
```

Main.java (con la suscripción)

```java
public class Main {
    public static void main(String[] args) {
        ShotBus bus = new ShotBus();
        Archer archer = new Archer(bus);
        bus.subscribe(event -> {
            if (event instanceof Miss) {
                System.out.println("Fallo");
            }
        });
        bus.subscribe(new StreakAchievement());

        boolean[] shots = {true, true, false, true, true, true, true,
                true, false, true, true, true, true, true};
        for (boolean hits : shots) {
            archer.shoot(hits);
        }
    }
}
```

Por qué funciona. Los disparos 1 y 2 son aciertos (racha 2); el tercero falla (el primer oyente escribe «Fallo» y el logro pone la racha a 0). Los disparos 4 a 8 son cinco aciertos: en el octavo la racha vale 5 y el logro se desbloquea. El noveno falla («Fallo») y los disparos 10 a 14 son otra racha de cinco: la racha vuelve a valer 5, pero `unlocked` ya es `true` y no se escribe nada. El `switch` sobre la interfaz sellada no lleva `default`: si alguien añade un tercer tipo de disparo, el compilador obliga a decidir qué hace el logro con él. Ni `Archer` ni `ShotBus` cambian: el logro es solo un oyente más, que se suscribe en `Main`.

Errores típicos:

- No poner la racha a cero en el fallo: cuenta aciertos totales, no seguidos.

- Olvidar el indicador `unlocked`: la segunda racha escribiría el logro otra vez.
- Escribir `streak >= GOAL` sin el indicador: el logro se repetiría con el sexto acierto y los siguientes.
- Añadir un `default` «por si acaso»: funciona, pero se pierde el aviso del compilador cuando aparece un evento nuevo.

Solución del ejercicio 9.4 enunciado en la pág. 350

Partes 1 a 3 en orden. Se repiten los archivos enteros (también los que solo cambian un poco), para que la solución esté completa. Los archivos que no aparecen (`Fighter`, las IA, los biomas, el mapa, `engine/save/`...) no cambian.

Parte 1: engine/event/

engine/event/GameEvent.java

```
package emberdeep.engine.event;

import emberdeep.engine.combat.damage.DamageType;
import java.util.List;

public sealed interface GameEvent {
    record DamageDealt(String attacker, String target,
                       boolean targetIsHero, int amount, DamageType type,
                       int targetHp, int targetMaxHp)
            implements GameEvent {}

    record DamageDodged(String attacker, String target,
                        boolean targetIsHero) implements GameEvent {}

    record FighterDefeated(String name, boolean isHero)
            implements GameEvent {}

    record Defended(String name, int shield, int defenseBonus)
            implements GameEvent {}

    record ItemUsed(String user, String target, String itemName,
                    int targetHp, int targetMaxHp) implements GameEvent {}

    record CombatEnded(boolean heroesWon, List<Integer> heroesHp)
            implements GameEvent {
        public CombatEnded {
            heroesHp = List.copyOf(heroesHp);
        }
    }

    record AchievementUnlocked(String name) implements GameEvent {}
}
```

engine/event/GameListener.java

```java
package emberdeep.engine.event;

@FunctionalInterface
public interface GameListener {
    void onEvent(GameEvent event);
}
```

engine/event/EventBus.java

```java
package emberdeep.engine.event;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

public class EventBus {
    private final List<GameListener> listeners = new ArrayList<>();

    public void subscribe(GameListener listener) {
        listeners.add(Objects.requireNonNull(listener));
    }

    public void unsubscribe(GameListener listener) {
        listeners.remove(listener);
    }

    // se recorre una copia: un oyente puede darse de baja o publicar
    public void publish(GameEvent event) {
        for (GameListener listener : List.copyOf(listeners)) {
            listener.onEvent(event);
        }
    }
}
```

Parte 2: el motor publica. `engine/combat/Combat.java`, en cuatro tramos seguidos dentro de la misma clase:

Combat.java (1/4: imports, campos y constructor)

```
package emberdeep.engine.combat;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.command.CommandHistory;
import emberdeep.engine.combat.damage.Damage;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.event.EventBus;
import emberdeep.engine.event.GameEvent;
import emberdeep.engine.event.GameEvent.CombatEnded;
import emberdeep.engine.event.GameEvent.DamageDealt;
import emberdeep.engine.event.GameEvent.DamageDodged;
import emberdeep.engine.event.GameEvent.FighterDefeated;
import java.util.List;

public class Combat {
    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final DamageFormula formula;
    private final DamagePipeline pipeline;
    private final EventBus bus;
    private final CommandHistory history = new CommandHistory();
    private int round = 1;
    private boolean ended;

    public Combat(List<Fighter> heroes, List<Fighter> enemies,
                  DamageFormula formula, DamagePipeline pipeline,
                  EventBus bus) {
        this.heroes = heroes;
        this.enemies = enemies;
        this.formula = formula;
        this.pipeline = pipeline;
        this.bus = bus;
    }
```

Combat.java (2/4: estado y ronda)

```
    public int round() {
        return round;
    }

    public boolean heroesWon() {
        return enemies.stream().noneMatch(Fighter::isAlive);
    }

    public boolean heroesLost() {
        return heroes.stream().noneMatch(Fighter::isAlive);
    }

    public boolean isOver() {
        return heroesWon() || heroesLost();
    }

    public void restoreRound(int round) {
        if (round < 1) {
            throw new IllegalArgumentException("La ronda empieza en 1");
        }
        this.round = round;
    }

    // cap. 9: los comandos publican por aquí
    public void publish(GameEvent event) {
        bus.publish(event);
    }
```

Combat.java (3/4: ejecutar y golpear)

```
    // los héroes ejecutan sus comandos por aquí
    public void perform(CombatCommand command) {
        history.run(command);
        checkEnd();
    }

    public boolean undo() {
        return history.undo();
    }

    public int damageOf(Fighter attacker, Fighter target) {
        return formula.damage(attacker.stats().attack(),
                              target.stats().defense());
    }

    public int deal(Fighter attacker, Fighter target, Damage damage) {
        int dealt = pipeline.apply(target, damage);
        target.takeDamage(dealt);
        boolean isHero = heroes.contains(target);
        if (dealt == 0) {
            publish(new DamageDodged(attacker.name(), target.name(),
                    isHero));
        } else {
            publish(new DamageDealt(attacker.name(), target.name(), isHero,
                    dealt, damage.type(),
                    target.hp(), target.stats().maxHp()));
        }
        if (!target.isAlive()) {
            publish(new FighterDefeated(target.name(), isHero));
        }
        return dealt;
    }
```

Combat.java (4/4: fase enemiga y fin del combate)

```
    public void enemyPhase() {
        for (Fighter enemy : enemies) {
            if (!enemy.isAlive()) {
                continue;
            }
            if (heroesLost()) {
                break;
            }
            enemy.act(this, heroes).execute();
        }
        checkEnd();
        round++;
        heroes.forEach(Fighter::tickBuffs);
        enemies.forEach(Fighter::tickBuffs);
        history.clear();
    }

    // publica el fin del combate una sola vez
    private void checkEnd() {
        if (ended || !isOver()) {
            return;
        }
        ended = true;
        publish(new CombatEnded(heroesWon(),
                heroes.stream().map(Fighter::hp).toList()));
    }
}
```

engine/combat/command/

CombatCommand.java

```
package emberdeep.engine.combat.command;

public interface CombatCommand {
    void execute();
    void undo();
}
```

CommandHistory.java

```
package emberdeep.engine.combat.command;

import java.util.ArrayDeque;
import java.util.Deque;

public class CommandHistory {
    private final Deque<CombatCommand> done = new ArrayDeque<>();

    public void run(CombatCommand command) {
        command.execute();
        done.push(command);
    }

    public boolean undo() {
        if (done.isEmpty()) {
            return false;
        }
        done.pop().undo();
        return true;
    }

    public int size() { return done.size(); }
    public void clear() { done.clear(); }
}
```

Attack.java

```java
package emberdeep.engine.combat.command;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.Damage;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.stats.StatSource;

public class Attack implements CombatCommand {
    private final Combat combat;
    private final Fighter attacker;
    private final Fighter target;
    private int hpBefore;
    private int shieldBefore;
    private StatSource statsBefore;

    public Attack(Combat combat, Fighter attacker, Fighter target) {
        this.combat = combat;
        this.attacker = attacker;
        this.target = target;
    }

    @Override
    public void execute() {
        hpBefore = target.hp();
        shieldBefore = target.shield();
        statsBefore = target.stats();
        int raw = combat.damageOf(attacker, target);
        combat.deal(attacker, target, new Damage(raw, DamageType.PHYSICAL));
    }

    @Override
    public void undo() {
        target.restore(hpBefore, shieldBefore, statsBefore);
    }
}
```

Defend.java

```java
package emberdeep.engine.combat.command;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.event.GameEvent.Defended;
import emberdeep.engine.stats.StatKind;
import emberdeep.engine.stats.StatSource;

public class Defend implements CombatCommand {
    private static final int SHIELD = 5;
    private static final int DEFENSE_BONUS = 2;

    private final Combat combat;
    private final Fighter user;
    private int hpBefore;
    private int shieldBefore;
    private StatSource statsBefore;

    public Defend(Combat combat, Fighter user) {
        this.combat = combat;
        this.user = user;
    }

    @Override
    public void execute() {
        hpBefore = user.hp();
        shieldBefore = user.shield();
        statsBefore = user.stats();
        user.addShield(SHIELD);
        user.addBuff(StatKind.DEFENSE, DEFENSE_BONUS, 1);
        combat.publish(new Defended(user.name(), SHIELD, DEFENSE_BONUS));
    }

    @Override
    public void undo() {
        user.restore(hpBefore, shieldBefore, statsBefore);
    }
}
```

UseSkill.java

```java
package emberdeep.engine.combat.command;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Skill;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.Damage;
import emberdeep.engine.stats.StatSource;

public class UseSkill implements CombatCommand {
    private final Combat combat;
    private final Fighter user;
    private final Skill skill;
    private final Fighter target;
    private int hpBefore;
    private int shieldBefore;
    private StatSource statsBefore;

    public UseSkill(Combat combat, Fighter user,
                    Skill skill, Fighter target) {
        this.combat = combat;
        this.user = user;
        this.skill = skill;
        this.target = target;
    }

    @Override
    public void execute() {
        hpBefore = target.hp();
        shieldBefore = target.shield();
        statsBefore = target.stats();
        combat.deal(user, target, new Damage(skill.power(), skill.type()));
    }

    @Override
    public void undo() {
        target.restore(hpBefore, shieldBefore, statsBefore);
    }
}
```

UseItem.java (1/2: datos)

```
package emberdeep.engine.combat.command;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.Damage;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.event.GameEvent.ItemUsed;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.Item;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.stats.StatSource;

public class UseItem implements CombatCommand {
    private final GameSession session;
    private final Fighter user;
    private final Fighter target;
    private final Item item;
    private Inventory inventoryBefore;
    private int hpBefore;
    private int shieldBefore;
    private StatSource statsBefore;

    public UseItem(GameSession session, Fighter user,
                   Fighter target, Item item) {
        this.session = session;
        this.user = user;
        this.target = target;
        this.item = item;
    }
```

UseItem.java (2/2: execute y undo)

```
    @Override
    public void execute() {
        inventoryBefore = session.inventory();
        hpBefore = target.hp();
        shieldBefore = target.shield();
        statsBefore = target.stats();
        session.replaceInventory(inventoryBefore.without(item));
        Combat combat = session.combat();
        switch (item.kind()) {
            case HEALING -> {
                target.heal(item.power());
                combat.publish(new ItemUsed(user.name(), target.name(),
                        item.name(), target.hp(), target.stats().maxHp()));
            }
            case BOMB -> combat.deal(user, target,
                    new Damage(item.power(), DamageType.FIRE));
        }
    }

    @Override
    public void undo() {
        session.replaceInventory(inventoryBefore);
        target.restore(hpBefore, shieldBefore, statsBefore);
    }
}
```

engine/session/GameSession.java, en cuatro tramos seguidos dentro de la misma clase (solo cambian el `import`, el campo `events`, la creación de `Combat` y el acceso `events()`):

GameSession.java (1/4: imports y campos)

```java
package emberdeep.engine.session;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.event.EventBus;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.save.FighterState;
import emberdeep.engine.save.Snapshot;
import emberdeep.engine.stats.ArmorBonus;
import emberdeep.engine.stats.BaseStats;
import emberdeep.engine.stats.StatSource;
import emberdeep.engine.stats.WeaponBonus;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.List;
import java.util.stream.Stream;

public class GameSession {
    private static final int SWORD_ATTACK = 2;
    private static final int ARMOR_VALUE = 1;

    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final Combat combat;
    private final EventBus events = new EventBus();
    private Inventory inventory;
    private int floor = 1;
    private int x;
    private int y;
```

GameSession.java (2/4: constructor y equipo)

```
    public GameSession(List<Hero> heroes, BiomeFactory biome,
                       List<EnemyKind> kinds, DamageFormula formula,
                       DamagePipeline pipeline) {
        this.heroes = heroes.stream()
                .map(hero -> Fighter.ofHero(hero, gearOf(hero)))
                .toList();
        this.enemies = kinds.stream()
                .map(biome::enemy)
                .map(Fighter::ofEnemy)
                .toList();
        this.combat = new Combat(this.heroes, this.enemies,
                formula, pipeline, events);
        this.inventory = new Inventory(List.of())
                .with(biome.item(ItemKind.HEALING))
                .with(biome.item(ItemKind.HEALING))
                .with(biome.item(ItemKind.BOMB));
    }

    private static StatSource gearOf(Hero hero) {
        return new ArmorBonus(
                new WeaponBonus(new BaseStats(hero.stats()), SWORD_ATTACK),
                ARMOR_VALUE);
    }
```

GameSession.java (3/4: accesos y posición)

```
    public List<Fighter> heroes() { return heroes; }
    public List<Fighter> enemies() { return enemies; }
    public Combat combat() { return combat; }
    public EventBus events() { return events; }
    public int round() { return combat.round(); }
    public Inventory inventory() { return inventory; }
    public int floor() { return floor; }
    public int x() { return x; }
    public int y() { return y; }

    public void replaceInventory(Inventory next) {
        this.inventory = next;
    }

    public void setLocation(int floor, int x, int y) {
        this.floor = floor;
        this.x = x;
        this.y = y;
    }
```

GameSession.java (4/4: instantánea y restauración)

```
    private List<Fighter> allFighters() {
        return Stream.concat(heroes.stream(), enemies.stream()).toList();
    }

    public Snapshot createSnapshot() {
        List<FighterState> states = allFighters().stream()
                .map(f -> new FighterState(f.hp(), f.shield(), f.stats()))
                .toList();
        return new Snapshot(floor, x, y, combat.round(), states, inventory);
    }

    public void restore(Snapshot snapshot) {
        List<Fighter> all = allFighters();
        if (snapshot.fighters().size() != all.size()) {
            throw new IllegalArgumentException(
                    "La instantánea es de otra partida");
        }
        for (int i = 0; i < all.size(); i++) {
            FighterState state = snapshot.fighters().get(i);
            all.get(i).restore(state.hp(), state.shield(), state.stats());
        }
        combat.restoreRound(snapshot.round());
        inventory = snapshot.inventory();
        floor = snapshot.floor();
        x = snapshot.x();
        y = snapshot.y();
    }
}
```

ui/listener/

ui/listener/CombatLog.java (1/2: imports y campos)

```
1  package emberdeep.ui.listener;
2
3  import emberdeep.engine.combat.damage.DamageType;
4  import emberdeep.engine.event.GameEvent;
5  import emberdeep.engine.event.GameEvent.AchievementUnlocked;
6  import emberdeep.engine.event.GameEvent.CombatEnded;
7  import emberdeep.engine.event.GameEvent.DamageDealt;
8  import emberdeep.engine.event.GameEvent.DamageDodged;
9  import emberdeep.engine.event.GameEvent.Defended;
10 import emberdeep.engine.event.GameEvent.FighterDefeated;
11 import emberdeep.engine.event.GameEvent.ItemUsed;
12 import emberdeep.engine.event.GameListener;
13 import java.io.PrintStream;
14
15 public class CombatLog implements GameListener {
16     private final PrintStream out;
17
18     public CombatLog(PrintStream out) {
19         this.out = out;
20     }
```

ui/listener/CombatLog.java (2/2: onEvent y tipo de daño)

```
    @Override
    public void onEvent(GameEvent event) {
        switch (event) {
            case DamageDealt e -> out.println(e.attacker() + " hace "
                    + e.amount() + " de daño " + kind(e.type())
                    + " a " + e.target() + ".");
            case DamageDodged e -> out.println("El golpe de "
                    + e.attacker() + " a " + e.target() + " no hace daño.");
    ↪
            case FighterDefeated e -> out.println(e.name()
                    + " queda fuera de combate.");
            case Defended e -> out.println(e.name() + " se cubre: +"
                    + e.shield() + " de escudo y +" + e.defenseBonus()
                    + " de defensa este turno.");
            case ItemUsed e -> out.println(e.user() + " usa "
                    + e.itemName() + " sobre " + e.target() + ".");
            case CombatEnded e -> out.println(e.heroesWon()
                    ? "¡Victoria!" : "Derrota...");
            case AchievementUnlocked e -> out.println(
                    "¡Logro desbloqueado: " + e.name() + "!");
        }
    }

    private static String kind(DamageType type) {
        return switch (type) {
            case PHYSICAL -> "físico";
            case FIRE -> "de fuego";
        };
    }
}
```

ui/listener/HudListener.java

```java
package emberdeep.ui.listener;

import emberdeep.engine.event.GameEvent;
import emberdeep.engine.event.GameEvent.AchievementUnlocked;
import emberdeep.engine.event.GameEvent.CombatEnded;
import emberdeep.engine.event.GameEvent.DamageDealt;
import emberdeep.engine.event.GameEvent.DamageDodged;
import emberdeep.engine.event.GameEvent.Defended;
import emberdeep.engine.event.GameEvent.FighterDefeated;
import emberdeep.engine.event.GameEvent.ItemUsed;
import emberdeep.engine.event.GameListener;
import java.io.PrintStream;

public class HudListener implements GameListener {
    private static final int WIDTH = 10;

    private final PrintStream out;

    public HudListener(PrintStream out) {
        this.out = out;
    }

    @Override
    public void onEvent(GameEvent event) {
        switch (event) {
            case DamageDealt e -> bar(e.target(), e.targetHp(),
                    e.targetMaxHp());
            case ItemUsed e -> bar(e.target(), e.targetHp(),
                    e.targetMaxHp());
            case DamageDodged e -> { }
            case FighterDefeated e -> { }
            case Defended e -> { }
            case CombatEnded e -> { }
            case AchievementUnlocked e -> { }
        }
    }

    // casillas llenas = hp * 10 / maxHp, redondeado hacia arriba
    private void bar(String name, int hp, int maxHp) {
        int filled = (hp * WIDTH + maxHp - 1) / maxHp;
        out.println("  " + name + " [" + "#".repeat(filled)
                + ".".repeat(WIDTH - filled) + "] " + hp + "/" + maxHp);
    }
}
```

ui/ConsoleMenu.java (sustituye a `ConsoleUi`, que se borra), en siete tramos seguidos dentro de la misma clase. Respecto a `ConsoleUi` cambia el nombre, desaparece el método `print`, el bucle ya no imprime los mensajes del motor ni el final, y `Defend` recibe el `Combat`:

ConsoleMenu.java (1/7: imports, campos y fórmula)

```
package emberdeep.ui;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Skill;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.Attack;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.command.Defend;
import emberdeep.engine.combat.command.UseItem;
import emberdeep.engine.combat.command.UseSkill;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.save.SnapshotStore;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.TileType;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.input.InputSource;
import java.io.PrintStream;
import java.util.List;
import java.util.stream.Collectors;

public class ConsoleMenu {
    private enum Turn { ACTED, UNDONE, RESTORED }

    private final InputSource input;
    private final PrintStream out;
    private final SnapshotStore checkpoints = new SnapshotStore();
    private int checkpointHero;     // a qué héroe le tocaba al guardar

    public ConsoleMenu(InputSource input, PrintStream out) {
        this.input = input;
        this.out = out;
    }

    public int askFormula() {
        out.println("Fórmula de daño:");
        out.println("  [1] plana: ataque - defensa");
        out.println("  [2] escalada: ataque x 100 / (100 + defensa)");
        return ask(2);
    }
```

ConsoleMenu.java (2/7: la partida)

```
    public void run(GameSession session) {
        Combat combat = session.combat();
        out.println("=== Emberdeep ===");
        while (!combat.isOver()) {
            out.println("--- Ronda " + session.round() + " ---");
            List<Fighter> acting = aliveHeroes(session);
            int i = 0;
            while (i < acting.size() && !combat.isOver()) {
                Turn turn = heroTurn(session, acting.get(i), i);
                switch (turn) {
                    case ACTED -> i++;
                    case UNDONE -> i = Math.max(0, i - 1);
                    case RESTORED -> {
                        acting = aliveHeroes(session);
                        i = checkpointHero;
                    }
                }
            }
            if (!combat.isOver()) {
                combat.enemyPhase();
            }
        }
    }

    private List<Fighter> aliveHeroes(GameSession session) {
        return session.heroes().stream().filter(Fighter::isAlive).toList();
    }
```

ConsoleMenu.java (3/7: el turno de un héroe)

```
    // ACTED: el héroe actúa; UNDONE: se deshace la acción anterior;
    // RESTORED: se vuelve a un punto de control
    private Turn heroTurn(GameSession session, Fighter hero, int index) {
        Combat combat = session.combat();
        while (true) {
            out.println("Héroes:   " + describe(session.heroes()));
            out.println("Enemigos: " + describe(session.enemies()));
            out.println("Objetos:  " + describeItems(session.inventory()));
            out.println("Turno de " + hero.name()
                    + ". [1] atacar  [2] defender  [3] objeto"
                    + "  [4] habilidad  [u] deshacer"
                    + "  [c] guardar  [r] volver");
            String key = input.next();
            if (key.equals("u")) {
                if (combat.undo()) {
                    out.println("Deshaces la última acción.");
                    return Turn.UNDONE;
                }
                out.println("No hay nada que deshacer en esta ronda.");
                continue;
            }
            if (key.equals("c")) {
                saveCheckpoint(session, index);
                continue;
            }
            if (key.equals("r")) {
                if (loadCheckpoint(session)) {
                    return Turn.RESTORED;
                }
                continue;
            }
            CombatCommand command = switch (key) {
                case "1" -> new Attack(combat, hero,
                        askTarget("Objetivo", session.enemies()));
                case "2" -> new Defend(combat, hero);
                case "3" -> chooseItem(session, hero);
                case "4" -> chooseSkill(session, hero);
                default -> {
                    out.println("Tecla no válida.");
                    yield null;
                }
            };
            if (command != null) {
                combat.perform(command);
                return Turn.ACTED;
            }
        }
    }
```

ConsoleMenu.java (4/7: guardar y volver)

```
    private void saveCheckpoint(GameSession session, int index) {
        checkpoints.push(session.createSnapshot());
        checkpointHero = index;
        out.println("Punto de control guardado ("
                + checkpoints.size() + ").");
    }

    // false si todavía no hay ninguno
    private boolean loadCheckpoint(GameSession session) {
        if (checkpoints.isEmpty()) {
            out.println("No hay puntos de control.");
            return false;
        }
        session.restore(checkpoints.last());
        out.println("Vuelves al último punto de control (ronda "
                + session.round() + ").");
        return true;
    }
```

ConsoleMenu.java (5/7: objetos, habilidades y objetivos)

```
    private CombatCommand chooseItem(GameSession session, Fighter hero) {
        List<Item> items = session.inventory().items();
        if (items.isEmpty()) {
            out.println("No te quedan objetos.");
            return null;
        }
        Item item = items.get(choose("Objeto",
                items.stream().map(Item::name).toList()));
        Fighter target = item.kind() == ItemKind.HEALING
                ? askTarget("Aliado", session.heroes())
                : askTarget("Objetivo", session.enemies());
        return new UseItem(session, hero, target, item);
    }

    private CombatCommand chooseSkill(GameSession session, Fighter hero) {
        List<Skill> skills = hero.skills();
        if (skills.isEmpty()) {
            out.println("No tienes habilidades.");
            return null;
        }
        Skill skill = skills.get(choose("Habilidad",
                skills.stream().map(Skill::name).toList()));
        Fighter target = askTarget("Objetivo", session.enemies());
        return new UseSkill(session.combat(), hero, skill, target);
    }

    private Fighter askTarget(String title, List<Fighter> group) {
        List<Fighter> alive = group.stream()
                .filter(Fighter::isAlive).toList();
        return alive.get(choose(title,
                alive.stream().map(Fighter::name).toList()));
    }
```

ConsoleMenu.java (6/7: menús, entrada y auxiliares)

```
    // Muestra "Título:  [1] a  [2] b" y devuelve la posición (desde 0)
    private int choose(String title, List<String> labels) {
        StringBuilder menu = new StringBuilder(title + ":");
        for (int i = 0; i < labels.size(); i++) {
            menu.append("  [").append(i + 1).append("] ")
                    .append(labels.get(i));
        }
        out.println(menu);
        return ask(labels.size()) - 1;
    }

    private int ask(int max) {
        while (true) {
            String line = input.next();
            try {
                int n = Integer.parseInt(line);
                if (n >= 1 && n <= max) {
                    return n;
                }
            } catch (NumberFormatException e) {
                // no es un número: se avisa más abajo
            }
            out.println("Escribe un número entre 1 y " + max + ".");
        }
    }

    private String describe(List<Fighter> group) {
        return group.stream()
                .map(f -> f.name() + " " + f.hp() + "/" + f.stats().maxHp())
                .collect(Collectors.joining(", "));
    }

    private String describeItems(Inventory inventory) {
        if (inventory.items().isEmpty()) {
            return "(ninguno)";
        }
        return inventory.items().stream()
                .map(Item::name)
                .collect(Collectors.joining(", "));
    }
```

ConsoleMenu.java (7/7: modo mapa y llave final)

```
    public boolean explore(GameMap map, TileTypes types) {
        while (true) {
            out.print(map.render());
            out.println("[w a s d] mover, [t] casillas, [q] salir");
            String key = input.next();
            switch (key) {
                case "w" -> step(map, 0, -1);
                case "a" -> step(map, -1, 0);
                case "s" -> step(map, 0, 1);
                case "d" -> step(map, 1, 0);
                case "t" -> showTiles(map, types);
                case "q" -> { return false; }
                default -> out.println("Tecla no válida.");
            }
            if (map.onStairs()) {
                out.println("Bajas por la escalera.");
                return true;
            }
        }
    }

    private void step(GameMap map, int dx, int dy) {
        if (!map.move(dx, dy)) {
            out.println("No puedes pasar.");
        }
    }

    private void showTiles(GameMap map, TileTypes types) {
        TileType corner = map.tileAt(0, 0);
        TileType opposite = map.tileAt(map.width() - 1, map.height() - 1);
        out.println("Casillas: " + map.width() * map.height()
                + ", TileType creados: " + types.createdCount());
        out.println("Esquinas opuestas, mismo objeto: "
                + (corner == opposite));
    }
}
```

app/Main.java, en cuatro tramos seguidos dentro de la misma clase. Las líneas marcadas «parte 3» (el `import` del tracker y su suscripción) las añades en la parte 3; hasta entonces el `Main` funciona sin ellas:

Main.java (1/4: imports)

```
package emberdeep.app;

import emberdeep.engine.achievement.AchievementTracker;   // parte 3
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.event.EventBus;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.Dungeon;
import emberdeep.engine.world.DungeonBuilder;
import emberdeep.engine.world.Floor;
import emberdeep.engine.world.biome.BiomeFactory;
import emberdeep.engine.world.biome.CavernBiome;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.MapData;
import emberdeep.engine.world.map.MapParser;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.ConsoleMenu;
import emberdeep.ui.input.ConsoleInput;
import emberdeep.ui.input.InputSource;
import emberdeep.ui.input.ScriptInput;
import emberdeep.ui.listener.CombatLog;
import emberdeep.ui.listener.HudListener;
import java.io.IOException;
import java.nio.file.Path;
import java.util.Arrays;
import java.util.List;
import java.util.Random;
import java.util.Scanner;
```

Main.java (2/4: arranque y entrada)

```
public class Main {
    private static final Skill POWER_STRIKE =
            new Skill("Golpe poderoso", 12, DamageType.PHYSICAL);
    private static final Skill FLARE =
            new Skill("Llamarada", 10, DamageType.FIRE);

    private final GameConfig config;

    private Main(GameConfig config) {
        this.config = config;
    }

    public static void main(String[] args) throws IOException {
        GameConfig config = new GameConfig(42L, 3, new FlatFormula());
        new Main(config).play(args);
    }

    private void play(String[] args) throws IOException {
        String script = scriptOf(args);
        String[] options = optionsOf(args);
        int floorNumber = chooseFloor(options);
        InputSource input = script == null
                ? new ConsoleInput(new Scanner(System.in), System.out)
                : new ScriptInput(Path.of(script), System.out);
        ConsoleMenu menu = new ConsoleMenu(input, System.out);
        Random random = new Random(config.seed());

        BiomeFactory crypt = new CryptBiome(random);
        BiomeFactory cavern = new CavernBiome(random);
        List<EnemyKind> cryptKinds = List.of(
                EnemyKind.SKELETON, EnemyKind.ASH_RAT, EnemyKind.ZOMBIE);
        List<EnemyKind> cavernKinds = List.of(
                EnemyKind.GOBLIN, EnemyKind.BAT, EnemyKind.SPIDER);
```

Main.java (3/4: mazmorra, partida y modo mapa)

```
        Dungeon dungeon = new DungeonBuilder()
                .floor().biome(crypt).enemies(cryptKinds)
                        .map(MapData.floor(1)).done()
                .floor().biome(cavern).enemies(cavernKinds)
                        .map(MapData.floor(2)).done()
                .floor().biome(crypt).enemies(cryptKinds)
                        .map(MapData.floor(3)).done()
                .build();

        if (options.length > 1 && options[1].equals("map")) {
            explore(menu, dungeon, floorNumber);
            return;
        }
        DamageFormula formula = chooseFormula(menu, options,
                script != null);
        Floor floor = dungeon.floors().get(floorNumber - 1);
        GameSession session = new GameSession(heroes(), floor.biome(),
                floor.enemies(), formula, new DamagePipeline(random, 0.0));
        GameMap entrance = new MapParser(new TileTypes())
                .parse(floor.map());
        session.setLocation(floorNumber,
                entrance.partyX(), entrance.partyY());
        EventBus events = session.events();
        events.subscribe(new CombatLog(System.out));
        events.subscribe(new HudListener(System.out));
        // parte 3: el tracker va el último
        events.subscribe(new AchievementTracker(events));
        menu.run(session);
    }

    private void explore(ConsoleMenu menu, Dungeon dungeon,
                         int firstFloor) {
        TileTypes types = new TileTypes();
        MapParser parser = new MapParser(types);
        int number = firstFloor;
        while (true) {
            Floor floor = dungeon.floors().get(number - 1);
            boolean descended =
                    menu.explore(parser.parse(floor.map()), types);
            if (!descended || number == dungeon.floors().size()) {
                return;
            }
            number++;
        }
    }
```

Main.java (4/4: argumentos, héroes y llave final)

```
    // El guion es el argumento que acaba en .txt (parte 3)
    private static String scriptOf(String[] args) {
        return Arrays.stream(args)
                .filter(arg -> arg.endsWith(".txt"))
                .findFirst().orElse(null);
    }

    private static String[] optionsOf(String[] args) {
        return Arrays.stream(args)
                .filter(arg -> !arg.endsWith(".txt"))
                .toArray(String[]::new);
    }

    private DamageFormula chooseFormula(ConsoleMenu menu, String[] options,
                                        boolean scripted) {
        boolean useDefault = scripted
                || (options.length > 1 && options[1].equals("default"));
        if (useDefault) {
            return config.defaultFormula();
        }
        return menu.askFormula() == 1
                ? new FlatFormula() : new ScaledFormula();
    }

    private int chooseFloor(String[] options) {
        int floor = options.length > 0 ? Integer.parseInt(options[0]) : 1;
        if (floor < 1 || floor > config.floors()) {
            throw new IllegalArgumentException(
                    "La planta debe estar entre 1 y " + config.floors());
        }
        return floor;
    }

    private List<Hero> heroes() {
        return List.of(
                new HeroBuilder().name("Aria").stats(new Stats(30, 8, 3))
                        .skill(POWER_STRIKE).skill(FLARE).build(),
                new HeroBuilder().name("Borin").stats(new Stats(40, 6, 5))
                        .skill(POWER_STRIKE).skill(FLARE).build(),
                new HeroBuilder().name("Cael").stats(new Stats(24, 9, 2))
                        .skill(POWER_STRIKE).skill(FLARE).build());
    }
}
```

Parte 3: engine/achievement/AchievementTracker.java, en dos tramos seguidos dentro de la misma clase:

AchievementTracker.java (1/2: imports, campos y constructor)

```java
package emberdeep.engine.achievement;

import emberdeep.engine.event.EventBus;
import emberdeep.engine.event.GameEvent;
import emberdeep.engine.event.GameEvent.AchievementUnlocked;
import emberdeep.engine.event.GameEvent.CombatEnded;
import emberdeep.engine.event.GameEvent.DamageDealt;
import emberdeep.engine.event.GameEvent.DamageDodged;
import emberdeep.engine.event.GameEvent.Defended;
import emberdeep.engine.event.GameEvent.FighterDefeated;
import emberdeep.engine.event.GameEvent.ItemUsed;
import emberdeep.engine.event.GameListener;
import java.util.HashSet;
import java.util.Set;

public class AchievementTracker implements GameListener {
    private static final int DODGES_NEEDED = 5;

    private final EventBus bus;
    private final Set<String> unlocked = new HashSet<>();
    private int heroDodges;

    public AchievementTracker(EventBus bus) {
        this.bus = bus;
    }
```

AchievementTracker.java (2/2: onEvent y unlock)

```
    @Override
    public void onEvent(GameEvent event) {
        switch (event) {
            case DamageDealt e -> {
                if (!e.targetIsHero()) {
                    unlock("Primera sangre");
                }
            }
            case DamageDodged e -> {
                if (e.targetIsHero()) {
                    heroDodges++;
                    if (heroDodges == DODGES_NEEDED) {
                        unlock("Intocable");
                    }
                }
            }
            case CombatEnded e -> {
                if (e.heroesWon() && e.heroesHp().contains(1)) {
                    unlock("Último aliento");
                }
            }
            case FighterDefeated e -> { }
            case Defended e -> { }
            case ItemUsed e -> { }
            case AchievementUnlocked e -> { }
        }
    }

    private void unlock(String name) {
        if (unlocked.add(name)) {
            bus.publish(new AchievementUnlocked(name));
        }
    }
}
```

Por qué funciona (parte 1). `Check09a` suscribe una lambda, publica un `DamageDodged` (llega: el `toString` de un `record` lista sus campos), la da de baja con la *misma referencia* (`printer`) y publica un `FighterDefeated`, que ya no llega. El bus no sabe nada de las clases del juego: solo de `GameEvent`.

Por qué funciona (parte 2). Todo golpe pasa por `Combat.deal`, así que un solo sitio publica `DamageDealt`, `DamageDodged` y `FighterDefeated`, venga el golpe de un ataque, de una habilidad o de una bomba. `Attack`, `UseSkill` y la bomba de `UseItem` solo cambian de qué atacante cuentan; `Defend` y la poción publican su propio hecho. `CombatEnded` sale de `checkEnd()`, que se llama al terminar `perform` (cae el último enemigo) y al terminar la fase enemiga (caen los héroes) y se publica una sola vez gracias al indicador `ended`. En el guion: la bomba deja al Zombi en 12 de 22, y la barra tiene $\lceil 120/22 \rceil = 6$ casillas llenas; Borin deja al Esqueleto en 12 de 18 ($\lceil 120/18 \rceil = 7$); Cael se cubre y el Esqueleto (que elige al azar con la semilla 42 y cae en Cael) y la Rata (que va a por el héroe con menos vida, Cael) se quedan en cero daño, absorbidos por el escudo; el Zombi, que va a por el primero, hace 1 a Aria. `ConsoleMenu` sigue pintando las líneas «Héroes:», «Enemigos:» y «Objetos:» y el menú, porque eso es preguntar, no anunciar. El `grep` de `System.out` en `engine/` no

encuentra nada: el motor ya no escribe en ningún sitio.

Por qué funciona (parte 3). En `Check09c` el héroe tiene 7 de vida y defensa 0 (con la armadura de 1 del equipo). Con `Defend`, el golpe del Esqueleto pasa por la fórmula con la defensa subida a 2: $7 - 2 = 5$, y el escudo de 5 absorbe justo esos 5, así que el daño final es 0 y se publica un `DamageDodged`. Al quinto, el tracker cuenta 5 y publica «Intocable». En la sexta ronda no hay defensa ni escudo: el golpe de 12 de la habilidad deja al Esqueleto en 6 (publica `DamageDealt` con `targetIsHero` falso, y sale «Primera sangre») y el golpe del Esqueleto pasa la fórmula con $7 - 0 = 7$ y la armadura lo deja en 6: el héroe se queda con 1 de vida. En la séptima ronda, el ataque del héroe ($10 - 2 = 8$) mata al Esqueleto, `perform` llama a `checkEnd()` y el `CombatEnded` lleva `heroesHp` $= [1]$: sale «Último aliento». Fíjate en el orden de las líneas: el log escribe cada evento antes de que el tracker publique su logro, porque el tracker está suscrito el último.

Errores típicos:

- Poner el tracker antes que el log: los logros saldrían antes de la frase del golpe que los provoca.
- Dejar que `Combat` imprima algo «solo para depurar»: rompe la regla de que `engine` no usa `System.out`.
- Publicar `CombatEnded` en `enemyPhase()` solamente: si el último enemigo cae a manos de un héroe, el combate termina en la fase de los héroes y el evento no llegaría nunca (y «Último aliento» tampoco).
- Olvidar el `boolean ended`: el evento saldría dos veces si algo vuelve a llamar a `checkEnd()` con el combate ya terminado.
- Meter un `Fighter` en un evento: el oyente vería el estado de ahora, no el del momento del golpe, y podría modificarlo.
- Usar `default` en los `switch` de los oyentes: se pierde el aviso del compilador cuando se añade un evento (los capítulos 11 y 12 lo harán).
- Calcular las casillas de la barra con $hp \cdot 10/maxHp$ sin redondear hacia arriba: un héroe con 1 de vida de 30 dibujaría una barra vacía.

Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo 10: `engine/event/` con `GameEvent` (siete records), `GameListener` y `EventBus`; `engine/achievement/` con `AchievementTracker`; `ui/listener/` con `CombatLog` y `HudListener`; `ui/ConsoleMenu` en lugar de `ConsoleUi`; el motor publica y no devuelve cadenas; y `Main` suscribe log, HUD y tracker, en ese orden. Borra `app/Check09a.java`, `app/Check09c.java` y `scripts/check09.txt`.

CAPÍTULO 10

Bridge: vistas y salidas que crecen por separado

Objetivos del capítulo

- Reconocer el olor que pide un Bridge: una clase por cada combinación de dos cosas que cambian por separado (`AnsiHud`, `PlainHud`, `AnsiLog`, `PlainLog`...).
- Aplicar Bridge: una abstracción (`View`) que guarda por constructor un implementador (`Output`), de modo que cada lado crece sin tocar al otro.
- Diseñar la interfaz del implementador con operaciones de bajo nivel y explicar por qué.
- Escribir códigos de escape ANSI a mano para pintar de color en una consola, sin librerías.
- Distinguir Bridge de Strategy: dos jerarquías que crecen a la vez frente a un algoritmo que se elige.
- Dar el hito H11: tres vistas y tres salidas unidas por un puente, nueve combinaciones con seis clases.

En el capítulo anterior el motor dejó de hablar y empezó a anunciar. Ahora la pantalla de Emberdeep tiene dos oyentes que escriben en la consola: `CombatLog` cuenta lo que pasa y `HudListener` dibuja barras de vida. Funciona, pero los dos escriben directamente en un `PrintStream`, y eso va a dejar de bastar. Quieres color en la terminal, texto limpio cuando juegas con un guion o guardas la salida en un fichero, y una forma de recoger la salida en memoria para comparar dos partidas. Cada una de esas cosas afecta a *todo* lo que se muestra. El patrón que lo ordena es Bridge: separar «qué se muestra» de «dónde y cómo se escribe» para que cada lado crezca sin multiplicar las clases del otro. En este capítulo lo construyes, aprendes los códigos ANSI que pintan el color y lo comparas con Strategy, con el que se confunde.

10.1 El olor que pide Bridge: una clase por cada combinación

Mira lo que tienes. Dos oyentes (la vista del log y la vista del HUD) y una única forma de escribir (el `PrintStream`). Te piden tres formas:

1. **Color ANSI** en la terminal: las barras de vida en verde o en rojo según queda poca vida.
2. **Texto plano**, sin ningún código de color, para el modo guion y para redirigir a un fichero.
3. **Un búfer en memoria**, para poder comparar lo que se habría mostrado en dos partidas iguales sin leer la pantalla.

La salida más directa es una clase por cada pareja. Sería algo así (el código es hipotético; no existe en el proyecto):

Hipotético: una clase por combinación

```
public class AnsiHud implements GameListener {
    private final PrintStream out;
    // ... constructor

    private void bar(String name, int hp, int maxHp) {
        int filled = (hp * 10 + maxHp - 1) / maxHp;
        out.println("\u001B[32m  " + name + " ["
                + "#".repeat(filled) + ".".repeat(10 - filled)
                + "] " + hp + "/" + maxHp + "\u001B[0m");
    }
}

public class PlainHud implements GameListener {
    private final PrintStream out;
    // ... constructor

    private void bar(String name, int hp, int maxHp) {
        int filled = (hp * 10 + maxHp - 1) / maxHp;     // la misma cuenta
        out.println("  " + name + " ["
                + "#".repeat(filled) + ".".repeat(10 - filled)
                + "] " + hp + "/" + maxHp);
    }
}
// y además: BufferHud, AnsiLog, PlainLog, BufferLog...
```

Funciona, y tiene tres problemas que crecen a la vez:

1. **Las clases se multiplican.** Con tres vistas (log, HUD y mapa) y tres salidas hacen falta $3 \times 3 = 9$ clases. Si añades una cuarta salida, no escribes una clase: escribes tres. Si añades una cuarta vista, escribes otras tres.
2. **La misma lógica se repite.** La cuenta de las casillas de la barra está copiada en cada clase del HUD. Cuando cambie (por ejemplo, barras de 20 casillas), habrá que acordarse de tres sitios, y nadie avisa si te olvidas de uno.
3. **Dos decisiones están pegadas en el mismo sitio.** «Qué es una barra de vida» es una decisión de diseño de la pantalla; «los códigos de color» son un detalle de la terminal. Cambiar una obliga a releer la otra.

Vistas	Salidas	Clases con una por pareja	Clases con Bridge
3	3	9	3 + 3
4	3	12	4 + 3
4	4	16	4 + 4
10	10	100	10 + 10

(Con Bridge se añaden además dos piezas fijas, la base de cada lado, que no crecen.) Este es el olor: **dos cosas que cambian por separado y que, por herencia, obligan a una clase por cada pareja**. Bridge lo cura repartiendo cada cosa en su propia jerarquía y uniéndolas por composición. Su intención en el catálogo es desacoplar una abstracción de su implementación para que las dos puedan variar de forma independiente [9]. Head First lo trata entre los patrones de su apéndice [7], y el catálogo de Refactoring.Guru lo explica con el ejemplo de unas

formas geométricas y sus colores: dividir una clase grande en dos jerarquías, la abstracción y la implementación, que se desarrollan por separado [24]. La idea cabe en una frase: la vista *tiene* una salida en lugar de *ser* una salida concreta.

10.2 Piezas del patrón Bridge

Las palabras «abstracción» e «implementación» del catálogo no significan aquí «interfaz» y «clase» de Java. La *abstracción* es lo que el resto del programa usa y entiende (en Emberdeep, lo que se muestra: el log, el HUD, el mapa). La *implementación* es el lado que hace el trabajo de bajo nivel (cómo y dónde se escribe). Lo que las une es una referencia: la abstracción guarda un implementador y le delega.

Rol	Clase en el ejemplo	Propósito	Paquete
Abstracción (clase abstracta)	`View`	Guarda el `Output` que recibe y ofrece a las vistas escribir una línea	`ui.view`
Abstracciones refinadas (clases)	`HudView`, `LogView`, `MapView`	Deciden qué se muestra: barras de vida, frases del combate, el mapa	`ui.view`
Implementador (interfaz)	`Output`	Dos operaciones de bajo nivel: `write` y `writeStyled`	`ui.output`
Implementadores concretos (clases)	`AnsiOutput`, `PlainOutput`, `BufferOutput`	Deciden cómo y dónde se escribe: color ANSI, texto sin color, memoria	`ui.output`
Cliente (clase)	`Main`	Monta cada pareja vista-salida	`app`

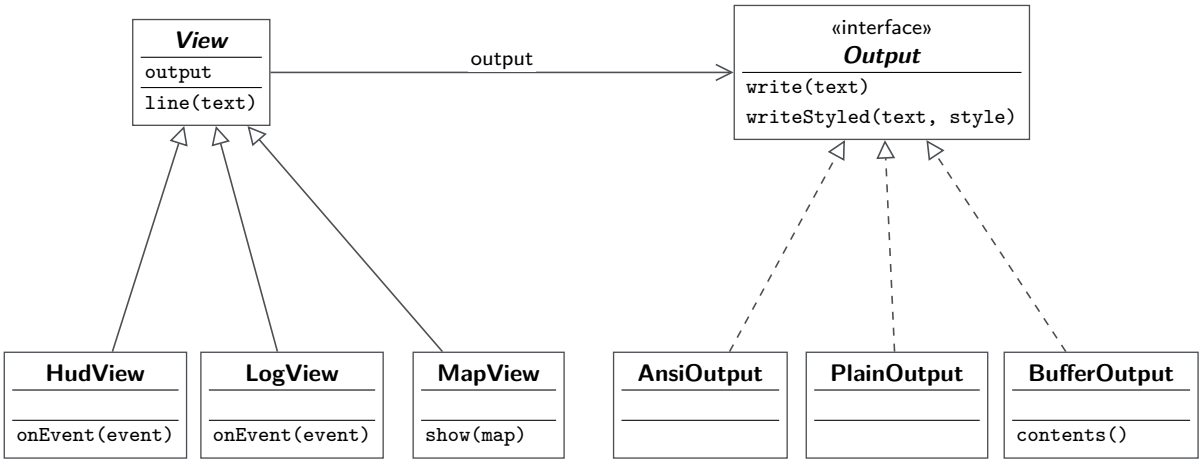


Diagrama UML del patrón Bridge: dos jerarquías, una de vistas y otra de salidas, unidas por la referencia que guarda `View`.

En UML, la unión entre las dos jerarquías se dibuja como una asociación, la flecha abierta de `View` a `Output` [6]. Es la única flecha que cruza de un lado al otro, y es el «puente». Fíjate en lo

que *no* hay: ninguna vista nombra una salida concreta, y ninguna salida sabe que existen las vistas. Las dos piezas se encuentran solo en el `Main`, que decide qué salida recibe cada vista.

10.3 El ejemplo de Bridge, pieza a pieza: `HudView` y `Output`

El estilo. Las vistas necesitan decir «esto va en rojo» sin saber qué es el rojo. Para eso hay un enumerado cerrado con tres valores, que son intenciones y no colores:

ui/output/Style.java

```
package emberdeep.ui.output;

public enum Style {
    NORMAL, DANGER, GOOD
}
```

`NORMAL` es «sin énfasis», `DANGER` avisa de algo malo (poca vida, un golpe a un héroe) y `GOOD` de algo bueno (un logro, una victoria). Cada salida traduce la intención como pueda: la de ANSI la pinta, la de texto plano la ignora.

El implementador. La interfaz es deliberadamente pequeña y de bajo nivel:

ui/output/Output.java

```
package emberdeep.ui.output;

public interface Output {
    // escribe el texto tal cual, sin añadir salto de línea
    void write(String text);

    // igual, pero con el énfasis que pide el estilo
    void writeStyled(String text, Style style);
}
```

Dos decisiones merecen un comentario. La primera: las operaciones son *primitivas*. No hay un `drawHealthBar`: eso sería llevar a la salida el trabajo de la vista, y cada implementador tendría que repetirlo. Un buen implementador ofrece pocas operaciones sencillas con las que cualquier abstracción pueda construir lo que necesite. La segunda: `write` no añade el salto de línea. Así una vista puede escribir una línea con partes de estilos distintos (el mapa lo hará: un color por casilla), y es la vista quien decide dónde acaba la línea.

Dos implementadores sencillos. La salida de texto plano escribe en un `PrintStream` e ignora el estilo; la salida de búfer acumula en memoria y devuelve lo acumulado:

ui/output/PlainOutput.java

```
package emberdeep.ui.output;

import java.io.PrintStream;

public class PlainOutput implements Output {
    private final PrintStream out;

    public PlainOutput(PrintStream out) {
```

```
        this.out = out;
    }

    @Override
    public void write(String text) {
        out.print(text);
    }

    @Override
    public void writeStyled(String text, Style style) {
        write(text);            // sin color: el estilo se ignora
    }
}
```

ui/output/BufferOutput.java

```
package emberdeep.ui.output;

public class BufferOutput implements Output {
    private final StringBuilder text = new StringBuilder();

    @Override
    public void write(String part) {
        text.append(part);
    }

    @Override
    public void writeStyled(String part, Style style) {
        text.append(part);      // guarda el texto, no el estilo
    }

    public String contents() {
        return text.toString();
    }
}
```

`BufferOutput` es el tercer implementador y el que más enseña: no es una «pantalla», y aun así encaja, porque el implementador es cualquier destino de escritura. Su método `contents()` no está en `Output`: lo usa quien conoce el búfer concreto (una comprobación, o más adelante una prueba), no las vistas.

La abstracción. `View` guarda el `Output` y ofrece dos ayudas para escribir líneas completas. Es abstracta porque no tiene sentido crear una vista «en blanco»:

ui/view/View.java

```
package emberdeep.ui.view;

import emberdeep.ui.output.Output;
import emberdeep.ui.output.Style;
import java.util.Objects;

public abstract class View {
    protected final Output output;

```

```
    protected View(Output output) {
        this.output = Objects.requireNonNull(output);
    }

    // una línea sin énfasis
    protected void line(String text) {
        output.write(text + "\n");
    }

    // una línea con énfasis; el salto de línea queda fuera del color
    protected void line(String text, Style style) {
        output.writeStyled(text, style);
        output.write("\n");
    }
}
```

Este es el puente: el campo `output` de tipo `Output` (la interfaz, nunca una clase concreta), que llega por el constructor. El salto de línea va fuera del texto con estilo a propósito: así un código de color se abre y se cierra *antes* del salto, y el color no se queda activo al final de la línea. Y es siempre `"\n"`, no el separador del sistema, para que dos partidas den exactamente el mismo texto en cualquier sistema operativo.

Una abstracción refinada. `HudView` es el `HudListener` del capítulo anterior, con una diferencia: ya no sabe escribir, solo sabe qué escribir. Reutiliza el `switch` sin `default` y la cuenta de las casillas, y decide además un estilo según la vida que queda:

ui/view/HudView.java (1/2: onEvent)

```
package emberdeep.ui.view;

import emberdeep.engine.event.GameEvent;
import emberdeep.engine.event.GameEvent.AchievementUnlocked;
import emberdeep.engine.event.GameEvent.CombatEnded;
import emberdeep.engine.event.GameEvent.DamageDealt;
import emberdeep.engine.event.GameEvent.DamageDodged;
import emberdeep.engine.event.GameEvent.Defended;
import emberdeep.engine.event.GameEvent.FighterDefeated;
import emberdeep.engine.event.GameEvent.ItemUsed;
import emberdeep.engine.event.GameListener;
import emberdeep.ui.output.Output;
import emberdeep.ui.output.Style;

public class HudView extends View implements GameListener {
    private static final int WIDTH = 10;

    public HudView(Output output) {
        super(output);
    }

    @Override
    public void onEvent(GameEvent event) {
        switch (event) {
            case DamageDealt e -> bar(e.target(), e.targetHp(),
                    e.targetMaxHp());
            case ItemUsed e -> bar(e.target(), e.targetHp(),
                    e.targetMaxHp());
```

```
            case DamageDodged e -> { }
            case FighterDefeated e -> { }
            case Defended e -> { }
            case CombatEnded e -> { }
            case AchievementUnlocked e -> { }
        }
    }
```

ui/view/HudView.java (2/2: la barra y su estilo)

```
    // casillas llenas = hp * 10 / maxHp, redondeado hacia arriba
    private void bar(String name, int hp, int maxHp) {
        int filled = (hp * WIDTH + maxHp - 1) / maxHp;
        line("  " + name + " [" + "#".repeat(filled)
                + ".".repeat(WIDTH - filled) + "] " + hp + "/" + maxHp,
                styleOf(hp, maxHp));
    }

    // rojo con un cuarto de vida o menos, verde con más de la mitad
    private static Style styleOf(int hp, int maxHp) {
        if (hp * 4 <= maxHp) {
            return Style.DANGER;
        }
        return hp * 2 > maxHp ? Style.GOOD : Style.NORMAL;
    }
}
```

La clase implementa dos cosas a la vez, y conviene no mezclarlas: `extends View` es el lado del puente (de dónde saca dónde escribir) e `implements GameListener` es el lado de Observer (de dónde saca qué contar). Los dos patrones del capítulo 9 y de este trabajan juntos sin pisarse.

Un ejemplo con números. Aria lanza la bomba de ceniza al Zombi, que queda en 12 de 22. El motor publica un `DamageDealt` con `targetHp` 12 y `targetMaxHp` 22, y `HudView` calcula $\lceil 120/22 \rceil = 6$ casillas llenas. Como $12 \cdot 4 = 48$ es mayor que 22, no está en peligro, y como $12 \cdot 2 = 24$ es mayor que 22, el estilo es `GOOD`. La misma vista, con la salida que reciba, escribe lo siguiente (en la segunda línea, `^[` representa el carácter ESC):

Tres salidas, el mismo evento

```
PlainOutput:    "  Zombi [######....] 12/22\n"
AnsiOutput:     "^[[32m  Zombi [######....] 12/22^[[0m\n"
BufferOutput:   contents() == "  Zombi [######....] 12/22\n"
```

`HudView` no ha cambiado entre las tres líneas: lo que cambia es el objeto que se le pasó al construirla. Montar las nueve combinaciones de tres vistas y tres salidas no requiere escribir una clase más; es solo cuestión de qué se pasa en cada `new`:

Montaje (ejemplo)

```
Output color = new AnsiOutput(System.out);
Output plain = new PlainOutput(System.out);
Output memory = new BufferOutput();

events.subscribe(new HudView(color));    // HUD en color
```

```
6 events.subscribe(new LogView(plain));    // log sin color
7 events.subscribe(new HudView(memory));   // otro HUD, esta vez en memoria
```

10.4 Si ya lo conoces: lo nuevo en Bridge es separar vistas y salidas

Si ya conocías Bridge, lo nuevo aquí es:

- **Los dos ejes no son «formas y colores».** El ejemplo de los manuales mezcla dos adjetivos de un mismo objeto. Aquí los ejes son dos *responsabilidades*: qué se cuenta (una vista) y dónde se escribe (una salida). Es el mismo patrón, pero se reconoce mejor: si cambiar «cómo se escribe» te obliga a tocar «qué se cuenta», hay un puente que falta.
- **La interfaz del implementador es de bajo nivel.** El patrón no te dice cómo de grande debe ser `Output`; el diseño sí. Dos operaciones (`write` y `writeStyled`) bastan para construir una barra, una frase o un mapa. Si le pidieras `drawHud`, la salida acabaría repitiendo la lógica de las vistas.
- **Se decide al diseñar, no después.** El catálogo lo distingue de Adapter: Adapter hace trabajar juntas clases que ya existían y no encajan, y suele aplicarse cuando el sistema ya está hecho; Bridge se usa desde el principio de un diseño para que abstracción e implementación varíen por separado [9]. Aquí nace en el momento en que aparece el segundo eje.
- **Un implementador no tiene por qué ser un dispositivo.** `BufferOutput` no pinta nada: guarda. Es un implementador válido porque cumple el contrato, y el que más facilita comprobar cosas sin leer una pantalla.
- **Se parece a Strategy en el diagrama, y no es lo mismo.** Lo separamos en la sección 10.6.

10.5 Bridge y los colores ANSI escritos a mano

Falta el implementador que da color. Una terminal pinta el texto de colores cuando recibe, mezcladas con él, unas secuencias especiales que empiezan por el carácter de control *escape* (ESC, el número 27). No hace falta ninguna librería: son cadenas de texto.

Códigos de escape ANSI, sin librerías

Una secuencia de color tiene la forma `ESC [ n m`, donde `n` es un número que dice qué activar. En Java, el carácter ESC se escribe `\u001B` dentro de una cadena. Los tres que usa Emberdeep:

Secuencia	Efecto	En Java
`ESC[31m`	Texto en rojo	`"\u001B[31m"`
`ESC[32m`	Texto en verde	`"\u001B[32m"`
`ESC[0m`	Vuelve al aspecto normal	`"\u001B[0m"`

La regla es siempre la misma: se abre el color, se escribe el texto y se cierra con `ESC[0m`. Si te olvidas del cierre, el color se queda activo y pinta todo lo que venga después, incluido el *prompt* de tu terminal. Una terminal que no entienda estas secuencias (o un fichero al que

rediriges la salida) las muestra como caracteres sueltos, y la salida se llena de basura. Por eso existe la salida de texto plano.

ui/output/AnsiOutput.java

```java
package emberdeep.ui.output;

import java.io.PrintStream;

public class AnsiOutput implements Output {
    private static final String ESC = "\u001B[";
    private static final String RESET = ESC + "0m";

    private final PrintStream out;

    public AnsiOutput(PrintStream out) {
        this.out = out;
    }

    @Override
    public void write(String text) {
        out.print(text);
    }

    @Override
    public void writeStyled(String text, Style style) {
        String code = codeOf(style);
        if (code.isEmpty()) {
            out.print(text);
        } else {
            out.print(code + text + RESET);
        }
    }

    private static String codeOf(Style style) {
        return switch (style) {
            case NORMAL -> "";
            case DANGER -> ESC + "31m";
            case GOOD -> ESC + "32m";
        };
    }
}
```

Tres detalles. `NORMAL` no abre ningún color, así que no hay nada que cerrar. El `switch` sobre `Style` no lleva `default`: es un enumerado cerrado y, si alguien añade un cuarto estilo, el compilador marca este método (otra vez la ventaja de los tipos cerrados que viste en el capítulo 9). Y todo el conocimiento de los códigos vive en esta clase: si mañana el peligro debe ser naranja, se cambia una línea aquí y ninguna vista se entera.

10.6 Bridge y Strategy: dos diagramas parecidos, dos problemas

Mira los dos diagramas de esta sección. Cada uno tiene una clase que guarda una referencia a una interfaz, y detrás de la interfaz varias clases. Dibujados con cuatro cajas, casi se pueden confundir. Lo que los separa es el problema que resuelven:

	Bridge	**Strategy**
Problema	Dos ejes que cambian por separado y se multiplicarían por herencia	Un solo algoritmo con varias versiones, entre las que se elige
Jerarquías	Dos: la de la abstracción y la del implementador, que crecen las dos	Una: la de las estrategias; el contexto no tiene subclases
Quién crece	Cualquiera de los dos lados, sin tocar el otro	Se añaden estrategias; el contexto no cambia
Qué guarda la referencia	Una abstracción refinada (`HudView`) que delega el trabajo de bajo nivel	El contexto (`Fighter`) que delega una decisión
Cómo se nota	Contar clases: sin puente serían $N \times M$	Hay un `if` o un `switch` sobre «cuál usar»
En Emberdeep	`HudView` y `AnsiOutput`	`Fighter` y `LowestHpAi`

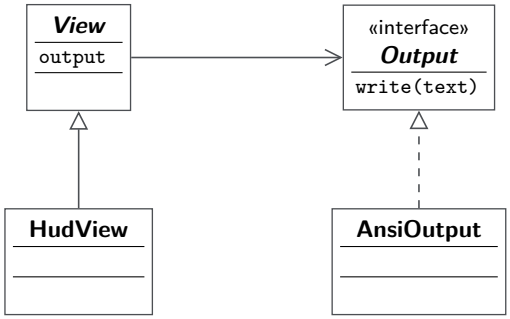


Diagrama UML del patrón Bridge: dos jerarquías unidas por una referencia.

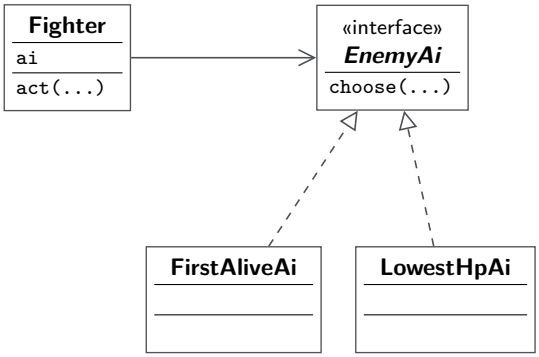


Diagrama UML del patrón Strategy: una sola jerarquía de algoritmos.

El criterio rápido es una pregunta: *¿cuántas cosas distintas crecen?* Si añades una salida y el resto del programa no se entera, y añades una vista y las salidas tampoco, hay dos ejes: Bridge. Si lo que añades es siempre «otra forma de hacer lo mismo» y quien la usa solo tiene que elegir una (la IA de un enemigo, la fórmula de daño), es un único eje: Strategy. En el código se ven parecidos: un campo con una interfaz que se recibe por el constructor. Y hay otra pista: una estrategia suele caber en una función y por eso puede ser una lambda, como `DamageFormula`; un implementador de Bridge suele tener varias operaciones relacionadas, y una interfaz con más de un método abstracto ya no se puede escribir como lambda [26]. El ejercicio 10.3 te pide aplicar este criterio a cuatro situaciones.

10.7 Bridge: cuándo sí y cuándo no

Compensa cuando hay de verdad dos ejes que van a seguir creciendo: más vistas y más salidas, más informes y más formatos, más tipos de aviso y más canales. También cuando quieres cambiar el implementador en tiempo de ejecución (aquí, `--plain` o `--ansi` al arrancar) o poder probar una abstracción con un implementador falso (`BufferOutput`) sin tocar una pantalla.

No compensa si solo hay un eje. Un solo tipo de aviso con dos canales es un Strategy, o una interfaz con dos clases; montar una jerarquía de abstracciones «por si algún día» es una indirección que nadie usa. Tampoco compensa si los dos lados están tan ligados que cada cambio de uno obliga a cambiar el otro: entonces no son independientes, y el puente solo añade clases. Y tiene costes reales:

- **Una indirección más al leer.** Para saber qué sale por pantalla hay que mirar la vista *y* saber qué salida le pasó el `Main`.
- **La interfaz del implementador es difícil de acertar.** Si es demasiado de alto nivel (`drawHud`), el implementador repite lógica; si es demasiado de bajo nivel (escribir carácter a carácter), la abstracción se llena de detalle. Aquí, dos operaciones han bastado para tres vistas, pero una cuarta vista con necesidades nuevas (posicionar el cursor, borrar pantalla) obligaría a ampliarla, y con ella a las tres salidas.
- **Se pierde información en el camino.** `PlainOutput` y `BufferOutput` tiran el estilo: una comprobación que compare dos búferes no notará que una partida pintó un golpe de rojo y la otra de verde. El puente deja que cada lado decida qué hace con lo que recibe.
- **El montaje lo hace alguien.** Alguien tiene que saber qué vista va con qué salida: el `Main`. El puente reparte las responsabilidades, pero no elimina la de elegir pareja.

Dos veces separar, dos patrones distintos

En este proyecto ya has separado «cómo se calcula» (Strategy, capítulo 2) y ahora «qué se cuenta frente a dónde se escribe» (Bridge). Un tercer patrón, que este libro no desarrolla, se confunde a veces con los dos: el *Adapter* hace de traductor entre dos clases que ya existían y no encajan. Si el problema es «esta clase no habla el idioma de aquella», es un adaptador; si es «dos cosas que cambian por separado», es un puente.

10.8 En el proyecto: H11 con Bridge

Este capítulo avanza el hito H11, y el trabajo se reparte en **tres pasos** que se comprueban uno a uno. Cambios del hito:

- **Nuevo paquete** `ui/output/`, con `Output`, `Style`, `AnsiOutput`, `PlainOutput` y `BufferOutput`.
- **Nuevo paquete** `ui/view/`, con `View`, `HudView`, `LogView` y `MapView`.
- **Desaparece** `ui/listener/`. `CombatLog` pasa a ser `LogView` y `HudListener` pasa a ser `HudView`. `MapView` es nueva: dibuja el mapa del modo mapa, que hasta ahora imprimía el propio `ConsoleMenu`.
- **Cambian** `ConsoleMenu`, que recibe un `MapView` por constructor y ya no llama a `GameMap.render()`, y `Main`, que elige la salida y se la pasa a las tres vistas.

Qué cambia en la partida. Las reglas y los números no cambian, y con salida de texto plano el texto es idéntico al del capítulo anterior, letra por letra. Cambia lo que se puede elegir. `Main` acepta dos opciones nuevas, que pueden ir en cualquier posición:

Opción	Efecto
`--plain`	Usa `PlainOutput`: texto sin color.
`--ansi`	Usa `AnsiOutput`: colores ANSI.
ninguna	`PlainOutput` si hay guion de teclas, `AnsiOutput` si se juega a mano.

Pedir las dos a la vez es un error (`IllegalArgumentException`). El modo guion usa texto plano por defecto porque su salida se compara y se guarda en ficheros, y los códigos de color la ensuciarían. `BufferOutput` no se elige por argumento: lo usan las comprobaciones del hito.

Los tres pasos y qué debes ver en cada uno.

- **Paso 1, el lado de las salidas.** `Style`, `Output` y las tres salidas. Resultado: un programa de comprobación escribe la misma línea con las tres y se ve el texto plano, el texto con códigos ESC y el contenido del búfer.
- **Paso 2, el lado de las vistas.** `View`, `HudView`, `LogView` y `MapView`; `ConsoleMenu` y `Main` cambian. Resultado: el guion de comprobación del capítulo anterior da exactamente la misma salida con `--plain` y, con `--ansi`, las barras y los golpes llevan sus códigos de color.
- **Paso 3, el búfer.** Un programa de comprobación juega dos veces la misma partida con la misma semilla, cada vez con un `BufferOutput`, y comprueba que el texto acumulado es el mismo.

Hito 11: vistas y salidas unidas por un puente

Tras este hito, lo que Emberdeep muestra y el sitio donde lo escribe son dos jerarquías independientes: tres vistas, tres salidas, nueve combinaciones con seis clases. La estructura:

Estructura de carpetas tras el capítulo 10

```
emberdeep/
|-- settings.gradle
|-- build.gradle                    (con JUnit 6.1.3)
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
|-- scripts/
|   `-- demo.txt
`-- src/
    |-- main/java/emberdeep/
    |   |-- app/
    |   |   |-- Main.java              (composition root)
    |   |   `-- GameConfig.java
    |   |-- engine/
    |   |   |-- character/             Stats, Hero, HeroBuilder, Enemy,
    |   |   |                          EnemyKind, Fighter, Skill,
    |   |   |                          EnemyTemplate, TemplateRegistry
    |   |   |-- stats/                 StatSource, StatKind, BaseStats,
    |   |   |                          StatDecorator, WeaponBonus,
    |   |   |                          ArmorBonus, TempBuff
    |   |   |-- combat/
    |   |   |   |-- Combat.java
    |   |   |   |-- ai/                EnemyAi, FirstAliveAi, LowestHpAi,
    |   |   |   |                      RandomTargetAi, SkillfulAi
    |   |   |   |-- command/           CombatCommand, Attack, Defend,
    |   |   |   |                      UseItem, UseSkill, CommandHistory
    |   |   |   |-- damage/            DamageFormula, FlatFormula,
    |   |   |   |                      ScaledFormula, Damage, DamageType
    |   |   |   `-- defense/           DamageHandler, DamagePipeline,
    |   |   |                          DodgeHandler, ShieldHandler,
    |   |   |                          ArmorHandler, ResistanceHandler
    |   |   |-- item/                  Item, ItemKind, Inventory
    |   |   |-- event/                 GameEvent, GameListener, EventBus
    |   |   |-- achievement/           AchievementTracker
    |   |   |-- save/                  Snapshot, FighterState,
    |   |   |                          SnapshotStore
    |   |   |-- session/               GameSession
    |   |   `-- world/
    |   |       |-- Dungeon.java, Floor.java,
    |   |       |   DungeonBuilder.java
    |   |       |-- biome/             BiomeFactory, CryptBiome,
    |   |       |                      CavernBiome
    |   |       `-- map/               TileType, TileTypes, GameMap,
    |   |                              MapParser, MapData
    |   `-- ui/
    |       |-- ConsoleMenu.java
    |       |-- input/                 InputSource, ConsoleInput,
    |       |                          ScriptInput
    |       |-- view/                  View, MapView, HudView, LogView
    |       `-- output/                Output, Style, AnsiOutput,
    |                                  PlainOutput, BufferOutput
    `-- test/java/emberdeep/
        `-- engine/save/               SnapshotTest
```

Lo practicas en el ejercicio 10.4. Antes, tres ejercicios más pequeños, en otros dominios.

10.9 Ejercicios

Ejercicio 10.1: Doce clases para tres formas y cuatro colores ★★★☆☆

No es de Emberdeep. Un programa de dibujo tiene tres formas (círculo, cuadrado y triángulo) y cuatro colores (rojo, verde, azul y negro). Alguien lo ha resuelto con herencia: una clase abstracta `Shape` y una clase por cada pareja forma-color, todas hijas directas de `Shape`. Estas son dos de las doce, y las otras diez siguen el mismo patrón:

`Shape.java y dos de las doce clases`

```
public abstract class Shape {
    public abstract String draw();
}

public class CircleRed extends Shape {
    @Override
    public String draw() {
        return "círculo rojo";
    }
}

public class SquareBlue extends Shape {
    @Override
    public String draw() {
        return "cuadrado azul";
    }
}
```

Las doce clases se llaman `CircleRed`, `CircleGreen`, `CircleBlue`, `CircleBlack`, `SquareRed`, `SquareGreen`, `SquareBlue`, `SquareBlack`, `TriangleRed`, `TriangleGreen`, `TriangleBlue` y `TriangleBlack`. Responde por escrito, sin programar:

1. ¿Cuántas clases hay en total, contando `Shape`?
2. Rediseña el programa con Bridge. Escribe, en texto o dibujando un diagrama, cuál es la abstracción, cuál el implementador y cuáles las clases de cada lado. ¿Cuántas clases hay ahora?
3. Se pide una cuarta forma (la estrella) y un quinto color (el amarillo). ¿Cuántas clases nuevas hay que escribir en cada diseño, y cuántas clases tiene cada uno al final?
4. En tu diseño con Bridge, ¿qué línea de código es la que «une» la forma con el color?

Está bien si das los cuatro números (cuenta y recuenta), si tu diagrama deja claro que *ninguna* forma nombra un color concreto y si la respuesta 4 señala un campo y su asignación, no una subclase.

Pista: *Para el diagrama, copia el de la sección 10.6 y cambia los nombres. Una forma con color sigue siendo una forma, pero el color lo* tiene.

Lectura de código ▪ solución en la pág. 413

Ejercicio 10.2: Informes y exportadores con un puente ★★★☆☆

No es de Emberdeep. Crea una carpeta vacía y, dentro, archivos sin paquete. Una aplicación genera informes de ventas y quiere exportarlos en varios formatos. Hay dos tipos de informe y, de momento, dos formatos, pero los dos lados van a crecer. Parte de este registro:

Sale.java

```
public record Sale(String product, int units) {}
```

Diseña con Bridge, en este orden:

1. `Exporter`: interfaz con dos métodos, `String title(String text)` y `String row(String name, int value)`. Cada uno devuelve el trozo de texto ya formateado, con su salto de línea si el formato lo lleva.
2. `TextExporter`: `title` devuelve el texto, un salto de línea, tantos signos `=` como letras tenga el texto y otro salto; `row` devuelve `nombre: valor` y un salto. `HtmlExporter`: `title` devuelve `<h1>texto</h1>` y un salto; `row` devuelve `<p>nombre: valor</p>` y un salto.
3. `Report`: clase abstracta que recibe un `Exporter` en el constructor y declara `String render(List<Sale> sales)`. `SummaryReport` devuelve el título `Resumen` y una sola fila `Total` con la suma de las unidades. `DetailReport` devuelve el título `Detalle` y una fila por venta, con el producto y sus unidades.
4. `Main`: con las ventas `Poción` (3 unidades) y `Bomba` (2), imprime, para cada exportador, una línea `--- NombreDeLaClase ---` (con `getClass().getSimpleName()`) y a continuación los dos informes.
5. **Sin tocar ningún informe**, añade `CsvExporter`: `title` no devuelve nada (una cadena vacía) y `row` devuelve `nombre,valor` y un salto. Añádelo a la lista de `Main`.

Compila con `javac *.java` y ejecuta con `java Main`. Está bien si la salida es exactamente:

Salida esperada

```
--- TextExporter ---
Resumen
=======
Total: 5
Detalle
=======
Poción: 3
Bomba: 2
--- HtmlExporter ---
<h1>Resumen</h1>
<p>Total: 5</p>
<h1>Detalle</h1>
<p>Poción: 3</p>
<p>Bomba: 2</p>
--- CsvExporter ---
Total,5
Poción,3
Bomba,2
```

y si, para añadir el CSV, no has modificado ni `Report`, ni `SummaryReport`, ni `DetailReport`,

ni los otros dos exportadores. Si te hace falta tocarlos, el puente está mal puesto.

Pista: *Los informes no deben nombrar ninguna clase de exportador: solo reciben un* `Exporter` *y le piden el título y las filas. Para la suma sirve* `sales.stream().mapToInt(Sale::units).sum()`.

Práctica ▪ solución en la pág. 416

Ejercicio 10.3: ¿Strategy o Bridge? ★★★★☆

Para cada una de estas cuatro situaciones decide si usarías Strategy, Bridge o ninguno de los dos, y explica el criterio que te hace decidir. No hay que escribir el código completo, pero sí nombrar las piezas.

A. En Emberdeep, el jugador elige al arrancar si el daño se calcula con la fórmula plana o con la escalada, y mañana se quiere una tercera.

B. Un editor de dibujo tiene herramientas (lápiz, goma, relleno) que deben funcionar sobre lienzos distintos (pantalla, imagen PNG, fichero SVG). Se esperan herramientas nuevas y lienzos nuevos.

C. Una aplicación envía avisos (bienvenida, alerta, recordatorio) por canales distintos (correo, SMS, notificación del móvil). Cada aviso tiene su propio formato y el equipo prevé más avisos y más canales.

D. En Emberdeep, cada enemigo tiene un tipo (`SKELETON`, `ZOMBIE`...) y una inteligencia artificial (`FirstAliveAi`, `LowestHpAi`...). Alguien propone «un puente entre los tipos de enemigo y sus IA».

Está bien si: (1) cada decisión se justifica con una pregunta sobre el problema (cuántos ejes crecen, qué se elige y quién lo elige), no con el nombre del patrón; (2) en las situaciones de Bridge nombras la abstracción y el implementador, y en las de Strategy el contexto y la estrategia; (3) en la situación D aclaras qué es un tipo de enemigo en el proyecto (una jerarquía de clases o un dato) y qué cambia en la decisión según la respuesta; (4) para al menos una situación dices qué haría que cambiaras de opinión.

Pista: *Cuenta los ejes: ¿hay dos cosas que se pueden añadir sin tocar la otra? ¿Es el segundo eje una jerarquía de clases o solo un valor?*

Diseño ▪ solución en la pág. 419

Ejercicio 10.4: Hito 11: tres vistas y tres salidas con Bridge ★★★★☆

Implementa el hito H11 partiendo del estado del capítulo anterior (solución del ejercicio 9.4), en tres partes. Comprueba cada una antes de pasar a la siguiente.

Parte 1 (las salidas). Crea el paquete `ui/output/`:

1. `Style`: enumerado con `NORMAL`, `DANGER` y `GOOD`.
2. `Output`: interfaz con `void write(String text)` (escribe el texto tal cual, sin salto de línea) y `void writeStyled(String text, Style style)`.
3. `PlainOutput(PrintStream out)`: escribe con `out.print` y trata el estilo como si no existiera.
4. `AnsiOutput(PrintStream out)`: igual, pero `DANGER` escribe el texto entre `"\u001B[31m"` y `"\u001B[0m"`, y `GOOD` entre `"\u001B[32m"` y `"\u001B[0m"`. `NORMAL` no añade nada. El `switch` sobre `Style` no lleva `default`.
5. `BufferOutput`: acumula todo lo escrito (sin los estilos) y lo devuelve con `String contents()`.

Está bien si este archivo temporal, `app/Check10a.java`, escribe lo que se indica debajo (el carácter ESC se ve como `^[` porque la salida pasa por `cat -v`):

app/Check10a.java (parte 1; bórralo al terminar)

```java
package emberdeep.app;

import emberdeep.ui.output.AnsiOutput;
import emberdeep.ui.output.BufferOutput;
import emberdeep.ui.output.Output;
import emberdeep.ui.output.PlainOutput;
import emberdeep.ui.output.Style;
import java.util.List;

public class Check10a {
    public static void main(String[] args) {
        BufferOutput buffer = new BufferOutput();
        List<Output> outputs = List.of(new PlainOutput(System.out),
                new AnsiOutput(System.out), buffer);
        for (Output output : outputs) {
            output.write("Vida: ");
            output.writeStyled("5/30", Style.DANGER);
            output.write("\n");
        }
        System.out.print("buffer = " + buffer.contents());
    }
}
```

Se compila y se ejecuta desde emberdeep/

```
$ ./gradlew -q classes
$ java -cp build/classes/java/main emberdeep.app.Check10a | cat -v
Vida: 5/30
Vida: ^[[31m5/30^[[0m
buffer = Vida: 5/30
```

Parte 2 (las vistas). Crea el paquete `ui/view/` y retira `ui/listener/`:

1. `View`: clase abstracta con el campo `protected final Output output` (rechaza `null` con `Objects.requireNonNull`), un constructor protegido que lo recibe, y dos métodos protegidos: `line(String text)`, que escribe el texto seguido de `"\n"`, y `line(String text, Style style)`, que escribe el texto con estilo y después, *fuera* del estilo, el salto de línea.

2. `LogView(Output output)`: `extends View implements GameListener`. Escribe las mismas frases que `CombatLog` (con `line`), con este estilo: `DANGER` si el golpeado o el caído es un héroe, `GOOD` para `ItemUsed`, para el logro y para la victoria, `DANGER` para la derrota y `NORMAL` en el resto.

3. `HudView(Output output)`: el `HudListener` anterior, escribiendo con `line`. Estilo de la barra: `DANGER` si $hp \cdot 4 \leq maxHp$, `GOOD` si $hp \cdot 2 > maxHp$ y `NORMAL` en los demás casos.

4. `MapView(Output output)`: `extends View` y tiene un método `void show(GameMap`

`map`) que escribe las filas del mapa, casilla a casilla, con `writeStyled` y un salto de línea al final de cada fila. La casilla del grupo se pinta con `@` y estilo `GOOD`; las demás con su glifo y estilo `DANGER` si el color simbólico del tipo es `"red"`, `GOOD` si es `"cyan"` y `NORMAL` en los demás casos.

5. `ConsoleMenu` recibe un tercer parámetro, `MapView mapView`, y en `explore` sustituye `out.print(map.render())` por `mapView.show(map)`.
6. `Main`: acepta `--plain` y `--ansi` en cualquier posición (no cuentan como número de planta ni como segundo argumento). Sin ninguna, usa `PlainOutput` si hay guion y `AnsiOutput` si no lo hay; con las dos lanza `IllegalArgumentException`. Crea una única salida con `System.out` y se la pasa a las tres vistas. Suscribe en este orden `LogView`, `HudView` y el `AchievementTracker`.
7. Borra `CombatLog` y `HudListener`.

Está bien si, con el guion `scripts/check10.txt` (el mismo de la parte 2 del capítulo anterior: bomba al Zombi, Borin al Esqueleto, Cael se cubre; bórralo al terminar):

`scripts/check10.txt (parte 2; bórralo al terminar)`

```
# Ronda 1: Aria lanza la bomba al Zombi
3
3
3
# Borin ataca al Esqueleto
1
1
# Cael se cubre
2
```

las dos ejecuciones siguientes dan archivos idénticos, y esos archivos son idénticos a lo que daba el hito anterior (sin códigos de color, con las mismas frases y barras):

Texto plano: por defecto y pidiéndolo

```
$ ./gradlew -q run --args="scripts/check10.txt" > a.txt 2>/dev/null
$ ./gradlew -q run --args="scripts/check10.txt --plain" > b.txt 2>/dev/null
$ cmp a.txt b.txt && echo iguales
iguales
```

y, pidiendo color, solo salen con códigos las líneas con estilo (la barra del Zombi, la del Esqueleto, el golpe del Zombi a Aria y la barra de Aria):

Pidiendo color (solo las líneas con ESC)

```
$ ./gradlew -q run --args="scripts/check10.txt --ansi" 2>/dev/null \
    | grep -a "$(printf '\033')" | cat -v
^[[32m  Zombi [######....] 12/22^[[0m
^[[32m  Esqueleto [#######...] 12/18^[[0m
^[[31mZombi hace 1 de daño físico a Aria.^[[0m
^[[32m  Aria [##########] 29/30^[[0m
```

Además, el modo mapa sigue funcionando. Con `--ansi` dibuja el mapa de la planta 1 con la `@` en verde y las trampas `^` en rojo, y con `--plain` el mismo mapa del capítulo 5 sin ningún

código:

Modo mapa (se sale con q)

```
$ printf 'q\n' | ./gradlew -q run --args="1 map --ansi"
$ printf 'q\n' | ./gradlew -q run --args="1 map --plain"
```

Parte 3 (el búfer). Escribe un programa temporal, `app/Check10c.java`, que juega una partida con la semilla 42 y una única heroína, Aria (30 de vida, 8 de ataque, 3 de defensa, sin habilidades), contra un Esqueleto de la cripta. Debe hacer en este orden: Aria ataca; fase enemiga; Aria se defiende; fase enemiga; Aria ataca dos veces seguidas. El método `play(Output output)` suscribe un `LogView` y un `HudView` a la salida recibida y juega la partida. `main` juega dos veces con dos `BufferOutput` distintos, imprime si los dos textos son iguales y vuelve a jugar con un `PlainOutput` sobre `System.out` para que veas el texto. Está bien si la salida es:

Salida esperada

```
Iguales: true
Aria hace 8 de daño físico a Esqueleto.
  Esqueleto [######....] 10/18
Esqueleto hace 3 de daño físico a Aria.
  Aria [#########.] 27/30
Aria se cubre: +5 de escudo y +2 de defensa este turno.
El golpe de Esqueleto a Aria no hace daño.
Aria hace 8 de daño físico a Esqueleto.
  Esqueleto [##........] 2/18
Aria hace 8 de daño físico a Esqueleto.
  Esqueleto [..........] 0/18
Esqueleto queda fuera de combate.
¡Victoria!
```

Borra `Check10a.java`, `Check10c.java` y `scripts/check10.txt`.

Pista: *Para el estilo del mapa, `TileType` ya trae el color en el campo `color()`, y `GameMap` te da `width()`, `height()`, `tileAt(x, y)`, `partyX()` y `partyY()`. Para las opciones de `Main`, `Arrays.asList(args).contains("-plain")` basta.*

Proyecto ▪ solución en la pág. 421

10.10 Resumen

Resumen

- El olor de Bridge es una clase por cada combinación de dos cosas que cambian por separado (`AnsiHud`, `PlainHud`, `AnsiLog`...): las clases se multiplican ($N \times M$), la lógica se repite y dos decisiones quedan pegadas. Bridge pone cada cosa en su jerarquía y las une por composición ($N + M$).
- Piezas: la abstracción (`View`, clase abstracta que guarda el `Output`), las abstracciones refinadas (`HudView`, `LogView`, `MapView`), el implementador (`Output`, interfaz) y los implementadores concretos (`AnsiOutput`, `PlainOutput`, `BufferOutput`). El cliente (`Main`) monta las parejas.

- La interfaz del implementador debe ser de bajo nivel: `write` y `writeStyled` bastan para construir barras, frases y mapas, y las vistas no repiten nada en las salidas. `Style` cierra con un enumerado las intenciones (normal, peligro, bien) para que las vistas no conozcan ningún color.
- Los códigos ANSI son cadenas que empiezan por el carácter ESC (en Java, `\u001B`): `ESC[31m` rojo, `ESC[32m` verde y `ESC[0m` para cerrar. Se abre, se escribe y se cierra. Una terminal que no los entiende (o un fichero) los muestra como basura: por eso existe la salida de texto plano, que es la del modo guion.
- Bridge y Strategy se dibujan parecido y no son lo mismo. Bridge: dos ejes que crecen y se multiplicarían por herencia. Strategy: un solo algoritmo con varias versiones. Pregunta clave: ¿cuántas cosas distintas crecen?
- Bridge compensa con dos ejes que seguirán creciendo, o para cambiar un implementador o probar con uno falso. No compensa con un solo eje. Cuesta una indirección más, una interfaz difícil de acertar, información que se pierde por el camino (el estilo en un búfer) y alguien que monte las parejas.
- H11: `ui/output/` y `ui/view/` sustituyen a `ui/listener/` (`CombatLog` es `LogView`, `HudListener` es `HudView`); `MapView` dibuja el mapa; `ConsoleMenu` recibe el `MapView`; `Main` acepta `--plain` y `--ansi`, y con guion usa texto plano por defecto.

Soluciones de los ejercicios

Solución del ejercicio 10.1 enunciado en la pág. 407

1. Con herencia. Hay 13 clases: `Shape` y las 12 parejas (3×4).
2. Con Bridge. La abstracción es la forma y el implementador es el color. Cada forma *tiene* un color en lugar de *ser* de un color:

Diagrama en texto de la solución con Bridge

```
Abstracción                          Implementador

  Shape (abstracta)  ------------>   Color (interfaz)
  campo: Color color                 String name()
    ^                                  ^
    |-- Circle                         |-- Red
    |-- Square                         |-- Green
    |-- Triangle                       |-- Blue
                                       `-- Black
```

Ahora hay $1 + 3 + 1 + 4 = 9$ clases e interfaces: `Shape`, tres formas, `Color` y cuatro colores. Este es el código de esa versión, que no hace falta escribir en el ejercicio, pero que se ejecuta tal cual con `javac *.java` y `java Main`:

`Color.java, Red.java y Blue.java`

```java
public interface Color {
    String name();
}

public class Red implements Color {
    @Override
    public String name() {
        return "rojo";
    }
}

public class Blue implements Color {
    @Override
    public String name() {
        return "azul";
    }
}
```

(Con `Green` y `Black`, iguales en estructura, devolviendo `"verde"` y `"negro"`, están los cuatro colores.)

Shape.java, Circle.java y Square.java

```java
public abstract class Shape {
    protected final Color color;

    protected Shape(Color color) {
        this.color = color;
    }

    public abstract String draw();
}

public class Circle extends Shape {
    public Circle(Color color) {
        super(color);
    }

    @Override
    public String draw() {
        return "círculo " + color.name();
    }
}

public class Square extends Shape {
    public Square(Color color) {
        super(color);
    }

    @Override
    public String draw() {
        return "cuadrado " + color.name();
    }
}
```

(`Triangle`, igual, con `"triángulo "`.) Y un `Main` que muestra cualquier pareja sin clases nuevas:

Main.java

```java
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Shape> shapes = List.of(
                new Circle(new Red()),
                new Square(new Blue()),
                new Triangle(new Green()));
        for (Shape shape : shapes) {
            System.out.println(shape.draw());
        }
    }
}
```

Salida

```
círculo rojo
cuadrado azul
triángulo verde
```

3. Añadir una forma y un color.

	Con herencia	Con Bridge
Cuarta forma (estrella)	4 clases nuevas (una por color)	1 clase (`Star`)
Quinto color (amarillo)	4 clases nuevas (una por forma, ya con 4 formas)	1 clase (`Yellow`)
Total final	$1 + 4 \times 5 = 21$	$1 + 4 + 1 + 5 = 11$

Si el quinto color se añade antes de la cuarta forma, los números intermedios cambian (3 y 5), pero el total final es el mismo: 21 frente a 11. Con herencia se paga cada *pareja*; con Bridge, cada *pieza*.

4. Qué línea une los dos lados. Es el constructor de `Shape`, que guarda el color en el campo `protected final Color color`, y la llamada `new Circle(new Red())` que lo rellena. Es un campo de tipo interfaz, no una subclase: la forma conoce el *contrato* `Color`, nunca una clase concreta de color.

Errores típicos:

- Contar 12 en la pregunta 1 y olvidar la clase base, o contar solo las formas y los colores (7) en la pregunta 2 y olvidar `Shape` y `Color`.
- Dibujar a `Circle` heredando de `Red`: se vuelve a la herencia por pareja.
- Dar el número de clases nuevas sin tener en cuenta los colores ya existentes: al añadir la estrella en el diseño con herencia hacen falta cuatro, no una.

Solución del ejercicio 10.2

enunciado en la pág. 408

Nueve archivos sin paquete, en la misma carpeta (el `Sale` es el del enunciado).

Exporter.java

```java
public interface Exporter {
    String title(String text);
    String row(String name, int value);
}
```

TextExporter.java

```
public class TextExporter implements Exporter {
    @Override
    public String title(String text) {
        return text + "\n" + "=".repeat(text.length()) + "\n";
    }

    @Override
    public String row(String name, int value) {
        return name + ": " + value + "\n";
    }
}
```

HtmlExporter.java

```
public class HtmlExporter implements Exporter {
    @Override
    public String title(String text) {
        return "<h1>" + text + "</h1>\n";
    }

    @Override
    public String row(String name, int value) {
        return "<p>" + name + ": " + value + "</p>\n";
    }
}
```

Report.java

```
import java.util.List;

public abstract class Report {
    protected final Exporter exporter;

    protected Report(Exporter exporter) {
        this.exporter = exporter;
    }

    public abstract String render(List<Sale> sales);
}
```

SummaryReport.java

```java
import java.util.List;

public class SummaryReport extends Report {
    public SummaryReport(Exporter exporter) {
        super(exporter);
    }

    @Override
    public String render(List<Sale> sales) {
        int total = sales.stream().mapToInt(Sale::units).sum();
        return exporter.title("Resumen") + exporter.row("Total", total);
    }
}
```

DetailReport.java

```java
import java.util.List;

public class DetailReport extends Report {
    public DetailReport(Exporter exporter) {
        super(exporter);
    }

    @Override
    public String render(List<Sale> sales) {
        StringBuilder text = new StringBuilder(exporter.title("Detalle"));
        for (Sale sale : sales) {
            text.append(exporter.row(sale.product(), sale.units()));
        }
        return text.toString();
    }
}
```

El exportador de CSV es un archivo nuevo, y es lo único que se añade:

CsvExporter.java

```java
public class CsvExporter implements Exporter {
    @Override
    public String title(String text) {
        return "";              // un CSV no tiene título
    }

    @Override
    public String row(String name, int value) {
        return name + "," + value + "\n";
    }
}
```

Main.java

```java
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Sale> sales = List.of(new Sale("Poción", 3),
                new Sale("Bomba", 2));
        List<Exporter> exporters = List.of(new TextExporter(),
                new HtmlExporter(), new CsvExporter());
        for (Exporter exporter : exporters) {
            System.out.println("--- "
                    + exporter.getClass().getSimpleName() + " ---");
            System.out.print(new SummaryReport(exporter).render(sales));
            System.out.print(new DetailReport(exporter).render(sales));
        }
    }
}
```

Por qué funciona. Los informes (la abstracción) deciden *qué* se cuenta: un total o una fila por venta. Los exportadores (el implementador) deciden *cómo* se escribe un título o una fila. Los informes solo conocen la interfaz `Exporter`, que se les pasa por el constructor (el «puente»), y le piden dos operaciones sencillas. Por eso el CSV es una clase nueva y nada más: en cuanto cumple `title` y `row`, los dos informes ya saben usarlo, y los dos informes juntos con tres formatos dan seis combinaciones con cinco clases de pegamento, sin ninguna clase por pareja. El `title` vacío del CSV no es un truco: el contrato dice «devuelve el trozo ya formateado», y un formato sin títulos formatea un título como nada.

Errores típicos:

- Poner un `if (exporter instanceof CsvExporter)` en un informe: es el síntoma de que el contrato no basta. Debe arreglarse en el exportador, no en el informe.
- Crear `SummaryTextReport`, `SummaryHtmlReport`... (la explosión de clases): es exactamente lo que el puente evita.
- Que `Report` cree el exportador por dentro (`new TextExporter()`): el informe quedaría atado a un formato. El exportador se recibe, no se crea.
- Olvidar el `"\n"` en `row`: las filas saldrían pegadas.

Solución del ejercicio 10.3 enunciado en la pág. 409

Solución de referencia.

	Patrón	Piezas	Criterio
A	Strategy	Contexto: `Combat`. Estrategia: `DamageFormula`, con `FlatFormula` y `ScaledFormula`, y una tercera clase o lambda	Un solo eje: «cómo se calcula el daño». Añadir una fórmula no obliga a tocar nada más. Quien elige es el `Main`.
B	Bridge	Abstracción: `Tool`, con `Pen`, `Eraser`, `Fill`. Implementador: `Canvas`, con `ScreenCanvas`, `PngCanvas`, `SvgCanvas`	Dos ejes que crecen a la vez: herramientas y lienzos. Por herencia serían $3 \times 3 = 9$ clases. El implementador debe ser de bajo nivel (poner un píxel, trazar una línea).
C	Bridge	Abstracción: `Notice`, con `Welcome`, `Alert`, `Reminder`. Implementador: `Channel`, con `MailChannel`, `SmsChannel`, `PushChannel`	Los avisos y los canales crecen por separado y cada aviso tiene su formato: el aviso decide el contenido, el canal decide la entrega. Dos ejes.
D	Strategy (si el tipo es un dato)	Contexto: `Fighter` (ya guarda `ai`). Estrategia: `EnemyAi`. El tipo de enemigo es un valor de `EnemyKind`	`EnemyKind` es un *enumerado*, no una jerarquía: no hay un segundo eje de clases que crezca. Un puente de un lado solo es una indirección sin uso.

Qué haría cambiar de opinión. En A, si la fórmula dependiera además de un segundo eje que crece (por ejemplo, tipos de golpe con reglas propias que también varían), habría dos jerarquías. En C, si solo existiera un tipo de aviso y nadie previera más, dejaría de haber dos ejes y bastaría una interfaz `Channel` elegida por el cliente: Strategy. En D, si cada tipo de enemigo fuera una clase con su propio comportamiento de combate y las IA fueran otra jerarquía que crece, entonces sí serían dos ejes y el puente tendría sentido; con un enumerado y datos, no. La pregunta de fondo es siempre la misma: cuántas jerarquías crecen y si una se puede añadir sin tocar la otra.

Criterios para valorar otras soluciones.

- Cada respuesta se apoya en cuántos ejes crecen y en cuál es el segundo eje, no en «Strategy es para algoritmos» ni en una definición memorizada.
- En A y D hay un contexto, una interfaz y varias implementaciones; en B y C hay dos jerarquías unidas por una referencia, y las dos tienen al menos dos variantes.
- En B y C, el implementador tiene operaciones de bajo nivel; una interfaz como `Canvas.drawEverything()` o `Channel.sendWelcomeMessage()` lleva el trabajo de la abstracción al implementador y hace fallar el diseño.
- En D se distingue entre un enumerado (un valor) y una jerarquía de clases, y se argumenta la condición en que cambiaría la respuesta.

Alternativa válida para C. Una solución que plantee un único `Notice` (un record con texto y tipo) y un `Channel` elegido por el cliente (Strategy) es defendible si los avisos no se diferencian en comportamiento, sino solo en datos. Deja de ser válida en cuanto cada aviso necesita su propia plantilla o su propia lógica y esas diferencias van a crecer.

Errores típicos:

- Responder Bridge a las cuatro porque «hay una interfaz con varias clases detrás»: eso también lo tiene Strategy.
- En D, aceptar el puente porque hay dos palabras (tipo e IA) sin comprobar si el tipo

es una jerarquía o un dato.

- Responder sin nombrar las piezas: «sería Bridge» sin decir qué es la abstracción y qué es el implementador.

Solución del ejercicio 10.4 enunciado en la pág. 409

Partes 1 a 3 en orden. Se repiten los archivos enteros (también los que solo cambian un poco), para que la solución esté completa. Los archivos que no aparecen (todo `engine/`, `ui/input/`, `GameConfig`...) no cambian respecto del hito anterior.

Parte 1: ui/output/

`ui/output/Style.java`

```
package emberdeep.ui.output;

public enum Style {
    NORMAL, DANGER, GOOD
}
```

`ui/output/Output.java`

```
package emberdeep.ui.output;

public interface Output {
    // escribe el texto tal cual, sin añadir salto de línea
    void write(String text);

    // igual, pero con el énfasis que pide el estilo
    void writeStyled(String text, Style style);
}
```

ui/output/PlainOutput.java

```
package emberdeep.ui.output;

import java.io.PrintStream;

public class PlainOutput implements Output {
    private final PrintStream out;

    public PlainOutput(PrintStream out) {
        this.out = out;
    }

    @Override
    public void write(String text) {
        out.print(text);
    }

    @Override
    public void writeStyled(String text, Style style) {
        write(text);            // sin color: el estilo se ignora
    }
}
```

ui/output/AnsiOutput.java

```java
package emberdeep.ui.output;

import java.io.PrintStream;

public class AnsiOutput implements Output {
    private static final String ESC = "\u001B[";
    private static final String RESET = ESC + "0m";

    private final PrintStream out;

    public AnsiOutput(PrintStream out) {
        this.out = out;
    }

    @Override
    public void write(String text) {
        out.print(text);
    }

    @Override
    public void writeStyled(String text, Style style) {
        String code = codeOf(style);
        if (code.isEmpty()) {
            out.print(text);
        } else {
            out.print(code + text + RESET);
        }
    }

    private static String codeOf(Style style) {
        return switch (style) {
            case NORMAL -> "";
            case DANGER -> ESC + "31m";
            case GOOD -> ESC + "32m";
        };
    }
}
```

ui/output/BufferOutput.java

```java
package emberdeep.ui.output;

public class BufferOutput implements Output {
    private final StringBuilder text = new StringBuilder();

    @Override
    public void write(String part) {
        text.append(part);
    }

    @Override
    public void writeStyled(String part, Style style) {
        text.append(part);      // guarda el texto, no el estilo
    }

    public String contents() {
        return text.toString();
    }
}
```

Parte 2: ui/view/

ui/view/View.java

```java
package emberdeep.ui.view;

import emberdeep.ui.output.Output;
import emberdeep.ui.output.Style;
import java.util.Objects;

public abstract class View {
    protected final Output output;

    protected View(Output output) {
        this.output = Objects.requireNonNull(output);
    }

    // una línea sin énfasis
    protected void line(String text) {
        output.write(text + "\n");
    }

    // una línea con énfasis; el salto de línea queda fuera del color
    protected void line(String text, Style style) {
        output.writeStyled(text, style);
        output.write("\n");
    }
}
```

ui/view/HudView.java (1/2: imports y onEvent)

```
package emberdeep.ui.view;

import emberdeep.engine.event.GameEvent;
import emberdeep.engine.event.GameEvent.AchievementUnlocked;
import emberdeep.engine.event.GameEvent.CombatEnded;
import emberdeep.engine.event.GameEvent.DamageDealt;
import emberdeep.engine.event.GameEvent.DamageDodged;
import emberdeep.engine.event.GameEvent.Defended;
import emberdeep.engine.event.GameEvent.FighterDefeated;
import emberdeep.engine.event.GameEvent.ItemUsed;
import emberdeep.engine.event.GameListener;
import emberdeep.ui.output.Output;
import emberdeep.ui.output.Style;

public class HudView extends View implements GameListener {
    private static final int WIDTH = 10;

    public HudView(Output output) {
        super(output);
    }

    @Override
    public void onEvent(GameEvent event) {
        switch (event) {
            case DamageDealt e -> bar(e.target(), e.targetHp(),
                    e.targetMaxHp());
            case ItemUsed e -> bar(e.target(), e.targetHp(),
                    e.targetMaxHp());
            case DamageDodged e -> { }
            case FighterDefeated e -> { }
            case Defended e -> { }
            case CombatEnded e -> { }
            case AchievementUnlocked e -> { }
        }
    }
```

ui/view/HudView.java (2/2: la barra y su estilo)

```
    // casillas llenas = hp * 10 / maxHp, redondeado hacia arriba
    private void bar(String name, int hp, int maxHp) {
        int filled = (hp * WIDTH + maxHp - 1) / maxHp;
        line("  " + name + " [" + "#".repeat(filled)
                + ".".repeat(WIDTH - filled) + "] " + hp + "/" + maxHp,
                styleOf(hp, maxHp));
    }

    // rojo con un cuarto de vida o menos, verde con más de la mitad
    private static Style styleOf(int hp, int maxHp) {
        if (hp * 4 <= maxHp) {
            return Style.DANGER;
        }
        return hp * 2 > maxHp ? Style.GOOD : Style.NORMAL;
    }
}
```

ui/view/LogView.java (1/2: imports y onEvent)

```
package emberdeep.ui.view;

import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.event.GameEvent;
import emberdeep.engine.event.GameEvent.AchievementUnlocked;
import emberdeep.engine.event.GameEvent.CombatEnded;
import emberdeep.engine.event.GameEvent.DamageDealt;
import emberdeep.engine.event.GameEvent.DamageDodged;
import emberdeep.engine.event.GameEvent.Defended;
import emberdeep.engine.event.GameEvent.FighterDefeated;
import emberdeep.engine.event.GameEvent.ItemUsed;
import emberdeep.engine.event.GameListener;
import emberdeep.ui.output.Output;
import emberdeep.ui.output.Style;

public class LogView extends View implements GameListener {
    public LogView(Output output) {
        super(output);
    }

    @Override
    public void onEvent(GameEvent event) {
        switch (event) {
            case DamageDealt e -> line(e.attacker() + " hace "
                    + e.amount() + " de daño " + kind(e.type())
                    + " a " + e.target() + ".",
                    e.targetIsHero() ? Style.DANGER : Style.NORMAL);
            case DamageDodged e -> line("El golpe de " + e.attacker()
                    + " a " + e.target() + " no hace daño.");
            case FighterDefeated e -> line(e.name()
                    + " queda fuera de combate.",
                    e.isHero() ? Style.DANGER : Style.NORMAL);
```

ui/view/LogView.java (2/2: resto de eventos y tipo de daño)

```
            case Defended e -> line(e.name() + " se cubre: +"
                    + e.shield() + " de escudo y +" + e.defenseBonus()
                    + " de defensa este turno.");
            case ItemUsed e -> line(e.user() + " usa " + e.itemName()
                    + " sobre " + e.target() + ".", Style.GOOD);
            case CombatEnded e -> line(e.heroesWon()
                    ? "¡Victoria!" : "Derrota...",
                    e.heroesWon() ? Style.GOOD : Style.DANGER);
            case AchievementUnlocked e -> line(
                    "¡Logro desbloqueado: " + e.name() + "!", Style.GOOD);
        }
    }

    private static String kind(DamageType type) {
        return switch (type) {
            case PHYSICAL -> "físico";
            case FIRE -> "de fuego";
        };
    }
}
```

ui/view/MapView.java

```
package emberdeep.ui.view;

import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.TileType;
import emberdeep.ui.output.Output;
import emberdeep.ui.output.Style;

public class MapView extends View {
    public MapView(Output output) {
        super(output);
    }

    public void show(GameMap map) {
        for (int y = 0; y < map.height(); y++) {
            for (int x = 0; x < map.width(); x++) {
                if (x == map.partyX() && y == map.partyY()) {
                    output.writeStyled("@", Style.GOOD);
                } else {
                    TileType tile = map.tileAt(x, y);
                    output.writeStyled(String.valueOf(tile.glyph()),
                            styleOf(tile));
                }
            }
            output.write("\n");
        }
    }

    // el color simbólico del capítulo 5, por fin pintado
    private static Style styleOf(TileType tile) {
        return switch (tile.color()) {
            case "red" -> Style.DANGER;
            case "cyan" -> Style.GOOD;
            default -> Style.NORMAL;
        };
    }
}
```

ui/ConsoleMenu.java (cambian el constructor y `explore`; el resto es el del hito anterior), en siete tramos seguidos dentro de la misma clase:

ui/ConsoleMenu.java (1/7: imports, campos y fórmula)

```java
package emberdeep.ui;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Skill;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.Attack;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.command.Defend;
import emberdeep.engine.combat.command.UseItem;
import emberdeep.engine.combat.command.UseSkill;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.save.SnapshotStore;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.TileType;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.input.InputSource;
import emberdeep.ui.view.MapView;
import java.io.PrintStream;
import java.util.List;
import java.util.stream.Collectors;

public class ConsoleMenu {
    private enum Turn { ACTED, UNDONE, RESTORED }

    private final InputSource input;
    private final PrintStream out;
    private final MapView mapView;
    private final SnapshotStore checkpoints = new SnapshotStore();
    private int checkpointHero;     // a qué héroe le tocaba al guardar

    public ConsoleMenu(InputSource input, PrintStream out,
                       MapView mapView) {
        this.input = input;
        this.out = out;
        this.mapView = mapView;
    }

    public int askFormula() {
        out.println("Fórmula de daño:");
        out.println("  [1] plana: ataque - defensa");
        out.println("  [2] escalada: ataque x 100 / (100 + defensa)");
        return ask(2);
    }
```

ui/ConsoleMenu.java (2/7: la partida)

```
    public void run(GameSession session) {
        Combat combat = session.combat();
        out.println("=== Emberdeep ===");
        while (!combat.isOver()) {
            out.println("--- Ronda " + session.round() + " ---");
            List<Fighter> acting = aliveHeroes(session);
            int i = 0;
            while (i < acting.size() && !combat.isOver()) {
                Turn turn = heroTurn(session, acting.get(i), i);
                switch (turn) {
                    case ACTED -> i++;
                    case UNDONE -> i = Math.max(0, i - 1);
                    case RESTORED -> {
                        acting = aliveHeroes(session);
                        i = checkpointHero;
                    }
                }
            }
            if (!combat.isOver()) {
                combat.enemyPhase();
            }
        }
    }

    private List<Fighter> aliveHeroes(GameSession session) {
        return session.heroes().stream().filter(Fighter::isAlive).toList();
    }
```

ui/ConsoleMenu.java (3/7: el turno de un héroe)

```
    // ACTED: el héroe actúa; UNDONE: se deshace la acción anterior;
    // RESTORED: se vuelve a un punto de control
    private Turn heroTurn(GameSession session, Fighter hero, int index) {
        Combat combat = session.combat();
        while (true) {
            out.println("Héroes:   " + describe(session.heroes()));
            out.println("Enemigos: " + describe(session.enemies()));
            out.println("Objetos:  " + describeItems(session.inventory()));
            out.println("Turno de " + hero.name()
                    + ". [1] atacar  [2] defender  [3] objeto"
                    + "  [4] habilidad  [u] deshacer"
                    + "  [c] guardar  [r] volver");
            String key = input.next();
            if (key.equals("u")) {
                if (combat.undo()) {
                    out.println("Deshaces la última acción.");
                    return Turn.UNDONE;
                }
                out.println("No hay nada que deshacer en esta ronda.");
                continue;
            }
            if (key.equals("c")) {
                saveCheckpoint(session, index);
                continue;
            }
            if (key.equals("r")) {
                if (loadCheckpoint(session)) {
                    return Turn.RESTORED;
                }
                continue;
            }
            CombatCommand command = switch (key) {
                case "1" -> new Attack(combat, hero,
                        askTarget("Objetivo", session.enemies()));
                case "2" -> new Defend(combat, hero);
                case "3" -> chooseItem(session, hero);
                case "4" -> chooseSkill(session, hero);
                default -> {
                    out.println("Tecla no válida.");
                    yield null;
                }
            };
            if (command != null) {
                combat.perform(command);
                return Turn.ACTED;
            }
        }
    }
```

ui/ConsoleMenu.java (4/7: guardar y volver)

```
    private void saveCheckpoint(GameSession session, int index) {
        checkpoints.push(session.createSnapshot());
        checkpointHero = index;
        out.println("Punto de control guardado ("
                + checkpoints.size() + ").");
    }

    // false si todavía no hay ninguno
    private boolean loadCheckpoint(GameSession session) {
        if (checkpoints.isEmpty()) {
            out.println("No hay puntos de control.");
            return false;
        }
        session.restore(checkpoints.last());
        out.println("Vuelves al último punto de control (ronda "
                + session.round() + ").");
        return true;
    }
```

ui/ConsoleMenu.java (5/7: objetos, habilidades y objetivos)

```
    private CombatCommand chooseItem(GameSession session, Fighter hero) {
        List<Item> items = session.inventory().items();
        if (items.isEmpty()) {
            out.println("No te quedan objetos.");
            return null;
        }
        Item item = items.get(choose("Objeto",
                items.stream().map(Item::name).toList()));
        Fighter target = item.kind() == ItemKind.HEALING
                ? askTarget("Aliado", session.heroes())
                : askTarget("Objetivo", session.enemies());
        return new UseItem(session, hero, target, item);
    }

    private CombatCommand chooseSkill(GameSession session, Fighter hero) {
        List<Skill> skills = hero.skills();
        if (skills.isEmpty()) {
            out.println("No tienes habilidades.");
            return null;
        }
        Skill skill = skills.get(choose("Habilidad",
                skills.stream().map(Skill::name).toList()));
        Fighter target = askTarget("Objetivo", session.enemies());
        return new UseSkill(session.combat(), hero, skill, target);
    }

    private Fighter askTarget(String title, List<Fighter> group) {
        List<Fighter> alive = group.stream()
                .filter(Fighter::isAlive).toList();
        return alive.get(choose(title,
                alive.stream().map(Fighter::name).toList()));
    }
```

ui/ConsoleMenu.java (6/7: menús, entrada y auxiliares)

```
    // Muestra "Título:  [1] a  [2] b" y devuelve la posición (desde 0)
    private int choose(String title, List<String> labels) {
        StringBuilder menu = new StringBuilder(title + ":");
        for (int i = 0; i < labels.size(); i++) {
            menu.append("  [").append(i + 1).append("] ")
                    .append(labels.get(i));
        }
        out.println(menu);
        return ask(labels.size()) - 1;
    }

    private int ask(int max) {
        while (true) {
            String line = input.next();
            try {
                int n = Integer.parseInt(line);
                if (n >= 1 && n <= max) {
                    return n;
                }
            } catch (NumberFormatException e) {
                // no es un número: se avisa más abajo
            }
            out.println("Escribe un número entre 1 y " + max + ".");
        }
    }

    private String describe(List<Fighter> group) {
        return group.stream()
                .map(f -> f.name() + " " + f.hp() + "/"
                        + f.stats().maxHp())
                .collect(Collectors.joining(", "));
    }

    private String describeItems(Inventory inventory) {
        if (inventory.items().isEmpty()) {
            return "(ninguno)";
        }
        return inventory.items().stream()
                .map(Item::name)
                .collect(Collectors.joining(", "));
    }
```

ui/ConsoleMenu.java (7/7: modo mapa y llave final)

```
    public boolean explore(GameMap map, TileTypes types) {
        while (true) {
            mapView.show(map);
            out.println("[w a s d] mover, [t] casillas, [q] salir");
            String key = input.next();
            switch (key) {
                case "w" -> step(map, 0, -1);
                case "a" -> step(map, -1, 0);
                case "s" -> step(map, 0, 1);
                case "d" -> step(map, 1, 0);
                case "t" -> showTiles(map, types);
                case "q" -> { return false; }
                default -> out.println("Tecla no válida.");
            }
            if (map.onStairs()) {
                out.println("Bajas por la escalera.");
                return true;
            }
        }
    }

    private void step(GameMap map, int dx, int dy) {
        if (!map.move(dx, dy)) {
            out.println("No puedes pasar.");
        }
    }

    private void showTiles(GameMap map, TileTypes types) {
        TileType corner = map.tileAt(0, 0);
        TileType opposite = map.tileAt(map.width() - 1, map.height() - 1);
        out.println("Casillas: " + map.width() * map.height()
                + ", TileType creados: " + types.createdCount());
        out.println("Esquinas opuestas, mismo objeto: "
                + (corner == opposite));
    }
}
```

app/Main.java, en cinco tramos seguidos dentro de la misma clase (cambian los imports, la creación de la salida y del menú, la suscripción de las vistas, `optionsOf` y el método nuevo `chooseOutput`):

Main.java (1/5: imports)

```
package emberdeep.app;

import emberdeep.engine.achievement.AchievementTracker;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.event.EventBus;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.Dungeon;
import emberdeep.engine.world.DungeonBuilder;
import emberdeep.engine.world.Floor;
import emberdeep.engine.world.biome.BiomeFactory;
import emberdeep.engine.world.biome.CavernBiome;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.MapData;
import emberdeep.engine.world.map.MapParser;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.ConsoleMenu;
import emberdeep.ui.input.ConsoleInput;
import emberdeep.ui.input.InputSource;
import emberdeep.ui.input.ScriptInput;
import emberdeep.ui.output.AnsiOutput;
import emberdeep.ui.output.Output;
import emberdeep.ui.output.PlainOutput;
import emberdeep.ui.view.HudView;
import emberdeep.ui.view.LogView;
import emberdeep.ui.view.MapView;
import java.io.IOException;
import java.nio.file.Path;
import java.util.Arrays;
import java.util.List;
import java.util.Random;
import java.util.Scanner;
```

Main.java (2/5: arranque y entrada)

```
public class Main {
    private static final Skill POWER_STRIKE =
            new Skill("Golpe poderoso", 12, DamageType.PHYSICAL);
    private static final Skill FLARE =
            new Skill("Llamarada", 10, DamageType.FIRE);

    private final GameConfig config;

    private Main(GameConfig config) {
        this.config = config;
    }

    public static void main(String[] args) throws IOException {
        GameConfig config = new GameConfig(42L, 3, new FlatFormula());
        new Main(config).play(args);
    }

    private void play(String[] args) throws IOException {
        String script = scriptOf(args);
        String[] options = optionsOf(args);
        int floorNumber = chooseFloor(options);
        InputSource input = script == null
                ? new ConsoleInput(new Scanner(System.in), System.out)
                : new ScriptInput(Path.of(script), System.out);
        Output output = chooseOutput(args, script != null);
        ConsoleMenu menu = new ConsoleMenu(input, System.out,
                new MapView(output));
        Random random = new Random(config.seed());

        BiomeFactory crypt = new CryptBiome(random);
        BiomeFactory cavern = new CavernBiome(random);
        List<EnemyKind> cryptKinds = List.of(
                EnemyKind.SKELETON, EnemyKind.ASH_RAT, EnemyKind.ZOMBIE);
        List<EnemyKind> cavernKinds = List.of(
                EnemyKind.GOBLIN, EnemyKind.BAT, EnemyKind.SPIDER);
```

Main.java (3/5: mazmorra, partida y modo mapa)

```
        Dungeon dungeon = new DungeonBuilder()
                .floor().biome(crypt).enemies(cryptKinds)
                        .map(MapData.floor(1)).done()
                .floor().biome(cavern).enemies(cavernKinds)
                        .map(MapData.floor(2)).done()
                .floor().biome(crypt).enemies(cryptKinds)
                        .map(MapData.floor(3)).done()
                .build();

        if (options.length > 1 && options[1].equals("map")) {
            explore(menu, dungeon, floorNumber);
            return;
        }
        DamageFormula formula = chooseFormula(menu, options,
                script != null);
        Floor floor = dungeon.floors().get(floorNumber - 1);
        GameSession session = new GameSession(heroes(), floor.biome(),
                floor.enemies(), formula, new DamagePipeline(random, 0.0));
        GameMap entrance = new MapParser(new TileTypes())
                .parse(floor.map());
        session.setLocation(floorNumber,
                entrance.partyX(), entrance.partyY());
        EventBus events = session.events();
        events.subscribe(new LogView(output));
        events.subscribe(new HudView(output));
        // el tracker va el último
        events.subscribe(new AchievementTracker(events));
        menu.run(session);
    }

    private void explore(ConsoleMenu menu, Dungeon dungeon,
                         int firstFloor) {
        TileTypes types = new TileTypes();
        MapParser parser = new MapParser(types);
        int number = firstFloor;
        while (true) {
            Floor floor = dungeon.floors().get(number - 1);
            boolean descended =
                    menu.explore(parser.parse(floor.map()), types);
            if (!descended || number == dungeon.floors().size()) {
                return;
            }
            number++;
        }
    }
```

Main.java (4/5: argumentos, fórmula y planta)

```
    // El guion es el argumento que acaba en .txt
    private static String scriptOf(String[] args) {
        return Arrays.stream(args)
                .filter(arg -> arg.endsWith(".txt"))
                .findFirst().orElse(null);
    }

    private static String[] optionsOf(String[] args) {
        return Arrays.stream(args)
                .filter(arg -> !arg.endsWith(".txt")
                        && !arg.startsWith("--"))
                .toArray(String[]::new);
    }

    private DamageFormula chooseFormula(ConsoleMenu menu, String[] options,
                                        boolean scripted) {
        boolean useDefault = scripted
                || (options.length > 1 && options[1].equals("default"));
        if (useDefault) {
            return config.defaultFormula();
        }
        return menu.askFormula() == 1
                ? new FlatFormula() : new ScaledFormula();
    }

    private int chooseFloor(String[] options) {
        int floor = options.length > 0 ? Integer.parseInt(options[0]) : 1;
        if (floor < 1 || floor > config.floors()) {
            throw new IllegalArgumentException(
                    "La planta debe estar entre 1 y " + config.floors());
        }
        return floor;
    }
```

Main.java (5/5: la salida, héroes y llave final)

```
    // --plain o --ansi mandan; si no, plano con guion y color a mano
    private static Output chooseOutput(String[] args, boolean scripted) {
        List<String> all = Arrays.asList(args);
        boolean plain = all.contains("--plain");
        boolean ansi = all.contains("--ansi");
        if (plain && ansi) {
            throw new IllegalArgumentException(
                    "Elige --plain o --ansi, no las dos");
        }
        if (ansi || (!plain && !scripted)) {
            return new AnsiOutput(System.out);
        }
        return new PlainOutput(System.out);
    }

    private List<Hero> heroes() {
        return List.of(
                new HeroBuilder().name("Aria").stats(new Stats(30, 8, 3))
                        .skill(POWER_STRIKE).skill(FLARE).build(),
                new HeroBuilder().name("Borin").stats(new Stats(40, 6, 5))
                        .skill(POWER_STRIKE).skill(FLARE).build(),
                new HeroBuilder().name("Cael").stats(new Stats(24, 9, 2))
                        .skill(POWER_STRIKE).skill(FLARE).build());
    }
}
```

Parte 3: la comprobación del búfer.

app/Check10c.java (1/2: imports y main)

```java
package emberdeep.app;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.Attack;
import emberdeep.engine.combat.command.Defend;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.ui.output.BufferOutput;
import emberdeep.ui.output.Output;
import emberdeep.ui.output.PlainOutput;
import emberdeep.ui.view.HudView;
import emberdeep.ui.view.LogView;
import java.util.List;
import java.util.Random;

public class Check10c {
    public static void main(String[] args) {
        BufferOutput first = new BufferOutput();
        BufferOutput second = new BufferOutput();
        play(first);
        play(second);
        System.out.println("Iguales: "
                + first.contents().equals(second.contents()));
        play(new PlainOutput(System.out));
    }
```

app/Check10c.java (2/2: play)

```
    // el mismo guion de órdenes y la misma semilla en cada llamada
    private static void play(Output output) {
        Random random = new Random(42);
        Hero aria = new HeroBuilder().name("Aria")
                .stats(new Stats(30, 8, 3)).build();
        GameSession session = new GameSession(List.of(aria),
                new CryptBiome(random), List.of(EnemyKind.SKELETON),
                new FlatFormula(), new DamagePipeline(random, 0.0));
        session.events().subscribe(new LogView(output));
        session.events().subscribe(new HudView(output));

        Combat combat = session.combat();
        Fighter hero = session.heroes().get(0);
        Fighter skeleton = session.enemies().get(0);
        combat.perform(new Attack(combat, hero, skeleton));
        combat.enemyPhase();
        combat.perform(new Defend(combat, hero));
        combat.enemyPhase();
        combat.perform(new Attack(combat, hero, skeleton));
        combat.perform(new Attack(combat, hero, skeleton));
    }
}
```

Por qué funciona (parte 1). `Check10a` recorre las tres salidas con la misma secuencia: texto, texto con estilo, salto de línea. La de texto plano ignora el estilo y escribe `Vida: 5/30`; la de ANSI abre el rojo (`ESC[31m`), escribe `5/30` y cierra (`ESC[0m`), y `cat -v` muestra el carácter ESC como `^[`; el búfer guarda el texto sin estilo. Las tres cumplen el mismo contrato y el bucle no sabe cuál es cuál.

Por qué funciona (parte 2). Las vistas no tienen ninguna referencia a `PrintStream`: escriben en el `Output` que reciben. `LogView` y `HudView` son los dos oyentes del capítulo anterior con otro destino, y con `PlainOutput` dan las mismas bytes porque `line` escribe el texto y un salto `"\n"`, que es lo que escribía `println` en Linux y macOS. La ejecución con guion usa texto plano por defecto, y por eso `a.txt` y `b.txt` (con `--plain`) son iguales. Con `--ansi` solo llevan código las líneas con estilo distinto de `NORMAL`: Zombi a 12/22 (más de la mitad, verde), Esqueleto a 12/18 (verde), el golpe del Zombi a Aria (un héroe golpeado, rojo) y Aria a 29/30 (verde). Las líneas de log con `NORMAL`, como «Aria hace 10 de daño de fuego a Zombi.», no cambian. Los menús y los mensajes de `ConsoleMenu` siguen saliendo por su `PrintStream`, sin estilo. Esa es la frontera de este hito: el puente cubre lo que el motor anuncia y el mapa, no las preguntas del menú. En el modo mapa, `MapView.show` escribe las mismas filas que `GameMap.render()` (los mismos glifos, la `@` en la posición del grupo y un salto al final de cada fila), casilla a casilla con `writeStyled`, y por eso con `--plain` el mapa es el del capítulo 5. Que `Main` lance `IllegalArgumentException` si recibe las dos opciones evita elegir una en silencio.

Por qué funciona (parte 3). En `Check10c`, Aria tiene 30 de vida y 3 de defensa, y con el equipo ataque 10 y armadura 1. Ronda 1: ataca al Esqueleto (18 de vida, defensa 2): $10 - 2 = 8$, queda en 10, y $\lceil 100/18 \rceil = 6$ casillas llenas. El Esqueleto, con ataque 7, le hace $7 - 3 = 4$ por la fórmula y $4 - 1 = 3$ por la armadura: Aria queda en 27 ($\lceil 270/30 \rceil = 9$ casillas). Ronda 2: Aria se cubre (+5 de escudo y +2 de defensa); el golpe del Esqueleto pasa por la fórmula con $7 - 5 = 2$ y el escudo lo absorbe entero, así que el daño final es 0 y se publica el golpe sin daño. Ronda 3: otro ataque de 8 deja al Esqueleto en 2

($\lceil 20/18 \rceil = 2$ casillas) y el siguiente lo remata (vida 0, barra vacía), con la derrota del Esqueleto y la victoria detrás. El texto no depende del azar: el Esqueleto tiene un único objetivo posible, y el `DamagePipeline` se crea con una esquiva del 0 %. Por eso las dos partidas con `BufferOutput` acumulan exactamente el mismo texto y se imprime `Iguales: true`.

Errores típicos:

- Escribir el salto de línea *dentro* del texto con estilo: el código de cierre quedaría después del salto de línea y el color se quedaría activo al empezar la siguiente línea.
- Olvidar `ESC[0m`: el color sigue activo y pinta lo que venga después, también el *prompt* de la terminal.
- Pasar una clase concreta (`AnsiOutput`) donde la vista pide `Output`: la vista deja de poder cambiar de salida y se pierde el puente.
- Dejar un `System.out.println` dentro de una vista: salta el puente y esa línea sale en color aunque pidas `--plain`.
- Contar las opciones `--plain` y `--ansi` como argumentos posicionales: el número de planta dejaría de ser el primer argumento.
- Suscribir el `AchievementTracker` antes que las vistas: el logro saldría antes de la frase del golpe que lo provoca (igual que en el capítulo 9).
- Comparar los búferes con `==` en lugar de `equals`: compara referencias y daría siempre `false`.

Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo 11: `ui/output/` con `Output`, `Style`, `AnsiOutput`, `PlainOutput` y `BufferOutput`; `ui/view/` con `View`, `HudView`, `LogView` y `MapView`; `ui/ConsoleMenu` con tres parámetros de constructor; `ui/listener/` desaparecida; y `Main` con las opciones `--plain` y `--ansi`. Borra `app/Check10a.java`, `app/Check10c.java` y `scripts/check10.txt`.

CAPÍTULO 11

Mediator: la arena que coordina el combate

Objetivos del capítulo

- Reconocer el olor que pide un Mediator: varios objetos que se referencian entre sí, con reglas de coordinación repartidas por todos ellos.
- Contar las dependencias de la versión sin mediador y de la versión con él, y saber cuándo el número mejora y cuándo no.
- Aplicar Mediator: una interfaz pequeña (`TurnMediator`), un mediador concreto (`CombatArena`) y colegas (`Fighter`) que solo conocen la interfaz.
- Distinguir Mediator de Observer por quién decide y en qué dirección van los mensajes.
- Evitar que el mediador se convierta en un «dios» que lo sabe todo.
- Dar el hito H12: provocar, curar a un aliado, invocar refuerzos y enfurecer al que queda, con cuatro interacciones.

Hasta ahora, en Emberdeep, cada combatiente actuaba sobre otro y listo: un ataque, una bomba, una habilidad con un número. Pero tu equipo pide cosas nuevas que afectan a *varios* combatientes a la vez: un guerrero que *provoca* para que todos los enemigos le ataquen a él, una clériga que *cura* a un compañero, un nigromante que *invoca* un esqueleto y un enemigo que *se enfurece* cuando cae uno de sus aliados. Ninguna de estas cosas la puede decidir un combatiente solo: hay que mirar al resto, mover a varios y mantener unas reglas. Y todo eso hay que ponerlo en algún sitio. El patrón que te dice dónde es Mediator: un objeto que conoce a todos los participantes y por el que pasa toda la coordinación, de modo que los participantes ya no tienen que conocerse entre sí. En este capítulo lo construyes con *solo cuatro* interacciones, ves cómo se compara con Observer (con el que se confunde) y aprendes a vigilar su peor defecto: crecer sin límite.

11.1 El olor que pide Mediator: héroes y enemigos con referencias entre sí

Empieza por la solución más directa, la que escribirías sin pensar en patrones: que cada `Fighter` guarde referencias a los demás y se las apañe. Sería algo así (el código es hipotético; no existe en el proyecto):

Hipotético: Fighter con referencias a todos

```
public class Fighter {
    private final List<Fighter> allies = new ArrayList<>();   // su bando
    private final List<Fighter> rivals = new ArrayList<>();   // el contrario
    private Fighter tauntedBy;                // quién me ha provocado

```

```
    public void taunt() {
        for (Fighter rival : rivals) {
            rival.tauntedBy = this;           // toco el estado de otro
        }
    }

    public void healAlly(Fighter ally, int amount) {
        if (allies.contains(ally) && ally.isAlive()) {
            ally.heal(amount);
        }
    }

    public void takeDamage(int amount) {
        hp = Math.max(0, hp - amount);
        if (hp == 0) {
            for (Fighter ally : allies) {     // aviso a todos mis aliados
                ally.rage();
            }
        }
    }

    public void summon(Fighter reinforcement) {
        for (Fighter ally : allies) {         // les cuento que hay uno más
            ally.allies.add(reinforcement);
            reinforcement.allies.add(ally);
        }
        for (Fighter rival : rivals) {
            rival.rivals.add(reinforcement);
            reinforcement.rivals.add(rival);
        }
        allies.add(reinforcement);
        reinforcement.allies.add(this);
    }
}
```

Funciona con dos o tres combatientes, y tiene cuatro problemas que crecen a la vez:

1. **Cada combatiente conoce a todos los demás.** Con N combatientes hay $N(N-1)$ referencias de uno a otro. En un combate de tres héroes y tres enemigos, 30.
2. **Cualquiera puede tocar el estado de cualquiera.** `taunt()` escribe en un campo de otro objeto; `summon` reescribe las listas de todos. Una errata en un `Fighter` rompe el estado de otros cinco.
3. **Las reglas de coordinación están repartidas.** La provocación vive en `taunt()` y en lo que lea `tauntedBy`; el furor, en `takeDamage` y en `rage`; la invocación, en `summon`. Para saber «qué pasa cuando cae un enemigo» hay que leer todo `Fighter`.
4. **Un `Fighter` no se puede probar solo.** Para comprobar el furor necesitas montar un bando entero, con sus listas cruzadas. No hay forma de poner un sustituto.

N	**Sin mediador:** $N(N-1)$	**Con mediador: de los colegas**	**Con mediador: total** $(2N)$
2	2	2	4
3	6	3	6
6	30	6	12
10	90	10	20

La tabla pide honestidad. Con mediador, cada colega guarda *una* referencia (al mediador), pero el mediador guarda N (a todos): en total son $2N$. Con dos combatientes el mediador sale peor (4 frente a 2), con tres empatan, y a partir de ahí la diferencia crece rápido. Pero el número de flechas no es lo importante. Lo importante es *quién conoce a quién*: antes, todos conocían a todos; ahora, cada colega solo conoce una interfaz pequeña, y todo lo demás vive en un único sitio.

Este es el olor: **un grupo de objetos que se referencian entre sí, con reglas de coordinación repartidas entre ellos**. Mediator lo cura sacando la coordinación a un objeto aparte. Su intención en el catálogo es definir un objeto que encapsule cómo interactúan otros, de modo que éstos no se refieran entre sí de forma explícita y puedan variar sus interacciones por separado [9]. Head First lo trata entre los patrones de su apéndice [7], y el catálogo de Refactoring.Guru lo resume como restringir la comunicación directa entre los objetos y obligarlos a colaborar solo a través del mediador [24]. La idea cabe en una frase: los colegas *piden* al mediador en lugar de *hablarse*.

11.2 Piezas del patrón Mediator

Los roles del patrón son cuatro (mediador, mediador concreto, colega y cliente), a los que se suma un colaborador. En Emberdeep, todos tienen nombre propio:

Rol	Clase en el ejemplo	Propósito	Paquete
Mediador (interfaz)	`TurnMediator`	Las cuatro interacciones que un colega puede pedir: provocar, curar, invocar y avisar de una caída	`combat.arena`
Mediador concreto (clase)	`CombatArena`	Conoce a todos los combatientes, aplica las reglas y mueve a varios a la vez	`combat.arena`
Colega (clase)	`Fighter`	Se registra en la arena, solo conoce `TurnMediator` y le avisa cuando cae (`allyDefeated`)	`character`
Clientes (clases)	`UseSkill`, `Combat`	`UseSkill` recibe el `TurnMediator` por constructor y pide `taunt`, `healAlly` o `summon`; `Combat` crea la arena y le consulta `targetsFor`	`combat.command`, `combat`
Colaborador (interfaz)	`BiomeFactory`	La arena la usa en `summon` para crear el refuerzo	`world.biome`

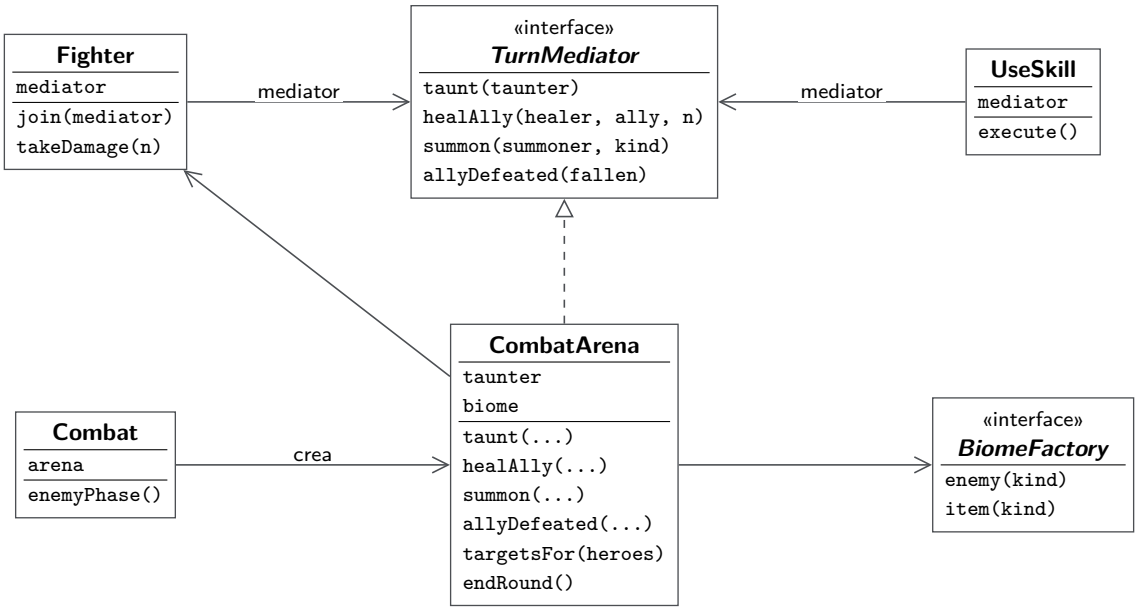


Diagrama UML del patrón Mediator: los colegas (`Fighter`) y los clientes (`UseSkill`) solo conocen la interfaz; la arena los conoce a todos.

Mira las flechas. Ningún `Fighter` apunta a otro `Fighter`: apuntan todos a `TurnMediator`. Y la arena, que sí los conoce, llega a ellos con una asociación que se dibuja desde la arena hacia el colega. Hay otras dos piezas que no son colegas aunque aparezcan en la interfaz de las

habilidades: `Taunt`, `Heal` y `Summon` son *datos* (registros inmutables que dicen qué habilidad es, con sus cifras). No conocen a nadie; es `UseSkill` quien, al leerlos, llama al mediador.

11.3 El ejemplo de Mediator, pieza a pieza: `taunt` y `healAlly`

Las habilidades como datos. Hasta el capítulo anterior, una habilidad era un registro `Skill(name, power, type)`: un nombre, una potencia y un tipo de daño. Provocar, curar e invocar no encajan ahí: una provocación no tiene potencia, una cura tiene una cantidad y una invocación tiene un tipo de enemigo. En lugar de rellenar un registro con campos que unas veces valen y otras no, `Skill` se convierte en una `sealed interface` (la sellada del capítulo 9) con una variante por cada forma:

engine/character/Skill.java

```
package emberdeep.engine.character;

import emberdeep.engine.combat.damage.DamageType;

public sealed interface Skill {
    String name();

    // el antiguo record Skill(name, power, type), con otro nombre
    record Strike(String name, int power, DamageType type)
            implements Skill {}

    record Taunt(String name) implements Skill {}

    record Heal(String name, int amount) implements Skill {}

    record Summon(String name, EnemyKind kind) implements Skill {}
}
```

`Strike` es el antiguo `Skill`, renombrado. Las otras tres son nuevas. Como todas exponen un `name()`, el menú sigue listando habilidades por su nombre sin enterarse del cambio. Pero el renombrado tiene un **coste real** que conviene ver con claridad: todo código que escribía `new Skill(...)` deja de compilar (las plantillas del capítulo 5, los élites, las constantes de `Main`) y todo código que leía `skill.power()` también (`UseSkill`). El compilador te lleva de la mano a cada sitio, y ésa es la ventaja del tipo cerrado, pero el cambio de forma de un tipo que ya usa medio proyecto no es gratis. Se hace aquí, y no desde el capítulo 5, porque entonces habría hecho falta una interfaz sellada que solo tuviera una variante.

La interfaz del mediador. Cuatro métodos, uno por interacción. Es lo único que conocen los colegas y los clientes:

engine/combat/arena/TurnMediator.java

```
package emberdeep.engine.combat.arena;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;

public interface TurnMediator {
    // provocar: esta ronda, los enemigos solo atacan a quien provoca
    void taunt(Fighter taunter);
```

```
9
10      // curar a un aliado; devuelve la vida que ha recuperado de verdad
11      int healAlly(Fighter healer, Fighter ally, int amount);
12
13      // un enemigo invoca un refuerzo del tipo dado y lo devuelve
14      Fighter summon(Fighter summoner, EnemyKind kind);
15
16      // un colega avisa de que ha caído
17      void allyDefeated(Fighter fallen);
18  }
```

Fíjate en que dos métodos *devuelven* algo (la vida recuperada, el refuerzo). Es una de las diferencias con Observer: el que pide espera una respuesta. Y la interfaz es deliberadamente pequeña: si mañana te piden una quinta interacción, será un método nuevo en un único sitio, no una referencia más en cada colega.

El colega. `Fighter` solo gana un campo de tipo `TurnMediator` y un método para recibirlo. Cuando un combatiente cae, avisa; no sabe a quién:

Fragmento de Fighter: el colega

```
1  private TurnMediator mediator;     // null hasta unirse a una arena
2
3  // la arena llama a join al empezar el combate y con cada refuerzo
4  public void join(TurnMediator mediator) {
5      this.mediator = mediator;
6  }
7
8  public void takeDamage(int amount) {
9      boolean wasAlive = hp > 0;
10     hp = Math.max(0, hp - amount);
11     if (wasAlive && hp == 0 && mediator != null) {
12         mediator.allyDefeated(this);
13     }
14 }
```

Dos detalles. El aviso solo sale cuando el combatiente *pasa* de vivo a muerto: repetir un golpe sobre un caído no avisa dos veces. Y la comprobación `mediator != null` permite crear un `Fighter` suelto, sin arena, que es justo lo que necesitas para probarlo solo (es lo que haces con el sensor del ejercicio 11.3).

El mediador concreto. `CombatArena` guarda las listas de héroes y enemigos y la fábrica del bioma. Estos son los dos primeros métodos y la consulta de la provocación:

Fragmento de CombatArena: taunt, healAlly y targetsFor

```
1  private Fighter taunter;                // null si nadie provoca
2
3  @Override
4  public void taunt(Fighter taunter) {
5      this.taunter = taunter;
6  }
7
8  @Override
9  public int healAlly(Fighter healer, Fighter ally, int amount) {
10     if (!ally.isAlive() || !sameSide(healer, ally)) {
```

```
        throw new IllegalArgumentException(
                "Solo se cura a un aliado que sigue vivo");
    }
    return ally.heal(amount);
}

// Con provocación vigente, solo queda quien provoca.
public List<Fighter> targetsFor(List<Fighter> candidates) {
    if (taunter != null && taunter.isAlive()) {
        return List.of(taunter);
    }
    return candidates;
}

private boolean sameSide(Fighter a, Fighter b) {
    return heroes.contains(a) == heroes.contains(b);
}
```

Mira cómo se reparte el trabajo en `healAlly`. Las *reglas de coordinación* (que el curado esté vivo y sea del mismo bando) las pone la arena, porque exigen mirar a dos combatientes. La *aritmética* (curar hasta el máximo y decir cuánto se ha curado) se queda en `Fighter.heal`, que ya existía desde el capítulo 7. Esa frontera es la que evita que el mediador crezca (más en la sección 11.5).

`targetsFor` no es una de las cuatro interacciones: no la llama un colega, la llama `Combat`. Y no cambia las IA del capítulo 2. Antes de que cada enemigo actúe, `Combat` le pasa a la arena la lista de héroes y recibe otra, que puede ser más corta:

Fragmento de Combat: enemyPhase con la arena

```
public void enemyPhase() {
    // se recorre una copia: un enemigo puede invocar refuerzos
    for (Fighter enemy : List.copyOf(enemies)) {
        if (!enemy.isAlive()) {
            continue;
        }
        if (heroesLost()) {
            break;
        }
        enemy.act(this, arena.targetsFor(heroes)).execute();
    }
    // ... fin del combate, ronda, buffs, arena.endRound() y history
}
```

Un ejemplo con números. Borin lanza «Grito de guerra» (una `Taunt`). `UseSkill` llama a `mediator.taunt(borin)`, y la arena apunta a Borin. En la fase enemiga, para la Rata de ceniza, `Combat` pide `arena.targetsFor(heroes)` y obtiene `[Borin]`. La Rata llama a `choose` sobre esa lista de un solo elemento: `LowestHpAi` solo puede elegir a Borin, aunque Aria tenga menos vida. Para el Zombi pasa lo mismo con `FirstAliveAi`. Al terminar la ronda, `Combat` llama a `arena.endRound()`, que olvida la provocación: dura exactamente una ronda.

Las otras dos interacciones. `summon` y `allyDefeated` siguen el mismo patrón. La arena crea el refuerzo con la fábrica del bioma, lo une a sí misma y lo añade al final de la lista de enemigos; y cuando cae un enemigo, elige al primero que siga vivo y le pone un buff de ataque de un turno, con el decorador `TempBuff` del capítulo 6:

Fragmento de CombatArena: summon y allyDefeated

```
@Override
public Fighter summon(Fighter summoner, EnemyKind kind) {
    if (!enemies.contains(summoner)) {
        throw new IllegalArgumentException(
                "Solo los enemigos invocan");
    }
    Fighter reinforcement = Fighter.ofEnemy(biome.enemy(kind));
    reinforcement.join(this);
    enemies.add(reinforcement);
    return reinforcement;
}

@Override
public void allyDefeated(Fighter fallen) {
    if (!enemies.contains(fallen)) {
        return;          // la caída de un héroe no enfurece a nadie
    }
    enemies.stream().filter(Fighter::isAlive).findFirst()
            .ifPresent(enemy ->
                    enemy.addBuff(StatKind.ATTACK, FURY_BONUS, 1));
}
```

Tres cosas que merecen un comentario. Primera: el refuerzo se crea con `biome.enemy(kind)`, que copia la plantilla del `TemplateRegistry` (capítulo 5); por eso el `kind` de una `Summon` tiene que ser un tipo del bioma de la planta, y si no lo es el registro lanza su `IllegalArgumentException`. Segunda: `enemyPhase` recorre una *copia* de la lista de enemigos, porque `summon` añade a la original mientras se recorre, y modificar una lista mientras la recorres lanza `ConcurrentModificationException`, igual que te pasó con el `EventBus` en el capítulo 9. El refuerzo no actúa hasta la ronda siguiente. Tercera: el furor no publica ningún evento; se nota en el daño.

Quién llama a la arena. El comando `UseSkill` (capítulo 7) recibe el `TurnMediator` por constructor y, según la habilidad, hace una cosa u otra. El `switch` sobre una `sealed interface` no necesita `default`: si alguien añade una variante, el compilador marca este sitio.

Fragmento de UseSkill: despachar según la habilidad

```
switch (skill) {
    case Strike strike -> combat.deal(user, target,
            new Damage(strike.power(), strike.type()));
    case Taunt taunt -> {
        mediator.taunt(user);
        combat.publish(new SkillCast(user.name(),
                combat.isHero(user), taunt.name(), user.name()));
    }
    case Heal heal -> {
        int healed = mediator.healAlly(user, target,
                heal.amount());
        combat.publish(new FighterHealed(target.name(),
                combat.isHero(target), healed,
                target.hp(), target.stats().maxHp()));
    }
    case Summon summon -> {
        Fighter reinforcement =
                mediator.summon(user, summon.kind());
```

```
19          combat.publish(new SkillCast(user.name(),
20                  combat.isHero(user), summon.name(),
21                  reinforcement.name()));
22      }
23  }
```

Quién lanza qué. `Taunt` y `Heal` son habilidades de *héroes*: `Main` se las da a dos de ellos (Borin provoca; Aria cura), junto con sus `Strike`. `Summon` es un *dato* de una plantilla de enemigo: el nuevo Nigromante de la cripta, que la lanza su `SkillfulAi`. Esa IA solo usa `Strike` y `Summon`; si le dieras una `Taunt`, lanzaría una excepción (un enemigo no cura ni provoca). Y la IA *no* cambia el modo de elegir objetivo: `choose` sigue igual, y lo que cambia es la lista de héroes que le pasan.

Eventos nuevos. Como `GameEvent` es sellada, curar e invocar necesitan eventos propios, y los publica `UseSkill`: `FighterHealed(name, isHero, amount, hp, maxHp)` tras una cura y `SkillCast(user, userIsHero, skillName, targetName)` al provocar o invocar (`targetName` es el provocador o el refuerzo). El compilador señala los tres `switch` que no tienen `default`: `LogView`, `HudView` y `AchievementTracker`. Es otro coste real de los tipos sellados, el mismo que anunciaba el capítulo 9. Las vistas escriben una línea nueva («Borin recupera 3 de vida.») y el tracker los ignora con una rama vacía.

11.4 Mediator y Observer: quién decide

En Emberdeep ya tienes un objeto en medio que evita que dos partes se conozcan: el `EventBus` del capítulo 9. A primera vista, Mediator y Observer se parecen mucho, y el propio catálogo reconoce que la diferencia entre ambos suele resultar esquiva y que a veces se pueden aplicar los dos a la vez [24]. Lo que los separa es la dirección y el poder:

	Observer	**Mediator**
Dirección	Uno anuncia y muchos escuchan	Muchos piden y uno decide
Quién decide	Cada oyente decide por su cuenta; el emisor no manda sobre ellos	El mediador decide y puede cambiar el estado de varios colegas
Respuesta	Ninguna: el emisor no espera nada	Puede devolver algo (`healAlly` devuelve lo curado)
Quién conoce a quién	El emisor conoce una interfaz de oyente; los oyentes no se conocen entre sí	Los colegas conocen la interfaz del mediador; el mediador conoce a todos los colegas
Altas y bajas	Dinámicas: `subscribe` y `unsubscribe`	Fijas: la arena une a los colegas al empezar y a cada refuerzo
En Emberdeep	`EventBus` y `LogView`	`TurnMediator` y `Fighter`

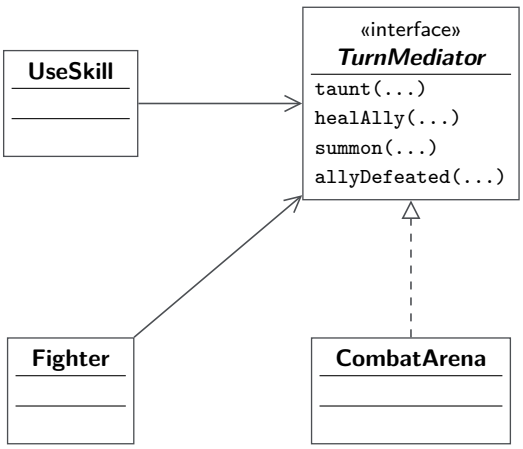


Diagrama UML del patrón Mediator: los que piden solo conocen la interfaz.

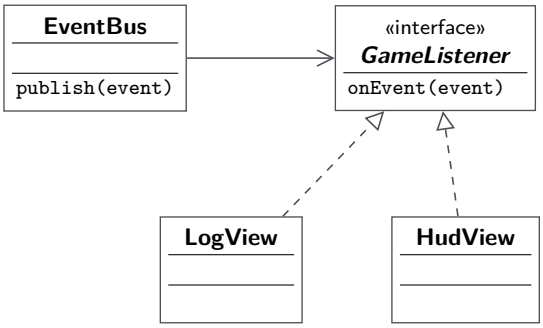


Diagrama UML del patrón Observer: el emisor anuncia a quien se suscribió.

La pregunta práctica es: *¿alguien tiene que decidir algo mirando a varios?* Si lo que tienes es «cuando pase X, que se enteren los interesados» y a ti no te importa qué hagan, es Observer: el `EventBus` difunde un `DamageDealt` y el log, el HUD y los logros reaccionan cada uno a su manera. Si lo que tienes es «para hacer X hay que comprobar reglas y mover a varios», es Mediator: cuando un enemigo cae, *alguien* tiene que decidir cuál de los otros se enfurece. Y los dos conviven sin pisarse. Mira `UseSkill` al curar: pide la cura al mediador (Mediator: hay una regla y una respuesta) y después publica `FighterHealed` (Observer: es un hecho, y que el log o el HUD lo cuenten no cambia nada en el combate). El ejercicio 11.4 te pide aplicar este criterio a cuatro situaciones.

11.5 Mediator: el riesgo del «dios» y cómo evitarlo

El defecto clásico de Mediator es que, como todo pasa por él, acaba sabiéndolo todo. Cada regla nueva «ya que está» se mete en el mediador, y un día el mediador es la clase más grande del proyecto, la que nadie se atreve a tocar. El catálogo lo advierte: con el tiempo, un mediador puede convertirse en un *objeto dios* [24]. Lo que has hecho es sustituir una maraña de referencias por un único nudo, y un nudo enorme sigue siendo un problema. Hay cuatro hábitos que lo evitan, y la arena de Emberdeep los cumple:

- **Pocas interacciones.** La interfaz tiene cuatro métodos. Si cada funcionalidad nueva añade una, tienes una pista de que algo no va bien.
- **El mediador coordina; el colega hace su trabajo.** La arena decide *a quién* curar o *quién* se enfurece; no calcula cuánto cura (eso es `Fighter.heal`), ni cuánto daño hace un golpe (eso es la fórmula y la tubería del capítulo 6).
- **Nada de `if` por tipo de colega.** Si la arena empezara a preguntar «¿eres un esqueleto?» para tratarlo distinto, esa lógica pertenece a una plantilla o a una estrategia, no al mediador.
- **Una frase.** Debes poder describir qué hace el mediador en una frase sin «y además»: «la arena decide los efectos que un combatiente tiene sobre otros». Si necesitas dos frases, probablemente son dos mediadores.

Se queda en el colega o en otra pieza	Va a la arena
Cuánta vida recupera una cura (`Fighter.heal`)	Que la cura solo vaya a un aliado vivo del mismo bando
Cuánto daño hace un golpe (`DamageFormula`, `DamagePipeline`)	Que, mientras alguien provoca, solo se pueda atacar a ese
Cómo se crea un esqueleto (plantilla y `BiomeFactory`)	Que un enemigo invoque y el refuerzo se una al combate
Cuánto dura un buff (`TempBuff`)	Quién se enfurece cuando cae un aliado

11.6 Mediator: cuándo sí y cuándo no

Compensa cuando un grupo de objetos se influyen entre sí de formas que cambian (formularios cuyos campos se activan o se desactivan unos a otros, una sala de chat, el control de una torre de aviones, un combate con provocaciones) y quieres que cada participante sea reutilizable y se pueda probar con un mediador falso. También cuando quieres cambiar las reglas de interacción en un solo sitio.

No compensa si hay dos objetos (un mediador entre dos sale peor que una referencia directa, como viste en la tabla), si las interacciones son pocas y fijas, o si lo que necesitas es difundir hechos sin esperar respuesta, que es Observer. Tiene costes reales:

- **Riesgo de «dios».** Descrito arriba; es el peor.
- **Las reglas están lejos de quien las provoca.** Quien lee `Fighter` no ve que, al caer, otro se enfurece: hay que ir a la arena. El acoplamiento no desaparece, se traslada, y lo que ganas en independencia lo pagas en indirección.
- **Un único punto por el que pasa todo.** Un error en la arena afecta a todos los colegas. Y si crece un segundo grupo de colegas, el mediador se complica.

Mediator no es Facade

Un patrón que verás en el capítulo 14 se parece a éste: Facade también reúne la colaboración de muchas clases en un sitio. La diferencia es que Facade *simplifica* el acceso a un subsistema y el subsistema no sabe que existe, mientras que en Mediator los colegas *conocen* al mediador y hablan con él.

11.7 En el proyecto: H12 con Mediator

Este capítulo avanza el hito H12. Cambios del hito:

- **Nuevo paquete** `combat/arena/`, con `TurnMediator` y `CombatArena`.
- **Cambia la forma de** `Skill`: pasa de `record` a `sealed interface` con `Strike`, `Taunt`, `Heal` y `Summon`. Las plantillas del capítulo 5 (y los élites) y las habilidades de los héroes pasan a ser `Strike`.
- **Nuevo tipo de enemigo**: `EnemyKind.NECROMANCER`, con una plantilla en `CryptBiome` cuya habilidad es una `Summon` de esqueleto. Es el primero que usa la planta 3 (Esqueleto y Nigromante).

- **Cambian** `Fighter` (`join` y el aviso al caer), `Combat` (crea la arena, la consulta con `targetsFor`, recibe la `BiomeFactory`), `UseSkill` (recibe el mediador y despacha con un `switch`), `SkillfulAi`, `GameSession` (lista de enemigos modificable), `ConsoleMenu`, `Main` y `GameEvent`, con `FighterHealed` y `SkillCast`.
- **Se actualizan** los `switch` de `LogView`, `HudView` y `AchievementTracker` (el capítulo 9 lo anunció: lo señala el compilador).

Qué cambia en la partida. Borin gana una tercera habilidad, «Grito de guerra» (`Taunt`), y Aria otra, «Luz curativa» (`Heal` de 8). La planta 3 ya no es una repetición de la 1: lleva un Esqueleto y un Nigromante, que cada tercera ronda invoca otro esqueleto. Y, sobre todo, **cambian algunos números**: cuando cae un enemigo, otro se enfurece y ataca con +2 durante esa fase enemiga. La partida de la cripta con «atacar al primero» pasa de victoria en la ronda 3 con 28, 40 y 22 de vida a la misma victoria con **26, 40 y 20**; la de la caverna, de 30, 40 y 20 a **30, 40 y 18**. Es un cambio de regla deliberado (el furor), no un error.

Límites que esta versión no resuelve (se declaran, no se ocultan):

1. **`restore` descarta los refuerzos.** Al volver a un punto de control con `r`, los refuerzos invocados *después* desaparecen (la instantánea no los conoce, y el `Snapshot` no cambia en este capítulo). Los invocados *antes* sí quedan guardados, porque ya estaban en la lista.
2. **`u` no deshace la provocación ni el furor.** El deshacer del capítulo 7 repone vida, escudo y buffs del objetivo de un comando, pero no toca el estado de la arena. Una provocación o un furor ya concedidos duran hasta que acaba la ronda aunque deshagas la acción que los causó.
3. **El furor no tiene evento.** Se nota en el daño, no en una línea del log.

Las tres partes del ejercicio.

- **Parte 1, la arena.** Las dos piezas nuevas del paquete, el colega (`Fighter`) y los cambios de `Combat` y `GameSession`. Resultado: el proyecto compila y las partidas de «atacar siempre al primero» ya muestran el furor (las vidas nuevas de arriba).
- **Parte 2, las habilidades.** `Skill` sellada, el Nigromante, los eventos nuevos y los tres `switch`, `UseSkill`, `SkillfulAi`, el menú y `Main`. Resultado: el proyecto compila y `./gradlew test` sigue pasando.
- **Parte 3, la comprobación.** Un programa temporal juega tres rondas con una provocación, una cura, un furor y una invocación, y escribe un texto que debes reproducir.

Hito 12: la arena que coordina el combate

Tras este hito, los combatientes no se conocen entre sí: hablan con una arena que aplica las cuatro interacciones y que ve a todos. Conviven un patrón de comportamiento con el que ya tenías (Observer) y uno nuevo (Mediator) sin que ninguno invada al otro. La estructura:

Estructura de carpetas tras el capítulo 11 (1/2)

```
emberdeep/
|-- settings.gradle
|-- build.gradle                    (con JUnit 6.1.3)
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
|-- scripts/
|   `-- demo.txt
`-- src/
    |-- main/java/emberdeep/
    |   |-- app/
    |   |   |-- Main.java              (composition root)
    |   |   `-- GameConfig.java
    |   |-- engine/
    |   |   |-- character/             Stats, Hero, HeroBuilder, Enemy,
    |   |   |                          EnemyKind, Fighter, Skill,
    |   |   |                          EnemyTemplate, TemplateRegistry
    |   |   |-- stats/                 StatSource, StatKind, BaseStats,
    |   |   |                          StatDecorator, WeaponBonus,
    |   |   |                          ArmorBonus, TempBuff
    |   |   |-- combat/
    |   |   |   |-- Combat.java
    |   |   |   |-- ai/                EnemyAi, FirstAliveAi, LowestHpAi,
    |   |   |   |                      RandomTargetAi, SkillfulAi
    |   |   |   |-- arena/             TurnMediator, CombatArena
    |   |   |   |-- command/           CombatCommand, Attack, Defend,
    |   |   |   |                      UseItem, UseSkill, CommandHistory
    |   |   |   |-- damage/            DamageFormula, FlatFormula,
    |   |   |   |                      ScaledFormula, Damage, DamageType
    |   |   |   `-- defense/           DamageHandler, DamagePipeline,
    |   |   |                          DodgeHandler, ShieldHandler,
    |   |   |                          ArmorHandler, ResistanceHandler
```

Estructura de carpetas tras el capítulo 11 (2/2)

```
    |   |   |-- item/                  Item, ItemKind, Inventory
    |   |   |-- event/                 GameEvent, GameListener, EventBus
    |   |   |-- achievement/           AchievementTracker
    |   |   |-- save/                  Snapshot, FighterState,
    |   |   |                          SnapshotStore
    |   |   |-- session/               GameSession
    |   |   `-- world/
    |   |       |-- Dungeon.java, Floor.java,
    |   |       |   DungeonBuilder.java
    |   |       |-- biome/             BiomeFactory, CryptBiome,
    |   |       |                      CavernBiome
    |   |       `-- map/               TileType, TileTypes, GameMap,
    |   |                              MapParser, MapData
    |   `-- ui/
    |       |-- ConsoleMenu.java
    |       |-- input/                 InputSource, ConsoleInput,
    |       |                          ScriptInput
    |       |-- view/                  View, MapView, HudView, LogView
    |       `-- output/                Output, Style, AnsiOutput,
    |                                  PlainOutput, BufferOutput
    `-- test/java/emberdeep/
        `-- engine/save/               SnapshotTest
```

Lo practicas en el ejercicio 11.5. Antes, cuatro ejercicios más pequeños.

11.8 Ejercicios

Ejercicio 11.1: Una sala de chat sin participantes que se conozcan ★★★☆☆

No es de Emberdeep. Crea una carpeta vacía y, dentro, archivos sin paquete. Vas a escribir una sala de chat con tres participantes que se envían mensajes *solo* a través de un mediador. Ningún participante debe guardar una referencia a otro.

1. `ChatRoom`: interfaz con tres métodos, `void join(User user)`, `void leave(User user)` y `void send(User from, String text)`.
2. `User`: guarda su nombre y un `ChatRoom` que recibe por constructor (`User(String name, ChatRoom room)`). Tiene `String name()`, `void say(String text)`, que pide a la sala que envíe el texto (el remitente es él mismo), y `void receive(String from, String text)`, que imprime `[destinatario] remitente: texto`.
3. `Room`: implementa `ChatRoom`. Guarda la lista de usuarios; `send` entrega el texto a todos los usuarios *menos* al remitente, en el orden en que se unieron.
4. `Main`: crea una sala y tres usuarios (`Ana`, `Beto` y `Carla`), los une en ese orden, hace que Ana diga «Hola», da de baja a Beto y hace que Carla diga «¿Quedamos?».

Compila con `javac *.java` y ejecuta con `java Main`. Está bien si la salida es exactamente:

Salida esperada

```
[Beto] Ana: Hola
[Carla] Ana: Hola
[Ana] Carla: ¿Quedamos?
```

y si `User` no tiene ningún campo de tipo `User` (ni una lista de ellos). Dar de baja a Beto no debe haber exigido tocar el código de Ana ni de Carla.

Pista: *`User` depende de la* interfaz *`ChatRoom`, no de `Room`. Para no entregar al remitente su propio mensaje, compara referencias dentro de `send`.*

Práctica ▪ solución en la pág. 466

Ejercicio 11.2: Tres aparatos que se conocen entre sí ★★★☆☆

No es de Emberdeep. Una casa tiene una calefacción, una ventana y una luz de aviso. Las tres clases se referencian entre sí. Léelas:

Heater.java

```java
public class Heater {
    private Window window;
    private Light light;
    private boolean on;

    public void connect(Window window, Light light) {
        this.window = window;
        this.light = light;
    }

    public boolean isOn() { return on; }

    public void turnOn() {
        if (window.isOpen()) {
            window.close();         // no se calienta con la ventana abierta
        }
        on = true;
        light.refresh();
    }

    public void turnOff() {
        on = false;
        light.refresh();
    }
}
```

Window.java

```
public class Window {
    private Heater heater;
    private Light light;
    private boolean open;

    public void connect(Heater heater, Light light) {
        this.heater = heater;
        this.light = light;
    }

    public boolean isOpen() { return open; }

    public void open() {
        open = true;
        if (heater.isOn()) {
            heater.turnOff();       // ventana abierta: calefacción apagada
        }
        light.refresh();
    }

    public void close() {
        open = false;
        light.refresh();
    }
}
```

Light.java

```
public class Light {
    private Heater heater;
    private Window window;
    private String color = "blanca";

    public void connect(Heater heater, Window window) {
        this.heater = heater;
        this.window = window;
    }

    public String color() { return color; }

    public void refresh() {
        if (window.isOpen()) {
            color = "azul";
        } else if (heater.isOn()) {
            color = "naranja";
        } else {
            color = "blanca";
        }
    }
}
```

Y este es el montaje, con las tres operaciones que se ejecutan:

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Heater heater = new Heater();
        Window window = new Window();
        Light light = new Light();
        heater.connect(window, light);
        window.connect(heater, light);
        light.connect(heater, window);

        heater.turnOn();
        window.open();
        heater.turnOn();
        System.out.println(light.color());
    }
}
```

Responde por escrito, sin ejecutar nada:

1. Dibuja las dependencias: una flecha de una clase a otra por cada campo que apunta a otra de las tres. ¿Cuántas flechas hay? ¿Por qué las clases se conectan con `connect` y no reciben a las otras en el constructor?
2. Sigue las tres operaciones de `Main`. ¿En qué orden se llama a `refresh()` y cuántas veces? ¿Qué imprime el programa?
3. La misma idea («la ventana abierta y la calefacción encendida no pueden convivir») aparece en dos clases. ¿En cuáles? ¿Qué otra regla vive en una tercera?
4. Rediséñalo con un mediador (una cuarta clase que conoce a las tres, mientras que cada una solo conoce al mediador). ¿Cuántas referencias entre aparatos desaparecen? ¿Cuántas referencias hay en total en el nuevo diseño?
5. Se añade un ventilador (`Fan`) que también reacciona a la ventana y a la calefacción. ¿Cuántas referencias nuevas hacen falta con cada diseño?

Está bien si das los números (cuéntalos dos veces), si la respuesta 2 enumera las cinco llamadas en orden y si la respuesta 4 distingue entre las referencias *de los aparatos* y las *totales*.

Pista: *En la pregunta 2, cada llamada a `refresh()` está dentro de un método que alguien ha llamado: apunta la pila de llamadas de cada operación, empezando por la última que entra.*

Lectura de código ▪ solución en la pág. 468

Ejercicio 11.3: Probar un sensor con un mediador falso ★★★☆☆

No es de Emberdeep, pero usa el `build.gradle` con JUnit de la sección 8.6 (guía oficial de JUnit [15]). Un sensor avisa a un mediador cuando la lectura pasa de un límite y cuando vuelve a la normalidad. Estas son las dos piezas, ya completas. Guárdalas en `src/main/java/scratch/`:

src/main/java/scratch/Hub.java

```java
package scratch;

public interface Hub {
    void alert(String sensor, int level);
    void clear(String sensor);
}
```

src/main/java/scratch/Sensor.java

```java
package scratch;

public class Sensor {
    private final String name;
    private final int limit;
    private final Hub hub;
    private boolean alarmed;

    public Sensor(String name, int limit, Hub hub) {
        this.name = name;
        this.limit = limit;
        this.hub = hub;
    }

    public void read(int level) {
        if (level > limit && !alarmed) {
            alarmed = true;
            hub.alert(name, level);
        } else if (level <= limit && alarmed) {
            alarmed = false;
            hub.clear(name);
        }
    }
}
```

Para probar el sensor *sin* un mediador real necesitas un doble de prueba: una clase tuya que implementa `Hub` y se limita a apuntar las llamadas que recibe en una lista. Esta es la plantilla; está sin terminar. Guárdala en `src/test/java/scratch/RecordingHub.java` y añade el método que falta:

src/test/java/scratch/RecordingHub.java (plantilla)

```java
package scratch;

import java.util.ArrayList;
import java.util.List;

class RecordingHub implements Hub {
    final List<String> calls = new ArrayList<>();

    @Override
    public void alert(String sensor, int level) {
        calls.add("alert " + sensor + " " + level);
    }

    // falta clear(String sensor): debe apuntar "clear " + sensor
}
```

Crea `src/test/java/scratch/SensorTest.java` con **tres** pruebas. En cada una, crea un `RecordingHub` y un `Sensor` de nombre `Temp` y límite 50:

1. Leer 30 y luego 50 no avisa a nadie (el límite se supera con más de 50).
2. Leer 60 y luego 70 avisa una sola vez: la lista de llamadas es `alert Temp 60`.
3. Leer 60, luego 40 y luego 30 avisa del aviso y de la vuelta a la normalidad una sola vez cada uno: `alert Temp 60` y `clear Temp`.

Está bien si `./gradlew test` termina con `BUILD SUCCESSFUL` y, además, si quitas a propósito `&& !alarmed` del primer `if` de `read` la segunda prueba falla (compruébalo y vuelve a ponerlo). Cuando termines, borra las carpetas `scratch`.

Pista: *Compara la lista completa con `assertEquals(List.of(...), hub.calls)`; para la primera prueba, `assertTrue(hub.calls.isEmpty())`. Importa `List` en la prueba.*

Test ▪ solución en la pág. 469

Ejercicio 11.4: ¿Mediator u Observer? ★★★★☆

Para cada una de estas cuatro situaciones de Emberdeep decide si la resolverías con Mediator, con Observer o con los dos, y justifica quién toma la decisión. No hay que escribir el código completo, pero sí nombrar las piezas.

A. La barra de vida del HUD se vuelve a dibujar cada vez que alguien recibe daño.

B. Se añade un Sacerdote enemigo que, en su turno, cura al enemigo que más vida haya perdido. Debe averiguar quién es, comprobar que sigue vivo y aplicar la cura.

C. El jugador quiere un sonido cada vez que cae un combatiente y, aparte, un contador de bajas de la partida.

D. Cuando cae un héroe, el log lo cuenta y, además, cada héroe que sigue vivo gana +1 de defensa durante dos rondas.

Está bien si: (1) cada decisión se apoya en una pregunta sobre el problema (quién decide, si hay respuesta, cuántos participantes se tocan), no en el nombre del patrón; (2) para Mediator nombras el mediador, los colegas y qué método usa cada uno, y para Observer el emisor, el evento y los oyentes; (3) en D separas lo que difundirías de lo que coordinarías; (4) para al menos una situación dices qué haría que cambiaras de opinión.

Pista: *Pregúntate: ¿alguien tiene que decidir algo mirando a varios, o solo hay que avisar? ¿El que avisa espera una respuesta?*

Diseño ▪ solución en la pág. 472

Ejercicio 11.5: Hito 12: la arena con sus cuatro interacciones ★★★★☆

Implementa el hito H12 partiendo del estado del capítulo anterior (solución del ejercicio 10.4), en tres partes. Comprueba cada una antes de pasar a la siguiente.

Parte 1 (la arena). Crea el paquete `engine/combat/arena/` y cambia las piezas que lo usan:

1. `TurnMediator`: interfaz con exactamente cuatro métodos: `void taunt(Fighter taunter)`, `int healAlly(Fighter healer, Fighter ally, int amount)`, `Fighter summon(Fighter summoner, EnemyKind kind)` y `void allyDefeated(Fighter fallen)`.
2. `CombatArena implements TurnMediator`, con constructor `CombatArena(List<Fighter> heroes, List<Fighter> enemies, BiomeFactory biome)` que une a todos los combatientes con `join`. `taunt` apunta a quien provoca. `healAlly` lanza `IllegalArgumentException` si el aliado no está vivo o es del otro bando, y si no devuelve `ally.heal(amount)`. `summon` exige que el invocador sea un enemigo, crea el refuerzo con `Fighter.ofEnemy(biome.enemy(kind))` y lo añade a la lista de enemigos y lo une a la arena. `allyDefeated` no hace nada si cae un héroe; si cae un enemigo, pone al primer enemigo vivo un `addBuff(StatKind.ATTACK, 2, 1)`. Además, fuera de la interfaz: `List<Fighter> targetsFor(List<Fighter> candidates)` (la lista con solo el provocador, si hay provocación vigente y sigue vivo; si no, la lista recibida) y `void endRound()` (olvida la provocación).
3. `Fighter`: nuevo campo `TurnMediator mediator` (`null` al principio), `void join(TurnMediator mediator)` y, en `takeDamage`, un aviso `mediator.allyDefeated(this)` cuando el combatiente pasa de vivo a muerto (y solo entonces), si tiene mediador.
4. `Combat`: sexto parámetro de constructor `BiomeFactory biome`; crea la arena; `TurnMediator mediator()` y `boolean isHero(Fighter fighter)`. En `enemyPhase`, recorre una copia de la lista de enemigos (`List.copyOf`), pasa a `act` la lista `arena.targetsFor(heroes)` y llama a `arena.endRound()` al cerrar la ronda.
5. `GameSession`: la lista de enemigos pasa a ser un `ArrayList` (modificable), `Combat` recibe el bioma y `restore` descarta los refuerzos que no estuvieran en la instantánea (con `subList(...).clear()`), manteniendo la excepción si la instantánea es de otra partida.

Está bien si el proyecto compila y, con la cripta y siempre «atacar al primero» (`1` y `1`), la ronda 3 empieza con el HUD de arriba mostrando 26, 40 y 20 de vida y la partida termina en victoria; con la caverna, la ronda 2 empieza con 30, 40 y 18:

Se ejecuta desde emberdeep/ (solo la última línea «Héroes»)

```
$ yes 1 | head -n 20 | ./gradlew -q run --args="1 default" \
    | grep -a -o "Héroes:.*" | tail -n 1
Héroes:   Aria 26/30, Borin 40/40, Cael 20/24
$ yes 1 | head -n 20 | ./gradlew -q run --args="2 default" \
    | grep -a -o "Héroes:.*" | tail -n 1
Héroes:   Aria 30/30, Borin 40/40, Cael 18/24
```

Parte 2 (las habilidades). Cambia la forma de `Skill` y todo lo que depende de ella:

1. `Skill`: `sealed interface` con `String name()` y los cuatro registros anidados `Strike(String name, int power, DamageType type)`, `Taunt(String name)`, `Heal(String name, int amount)` y `Summon(String name, EnemyKind kind)`.
2. `EnemyKind`: nuevo valor `NECROMANCER`, de la cripta. En `CryptBiome`, una plantilla «Nigromante» (14 de vida, 5 de ataque, 1 de defensa, `SkillfulAi` sobre `FirstAliveAi`) con una habilidad `Summon("Alzar esqueleto", EnemyKind.SKELETON)`. Las plantillas existentes, los dos biomas y `EnemyTemplate.asElite` usan `Skill.Strike` donde usaban `Skill`.
3. `GameEvent`: dos records nuevos, `FighterHealed(String name, boolean isHero, int amount, int hp, int maxHp)` y `SkillCast(String user, boolean userIsHero, String skillName, String targetName)`. `LogView` escribe `Borin ↪ recupera 3 de vida.` (estilo `GOOD`) y, para `SkillCast`, `X usa S.` si el objetivo es quien la lanza o `X usa S sobre T.` si no; `HudView` dibuja la barra del curado con `FighterHealed` y no hace nada con `SkillCast`; `AchievementTracker` ignora los dos.
4. `UseSkill`: constructor `UseSkill(Combat combat, TurnMediator mediator, Fighter user, Skill skill, Fighter target)` y un `switch` sin `default`. `Strike`: como antes. `Taunt`: `mediator.taunt(user)` y `SkillCast` con el propio lanzador como objetivo. `Heal`: `mediator.healAlly` y `FighterHealed` con lo curado de verdad y la vida resultante. `Summon`: `mediator.summon` y `SkillCast` con el refuerzo como objetivo.
5. `SkillfulAi`: usa `combat.mediator()` y apunta con un `switch`: `Strike` al objetivo elegido, `Summon` a sí mismo, y `Taunt` y `Heal` lanzan `IllegalStateException`.
6. `ConsoleMenu.chooseSkill`: el objetivo depende de la habilidad (`Strike`: un enemigo; `Heal`: un héroe; `Taunt`: el propio héroe). `Main`: constantes `Strike`; Aria gana «Luz curativa» (`Heal`, 8) y Borin «Grito de guerra» (`Taunt`), detrás de sus dos habilidades de siempre; la planta 3 lleva `SKELETON` y `NECROMANCER`.

Está bien si `./gradlew -q classes` compila y `./gradlew test` sigue pasando (`SnapshotTest` no necesita cambios).

Parte 3 (la comprobación). Escribe un programa temporal, `app/Check12.java`. Crea Aria (30, 8, 3) con la habilidad «Luz curativa» (8) y Borin (40, 6, 5) con «Grito de guerra», una cripta con la semilla 42 y los enemigos `ASH_RAT`, `ZOMBIE` y `NECROMANCER`, la fórmula plana, la esquiva al 0 % y un `LogView` sobre un `PlainOutput`. Juega, en este orden: ronda 1, Borin provoca y después fase enemiga; imprime la vida de Borin. Ronda 2: Aria cura a Borin, Aria ataca a la Rata, imprime el ataque del Zombi, fase enemiga, imprime de nuevo el ataque del Zombi y la vida de Aria. Ronda 3: solo fase enemiga; imprime cuántos enemigos hay y el nombre y la vida del cuarto. Está bien si la salida es exactamente:

Salida esperada de Check12

```
Borin usa Grito de guerra.
Rata de ceniza hace 1 de daño físico a Borin.
Zombi hace 1 de daño físico a Borin.
Nigromante hace 1 de daño físico a Borin.
Borin: 37/40
Borin recupera 3 de vida.
Aria hace 10 de daño físico a Rata de ceniza.
Rata de ceniza queda fuera de combate.
Ataque del Zombi furioso: 7
Zombi hace 3 de daño físico a Aria.
Nigromante hace 1 de daño físico a Aria.
Ataque del Zombi tras la ronda: 5
Aria: 26/30
Zombi hace 6 de daño físico a Aria.
Nigromante usa Alzar esqueleto sobre Esqueleto.
Enemigos: 4
Esqueleto: 18/18
```

Borra `Check12.java` al terminar.

Pista: *Para lanzar una habilidad de un héroe a mano:* `combat.perform(new UseSkill(combat, combat.mediator(), user, skill, target))`. *Las habilidades de un combatiente están en* `user.skills()`.

Proyecto ▪ solución en la pág. 473

11.9 Resumen

Resumen

- El olor de Mediator es un grupo de objetos que se referencian entre sí, con reglas de coordinación repartidas: $N(N-1)$ referencias, estado de unos tocado por otros y un colega imposible de probar solo. Mediator saca la coordinación a un objeto aparte; los colegas guardan una referencia al mediador. El mediador guarda N, así que el total es $2N$: mejora desde tres participantes, y la ganancia real es quién conoce a quién.
- Piezas: el mediador (`TurnMediator`, interfaz), el mediador concreto (`CombatArena`), los colegas (`Fighter`) y los clientes (`UseSkill`, `Combat`). `Taunt`, `Heal` y `Summon` son datos, no colegas. La arena usa la `BiomeFactory` para crear refuerzos.
- Las cuatro interacciones: `taunt`, `healAlly`, `summon` y `allyDefeated`. Dos devuelven algo. La provocación se aplica sin cambiar las IA: `Combat` pasa a `act` la lista que le da `targetsFor`.
- Mediator frente a Observer: Observer anuncia y no espera nada; Mediator recibe peticiones, decide y puede mover a varios. Conviven: `UseSkill` pide al mediador y después publica el hecho.
- El riesgo es el «dios»: un mediador que lo sabe todo. Se evita con pocas interacciones, con la aritmética en el colega y las reglas de coordinación en el mediador, sin `if` por tipo de colega y con una descripción de una sola frase.
- Mediator compensa con un grupo que se influye de formas que cambian; no, con dos

objetos o con difusión de hechos. Cuesta indirección, un punto único por el que pasa todo y reglas lejos de quien las provoca.

- H12: `engine/combat/arena/`; `Skill` es una `sealed interface` con `Strike`, `Taunt`, `Heal` y `Summon`; `GameEvent` gana `FighterHealed` y `SkillCast` (y tres `switch` se actualizan); un Nigromante en la planta 3. Límites declarados: `restore` descarta los refuerzos, `u` no deshace provocación ni furor y el furor no tiene evento.

Soluciones de los ejercicios

Solución del ejercicio 11.1

enunciado en la pág. 457

Cuatro archivos sin paquete, en la misma carpeta.

ChatRoom.java

```java
public interface ChatRoom {
    void join(User user);
    void leave(User user);
    void send(User from, String text);
}
```

User.java

```java
public class User {
    private final String name;
    private final ChatRoom room;

    public User(String name, ChatRoom room) {
        this.name = name;
        this.room = room;
    }

    public String name() {
        return name;
    }

    // pide a la sala que lo envíe; no sabe a quién llega
    public void say(String text) {
        room.send(this, text);
    }

    public void receive(String from, String text) {
        System.out.println("[" + name + "] " + from + ": " + text);
    }
}
```

Room.java

```java
import java.util.ArrayList;
import java.util.List;

public class Room implements ChatRoom {
    private final List<User> users = new ArrayList<>();

    @Override
    public void join(User user) {
        users.add(user);
    }

    @Override
    public void leave(User user) {
        users.remove(user);
    }

    @Override
    public void send(User from, String text) {
        for (User user : users) {
            if (user != from) {
                user.receive(from.name(), text);
            }
        }
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Room room = new Room();
        User ana = new User("Ana", room);
        User beto = new User("Beto", room);
        User carla = new User("Carla", room);
        room.join(ana);
        room.join(beto);
        room.join(carla);

        ana.say("Hola");
        room.leave(beto);
        carla.say("¿Quedamos?");
    }
}
```

Salida

```
[Beto] Ana: Hola
[Carla] Ana: Hola
[Ana] Carla: ¿Quedamos?
```

Por qué funciona. El mediador es `Room`: es el único que conoce la lista de usuarios. Los participantes solo guardan un `ChatRoom`, la interfaz. Cuando Ana habla, `say` pide a la sala

que envíe y la sala recorre su lista en orden, saltándose a Ana (compara referencias con `!=`). Al dar de baja a Beto, solo cambia la lista de la sala: Ana y Carla ni se enteran, y por eso el segundo mensaje llega solo a Ana. Es la tabla del capítulo en pequeño: tres participantes, tres referencias de los colegas al mediador y ninguna entre ellos.

Errores típicos:

- Guardar en `User` una lista de otros usuarios «para mandarles el mensaje»: se vuelve a la maraña de referencias y la sala deja de servir.
- Que `User` reciba un `Room` en lugar de un `ChatRoom`: no se puede sustituir la sala por otra (ni por un doble de prueba).
- Entregar el mensaje también al remitente: Ana se vería su propio «Hola».
- Quitar un usuario de la lista *mientras* se recorre para enviar: lanzaría `ConcurrentModificationException`. Aquí `leave` se llama fuera de `send`.

Solución del ejercicio 11.2

enunciado en la pág. 458

1. Las dependencias. Cada clase tiene dos campos que apuntan a las otras dos:

Dependencias directas entre los tres aparatos

```
Heater ----> Window      Heater ----> Light
Window ----> Heater      Window ----> Light
Light  ----> Heater      Light  ----> Window
```

Son $3 \times 2 = 6$ flechas, que es $N(N-1)$ con $N = 3$. No se pueden pasar por el constructor porque la dependencia es *circular*: para crear la calefacción harían falta la ventana y la luz, que a su vez necesitan la calefacción. Por eso se crean vacías y se conectan después con `connect`.

2. La traza. Llamadas a `refresh()`, en orden:

1. `heater.turnOn()`: la ventana está cerrada, así que `on = true` y se llama a `refresh()` (1): la luz queda `naranja`.
2. `window.open()`: `open = true`; como la calefacción está encendida llama a `heater.turnOff()`, que pone `on = false` y llama a `refresh()` (2): la luz queda `azul`. Al volver, `open()` llama a `refresh()` otra vez (3): sigue `azul`.
3. `heater.turnOn()`: la ventana está abierta, así que llama a `window.close()`, que pone `open = false` y llama a `refresh()` (4): ahora la ventana está cerrada y la calefacción apagada, así que la luz queda `blanca`. Después `on = true` y `refresh()` (5): la luz queda `naranja`.

En total, **cinco** llamadas a `refresh()`, y el programa imprime `naranja`. La llamada (4) es un paso intermedio que nadie quería: una luz que se pone blanca un instante porque dos aparatos se llaman entre sí.

3. Dónde viven las reglas. «Ventana abierta y calefacción encendida no pueden convivir» está escrita dos veces, una en cada sentido: en `Window.open()` (abrir apaga la calefacción) y en `Heater.turnOn()` (encender cierra la ventana). La tercera regla, «de qué color es la luz», vive en `Light.refresh()`. Para entender o cambiar la convivencia hay que leer las tres clases.

4. Con un mediador. Una cuarta clase (por ejemplo, `Home`) conoce a las tres; cada aparato solo conoce a `Home`.

	Sin mediador	Con mediador
Referencias entre aparatos	6	0
Referencias de los aparatos (al mediador)	0	3
Referencias del mediador (a los aparatos)	0	3
Total	6	6

Desaparecen las 6 referencias entre aparatos. Los números totales son iguales con tres aparatos; lo que cambia es que cada aparato solo conoce una interfaz y que las dos reglas de convivencia y la del color pasan a vivir en `Home`, juntas.

5. Un ventilador. Sin mediador, el ventilador necesita una referencia a cada uno de los tres aparatos y cada uno de ellos una al ventilador: $3 + 3 = 6$ referencias nuevas (el total pasa de 6 a $4 \times 3 = 12$). Con mediador, el ventilador guarda una referencia al mediador y el mediador una al ventilador: 2 referencias nuevas (el total pasa de 6 a $2 \times 4 = 8$). Es la tabla del capítulo: $N(N-1)$ crece con el cuadrado y $2N$, con N.

Criterios para valorar. La respuesta 1 da 6 y justifica el `connect`; la 2 enumera cinco llamadas y la salida `naranja`; la 3 nombra `Window` y `Heater` para la regla repetida y `Light` para el color; la 4 distingue 6 que desaparecen de 6 totales (no promete que el total baje con tres aparatos); la 5 da 6 frente a 2.

Errores típicos:

- Decir que con el mediador «hay menos flechas» con tres aparatos: el total es el mismo; lo que se reduce es el acoplamiento de cada aparato.
- Contar solo cuatro llamadas a `refresh()`: olvidar la que ocurre dentro de `heater.turnOff()` al abrir la ventana, o la de `window.close()`.
- Pensar que `connect` es una elección de estilo: viene de la circularidad.

Solución del ejercicio 11.3 enunciado en la pág. 460

Tres archivos de prueba y de apoyo en `src/test/java/scratch/` (y los dos del enunciado en `src/main/java/scratch/`). El doble de prueba completo:

src/test/java/scratch/RecordingHub.java

```
package scratch;

import java.util.ArrayList;
import java.util.List;

class RecordingHub implements Hub {
    final List<String> calls = new ArrayList<>();

    @Override
    public void alert(String sensor, int level) {
        calls.add("alert " + sensor + " " + level);
    }

    @Override
    public void clear(String sensor) {
        calls.add("clear " + sensor);
    }
}
```

Y las tres pruebas:

src/test/java/scratch/SensorTest.java

```java
package scratch;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

import java.util.List;
import org.junit.jupiter.api.Test;

class SensorTest {

    @Test
    void readingsBelowTheLimitMakeNoCalls() {
        RecordingHub hub = new RecordingHub();
        Sensor sensor = new Sensor("Temp", 50, hub);

        sensor.read(30);
        sensor.read(50);

        assertTrue(hub.calls.isEmpty());
    }

    @Test
    void crossingTheLimitAlertsOnlyOnce() {
        RecordingHub hub = new RecordingHub();
        Sensor sensor = new Sensor("Temp", 50, hub);

        sensor.read(60);
        sensor.read(70);

        assertEquals(List.of("alert Temp 60"), hub.calls);
    }

    @Test
    void goingBackBelowTheLimitClearsOnlyOnce() {
        RecordingHub hub = new RecordingHub();
        Sensor sensor = new Sensor("Temp", 50, hub);

        sensor.read(60);
        sensor.read(40);
        sensor.read(30);

        assertEquals(List.of("alert Temp 60", "clear Temp"), hub.calls);
    }
}
```

Por qué funciona. El `Sensor` es un colega: solo conoce la interfaz `Hub`. Eso permite pasarle un sustituto que no hace nada útil salvo apuntar qué se le ha pedido, y comprobar con `assertEquals` que el sensor pidió exactamente lo que debía y en el orden esperado. No hace falta montar un mediador real ni los otros colegas: la prueba es corta, rápida y falla por una sola razón. La primera lectura (50) comprueba el borde: el sensor avisa con *más* de 50, no con 50. Si quitas `&& !alarmed`, la lectura de 70 vuelve a avisar y la lista tiene dos `alert`: falla la segunda prueba, y la tercera también, porque la lista de llamadas ya no coincide. Con el `Fighter` de Emberdeep se haría igual: `join` recibe un `TurnMediator`

falso que apunta `allyDefeated`, y una prueba comprueba que el aviso sale una sola vez, cuando la vida llega a 0. Mediator hace posible esta prueba porque el colega depende de una interfaz y no de una arena.

Errores típicos:

- Dejar los archivos de prueba en `src/main/java`: Gradle los compila como código del juego y falla por no encontrar `org.junit`.
- Probar el sensor con un mediador real: la prueba fallaría por motivos ajenos al sensor y dejaría de ser una prueba de una sola pieza.
- Comparar las llamadas con `==` en lugar de con `assertEquals`: compara referencias y falla siempre.
- Reutilizar un mismo `RecordingHub` entre pruebas: la lista de una contaminaría la siguiente. Cada prueba prepara el suyo.

Solución del ejercicio 11.4 enunciado en la pág. 462

Solución de referencia.

	Patrón	Piezas	Criterio
A	Observer	Emisor: `Combat` (publica `DamageDealt` en el `EventBus`). Oyente: `HudView`	Se anuncia un hecho y nadie responde; el HUD decide solo cómo pintarlo. Se pueden añadir más oyentes sin tocar al emisor.
B	Mediator	Mediador: `TurnMediator` con un método nuevo (por ejemplo, `healWeakest(healer)`) y `CombatArena`. Colega: `Fighter`. Cliente: el comando de la habilidad	Alguien tiene que mirar a varios (¿quién ha perdido más vida?), comprobar una regla (que siga vivo) y cambiar el estado de otro. Hay respuesta: cuánto se ha curado.
C	Observer	Emisor: `Combat` (publica `FighterDefeated`). Oyentes: una vista de sonido y un contador de bajas	Dos oyentes independientes reaccionan al mismo hecho y a nadie le importa lo que hagan.
D	Los dos	Observer: `FighterDefeated` lo cuenta `LogView`. Mediator: `allyDefeated(fallen)` en la arena, que da +1 de defensa a los héroes vivos con un `TempBuff`	Contar la caída es difundir un hecho (Observer). Dar una bonificación a los demás es una decisión que toca a varios (Mediator).

Qué haría cambiar de opinión. En B, si el Sacerdote no tuviera que decidir a quién curar (curaría siempre a un objetivo que le dan) bastaría un comando y ni siquiera haría falta el mediador. En C, si el contador de bajas tuviera que impedir que cayera alguien por encima de un máximo, ya no sería difundir sino decidir: Mediator. En A, si el HUD tuviera que *pedir* al motor que redibujara otra cosa, hablaríamos de coordinación y no de anuncio.

Criterios para valorar otras soluciones.

- Cada decisión se apoya en si hay una respuesta, en quién decide y en cuántos participantes se tocan, no en «Observer es para eventos».
- Para Mediator se nombra un mediador con métodos, los colegas que lo usan y quién

es el cliente; para Observer, el emisor, el evento y al menos un oyente concreto.

- En D se separan el hecho y la decisión; una respuesta que solo diga «los dos» sin repartir quién hace qué no es suficiente.
- Se evita la solución «un solo mediador que lo hace todo» (el riesgo del «dios»): cada interacción nueva es un método con un propósito claro.

Alternativa válida para B. Hacerlo con Observer: el Sacerdote se suscribe a `DamageDealt`, apunta quién ha perdido más vida y, en su turno, se cura a ése. Es defendible si el Sacerdote es el único que necesita esa información y no hay reglas que comprobar fuera de él. Deja de serlo en cuanto aparezca una segunda regla (por ejemplo, «no curar a un caído» o «solo una cura por ronda»): esas reglas necesitan un sitio común, y el sitio es un mediador.

Errores típicos:

- Responder Mediator a todo porque «hay un objeto en medio»: el `EventBus` también lo es.
- Responder Observer a B porque «alguien se entera de que hay un herido»: se pierde que el que pide una cura espera una respuesta y que alguien debe aplicar reglas.
- En D, resolver todo dentro del `LogView` (que cuenta la caída *y* da la defensa): una vista nunca debería cambiar el combate.

Solución del ejercicio 11.5

enunciado en la pág. 463

Partes 1 a 3 en orden. Se repiten los archivos enteros (también los que solo cambian un poco), para que la solución esté completa. Los archivos que no aparecen (el resto de `engine/`, `ui/input/`, `ui/output/`, `MapView`, `View`, `GameConfig`, `SnapshotTest`...) no cambian respecto del hito anterior.

Parte 1: la arena. Archivos nuevos: `TurnMediator` y `CombatArena`. Cambian `Fighter` (campo `mediator`, `join` y el aviso en `takeDamage`), `Combat` (sexto parámetro del constructor, la arena, `mediator()`, `isHero`, y la fase enemiga con la copia, `targetsFor` y `endRound`) y `GameSession` (lista de enemigos modificable, el bioma para `Combat` y `restore` que descarta refuerzos).

engine/combat/arena/TurnMediator.java

```java
package emberdeep.engine.combat.arena;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;

public interface TurnMediator {
    // provocar: esta ronda, los enemigos solo atacan a quien provoca
    void taunt(Fighter taunter);

    // curar a un aliado; devuelve la vida que ha recuperado de verdad
    int healAlly(Fighter healer, Fighter ally, int amount);

    // un enemigo invoca un refuerzo del tipo dado y lo devuelve
    Fighter summon(Fighter summoner, EnemyKind kind);

    // un colega avisa de que ha caído
    void allyDefeated(Fighter fallen);
}
```

engine/combat/arena/CombatArena.java (1/2)

```
package emberdeep.engine.combat.arena;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.stats.StatKind;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.List;

public class CombatArena implements TurnMediator {
    private static final int FURY_BONUS = 2;

    private final List<Fighter> heroes;
    private final List<Fighter> enemies;    // la lista viva de Combat
    private final BiomeFactory biome;
    private Fighter taunter;                // null si nadie provoca

    public CombatArena(List<Fighter> heroes, List<Fighter> enemies,
                       BiomeFactory biome) {
        this.heroes = heroes;
        this.enemies = enemies;
        this.biome = biome;
        heroes.forEach(fighter -> fighter.join(this));
        enemies.forEach(fighter -> fighter.join(this));
    }

    @Override
    public void taunt(Fighter taunter) {
        this.taunter = taunter;
    }

    @Override
    public int healAlly(Fighter healer, Fighter ally, int amount) {
        if (!ally.isAlive() || !sameSide(healer, ally)) {
            throw new IllegalArgumentException(
                    "Solo se cura a un aliado que sigue vivo");
        }
        return ally.heal(amount);
    }

    @Override
    public Fighter summon(Fighter summoner, EnemyKind kind) {
        if (!enemies.contains(summoner)) {
            throw new IllegalArgumentException(
                    "Solo los enemigos invocan");
        }
        Fighter reinforcement = Fighter.ofEnemy(biome.enemy(kind));
        reinforcement.join(this);
        enemies.add(reinforcement);
        return reinforcement;
    }
```

engine/combat/arena/CombatArena.java (2/2)

```
    @Override
    public void allyDefeated(Fighter fallen) {
        if (!enemies.contains(fallen)) {
            return;          // la caída de un héroe no enfurece a nadie
        }
        enemies.stream().filter(Fighter::isAlive).findFirst()
                .ifPresent(enemy ->
                        enemy.addBuff(StatKind.ATTACK, FURY_BONUS, 1));
    }

    // No es una interacción de los colegas: la consulta Combat antes de
    // que cada enemigo actúe. Con provocación vigente, solo queda quien
    // provoca.
    public List<Fighter> targetsFor(List<Fighter> candidates) {
        if (taunter != null && taunter.isAlive()) {
            return List.of(taunter);
        }
        return candidates;
    }

    // Tampoco lo es: Combat la llama al cerrar la ronda
    public void endRound() {
        taunter = null;
    }

    private boolean sameSide(Fighter a, Fighter b) {
        return heroes.contains(a) == heroes.contains(b);
    }
}
```

engine/character/Fighter.java (1/3)

```
package emberdeep.engine.character;

import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.ai.EnemyAi;
import emberdeep.engine.combat.arena.TurnMediator;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.stats.BaseStats;
import emberdeep.engine.stats.StatKind;
import emberdeep.engine.stats.StatSource;
import emberdeep.engine.stats.TempBuff;
import java.util.List;

public class Fighter {
    private final String name;
    private final EnemyKind kind;      // null en los héroes
    private final StatSource gear;     // equipo permanente
    private StatSource stats;          // gear más los buffs del combate
    private final EnemyAi ai;          // null en los héroes
    private final List<Skill> skills;
    private int hp;
    private int shield;
    private TurnMediator mediator;     // null hasta unirse a una arena

    private Fighter(String name, EnemyKind kind, StatSource gear,
                    EnemyAi ai, List<Skill> skills) {
        this.name = name;
        this.kind = kind;
        this.gear = gear;
        this.stats = gear;
        this.ai = ai;
        this.skills = skills;
        this.hp = gear.maxHp();
    }

    public static Fighter ofHero(Hero hero, StatSource gear) {
        return new Fighter(hero.name(), null, gear, null, hero.skills());
    }

    public static Fighter ofEnemy(Enemy enemy) {
        return new Fighter(enemy.name(), enemy.kind(),
                new BaseStats(enemy.stats()), enemy.ai(), enemy.skills());
    }
```

engine/character/Fighter.java (2/3)

```
    public String name() { return name; }
    public StatSource stats() { return stats; }
    public StatSource gear() { return gear; }
    public List<Skill> skills() { return skills; }
    public int hp() { return hp; }
    public boolean isAlive() { return hp > 0; }
    public int shield() { return shield; }

    public void addShield(int amount) {
        shield += amount;
    }

    public int absorbShield(int damage) {
        int absorbed = Math.min(shield, damage);
        shield -= absorbed;
        return damage - absorbed;
    }

    public boolean resists(DamageType type) {
        return kind != null && kind.resistances().contains(type);
    }

    public void takeDamage(int amount) {
        boolean wasAlive = hp > 0;
        hp = Math.max(0, hp - amount);
        if (wasAlive && hp == 0 && mediator != null) {
            mediator.allyDefeated(this);
        }
    }

    // la arena llama a join al empezar el combate y con cada refuerzo
    public void join(TurnMediator mediator) {
        this.mediator = mediator;
    }

    public int heal(int amount) {
        int healed = Math.min(amount, stats.maxHp() - hp);
        hp += healed;
        return healed;
    }

    // fija a la vez los tres números que apuntan los comandos
    public void restore(int hp, int shield, StatSource stats) {
        this.hp = hp;
        this.shield = shield;
        this.stats = stats;
    }
```

engine/character/Fighter.java (3/3)

```
    public void addBuff(StatKind stat, int bonus, int turns) {
        stats = new TempBuff(stats, stat, bonus, turns);
    }

    public void tickBuffs() {
        stats = stats.tick();
    }

    public CombatCommand act(Combat combat, List<Fighter> heroes) {
        if (ai == null) {
            throw new IllegalStateException(
                    name + " no tiene IA: es un héroe");
        }
        return ai.act(combat, this, heroes);
    }
}
```

engine/combat/Combat.java (1/3)

```
package emberdeep.engine.combat;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.arena.CombatArena;
import emberdeep.engine.combat.arena.TurnMediator;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.command.CommandHistory;
import emberdeep.engine.combat.damage.Damage;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.event.EventBus;
import emberdeep.engine.event.GameEvent;
import emberdeep.engine.event.GameEvent.CombatEnded;
import emberdeep.engine.event.GameEvent.DamageDealt;
import emberdeep.engine.event.GameEvent.DamageDodged;
import emberdeep.engine.event.GameEvent.FighterDefeated;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.List;

public class Combat {
    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final DamageFormula formula;
    private final DamagePipeline pipeline;
    private final EventBus bus;
    private final CommandHistory history = new CommandHistory();
    private final CombatArena arena;
    private int round = 1;
    private boolean ended;

    public Combat(List<Fighter> heroes, List<Fighter> enemies,
                  DamageFormula formula, DamagePipeline pipeline,
                  EventBus bus, BiomeFactory biome) {
        this.heroes = heroes;
        this.enemies = enemies;
        this.formula = formula;
        this.pipeline = pipeline;
        this.bus = bus;
        this.arena = new CombatArena(heroes, enemies, biome);
    }

    public int round() {
        return round;
    }

    public boolean heroesWon() {
        return enemies.stream().noneMatch(Fighter::isAlive);
    }
```

engine/combat/Combat.java (2/3)

```
    public boolean heroesLost() {
        return heroes.stream().noneMatch(Fighter::isAlive);
    }

    public boolean isOver() {
        return heroesWon() || heroesLost();
    }

    public void restoreRound(int round) {
        if (round < 1) {
            throw new IllegalArgumentException("La ronda empieza en 1");
        }
        this.round = round;
    }

    // cap. 11: los comandos reciben el mediador de aquí
    public TurnMediator mediator() {
        return arena;
    }

    public boolean isHero(Fighter fighter) {
        return heroes.contains(fighter);
    }

    // cap. 9: los comandos publican por aquí
    public void publish(GameEvent event) {
        bus.publish(event);
    }

    // los héroes ejecutan sus comandos por aquí
    public void perform(CombatCommand command) {
        history.run(command);
        checkEnd();
    }

    public boolean undo() {
        return history.undo();
    }

    public int damageOf(Fighter attacker, Fighter target) {
        return formula.damage(attacker.stats().attack(),
                              target.stats().defense());
    }
```

engine/combat/Combat.java (3/3)

```
    public int deal(Fighter attacker, Fighter target, Damage damage) {
        int dealt = pipeline.apply(target, damage);
        target.takeDamage(dealt);
        boolean targetIsHero = isHero(target);
        if (dealt == 0) {
            publish(new DamageDodged(attacker.name(), target.name(),
                    targetIsHero));
        } else {
            publish(new DamageDealt(attacker.name(), target.name(),
                    targetIsHero, dealt, damage.type(),
                    target.hp(), target.stats().maxHp()));
        }
        if (!target.isAlive()) {
            publish(new FighterDefeated(target.name(), targetIsHero));
        }
        return dealt;
    }

    public void enemyPhase() {
        // se recorre una copia: un enemigo puede invocar refuerzos
        for (Fighter enemy : List.copyOf(enemies)) {
            if (!enemy.isAlive()) {
                continue;
            }
            if (heroesLost()) {
                break;
            }
            enemy.act(this, arena.targetsFor(heroes)).execute();
        }
        checkEnd();
        round++;
        heroes.forEach(Fighter::tickBuffs);
        enemies.forEach(Fighter::tickBuffs);
        arena.endRound();
        history.clear();
    }

    // publica el fin del combate una sola vez
    private void checkEnd() {
        if (ended || !isOver()) {
            return;
        }
        ended = true;
        publish(new CombatEnded(heroesWon(),
                heroes.stream().map(Fighter::hp).toList()));
    }
}
```

engine/session/GameSession.java (1/3)

```
package emberdeep.engine.session;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.event.EventBus;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.save.FighterState;
import emberdeep.engine.save.Snapshot;
import emberdeep.engine.stats.ArmorBonus;
import emberdeep.engine.stats.BaseStats;
import emberdeep.engine.stats.StatSource;
import emberdeep.engine.stats.WeaponBonus;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Stream;

public class GameSession {
    private static final int SWORD_ATTACK = 2;
    private static final int ARMOR_VALUE = 1;

    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final Combat combat;
    private final EventBus events = new EventBus();
    private Inventory inventory;
    private int floor = 1;
    private int x;
    private int y;
```

engine/session/GameSession.java (2/3)

```
    public GameSession(List<Hero> heroes, BiomeFactory biome,
                       List<EnemyKind> kinds, DamageFormula formula,
                       DamagePipeline pipeline) {
        this.heroes = heroes.stream()
                .map(hero -> Fighter.ofHero(hero, gearOf(hero)))
                .toList();
        // mutable: la arena añade los refuerzos al final de la lista
        this.enemies = new ArrayList<>(kinds.stream()
                .map(biome::enemy)
                .map(Fighter::ofEnemy)
                .toList());
        this.combat = new Combat(this.heroes, this.enemies,
                formula, pipeline, events, biome);
        this.inventory = new Inventory(List.of())
                .with(biome.item(ItemKind.HEALING))
                .with(biome.item(ItemKind.HEALING))
                .with(biome.item(ItemKind.BOMB));
    }

    private static StatSource gearOf(Hero hero) {
        return new ArmorBonus(
                new WeaponBonus(new BaseStats(hero.stats()), SWORD_ATTACK),
                ARMOR_VALUE);
    }

    public List<Fighter> heroes() { return heroes; }
    public List<Fighter> enemies() { return enemies; }
    public Combat combat() { return combat; }
    public EventBus events() { return events; }
    public int round() { return combat.round(); }
    public Inventory inventory() { return inventory; }
    public int floor() { return floor; }
    public int x() { return x; }
    public int y() { return y; }

    public void replaceInventory(Inventory next) {
        this.inventory = next;
    }

    public void setLocation(int floor, int x, int y) {
        this.floor = floor;
        this.x = x;
        this.y = y;
    }

    private List<Fighter> allFighters() {
        return Stream.concat(heroes.stream(), enemies.stream()).toList();
    }
```

engine/session/GameSession.java (3/3)

```
    public Snapshot createSnapshot() {
        List<FighterState> states = allFighters().stream()
                .map(f -> new FighterState(f.hp(), f.shield(), f.stats()))
                .toList();
        return new Snapshot(floor, x, y, combat.round(), states,
                inventory);
    }

    public void restore(Snapshot snapshot) {
        int saved = snapshot.fighters().size();
        if (saved < heroes.size() || saved > allFighters().size()) {
            throw new IllegalArgumentException(
                    "La instantánea es de otra partida");
        }
        // cap. 11: los refuerzos invocados después del punto de control
        // se descartan (siempre están al final de la lista de enemigos)
        enemies.subList(saved - heroes.size(), enemies.size()).clear();
        List<Fighter> all = allFighters();
        for (int i = 0; i < all.size(); i++) {
            FighterState state = snapshot.fighters().get(i);
            all.get(i).restore(state.hp(), state.shield(), state.stats());
        }
        combat.restoreRound(snapshot.round());
        inventory = snapshot.inventory();
        floor = snapshot.floor();
        x = snapshot.x();
        y = snapshot.y();
    }
}
```

Parte 2: las habilidades. Nuevos valores o variantes: `Skill` sellada, `EnemyKind.NECROMANCER` y los dos eventos. Cambian las plantillas (`Skill.Strike`) y la del Nigromante, `UseSkill` y `SkillfulAi` (el compilador marca ambos en cuanto `Skill` deja de ser un registro), los tres `switch` sobre `GameEvent`, `ConsoleMenu.chooseSkill` y `Main`.

engine/character/Skill.java

```java
package emberdeep.engine.character;

import emberdeep.engine.combat.damage.DamageType;

public sealed interface Skill {
    String name();

    // el antiguo record Skill(name, power, type), con otro nombre
    record Strike(String name, int power, DamageType type)
            implements Skill {}

    record Taunt(String name) implements Skill {}

    record Heal(String name, int amount) implements Skill {}

    record Summon(String name, EnemyKind kind) implements Skill {}
}
```

engine/character/EnemyKind.java

```java
package emberdeep.engine.character;

import emberdeep.engine.combat.damage.DamageType;
import java.util.Set;

public enum EnemyKind {
    SKELETON, ASH_RAT(Set.of(DamageType.FIRE)), ZOMBIE,    // cripta
    NECROMANCER,                                           // cripta
    GOBLIN, BAT, SPIDER;                                   // caverna

    private final Set<DamageType> resistances;

    EnemyKind() {
        this(Set.of());
    }

    EnemyKind(Set<DamageType> resistances) {
        this.resistances = resistances;
    }

    public Set<DamageType> resistances() {
        return resistances;
    }
}
```

engine/character/EnemyTemplate.java (1/2)

```
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;
import emberdeep.engine.combat.damage.DamageType;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class EnemyTemplate {
    private final EnemyKind kind;
    private String name;
    private Stats stats;
    private final List<Skill> skills;
    private final EnemyAi ai;

    public EnemyTemplate(EnemyKind kind, String name, Stats stats,
                         EnemyAi ai) {
        this(kind, name, stats, new ArrayList<>(), ai);
    }

    private EnemyTemplate(EnemyKind kind, String name, Stats stats,
                          List<Skill> skills, EnemyAi ai) {
        this.kind = kind;
        this.name = name;
        this.stats = stats;
        this.skills = skills;
        this.ai = ai;
    }

    public EnemyTemplate copy() {
        return new EnemyTemplate(kind, name, stats,
                new ArrayList<>(skills), ai);
    }

    public Enemy toEnemy() {
        return new Enemy(name, kind, stats, ai, List.copyOf(skills));
    }

    public EnemyTemplate asElite() {
        EnemyTemplate elite = copy();
        elite.setName("Élite " + name);
        elite.setStats(new Stats(stats.maxHp() * 3 / 2,
                stats.attack() + 2, stats.defense() + 1));
        elite.addSkill(new Skill.Strike("Furia", 12,
                DamageType.PHYSICAL));
        return elite;
    }
```

engine/character/EnemyTemplate.java (2/2)

```
    public EnemyTemplate addSkill(Skill skill) {
        skills.add(skill);
        return this;
    }

    public void setName(String name) { this.name = name; }
    public void setStats(Stats stats) { this.stats = stats; }

    public EnemyKind kind() { return kind; }
    public String name() { return name; }
    public Stats stats() { return stats; }
    public EnemyAi ai() { return ai; }

    public List<Skill> skills() {
        return Collections.unmodifiableList(skills);
    }
}
```

engine/world/biome/CryptBiome.java (1/2)

```java
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.EnemyTemplate;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Stats;
import emberdeep.engine.character.TemplateRegistry;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import emberdeep.engine.combat.ai.SkillfulAi;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import java.util.Random;

public class CryptBiome implements BiomeFactory {
    private final TemplateRegistry registry = new TemplateRegistry();

    public CryptBiome(Random random) {
        registry.register(new EnemyTemplate(EnemyKind.SKELETON,
                "Esqueleto", new Stats(18, 7, 2),
                new RandomTargetAi(random))
                .addSkill(new Skill.Strike("Golpe de hueso", 9,
                        DamageType.PHYSICAL)));
        registry.register(new EnemyTemplate(EnemyKind.ASH_RAT,
                "Rata de ceniza", new Stats(8, 5, 0), new LowestHpAi())
                .addSkill(new Skill.Strike("Mordisco sucio", 6,
                        DamageType.PHYSICAL)));
        registry.register(new EnemyTemplate(EnemyKind.ZOMBIE, "Zombi",
                new Stats(22, 5, 1), new SkillfulAi(new FirstAliveAi()))
                .addSkill(new Skill.Strike("Abrazo pútrido", 7,
                        DamageType.PHYSICAL)));
        // cap. 11: la primera plantilla con una habilidad Summon
        registry.register(new EnemyTemplate(EnemyKind.NECROMANCER,
                "Nigromante", new Stats(14, 5, 1),
                new SkillfulAi(new FirstAliveAi()))
                .addSkill(new Skill.Summon("Alzar esqueleto",
                        EnemyKind.SKELETON)));
    }

    @Override
    public Enemy enemy(EnemyKind kind) {
        return registry.get(kind).copy().toEnemy();
    }
```

engine/world/biome/CryptBiome.java (2/2)

```
    @Override
    public Item item(ItemKind kind) {
        return switch (kind) {
            case HEALING -> new Item("Poción de cripta", kind, 12);
            case BOMB -> new Item("Bomba de ceniza", kind, 10);
        };
    }
}
```

engine/world/biome/CavernBiome.java

```
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.EnemyTemplate;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Stats;
import emberdeep.engine.character.TemplateRegistry;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import java.util.Random;

public class CavernBiome implements BiomeFactory {
    private final TemplateRegistry registry = new TemplateRegistry();

    public CavernBiome(Random random) {
        registry.register(new EnemyTemplate(EnemyKind.GOBLIN, "Goblin",
                new Stats(14, 6, 1), new FirstAliveAi())
                .addSkill(new Skill.Strike("Puñalada", 8,
                        DamageType.PHYSICAL)));
        registry.register(new EnemyTemplate(EnemyKind.BAT, "Murciélago",
                new Stats(10, 6, 0), new LowestHpAi())
                .addSkill(new Skill.Strike("Chillido", 5,
                        DamageType.PHYSICAL)));
        registry.register(new EnemyTemplate(EnemyKind.SPIDER, "Araña",
                new Stats(16, 7, 2), new RandomTargetAi(random))
                .addSkill(new Skill.Strike("Picadura", 9,
                        DamageType.PHYSICAL)));
    }

    @Override
    public Enemy enemy(EnemyKind kind) {
        return registry.get(kind).copy().toEnemy();
    }

    @Override
    public Item item(ItemKind kind) {
        return switch (kind) {
            case HEALING -> new Item("Poción de caverna", kind, 10);
            case BOMB -> new Item("Bomba de roca", kind, 14);
        };
    }
}
```

engine/event/GameEvent.java

```
package emberdeep.engine.event;

import emberdeep.engine.combat.damage.DamageType;
import java.util.List;

public sealed interface GameEvent {
    record DamageDealt(String attacker, String target,
                       boolean targetIsHero, int amount, DamageType type,
                       int targetHp, int targetMaxHp)
            implements GameEvent {}

    record DamageDodged(String attacker, String target,
                        boolean targetIsHero) implements GameEvent {}

    record FighterDefeated(String name, boolean isHero)
            implements GameEvent {}

    record Defended(String name, int shield, int defenseBonus)
            implements GameEvent {}

    record ItemUsed(String user, String target, String itemName,
                    int targetHp, int targetMaxHp) implements GameEvent {}

    record CombatEnded(boolean heroesWon, List<Integer> heroesHp)
            implements GameEvent {
        public CombatEnded {
            heroesHp = List.copyOf(heroesHp);
        }
    }

    record AchievementUnlocked(String name) implements GameEvent {}

    // cap. 11: una cura de la habilidad Heal (con la vida ya repuesta)
    record FighterHealed(String name, boolean isHero, int amount,
                         int hp, int maxHp) implements GameEvent {}

    // cap. 11: Taunt y Summon; targetName es el provocador o el refuerzo
    record SkillCast(String user, boolean userIsHero, String skillName,
                     String targetName) implements GameEvent {}
}
```

engine/combat/command/UseSkill.java (1/2)

```
package emberdeep.engine.combat.command;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Skill.Heal;
import emberdeep.engine.character.Skill.Strike;
import emberdeep.engine.character.Skill.Summon;
import emberdeep.engine.character.Skill.Taunt;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.arena.TurnMediator;
import emberdeep.engine.combat.damage.Damage;
import emberdeep.engine.event.GameEvent.FighterHealed;
import emberdeep.engine.event.GameEvent.SkillCast;
import emberdeep.engine.stats.StatSource;

public class UseSkill implements CombatCommand {
    private final Combat combat;
    private final TurnMediator mediator;
    private final Fighter user;
    private final Skill skill;
    private final Fighter target;
    private int hpBefore;
    private int shieldBefore;
    private StatSource statsBefore;

    public UseSkill(Combat combat, TurnMediator mediator, Fighter user,
                    Skill skill, Fighter target) {
        this.combat = combat;
        this.mediator = mediator;
        this.user = user;
        this.skill = skill;
        this.target = target;
    }
```

engine/combat/command/UseSkill.java (2/2)

```
    @Override
    public void execute() {
        hpBefore = target.hp();
        shieldBefore = target.shield();
        statsBefore = target.stats();
        switch (skill) {
            case Strike strike -> combat.deal(user, target,
                    new Damage(strike.power(), strike.type()));
            case Taunt taunt -> {
                mediator.taunt(user);
                combat.publish(new SkillCast(user.name(),
                        combat.isHero(user), taunt.name(), user.name()));
            }
            case Heal heal -> {
                int healed = mediator.healAlly(user, target,
                        heal.amount());
                combat.publish(new FighterHealed(target.name(),
                        combat.isHero(target), healed,
                        target.hp(), target.stats().maxHp()));
            }
            case Summon summon -> {
                Fighter reinforcement =
                        mediator.summon(user, summon.kind());
                combat.publish(new SkillCast(user.name(),
                        combat.isHero(user), summon.name(),
                        reinforcement.name()));
            }
        }
    }

    @Override
    public void undo() {
        target.restore(hpBefore, shieldBefore, statsBefore);
    }
}
```

engine/combat/ai/SkillfulAi.java (1/2)

```java
package emberdeep.engine.combat.ai;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Skill.Heal;
import emberdeep.engine.character.Skill.Strike;
import emberdeep.engine.character.Skill.Summon;
import emberdeep.engine.character.Skill.Taunt;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.Attack;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.command.UseSkill;
import java.util.List;

public class SkillfulAi implements EnemyAi {
    private final EnemyAi targeting;

    public SkillfulAi(EnemyAi targeting) {
        this.targeting = targeting;
    }

    @Override
    public Fighter choose(Fighter self, List<Fighter> heroes) {
        return targeting.choose(self, heroes);
    }

    @Override
    public CombatCommand act(Combat combat, Fighter self,
                             List<Fighter> heroes) {
        Fighter target = choose(self, heroes);
        if (combat.round() % 3 == 0 && !self.skills().isEmpty()) {
            Skill skill = self.skills().get(0);
            return new UseSkill(combat, combat.mediator(), self, skill,
                    aimOf(skill, self, target));
        }
        return new Attack(combat, self, target);
    }
```

engine/combat/ai/SkillfulAi.java (2/2)

```
    // a quién apunta cada habilidad que un enemigo sabe lanzar
    private static Fighter aimOf(Skill skill, Fighter self,
                                 Fighter target) {
        return switch (skill) {
            case Strike strike -> target;
            case Summon summon -> self;
            case Taunt taunt -> throw new IllegalStateException(
                    self.name() + " no sabe provocar");
            case Heal heal -> throw new IllegalStateException(
                    self.name() + " no sabe curar");
        };
    }
}
```

ui/view/LogView.java (1/2)

```
package emberdeep.ui.view;

import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.event.GameEvent;
import emberdeep.engine.event.GameEvent.AchievementUnlocked;
import emberdeep.engine.event.GameEvent.CombatEnded;
import emberdeep.engine.event.GameEvent.DamageDealt;
import emberdeep.engine.event.GameEvent.DamageDodged;
import emberdeep.engine.event.GameEvent.Defended;
import emberdeep.engine.event.GameEvent.FighterDefeated;
import emberdeep.engine.event.GameEvent.FighterHealed;
import emberdeep.engine.event.GameEvent.ItemUsed;
import emberdeep.engine.event.GameEvent.SkillCast;
import emberdeep.engine.event.GameListener;
import emberdeep.ui.output.Output;
import emberdeep.ui.output.Style;

public class LogView extends View implements GameListener {
    public LogView(Output output) {
        super(output);
    }

    @Override
    public void onEvent(GameEvent event) {
        switch (event) {
            case DamageDealt e -> line(e.attacker() + " hace "
                    + e.amount() + " de daño " + kind(e.type())
                    + " a " + e.target() + ".",
                    e.targetIsHero() ? Style.DANGER : Style.NORMAL);
            case DamageDodged e -> line("El golpe de " + e.attacker()
                    + " a " + e.target() + " no hace daño.");
            case FighterDefeated e -> line(e.name()
                    + " queda fuera de combate.",
                    e.isHero() ? Style.DANGER : Style.NORMAL);
            case Defended e -> line(e.name() + " se cubre: +"
                    + e.shield() + " de escudo y +" + e.defenseBonus()
                    + " de defensa este turno.");
            case ItemUsed e -> line(e.user() + " usa " + e.itemName()
                    + " sobre " + e.target() + ".", Style.GOOD);
            case CombatEnded e -> line(e.heroesWon()
                    ? "¡Victoria!" : "Derrota...",
                    e.heroesWon() ? Style.GOOD : Style.DANGER);
            case AchievementUnlocked e -> line(
                    "¡Logro desbloqueado: " + e.name() + "!", Style.GOOD);
            case FighterHealed e -> line(e.name() + " recupera "
                    + e.amount() + " de vida.", Style.GOOD);
            case SkillCast e -> line(castText(e));
        }
    }
```

ui/view/LogView.java (2/2)

```
    // sin "sobre" cuando la habilidad es para quien la lanza (Taunt)
    private static String castText(SkillCast e) {
        String text = e.user() + " usa " + e.skillName();
        if (e.targetName().equals(e.user())) {
            return text + ".";
        }
        return text + " sobre " + e.targetName() + ".";
    }

    private static String kind(DamageType type) {
        return switch (type) {
            case PHYSICAL -> "físico";
            case FIRE -> "de fuego";
        };
    }
}
```

ui/view/HudView.java (1/2)

```
package emberdeep.ui.view;

import emberdeep.engine.event.GameEvent;
import emberdeep.engine.event.GameEvent.AchievementUnlocked;
import emberdeep.engine.event.GameEvent.CombatEnded;
import emberdeep.engine.event.GameEvent.DamageDealt;
import emberdeep.engine.event.GameEvent.DamageDodged;
import emberdeep.engine.event.GameEvent.Defended;
import emberdeep.engine.event.GameEvent.FighterDefeated;
import emberdeep.engine.event.GameEvent.FighterHealed;
import emberdeep.engine.event.GameEvent.ItemUsed;
import emberdeep.engine.event.GameEvent.SkillCast;
import emberdeep.engine.event.GameListener;
import emberdeep.ui.output.Output;
import emberdeep.ui.output.Style;

public class HudView extends View implements GameListener {
    private static final int WIDTH = 10;

    public HudView(Output output) {
        super(output);
    }

    @Override
    public void onEvent(GameEvent event) {
        switch (event) {
            case DamageDealt e -> bar(e.target(), e.targetHp(),
                    e.targetMaxHp());
            case ItemUsed e -> bar(e.target(), e.targetHp(),
                    e.targetMaxHp());
            case FighterHealed e -> bar(e.name(), e.hp(), e.maxHp());
            case DamageDodged e -> { }
            case FighterDefeated e -> { }
            case Defended e -> { }
            case CombatEnded e -> { }
            case AchievementUnlocked e -> { }
            case SkillCast e -> { }
        }
    }

    // casillas llenas = hp * 10 / maxHp, redondeado hacia arriba
    private void bar(String name, int hp, int maxHp) {
        int filled = (hp * WIDTH + maxHp - 1) / maxHp;
        line("  " + name + " [" + "#".repeat(filled)
                + ".".repeat(WIDTH - filled) + "] " + hp + "/" + maxHp,
                styleOf(hp, maxHp));
    }
```

ui/view/HudView.java (2/2)

```
    // rojo con un cuarto de vida o menos, verde con más de la mitad
    private static Style styleOf(int hp, int maxHp) {
        if (hp * 4 <= maxHp) {
            return Style.DANGER;
        }
        return hp * 2 > maxHp ? Style.GOOD : Style.NORMAL;
    }
}
```

engine/achievement/AchievementTracker.java (1/2)

```
package emberdeep.engine.achievement;

import emberdeep.engine.event.EventBus;
import emberdeep.engine.event.GameEvent;
import emberdeep.engine.event.GameEvent.AchievementUnlocked;
import emberdeep.engine.event.GameEvent.CombatEnded;
import emberdeep.engine.event.GameEvent.DamageDealt;
import emberdeep.engine.event.GameEvent.DamageDodged;
import emberdeep.engine.event.GameEvent.Defended;
import emberdeep.engine.event.GameEvent.FighterDefeated;
import emberdeep.engine.event.GameEvent.FighterHealed;
import emberdeep.engine.event.GameEvent.ItemUsed;
import emberdeep.engine.event.GameEvent.SkillCast;
import emberdeep.engine.event.GameListener;
import java.util.HashSet;
import java.util.Set;

public class AchievementTracker implements GameListener {
    private static final int DODGES_NEEDED = 5;

    private final EventBus bus;
    private final Set<String> unlocked = new HashSet<>();
    private int heroDodges;

    public AchievementTracker(EventBus bus) {
        this.bus = bus;
    }
```

engine/achievement/AchievementTracker.java (2/2)

```
    @Override
    public void onEvent(GameEvent event) {
        switch (event) {
            case DamageDealt e -> {
                if (!e.targetIsHero()) {
                    unlock("Primera sangre");
                }
            }
            case DamageDodged e -> {
                if (e.targetIsHero()) {
                    heroDodges++;
                    if (heroDodges == DODGES_NEEDED) {
                        unlock("Intocable");
                    }
                }
            }
            case CombatEnded e -> {
                if (e.heroesWon() && e.heroesHp().contains(1)) {
                    unlock("Último aliento");
                }
            }
            case FighterDefeated e -> { }
            case Defended e -> { }
            case ItemUsed e -> { }
            case AchievementUnlocked e -> { }
            case FighterHealed e -> { }
            case SkillCast e -> { }
        }
    }

    private void unlock(String name) {
        if (unlocked.add(name)) {
            bus.publish(new AchievementUnlocked(name));
        }
    }
}
```

ui/ConsoleMenu.java (1/7)

```
package emberdeep.ui;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Skill.Heal;
import emberdeep.engine.character.Skill.Strike;
import emberdeep.engine.character.Skill.Summon;
import emberdeep.engine.character.Skill.Taunt;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.Attack;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.command.Defend;
import emberdeep.engine.combat.command.UseItem;
import emberdeep.engine.combat.command.UseSkill;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.save.SnapshotStore;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.TileType;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.input.InputSource;
import emberdeep.ui.view.MapView;
import java.io.PrintStream;
import java.util.List;
import java.util.stream.Collectors;

public class ConsoleMenu {
    private enum Turn { ACTED, UNDONE, RESTORED }

    private final InputSource input;
    private final PrintStream out;
    private final MapView mapView;
    private final SnapshotStore checkpoints = new SnapshotStore();
    private int checkpointHero;     // a qué héroe le tocaba al guardar

    public ConsoleMenu(InputSource input, PrintStream out,
                       MapView mapView) {
        this.input = input;
        this.out = out;
        this.mapView = mapView;
    }

    public int askFormula() {
        out.println("Fórmula de daño:");
        out.println("  [1] plana: ataque - defensa");
        out.println("  [2] escalada: ataque x 100 / (100 + defensa)");
        return ask(2);
    }
```

ui/ConsoleMenu.java (2/7)

```
    public void run(GameSession session) {
        Combat combat = session.combat();
        out.println("=== Emberdeep ===");
        while (!combat.isOver()) {
            out.println("--- Ronda " + session.round() + " ---");
            List<Fighter> acting = aliveHeroes(session);
            int i = 0;
            while (i < acting.size() && !combat.isOver()) {
                Turn turn = heroTurn(session, acting.get(i), i);
                switch (turn) {
                    case ACTED -> i++;
                    case UNDONE -> i = Math.max(0, i - 1);
                    case RESTORED -> {
                        acting = aliveHeroes(session);
                        i = checkpointHero;
                    }
                }
            }
            if (!combat.isOver()) {
                combat.enemyPhase();
            }
        }
    }

    private List<Fighter> aliveHeroes(GameSession session) {
        return session.heroes().stream().filter(Fighter::isAlive).toList();
    }
```

ui/ConsoleMenu.java (3/7)

```
    // ACTED: el héroe actúa; UNDONE: se deshace la acción anterior;
    // RESTORED: se vuelve a un punto de control
    private Turn heroTurn(GameSession session, Fighter hero, int index) {
        Combat combat = session.combat();
        while (true) {
            out.println("Héroes:   " + describe(session.heroes()));
            out.println("Enemigos: " + describe(session.enemies()));
            out.println("Objetos:  " + describeItems(session.inventory()));
            out.println("Turno de " + hero.name()
                    + ". [1] atacar  [2] defender  [3] objeto"
                    + "  [4] habilidad  [u] deshacer"
                    + "  [c] guardar  [r] volver");
            String key = input.next();
            if (key.equals("u")) {
                if (combat.undo()) {
                    out.println("Deshaces la última acción.");
                    return Turn.UNDONE;
                }
                out.println("No hay nada que deshacer en esta ronda.");
                continue;
            }
            if (key.equals("c")) {
                saveCheckpoint(session, index);
                continue;
            }
            if (key.equals("r")) {
                if (loadCheckpoint(session)) {
                    return Turn.RESTORED;
                }
                continue;
            }
            CombatCommand command = switch (key) {
                case "1" -> new Attack(combat, hero,
                        askTarget("Objetivo", session.enemies()));
                case "2" -> new Defend(combat, hero);
                case "3" -> chooseItem(session, hero);
                case "4" -> chooseSkill(session, hero);
                default -> {
                    out.println("Tecla no válida.");
                    yield null;
                }
            };
            if (command != null) {
                combat.perform(command);
                return Turn.ACTED;
            }
        }
    }
```

ui/ConsoleMenu.java (4/7)

```
    private void saveCheckpoint(GameSession session, int index) {
        checkpoints.push(session.createSnapshot());
        checkpointHero = index;
        out.println("Punto de control guardado ("
                + checkpoints.size() + ").");
    }

    // false si todavía no hay ninguno
    private boolean loadCheckpoint(GameSession session) {
        if (checkpoints.isEmpty()) {
            out.println("No hay puntos de control.");
            return false;
        }
        session.restore(checkpoints.last());
        out.println("Vuelves al último punto de control (ronda "
                + session.round() + ").");
        return true;
    }

    private CombatCommand chooseItem(GameSession session, Fighter hero) {
        List<Item> items = session.inventory().items();
        if (items.isEmpty()) {
            out.println("No te quedan objetos.");
            return null;
        }
        Item item = items.get(choose("Objeto",
                items.stream().map(Item::name).toList()));
        Fighter target = item.kind() == ItemKind.HEALING
                ? askTarget("Aliado", session.heroes())
                : askTarget("Objetivo", session.enemies());
        return new UseItem(session, hero, target, item);
    }
```

ui/ConsoleMenu.java (5/7)

```
    private CombatCommand chooseSkill(GameSession session, Fighter hero) {
        List<Skill> skills = hero.skills();
        if (skills.isEmpty()) {
            out.println("No tienes habilidades.");
            return null;
        }
        Skill skill = skills.get(choose("Habilidad",
                skills.stream().map(Skill::name).toList()));
        Fighter target = switch (skill) {
            case Strike strike -> askTarget("Objetivo", session.enemies());
            case Heal heal -> askTarget("Aliado", session.heroes());
            case Taunt taunt -> hero;       // se lanza sobre sí mismo
            case Summon summon -> hero;     // solo la usan los enemigos
        };
        Combat combat = session.combat();
        return new UseSkill(combat, combat.mediator(), hero, skill,
                target);
    }

    private Fighter askTarget(String title, List<Fighter> group) {
        List<Fighter> alive = group.stream()
                .filter(Fighter::isAlive).toList();
        return alive.get(choose(title,
                alive.stream().map(Fighter::name).toList()));
    }

    // Muestra "Título:  [1] a  [2] b" y devuelve la posición (desde 0)
    private int choose(String title, List<String> labels) {
        StringBuilder menu = new StringBuilder(title + ":");
        for (int i = 0; i < labels.size(); i++) {
            menu.append("  [").append(i + 1).append("] ")
                    .append(labels.get(i));
        }
        out.println(menu);
        return ask(labels.size()) - 1;
    }
```

ui/ConsoleMenu.java (6/7)

```
    private int ask(int max) {
        while (true) {
            String line = input.next();
            try {
                int n = Integer.parseInt(line);
                if (n >= 1 && n <= max) {
                    return n;
                }
            } catch (NumberFormatException e) {
                // no es un número: se avisa más abajo
            }
            out.println("Escribe un número entre 1 y " + max + ".");
        }
    }

    private String describe(List<Fighter> group) {
        return group.stream()
                .map(f -> f.name() + " " + f.hp() + "/"
                        + f.stats().maxHp())
                .collect(Collectors.joining(", "));
    }

    private String describeItems(Inventory inventory) {
        if (inventory.items().isEmpty()) {
            return "(ninguno)";
        }
        return inventory.items().stream()
                .map(Item::name)
                .collect(Collectors.joining(", "));
    }
```

ui/ConsoleMenu.java (7/7)

```
    public boolean explore(GameMap map, TileTypes types) {
        while (true) {
            mapView.show(map);
            out.println("[w a s d] mover, [t] casillas, [q] salir");
            String key = input.next();
            switch (key) {
                case "w" -> step(map, 0, -1);
                case "a" -> step(map, -1, 0);
                case "s" -> step(map, 0, 1);
                case "d" -> step(map, 1, 0);
                case "t" -> showTiles(map, types);
                case "q" -> { return false; }
                default -> out.println("Tecla no válida.");
            }
            if (map.onStairs()) {
                out.println("Bajas por la escalera.");
                return true;
            }
        }
    }

    private void step(GameMap map, int dx, int dy) {
        if (!map.move(dx, dy)) {
            out.println("No puedes pasar.");
        }
    }

    private void showTiles(GameMap map, TileTypes types) {
        TileType corner = map.tileAt(0, 0);
        TileType opposite = map.tileAt(map.width() - 1, map.height() - 1);
        out.println("Casillas: " + map.width() * map.height()
                + ", TileType creados: " + types.createdCount());
        out.println("Esquinas opuestas, mismo objeto: "
                + (corner == opposite));
    }
}
```

app/Main.java (1/5)

```
package emberdeep.app;

import emberdeep.engine.achievement.AchievementTracker;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Skill.Heal;
import emberdeep.engine.character.Skill.Strike;
import emberdeep.engine.character.Skill.Taunt;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.event.EventBus;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.Dungeon;
import emberdeep.engine.world.DungeonBuilder;
import emberdeep.engine.world.Floor;
import emberdeep.engine.world.biome.BiomeFactory;
import emberdeep.engine.world.biome.CavernBiome;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.MapData;
import emberdeep.engine.world.map.MapParser;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.ConsoleMenu;
import emberdeep.ui.input.ConsoleInput;
import emberdeep.ui.input.InputSource;
import emberdeep.ui.input.ScriptInput;
import emberdeep.ui.output.AnsiOutput;
import emberdeep.ui.output.Output;
import emberdeep.ui.output.PlainOutput;
import emberdeep.ui.view.HudView;
import emberdeep.ui.view.LogView;
import emberdeep.ui.view.MapView;
import java.io.IOException;
import java.nio.file.Path;
import java.util.Arrays;
import java.util.List;
import java.util.Random;
import java.util.Scanner;
```

app/Main.java (2/5)

```
public class Main {
    private static final Skill POWER_STRIKE =
            new Strike("Golpe poderoso", 12, DamageType.PHYSICAL);
    private static final Skill FLARE =
            new Strike("Llamarada", 10, DamageType.FIRE);
    private static final Skill HEAL = new Heal("Luz curativa", 8);
    private static final Skill TAUNT = new Taunt("Grito de guerra");

    private final GameConfig config;

    private Main(GameConfig config) {
        this.config = config;
    }

    public static void main(String[] args) throws IOException {
        GameConfig config = new GameConfig(42L, 3, new FlatFormula());
        new Main(config).play(args);
    }
```

app/Main.java (3/5)

```
    private void play(String[] args) throws IOException {
        String script = scriptOf(args);
        String[] options = optionsOf(args);
        int floorNumber = chooseFloor(options);
        InputSource input = script == null
                ? new ConsoleInput(new Scanner(System.in), System.out)
                : new ScriptInput(Path.of(script), System.out);
        Output output = chooseOutput(args, script != null);
        ConsoleMenu menu = new ConsoleMenu(input, System.out,
                new MapView(output));
        Random random = new Random(config.seed());

        BiomeFactory crypt = new CryptBiome(random);
        BiomeFactory cavern = new CavernBiome(random);
        List<EnemyKind> cryptKinds = List.of(
                EnemyKind.SKELETON, EnemyKind.ASH_RAT, EnemyKind.ZOMBIE);
        List<EnemyKind> cavernKinds = List.of(
                EnemyKind.GOBLIN, EnemyKind.BAT, EnemyKind.SPIDER);
        List<EnemyKind> deepKinds = List.of(
                EnemyKind.SKELETON, EnemyKind.NECROMANCER);
        Dungeon dungeon = new DungeonBuilder()
                .floor().biome(crypt).enemies(cryptKinds)
                        .map(MapData.floor(1)).done()
                .floor().biome(cavern).enemies(cavernKinds)
                        .map(MapData.floor(2)).done()
                .floor().biome(crypt).enemies(deepKinds)
                        .map(MapData.floor(3)).done()
                .build();

        if (options.length > 1 && options[1].equals("map")) {
            explore(menu, dungeon, floorNumber);
            return;
        }
        DamageFormula formula = chooseFormula(menu, options,
                script != null);
        Floor floor = dungeon.floors().get(floorNumber - 1);
        GameSession session = new GameSession(heroes(), floor.biome(),
                floor.enemies(), formula, new DamagePipeline(random, 0.0));
        GameMap entrance = new MapParser(new TileTypes())
                .parse(floor.map());
        session.setLocation(floorNumber,
                entrance.partyX(), entrance.partyY());
        EventBus events = session.events();
        events.subscribe(new LogView(output));
        events.subscribe(new HudView(output));
        // el tracker va el último
        events.subscribe(new AchievementTracker(events));
        menu.run(session);
    }
```

app/Main.java (4/5)

```
    private void explore(ConsoleMenu menu, Dungeon dungeon,
                         int firstFloor) {
        TileTypes types = new TileTypes();
        MapParser parser = new MapParser(types);
        int number = firstFloor;
        while (true) {
            Floor floor = dungeon.floors().get(number - 1);
            boolean descended =
                    menu.explore(parser.parse(floor.map()), types);
            if (!descended || number == dungeon.floors().size()) {
                return;
            }
            number++;
        }
    }

    // El guion es el argumento que acaba en .txt
    private static String scriptOf(String[] args) {
        return Arrays.stream(args)
                .filter(arg -> arg.endsWith(".txt"))
                .findFirst().orElse(null);
    }

    private static String[] optionsOf(String[] args) {
        return Arrays.stream(args)
                .filter(arg -> !arg.endsWith(".txt")
                        && !arg.startsWith("--"))
                .toArray(String[]::new);
    }

    private DamageFormula chooseFormula(ConsoleMenu menu, String[] options,
                                        boolean scripted) {
        boolean useDefault = scripted
                || (options.length > 1 && options[1].equals("default"));
        if (useDefault) {
            return config.defaultFormula();
        }
        return menu.askFormula() == 1
                ? new FlatFormula() : new ScaledFormula();
    }

    private int chooseFloor(String[] options) {
        int floor = options.length > 0 ? Integer.parseInt(options[0]) : 1;
        if (floor < 1 || floor > config.floors()) {
            throw new IllegalArgumentException(
                    "La planta debe estar entre 1 y " + config.floors());
        }
        return floor;
    }
```

app/Main.java (5/5)

```
    // --plain o --ansi mandan; si no, plano con guion y color a mano
    private static Output chooseOutput(String[] args, boolean scripted) {
        List<String> all = Arrays.asList(args);
        boolean plain = all.contains("--plain");
        boolean ansi = all.contains("--ansi");
        if (plain && ansi) {
            throw new IllegalArgumentException(
                    "Elige --plain o --ansi, no las dos");
        }
        if (ansi || (!plain && !scripted)) {
            return new AnsiOutput(System.out);
        }
        return new PlainOutput(System.out);
    }

    private List<Hero> heroes() {
        return List.of(
                new HeroBuilder().name("Aria").stats(new Stats(30, 8, 3))
                        .skill(POWER_STRIKE).skill(FLARE)
                        .skill(HEAL).build(),
                new HeroBuilder().name("Borin").stats(new Stats(40, 6, 5))
                        .skill(POWER_STRIKE).skill(FLARE)
                        .skill(TAUNT).build(),
                new HeroBuilder().name("Cael").stats(new Stats(24, 9, 2))
                        .skill(POWER_STRIKE).skill(FLARE).build());
    }
}
```

Parte 3: la comprobación.

app/Check12.java (1/3)

```
package emberdeep.app;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.Attack;
import emberdeep.engine.combat.command.UseSkill;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.ui.output.PlainOutput;
import emberdeep.ui.view.LogView;
import java.util.List;
import java.util.Random;
```

app/Check12.java (2/3)

```
public class Check12 {
    public static void main(String[] args) {
        Random random = new Random(42);
        Hero ariaData = new HeroBuilder().name("Aria")
                .stats(new Stats(30, 8, 3))
                .skill(new Skill.Heal("Luz curativa", 8)).build();
        Hero borinData = new HeroBuilder().name("Borin")
                .stats(new Stats(40, 6, 5))
                .skill(new Skill.Taunt("Grito de guerra")).build();
        GameSession session = new GameSession(List.of(ariaData, borinData),
                new CryptBiome(random),
                List.of(EnemyKind.ASH_RAT, EnemyKind.ZOMBIE,
                        EnemyKind.NECROMANCER),
                new FlatFormula(), new DamagePipeline(random, 0.0));
        session.events().subscribe(
                new LogView(new PlainOutput(System.out)));

        Combat combat = session.combat();
        Fighter aria = session.heroes().get(0);
        Fighter borin = session.heroes().get(1);
        Fighter rat = session.enemies().get(0);
        Fighter zombie = session.enemies().get(1);

        // Ronda 1: Borin provoca y los tres enemigos le atacan a él
        combat.perform(cast(combat, borin, borin));
        combat.enemyPhase();
        System.out.println("Borin: " + borin.hp() + "/"
                + borin.stats().maxHp());

        // Ronda 2: Aria cura a Borin y mata a la Rata (furor del Zombi)
        combat.perform(cast(combat, aria, borin));
        combat.perform(new Attack(combat, aria, rat));
        System.out.println("Ataque del Zombi furioso: "
                + zombie.stats().attack());
        combat.enemyPhase();
        System.out.println("Ataque del Zombi tras la ronda: "
                + zombie.stats().attack());
        System.out.println("Aria: " + aria.hp() + "/"
                + aria.stats().maxHp());

        // Ronda 3: el Nigromante invoca un refuerzo
        combat.enemyPhase();
        Fighter reinforcement = session.enemies().get(3);
        System.out.println("Enemigos: " + session.enemies().size());
        System.out.println(reinforcement.name() + ": " + reinforcement.hp()
                + "/" + reinforcement.stats().maxHp());
    }
```

app/Check12.java (3/3)

```
    // la primera habilidad de quien la lanza, sobre el objetivo dado
    private static UseSkill cast(Combat combat, Fighter user,
                                 Fighter target) {
        return new UseSkill(combat, combat.mediator(), user,
                user.skills().get(0), target);
    }
}
```

Por qué funciona (parte 1). `Fighter.takeDamage` avisa a la arena solo cuando el combatiente pasa de vivo a muerto, y la arena, al construirse, une a todos con `join`. Cuando cae un enemigo, `allyDefeated` da `+2` de ataque durante un turno (un `TempBuff`) al primer enemigo vivo, y ese buff dura hasta que `Combat` avanza los buffs al cerrar la ronda. Con la cripta y siempre «atacar al primero»: en la ronda 1, Aria hace $10 - 2 = 8$ al Esqueleto, Borin $8 - 2 = 6$ y Cael $11 - 2 = 9$, que lo mata (18 de vida); como la Rata es la primera enemiga viva, se enfurece (ataque 7). En la fase enemiga la Rata elige al héroe con menos vida, Cael (24), y le hace $7 - 2 = 5$ por la fórmula y $5 - 1 = 4$ por la armadura (antes hacía 2); el Zombi le hace $5 - 3 = 2$ y $2 - 1 = 1$ a Aria, que queda en 29. En la ronda 2, Aria hace 10 a la Rata, que muere, y se enfurece el Zombi (ataque 7); Borin le hace $8 - 1 = 7$ y Cael $11 - 1 = 10$, así que el Zombi queda en 5; en la fase enemiga hace a Aria $7 - 3 = 4$ y $4 - 1 = 3$: queda en 26. Cuando empieza la ronda 3, el HUD de la línea «Héroes» muestra Aria 26/30, Borin 40/40 y Cael 20/24, y Aria remata al Zombi con $10 - 1 = 9$. En la caverna, en la ronda 1, Aria hace 9 y Borin 7 y matan al Goblin (14 de vida), se enfurece el Murciélago, Cael lo mata con 11 y se enfurece la Araña (ataque 9); la Araña elige al azar con la semilla 42 (el `nextInt(3)` da 2: Cael) y le hace $9 - 2 = 7$ y $7 - 1 = 6$: Cael queda en 18, y en la ronda 2 se acaba la partida. Los números de las plantas 1 y 2 cambian por el furor, y solo por él.

Por qué funciona (parte 2). `Skill` es una interfaz sellada con cuatro registros. Al pasarla de registro a interfaz, el compilador marca todos los sitios que usaban `new Skill(...)` o `skill.power()`: las plantillas, `asElite`, `Main`, `UseSkill`. Los `switch` de `UseSkill`, `SkillfulAi` y `ConsoleMenu` no tienen `default`: si mañana se añade `Venom` (capítulo 12), el compilador marca los tres. En `UseSkill`, `Strike` sigue siendo un golpe por la tubería; `Taunt` y `Summon` piden al mediador y publican `SkillCast`; `Heal` pide al mediador, que valida y cura, y `UseSkill` publica `FighterHealed` con lo curado de verdad. `SkillfulAi` sigue lanzando su primera habilidad cada tercera ronda: para un Zombi es un golpe (a quien elige `choose`) y para el Nigromante es una invocación (a sí mismo, porque no tiene objetivo). Lo único que ve el motor de las vistas son dos eventos más, y las tres clases que hacen un `switch` sobre ellos añaden sus ramas. `SnapshotTest` sigue pasando sin tocarlo: usa `HeroBuilder` sin habilidades y una `GameSession` con la misma firma.

Por qué funciona (parte 3). Se parte de Aria (ataque 10 con el equipo, defensa 3, armadura 1) y Borin (ataque 8, defensa 5, armadura 1) contra la Rata (8, 5, 0), el Zombi (22, 5, 1) y el Nigromante (14, 5, 1), con fórmula plana y esquiva al 0 %.

- *Ronda 1.* Borin provoca: la arena apunta a Borin y se publica `SkillCast` (el objetivo es él mismo, por eso el log dice «usa Grito de guerra.», sin «sobre»). En la fase enemiga, `targetsFor` devuelve solo a Borin, aunque Aria tenga menos vida: los tres le atacan. Cada golpe es $5 - 5 = 0$, que la fórmula sube a 1, y la armadura lo deja en $\max(1, 1 - 1) = 1$. Borin queda en 37. Al cerrar la ronda, `endRound` olvida la provocación.
- *Ronda 2.* Aria cura a Borin con 8, pero solo le faltan 3: `heal` lo limita y el evento dice

«recupera 3». Aria ataca a la Rata: $10 - 0 = 10$, y la Rata (8 de vida) cae; el aviso a la arena enfurece al primer enemigo vivo, el Zombi, y su ataque sube a 7. En la fase enemiga, sin provocación, `targetsFor` devuelve a los dos héroes y el Zombi (que elige al primero vivo, Aria) hace $7 - 3 = 4$ y, con la armadura, 3; el Nigromante, $5 - 3 = 2$ y 1. Aria queda en $30 - 3 - 1 = 26$. Al terminar la ronda, el buff expira y el ataque del Zombi vuelve a 5.

- *Ronda 3.* Es una tercera ronda, así que las dos IA usan su primera habilidad. El Zombi lanza «Abrazo pútrido», un `Strike` de 7 que no pasa por la fórmula: la armadura lo deja en 6 (Aria, 20). El Nigromante lanza su `Summon`: la arena pide un Esqueleto a la cripta, lo une a sí misma y lo añade a la lista; el `SkillCast` dice «Nigromante usa Alzar esqueleto sobre Esqueleto.». Como `enemyPhase` recorre una copia, el Esqueleto no actúa en esta ronda. Al final hay cuatro enemigos (la Rata caída sigue en la lista) y el Esqueleto tiene 18/18.

Errores típicos:

- Recorrer `enemies` directamente en `enemyPhase`: al invocar, la lista cambia mientras se recorre y salta `ConcurrentModificationException`.
- Dejar la lista de enemigos inmutable (`toList()`): `summon` lanza `UnsupportedOperationException`.
- Olvidar `reinforcement.join(this)`: el refuerzo cae y no enfurece a nadie.
- Avisar en `takeDamage` cada vez que `hp == 0`, sin la comprobación de que estaba vivo: un golpe más sobre un caído volvería a enfurecer a otro.
- Calcular la cura en la arena: la aritmética ya está en `Fighter.heal`, y la arena duplicaría su regla (y tendría que conocer el máximo de vida).
- No llamar a `endRound()`: la provocación duraría para siempre.
- Devolver al provocador en `targetsFor` aunque haya caído: la lista quedaría con un muerto y `choose` no tendría a quién elegir.
- Poner `default` en los `switch` sobre `Skill` o `GameEvent`: el compilador deja de avisar de los casos nuevos.

Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo 12: `engine/combat/arena/` con `TurnMediator` (cuatro interacciones) y `CombatArena`; `Skill` sellada con `Strike`, `Taunt`, `Heal` y `Summon`; `EnemyKind.NECROMANCER` y su plantilla en la cripta; `GameEvent` con nueve records; `Combat` con seis parámetros de constructor; y `Main` con las habilidades nuevas de Aria y Borin y la planta 3 con un Nigromante. Borra `app/Check12.java`.

Parte V

Estados, estructura y cierre

CAPÍTULO 12

State: estados de los personajes y fases de la partida

Objetivos del capítulo

- Reconocer el olor que pide State: banderas booleanas que se combinan y dejan combinaciones imposibles.
- Modelar los estados como tipos de una interfaz `sealed` y las transiciones como un `switch` sin `default`.
- Aplicar State a los personajes (`Status`: normal, envenenado, aturdido) y al ciclo de la partida (`GamePhase`).
- Limpiar lo que deja un combate (`Fighter.endCombat()`) y ver por qué un Memento tiene que guardar también el estado.
- Distinguir State de Strategy y saber qué pasa cuando hace falta más de un estado a la vez.
- Dar el hito H13: veneno, aturdimiento y fases de la partida, con el `Snapshot` al día.

Hasta ahora, un combatiente de Emberdeep solo tenía números: vida, escudo, estadísticas. Tu equipo pide efectos que *duran*: un veneno que quita vida al empezar cada turno durante tres turnos y un golpe que aturde y hace perder el turno. Y el juego, además, tiene momentos distintos: el menú, recorrer la mazmorra, combatir, el final. Los dos problemas tienen la misma forma: algo que está siempre en *uno* de varios modos y cuyo comportamiento depende del modo en que está. Eso es una máquina de estados, y State es el patrón que la escribe sin convertir el código en una maraña de `if`. Lo construyes con los tipos sellados de Java 21, que te dan lo que no daban las banderas: que las combinaciones imposibles no se puedan escribir y que el compilador te señale todos los sitios que hay que tocar cuando añades un estado.

12.1 El olor que pide State: banderas y combinaciones imposibles

Empieza por lo que escribirías sin pensar en patrones: una bandera por efecto y los números que cada efecto necesita. Sería algo así (el código es hipotético; no existe en el proyecto):

Hipotético: Fighter con una bandera por efecto

```
public class Fighter {
    private boolean poisoned;
    private int poisonTurns;       // solo significa algo si poisoned
    private int poisonDamage;      // idem
    private boolean stunned;
    private int stunTurns;         // solo significa algo si stunned

    // devuelve true si puede actuar
```

```
    public boolean startTurn() {
        if (poisoned) {
            hp = Math.max(0, hp - poisonDamage);
            poisonTurns--;
            if (poisonTurns == 0) {
                poisoned = false;          // ¿y poisonDamage?
            }
        }
        if (stunned) {
            stunTurns--;
            if (stunTurns == 0) {
                stunned = false;
            }
            return false;
        }
        return true;
    }

    public void poison(int turns, int damage) {
        poisoned = true;
        poisonTurns = turns;
        poisonDamage = damage;
    }

    public void stun(int turns) {
        stunned = true;
        stunTurns = turns;
    }
}
```

Funciona, y tiene cuatro problemas que crecen a la vez:

1. **Datos que solo valen a veces.** `poisonDamage` conserva un valor viejo cuando `poisoned` es falso, y nada impide leerlo. Cada efecto arrastra sus números sueltos dentro de `Fighter`.
2. **Combinaciones que nadie decidió.** Con dos banderas hay cuatro combinaciones. ¿Envenenado y aturdido a la vez es válido? El código lo permite sin que lo hayas decidido, y cualquier método que se olvide de una de las dos banderas se equivoca solo en algunas combinaciones.
3. **Una cadena de `if` por método.** `startTurn`, el log, el HUD y la IA tendrían que consultar las mismas banderas, cada uno a su manera.
4. **Añadir un efecto no avisa de nada.** Una bandera nueva, dos números nuevos y un `if` nuevo en cada método que debería tenerla en cuenta; si olvidas uno, compila.

Efectos	Combinaciones de banderas (2^n)	Estados si solo vale uno a la vez ($n+1$)
2	4	3
3	8	4
4	16	5
5	32	6

La tabla pone cifras a la intuición. Con las banderas, cada efecto nuevo duplica las combinaciones

que hay que vigilar; si lo que quieres es que solo valga *un* modo a la vez, solo hay $n + 1$ casos legales (normal más uno por efecto) y las banderas permiten todos los demás. Robert Nystrom cuenta el mismo callejón sin salida con una heroína de plataformas: cada bandera arregla un fallo y abre otro, y cuando de un puñado de banderas solo una puede ser verdadera a la vez, es la pista de que lo que quieres es un modo único, primero un `enum` y, si cada modo tiene datos y comportamiento propios, State [19].

Este es el olor: **un objeto cuyo comportamiento depende de un modo que dura, escrito como banderas que se combinan**. State lo cura convirtiendo cada modo en un valor de un tipo, de modo que el objeto está siempre en exactamente uno. En el catálogo, su intención es permitir que un objeto altere su comportamiento cuando cambia su estado interno, de modo que parezca que cambia de clase [9, 24]. Head First le dedica un capítulo entero [7].

12.2 Piezas del patrón State

Los roles son tres, el estado, los estados concretos y el contexto que los guarda, y a ellos se suma la transición, que aquí es un método. En Emberdeep:

Rol	Clase en el ejemplo	Propósito	Paquete
Estado (interfaz sellada)	`Status`	Los estados posibles de un combatiente	`combat.status`
Estados concretos (records)	`Normal`, `Poisoned`, `Stunned` (ejercicio)	Cada estado, con sus propios datos	`combat.status`
Contexto (clase)	`Fighter`	Guarda el estado actual y lo cambia	`character`
Transiciones (método)	`next()` con `switch`	Dice qué estado sigue a cada uno	`combat.status`
Segundo uso (sellada y clase)	`GamePhase` y `GameFlow`	Las fases de la partida; `GameFlow` decide la siguiente	`engine.flow`

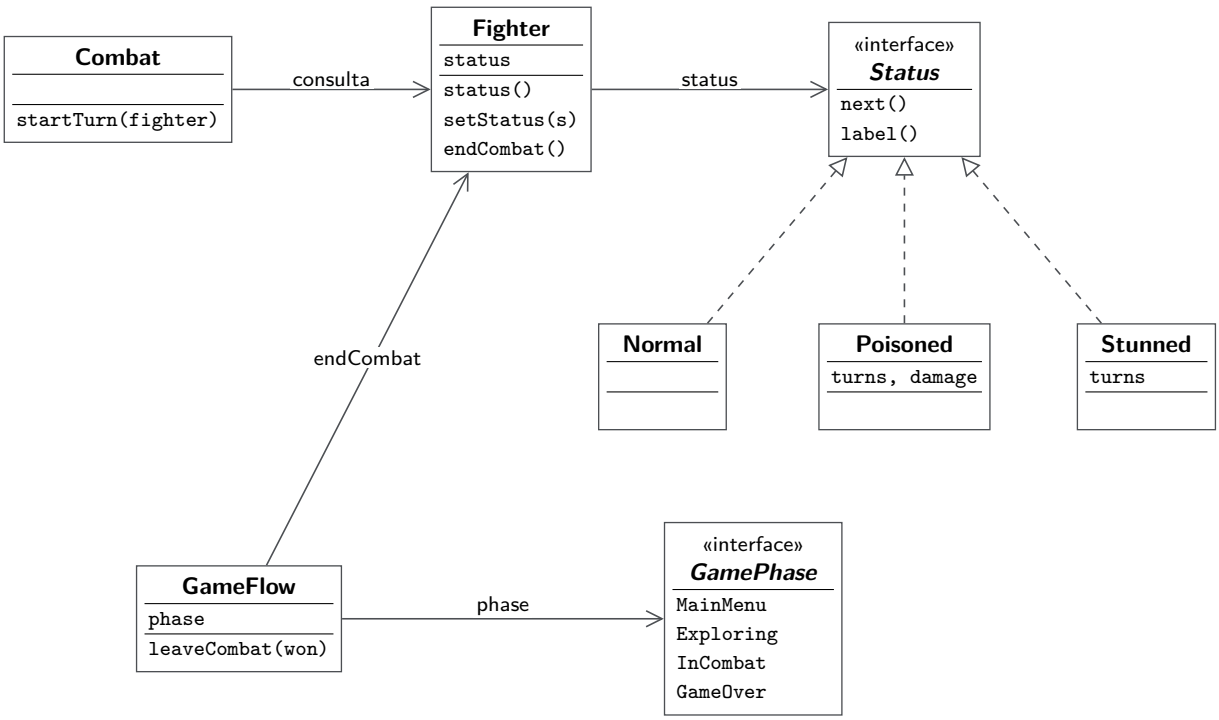


Diagrama UML del patrón State: `Fighter` guarda un `Status`; `GameFlow`, una `GamePhase` (cuatro records).

Mira dos cosas del diagrama. `Fighter` solo conoce la interfaz `Status`, no sus variantes. Y `Combat` es quien *consulta* el estado al empezar cada turno, pero no decide qué estado sigue: eso lo dice el propio estado con `next()`.

12.3 El ejemplo de State, pieza a pieza: `Status`

El estado como tipo. Cada modo es un record dentro de una interfaz sellada (la `sealed interface` del capítulo 9) [21]. El capítulo empieza con dos: `Normal`, que no lleva datos, y `Poisoned`, que lleva lo que necesita (cuántos turnos le quedan y cuánto daño hace cada uno). El tercero, `Stunned`, lo escribes tú en el hito.

`engine/combat/status/Status.java` (versión del texto)

```
1  package emberdeep.engine.combat.status;
2
3  public sealed interface Status {
4      record Normal() implements Status {}
5
6      // turns: turnos que le quedan; damage: daño que causa cada turno
7      record Poisoned(int turns, int damage) implements Status {}
8
9      // la transición: el estado que queda al consumir un turno
10     default Status next() {
11         return switch (this) {
12             case Normal normal -> normal;
13             case Poisoned poisoned -> poisoned.turns() <= 1
14                     ? new Normal()
15                     : new Poisoned(poisoned.turns() - 1,
16                             poisoned.damage());
17         };
```

```
    }
}
```

Tres cosas que mirar. Primera: los datos de cada efecto viven *dentro* del estado. `poisonDamage` ya no es un campo suelto de `Fighter`: solo existe mientras hay un `Poisoned`, y un `Normal` no tiene nada que leer por error. Segunda: `switch (this)` usa pattern matching sobre el propio tipo, y la transición es una sola expresión que devuelve el estado siguiente. No lleva `default`: como `Status` es sellada, el compilador sabe que solo hay dos casos y exige nombrarlos [4]. Tercera, y es la ventaja que no tenían las banderas: si añades un tercer record, el `switch` deja de compilar hasta que decidas qué estado le sigue. El hito te lo hará ver.

Un `Normal` sin datos podría ser un valor de un `enum`, y para una máquina de estados sin datos un `enum` es la versión más simple [19]. Aquí los tres estados comparten interfaz y dos de ellos llevan datos, y por eso son records.

El contexto. `Fighter` solo guarda el estado actual. No sabe qué significa cada uno:

Fragmento de Fighter: el contexto

```
private Status status = new Normal();   // todos nacen sin efectos

public Status status() { return status; }

public void setStatus(Status status) {
    this.status = Objects.requireNonNull(status);
}
```

Quién consulta el estado. Al empezar el turno de un combatiente, `Combat` mira su estado, aplica lo que ese estado hace y deja puesto el siguiente. Es el sitio donde el `switch` sobre los estados convierte «está envenenado» en «pierde vida»:

Fragmento de Combat: startTurn (versión del texto)

```
public boolean startTurn(Fighter fighter) {
    Status current = fighter.status();
    switch (current) {
        case Normal normal -> { }
        case Poisoned poisoned -> {
            int lost = Math.min(poisoned.damage(), fighter.hp());
            fighter.takeDamage(lost);
            publish(new StatusTick(fighter.name(), isHero(fighter),
                    current.label(), lost,
                    fighter.hp(), fighter.stats().maxHp()));
        }
    }
    fighter.setStatus(current.next());      // la transición
    return fighter.isAlive();
}
```

Un envenenado con tres turnos y dos de daño, a lo largo de tres turnos suyos, se ve así (Zombi, 22 de vida):

Turno	Estado al empezar	Vida	Estado al acabar `startTurn`
1	`Poisoned(3, 2)`	20	`Poisoned(2, 2)`
2	`Poisoned(2, 2)`	18	`Poisoned(1, 2)`
3	`Poisoned(1, 2)`	16	`Normal`

Cómo se pone un estado: la habilidad `Venom`. La transición de `next()` es la del *paso del tiempo*. Las que ocurren por un *hecho* (te envenenan) las hace quien provoca el hecho, llamando a `setStatus`. Es una habilidad nueva en la interfaz sellada del capítulo 11, con un caso nuevo en `UseSkill`, y el compilador te lleva de la mano a ese `switch`:

Fragmento de Skill y de UseSkill: Venom

```
// en Skill: turns turnos de veneno, damage de daño por turno
record Venom(String name, int turns, int damage) implements Skill {}

// en el switch de UseSkill.execute():
case Venom venom -> {
    target.setStatus(new Poisoned(venom.turns(), venom.damage()));
    combat.publish(new SkillCast(user.name(), combat.isHero(user),
            venom.name(), target.name()));
}
```

Como `GameEvent` es sellada, el motor necesita un evento para contar lo que pasa al empezar el turno: `StatusTick(name, isHero, statusName, amount, hp, maxHp)`. Con `amount` mayor que 0 es el daño del veneno; con `amount` igual a 0 y el estado aturdimiento, un turno perdido. El log lo cuenta («Zombi sufre 2 de daño por veneno.») y, como en el capítulo 11, el compilador señala los tres `switch` sin `default` sobre `GameEvent`: `LogView`, `HudView` y `AchievementTracker`.

Dos reglas que conviene decir en voz alta. Primera: un combatiente tiene *un solo* estado; si le aplican uno nuevo, sustituye al anterior (un envenenado al que aturden deja de estar envenenado). Es una decisión de diseño, no un descuido: es lo que significa «una máquina de estados» y tiene un límite que verás en la sección 12.5. Segunda: el veneno es daño directo, `takeDamage`, y no pasa por la tubería del capítulo 6; ignora escudo, armadura y resistencias.

12.4 State para el ciclo de la partida: `GamePhase`

El mismo patrón sirve a escala mayor. Una partida de Emberdeep está siempre en una fase: el menú principal, recorriendo una planta, en combate o terminada. Hoy esa información está repartida entre `Main`, que decide qué modo arranca, y el bucle de `ConsoleMenu`, que sabe cuándo acaba el combate. Con State, las fases son un tipo y las transiciones, un `switch`:

engine/flow/GamePhase.java

```
package emberdeep.engine.flow;

public sealed interface GamePhase {
    record MainMenu() implements GamePhase {}

    record Exploring(int floor) implements GamePhase {}

    record InCombat(int floor) implements GamePhase {}

```

```
    record GameOver(boolean won) implements GamePhase {}
}
```

Cada fase lleva solo lo que le hace falta: recorrer o combatir pide saber la planta, y el final, si se ganó. El contexto es `GameFlow`, que guarda la fase actual y ofrece una acción por cada hecho que la cambia. Estas son todas las transiciones; cualquier otra lanza `IllegalStateException`:

Fase actual	Acción	Fase siguiente
`MainMenu`	`start(floor)`	`Exploring(floor)`
`Exploring(f)`	`enterCombat()`	`InCombat(f)`
`Exploring(f)`	`descend()`	`Exploring(f + 1)`; en la última planta, `GameOver(true)`
`InCombat(f)`	`leaveCombat(true)`	`Exploring(f)`
`InCombat(f)`	`leaveCombat(false)`	`GameOver(false)`

La acción que importa es la que sale del combate, porque es el sitio donde se limpia lo que el combate dejó. Fíjate también en que es un `switch` sobre la fase actual:

Fragmento de GameFlow: salir del combate

```
public void leaveCombat(boolean heroesWon) {
    phase = switch (phase) {
        case InCombat inCombat -> {
            heroes.forEach(Fighter::endCombat);
            yield heroesWon
                    ? new Exploring(inCombat.floor())
                    : new GameOver(false);
        }
        case MainMenu menu -> throw notNow("salir del combate");
        case Exploring exploring -> throw notNow("salir del combate");
        case GameOver gameOver -> throw notNow("salir del combate");
    };
}
```

Qué limpia `endCombat()`. Un combate deja huellas que no deben viajar al siguiente: un estado de veneno o aturdimiento, el escudo que dejó un `Defend` y los `TempBuff` que no llegaron a caducar. Si la partida termina en la fase de los héroes, antes de que `enemyPhase` avance los buffs, nada los apagaría. La vida *no* se toca: las heridas se conservan. Y el equipo permanente tampoco: desde el capítulo 6, `Fighter` guarda aparte el equipo (`gear`) y la cadena actual (`stats`), que es el equipo más los `TempBuff`. Volver a `gear` apaga los buffs y deja la espada y la armadura:

Fragmento de Fighter: endCombat

```
public void endCombat() {
    status = new Normal();
    shield = 0;          // el que dejó un Defend
    stats = gear;        // fuera los TempBuff; el equipo se queda
}
```

Quién avisa a `GameFlow`. Nadie lo llama a mano al terminar el combate: `GameFlow` implementa `GameListener` y se suscribe al bus del capítulo 9. Solo le interesa un evento, así que no hace

falta un `switch` exhaustivo; basta un `instanceof`, que pregunta si el evento es un `CombatEnded` y, si lo es, lo deja ya convertido en `ended`:

Fragmento de GameFlow: reaccionar al fin del combate

```
@Override
public void onEvent(GameEvent event) {
    if (event instanceof CombatEnded ended
            && phase instanceof InCombat) {
        leaveCombat(ended.heroesWon());
    }
}
```

La partida de consola de Emberdeep solo tiene la fase de combate (el modo mapa sigue aparte), así que hoy la fase cambia una vez. Pero el motor ya sabe en qué fase está, y el capítulo 14 lo aprovecha: solo se podrá guardar en `Exploring`.

12.5 State: cuándo sí y cuándo no

Compensa cuando un objeto cambia de comportamiento según un modo que dura (efectos de estado, fases de una partida, el estado de una conexión o de un pedido), cuando las banderas se excluyen entre sí y cuando las transiciones se pueden listar. Se nota en que las preguntas «¿en qué modo está?» aparecen en varios métodos.

No compensa si hay dos modos y casi nada depende de ellos: un `boolean` basta, y el catálogo reconoce que aplicar el patrón puede ser excesivo cuando la máquina tiene pocos estados o rara vez cambia [24]. Tampoco, si lo que cambia es una política que elige quien llama y no depende del tiempo (eso es Strategy).

State frente a Strategy. Se parecen mucho, porque en los dos hay un objeto que delega en otro intercambiable, y se distinguen por quién cambia la pieza y por qué. En Strategy la elige el cliente desde fuera y rara vez se cambia: la `EnemyAi` de un enemigo se fija al crearlo. En State el cambio forma parte del dominio: el `Status` de un `Fighter` cambia solo con el paso de los turnos y con los hechos del combate, y un estado construye el siguiente (`Poisoned` produce un `Normal`). Por eso, según el catálogo, los estados concretos pueden conocerse entre sí y provocar transiciones, y las estrategias casi nunca [24].

Un estado a la vez: el límite. Una máquina de estados está en un único estado, y a veces eso no basta. Si quieres envenenado *y* aturdido, no puedes añadir un estado «envenenado y aturdido» para cada pareja: con n efectos independientes harían falta 2^n estados. Nystrom lo plantea con una heroína que lleva un arma: meter en una sola máquina lo que hace y lo que lleva exige $n \times m$ estados, mientras que con dos máquinas son $n + m$ [19]. El ejercicio 12.4 te pide diseñarlo.

Costes reales:

- **Más tipos.** Un estado nuevo es un record nuevo y un caso nuevo en cada `switch` sobre `Status`. Con `sealed` el compilador los lista (es la ventaja), pero el trabajo existe.
- **Las transiciones se reparten.** Las del paso del tiempo viven en `next()`; las que provoca un hecho, en quien llama a `setStatus` (aquí, `UseSkill`). Quien lee `Status` no ve todo el diagrama de estados.
- **Estilo de datos frente a estilo de clases.** En la forma clásica del patrón cada estado es una clase con sus métodos y añadir un estado es barato. Con records y `switch`, la lógica

de todos los estados está junta en un `switch` y añadir una *operación* nueva es barato, pero un estado nuevo toca todos los `switch`. Es la forma de programar con datos que defiende Goetz [10]; en este curso la usamos por la ayuda del compilador.

- **El Memento tiene que guardarlo.** Cada dato de estado que añades al juego se suma al `Snapshot`, y esa es la parte del hito que más se olvida (sección 12.6).

12.6 En el proyecto: H13 con State

Este capítulo avanza el hito H13. Cambios del hito:

- **Nuevos paquetes:** `combat/status/` (con `Status`) y `engine/flow/` (con `GamePhase` y `GameFlow`).
- **`Skill` gana dos variantes:** `Venom` y `Stun`. Los tres `switch` sin `default` sobre `Skill` (`UseSkill`, `SkillfulAi` y `ConsoleMenu.chooseSkill`) dejan de compilar hasta que añades los dos casos.
- **`GameEvent` gana `StatusTick`** (diez records) y se actualizan los tres `switch` sobre eventos: `LogView`, `HudView` y `AchievementTracker`.
- **Cambian** `Fighter` (estado, `setStatus`, `endCombat`), `Combat` (`startTurn`), `UseSkill` (los dos casos y el `undo` del estado), `ConsoleMenu` (los héroes consultan su estado al empezar la ronda) y `Main` (dos habilidades nuevas y el `GameFlow`).
- **Cambia el Memento:** `FighterState` gana el campo `status`, y con él `GameSession` y `SnapshotTest`.

Qué cambia en la partida. Cael gana una habilidad, «Daga envenenada» (envenena tres turnos, dos de daño por turno), y Borin otra, «Golpe aturdidor» (el objetivo pierde un turno). Ambas van al final de sus listas, así que las pulsaciones de `scripts/demo.txt` siguen valiendo. Ningún enemigo tiene todavía `Venom` ni `Stun` en su plantilla, aunque la `SkillfulAi` ya sabe lanzarlas. **Los números de las partidas de «atacar siempre al primero» no cambian** (nadie lanza veneno ni aturdimiento); solo cambia el texto de la línea «Habilidad:» de Borin y de Cael.

Límites que esta versión no resuelve (se declaran, no se ocultan):

1. **Un solo estado.** Aplicar uno nuevo sustituye al anterior. El ejercicio 12.4 explora cómo evitarlo.
2. **El veneno es daño directo.** Ignora escudo, armadura y resistencias, y puede matar (en ese caso se publica `FighterDefeated`).
3. **`u` deshace el lanzamiento, no el tictac.** Deshacer un `Venom` o un `Stun` repone el estado anterior del objetivo, pero el daño que el veneno ya hizo al empezar la ronda no se deshace: no es un comando.
4. **`GameFlow` no está en el `Snapshot`.** Los puntos de control se toman en combate, y `r` no cambia de fase. El modo mapa sigue aparte.

Los tres pasos. Cada uno termina con un resultado que puedes comprobar:

- **Paso 1, estados y habilidades.** `Status`, `Venom` y `Stun`, `StatusTick`, `Fighter.status`, `Combat.startTurn` y todo lo que el compilador señala. Resultado: el proyecto compila, `./gradlew test` sigue pasando y un programa temporal muestra un veneno que baja vida y un aturdimiento que salta un turno.

- **Paso 2, fin del combate y fases.** `Fighter.endCombat()`, `GamePhase` y `GameFlow`. Resultado: un héroe que se cubrió en el combate sale con el escudo a 0 y sin `TempBuff`, con la espada y la armadura intactas, y la fase llega a `GameOver`.
- **Paso 3, el Snapshot.** `FighterState` con `status`. Resultado: el compilador rompe `GameSession.createSnapshot`, y al arreglarlo `SnapshotTest` (con una prueba nueva) pasa con `./gradlew test`.

Hito 13: estados de los personajes y fases de la partida

Tras este hito, un combatiente puede estar envenenado o aturdido, el combate se limpia al terminar y la partida sabe en qué fase está. La estructura:

Estructura de carpetas tras el capítulo 12 (1/2)

```
emberdeep/
|-- settings.gradle
|-- build.gradle                    (con JUnit 6.1.3)
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
|-- scripts/
|   `-- demo.txt
`-- src/
    |-- main/java/emberdeep/
    |   |-- app/
    |   |   |-- Main.java               (composition root)
    |   |   `-- GameConfig.java
    |   |-- engine/
    |   |   |-- character/              Stats, Hero, HeroBuilder, Enemy,
    |   |   |                           EnemyKind, Fighter, Skill,
    |   |   |                           EnemyTemplate, TemplateRegistry
    |   |   |-- stats/                  StatSource, StatKind, BaseStats,
    |   |   |                           StatDecorator, WeaponBonus,
    |   |   |                           ArmorBonus, TempBuff
    |   |   |-- combat/
    |   |   |   |-- Combat.java
    |   |   |   |-- ai/                 EnemyAi, FirstAliveAi, LowestHpAi,
    |   |   |   |                       RandomTargetAi, SkillfulAi
    |   |   |   |-- arena/              TurnMediator, CombatArena
    |   |   |   |-- command/            CombatCommand, Attack, Defend,
    |   |   |   |                       UseItem, UseSkill, CommandHistory
    |   |   |   |-- damage/             DamageFormula, FlatFormula,
    |   |   |   |                       ScaledFormula, Damage, DamageType
    |   |   |   |-- defense/            DamageHandler, DamagePipeline,
    |   |   |   |                       DodgeHandler, ShieldHandler,
    |   |   |   |                       ArmorHandler, ResistanceHandler
    |   |   |   `-- status/             Status
```

Estructura de carpetas tras el capítulo 12 (2/2)

```
|   |   |-- item/                  Item, ItemKind, Inventory
|   |   |-- event/                 GameEvent, GameListener, EventBus
|   |   |-- flow/                  GamePhase, GameFlow
|   |   |-- achievement/           AchievementTracker
|   |   |-- save/                  Snapshot, FighterState,
|   |   |                          SnapshotStore
|   |   |-- session/               GameSession
|   |   `-- world/
|   |       |-- Dungeon.java, Floor.java,
|   |       |   DungeonBuilder.java
|   |       |-- biome/             BiomeFactory, CryptBiome,
|   |       |                      CavernBiome
|   |       `-- map/               TileType, TileTypes, GameMap,
|   |                              MapParser, MapData
|   `-- ui/
|       |-- ConsoleMenu.java
|       |-- input/                 InputSource, ConsoleInput,
|       |                          ScriptInput
|       |-- view/                  View, MapView, HudView, LogView
|       `-- output/                Output, Style, AnsiOutput,
|                                  PlainOutput, BufferOutput
`-- test/java/emberdeep/
    `-- engine/save/               SnapshotTest
```

Lo practicas en el ejercicio 12.5. Antes, cuatro ejercicios más pequeños.

12.7 Ejercicios

Ejercicio 12.1: Un semáforo de peatones con estados sellados ★★★☆☆

No es de Emberdeep. Crea una carpeta vacía y, dentro, archivos sin paquete. Vas a modelar un semáforo de peatones que recorre tres modos: rojo, verde y verde intermitente (que parpadea dos veces antes de volver al rojo).

1. `Signal`: `sealed interface` con tres records anidados: `Red()`, `Green()` y `Flashing(int blinksLeft)` (los parpadeos que le quedan).
2. `Signal next()`: método `default` con un `switch (this)` *sin* `default`. `Red` pasa a `Green`; `Green` pasa a `Flashing(2)`; `Flashing(n)` pasa a `Flashing(n - 1)` si `n` es mayor que 1 y, si no, a `Red`.
3. `String label()`: método `default`, también con un `switch` sin `default`: `"rojo"`, `"verde"` y, para el intermitente, `"intermitente "` seguido de los parpadeos que quedan.
4. `Main`: parte de un `Red`, imprime su `label()` y repite seis veces «avanzar con `next()` e imprimir».

Compila con `javac *.java` y ejecuta con `java Main`. Está bien si la salida es exactamente:

Salida esperada

```
rojo
verde
intermitente 2
intermitente 1
rojo
verde
intermitente 2
```

y si, al añadir a propósito un cuarto record `record Amber() implements Signal {}`, el compilador se queja de `next()` y de `label()` (quita el record después).

Pista: *Dentro de un método* `default` *de* `Signal`*, los nombres* `Red`*,* `Green` *y* `Flashing` *se escriben sin prefijo. En* `Main`*, el primer estado es* `new Signal.Red()`*.*

Práctica ▪ solución en la pág. 535

Ejercicio 12.2: Probar las transiciones del estado envenenado ★★★☆☆

No es de Emberdeep, pero usa el `build.gradle` con JUnit de la sección 8.6. Este es un estado reducido a lo que importa para la prueba, ya completo. Guárdalo en `src/main/java/scratch/Condition.java`:

src/main/java/scratch/Condition.java

```
1  package scratch;
2
3  public sealed interface Condition {
4      record Normal() implements Condition {}
5
6      record Poisoned(int turns, int damage) implements Condition {}
7
8      default Condition next() {
9          return switch (this) {
10             case Normal normal -> normal;
11             case Poisoned poisoned -> poisoned.turns() <= 1
12                     ? new Normal()
13                     : new Poisoned(poisoned.turns() - 1,
14                             poisoned.damage());
15         };
16     }
17 }
```

Crea `src/test/java/scratch/ConditionTest.java` con **cuatro** pruebas, una por transición. Recuerda que dos records con los mismos componentes son iguales con `equals`, así que `assertEquals(new Poisoned(2, 2), algo)` compara valores:

1. `Poisoned(3, 2).next()` es `Poisoned(2, 2)` (pierde un turno y conserva el daño).
2. `Poisoned(1, 2).next()` es `Normal()` (con un turno restante, el estado se acaba).
3. `Normal().next()` es `Normal()`.
4. Un `Poisoned(2, 5)` necesita exactamente dos pasos para volver a `Normal`: tras el primero es `Poisoned(1, 5)`, tras el segundo `Normal()`.

Está bien si `./gradlew test` termina con `BUILD SUCCESSFUL` y, además, si cambias a

propósito `<= 1` por `< 1` en `next` la segunda prueba falla (compruébalo y vuelve a ponerlo). Cuando termines, borra las carpetas `scratch`.

Pista: *Importa los records con* `import scratch.Condition.Poisoned;` *y* `import scratch.Condition.Normal;`*, y* `assertEquals` *de forma estática, como en* `SnapshotTest`*. El método de prueba puede llamarse como quieras, pero conviene que su nombre diga la transición.*

Test ▪ solución en la pág. 536

Ejercicio 12.3: De tres banderas a un estado sellado ★★★☆☆

No es de Emberdeep. Un monstruo se controla con tres banderas, y el código permite estados que no tienen sentido. Guarda esta clase y su `Main`, sin paquete:

Monster.java (con banderas)

```
public class Monster {
    private boolean sleeping;
    private boolean fleeing;
    private boolean attacking;

    public void fallAsleep() { sleeping = true; }

    public void provoke() {
        if (!fleeing) { attacking = true; }
    }

    public void scare() {
        if (!sleeping) { fleeing = true; }
    }

    public void calmDown() {
        sleeping = false;
        fleeing = false;
        attacking = false;
    }

    public String describe() {
        if (sleeping) { return "duerme"; }
        if (attacking) { return "ataca"; }
        if (fleeing) { return "huye"; }
        return "descansa";
    }
}
```

Main.java

```
1  public class Main {
2      public static void main(String[] args) {
3          Monster monster = new Monster();
4          System.out.println(monster.describe());
5          monster.fallAsleep();
6          System.out.println(monster.describe());
7          monster.scare();
8          System.out.println(monster.describe());
9          monster.provoke();
10         System.out.println(monster.describe());
11         monster.scare();
12         System.out.println(monster.describe());
13         monster.provoke();
14         System.out.println(monster.describe());
15         monster.calmDown();
16         System.out.println(monster.describe());
17     }
18 }
```

Con tres banderas hay $2^3 = 8$ combinaciones, y solo cuatro tienen sentido:

sleeping	fleeing	attacking	¿Tiene sentido?
no	no	no	Sí: descansa
sí	no	no	Sí: duerme
no	sí	no	Sí: huye
no	no	sí	Sí: ataca
sí	sí	no	No
sí	no	sí	No
no	sí	sí	No
sí	sí	sí	No

Ejecútalo antes de tocar nada: verás que la línea 4 imprime «duerme» aunque a un monstruo dormido se le acaba de provocar. Ahora refactoriza, sin cambiar las firmas públicas de `Monster` ni `Main`:

1. Crea `Mood`, una `sealed interface` con cuatro records sin datos: `Idle`, `Sleeping`, `Attacking` y `Fleeing`.
2. En `Mood`, un método `default` por operación, con `switch (this)` sin `default` cuando haga falta: `fallAsleep()`, `provoke()`, `scare()`, `calmDown()` (cada uno devuelve el `Mood` siguiente) y `label()`. Las transiciones son estas:

Operación	Idle	Sleeping	Attacking	Fleeing
`fallAsleep()`	Sleeping	Sleeping	Sleeping	Sleeping
`provoke()`	Attacking	Attacking	Attacking	Fleeing
`scare()`	Fleeing	Sleeping	Fleeing	Fleeing
`calmDown()`	Idle	Idle	Idle	Idle

3. `Monster` guarda un único campo `Mood mood` (empieza en `Idle`) y cada método de `Monster` es una línea que sustituye `mood` por el siguiente.

Está bien si `Main`, sin tocarlo, imprime exactamente

Salida esperada

```
descansa
duerme
duerme
ataca
huye
huye
descansa
```

y si en tu nueva `Monster` no queda ningún `boolean`.

Pista: *Los métodos que devuelven siempre el mismo valor (`fallAsleep` y `calmDown`) no necesitan `switch`. En `Mood.scare()`, el dormido se queda igual y, en los demás, solo cambia el que no huye.*

Práctica ▪ solución en la pág. 539

Ejercicio 12.4: Envenenado y aturdido a la vez ★★★★☆

Hoy un `Fighter` tiene un único `Status`: si aturdes a un envenenado, deja de estar envenenado. Tu equipo quiere que un personaje pueda estar envenenado y aturdido *a la vez*, y que mañana puedan sumarse `Frozen`, `Burning` y `Blessed` sin que el número de tipos se dispare. Diseña el cambio sobre Emberdeep. No hace falta escribir todo el proyecto, pero sí lo siguiente:

1. Cuántos estados necesitarías con la solución actual (un solo `Status`, un record por combinación) si cada uno de los cinco efectos puede estar activo o no, y por qué crece así.
2. Tu modelo: qué tipos y qué campos sustituyen al único `status` de `Fighter`.
3. Cómo cambian `Combat.startTurn` y el aviso al jugador (`StatusTick`) cuando hay varios efectos a la vez, y qué decide el orden en que se aplican.
4. Qué pasa si se aplica dos veces el mismo efecto (dos `Venom` seguidos): tu regla y por qué.
5. Qué tiene que guardar `FighterState` y por qué el Memento sigue siendo inmutable.

Está bien si: (1) das la cifra y la relacionas con 2^n; (2) tu modelo hace crecer el número de piezas con n y no con 2^n; (3) fijas una regla de acumulación para un mismo efecto y la defiendes; (4) el estado guardado en el `Snapshot` no se puede modificar después por quien lo guarda; (5) nombras al menos un coste de tu diseño.

Pista: *Piensa en cuántos «interruptores» independientes hay: cada efecto se puede activar y apagar sin saber nada de los demás. Compara cómo se combinan los estados en una sola máquina y en varias, y recuerda lo que hacía el capítulo 6 con los `TempBuff`.*

Diseño ▪ solución en la pág. 542

Ejercicio 12.5: Hito 13: estados y fases ★★★★☆

Implementa el hito H13 partiendo del estado del capítulo anterior (solución del ejercicio 11.5), en tres partes. Comprueba cada una antes de pasar a la siguiente.

Parte 1 (estados y habilidades).

1. `Status`, en `engine/combat/status/`: `sealed interface` con `Normal()`, `Poisoned(int turns, int damage)` y `Stunned(int turns)`. Un método `default Status next()` con `switch (this)` sin `default`: `Normal` se queda como está; `Poisoned` y `Stunned` pasan a `Normal` si les queda un turno o menos y, si no, se repiten con un turno menos. Un método `default String label()`: `"normal"`, `"veneno"` y `"aturdimiento"`.

2. `Skill`: dos variantes nuevas, `Venom(String name, int turns, int damage)` y `Stun(String name, int turns)`.

3. `GameEvent`: nuevo record `StatusTick(String name, boolean isHero, String statusName, int amount, int hp, int maxHp)`. `LogView` escribe `Zombi sufre 2 ↪ de daño por veneno.` si `amount` es mayor que 0 y `Zombi pierde el turno por ↪ aturdimiento.` si es 0 (estilo `DANGER` si es un héroe); `HudView` dibuja la barra solo si `amount` es mayor que 0; `AchievementTracker` lo ignora.

4. `Fighter`: campo `status` (empieza en `Normal`), `Status status()` y `void setStatus(Status status)`, que rechaza `null`.

5. `Combat`: `boolean startTurn(Fighter fighter)`. Un combatiente muerto no actúa. Con `Normal` no hace nada; con `Poisoned` le quita `min(damage, hp)` de vida, publica `StatusTick` y, si cae, publica `FighterDefeated` y no puede actuar; con `Stunned` publica `StatusTick` con `amount` 0 y no puede actuar. En todos los casos deja puesto `current.next()` y llama a `checkEnd()`. En `enemyPhase`, justo antes de que cada enemigo actúe, se llama a `startTurn` y se salta al que no puede.

6. `UseSkill`: casos `Venom` (pone `Poisoned(turns, damage)` al objetivo) y `Stun` (pone `Stunned(turns)`), ambos con `SkillCast` del lanzador al objetivo. Guarda el estado del objetivo con los otros tres valores y `undo()` lo repone. `SkillfulAi.aimOf`: `Venom` y `Stun` apuntan al objetivo elegido.

7. `ConsoleMenu`: los héroes consultan su estado *al empezar la ronda*, no al empezar su turno individual (si lo hicieran en el turno, deshacer con `u` y volver a un héroe aplicaría el veneno dos veces). Un método privado `startRound(session)` llama a `combat.startTurn` para cada héroe vivo y devuelve los que pueden actuar; el campo `acting` guarda esa lista, y `checkpointActing` la copia al pulsar `c` para que `r` recupere exactamente los héroes de ese momento (los aturdidos no vuelven a la lista). `chooseSkill`: `Venom` y `Stun` piden un enemigo como objetivo.

8. `Main`: `VENOM = new Venom("Daga envenenada", 3, 2)` para Cael y `STUN = new Stun("Golpe aturdidor", 1)` para Borin, ambas al final de sus listas.

Está bien si `./gradlew -q classes` compila, `./gradlew test` sigue pasando y el programa temporal `app/Check13a.java` imprime lo de abajo. Crea a Cael (24, 9, 2) con `Venom("Daga envenenada", 3, 2)` y a Borin (40, 6, 5) con `Stun("Golpe aturdidor", 1)`, una cripta con la semilla 42 y los enemigos `ASH_RAT` y `ZOMBIE`, la fórmula plana, la esquiva al 0 % y un `LogView` sobre un `PlainOutput`. Ronda 1: Cael lanza su primera habilidad sobre el Zombi, Borin la suya sobre la Rata, imprime `estadoDelZombi + " / " + estadoDeLaRata`, fase enemiga e imprime, para el Zombi y para la Rata, nombre, vida, «/», vida máxima, un espacio y el estado (el `toString` del record). Rondas 2 y 3: solo la fase enemiga. Al final imprime lo mismo para el Zombi y para Cael.

Salida esperada de Check13a

```
Cael usa Daga envenenada sobre Zombi.
Borin usa Golpe aturdidor sobre Rata de ceniza.
Poisoned[turns=3, damage=2] / Stunned[turns=1]
Rata de ceniza pierde el turno por aturdimiento.
Zombi sufre 2 de daño por veneno.
Zombi hace 2 de daño físico a Cael.
Zombi 20/22 Poisoned[turns=2, damage=2]
Rata de ceniza 8/8 Normal[]
Rata de ceniza hace 2 de daño físico a Cael.
Zombi sufre 2 de daño por veneno.
Zombi hace 2 de daño físico a Cael.
Rata de ceniza hace 2 de daño físico a Cael.
Zombi sufre 2 de daño por veneno.
Zombi hace 6 de daño físico a Cael.
Zombi 16/22 Normal[]
Cael 10/24 Normal[]
```

Parte 2 (fin del combate y fases).

1. `Fighter.endCombat()`: estado a `Normal`, `shield` a 0 y `stats = gear`. No toca la vida.
2. En `engine/flow/`: `GamePhase` (`sealed interface` con `MainMenu()`, `Exploring(int floor)`, `InCombat(int floor)` y `GameOver(boolean won)`) y `GameFlow implements GameListener`, con constructor `GameFlow(List<Fighter> heroes, int lastFloor)`, `GamePhase phase()` (empieza en `MainMenu`) y las acciones `start(int floor)`, `enterCombat()`, `leaveCombat(boolean heroesWon)` y `descend()`, cada una con un `switch` sobre la fase sin `default`, con la tabla de transiciones de la sección 12.4. Una acción en una fase que no la admite lanza `IllegalStateException` con el mensaje «No se puede X en la fase Y». `leaveCombat` llama a `endCombat()` de cada héroe. `onEvent` llama a `leaveCombat` cuando llega un `CombatEnded` y la fase es `InCombat`.
3. `Main`: crea el `GameFlow` con los héroes y `config.floors()`, llama a `start(floorNumber)` y `enterCombat()`, y lo suscribe al bus entre el `HudView` y el `AchievementTracker`.

Está bien si `./gradlew -q classes` compila y el programa temporal `app/Check13b.java` imprime lo de abajo. Crea a Borin (40, 6, 5), una cripta con la semilla 42 y una sola `ASH_RAT`, fórmula plana, esquiva 0 y un `LogView`. Crea el `GameFlow` con una sola planta como última, suscríbelo después del `LogView`, llama a `start(1)` y `enterCombat()`. Imprime el ataque, la armadura y la defensa de Borin; que se cubra con un `Defend` e imprime el escudo y la defensa y la fase; que ataque a la Rata (la mata) e imprime escudo, defensa, ataque y armadura; imprime la fase; llama a `descend()` e imprime la fase; y, dentro de un `try`, llama a `enterCombat()` e imprime el mensaje de la excepción.

Salida esperada de Check13b

```
Antes: ataque 8, armadura 1, defensa 5
Borin se cubre: +5 de escudo y +2 de defensa este turno.
Tras defender: escudo 5, defensa 7
Fase: InCombat[floor=1]
Borin hace 8 de daño físico a Rata de ceniza.
Rata de ceniza queda fuera de combate.
¡Victoria!
Tras el combate: escudo 0, defensa 5, ataque 8, armadura 1
Fase: Exploring[floor=1]
Fase: GameOver[won=true]
No se puede entrar en combate en la fase GameOver[won=true]
```

Parte 3 (el Snapshot).

1. `FighterState`: cuarto campo `Status status`. Ejecuta `./gradlew -q classes`: el compilador rechaza una llamada. Corrígela.
2. `GameSession.createSnapshot` guarda el estado de cada combatiente y `restore` lo repone con `setStatus`.
3. `SnapshotTest`: añade una segunda prueba, `restoreBringsBackStatus`: guarda una instantánea con Aria `Poisoned(3, 2)`, la deja `Stunned(1)`, restaura y comprueba que vuelve a `Poisoned(3, 2)`.

Está bien si `./gradlew test` termina con `BUILD SUCCESSFUL` (dos pruebas) y, al comentar la línea de `restore` que repone el estado, la prueba nueva falla con `expected: <Poisoned[turns=3, damage=2]> but was: <Stunned[turns=1]>`. Vuelve a ponerla. Borra `Check13a.java` y `Check13b.java`.

Pista: *Para lanzar una habilidad de un combatiente a mano:* `combat.perform(new UseSkill(combat, combat.mediator(), user, skill, target))`. *La primera habilidad de quien la lanza es* `user.skills().get(0)`.

Proyecto ▪ solución en la pág. 544

12.8 Resumen

Resumen

- El olor de State es un objeto cuyo comportamiento depende de un modo que dura, escrito como banderas: datos que solo valen a veces, 2^n combinaciones de las que pocas tienen sentido y un `if` por método. State convierte cada modo en un valor de un tipo y el objeto está siempre en exactamente uno.
- Piezas: el estado (`Status`, interfaz sellada), los estados concretos (`Normal`, `Poisoned`, `Stunned`, records con sus datos), el contexto (`Fighter`) y la transición, un método `next()` con `switch (this)` sin `default`. El compilador señala todos los `switch` cuando añades un estado.
- Las transiciones por el paso del tiempo viven en `next()`; las que provoca un hecho, en quien llama a `setStatus` (`UseSkill`). `Combat.startTurn` consulta el estado al empezar el turno y publica `StatusTick`.

- Segundo uso: `GamePhase` (`MainMenu`, `Exploring`, `InCombat`, `GameOver`) y `GameFlow`, que decide la siguiente fase, rechaza las acciones ilegales y llama a `Fighter.endCombat()` al salir del combate (estado a `Normal`, escudo a 0, `stats = gear`: se pierden los buffs, se conserva el equipo y la vida).
- State frente a Strategy: en Strategy el cliente elige la pieza desde fuera y rara vez cambia; en State el cambio forma parte del dominio y los estados pueden construirse unos a otros.
- Compensa con modos que duran y banderas que se excluyen; no, con dos modos y casi nada que dependa de ellos. Un estado a la vez es un límite: varios efectos independientes piden varias máquinas ($n + m$), no un estado por combinación ($n \times m$). Cuesta más tipos, transiciones repartidas y un campo más en el Memento.
- H13: `combat/status/` y `engine/flow/`; `Skill` gana `Venom` y `Stun`; `GameEvent` gana `StatusTick` (diez records, tres `switch` actualizados); `FighterState` gana `status`. El compilador avisa de quien crea el `FighterState`, pero no de quien olvida reponerlo: eso lo avisa la prueba.

Soluciones de los ejercicios

Solución del ejercicio 12.1 enunciado en la pág. 527

Dos archivos sin paquete, en la misma carpeta.

`Signal.java`

```java
public sealed interface Signal {
    record Red() implements Signal {}

    record Green() implements Signal {}

    // blinksLeft: parpadeos que quedan antes de volver al rojo
    record Flashing(int blinksLeft) implements Signal {}

    default Signal next() {
        return switch (this) {
            case Red red -> new Green();
            case Green green -> new Flashing(2);
            case Flashing flashing -> flashing.blinksLeft() > 1
                    ? new Flashing(flashing.blinksLeft() - 1)
                    : new Red();
        };
    }

    default String label() {
        return switch (this) {
            case Red red -> "rojo";
            case Green green -> "verde";
            case Flashing flashing ->
                    "intermitente " + flashing.blinksLeft();
        };
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Signal signal = new Signal.Red();
        System.out.println(signal.label());
        for (int i = 0; i < 6; i++) {
            signal = signal.next();
            System.out.println(signal.label());
        }
    }
}
```

Salida

```
rojo
verde
intermitente 2
intermitente 1
rojo
verde
intermitente 2
```

Por qué funciona. Cada color es un record y `Signal` es una interfaz sellada, así que el compilador conoce los tres únicos casos. En `next()` y `label()`, el `switch (this)` nombra los tres y no lleva `default`. Los datos del parpadeo viven en el estado: solo `Flashing` tiene `blinksLeft`, y nada en `Main` guarda un contador aparte. La traza: `Red` pasa a `Green`; `Green` pasa a `Flashing(2)`; con 2 parpadeos queda 1 y con 1 vuelve a `Red`. Al añadir `record Amber()` sin tocar nada más, los dos `switch` dejan de compilar y te señalan dónde decidir qué hace el ámbar.

Errores típicos:

- Poner un `default` en el `switch`: compila al añadir `Amber` y el nuevo color cae en un caso que nadie escribió para él.
- Pasar de `Flashing(1)` a `Flashing(0)`: el semáforo parpadearía una vez de más; la condición es `blinksLeft() > 1`.
- Guardar el contador de parpadeos fuera, en `Main` o en un campo mutable: justo lo que State evita.

Solución del ejercicio 12.2

enunciado en la pág. 528

Cuatro pruebas, una por transición. El código probado ya estaba completo en el enunciado; se repite aquí para tener el ejercicio entero. En `src/main/java/scratch/`:

src/main/java/scratch/Condition.java

```java
package scratch;

public sealed interface Condition {
    record Normal() implements Condition {}

    record Poisoned(int turns, int damage) implements Condition {}

    default Condition next() {
        return switch (this) {
            case Normal normal -> normal;
            case Poisoned poisoned -> poisoned.turns() <= 1
                    ? new Normal()
                    : new Poisoned(poisoned.turns() - 1,
                            poisoned.damage());
        };
    }
}
```

Y la prueba, en `src/test/java/scratch/`:

src/test/java/scratch/ConditionTest.java

```java
package scratch;

import static org.junit.jupiter.api.Assertions.assertEquals;

import scratch.Condition.Normal;
import scratch.Condition.Poisoned;
import org.junit.jupiter.api.Test;

class ConditionTest {

    @Test
    void poisonedLosesOneTurnAndKeepsItsDamage() {
        Condition next = new Poisoned(3, 2).next();

        assertEquals(new Poisoned(2, 2), next);
    }

    @Test
    void poisonedWithOneTurnLeftBecomesNormal() {
        Condition next = new Poisoned(1, 2).next();

        assertEquals(new Normal(), next);
    }

    @Test
    void normalStaysNormal() {
        Condition next = new Normal().next();

        assertEquals(new Normal(), next);
    }

    @Test
    void poisonedEndsAfterAsManyStepsAsTurns() {
        Condition condition = new Poisoned(2, 5);

        condition = condition.next();
        assertEquals(new Poisoned(1, 5), condition);
        condition = condition.next();
        assertEquals(new Normal(), condition);
    }
}
```

Por qué funciona. Cada prueba construye un estado, llama a `next()` y compara el resultado con el estado esperado. Como los records comparan por valor, no hace falta leer `turns()` y `damage()` uno a uno: `assertEquals(new Poisoned(2, 2), next)` comprueba las dos cifras a la vez (y que sea un `Poisoned`). La cuarta prueba encadena dos transiciones y comprueba el estado intermedio. Si cambias `<= 1` por `< 1`, `Poisoned(1, 2).next()` pasa a ser `Poisoned(0, 2)` y la segunda prueba falla con `expected: <Normal[]> but was: <Poisoned[turns=0, damage=2]>`.

Errores típicos:

- Probar solo el caso feliz (`Poisoned(3, 2)`): el error de «un turno de más o de menos» solo aparece en el límite, con un turno restante.

- Olvidar la prueba del estado que no cambia (`Normal`): la transición «quieta» también es una transición.
- Importar `assertEquals` sin `static`: no compila.
- Comparar con `==`: dos records distintos con los mismos valores no son la misma referencia, y la prueba fallaría aunque el código estuviera bien.

Solución del ejercicio 12.3 enunciado en la pág. 529

Dos archivos nuevos o reescritos, sin paquete; `Main` no cambia.

Mood.java

```java
public sealed interface Mood {
    record Idle() implements Mood {}

    record Sleeping() implements Mood {}

    record Attacking() implements Mood {}

    record Fleeing() implements Mood {}

    default Mood fallAsleep() {
        return new Sleeping();
    }

    default Mood provoke() {
        return switch (this) {
            case Idle idle -> new Attacking();
            case Sleeping sleeping -> new Attacking();
            case Attacking attacking -> attacking;
            case Fleeing fleeing -> fleeing;
        };
    }

    default Mood scare() {
        return switch (this) {
            case Idle idle -> new Fleeing();
            case Sleeping sleeping -> sleeping;
            case Attacking attacking -> new Fleeing();
            case Fleeing fleeing -> fleeing;
        };
    }

    default Mood calmDown() {
        return new Idle();
    }

    default String label() {
        return switch (this) {
            case Idle idle -> "descansa";
            case Sleeping sleeping -> "duerme";
            case Attacking attacking -> "ataca";
            case Fleeing fleeing -> "huye";
        };
    }
}
```

Monster.java

```
1  public class Monster {
2      private Mood mood = new Mood.Idle();
3
4      public void fallAsleep() {
5          mood = mood.fallAsleep();
6      }
7
8      public void provoke() {
9          mood = mood.provoke();
10     }
11
12     public void scare() {
13         mood = mood.scare();
14     }
15
16     public void calmDown() {
17         mood = mood.calmDown();
18     }
19
20     public String describe() {
21         return mood.label();
22     }
23 }
```

Salida

```
descansa
duerme
duerme
ataca
huye
huye
descansa
```

Por qué funciona. Con tres banderas, `provoke()` sobre un monstruo dormido deja `sleeping` y `attacking` a `true` a la vez, una de las cuatro combinaciones sin sentido, y `describe()` resuelve el empate por el orden de sus `if`: de ahí los «duerme» de las líneas 4 a 6 de la versión antigua. Con `Mood` el monstruo está en un solo valor, y las otras cuatro combinaciones de la tabla *no se pueden escribir*: no hay ningún valor de `Mood` que las represente. Las reglas están en la tabla de transiciones del enunciado, y cada fila es un método de `Mood`: `provoke()` despierta al dormido (pasa a `Attacking`) y no cambia a quien ya huye; `scare()` no asusta al dormido. `Monster` deja de tener lógica: cada método es una línea.

Un enum también valdría. Los cuatro modos no llevan datos, así que un `enum` con un `switch` elimina igualmente las combinaciones ilegales. Con records, cuando mañana `Fleeing` necesite saber *de quién* huye, ese dato vive dentro del estado.

Errores típicos:

- Dejar un campo `boolean` «por si acaso»: reabre la puerta a las combinaciones ilegales.
- Escribir `default -> this` en `scare()` para ahorrar casos: el compilador ya no te avisa cuando añadas un modo.

- Invertir la fila de `provoke()`: el que huye no se pone a atacar porque lo provoquen; sigue huyendo.

Solución del ejercicio 12.4 enunciado en la pág. 531

Solución de referencia.
1. Cuántos estados. Si cada uno de los cinco efectos puede estar activo o no, una sola `Status` necesitaría un record por combinación: $2^5 = 32$ (uno de ellos, `Normal`), cada uno repitiendo los datos de los efectos que contiene. Crece así porque cada efecto nuevo se combina con *todas* las combinaciones anteriores: duplica la cuenta.
2. El modelo. Los efectos son independientes, así que se guardan *por separado*. `Fighter` deja de tener un `Status` y pasa a tener un `Effects`, un record inmutable con la lista de efectos activos; cada efecto es un record de una interfaz sellada `Effect`. Las piezas crecen con n: un record por efecto y nada más.

Effect.java

```
public sealed interface Effect {
    record Poison(int turns, int damage) implements Effect {}

    record Stun(int turns) implements Effect {}

    record Freeze(int turns) implements Effect {}

    int turns();

    // el mismo efecto con un turno menos (puede quedar en 0)
    default Effect next() {
        return switch (this) {
            case Poison poison ->
                    new Poison(poison.turns() - 1, poison.damage());
            case Stun stun -> new Stun(stun.turns() - 1);
            case Freeze freeze -> new Freeze(freeze.turns() - 1);
        };
    }
}
```

Effects.java

```java
import java.util.ArrayList;
import java.util.List;

// Inmutable, como StatSource: un Memento puede guardarlo por referencia.
public record Effects(List<Effect> active) {
    public Effects {
        active = List.copyOf(active);
    }

    // un efecto nuevo sustituye al de su mismo tipo, y los demás siguen
    public Effects with(Effect effect) {
        List<Effect> next = new ArrayList<>();
        for (Effect old : active) {
            if (old.getClass() != effect.getClass()) {
                next.add(old);
            }
        }
        next.add(effect);
        return new Effects(next);
    }

    // al empezar el turno: todos avanzan y los que se acaban se van
    public Effects tick() {
        return new Effects(active.stream()
                .map(Effect::next)
                .filter(effect -> effect.turns() > 0)
                .toList());
    }
}
```

Main.java (comprobación)

```java
import java.util.List;

public class Main {
    public static void main(String[] args) {
        Effects effects = new Effects(List.of())
                .with(new Effect.Poison(2, 3))
                .with(new Effect.Stun(1));
        System.out.println(effects.active());

        effects = effects.with(new Effect.Poison(3, 1));
        System.out.println(effects.active());

        effects = effects.tick();
        System.out.println(effects.active());
    }
}
```

Salida

```
[Poison[turns=2, damage=3], Stun[turns=1]]
[Stun[turns=1], Poison[turns=3, damage=1]]
[Poison[turns=2, damage=1]]
```

3. `startTurn`. Recorre `effects.active()` en el orden en que se aplicaron y, por cada efecto, un `switch` sin `default`: `Poison` quita vida y publica su `StatusTick`; `Stun` publica el suyo con `amount` 0 y marca que el combatiente pierde el turno; `Freeze`, lo mismo con su regla. Al final, `setEffects(effects.tick())`. El evento no cambia: cada efecto publica el suyo. El orden es observable: si el veneno mata antes de que le toque al aturdimiento, el aturdimiento ya no se anuncia, así que hay que cortar el bucle cuando el combatiente cae.
4. El mismo efecto dos veces. Regla: *el último sustituye al anterior* (como muestra la segunda línea de la salida: el segundo veneno, de 3 turnos y 1 de daño, reemplaza al primero). Es simple, predecible y garantiza como máximo un efecto de cada tipo. Acumular sumaría daño sin límite, y renovar la duración con el máximo obligaría a decidir también qué pasa con el daño.
5. El Memento. `FighterState` guarda un `Effects` en lugar de un `Status`. Es inmutable (la lista se copia con `List.copyOf` y cada `Effect` es un record), y `with` y `tick` devuelven un objeto *nuevo* sin tocar el anterior; por eso se guarda por referencia, igual que la cadena `StatSource`.
Criterios para valorar otras soluciones. Sirve cualquier diseño que: dé $2^5 = 32$ y lo explique como producto de combinaciones; crezca con n en piezas; fije una regla para el mismo efecto y la defienda; guarde en el `Snapshot` algo que nadie pueda modificar después (colección copiada o inmutable); y nombre un coste (aquí: un tipo más, `Effects`; el orden de aplicación visible para el jugador; la comparación por `getClass()`, que sería frágil si los efectos tuvieran subclases).
Alternativa válida: una máquina por canal. Si los efectos se agrupan en pocos canales independientes (daño con el tiempo, control), basta un campo de estado por canal: `Status damage` (`Normal`, `Poisoned`, `Burning`) y `Status control` (`Free`, `Stunned`, `Frozen`). Es la solución de «dos máquinas» de Nystrom ($n+m$ en vez de $n \times m$) [19]. Cuesta menos código ahora, pero cada efecto nuevo tiene que caber en un canal (uno que no encaje obliga a añadir otro campo), y el `FighterState` guarda un campo por canal.
Errores típicos:

- Contestar 5! o 5^2 en lugar de 2^5: son combinaciones de presencia o ausencia, no de orden.
- Usar una lista mutable en `Fighter` y guardarla tal cual en el `Snapshot`: la restauración devolvería la misma lista modificada (el bug de la copia superficial del capítulo 8).
- No decidir qué pasa con dos efectos iguales y dejar que se acumulen por accidente.

Solución del ejercicio 12.5 enunciado en la pág. 531

Partes 1 a 3 en orden. Se repiten los archivos enteros (también los que solo cambian un poco), para que la solución esté completa. `Fighter` aparece una sola vez, ya con el método `endCombat()` que usarás en la parte 2, y `Main`, que se toca en las partes 1 y 2, aparece completo en la parte 2.
Parte 1: estados y habilidades.

engine/combat/status/Status.java

```java
package emberdeep.engine.combat.status;

public sealed interface Status {
    record Normal() implements Status {}

    // turns: turnos que le quedan; damage: daño que causa cada turno
    record Poisoned(int turns, int damage) implements Status {}

    record Stunned(int turns) implements Status {}

    // la transición: el estado que queda al consumir un turno
    default Status next() {
        return switch (this) {
            case Normal normal -> normal;
            case Poisoned poisoned -> poisoned.turns() <= 1
                    ? new Normal()
                    : new Poisoned(poisoned.turns() - 1,
                            poisoned.damage());
            case Stunned stunned -> stunned.turns() <= 1
                    ? new Normal()
                    : new Stunned(stunned.turns() - 1);
        };
    }

    // cómo lo nombran el log y el HUD
    default String label() {
        return switch (this) {
            case Normal normal -> "normal";
            case Poisoned poisoned -> "veneno";
            case Stunned stunned -> "aturdimiento";
        };
    }
}
```

engine/character/Skill.java

```
package emberdeep.engine.character;

import emberdeep.engine.combat.damage.DamageType;

public sealed interface Skill {
    String name();

    // el antiguo record Skill(name, power, type), con otro nombre
    record Strike(String name, int power, DamageType type)
            implements Skill {}

    record Taunt(String name) implements Skill {}

    record Heal(String name, int amount) implements Skill {}

    record Summon(String name, EnemyKind kind) implements Skill {}

    // cap. 12: envenena durante turns turnos, damage por turno
    record Venom(String name, int turns, int damage)
            implements Skill {}

    // cap. 12: el objetivo pierde los próximos turns turnos
    record Stun(String name, int turns) implements Skill {}
}
```

engine/event/GameEvent.java (1/2)

```
package emberdeep.engine.event;

import emberdeep.engine.combat.damage.DamageType;
import java.util.List;

public sealed interface GameEvent {
    record DamageDealt(String attacker, String target,
                       boolean targetIsHero, int amount, DamageType type,
                       int targetHp, int targetMaxHp)
            implements GameEvent {}

    record DamageDodged(String attacker, String target,
                        boolean targetIsHero) implements GameEvent {}

    record FighterDefeated(String name, boolean isHero)
            implements GameEvent {}

    record Defended(String name, int shield, int defenseBonus)
            implements GameEvent {}

    record ItemUsed(String user, String target, String itemName,
                    int targetHp, int targetMaxHp) implements GameEvent {}
```

engine/event/GameEvent.java (2/2)

```
    record CombatEnded(boolean heroesWon, List<Integer> heroesHp)
            implements GameEvent {
        public CombatEnded {
            heroesHp = List.copyOf(heroesHp);
        }
    }

    record AchievementUnlocked(String name) implements GameEvent {}

    // cap. 11: una cura de la habilidad Heal (con la vida ya repuesta)
    record FighterHealed(String name, boolean isHero, int amount,
                         int hp, int maxHp) implements GameEvent {}

    // cap. 11: Taunt y Summon; targetName es el provocador o el refuerzo
    record SkillCast(String user, boolean userIsHero, String skillName,
                     String targetName) implements GameEvent {}

    // cap. 12: el estado pesa al empezar el turno; amount > 0 es el daño
    // del veneno y amount = 0 con "aturdimiento" es un turno perdido
    record StatusTick(String name, boolean isHero, String statusName,
                      int amount, int hp, int maxHp)
            implements GameEvent {}
}
```

engine/character/Fighter.java (1/3)

```java
package emberdeep.engine.character;

import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.ai.EnemyAi;
import emberdeep.engine.combat.arena.TurnMediator;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.combat.status.Status;
import emberdeep.engine.combat.status.Status.Normal;
import emberdeep.engine.stats.BaseStats;
import emberdeep.engine.stats.StatKind;
import emberdeep.engine.stats.StatSource;
import emberdeep.engine.stats.TempBuff;
import java.util.List;
import java.util.Objects;

public class Fighter {
    private final String name;
    private final EnemyKind kind;      // null en los héroes
    private final StatSource gear;     // equipo permanente
    private StatSource stats;          // gear más los buffs del combate
    private final EnemyAi ai;          // null en los héroes
    private final List<Skill> skills;
    private int hp;
    private int shield;
    private TurnMediator mediator;     // null hasta unirse a una arena
    private Status status = new Normal();  // cap. 12: un solo estado

    private Fighter(String name, EnemyKind kind, StatSource gear,
                    EnemyAi ai, List<Skill> skills) {
        this.name = name;
        this.kind = kind;
        this.gear = gear;
        this.stats = gear;
        this.ai = ai;
        this.skills = skills;
        this.hp = gear.maxHp();
    }

    public static Fighter ofHero(Hero hero, StatSource gear) {
        return new Fighter(hero.name(), null, gear, null, hero.skills());
    }
```

engine/character/Fighter.java (2/3)

```
    public static Fighter ofEnemy(Enemy enemy) {
        return new Fighter(enemy.name(), enemy.kind(),
                new BaseStats(enemy.stats()), enemy.ai(), enemy.skills());
    }

    public String name() { return name; }
    public StatSource stats() { return stats; }
    public StatSource gear() { return gear; }
    public List<Skill> skills() { return skills; }
    public int hp() { return hp; }
    public boolean isAlive() { return hp > 0; }
    public int shield() { return shield; }
    public Status status() { return status; }

    public void addShield(int amount) {
        shield += amount;
    }

    public int absorbShield(int damage) {
        int absorbed = Math.min(shield, damage);
        shield -= absorbed;
        return damage - absorbed;
    }

    public boolean resists(DamageType type) {
        return kind != null && kind.resistances().contains(type);
    }

    public void takeDamage(int amount) {
        boolean wasAlive = hp > 0;
        hp = Math.max(0, hp - amount);
        if (wasAlive && hp == 0 && mediator != null) {
            mediator.allyDefeated(this);
        }
    }

    // la arena llama a join al empezar el combate y con cada refuerzo
    public void join(TurnMediator mediator) {
        this.mediator = mediator;
    }
```

engine/character/Fighter.java (3/3)

```
    public int heal(int amount) {
        int healed = Math.min(amount, stats.maxHp() - hp);
        hp += healed;
        return healed;
    }

    // fija a la vez los tres números que apuntan los comandos
    public void restore(int hp, int shield, StatSource stats) {
        this.hp = hp;
        this.shield = shield;
        this.stats = stats;
    }

    public void addBuff(StatKind stat, int bonus, int turns) {
        stats = new TempBuff(stats, stat, bonus, turns);
    }

    public void tickBuffs() {
        stats = stats.tick();
    }

    // cap. 12: UseSkill lo cambia al lanzar Venom o Stun, Combat lo
    // avanza con next() y restore lo repone
    public void setStatus(Status status) {
        this.status = Objects.requireNonNull(status);
    }

    // cap. 12: al terminar el combate no queda nada del combate
    public void endCombat() {
        status = new Normal();
        shield = 0;          // el que dejó un Defend
        stats = gear;        // fuera los TempBuff; el equipo se queda
    }

    public CombatCommand act(Combat combat, List<Fighter> heroes) {
        if (ai == null) {
            throw new IllegalStateException(
                    name + " no tiene IA: es un héroe");
        }
        return ai.act(combat, this, heroes);
    }
}
```

engine/combat/Combat.java (1/5)

```java
package emberdeep.engine.combat;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.arena.CombatArena;
import emberdeep.engine.combat.arena.TurnMediator;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.command.CommandHistory;
import emberdeep.engine.combat.damage.Damage;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.combat.status.Status;
import emberdeep.engine.combat.status.Status.Normal;
import emberdeep.engine.combat.status.Status.Poisoned;
import emberdeep.engine.combat.status.Status.Stunned;
import emberdeep.engine.event.EventBus;
import emberdeep.engine.event.GameEvent;
import emberdeep.engine.event.GameEvent.CombatEnded;
import emberdeep.engine.event.GameEvent.DamageDealt;
import emberdeep.engine.event.GameEvent.DamageDodged;
import emberdeep.engine.event.GameEvent.FighterDefeated;
import emberdeep.engine.event.GameEvent.StatusTick;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.List;

public class Combat {
    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final DamageFormula formula;
    private final DamagePipeline pipeline;
    private final EventBus bus;
    private final CommandHistory history = new CommandHistory();
    private final CombatArena arena;
    private int round = 1;
    private boolean ended;
```

engine/combat/Combat.java (2/5)

```
    public Combat(List<Fighter> heroes, List<Fighter> enemies,
                  DamageFormula formula, DamagePipeline pipeline,
                  EventBus bus, BiomeFactory biome) {
        this.heroes = heroes;
        this.enemies = enemies;
        this.formula = formula;
        this.pipeline = pipeline;
        this.bus = bus;
        this.arena = new CombatArena(heroes, enemies, biome);
    }

    public int round() {
        return round;
    }

    public boolean heroesWon() {
        return enemies.stream().noneMatch(Fighter::isAlive);
    }

    public boolean heroesLost() {
        return heroes.stream().noneMatch(Fighter::isAlive);
    }

    public boolean isOver() {
        return heroesWon() || heroesLost();
    }

    public void restoreRound(int round) {
        if (round < 1) {
            throw new IllegalArgumentException("La ronda empieza en 1");
        }
        this.round = round;
    }

    // cap. 11: los comandos reciben el mediador de aquí
    public TurnMediator mediator() {
        return arena;
    }
```

engine/combat/Combat.java (3/5)

```
    public boolean isHero(Fighter fighter) {
        return heroes.contains(fighter);
    }

    // cap. 9: los comandos publican por aquí
    public void publish(GameEvent event) {
        bus.publish(event);
    }

    // los héroes ejecutan sus comandos por aquí
    public void perform(CombatCommand command) {
        history.run(command);
        checkEnd();
    }

    public boolean undo() {
        return history.undo();
    }

    public int damageOf(Fighter attacker, Fighter target) {
        return formula.damage(attacker.stats().attack(),
                              target.stats().defense());
    }

    public int deal(Fighter attacker, Fighter target, Damage damage) {
        int dealt = pipeline.apply(target, damage);
        target.takeDamage(dealt);
        boolean targetIsHero = isHero(target);
        if (dealt == 0) {
            publish(new DamageDodged(attacker.name(), target.name(),
                    targetIsHero));
        } else {
            publish(new DamageDealt(attacker.name(), target.name(),
                    targetIsHero, dealt, damage.type(),
                    target.hp(), target.stats().maxHp()));
        }
        if (!target.isAlive()) {
            publish(new FighterDefeated(target.name(), targetIsHero));
        }
        return dealt;
    }
```

engine/combat/Combat.java (4/5)

```
    // Al empezar su turno, el combatiente consulta su estado: el veneno
    // le quita vida y el aturdimiento le quita el turno. Devuelve true
    // si puede actuar.
    public boolean startTurn(Fighter fighter) {
        if (!fighter.isAlive()) {
            return false;
        }
        Status current = fighter.status();
        boolean hero = isHero(fighter);
        boolean canAct = true;
        switch (current) {
            case Normal normal -> { }
            case Poisoned poisoned -> {
                int lost = Math.min(poisoned.damage(), fighter.hp());
                fighter.takeDamage(lost);
                publish(new StatusTick(fighter.name(), hero,
                        current.label(), lost,
                        fighter.hp(), fighter.stats().maxHp()));
                if (!fighter.isAlive()) {
                    publish(new FighterDefeated(fighter.name(), hero));
                    canAct = false;
                }
            }
            case Stunned stunned -> {
                publish(new StatusTick(fighter.name(), hero,
                        current.label(), 0,
                        fighter.hp(), fighter.stats().maxHp()));
                canAct = false;
            }
        }
        fighter.setStatus(current.next());
        checkEnd();
        return canAct;
    }
```

engine/combat/Combat.java (5/5)

```
    public void enemyPhase() {
        // se recorre una copia: un enemigo puede invocar refuerzos
        for (Fighter enemy : List.copyOf(enemies)) {
            if (!enemy.isAlive()) {
                continue;
            }
            if (heroesLost()) {
                break;
            }
            if (!startTurn(enemy)) {        // cap. 12: manda el estado
                continue;
            }
            enemy.act(this, arena.targetsFor(heroes)).execute();
        }
        checkEnd();
        round++;
        heroes.forEach(Fighter::tickBuffs);
        enemies.forEach(Fighter::tickBuffs);
        arena.endRound();
        history.clear();
    }

    // publica el fin del combate una sola vez
    private void checkEnd() {
        if (ended || !isOver()) {
            return;
        }
        ended = true;
        publish(new CombatEnded(heroesWon(),
                heroes.stream().map(Fighter::hp).toList()));
    }
}
```

engine/combat/command/UseSkill.java (1/3)

```
1  package emberdeep.engine.combat.command;
2
3  import emberdeep.engine.character.Fighter;
4  import emberdeep.engine.character.Skill;
5  import emberdeep.engine.character.Skill.Heal;
6  import emberdeep.engine.character.Skill.Strike;
7  import emberdeep.engine.character.Skill.Stun;
8  import emberdeep.engine.character.Skill.Summon;
9  import emberdeep.engine.character.Skill.Taunt;
10 import emberdeep.engine.character.Skill.Venom;
11 import emberdeep.engine.combat.Combat;
12 import emberdeep.engine.combat.arena.TurnMediator;
13 import emberdeep.engine.combat.damage.Damage;
14 import emberdeep.engine.combat.status.Status;
15 import emberdeep.engine.combat.status.Status.Poisoned;
16 import emberdeep.engine.combat.status.Status.Stunned;
17 import emberdeep.engine.event.GameEvent.FighterHealed;
18 import emberdeep.engine.event.GameEvent.SkillCast;
19 import emberdeep.engine.stats.StatSource;
20
21 public class UseSkill implements CombatCommand {
22     private final Combat combat;
23     private final TurnMediator mediator;
24     private final Fighter user;
25     private final Skill skill;
26     private final Fighter target;
27     private int hpBefore;
28     private int shieldBefore;
29     private StatSource statsBefore;
30     private Status statusBefore;
31
32     public UseSkill(Combat combat, TurnMediator mediator, Fighter user,
33                     Skill skill, Fighter target) {
34         this.combat = combat;
35         this.mediator = mediator;
36         this.user = user;
37         this.skill = skill;
38         this.target = target;
39     }
```

engine/combat/command/UseSkill.java (2/3)

```
    @Override
    public void execute() {
        hpBefore = target.hp();
        shieldBefore = target.shield();
        statsBefore = target.stats();
        statusBefore = target.status();
        switch (skill) {
            case Strike strike -> combat.deal(user, target,
                    new Damage(strike.power(), strike.type()));
            case Taunt taunt -> {
                mediator.taunt(user);
                combat.publish(new SkillCast(user.name(),
                        combat.isHero(user), taunt.name(), user.name()));
            }
            case Heal heal -> {
                int healed = mediator.healAlly(user, target,
                        heal.amount());
                combat.publish(new FighterHealed(target.name(),
                        combat.isHero(target), healed,
                        target.hp(), target.stats().maxHp()));
            }
            case Summon summon -> {
                Fighter reinforcement =
                        mediator.summon(user, summon.kind());
                combat.publish(new SkillCast(user.name(),
                        combat.isHero(user), summon.name(),
                        reinforcement.name()));
            }
            case Venom venom -> {
                target.setStatus(new Poisoned(venom.turns(),
                        venom.damage()));
                combat.publish(new SkillCast(user.name(),
                        combat.isHero(user), venom.name(),
                        target.name()));
            }
            case Stun stun -> {
                target.setStatus(new Stunned(stun.turns()));
                combat.publish(new SkillCast(user.name(),
                        combat.isHero(user), stun.name(),
                        target.name()));
            }
        }
    }
```

engine/combat/command/UseSkill.java (3/3)

```
    @Override
    public void undo() {
        target.restore(hpBefore, shieldBefore, statsBefore);
        target.setStatus(statusBefore);
    }
}
```

engine/combat/ai/SkillfulAi.java (1/2)

```java
package emberdeep.engine.combat.ai;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Skill.Heal;
import emberdeep.engine.character.Skill.Strike;
import emberdeep.engine.character.Skill.Stun;
import emberdeep.engine.character.Skill.Summon;
import emberdeep.engine.character.Skill.Taunt;
import emberdeep.engine.character.Skill.Venom;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.Attack;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.command.UseSkill;
import java.util.List;

public class SkillfulAi implements EnemyAi {
    private final EnemyAi targeting;

    public SkillfulAi(EnemyAi targeting) {
        this.targeting = targeting;
    }

    @Override
    public Fighter choose(Fighter self, List<Fighter> heroes) {
        return targeting.choose(self, heroes);
    }
```

engine/combat/ai/SkillfulAi.java (2/2)

```
    @Override
    public CombatCommand act(Combat combat, Fighter self,
                             List<Fighter> heroes) {
        Fighter target = choose(self, heroes);
        if (combat.round() % 3 == 0 && !self.skills().isEmpty()) {
            Skill skill = self.skills().get(0);
            return new UseSkill(combat, combat.mediator(), self, skill,
                    aimOf(skill, self, target));
        }
        return new Attack(combat, self, target);
    }

    // a quién apunta cada habilidad que un enemigo sabe lanzar
    private static Fighter aimOf(Skill skill, Fighter self,
                                 Fighter target) {
        return switch (skill) {
            case Strike strike -> target;
            case Summon summon -> self;
            case Venom venom -> target;
            case Stun stun -> target;
            case Taunt taunt -> throw new IllegalStateException(
                    self.name() + " no sabe provocar");
            case Heal heal -> throw new IllegalStateException(
                    self.name() + " no sabe curar");
        };
    }
}
```

ui/view/LogView.java (1/3)

```
package emberdeep.ui.view;

import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.event.GameEvent;
import emberdeep.engine.event.GameEvent.AchievementUnlocked;
import emberdeep.engine.event.GameEvent.CombatEnded;
import emberdeep.engine.event.GameEvent.DamageDealt;
import emberdeep.engine.event.GameEvent.DamageDodged;
import emberdeep.engine.event.GameEvent.Defended;
import emberdeep.engine.event.GameEvent.FighterDefeated;
import emberdeep.engine.event.GameEvent.FighterHealed;
import emberdeep.engine.event.GameEvent.ItemUsed;
import emberdeep.engine.event.GameEvent.SkillCast;
import emberdeep.engine.event.GameEvent.StatusTick;
import emberdeep.engine.event.GameListener;
import emberdeep.ui.output.Output;
import emberdeep.ui.output.Style;

public class LogView extends View implements GameListener {
    public LogView(Output output) {
        super(output);
    }
```

ui/view/LogView.java (2/3)

```
    @Override
    public void onEvent(GameEvent event) {
        switch (event) {
            case DamageDealt e -> line(e.attacker() + " hace "
                    + e.amount() + " de daño " + kind(e.type())
                    + " a " + e.target() + ".",
                    e.targetIsHero() ? Style.DANGER : Style.NORMAL);
            case DamageDodged e -> line("El golpe de " + e.attacker()
                    + " a " + e.target() + " no hace daño.");
            case FighterDefeated e -> line(e.name()
                    + " queda fuera de combate.",
                    e.isHero() ? Style.DANGER : Style.NORMAL);
            case Defended e -> line(e.name() + " se cubre: +"
                    + e.shield() + " de escudo y +" + e.defenseBonus()
                    + " de defensa este turno.");
            case ItemUsed e -> line(e.user() + " usa " + e.itemName()
                    + " sobre " + e.target() + ".", Style.GOOD);
            case CombatEnded e -> line(e.heroesWon()
                    ? "¡Victoria!" : "Derrota...",
                    e.heroesWon() ? Style.GOOD : Style.DANGER);
            case AchievementUnlocked e -> line(
                    "¡Logro desbloqueado: " + e.name() + "!", Style.GOOD);
            case FighterHealed e -> line(e.name() + " recupera "
                    + e.amount() + " de vida.", Style.GOOD);
            case SkillCast e -> line(castText(e));
            case StatusTick e -> line(tickText(e),
                    e.isHero() ? Style.DANGER : Style.NORMAL);
        }
    }
```

ui/view/LogView.java (3/3)

```
    // sin "sobre" cuando la habilidad es para quien la lanza (Taunt)
    private static String castText(SkillCast e) {
        String text = e.user() + " usa " + e.skillName();
        if (e.targetName().equals(e.user())) {
            return text + ".";
        }
        return text + " sobre " + e.targetName() + ".";
    }

    // amount > 0: daño del estado; amount = 0: turno perdido
    private static String tickText(StatusTick e) {
        if (e.amount() > 0) {
            return e.name() + " sufre " + e.amount() + " de daño por "
                    + e.statusName() + ".";
        }
        return e.name() + " pierde el turno por " + e.statusName() + ".";
    }

    private static String kind(DamageType type) {
        return switch (type) {
            case PHYSICAL -> "físico";
            case FIRE -> "de fuego";
        };
    }
}
```

ui/view/HudView.java (1/2)

```
package emberdeep.ui.view;

import emberdeep.engine.event.GameEvent;
import emberdeep.engine.event.GameEvent.AchievementUnlocked;
import emberdeep.engine.event.GameEvent.CombatEnded;
import emberdeep.engine.event.GameEvent.DamageDealt;
import emberdeep.engine.event.GameEvent.DamageDodged;
import emberdeep.engine.event.GameEvent.Defended;
import emberdeep.engine.event.GameEvent.FighterDefeated;
import emberdeep.engine.event.GameEvent.FighterHealed;
import emberdeep.engine.event.GameEvent.ItemUsed;
import emberdeep.engine.event.GameEvent.SkillCast;
import emberdeep.engine.event.GameEvent.StatusTick;
import emberdeep.engine.event.GameListener;
import emberdeep.ui.output.Output;
import emberdeep.ui.output.Style;

public class HudView extends View implements GameListener {
    private static final int WIDTH = 10;

    public HudView(Output output) {
        super(output);
    }
```

ui/view/HudView.java (2/2)

```
    @Override
    public void onEvent(GameEvent event) {
        switch (event) {
            case DamageDealt e -> bar(e.target(), e.targetHp(),
                    e.targetMaxHp());
            case ItemUsed e -> bar(e.target(), e.targetHp(),
                    e.targetMaxHp());
            case FighterHealed e -> bar(e.name(), e.hp(), e.maxHp());
            case DamageDodged e -> { }
            case FighterDefeated e -> { }
            case Defended e -> { }
            case CombatEnded e -> { }
            case AchievementUnlocked e -> { }
            case SkillCast e -> { }
            case StatusTick e -> {
                if (e.amount() > 0) {
                    bar(e.name(), e.hp(), e.maxHp());
                }
            }
        }
    }

    // casillas llenas = hp * 10 / maxHp, redondeado hacia arriba
    private void bar(String name, int hp, int maxHp) {
        int filled = (hp * WIDTH + maxHp - 1) / maxHp;
        line("  " + name + " [" + "#".repeat(filled)
                + ".".repeat(WIDTH - filled) + "] " + hp + "/" + maxHp,
                styleOf(hp, maxHp));
    }

    // rojo con un cuarto de vida o menos, verde con más de la mitad
    private static Style styleOf(int hp, int maxHp) {
        if (hp * 4 <= maxHp) {
            return Style.DANGER;
        }
        return hp * 2 > maxHp ? Style.GOOD : Style.NORMAL;
    }
}
```

engine/achievement/AchievementTracker.java (1/2)

```java
package emberdeep.engine.achievement;

import emberdeep.engine.event.EventBus;
import emberdeep.engine.event.GameEvent;
import emberdeep.engine.event.GameEvent.AchievementUnlocked;
import emberdeep.engine.event.GameEvent.CombatEnded;
import emberdeep.engine.event.GameEvent.DamageDealt;
import emberdeep.engine.event.GameEvent.DamageDodged;
import emberdeep.engine.event.GameEvent.Defended;
import emberdeep.engine.event.GameEvent.FighterDefeated;
import emberdeep.engine.event.GameEvent.FighterHealed;
import emberdeep.engine.event.GameEvent.ItemUsed;
import emberdeep.engine.event.GameEvent.SkillCast;
import emberdeep.engine.event.GameEvent.StatusTick;
import emberdeep.engine.event.GameListener;
import java.util.HashSet;
import java.util.Set;

public class AchievementTracker implements GameListener {
    private static final int DODGES_NEEDED = 5;

    private final EventBus bus;
    private final Set<String> unlocked = new HashSet<>();
    private int heroDodges;

    public AchievementTracker(EventBus bus) {
        this.bus = bus;
    }
```

engine/achievement/AchievementTracker.java (2/2)

```
    @Override
    public void onEvent(GameEvent event) {
        switch (event) {
            case DamageDealt e -> {
                if (!e.targetIsHero()) {
                    unlock("Primera sangre");
                }
            }
            case DamageDodged e -> {
                if (e.targetIsHero()) {
                    heroDodges++;
                    if (heroDodges == DODGES_NEEDED) {
                        unlock("Intocable");
                    }
                }
            }
            case CombatEnded e -> {
                if (e.heroesWon() && e.heroesHp().contains(1)) {
                    unlock("Último aliento");
                }
            }
            case FighterDefeated e -> { }
            case Defended e -> { }
            case ItemUsed e -> { }
            case AchievementUnlocked e -> { }
            case FighterHealed e -> { }
            case SkillCast e -> { }
            case StatusTick e -> { }
        }
    }

    private void unlock(String name) {
        if (unlocked.add(name)) {
            bus.publish(new AchievementUnlocked(name));
        }
    }
}
```

ui/ConsoleMenu.java (1/8)

```
package emberdeep.ui;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Skill.Heal;
import emberdeep.engine.character.Skill.Strike;
import emberdeep.engine.character.Skill.Stun;
import emberdeep.engine.character.Skill.Summon;
import emberdeep.engine.character.Skill.Taunt;
import emberdeep.engine.character.Skill.Venom;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.Attack;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.command.Defend;
import emberdeep.engine.combat.command.UseItem;
import emberdeep.engine.combat.command.UseSkill;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.save.SnapshotStore;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.TileType;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.input.InputSource;
import emberdeep.ui.view.MapView;
import java.io.PrintStream;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class ConsoleMenu {
    private enum Turn { ACTED, UNDONE, RESTORED }

    private final InputSource input;
    private final PrintStream out;
    private final MapView mapView;
    private final SnapshotStore checkpoints = new SnapshotStore();
    private int checkpointHero;     // a qué héroe le tocaba al guardar
    private List<Fighter> acting = List.of();       // quién actúa ahora
    private List<Fighter> checkpointActing = List.of();   // y al guardar
```

ui/ConsoleMenu.java (2/8)

```
    public ConsoleMenu(InputSource input, PrintStream out,
                       MapView mapView) {
        this.input = input;
        this.out = out;
        this.mapView = mapView;
    }

    public int askFormula() {
        out.println("Fórmula de daño:");
        out.println("  [1] plana: ataque - defensa");
        out.println("  [2] escalada: ataque x 100 / (100 + defensa)");
        return ask(2);
    }

    public void run(GameSession session) {
        Combat combat = session.combat();
        out.println("=== Emberdeep ===");
        while (!combat.isOver()) {
            out.println("--- Ronda " + session.round() + " ---");
            acting = startRound(session);
            int i = 0;
            while (i < acting.size() && !combat.isOver()) {
                Turn turn = heroTurn(session, acting.get(i), i);
                switch (turn) {
                    case ACTED -> i++;
                    case UNDONE -> i = Math.max(0, i - 1);
                    case RESTORED -> {
                        acting = checkpointActing;
                        i = checkpointHero;
                    }
                }
            }
            if (!combat.isOver()) {
                combat.enemyPhase();
            }
        }
    }

    private List<Fighter> aliveHeroes(GameSession session) {
        return session.heroes().stream().filter(Fighter::isAlive).toList();
    }
```

ui/ConsoleMenu.java (3/8)

```
    // Cap. 12: cada héroe consulta su estado al empezar la ronda. Actúan
    // los que siguen vivos y no pierden el turno.
    private List<Fighter> startRound(GameSession session) {
        List<Fighter> ready = new ArrayList<>();
        for (Fighter hero : aliveHeroes(session)) {
            if (session.combat().startTurn(hero)) {
                ready.add(hero);
            }
        }
        return ready;
    }
```

ui/ConsoleMenu.java (4/8)

```
    // ACTED: el héroe actúa; UNDONE: se deshace la acción anterior;
    // RESTORED: se vuelve a un punto de control
    private Turn heroTurn(GameSession session, Fighter hero, int index) {
        Combat combat = session.combat();
        while (true) {
            out.println("Héroes:   " + describe(session.heroes()));
            out.println("Enemigos: " + describe(session.enemies()));
            out.println("Objetos:  " + describeItems(session.inventory()));
            out.println("Turno de " + hero.name()
                    + ". [1] atacar  [2] defender  [3] objeto"
                    + "  [4] habilidad  [u] deshacer"
                    + "  [c] guardar  [r] volver");
            String key = input.next();
            if (key.equals("u")) {
                if (combat.undo()) {
                    out.println("Deshaces la última acción.");
                    return Turn.UNDONE;
                }
                out.println("No hay nada que deshacer en esta ronda.");
                continue;
            }
            if (key.equals("c")) {
                saveCheckpoint(session, index);
                continue;
            }
            if (key.equals("r")) {
                if (loadCheckpoint(session)) {
                    return Turn.RESTORED;
                }
                continue;
            }
            CombatCommand command = switch (key) {
                case "1" -> new Attack(combat, hero,
                        askTarget("Objetivo", session.enemies()));
                case "2" -> new Defend(combat, hero);
                case "3" -> chooseItem(session, hero);
                case "4" -> chooseSkill(session, hero);
                default -> {
                    out.println("Tecla no válida.");
                    yield null;
                }
            };
            if (command != null) {
                combat.perform(command);
                return Turn.ACTED;
            }
        }
    }
```

ui/ConsoleMenu.java (5/8)

```
    private void saveCheckpoint(GameSession session, int index) {
        checkpoints.push(session.createSnapshot());
        checkpointHero = index;
        checkpointActing = acting;
        out.println("Punto de control guardado ("
                + checkpoints.size() + ").");
    }

    // false si todavía no hay ninguno
    private boolean loadCheckpoint(GameSession session) {
        if (checkpoints.isEmpty()) {
            out.println("No hay puntos de control.");
            return false;
        }
        session.restore(checkpoints.last());
        out.println("Vuelves al último punto de control (ronda "
                + session.round() + ").");
        return true;
    }

    private CombatCommand chooseItem(GameSession session, Fighter hero) {
        List<Item> items = session.inventory().items();
        if (items.isEmpty()) {
            out.println("No te quedan objetos.");
            return null;
        }
        Item item = items.get(choose("Objeto",
                items.stream().map(Item::name).toList()));
        Fighter target = item.kind() == ItemKind.HEALING
                ? askTarget("Aliado", session.heroes())
                : askTarget("Objetivo", session.enemies());
        return new UseItem(session, hero, target, item);
    }
```

ui/ConsoleMenu.java (6/8)

```
    private CombatCommand chooseSkill(GameSession session, Fighter hero) {
        List<Skill> skills = hero.skills();
        if (skills.isEmpty()) {
            out.println("No tienes habilidades.");
            return null;
        }
        Skill skill = skills.get(choose("Habilidad",
                skills.stream().map(Skill::name).toList()));
        Fighter target = switch (skill) {
            case Strike strike -> askTarget("Objetivo", session.enemies());
            case Heal heal -> askTarget("Aliado", session.heroes());
            case Taunt taunt -> hero;       // se lanza sobre sí mismo
            case Venom venom -> askTarget("Objetivo", session.enemies());
            case Stun stun -> askTarget("Objetivo", session.enemies());
            case Summon summon -> hero;     // solo la usan los enemigos
        };
        Combat combat = session.combat();
        return new UseSkill(combat, combat.mediator(), hero, skill,
                target);
    }

    private Fighter askTarget(String title, List<Fighter> group) {
        List<Fighter> alive = group.stream()
                .filter(Fighter::isAlive).toList();
        return alive.get(choose(title,
                alive.stream().map(Fighter::name).toList()));
    }

    // Muestra "Título:  [1] a  [2] b" y devuelve la posición (desde 0)
    private int choose(String title, List<String> labels) {
        StringBuilder menu = new StringBuilder(title + ":");
        for (int i = 0; i < labels.size(); i++) {
            menu.append("  [").append(i + 1).append("] ")
                    .append(labels.get(i));
        }
        out.println(menu);
        return ask(labels.size()) - 1;
    }
```

ui/ConsoleMenu.java (7/8)

```
    private int ask(int max) {
        while (true) {
            String line = input.next();
            try {
                int n = Integer.parseInt(line);
                if (n >= 1 && n <= max) {
                    return n;
                }
            } catch (NumberFormatException e) {
                // no es un número: se avisa más abajo
            }
            out.println("Escribe un número entre 1 y " + max + ".");
        }
    }

    private String describe(List<Fighter> group) {
        return group.stream()
                .map(f -> f.name() + " " + f.hp() + "/"
                        + f.stats().maxHp())
                .collect(Collectors.joining(", "));
    }

    private String describeItems(Inventory inventory) {
        if (inventory.items().isEmpty()) {
            return "(ninguno)";
        }
        return inventory.items().stream()
                .map(Item::name)
                .collect(Collectors.joining(", "));
    }
```

ui/ConsoleMenu.java (8/8)

```
    public boolean explore(GameMap map, TileTypes types) {
        while (true) {
            mapView.show(map);
            out.println("[w a s d] mover, [t] casillas, [q] salir");
            String key = input.next();
            switch (key) {
                case "w" -> step(map, 0, -1);
                case "a" -> step(map, -1, 0);
                case "s" -> step(map, 0, 1);
                case "d" -> step(map, 1, 0);
                case "t" -> showTiles(map, types);
                case "q" -> { return false; }
                default -> out.println("Tecla no válida.");
            }
            if (map.onStairs()) {
                out.println("Bajas por la escalera.");
                return true;
            }
        }
    }

    private void step(GameMap map, int dx, int dy) {
        if (!map.move(dx, dy)) {
            out.println("No puedes pasar.");
        }
    }

    private void showTiles(GameMap map, TileTypes types) {
        TileType corner = map.tileAt(0, 0);
        TileType opposite = map.tileAt(map.width() - 1, map.height() - 1);
        out.println("Casillas: " + map.width() * map.height()
                + ", TileType creados: " + types.createdCount());
        out.println("Esquinas opuestas, mismo objeto: "
                + (corner == opposite));
    }
}
```

En `Main`, de esta parte solo hacen falta las dos habilidades nuevas (el archivo entero está en la parte 2):

Fragmento de app/Main.java: parte 1

```
// imports nuevos
import emberdeep.engine.character.Skill.Stun;
import emberdeep.engine.character.Skill.Venom;

// constantes nuevas, junto a TAUNT
private static final Skill VENOM = new Venom("Daga envenenada", 3, 2);
private static final Skill STUN = new Stun("Golpe aturdidor", 1);

// en heroes(): Borin acaba con .skill(TAUNT).skill(STUN) y Cael con
// .skill(POWER_STRIKE).skill(FLARE).skill(VENOM)
```

app/Check13a.java (1/3)

```
package emberdeep.app;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.UseSkill;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.ui.output.PlainOutput;
import emberdeep.ui.view.LogView;
import java.util.List;
import java.util.Random;
```

app/Check13a.java (2/3)

```
public class Check13a {
    public static void main(String[] args) {
        Random random = new Random(42);
        Hero caelData = new HeroBuilder().name("Cael")
                .stats(new Stats(24, 9, 2))
                .skill(new Skill.Venom("Daga envenenada", 3, 2)).build();
        Hero borinData = new HeroBuilder().name("Borin")
                .stats(new Stats(40, 6, 5))
                .skill(new Skill.Stun("Golpe aturdidor", 1)).build();
        GameSession session = new GameSession(
                List.of(caelData, borinData), new CryptBiome(random),
                List.of(EnemyKind.ASH_RAT, EnemyKind.ZOMBIE),
                new FlatFormula(), new DamagePipeline(random, 0.0));
        session.events().subscribe(
                new LogView(new PlainOutput(System.out)));

        Combat combat = session.combat();
        Fighter cael = session.heroes().get(0);
        Fighter borin = session.heroes().get(1);
        Fighter rat = session.enemies().get(0);
        Fighter zombie = session.enemies().get(1);

        // Ronda 1: Cael envenena al Zombi y Borin aturde a la Rata
        combat.perform(cast(combat, cael, zombie));
        combat.perform(cast(combat, borin, rat));
        System.out.println(zombie.status() + " / " + rat.status());
        combat.enemyPhase();
        System.out.println(show(zombie));
        System.out.println(show(rat));

        // Ronda 2: la Rata ya actúa; el veneno sigue
        combat.enemyPhase();

        // Ronda 3: el veneno se acaba y el Zombi usa su habilidad
        combat.enemyPhase();
        System.out.println(show(zombie));
        System.out.println(show(cael));
    }
```

app/Check13a.java (3/3)

```
    // la primera habilidad de quien la lanza, sobre el objetivo dado
    private static UseSkill cast(Combat combat, Fighter user,
                                 Fighter target) {
        return new UseSkill(combat, combat.mediator(), user,
                user.skills().get(0), target);
    }

    private static String show(Fighter fighter) {
        return fighter.name() + " " + fighter.hp() + "/"
                + fighter.stats().maxHp() + " " + fighter.status();
    }
}
```

Por qué funciona (parte 1). `Status` es una interfaz sellada con tres records. Cuando añades `Stunned`, dejan de compilar `next()`, `label()` y el `switch` de `Combat.startTurn`, y cuando añades `Venom` y `Stun` a `Skill`, los de `UseSkill`, `SkillfulAi.aimOf` y `ConsoleMenu.chooseSkill`; y al añadir `StatusTick` a `GameEvent`, los de `LogView`, `HudView` y `AchievementTracker`. Son ocho `switch` que el compilador te lista, uno tras otro.
Los números de `Check13a`, con Cael (ataque 11 y armadura 1 con el equipo, defensa 2), la Rata (8, 5, 0) y el Zombi (22, 5, 1), con fórmula plana y esquiva al 0 %:

- *Ronda 1, héroes.* Cael envenena al Zombi (`Poisoned(3, 2)`) y Borin aturde a la Rata (`Stunned(1)`); las dos son un `SkillCast`.
- *Ronda 1, enemigos.* La Rata pierde el turno: `startTurn` publica `StatusTick` con `amount` 0, deja `Normal` (de `Stunned(1)`, `next()` da `Normal`) y devuelve `false`, así que `enemyPhase` la salta. El Zombi pierde mín$(2, 22) = 2$ de vida (20), queda en `Poisoned(2, 2)` y ataca a Cael, el primero vivo: $5 - 2 = 3$ por la fórmula y $3 - 1 = 2$ por la armadura (Cael, 22).
- *Ronda 2.* La Rata ya está en `Normal` y actúa: elige a Cael, el de menos vida (22 frente a 40), y le hace 2 (20). El Zombi: veneno (18), `Poisoned(1, 2)` y ataque de 2 (Cael, 18).
- *Ronda 3.* Rata a Cael: 2 (16). El Zombi: último veneno (16), `Normal`, y, por ser una tercera ronda, usa «Abrazo pútrido» (un golpe de 7 que no pasa por la fórmula): la armadura lo deja en 6 (Cael, 10).

`ConsoleMenu` es la pieza más delicada de la parte. Los héroes consultan su estado al empezar la *ronda*, en `startRound`, y solo los que pueden actuar entran en `acting`. Si lo hicieras al empezar el turno de cada héroe, pulsar `u` para deshacer y volver al héroe anterior ejecutaría su `startTurn` otra vez y el veneno haría daño dos veces. Por la misma razón `r` no puede recalcular los héroes con `aliveHeroes`: un héroe aturdido esa ronda no está en `acting` y reaparecería. Por eso `saveCheckpoint` copia la lista (`checkpointActing`) y la restauración la recupera.
En `UseSkill`, `statusBefore` se guarda con los otros tres valores y `undo()` lo repone: un `u` tras lanzar el veneno lo quita. En `SkillfulAi.aimOf`, `Venom` y `Stun` apuntan al héroe que eligió `choose`, como un `Strike`. `./gradlew test` sigue pasando: `SnapshotTest` no menciona `status`.
Parte 2: fin del combate y fases.

engine/flow/GamePhase.java

```java
package emberdeep.engine.flow;

public sealed interface GamePhase {
    record MainMenu() implements GamePhase {}

    record Exploring(int floor) implements GamePhase {}

    record InCombat(int floor) implements GamePhase {}

    record GameOver(boolean won) implements GamePhase {}
}
```

engine/flow/GameFlow.java (1/3)

```java
package emberdeep.engine.flow;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.event.GameEvent;
import emberdeep.engine.event.GameEvent.CombatEnded;
import emberdeep.engine.event.GameListener;
import emberdeep.engine.flow.GamePhase.Exploring;
import emberdeep.engine.flow.GamePhase.GameOver;
import emberdeep.engine.flow.GamePhase.InCombat;
import emberdeep.engine.flow.GamePhase.MainMenu;
import java.util.List;

public class GameFlow implements GameListener {
    private final List<Fighter> heroes;
    private final int lastFloor;
    private GamePhase phase = new MainMenu();

    public GameFlow(List<Fighter> heroes, int lastFloor) {
        this.heroes = heroes;
        this.lastFloor = lastFloor;
    }

    public GamePhase phase() {
        return phase;
    }

    // del menú a recorrer la planta indicada
    public void start(int floor) {
        phase = switch (phase) {
            case MainMenu menu -> new Exploring(floor);
            case Exploring exploring -> throw notNow("empezar");
            case InCombat inCombat -> throw notNow("empezar");
            case GameOver gameOver -> throw notNow("empezar");
        };
    }
```

engine/flow/GameFlow.java (2/3)

```
    // de recorrer la planta a combatir en ella
    public void enterCombat() {
        phase = switch (phase) {
            case Exploring exploring -> new InCombat(exploring.floor());
            case MainMenu menu -> throw notNow("entrar en combate");
            case InCombat inCombat -> throw notNow("entrar en combate");
            case GameOver gameOver -> throw notNow("entrar en combate");
        };
    }

    // al salir de InCombat se limpia el combate: endCombat()
    public void leaveCombat(boolean heroesWon) {
        phase = switch (phase) {
            case InCombat inCombat -> {
                heroes.forEach(Fighter::endCombat);
                yield heroesWon
                        ? new Exploring(inCombat.floor())
                        : new GameOver(false);
            }
            case MainMenu menu -> throw notNow("salir del combate");
            case Exploring exploring -> throw notNow("salir del combate");
            case GameOver gameOver -> throw notNow("salir del combate");
        };
    }
```

engine/flow/GameFlow.java (3/3)

```
    // por la escalera: a la planta siguiente o, en la última, al final
    public void descend() {
        phase = switch (phase) {
            case Exploring exploring -> exploring.floor() >= lastFloor
                    ? new GameOver(true)
                    : new Exploring(exploring.floor() + 1);
            case MainMenu menu -> throw notNow("bajar");
            case InCombat inCombat -> throw notNow("bajar");
            case GameOver gameOver -> throw notNow("bajar");
        };
    }

    // solo le importa un evento: basta un instanceof, sin switch
    @Override
    public void onEvent(GameEvent event) {
        if (event instanceof CombatEnded ended
                && phase instanceof InCombat) {
            leaveCombat(ended.heroesWon());
        }
    }

    private IllegalStateException notNow(String action) {
        return new IllegalStateException(
                "No se puede " + action + " en la fase " + phase);
    }
}
```

app/Main.java (1/6)

```java
package emberdeep.app;

import emberdeep.engine.achievement.AchievementTracker;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Skill.Heal;
import emberdeep.engine.character.Skill.Strike;
import emberdeep.engine.character.Skill.Stun;
import emberdeep.engine.character.Skill.Taunt;
import emberdeep.engine.character.Skill.Venom;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.event.EventBus;
import emberdeep.engine.flow.GameFlow;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.Dungeon;
import emberdeep.engine.world.DungeonBuilder;
import emberdeep.engine.world.Floor;
import emberdeep.engine.world.biome.BiomeFactory;
import emberdeep.engine.world.biome.CavernBiome;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.MapData;
import emberdeep.engine.world.map.MapParser;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.ConsoleMenu;
import emberdeep.ui.input.ConsoleInput;
import emberdeep.ui.input.InputSource;
import emberdeep.ui.input.ScriptInput;
import emberdeep.ui.output.AnsiOutput;
import emberdeep.ui.output.Output;
import emberdeep.ui.output.PlainOutput;
import emberdeep.ui.view.HudView;
import emberdeep.ui.view.LogView;
import emberdeep.ui.view.MapView;
import java.io.IOException;
import java.nio.file.Path;
import java.util.Arrays;
import java.util.List;
import java.util.Random;
import java.util.Scanner;
```

app/Main.java (2/6)

```
public class Main {
    private static final Skill POWER_STRIKE =
            new Strike("Golpe poderoso", 12, DamageType.PHYSICAL);
    private static final Skill FLARE =
            new Strike("Llamarada", 10, DamageType.FIRE);
    private static final Skill HEAL = new Heal("Luz curativa", 8);
    private static final Skill TAUNT = new Taunt("Grito de guerra");
    private static final Skill VENOM = new Venom("Daga envenenada", 3, 2);
    private static final Skill STUN = new Stun("Golpe aturdidor", 1);

    private final GameConfig config;

    private Main(GameConfig config) {
        this.config = config;
    }

    public static void main(String[] args) throws IOException {
        GameConfig config = new GameConfig(42L, 3, new FlatFormula());
        new Main(config).play(args);
    }
```

app/Main.java (3/6)

```
    private void play(String[] args) throws IOException {
        String script = scriptOf(args);
        String[] options = optionsOf(args);
        int floorNumber = chooseFloor(options);
        InputSource input = script == null
                ? new ConsoleInput(new Scanner(System.in), System.out)
                : new ScriptInput(Path.of(script), System.out);
        Output output = chooseOutput(args, script != null);
        ConsoleMenu menu = new ConsoleMenu(input, System.out,
                new MapView(output));
        Random random = new Random(config.seed());

        BiomeFactory crypt = new CryptBiome(random);
        BiomeFactory cavern = new CavernBiome(random);
        List<EnemyKind> cryptKinds = List.of(
                EnemyKind.SKELETON, EnemyKind.ASH_RAT, EnemyKind.ZOMBIE);
        List<EnemyKind> cavernKinds = List.of(
                EnemyKind.GOBLIN, EnemyKind.BAT, EnemyKind.SPIDER);
        List<EnemyKind> deepKinds = List.of(
                EnemyKind.SKELETON, EnemyKind.NECROMANCER);
        Dungeon dungeon = new DungeonBuilder()
                .floor().biome(crypt).enemies(cryptKinds)
                        .map(MapData.floor(1)).done()
                .floor().biome(cavern).enemies(cavernKinds)
                        .map(MapData.floor(2)).done()
                .floor().biome(crypt).enemies(deepKinds)
                        .map(MapData.floor(3)).done()
                .build();
```

app/Main.java (4/6)

```
        if (options.length > 1 && options[1].equals("map")) {
            explore(menu, dungeon, floorNumber);
            return;
        }
        DamageFormula formula = chooseFormula(menu, options,
                script != null);
        Floor floor = dungeon.floors().get(floorNumber - 1);
        GameSession session = new GameSession(heroes(), floor.biome(),
                floor.enemies(), formula, new DamagePipeline(random, 0.0));
        GameMap entrance = new MapParser(new TileTypes())
                .parse(floor.map());
        session.setLocation(floorNumber,
                entrance.partyX(), entrance.partyY());
        // cap. 12: la partida de consola empieza en combate
        GameFlow flow = new GameFlow(session.heroes(), config.floors());
        flow.start(floorNumber);
        flow.enterCombat();
        EventBus events = session.events();
        events.subscribe(new LogView(output));
        events.subscribe(new HudView(output));
        events.subscribe(flow);
        // el tracker va el último
        events.subscribe(new AchievementTracker(events));
        menu.run(session);
    }

    private void explore(ConsoleMenu menu, Dungeon dungeon,
                         int firstFloor) {
        TileTypes types = new TileTypes();
        MapParser parser = new MapParser(types);
        int number = firstFloor;
        while (true) {
            Floor floor = dungeon.floors().get(number - 1);
            boolean descended =
                    menu.explore(parser.parse(floor.map()), types);
            if (!descended || number == dungeon.floors().size()) {
                return;
            }
            number++;
        }
    }
```

app/Main.java (5/6)

```
    // El guion es el argumento que acaba en .txt
    private static String scriptOf(String[] args) {
        return Arrays.stream(args)
                .filter(arg -> arg.endsWith(".txt"))
                .findFirst().orElse(null);
    }

    private static String[] optionsOf(String[] args) {
        return Arrays.stream(args)
                .filter(arg -> !arg.endsWith(".txt")
                        && !arg.startsWith("--"))
                .toArray(String[]::new);
    }

    private DamageFormula chooseFormula(ConsoleMenu menu, String[] options,
                                        boolean scripted) {
        boolean useDefault = scripted
                || (options.length > 1 && options[1].equals("default"));
        if (useDefault) {
            return config.defaultFormula();
        }
        return menu.askFormula() == 1
                ? new FlatFormula() : new ScaledFormula();
    }

    private int chooseFloor(String[] options) {
        int floor = options.length > 0 ? Integer.parseInt(options[0]) : 1;
        if (floor < 1 || floor > config.floors()) {
            throw new IllegalArgumentException(
                    "La planta debe estar entre 1 y " + config.floors());
        }
        return floor;
    }
```

app/Main.java (6/6)

```
    // --plain o --ansi mandan; si no, plano con guion y color a mano
    private static Output chooseOutput(String[] args, boolean scripted) {
        List<String> all = Arrays.asList(args);
        boolean plain = all.contains("--plain");
        boolean ansi = all.contains("--ansi");
        if (plain && ansi) {
            throw new IllegalArgumentException(
                    "Elige --plain o --ansi, no las dos");
        }
        if (ansi || (!plain && !scripted)) {
            return new AnsiOutput(System.out);
        }
        return new PlainOutput(System.out);
    }

    private List<Hero> heroes() {
        return List.of(
                new HeroBuilder().name("Aria").stats(new Stats(30, 8, 3))
                        .skill(POWER_STRIKE).skill(FLARE)
                        .skill(HEAL).build(),
                new HeroBuilder().name("Borin").stats(new Stats(40, 6, 5))
                        .skill(POWER_STRIKE).skill(FLARE)
                        .skill(TAUNT).skill(STUN).build(),
                new HeroBuilder().name("Cael").stats(new Stats(24, 9, 2))
                        .skill(POWER_STRIKE).skill(FLARE)
                        .skill(VENOM).build());
    }
}
```

app/Check13b.java (1/2)

```
package emberdeep.app;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.Attack;
import emberdeep.engine.combat.command.Defend;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.flow.GameFlow;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.ui.output.PlainOutput;
import emberdeep.ui.view.LogView;
import java.util.List;
import java.util.Random;
```

app/Check13b.java (2/2)

```java
public class Check13b {
    public static void main(String[] args) {
        Random random = new Random(42);
        Hero borinData = new HeroBuilder().name("Borin")
                .stats(new Stats(40, 6, 5)).build();
        GameSession session = new GameSession(List.of(borinData),
                new CryptBiome(random), List.of(EnemyKind.ASH_RAT),
                new FlatFormula(), new DamagePipeline(random, 0.0));
        GameFlow flow = new GameFlow(session.heroes(), 1);
        session.events().subscribe(
                new LogView(new PlainOutput(System.out)));
        session.events().subscribe(flow);

        Combat combat = session.combat();
        Fighter borin = session.heroes().get(0);
        Fighter rat = session.enemies().get(0);

        flow.start(1);
        flow.enterCombat();
        System.out.println("Antes: ataque " + borin.stats().attack()
                + ", armadura " + borin.stats().armor()
                + ", defensa " + borin.stats().defense());
        combat.perform(new Defend(combat, borin));
        System.out.println("Tras defender: escudo " + borin.shield()
                + ", defensa " + borin.stats().defense());
        System.out.println("Fase: " + flow.phase());

        // el golpe mata a la Rata: CombatEnded -> leaveCombat(true)
        combat.perform(new Attack(combat, borin, rat));
        System.out.println("Tras el combate: escudo " + borin.shield()
                + ", defensa " + borin.stats().defense()
                + ", ataque " + borin.stats().attack()
                + ", armadura " + borin.stats().armor());
        System.out.println("Fase: " + flow.phase());

        flow.descend();
        System.out.println("Fase: " + flow.phase());
        try {
            flow.enterCombat();
        } catch (IllegalStateException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Por qué funciona (parte 2). Borin tiene ataque $6 + 2 = 8$, armadura 1 y defensa 5. El `Defend` le da 5 de escudo y un `TempBuff` de +2 de defensa (7) que dura un turno. Como el combate termina en la fase de los héroes, `enemyPhase` nunca llega a avanzar los buffs y, sin `endCombat`, el escudo y la defensa de 7 viajarían al siguiente combate. La Rata (8 de vida y defensa 0) cae de un golpe de $8 - 0 = 8$; `perform` llama a `checkEnd`, que publica `CombatEnded`. Los oyentes lo reciben en orden de suscripción: el `LogView` escribe «¡Victoria!» y `GameFlow`, que está en `InCombat`, llama a `leaveCombat(true)`: `endCombat()` deja el escudo a 0 y `stats = gear` (la defensa vuelve a 5, el ataque sigue en 8 y la armadura

en 1, porque la espada y la armadura estaban en `gear`, no en los `TempBuff`), y la fase pasa a `Exploring(1)`. Con una sola planta como última, `descend()` lleva a `GameOver(true)`; desde ahí, `enterCombat()` lanza la excepción y su mensaje sale de `notNow`. En `Main`, la partida de consola entra en combate nada más empezar, así que `GameFlow` solo cambia de fase una vez. Las trazas de «atacar siempre al primero» no cambian: lo que se limpia al ganar no se imprime.

Parte 3: el Snapshot.

engine/save/FighterState.java

```
package emberdeep.engine.save;

import emberdeep.engine.combat.status.Status;
import emberdeep.engine.stats.StatSource;

public record FighterState(int hp, int shield, StatSource stats,
                           Status status) {}
```

engine/session/GameSession.java (1/3)

```
package emberdeep.engine.session;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.event.EventBus;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.save.FighterState;
import emberdeep.engine.save.Snapshot;
import emberdeep.engine.stats.ArmorBonus;
import emberdeep.engine.stats.BaseStats;
import emberdeep.engine.stats.StatSource;
import emberdeep.engine.stats.WeaponBonus;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Stream;

public class GameSession {
    private static final int SWORD_ATTACK = 2;
    private static final int ARMOR_VALUE = 1;

    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final Combat combat;
    private final EventBus events = new EventBus();
    private Inventory inventory;
    private int floor = 1;
    private int x;
    private int y;
```

engine/session/GameSession.java (2/3)

```
    public GameSession(List<Hero> heroes, BiomeFactory biome,
                       List<EnemyKind> kinds, DamageFormula formula,
                       DamagePipeline pipeline) {
        this.heroes = heroes.stream()
                .map(hero -> Fighter.ofHero(hero, gearOf(hero)))
                .toList();
        // mutable: la arena añade los refuerzos al final de la lista
        this.enemies = new ArrayList<>(kinds.stream()
                .map(biome::enemy)
                .map(Fighter::ofEnemy)
                .toList());
        this.combat = new Combat(this.heroes, this.enemies,
                formula, pipeline, events, biome);
        this.inventory = new Inventory(List.of())
                .with(biome.item(ItemKind.HEALING))
                .with(biome.item(ItemKind.HEALING))
                .with(biome.item(ItemKind.BOMB));
    }

    private static StatSource gearOf(Hero hero) {
        return new ArmorBonus(
                new WeaponBonus(new BaseStats(hero.stats()), SWORD_ATTACK),
                ARMOR_VALUE);
    }

    public List<Fighter> heroes() { return heroes; }
    public List<Fighter> enemies() { return enemies; }
    public Combat combat() { return combat; }
    public EventBus events() { return events; }
    public int round() { return combat.round(); }
    public Inventory inventory() { return inventory; }
    public int floor() { return floor; }
    public int x() { return x; }
    public int y() { return y; }

    public void replaceInventory(Inventory next) {
        this.inventory = next;
    }
```

engine/session/GameSession.java (3/3)

```
    public void setLocation(int floor, int x, int y) {
        this.floor = floor;
        this.x = x;
        this.y = y;
    }

    private List<Fighter> allFighters() {
        return Stream.concat(heroes.stream(), enemies.stream()).toList();
    }

    public Snapshot createSnapshot() {
        List<FighterState> states = allFighters().stream()
                .map(f -> new FighterState(f.hp(), f.shield(), f.stats(),
                        f.status()))
                .toList();
        return new Snapshot(floor, x, y, combat.round(), states,
                inventory);
    }

    public void restore(Snapshot snapshot) {
        int saved = snapshot.fighters().size();
        if (saved < heroes.size() || saved > allFighters().size()) {
            throw new IllegalArgumentException(
                    "La instantánea es de otra partida");
        }
        // cap. 11: los refuerzos invocados después del punto de control
        // se descartan (siempre están al final de la lista de enemigos)
        enemies.subList(saved - heroes.size(), enemies.size()).clear();
        List<Fighter> all = allFighters();
        for (int i = 0; i < all.size(); i++) {
            FighterState state = snapshot.fighters().get(i);
            all.get(i).restore(state.hp(), state.shield(), state.stats());
            all.get(i).setStatus(state.status());
        }
        combat.restoreRound(snapshot.round());
        inventory = snapshot.inventory();
        floor = snapshot.floor();
        x = snapshot.x();
        y = snapshot.y();
    }
}
```

src/test/java/emberdeep/engine/save/SnapshotTest.java (1/2)

```
package emberdeep.engine.save;

import static org.junit.jupiter.api.Assertions.assertEquals;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.combat.status.Status.Poisoned;
import emberdeep.engine.combat.status.Status.Stunned;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.biome.CryptBiome;
import java.util.List;
import java.util.Random;
import org.junit.jupiter.api.Test;

class SnapshotTest {

    private static GameSession newSession() {
        Random random = new Random(42);
        List<Hero> heroes = List.of(
                new HeroBuilder().name("Aria")
                        .stats(new Stats(30, 8, 3)).build());
        return new GameSession(heroes, new CryptBiome(random),
                List.of(EnemyKind.SKELETON),
                new FlatFormula(), new DamagePipeline(random, 0.0));
    }
```

src/test/java/emberdeep/engine/save/SnapshotTest.java (2/2)

```
    @Test
    void restoreBringsBackSavedState() {
        GameSession session = newSession();
        Fighter aria = session.heroes().get(0);
        aria.addShield(5);
        Snapshot saved = session.createSnapshot();

        aria.takeDamage(10);
        aria.absorbShield(5);
        session.replaceInventory(new Inventory(List.of()));
        session.restore(saved);

        assertEquals(30, aria.hp());
        assertEquals(5, aria.shield());
        assertEquals(3, session.inventory().items().size());
    }

    // cap. 12: el estado también forma parte del memento
    @Test
    void restoreBringsBackStatus() {
        GameSession session = newSession();
        Fighter aria = session.heroes().get(0);
        aria.setStatus(new Poisoned(3, 2));
        Snapshot saved = session.createSnapshot();

        aria.setStatus(new Stunned(1));
        session.restore(saved);

        assertEquals(new Poisoned(3, 2), aria.status());
    }
}
```

Por qué funciona (parte 3). Al añadir el cuarto campo a `FighterState`, el compilador rechaza `new FighterState(f.hp(), f.shield(), f.stats())` en `createSnapshot`: ese es el aviso del coste de Memento. Pero hay un aviso que el compilador *no* da: si se te olvida la línea de `restore` que repone el estado, todo compila. Lo avisa la prueba: `restoreBringsBackStatus` guarda una instantánea con `Poisoned(3, 2)`, deja al héroe `Stunned(1)` y comprueba que vuelve. Los estados son records inmutables, así que, como la cadena `StatSource`, se guardan por referencia y no hace falta copiarlos. La prueba del capítulo 8 sigue compilando tal cual, porque nunca construye un `FighterState`; lo que protege el campo nuevo es la prueba añadida.

Errores típicos:

- Poner un `default` en el `switch` sobre `Status`, `Skill` o `GameEvent`: el compilador deja de listar los sitios que faltan.
- Consultar el estado de los héroes en el turno de cada uno, y no al empezar la ronda: `u` y `r` duplican el veneno o devuelven al juego a un héroe aturdido.
- Hacer pasar el veneno por la tubería de defensa: no es un golpe, y el escudo lo absorbería.
- Olvidar `fighter.setStatus(current.next())`: el veneno duraría para siempre.

- Avisar de la caída por veneno sin comprobar `isAlive()`: el combatiente muerto seguiría actuando.
- Poner `stats = gear` en `restore` o en lugar de en `endCombat`: los buffs del punto de control desaparecerían al volver.
- Que `GameFlow` limpie solo al ganar: si pierdes, la partida acaba, pero un `GameOver` no debería dejar escudos activos si luego se reutiliza el `Fighter`.
- Olvidar la prueba de `status` en `SnapshotTest`: el campo nuevo se guardaría sin que nada compruebe que se repone.

Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo 13: `engine/combat/status/` con `Status`; `engine/flow/` con `GamePhase` y `GameFlow`; `Skill` con `Venom` y `Stun`; `GameEvent` con diez records; `FighterState` con `status`; `SnapshotTest` con dos pruebas; y `Main` con las dos habilidades nuevas de Cael y Borin y el `GameFlow`. Borra `app/Check13a.java` y `app/Check13b.java`.

CAPÍTULO 13

Composite: oleadas e inventario anidados

Objetivos del capítulo

- Reconocer el olor que pide Composite: una estructura con elementos sueltos y grupos de elementos que se recorre con bucles anidados de profundidad fija.
- Modelar la hoja y el compuesto como tipos de una interfaz `sealed` y recorrerlos con un `switch` recursivo sin `default`.
- Usar `Optional` para devolver un resultado que puede no existir (`find`).
- Mantener la estructura inmutable: `with` y `without` devuelven una `Bag` nueva y el `Snapshot` la guarda sin copiarla.
- Saber cuándo un `switch` sobre una interfaz sellada basta y cuándo se echa de menos el Visitor clásico.
- Dar el hito H14: inventario con bolsas dentro de bolsas, oleadas compuestas y el `Snapshot` al día.

Hasta ahora, el inventario de Emberdeep era una lista plana de objetos. Tu equipo pide mochilas con bolsas dentro: una bolsa de cinturón para la bomba, otra para los frascos. Y el diseñador de niveles quiere describir las plantas con oleadas: «dos esqueletos y, después, una subola con una rata y un zombi». Los dos problemas tienen la misma forma: una cosa que puede ser *un elemento* o *un grupo de elementos*, y el grupo puede contener otros grupos. Composite es el patrón que permite preguntar lo mismo a un elemento y a un grupo (¿cuántos objetos hay?, ¿cuánto pesa?) sin que quien pregunta tenga que saber cuál de los dos tiene delante. Con los tipos sellados de Java 21 lo escribes con un `switch` que el compilador vigila.

13.1 El olor que pide Composite: bolsas dentro de bolsas con bucles anidados

Empieza por lo que escribirías sin pensar en patrones. Una mochila es una lista; a veces lo que hay dentro es un objeto y a veces es otra lista. Contar los objetos sería algo así (el código es hipotético; no existe en el proyecto):

Hipotético: contar objetos con bucles anidados

```
static int countItems(List<Object> backpack) {
    int total = 0;
    for (Object a : backpack) {
        if (a instanceof Item) {
            total++;
        } else if (a instanceof List<?> bag) {
            for (Object b : bag) {
```

```
                if (b instanceof Item) {
                    total++;
                }
                // ¿y si b es otra bolsa? Haría falta un tercer bucle
            }
        }
    }
    return total;
}
```

Funciona mientras las bolsas no estén dentro de otras bolsas, y tiene cuatro problemas:

1. **La profundidad está escrita en el código.** Cada nivel de anidamiento es un bucle más. Una bolsa dentro de una bolsa dentro de la mochila se cuenta mal sin avisar.
2. **Se pierden los tipos.** La lista es de `Object`: el compilador no sabe qué puede haber dentro, y un `instanceof` olvidado compila.
3. **Cada operación repite los bucles.** Contar, sumar el poder, buscar un objeto y quitarlo necesitan el mismo recorrido, copiado cuatro veces.
4. **Quien pregunta distingue entre elemento y grupo.** El `if` sobre el tipo está en todas partes, en vez de estar una sola vez.

Este es el olor: **una estructura donde un elemento y un grupo de elementos se tratan de forma distinta, aunque se les pregunta lo mismo, y los grupos se anidan**. Composite lo cura haciendo que elemento y grupo sean dos variantes de *un mismo tipo*. En el catálogo, su intención es componer objetos en estructuras de árbol para representar jerarquías parte-todo y permitir que los clientes traten de manera uniforme los objetos individuales y las composiciones [9, 24]. Head First lo cuenta con menús que contienen submenús [7].

13.2 Piezas del patrón Composite

Los roles son tres: el componente, la hoja y el compuesto, más la operación recursiva que los recorre. En Emberdeep, con el inventario como ejemplo y las oleadas como segundo uso:

Rol	Clase en el ejemplo	Propósito	Paquete
Componente (interfaz sellada)	`InventoryNode`	Lo que puede haber en una bolsa	`engine.item`
Hoja (record)	`ItemSlot`	Un objeto, sin nada dentro	`engine.item`
Compuesto (record)	`Bag`	Una bolsa con nodos dentro	`engine.item`
Operación recursiva (método)	`countItems`, `totalPower`, `find` con `switch`	Se pregunta igual a la hoja y al compuesto	`engine.item`
Segundo composite (interfaz sellada y records)	`WaveNode`, `EnemyGroup`, `Wave` (ejercicio)	Oleadas con subolas; el ejercicio completa `enemies()`	`engine.world.wave`
Cliente (clase)	`GameSession`	Guarda la `Bag` raíz y la sustituye	`engine.session`

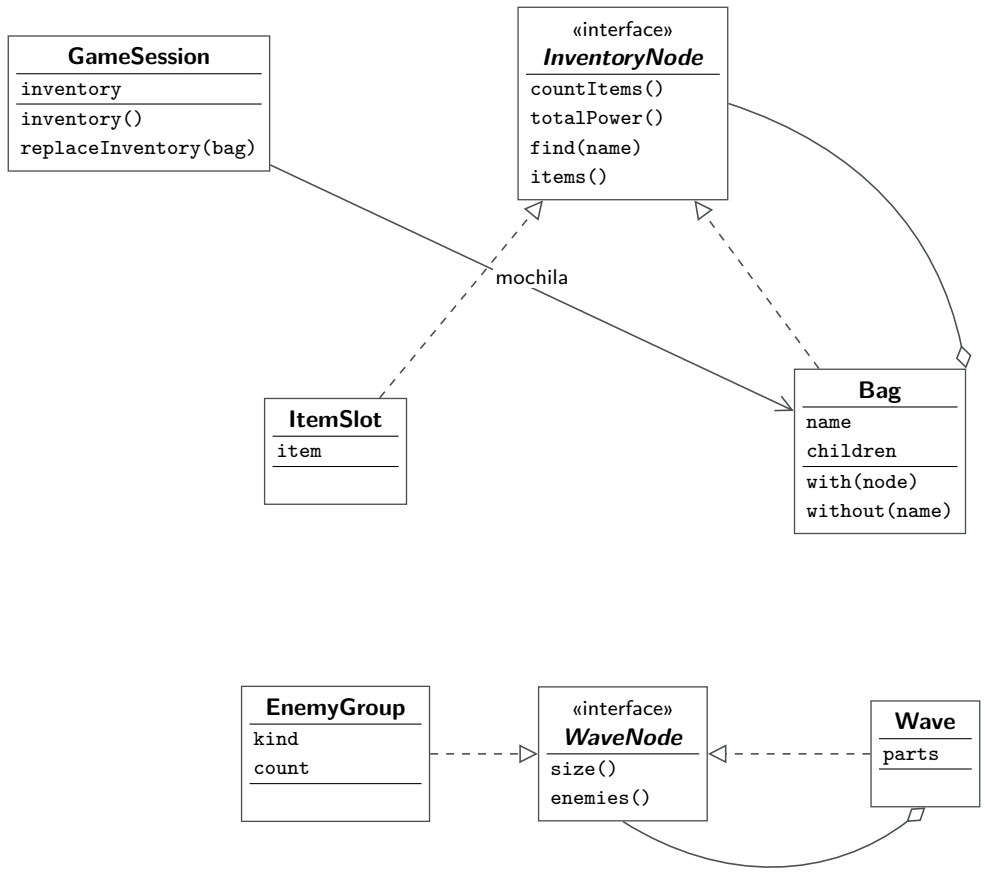


Diagrama UML del patrón Composite: `Bag` y `Wave` contienen una lista de su propio componente (rombo); `ItemSlot` y `EnemyGroup` son las hojas.

Mira dos cosas del diagrama. La flecha con rombo vuelve al componente, no a la hoja: una `Bag` guarda *nodos*, y por eso puede guardar otras bolsas. Y `GameSession` conoce una `Bag`, no los detalles de lo que contiene. Una nota sobre el papel de `UseItem` (capítulo 7): no forma parte del patrón. Es un comando que le pide a `GameSession` la bolsa raíz, busca con `find`, quita con `without` y guarda la bolsa anterior para su `undo`.

13.3 El ejemplo de Composite, pieza a pieza: `InventoryNode`

El componente y sus dos variantes. Es la `sealed interface` del capítulo 12 con dos records dentro: `ItemSlot`, que envuelve un `Item`, y `Bag`, que lleva un nombre y una lista de nodos [21]. El constructor compacto de `Bag` copia la lista con `List.copyOf`, como `Inventory` en el capítulo 3: nadie de fuera puede cambiarla después.

`engine/item/InventoryNode.java (versión del texto, 1/2)`

```
1  package emberdeep.engine.item;
2
3  import java.util.List;
4
5  public sealed interface InventoryNode {
6      // la hoja: un objeto
7      record ItemSlot(Item item) implements InventoryNode {}
8
9      // el compuesto: una bolsa con nodos dentro (hojas u otras bolsas)
```

```
    record Bag(String name, List<InventoryNode> children)
            implements InventoryNode {
        public Bag {
            children = List.copyOf(children);
        }
    }

    // la operación recursiva: se pregunta igual a una hoja y a una bolsa
    default int countItems() {
        return switch (this) {
            case ItemSlot slot -> 1;
            case Bag bag -> bag.children().stream()
                    .mapToInt(InventoryNode::countItems)
                    .sum();
        };
    }
}
```

Tres cosas que mirar. Primera: el **caso base** es la hoja: un `ItemSlot` cuenta 1 y no llama a nadie. Segunda: el caso recursivo es el compuesto, que suma lo que cuenta cada hijo *llamando al mismo método*: no sabe, ni le importa, si el hijo es una hoja o otra bolsa. Una bolsa vacía suma 0 y no hace falta un caso especial. Tercera: como en `Status`, el `switch` no lleva `default`. `InventoryNode` es sellada, solo tiene esas dos variantes y el compilador exige nombrarlas [4].

Sobre la mochila inicial de Emberdeep (dos pociones sueltas y una bolsa de cinturón con la bomba), la llamada se despliega así:

Traza de countItems sobre la mochila inicial

```
countItems(mochila)
|-- countItems(Poción de cripta)            = 1
|-- countItems(Poción de cripta)            = 1
`-- countItems(bolsa de cinturón)
    `-- countItems(Bomba de ceniza)         = 1
        la bolsa suma 1
mochila suma 1 + 1 + 1 = 3
```

Más operaciones recursivas. Todas tienen la misma forma: la hoja responde sola, el compuesto combina las respuestas de sus hijos. La suma del poder usa además un *patrón de record*, que descompone el record en la propia etiqueta del `case` (Java 21) [3]:

engine/item/InventoryNode.java: totalPower

```
default int totalPower() {
    return switch (this) {
        case ItemSlot(Item item) -> item.power();
        case Bag bag -> bag.children().stream()
                .mapToInt(InventoryNode::totalPower)
                .sum();
    };
}
```

`ItemSlot(Item item)` hace dos cosas a la vez: comprueba que el nodo es un `ItemSlot` y deja su componente en la variable `item`. Es lo mismo que `case ItemSlot slot` y `slot.item()`, escrito más corto.

Optional: un resultado que puede no existir

Buscar un objeto por su nombre puede no encontrar nada. La forma clásica de decirlo es devolver `null`, y el problema es que el tipo de retorno no avisa: quien llama tiene que acordarse de comprobar, y si no, falla más tarde con un `NullPointerException`. `Optional<T>` es una caja que contiene un valor o está vacía, y el propio tipo lo cuenta. Es del JDK (`java.util.Optional`) y no hay nada que instalar. Lo que necesitas en este capítulo:

- Crear: `Optional.of(x)` (con un valor que no sea `null`) y `Optional.empty()` (la caja vacía).
- Preguntar: `isPresent()` (tiene valor) e `isEmpty()` (no lo tiene).
- Sacar el valor con una salida para el caso vacío: `orElse(otro)` (el valor de reserva), `orElseThrow()` (lanza `NoSuchElementException` si está vacía) o `orElseThrow(() -> new X(...))` (lanza la excepción que elijas). Evita `get()`: hace lo mismo que `orElseThrow()` pero con un nombre que invita a olvidar el caso vacío.
- Transformar sin sacar: `map(f)` aplica `f` al valor si lo hay y deja vacía la caja si no.
- Verla como una colección de cero o un elemento: `stream()`. Sirve para encadenar varias búsquedas y quedarse con la primera que acierte.

Úsalo como tipo de *retorno* de lo que puede no encontrar nada; no como campo ni como parámetro [5].

`Optional: tres formas de usar un resultado que puede faltar`

```
Optional<Item> bomb = backpack.find("Bomba de ceniza");

// 1. un valor de reserva si no hay nada
String name = bomb.map(Item::name).orElse("nada");

// 2. exigir que esté: si falta, es un fallo del programa
Item potion = backpack.find("Poción de cripta").orElseThrow();

// 3. solo preguntar
boolean hasMace = backpack.find("Maza").isPresent();
```

Buscar: `find` con `Optional`. La hoja responde sola (su objeto, si el nombre coincide); la bolsa pregunta a cada hijo y se queda con la primera respuesta que no esté vacía. Los `stream` son perezosos: en cuanto un hijo acierta, `findFirst` corta y no se recorre el resto.

`engine/item/InventoryNode.java: find e items`

```
default Optional<Item> find(String itemName) {
    return switch (this) {
        case ItemSlot slot -> slot.item().name().equals(itemName)
                ? Optional.of(slot.item())
                : Optional.empty();
        case Bag bag -> bag.children().stream()
                .map(child -> child.find(itemName))
                .flatMap(Optional::stream)
                .findFirst();
    };
}

```

```
// todos los objetos, aplanados, en el orden del recorrido
default List<Item> items() {
    return switch (this) {
        case ItemSlot slot -> List.of(slot.item());
        case Bag bag -> bag.children().stream()
                .flatMap(child -> child.items().stream())
                .toList();
    };
}
```

El `map` de la bolsa produce un `Stream` de cajas, una por hijo. Con `flatMap(Optional::stream)` cada caja pasa a ser cero o un elemento y se juntan todas: las cajas vacías desaparecen y `findFirst` se queda con el primer objeto que existe, en profundidad. El resultado es otra caja: vacía si ninguna rama acertó.

Cambiar sin mutar: `with` y `without`. Las operaciones que cambian el contenido van *solo* en `Bag`, no en la interfaz: una hoja no puede añadir nada a sí misma. Ambas devuelven una `Bag` nueva y la original queda como estaba, igual que `Inventory` en el capítulo 3:

engine/item/InventoryNode.java: with y without (dentro de Bag)

```
public Bag with(InventoryNode node) {
    List<InventoryNode> next = new ArrayList<>(children);
    next.add(node);
    return new Bag(name, next);
}

// quita el primer objeto con ese nombre, a cualquier profundidad
public Bag without(String itemName) {
    List<InventoryNode> next = new ArrayList<>();
    boolean removed = false;
    for (InventoryNode child : children) {
        if (removed) {
            next.add(child);
            continue;
        }
        switch (child) {
            case ItemSlot slot -> {
                if (slot.item().name().equals(itemName)) {
                    removed = true;        // no se copia: queda fuera
                } else {
                    next.add(slot);
                }
            }
            case Bag bag -> {
                if (bag.find(itemName).isPresent()) {
                    next.add(bag.without(itemName));   // recursión
                    removed = true;
                } else {
                    next.add(bag);
                }
            }
        }
    }
    return new Bag(name, next);
}
```

Mira lo que se copia y lo que no. `without` construye listas nuevas solo a lo largo del camino que lleva hasta el objeto; las demás ramas y hojas se *reutilizan* tal cual, porque son inmutables y compartirlas no puede dar sorpresas. Quitar la bomba de la mochila inicial deja esto:

	Antes (`start`)	**Después (`start.without("Bomba de ceniza")`)**
La mochila	`Bag` con 3 nodos	Otra `Bag`, con 3 nodos
Las dos pociones	Dos `ItemSlot`	Los mismos dos objetos (misma referencia)
La bolsa de cinturón	Con la bomba dentro	Otra `Bag`, vacía
`countItems()`	3	2

Esa es la razón de que el Memento pueda guardar la bolsa *por referencia*: la `Bag` guardada no cambia nunca. Si `Bag` tuviera una lista mutable y un `add`, volvería el error del capítulo 8: el `Snapshot` y la sesión compartirían la misma lista y restaurar no restauraría nada.

13.4 Composite para oleadas: `WaveNode`

El mismo patrón sirve para describir quién sale a combatir. Un `EnemyGroup` es la hoja («2 esqueletos») y una `Wave` es el compuesto: una lista de grupos y de otras oleadas. La forma es idéntica a la del inventario; lo único que cambia es qué guarda la hoja:

engine/world/wave/WaveNode.java (versión del texto)

```
package emberdeep.engine.world.wave;

import emberdeep.engine.character.EnemyKind;
import java.util.List;

public sealed interface WaveNode {
    record EnemyGroup(EnemyKind kind, int count) implements WaveNode {
        public EnemyGroup {
            if (count < 1) {
                throw new IllegalArgumentException(
                        "Un grupo necesita al menos un enemigo");
            }
        }
    }

    record Wave(List<WaveNode> parts) implements WaveNode {
        public Wave {
            parts = List.copyOf(parts);
        }
    }

    // cuántos enemigos hay en total, a cualquier profundidad
    default int size() {
        return switch (this) {
            case EnemyGroup group -> group.count();
            case Wave wave -> wave.parts().stream()
                    .mapToInt(WaveNode::size)
```

```
                        .sum();
            };
        }
}
```

Dos esqueletos seguidos de una subola con una rata y un zombi se escriben con tres `new` anidados, y `size()` da 4 sin saber cuántos niveles hay. Lo que la planta necesita no es un número, sino la lista de enemigos en orden: aplanar la oleada con otra operación recursiva, `enemies()`, es parte del hito.

13.5 Composite con sealed y switch frente al Visitor clásico

Una operación recursiva puede vivir en tres sitios, y la decisión importa más de lo que parece. Sea cual sea, el problema de fondo es el mismo: tienes *tipos* (hoja, bolsa) y *operaciones* (contar, sumar, buscar), y cada vez que añades una de las dos cosas hay que tocar la otra.

Dónde vive la operación	**Operación nueva**	**Tipo nuevo**	**La lógica está...**
Método en cada nodo (el Composite clásico)	Hay que tocar todos los tipos	Una clase nueva y ya	Repartida por tipos
`switch` sobre la interfaz sellada	Una función nueva	El compilador lista cada `switch`	Junta por operación
Visitor clásico	Una clase `Visitor` nueva	Un método más en la interfaz y en cada `Visitor`	Junta por operación

En este capítulo usamos el `switch`: en un inventario las operaciones nuevas aparecen más a menudo que los tipos de nodo, y el `switch` las pone en una sola función. El precio es el de State: un tipo nuevo de nodo toca cada `switch`.

El Visitor clásico, solo para reconocerlo

Visitor es un patrón del catálogo [9] que Emberdeep no desarrolla. Antes de que Java tuviera tipos sellados, era la forma de poner operaciones fuera de los nodos: cada nodo tiene un método `accept(visitor)` que llama al método del visitor que corresponde a su tipo. Contar objetos se escribiría así (hipotético):

Hipotético: contar objetos con un Visitor clásico

```
interface NodeVisitor<R> {
    R visitSlot(ItemSlot slot);
    R visitBag(Bag bag);
}

// en cada nodo: accept(v) llama al visitXxx que le corresponde
// <R> R accept(NodeVisitor<R> v) { return v.visitXxx(this); }

class CountVisitor implements NodeVisitor<Integer> {
    public Integer visitSlot(ItemSlot slot) {
```

```
        return 1;
    }

    public Integer visitBag(Bag bag) {
        int total = 0;
        for (InventoryNode child : bag.children()) {
            total += child.accept(this);
        }
        return total;
    }
}
```

Para las operaciones sobre datos con un número cerrado de tipos, los tipos sellados y el `switch` cubren lo que hacía el Visitor con mucho menos código: lo que conserva es lo importante, que añadir un tipo obliga a revisar todas las operaciones [23]. Además, un `switch` con patrones es más flexible, porque puede mirar dentro de un record y combinar casos [10]. El Visitor sigue teniendo sentido si los nodos son una biblioteca que no controlas y quieres operaciones nuevas sin tocar sus clases, o si el árbol no es sellado. El ejercicio 13.4 te pide decidir entre las tres formas.

13.6 Composite: cuándo sí y cuándo no

Compensa cuando hay una jerarquía parte-todo real (carpetas y archivos, menús y submenús, oleadas y subolas, bolsas y objetos), cuando los clientes quieren tratar igual la hoja y el grupo y cuando la profundidad no se conoce de antemano. Se nota en que escribes dos veces el mismo recorrido con un `if` sobre el tipo.

No compensa si la estructura es plana y lo seguirá siendo: una lista de objetos sin bolsas es un `Inventory` del capítulo 3, y envolver cada objeto en un `ItemSlot` sería ceremonia. Tampoco, si la hoja y el grupo se usan de maneras tan distintas que casi ninguna operación tiene sentido para los dos.

Costes reales:

- **Todo es un nodo.** El tipo no te impide meter una bolsa donde esperabas un objeto. Aquí es una decisión: una `Bag` puede contener bolsas. Un ciclo (una bolsa dentro de sí misma) es imposible porque los records son inmutables: para construir una bolsa hay que tener ya sus hijos.
- **Dónde van `add` y `remove`.** El catálogo discute si las operaciones que gestionan hijos van en el componente (transparencia: todo se trata igual) o solo en el compuesto (seguridad: una hoja no recibe un `add` que no entiende) [9]. Aquí `with` y `without` van solo en `Bag`: la interfaz tiene las consultas, que son las que tiene sentido preguntar a cualquier nodo.
- **Recorridos completos.** `find` y `countItems` visitan el árbol entero en el peor caso; para un inventario es irrelevante. Una recursión muy profunda agotaría la pila de llamadas, algo que no ocurre con bolsas de mochila.
- **Un tipo nuevo toca cada `switch`.** Es el mismo coste de State y de `Skill`: el compilador lo señala, pero el trabajo existe.

Composite frente a Decorator. Los dos son recursivos: algo del tipo T contiene algo del tipo T. Un Decorator (capítulo 6) envuelve *uno* y le añade comportamiento; un Composite contiene *varios* y los trata como uno solo.

13.7 En el proyecto: H14 con Composite

Este capítulo avanza el hito H14. Cambios del hito:

- **Nuevo paquete:** `world/wave/` (con `WaveNode`).
- **Cambia el inventario:** `Inventory` desaparece; `engine/item/` gana `InventoryNode` (con `ItemSlot` y `Bag`). `GameSession` guarda ahora una `Bag` raíz, «mochila», y la sustituye por la que devuelven `with` y `without`.
- **Cambian** `GameSession`, `UseItem` (busca con `find`, quita con `without` y su `undo` repone la bolsa guardada), `ConsoleMenu` (el tipo de un método) y `Main` (la planta 3 se describe con una oleada).
- **Cambia el Memento:** `Snapshot` guarda una `Bag`, no un `Inventory`, y `SnapshotTest` se actualiza.

Qué cambia en la partida. La mochila inicial sigue teniendo tres objetos, pero ahora la bomba está en una «bolsa de cinturón» dentro de la mochila; el menú aplana el árbol con `items()` y muestra los objetos en el mismo orden de siempre. La planta 3 se describe con una oleada (un esqueleto y una subola con el nigromante) que se aplana en la misma lista de antes. **Las trazas de las partidas no cambian**: el cambio es de estructura, no de reglas.

Límites que esta versión no resuelve (se declaran, no se ocultan):

1. **El jugador no ve las bolsas.** El menú lista los objetos aplanados; mover objetos entre bolsas no es una acción del juego.
2. **`without` quita el primer objeto con ese nombre** (en profundidad, en el orden del recorrido). Con dos pociones iguales, da igual cuál; si importara, haría falta un identificador.
3. **Las bolsas no tienen capacidad.** Cabe todo lo que se añada.
4. **Las oleadas solo describen la planta 3.** Las plantas 1 y 2 siguen con una lista de `EnemyKind`, que es lo que `Floor` guarda.
5. **El guardado a fichero llega en el capítulo 14**, que aplanará la bolsa.

Los tres pasos. Cada uno termina con un resultado que puedes comprobar:

- **Paso 1, inventario inmutable.** `InventoryNode`, `GameSession`, `UseItem`, el tipo de `ConsoleMenu.describeItems` y el borrado de `Inventory`. Para que el proyecto compile, el campo del `Snapshot` cambia ya de tipo (una palabra); su explicación y su prueba son el paso 3. Resultado: el proyecto compila con `./gradlew -q classes` y un programa temporal muestra una bomba encontrada en una bolsa anidada, una poción usada y deshecha, y la bolsa anterior intacta.
- **Paso 2, oleadas.** `WaveNode` con `enemies()` y la planta 3 de `Main`. Resultado: una oleada con una subola se aplana en la lista correcta y una `GameSession` nace con esos enemigos.
- **Paso 3, el Snapshot.** `SnapshotTest`. Resultado: `./gradlew test` primero falla a compilar (la prueba aún nombra `Inventory`) y, al arreglarla, pasa con tres pruebas.

Hito 14: inventario anidado y oleadas compuestas

Tras este hito, la mochila es un árbol inmutable de bolsas y objetos, las plantas se pueden describir con oleadas y el punto de control guarda el inventario sin copiarlo. La estructura:

Estructura de carpetas tras el capítulo 13 (1/2)

```
emberdeep/
|-- settings.gradle
|-- build.gradle                    (con JUnit 6.1.3)
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
|-- scripts/
|   `-- demo.txt
`-- src/
    |-- main/java/emberdeep/
    |   |-- app/
    |   |   |-- Main.java              (composition root)
    |   |   `-- GameConfig.java
    |   |-- engine/
    |   |   |-- character/             Stats, Hero, HeroBuilder, Enemy,
    |   |   |                          EnemyKind, Fighter, Skill,
    |   |   |                          EnemyTemplate, TemplateRegistry
    |   |   |-- stats/                 StatSource, StatKind, BaseStats,
    |   |   |                          StatDecorator, WeaponBonus,
    |   |   |                          ArmorBonus, TempBuff
    |   |   |-- combat/
    |   |   |   |-- Combat.java
    |   |   |   |-- ai/                EnemyAi, FirstAliveAi, LowestHpAi,
    |   |   |   |                      RandomTargetAi, SkillfulAi
    |   |   |   |-- arena/             TurnMediator, CombatArena
    |   |   |   |-- command/           CombatCommand, Attack, Defend,
    |   |   |   |                      UseItem, UseSkill, CommandHistory
    |   |   |   |-- damage/            DamageFormula, FlatFormula,
    |   |   |   |                      ScaledFormula, Damage, DamageType
    |   |   |   |-- defense/           DamageHandler, DamagePipeline,
    |   |   |   |                      DodgeHandler, ShieldHandler,
    |   |   |   |                      ArmorHandler, ResistanceHandler
    |   |   |   `-- status/            Status
```

Estructura de carpetas tras el capítulo 13 (2/2)

```
|   |   |-- item/                  Item, ItemKind, InventoryNode
|   |   |-- event/                 GameEvent, GameListener, EventBus
|   |   |-- flow/                  GamePhase, GameFlow
|   |   |-- achievement/           AchievementTracker
|   |   |-- save/                  Snapshot, FighterState,
|   |   |                          SnapshotStore
|   |   |-- session/               GameSession
|   |   `-- world/
|   |       |-- Dungeon.java, Floor.java,
|   |       |   DungeonBuilder.java
|   |       |-- biome/             BiomeFactory, CryptBiome,
|   |       |                      CavernBiome
|   |       |-- map/               TileType, TileTypes, GameMap,
|   |       |                      MapParser, MapData
|   |       `-- wave/              WaveNode
|   `-- ui/
|       |-- ConsoleMenu.java
|       |-- input/                 InputSource, ConsoleInput,
|       |                          ScriptInput
|       |-- view/                  View, MapView, HudView, LogView
|       `-- output/                Output, Style, AnsiOutput,
|                                  PlainOutput, BufferOutput
`-- test/java/emberdeep/
    `-- engine/save/               SnapshotTest
```

Lo practicas en el ejercicio 13.5. Antes, cuatro ejercicios más pequeños.

13.8 Ejercicios

Ejercicio 13.1: El tamaño de un árbol de archivos ★★★☆☆

No es de Emberdeep. Crea una carpeta vacía y, dentro, archivos sin paquete. Vas a modelar un árbol de carpetas y archivos y a recorrerlo con un `switch` sobre una interfaz sellada.

1. `Entry`: `sealed interface` con dos records anidados: `FileNode(String name, int size)` y `FolderNode(String name, List<Entry> children)`. El constructor compacto de `FolderNode` copia la lista con `List.copyOf`.
2. `int totalSize()`: método `default` con un `switch (this)` *sin* `default`. Un archivo devuelve su tamaño; una carpeta, la suma de lo que devuelven sus hijos.
3. `int countFiles()`: otro método `default` con la misma forma: un archivo cuenta 1 y una carpeta suma los de sus hijos.
4. `Main`: construye una carpeta `raiz` con el archivo `notas.txt` (120), una carpeta `fotos` con `a.png` (300) y `b.png` (450), y una carpeta `vacia` sin nada. Imprime una línea por `raiz`, `fotos` y `vacia`.

Compila con `javac *.java` y ejecuta con `java Main`. Está bien si la salida es exactamente:

Salida esperada

```
raiz: 870 bytes en 3 archivos
fotos: 750 bytes en 2 archivos
vacia: 0 bytes en 0 archivos
```

y si, al añadir a propósito un tercer record `record Link(String name) implements Entry {}`, el compilador se queja de `totalSize()` y de `countFiles()` (quita el record después).

Pista: *Dentro de la interfaz, los records se nombran sin prefijo. En `Main`, hay que escribir `new Entry.FileNode(...)` porque los archivos no tienen paquete. Una carpeta vacía no necesita un caso especial: la suma de nada es 0.*

Práctica ▪ solución en la pág. 608

Ejercicio 13.2: Un contador que cuenta de más ★★★☆☆

No es de Emberdeep. Este código cuenta los objetos de una mochila, pero devuelve un número distinto del esperado. Léelo sin ejecutarlo:

Thing.java

```java
import java.util.List;

public sealed interface Thing {
    record Slot(String name) implements Thing {}

    record Pouch(String name, List<Thing> things) implements Thing {}

    default int countItems() {
        return switch (this) {
            case Slot slot -> 1;
            case Pouch pouch -> 1 + pouch.things().stream()
                    .mapToInt(Thing::countItems)
                    .sum();
        };
    }
}
```

Main.java

```java
import java.util.List;

public class Main {
    public static void main(String[] args) {
        Thing empty = new Thing.Pouch("vacía", List.of());
        Thing inner = new Thing.Pouch("bolsa", List.of(empty));
        Thing backpack = new Thing.Pouch("mochila", List.of(
                new Thing.Slot("poción"), inner));
        System.out.println(backpack.countItems());
    }
}
```

La mochila tiene *un solo* objeto: la poción. Responde:

1. Dibuja el árbol de llamadas de `backpack.countItems()` y apunta lo que devuelve

cada una, de las hojas hacia arriba.

2. ¿Qué imprime el programa?
3. ¿Qué expresión es la culpable y por qué?
4. Escribe la versión corregida de `countItems` y di cuánto devuelve para una bolsa vacía y por qué ese valor es el que hace funcionar la recursión.

Está bien si tu traza llega al mismo número que la ejecución real, si señalas la expresión exacta y si tu versión corregida imprime 1.

Lectura de código ▪ solución en la pág. 610

Ejercicio 13.3: La ruta de un objeto ★★★☆☆

No es de Emberdeep. Guarda estos dos archivos, sin paquete. El árbol ya está hecho; tú escribes el cuerpo de `pathTo`:

`Node.java`

```
import java.util.List;
import java.util.Optional;

public sealed interface Node {
    record Slot(String name) implements Node {}

    record Pouch(String name, List<Node> children) implements Node {
        public Pouch {
            children = List.copyOf(children);
        }
    }

    default Optional<String> pathTo(String target) {
        return Optional.empty();    // sustituye esta línea
    }
}
```

`Main.java`

```
import java.util.List;

public class Main {
    public static void main(String[] args) {
        Node pocket = new Node.Pouch("bolsillo", List.of(
                new Node.Slot("poción")));
        Node bag = new Node.Pouch("bolsa", List.of(
                new Node.Slot("bomba"), pocket));
        Node backpack = new Node.Pouch("mochila", List.of(
                new Node.Slot("mapa"), bag, new Node.Slot("poción")));
        for (String name : List.of("poción", "bomba", "mapa", "espada")) {
            System.out.println(name + ": "
                    + backpack.pathTo(name).orElse("no está"));
        }
    }
}
```

`pathTo(target)` devuelve la ruta completa del *primer* objeto que se llame `target`, recorriendo en profundidad y en el orden de la lista, con los nombres separados por `/` y empezando por el nombre de la bolsa más externa, o `Optional.empty()` si no existe. Está bien si la salida es exactamente:

Salida esperada

```
poción: mochila/bolsa/bolsillo/poción
bomba: mochila/bolsa/bomba
mapa: mochila/mapa
espada: no está
```

Pista: *Una hoja devuelve su propio nombre si coincide. Una bolsa pregunta a cada hijo, se queda con la primera respuesta que no esté vacía y, si la hay, le antepone su nombre y una barra con* `map`*.*

Práctica ▪ solución en la pág. 612

Ejercicio 13.4: Tres operaciones y un tipo de nodo nuevos ★★★★☆

El equipo de Emberdeep acaba de aprobar dos cambios en el inventario del capítulo, y los dos llegan a la vez:

- **Tres operaciones nuevas:** `weight()` (el peso total, con un peso por objeto), `describe()` (un texto con la mochila en forma de árbol, para el menú) y `sellValue()` (el oro que daría vender todo).
- **Un tipo de nodo nuevo:** `LockedBag`, una bolsa con llave que no deja ver lo que contiene hasta abrirla (para `describe()` y `find`, vale como un solo objeto opaco).

Decide cómo implementarlo: métodos en cada nodo, un `switch` sobre la interfaz sellada o un Visitor clásico, y justifícalo. Entrega:

1. Una tabla con las tres formas y lo que cuesta, en cada una, añadir las tres operaciones y añadir el tipo nuevo.
2. Tu elección y por qué, con lo que sabes de este proyecto (cuántas operaciones y cuántos tipos crecen, y quién escribe cada cosa).
3. El código de *una* de las tres operaciones en tu forma elegida, incluido el tratamiento de `LockedBag`.
4. Qué te diría el compilador al añadir `LockedBag`, según la forma elegida.

Está bien si: (1) la tabla distingue operación nueva de tipo nuevo en las tres formas; (2) tu elección usa un argumento sobre qué crece más a menudo y no solo un gusto; (3) el código compila con la forma elegida; (4) nombras al menos un coste de tu elección.

Pista: *Cuenta cuántos sitios hay que tocar en cada caso: tres operaciones nuevas son tres funciones o tres métodos en cada tipo; un tipo nuevo es un caso en cada operación o una clase sola.*

Diseño ▪ solución en la pág. 613

Ejercicio 13.5: Hito 14: inventario anidado y oleadas ★★★★☆

Implementa el hito H14 partiendo del estado del capítulo anterior (solución del ejercicio 12.5), en tres partes. Comprueba cada una antes de pasar a la siguiente.

Parte 1 (inventario inmutable).

1. `InventoryNode`, en `engine/item/`: `sealed interface` con `ItemSlot(Item item)` y

`Bag(String name, List<InventoryNode> children)`; el constructor compacto de `Bag` copia la lista con `List.copyOf`. Cuatro métodos `default` con `switch (this)` sin `default`: `int countItems()`, `int totalPower()`, `Optional<Item> find(String itemName)` (primer objeto con ese nombre, en profundidad) y `List<Item> items()` (todos los objetos, aplanados). En `Bag`: `Bag with(InventoryNode node)` (añade al final) y `Bag without(String itemName)` (quita el primer objeto con ese nombre, a cualquier profundidad; si no está, devuelve una bolsa igual). Ninguno cambia la bolsa original.

2. `GameSession`: el campo, `inventory()` y `replaceInventory(Bag)` pasan a la `Bag`. La mochila inicial es una `Bag` llamada `"mochila"` con dos pociones sueltas (`biome.item(ItemKind.HEALING)`) y una `Bag` llamada `"bolsa de cinturón"` que contiene la bomba (`ItemKind.BOMB`).

3. `UseItem`: guarda la `Bag` anterior, comprueba con `find` que el objeto sigue en la mochila (si no, lanza `IllegalStateException`), sustituye la mochila por `before.without(item.name())` y su `undo()` repone la guardada. El resto (curar, bomba, restaurar al objetivo) no cambia.

4. `ConsoleMenu`: el único cambio es el tipo del parámetro de `describeItems` y su `import`. `Snapshot`: el último campo pasa de `Inventory` a `Bag`. Borra `Inventory.java`.

Está bien si `./gradlew -q classes` compila y el programa temporal `app/Check14a.java` (compílalo con `./gradlew -q classes` y ejecútalo con `java -cp build/classes/java/main emberdeep.app.Check14a`) imprime lo de abajo. Crea una sesión con Aria (30, 8, 3), una cripta con la semilla 42 y un esqueleto, fórmula plana y esquiva 0. Guarda la mochila inicial en `start`. Imprime su número de objetos y su poder total; el nombre de la bomba encontrada con `find` (o «nada») y si `find` encuentra una «Maza». Quita 20 de vida a Aria y haz que se beba una «Poción de cripta» con un `UseItem` (el objeto lo da `find`): imprime la vida de Aria, los objetos de la mochila actual y los de `start`. Deshaz el comando: imprime la vida y si la mochila actual es *la misma referencia* que `start`. Lanza la bomba al esqueleto con otro `UseItem` (sin deshacerlo): imprime el esqueleto (vida y máximo), si la mochila actual encuentra la bomba y si `start` la encuentra.

Salida esperada de Check14a

```
Objetos: 3, poder total 34
Bomba: Bomba de ceniza
Maza: false
Poción: vida 22, mochila 2 objetos, la anterior 3
Deshecho: vida 10, misma mochila: true
Bomba: Esqueleto 8/18, en la mochila: false, en la anterior: true
```

Parte 2 (oleadas).

1. `WaveNode`, en `engine/world/wave/`: `sealed interface` con `EnemyGroup(EnemyKind kind, int count)` (lanza `IllegalArgumentException` con el mensaje «Un grupo necesita al menos un enemigo» si `count < 1`) y `Wave(List<WaveNode> parts)` (copia la lista). Dos métodos `default`: `int size()` y `List<EnemyKind> enemies()` (los enemigos en orden de recorrido, aplanados; un grupo con `count` 2 da dos veces su `kind`).

2. `Main`: la lista `deepKinds` de la planta 3 sale de una oleada con un `EnemyGroup(SKELETON, 1)` y una subola con un `EnemyGroup(NECROMANCER, 1)`,

aplanada con `enemies()`.

Está bien si `./gradlew -q classes` compila y el programa temporal `app/Check14b.java` imprime lo de abajo. Construye la oleada `Wave(EnemyGroup(SKELETON, 2), Wave(EnemyGroup(ASH_RAT, 1), EnemyGroup(ZOMBIE, 1)))`; imprime su `size()`, su `enemies()` y, tras crear una `GameSession` (Aria, cripta con semilla 42, fórmula plana, esquiva 0) con esos enemigos, sus nombres separados por coma. Por último, dentro de un `try`, crea un `EnemyGroup` de cantidad 0 e imprime el mensaje de la excepción.

Salida esperada de Check14b

```
Tamaño: 4
Enemigos: [SKELETON, SKELETON, ASH_RAT, ZOMBIE]
En la sesión: Esqueleto, Esqueleto, Rata de ceniza, Zombi
Grupo vacío: Un grupo necesita al menos un enemigo
```

Parte 3 (el Snapshot).

1. Ejecuta `./gradlew test`: el compilador rechaza `SnapshotTest` (importa `Inventory`, que ya no existe). Es el aviso del coste de Memento.
2. Actualiza la primera prueba: vacía la mochila con `new Bag("mochila", List.of())` y comprueba, tras `restore`, que `countItems()` vuelve a 3.
3. Añade una tercera prueba, `savedBagIsNotChangedByLaterUse`: guarda una instantánea, quita después una «Poción de cripta» de la mochila de la sesión y comprueba que la sesión tiene 2 objetos y la *instantánea*, 3.

Está bien si `./gradlew test` termina con `BUILD SUCCESSFUL` (tres pruebas) y, al comentar la línea de `restore` que repone el inventario, la primera prueba falla con `expected: <3> but was: <0>`. Vuelve a ponerla. Borra `Check14a.java` y `Check14b.java`.

Pista: *Para que un `UseItem` se ejecute a mano: `new UseItem(session, user, target, item).execute()`, y `undo()` lo deshace. Para comparar referencias, `==`.*

Proyecto ▪ solución en la pág. 618

13.9 Resumen

Resumen

- El olor de Composite es una estructura con elementos y grupos de elementos que se anidan, recorrida con bucles de profundidad fija, tipos perdidos y el mismo `if` sobre el tipo repetido en cada operación. Composite hace que elemento y grupo sean dos variantes de un mismo tipo.
- Piezas: el componente (`InventoryNode`, interfaz sellada), la hoja (`ItemSlot`), el compuesto (`Bag`, que guarda una lista de componentes) y la operación recursiva (`countItems`, `totalPower`, `find`): la hoja es el caso base y el compuesto combina lo que responden sus hijos con la misma llamada.
- Con tipos sellados, la operación es un `switch (this)` sin `default`; un patrón de record (`ItemSlot(Item item)`) descompone y comprueba a la vez.
- `Optional` expresa en el tipo que un resultado puede no existir: `of`, `empty`, `map`, `orElse`,

`orElseThrow` y `stream`; se usa como retorno, no como campo ni parámetro, y se evita `get()`.

- La estructura es inmutable: `with` y `without` (solo en `Bag`) devuelven una `Bag` nueva, comparten las ramas que no cambian y el `Snapshot` guarda la bolsa por referencia sin riesgo de copia superficial.
- Una operación puede vivir en cada nodo, en un `switch` sobre la sellada o en un Visitor clásico. El `switch` es barato para operaciones nuevas y obliga a revisar todos los `switch` al añadir un tipo; el Visitor solo compensa si no controlas los nodos.
- Compensa con jerarquías parte-todo de profundidad desconocida; no, con estructuras planas. Cuesta un tipo más por variante, recorridos completos y el cuidado de dónde van las operaciones de gestión de hijos.
- H14: `Inventory` desaparece en favor de `InventoryNode`; `GameSession` sustituye su `Bag` raíz; `UseItem` busca con `find`, quita con `without` y su `undo` repone la bolsa guardada; `WaveNode` describe la planta 3; `Snapshot` guarda una `Bag` y `SnapshotTest` tiene tres pruebas.

Soluciones de los ejercicios

Solución del ejercicio 13.1 enunciado en la pág. 602

Dos archivos sin paquete, en la misma carpeta.

Entry.java

```
import java.util.List;

public sealed interface Entry {
    record FileNode(String name, int size) implements Entry {}

    record FolderNode(String name, List<Entry> children)
            implements Entry {
        public FolderNode {
            children = List.copyOf(children);
        }
    }

    default int totalSize() {
        return switch (this) {
            case FileNode file -> file.size();
            case FolderNode folder -> folder.children().stream()
                    .mapToInt(Entry::totalSize)
                    .sum();
        };
    }

    default int countFiles() {
        return switch (this) {
            case FileNode file -> 1;
            case FolderNode folder -> folder.children().stream()
                    .mapToInt(Entry::countFiles)
                    .sum();
        };
    }
}
```

Main.java

```java
import java.util.List;

public class Main {
    public static void main(String[] args) {
        Entry photos = new Entry.FolderNode("fotos", List.of(
                new Entry.FileNode("a.png", 300),
                new Entry.FileNode("b.png", 450)));
        Entry empty = new Entry.FolderNode("vacia", List.of());
        Entry root = new Entry.FolderNode("raiz", List.of(
                new Entry.FileNode("notas.txt", 120), photos, empty));

        show("raiz", root);
        show("fotos", photos);
        show("vacia", empty);
    }

    private static void show(String label, Entry entry) {
        System.out.println(label + ": " + entry.totalSize()
                + " bytes en " + entry.countFiles() + " archivos");
    }
}
```

Salida

```
raiz: 870 bytes en 3 archivos
fotos: 750 bytes en 2 archivos
vacia: 0 bytes en 0 archivos
```

Por qué funciona. El archivo es el caso base: responde con su tamaño, o con 1, y no llama a nadie. La carpeta combina lo que responden sus hijos con la *misma* llamada (`Entry::totalSize`), sin saber si cada hijo es un archivo u otra carpeta. La carpeta vacía no necesita un caso especial: la suma de un `stream` sin elementos es 0. $120+(300+450)+0 = 870$. Al ser `Entry` sellada, el `switch` no lleva `default` y, si añades `Link`, el compilador señala los dos métodos.

Errores típicos:

- Poner un `default` en el `switch`: el compilador deja de avisar del tipo nuevo.
- Escribir `FileNode` sin el prefijo `Entry.` en `Main`: sin paquete, no se puede importar un record anidado, y hay que nombrarlo con el tipo que lo contiene.
- Sumar `1` a la carpeta (la carpeta no es un archivo) y contar de más: es el error del ejercicio 13.2.

Solución del ejercicio 13.2

enunciado en la pág. 603

1. El árbol de llamadas, de las hojas hacia arriba (la bolsa vacía es la primera que contesta):

Traza de backpack.countItems() (versión con el error)

```
countItems(mochila)
|-- countItems(poción)                  = 1
`-- countItems(bolsa)
    `-- countItems(vacía)
        no tiene hijos: suma 0
        devuelve 1 + 0                  = 1
    devuelve 1 + 1                      = 2
devuelve 1 + (1 + 2)                    = 4
```

2. Qué imprime. 4, y debería imprimir 1: solo hay una poción.
3. La culpable es la expresión `1 + pouch.things().stream()...`: la rama `Pouch` suma 1 por la propia bolsa, además de lo que cuentan sus hijos. Tres bolsas (mochila, bolsa y vacía) suman tres de más: $1 + 3 = 4$. Una bolsa no es un objeto; es un contenedor, y solo las hojas (`Slot`) cuentan 1.
4. Versión corregida.

Thing.java (corregido)

```java
import java.util.List;

public sealed interface Thing {
    record Slot(String name) implements Thing {}

    record Pouch(String name, List<Thing> things) implements Thing {}

    default int countItems() {
        return switch (this) {
            case Slot slot -> 1;
            case Pouch pouch -> pouch.things().stream()
                    .mapToInt(Thing::countItems)
                    .sum();
        };
    }
}
```

Main.java (sin cambios en la lógica)

```java
import java.util.List;

public class Main {
    public static void main(String[] args) {
        Thing empty = new Thing.Pouch("vacía", List.of());
        Thing inner = new Thing.Pouch("bolsa", List.of(empty));
        Thing backpack = new Thing.Pouch("mochila", List.of(
                new Thing.Slot("poción"), inner));
        System.out.println(backpack.countItems());
        System.out.println(empty.countItems());
    }
}
```

Salida

```
1
0
```

Una bolsa vacía devuelve 0 porque la suma de un `stream` sin elementos es 0. Ese 0 es el **caso base** de la recursión, junto con la hoja (1): sin él, una bolsa que no contiene nada contaría como un objeto, y es lo que arrastraba el error hasta arriba.

Errores típicos:

- Arreglarlo restando 1 al final en `main`: el contador seguiría mal con otra forma de árbol, y el fallo está en el método.
- Contar la bolsa para «incluir el contenedor» cuando lo que se pregunta es cuántos objetos hay.
- Trazar de arriba abajo sin escribir los valores que devuelve cada llamada: se pierde la cuenta en cuanto hay tres niveles.

Solución del ejercicio 13.3

enunciado en la pág. 604

Node.java

```java
import java.util.List;
import java.util.Optional;

public sealed interface Node {
    record Slot(String name) implements Node {}

    record Pouch(String name, List<Node> children) implements Node {
        public Pouch {
            children = List.copyOf(children);
        }
    }

    default Optional<String> pathTo(String target) {
        return switch (this) {
            case Slot slot -> slot.name().equals(target)
                    ? Optional.of(slot.name())
                    : Optional.empty();
            case Pouch pouch -> pouch.children().stream()
                    .map(child -> child.pathTo(target))
                    .flatMap(Optional::stream)
                    .findFirst()
                    .map(rest -> pouch.name() + "/" + rest);
        };
    }
}
```

Main.java

```java
import java.util.List;

public class Main {
    public static void main(String[] args) {
        Node pocket = new Node.Pouch("bolsillo", List.of(
                new Node.Slot("poción")));
        Node bag = new Node.Pouch("bolsa", List.of(
                new Node.Slot("bomba"), pocket));
        Node backpack = new Node.Pouch("mochila", List.of(
                new Node.Slot("mapa"), bag, new Node.Slot("poción")));
        for (String name : List.of("poción", "bomba", "mapa", "espada")) {
            System.out.println(name + ": "
                    + backpack.pathTo(name).orElse("no está"));
        }
    }
}
```

Salida

```
poción: mochila/bolsa/bolsillo/poción
bomba: mochila/bolsa/bomba
mapa: mochila/mapa
espada: no está
```

Por qué funciona. La hoja devuelve una caja con su nombre si coincide y una caja vacía si no. La bolsa pregunta a cada hijo (`map` produce un `Stream` de cajas); `flatMap(Optional::stream)` descarta las vacías; `findFirst` se queda con la primera que tiene valor y devuelve otra caja, vacía si ninguna acertó. Ese `map` final antepone el nombre de la bolsa y una barra *solo si hay ruta*: sobre una caja vacía no hace nada, así que no hace falta un `if`. Como el `Stream` es perezoso, la segunda «poción» de la mochila, la que está sin bolsa, ni siquiera se mira: la primera ruta en profundidad gana.

Errores típicos:

- Devolver `null` para «no está»: la firma dice `Optional`, y quien llama esperaría una caja, no un `NullPointerException`.
- Usar `get()` sobre la caja del hijo: lanza `NoSuchElementException` en cuanto un hijo no tiene el objeto.
- Anteponer el nombre de la bolsa a *todos* los hijos, incluidos los que no contienen el objeto, y devolver rutas que no existen.
- Recorrer todos los hijos y quedarse con la última ruta en lugar de la primera.

Solución del ejercicio 13.4

enunciado en la pág. 605

Solución de referencia.
1. Qué cuesta en cada forma.

Forma	Añadir las tres operaciones	Añadir `LockedBag`
Método en cada nodo	Tres métodos en la interfaz y en cada tipo existente	Una clase nueva con todos sus métodos; no se toca nada existente
`switch` sobre la sellada	Tres funciones nuevas, con un caso por tipo; no se toca ningún tipo	Un caso en cada `switch` (los cuatro actuales y los tres nuevos); el compilador los lista
Visitor clásico	Tres clases `Visitor`; antes, un `accept` en cada nodo y reescribir las cuatro operaciones actuales como visitors	Un método `visitLocked` en la interfaz `Visitor` y en cada `Visitor`

2. Elección: el `switch` sobre la interfaz sellada. En este proyecto lo que crece son las *operaciones*: ya hay cuatro (`countItems`, `totalPower`, `find`, `items`), llegan tres más y vendrán otras (el capítulo 14 aplana la bolsa para guardarla). Los tipos de nodo, en cambio, son pocos y los controlamos nosotros: uno nuevo cada cierto tiempo. Con el `switch`, cada operación nueva es una función en un sitio y el código sigue como está; con métodos en los nodos, cada operación se reparte por todas las clases; y el Visitor obligaría a reescribir lo que ya funciona (`accept` en cada nodo y las cuatro operaciones como clases) para ahorrar un esfuerzo que aquí no hace falta, porque los nodos son nuestros y están sellados. Coste que asumo: `LockedBag` toca siete `switch`, y la lógica de «qué es una bolsa cerrada» queda repartida entre ellos.

3. El código de una operación. `describe()` con `LockedBag` como nodo opaco. Tres archivos sin paquete:

`Item.java`

```
1 public record Item(String name) {}
```

InventoryNode.java

```
import java.util.List;
import java.util.stream.Collectors;

public sealed interface InventoryNode {
    record ItemSlot(Item item) implements InventoryNode {}

    record Bag(String name, List<InventoryNode> children)
            implements InventoryNode {
        public Bag {
            children = List.copyOf(children);
        }
    }

    // opaca: no se ve lo que hay dentro hasta abrirla con la llave
    record LockedBag(String name, List<InventoryNode> children,
                     String key) implements InventoryNode {
        public LockedBag {
            children = List.copyOf(children);
        }
    }

    default String describe() {
        return describeAt("");
    }

    private String describeAt(String indent) {
        return switch (this) {
            case ItemSlot slot -> indent + slot.item().name() + "\n";
            case Bag bag -> indent + bag.name() + "/\n"
                    + bag.children().stream()
                            .map(child -> child.describeAt(indent + "  "))
                            .collect(Collectors.joining());
            case LockedBag locked ->
                    indent + locked.name() + " (cerrada)\n";
        };
    }
}
```

Main.java

```java
import java.util.List;

public class Main {
    public static void main(String[] args) {
        InventoryNode backpack = new InventoryNode.Bag("mochila", List.of(
                new InventoryNode.ItemSlot(new Item("Poción de cripta")),
                new InventoryNode.Bag("bolsa de cinturón", List.of(
                        new InventoryNode.ItemSlot(
                                new Item("Bomba de ceniza")))),
                new InventoryNode.LockedBag("cofre", List.of(
                        new InventoryNode.ItemSlot(new Item("Anillo"))),
                        "llave de hierro")));
        System.out.print(backpack.describe());
    }
}
```

Salida

```
mochila/
  Poción de cripta
  bolsa de cinturón/
    Bomba de ceniza
  cofre (cerrada)
```

El anillo del cofre no aparece: el caso `LockedBag` no mira los hijos. El método recursivo auxiliar es `private` (Java permite métodos privados en una interfaz) para no exponer el parámetro de sangría.

4. Qué avisa el compilador. Con el `switch` sobre la sellada, al añadir `LockedBag` cada `switch` sin `default` deja de compilar con el mensaje de que no cubre todos los valores posibles de entrada; con métodos en los nodos, la clase nueva no compila hasta implementar los métodos abstractos que le faltan; con un Visitor, cada visitor existente no compila hasta tener `visitLocked`. En las tres formas el compilador ayuda, y esa es la propiedad que no hay que perder (por eso el `switch` no lleva `default`).

Criterios para valorar otras soluciones. Sirve cualquier elección que: distinga en la tabla operación nueva de tipo nuevo en las tres formas; argumente con lo que crece más a menudo en el proyecto; incluya código que compile con la forma elegida y trate `LockedBag`; y nombre un coste. También es válido elegir métodos en los nodos si se defiende que los tipos van a crecer más que las operaciones.

Alternativa válida: Visitor clásico. Compensa cuando los nodos son una biblioteca que no controlas (no puedes añadir métodos ni sellarlos), cuando el árbol no es sellado o cuando quieres que las operaciones estén fuera del modelo y sean intercambiables. Se paga con un `accept` en cada nodo y una interfaz que crece con cada tipo. La misma `describe()`, como programa completo en un solo archivo:

VisitorDemo.java (1/2)

```
import java.util.List;

public class VisitorDemo {
    record Item(String name) {}

    interface Node {
        <R> R accept(Visitor<R> visitor);
    }

    interface Visitor<R> {
        R visitSlot(Slot slot);
        R visitBag(Bag bag);
        R visitLocked(Locked locked);
    }

    record Slot(Item item) implements Node {
        public <R> R accept(Visitor<R> v) { return v.visitSlot(this); }
    }

    record Bag(String name, List<Node> children) implements Node {
        public <R> R accept(Visitor<R> v) { return v.visitBag(this); }
    }

    record Locked(String name) implements Node {
        public <R> R accept(Visitor<R> v) { return v.visitLocked(this); }
    }
```

VisitorDemo.java (2/2)

```
    static class Describe implements Visitor<String> {
        private final String indent;

        Describe(String indent) { this.indent = indent; }

        public String visitSlot(Slot slot) {
            return indent + slot.item().name() + "\n";
        }

        public String visitBag(Bag bag) {
            StringBuilder text =
                    new StringBuilder(indent + bag.name() + "/\n");
            for (Node child : bag.children()) {
                text.append(child.accept(new Describe(indent + "  ")));
            }
            return text.toString();
        }

        public String visitLocked(Locked locked) {
            return indent + locked.name() + " (cerrada)\n";
        }
    }

    public static void main(String[] args) {
        Node backpack = new Bag("mochila", List.of(
                new Slot(new Item("Poción de cripta")),
                new Locked("cofre")));
        System.out.print(backpack.accept(new Describe("")));
    }
}
```

Salida de VisitorDemo

```
mochila/
  Poción de cripta
  cofre (cerrada)
```

Errores típicos:

- Elegir por gusto («me gusta el Visitor») y no por lo que crece en el proyecto.
- Añadir un `default` al `switch` para que `LockedBag` compile: la bolsa cerrada se trataría como «lo que sea» y el compilador dejaría de avisar.
- En el Visitor, olvidar que `accept` se escribe en *cada* nodo: es la parte que ya no hace falta con un tipo sellado.

Solución del ejercicio 13.5 enunciado en la pág. 605

Parte del estado del capítulo 12. Primero, los archivos nuevos; después, los que cambian. **Parte 1: el inventario inmutable.**

engine/item/InventoryNode.java (1/2)

```
package emberdeep.engine.item;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public sealed interface InventoryNode {
    record ItemSlot(Item item) implements InventoryNode {}

    record Bag(String name, List<InventoryNode> children)
            implements InventoryNode {
        public Bag {
            children = List.copyOf(children);
        }

        public Bag with(InventoryNode node) {
            List<InventoryNode> next = new ArrayList<>(children);
            next.add(node);
            return new Bag(name, next);
        }

        public Bag without(String itemName) {
            List<InventoryNode> next = new ArrayList<>();
            boolean removed = false;
            for (InventoryNode child : children) {
                if (removed) {
                    next.add(child);
                    continue;
                }
                switch (child) {
                    case ItemSlot slot -> {
                        if (slot.item().name().equals(itemName)) {
                            removed = true;
                        } else {
                            next.add(slot);
                        }
                    }
                    case Bag bag -> {
                        if (bag.find(itemName).isPresent()) {
                            next.add(bag.without(itemName));
                            removed = true;
                        } else {
                            next.add(bag);
                        }
                    }
                }
            }
            return new Bag(name, next);
        }
    }
```

engine/item/InventoryNode.java (2/2)

```
    default int countItems() {
        return switch (this) {
            case ItemSlot slot -> 1;
            case Bag bag -> bag.children().stream()
                    .mapToInt(InventoryNode::countItems)
                    .sum();
        };
    }

    default int totalPower() {
        return switch (this) {
            case ItemSlot(Item item) -> item.power();
            case Bag bag -> bag.children().stream()
                    .mapToInt(InventoryNode::totalPower)
                    .sum();
        };
    }

    default Optional<Item> find(String itemName) {
        return switch (this) {
            case ItemSlot slot -> slot.item().name().equals(itemName)
                    ? Optional.of(slot.item())
                    : Optional.empty();
            case Bag bag -> bag.children().stream()
                    .map(child -> child.find(itemName))
                    .flatMap(Optional::stream)
                    .findFirst();
        };
    }

    default List<Item> items() {
        return switch (this) {
            case ItemSlot slot -> List.of(slot.item());
            case Bag bag -> bag.children().stream()
                    .flatMap(child -> child.items().stream())
                    .toList();
        };
    }
}
```

Esas dos mitades son un solo archivo (la segunda continúa dentro de la interfaz, después de la llave que cierra `Bag`). Borra `engine/item/Inventory.java`. Ahora el `Snapshot`, que cambia de tipo en una palabra:

engine/save/Snapshot.java

```
package emberdeep.engine.save;

import emberdeep.engine.item.InventoryNode.Bag;
import java.util.List;

public record Snapshot(int floor, int x, int y, int round,
                       List<FighterState> fighters, Bag inventory) {
    public Snapshot {
        fighters = List.copyOf(fighters);
    }
}
```

Y ahora `GameSession`, entera, en tres tramos seguidos de la misma clase (cambian los `import`, el campo, la mochila inicial, `inventory()` y `replaceInventory`):

engine/session/GameSession.java (1/3)

```
package emberdeep.engine.session;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.event.EventBus;
import emberdeep.engine.item.InventoryNode.Bag;
import emberdeep.engine.item.InventoryNode.ItemSlot;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.save.FighterState;
import emberdeep.engine.save.Snapshot;
import emberdeep.engine.stats.ArmorBonus;
import emberdeep.engine.stats.BaseStats;
import emberdeep.engine.stats.StatSource;
import emberdeep.engine.stats.WeaponBonus;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Stream;

public class GameSession {
    private static final int SWORD_ATTACK = 2;
    private static final int ARMOR_VALUE = 1;

    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final Combat combat;
    private final EventBus events = new EventBus();
    private Bag inventory;
    private int floor = 1;
    private int x;
    private int y;
```

engine/session/GameSession.java (2/3)

```
    public GameSession(List<Hero> heroes, BiomeFactory biome,
                       List<EnemyKind> kinds, DamageFormula formula,
                       DamagePipeline pipeline) {
        this.heroes = heroes.stream()
                .map(hero -> Fighter.ofHero(hero, gearOf(hero)))
                .toList();
        // mutable: la arena añade los refuerzos al final de la lista
        this.enemies = new ArrayList<>(kinds.stream()
                .map(biome::enemy)
                .map(Fighter::ofEnemy)
                .toList());
        this.combat = new Combat(this.heroes, this.enemies,
                formula, pipeline, events, biome);
        // cap. 13: la bomba va en una bolsa dentro de la mochila
        this.inventory = new Bag("mochila", List.of())
                .with(new ItemSlot(biome.item(ItemKind.HEALING)))
                .with(new ItemSlot(biome.item(ItemKind.HEALING)))
                .with(new Bag("bolsa de cinturón", List.of())
                        .with(new ItemSlot(biome.item(ItemKind.BOMB))));
    }

    private static StatSource gearOf(Hero hero) {
        return new ArmorBonus(
                new WeaponBonus(new BaseStats(hero.stats()), SWORD_ATTACK),
                ARMOR_VALUE);
    }

    public List<Fighter> heroes() { return heroes; }
    public List<Fighter> enemies() { return enemies; }
    public Combat combat() { return combat; }
    public EventBus events() { return events; }
    public int round() { return combat.round(); }
    public Bag inventory() { return inventory; }
    public int floor() { return floor; }
    public int x() { return x; }
    public int y() { return y; }

    public void replaceInventory(Bag next) {
        this.inventory = next;
    }
```

engine/session/GameSession.java (3/3)

```
    public void setLocation(int floor, int x, int y) {
        this.floor = floor;
        this.x = x;
        this.y = y;
    }

    private List<Fighter> allFighters() {
        return Stream.concat(heroes.stream(), enemies.stream()).toList();
    }

    public Snapshot createSnapshot() {
        List<FighterState> states = allFighters().stream()
                .map(f -> new FighterState(f.hp(), f.shield(), f.stats(),
                        f.status()))
                .toList();
        return new Snapshot(floor, x, y, combat.round(), states,
                inventory);
    }

    public void restore(Snapshot snapshot) {
        int saved = snapshot.fighters().size();
        if (saved < heroes.size() || saved > allFighters().size()) {
            throw new IllegalArgumentException(
                    "La instantánea es de otra partida");
        }
        // cap. 11: los refuerzos invocados después del punto de control
        // se descartan (siempre están al final de la lista de enemigos)
        enemies.subList(saved - heroes.size(), enemies.size()).clear();
        List<Fighter> all = allFighters();
        for (int i = 0; i < all.size(); i++) {
            FighterState state = snapshot.fighters().get(i);
            all.get(i).restore(state.hp(), state.shield(), state.stats());
            all.get(i).setStatus(state.status());
        }
        combat.restoreRound(snapshot.round());
        inventory = snapshot.inventory();
        floor = snapshot.floor();
        x = snapshot.x();
        y = snapshot.y();
    }
}
```

engine/combat/command/UseItem.java (1/2)

```
package emberdeep.engine.combat.command;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.Damage;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.event.GameEvent.ItemUsed;
import emberdeep.engine.item.InventoryNode.Bag;
import emberdeep.engine.item.Item;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.stats.StatSource;

public class UseItem implements CombatCommand {
    private final GameSession session;
    private final Fighter user;
    private final Fighter target;
    private final Item item;
    private Bag bagBefore;
    private int hpBefore;
    private int shieldBefore;
    private StatSource statsBefore;

    public UseItem(GameSession session, Fighter user,
                   Fighter target, Item item) {
        this.session = session;
        this.user = user;
        this.target = target;
        this.item = item;
    }
```

engine/combat/command/UseItem.java (2/2)

```
    @Override
    public void execute() {
        bagBefore = session.inventory();
        if (bagBefore.find(item.name()).isEmpty()) {
            throw new IllegalStateException("No tienes " + item.name());
        }
        hpBefore = target.hp();
        shieldBefore = target.shield();
        statsBefore = target.stats();
        session.replaceInventory(bagBefore.without(item.name()));
        Combat combat = session.combat();
        switch (item.kind()) {
            case HEALING -> {
                target.heal(item.power());
                combat.publish(new ItemUsed(user.name(), target.name(),
                        item.name(), target.hp(), target.stats().maxHp()));
            }
            case BOMB -> combat.deal(user, target,
                    new Damage(item.power(), DamageType.FIRE));
        }
    }

    @Override
    public void undo() {
        session.replaceInventory(bagBefore);
        target.restore(hpBefore, shieldBefore, statsBefore);
    }
}
```

ConsoleMenu.java, completo. Cambian dos cosas respecto al capítulo 12: el `import` de `Inventory` pasa a ser el de `Bag`, y `describeItems` recibe una `Bag`. `chooseItem` sigue valiendo: `session.inventory().items()` es ahora el `items()` recursivo de la interfaz, que devuelve la lista aplanada.

ui/ConsoleMenu.java (1/6)

```
package emberdeep.ui;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Skill.Heal;
import emberdeep.engine.character.Skill.Strike;
import emberdeep.engine.character.Skill.Stun;
import emberdeep.engine.character.Skill.Summon;
import emberdeep.engine.character.Skill.Taunt;
import emberdeep.engine.character.Skill.Venom;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.Attack;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.command.Defend;
import emberdeep.engine.combat.command.UseItem;
import emberdeep.engine.combat.command.UseSkill;
import emberdeep.engine.item.InventoryNode.Bag;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.save.SnapshotStore;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.TileType;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.input.InputSource;
import emberdeep.ui.view.MapView;
import java.io.PrintStream;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class ConsoleMenu {
    private enum Turn { ACTED, UNDONE, RESTORED }

    private final InputSource input;
    private final PrintStream out;
    private final MapView mapView;
    private final SnapshotStore checkpoints = new SnapshotStore();
    private int checkpointHero;     // a qué héroe le tocaba al guardar
    private List<Fighter> acting = List.of();       // quién actúa ahora
    private List<Fighter> checkpointActing = List.of();   // y al guardar
    public ConsoleMenu(InputSource input, PrintStream out,
                       MapView mapView) {
        this.input = input;
        this.out = out;
        this.mapView = mapView;
    }
```

ui/ConsoleMenu.java (2/6)

```
    public int askFormula() {
        out.println("Fórmula de daño:");
        out.println("  [1] plana: ataque - defensa");
        out.println("  [2] escalada: ataque x 100 / (100 + defensa)");
        return ask(2);
    }

    public void run(GameSession session) {
        Combat combat = session.combat();
        out.println("=== Emberdeep ===");
        while (!combat.isOver()) {
            out.println("--- Ronda " + session.round() + " ---");
            acting = startRound(session);
            int i = 0;
            while (i < acting.size() && !combat.isOver()) {
                Turn turn = heroTurn(session, acting.get(i), i);
                switch (turn) {
                    case ACTED -> i++;
                    case UNDONE -> i = Math.max(0, i - 1);
                    case RESTORED -> {
                        acting = checkpointActing;
                        i = checkpointHero;
                    }
                }
            }
            if (!combat.isOver()) {
                combat.enemyPhase();
            }
        }
    }

    private List<Fighter> aliveHeroes(GameSession session) {
        return session.heroes().stream().filter(Fighter::isAlive).toList();
    }
    // Cap. 12: cada héroe consulta su estado al empezar la ronda. Actúan
    // los que siguen vivos y no pierden el turno.
    private List<Fighter> startRound(GameSession session) {
        List<Fighter> ready = new ArrayList<>();
        for (Fighter hero : aliveHeroes(session)) {
            if (session.combat().startTurn(hero)) {
                ready.add(hero);
            }
        }
        return ready;
    }
    // ACTED: el héroe actúa; UNDONE: se deshace la acción anterior;
    // RESTORED: se vuelve a un punto de control
    private Turn heroTurn(GameSession session, Fighter hero, int index) {
```

ui/ConsoleMenu.java (3/6)

```
        Combat combat = session.combat();
        while (true) {
            out.println("Héroes:   " + describe(session.heroes()));
            out.println("Enemigos: " + describe(session.enemies()));
            out.println("Objetos:  " + describeItems(session.inventory()));
            out.println("Turno de " + hero.name()
                    + ". [1] atacar  [2] defender  [3] objeto"
                    + "  [4] habilidad  [u] deshacer"
                    + "  [c] guardar  [r] volver");
            String key = input.next();
            if (key.equals("u")) {
                if (combat.undo()) {
                    out.println("Deshaces la última acción.");
                    return Turn.UNDONE;
                }
                out.println("No hay nada que deshacer en esta ronda.");
                continue;
            }
            if (key.equals("c")) {
                saveCheckpoint(session, index);
                continue;
            }
            if (key.equals("r")) {
                if (loadCheckpoint(session)) {
                    return Turn.RESTORED;
                }
                continue;
            }
            CombatCommand command = switch (key) {
                case "1" -> new Attack(combat, hero,
                        askTarget("Objetivo", session.enemies()));
                case "2" -> new Defend(combat, hero);
                case "3" -> chooseItem(session, hero);
                case "4" -> chooseSkill(session, hero);
                default -> {
                    out.println("Tecla no válida.");
                    yield null;
                }
            };
            if (command != null) {
                combat.perform(command);
                return Turn.ACTED;
            }
        }
    }
    private void saveCheckpoint(GameSession session, int index) {
        checkpoints.push(session.createSnapshot());
        checkpointHero = index;
```

ui/ConsoleMenu.java (4/6)

```
        checkpointActing = acting;
        out.println("Punto de control guardado ("
                + checkpoints.size() + ").");
    }

    // false si todavía no hay ninguno
    private boolean loadCheckpoint(GameSession session) {
        if (checkpoints.isEmpty()) {
            out.println("No hay puntos de control.");
            return false;
        }
        session.restore(checkpoints.last());
        out.println("Vuelves al último punto de control (ronda "
                + session.round() + ").");
        return true;
    }

    private CombatCommand chooseItem(GameSession session, Fighter hero) {
        List<Item> items = session.inventory().items();
        if (items.isEmpty()) {
            out.println("No te quedan objetos.");
            return null;
        }
        Item item = items.get(choose("Objeto",
                items.stream().map(Item::name).toList()));
        Fighter target = item.kind() == ItemKind.HEALING
                ? askTarget("Aliado", session.heroes())
                : askTarget("Objetivo", session.enemies());
        return new UseItem(session, hero, target, item);
    }
    private CombatCommand chooseSkill(GameSession session, Fighter hero) {
        List<Skill> skills = hero.skills();
        if (skills.isEmpty()) {
            out.println("No tienes habilidades.");
            return null;
        }
        Skill skill = skills.get(choose("Habilidad",
                skills.stream().map(Skill::name).toList()));
        Fighter target = switch (skill) {
            case Strike strike -> askTarget("Objetivo", session.enemies());
            case Heal heal -> askTarget("Aliado", session.heroes());
            case Taunt taunt -> hero;       // se lanza sobre sí mismo
            case Venom venom -> askTarget("Objetivo", session.enemies());
            case Stun stun -> askTarget("Objetivo", session.enemies());
            case Summon summon -> hero;     // solo la usan los enemigos
        };
        Combat combat = session.combat();
        return new UseSkill(combat, combat.mediator(), hero, skill,
```

ui/ConsoleMenu.java (5/6)

```
                target);
    }

    private Fighter askTarget(String title, List<Fighter> group) {
        List<Fighter> alive = group.stream()
                .filter(Fighter::isAlive).toList();
        return alive.get(choose(title,
                alive.stream().map(Fighter::name).toList()));
    }

    // Muestra "Título:  [1] a  [2] b" y devuelve la posición (desde 0)
    private int choose(String title, List<String> labels) {
        StringBuilder menu = new StringBuilder(title + ":");
        for (int i = 0; i < labels.size(); i++) {
            menu.append("  [").append(i + 1).append("] ")
                    .append(labels.get(i));
        }
        out.println(menu);
        return ask(labels.size()) - 1;
    }
    private int ask(int max) {
        while (true) {
            String line = input.next();
            try {
                int n = Integer.parseInt(line);
                if (n >= 1 && n <= max) {
                    return n;
                }
            } catch (NumberFormatException e) {
                // no es un número: se avisa más abajo
            }
            out.println("Escribe un número entre 1 y " + max + ".");
        }
    }

    private String describe(List<Fighter> group) {
        return group.stream()
                .map(f -> f.name() + " " + f.hp() + "/"
                        + f.stats().maxHp())
                .collect(Collectors.joining(", "));
    }
```

ui/ConsoleMenu.java (6/6)

```
    private String describeItems(Bag inventory) {
        if (inventory.items().isEmpty()) {
            return "(ninguno)";
        }
        return inventory.items().stream()
                .map(Item::name)
                .collect(Collectors.joining(", "));
    }

    public boolean explore(GameMap map, TileTypes types) {
        while (true) {
            mapView.show(map);
            out.println("[w a s d] mover, [t] casillas, [q] salir");
            String key = input.next();
            switch (key) {
                case "w" -> step(map, 0, -1);
                case "a" -> step(map, -1, 0);
                case "s" -> step(map, 0, 1);
                case "d" -> step(map, 1, 0);
                case "t" -> showTiles(map, types);
                case "q" -> { return false; }
                default -> out.println("Tecla no válida.");
            }
            if (map.onStairs()) {
                out.println("Bajas por la escalera.");
                return true;
            }
        }
    }

    private void step(GameMap map, int dx, int dy) {
        if (!map.move(dx, dy)) {
            out.println("No puedes pasar.");
        }
    }

    private void showTiles(GameMap map, TileTypes types) {
        TileType corner = map.tileAt(0, 0);
        TileType opposite = map.tileAt(map.width() - 1, map.height() - 1);
        out.println("Casillas: " + map.width() * map.height()
                + ", TileType creados: " + types.createdCount());
        out.println("Esquinas opuestas, mismo objeto: "
                + (corner == opposite));
    }
}
```

Y el programa temporal de la parte 1:

app/Check14a.java (1/2)

```java
package emberdeep.app;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.command.UseItem;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.item.InventoryNode.Bag;
import emberdeep.engine.item.Item;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.biome.CryptBiome;
import java.util.List;
import java.util.Random;

public class Check14a {
    public static void main(String[] args) {
        Random random = new Random(42);
        List<Hero> heroes = List.of(new HeroBuilder().name("Aria")
                .stats(new Stats(30, 8, 3)).build());
        GameSession session = new GameSession(heroes,
                new CryptBiome(random), List.of(EnemyKind.SKELETON),
                new FlatFormula(), new DamagePipeline(random, 0.0));
        Fighter aria = session.heroes().get(0);
        Fighter skeleton = session.enemies().get(0);

        Bag start = session.inventory();
        System.out.println("Objetos: " + start.countItems()
                + ", poder total " + start.totalPower());
        System.out.println("Bomba: " + start.find("Bomba de ceniza")
                .map(Item::name).orElse("nada"));
        System.out.println("Maza: " + start.find("Maza").isPresent());
```

app/Check14a.java (2/2)

```
1
2           aria.takeDamage(20);
3           Item potion = start.find("Poción de cripta").orElseThrow();
4           UseItem drink = new UseItem(session, aria, aria, potion);
5           drink.execute();
6           System.out.println("Poción: vida " + aria.hp() + ", mochila "
7                   + session.inventory().countItems()
8                   + " objetos, la anterior " + start.countItems());
9           drink.undo();
10          System.out.println("Deshecho: vida " + aria.hp()
11                  + ", misma mochila: " + (session.inventory() == start));
12
13          Item bomb = start.find("Bomba de ceniza").orElseThrow();
14          new UseItem(session, aria, skeleton, bomb).execute();
15          System.out.println("Bomba: Esqueleto " + skeleton.hp() + "/"
16                  + skeleton.stats().maxHp() + ", en la mochila: "
17                  + session.inventory().find("Bomba de ceniza").isPresent()
18                  + ", en la anterior: "
19                  + start.find("Bomba de ceniza").isPresent());
20      }
21  }
```

Salida de Check14a

```
Objetos: 3, poder total 34
Bomba: Bomba de ceniza
Maza: false
Poción: vida 22, mochila 2 objetos, la anterior 3
Deshecho: vida 10, misma mochila: true
Bomba: Esqueleto 8/18, en la mochila: false, en la anterior: true
```

Por qué funciona (parte 1). La mochila inicial tiene dos pociones de 12 y la bomba de 10: $12 + 12 + 10 = 34$ de poder y tres objetos, aunque la bomba esté dentro de la bolsa de cinturón: `countItems`, `totalPower` y `find` bajan solos por la bolsa. Al beber, `UseItem` guarda la mochila actual (`bagBefore`), comprueba con `find` que la poción sigue ahí y pone `bagBefore.without(...)`: una `Bag` *nueva* con dos objetos. La anterior sigue teniendo tres, porque nada la ha cambiado. Aria tenía $30 - 20 = 10$ de vida y la poción cura 12: 22. Con `undo()`, `replaceInventory(bagBefore)` devuelve *el mismo objeto* (`==` da `true`, no solo «igual») y `restore` repone la vida. La bomba hace 10 al esqueleto (18 de vida, sin resistencia al fuego): queda en 8. Esa vez la bolsa de cinturón de la mochila nueva queda vacía, mientras que la de `start` sigue con la bomba, y las dos pociones sueltas son los *mismos* objetos en las dos mochilas.

Parte 2: las oleadas.

engine/world/wave/WaveNode.java

```java
package emberdeep.engine.world.wave;

import emberdeep.engine.character.EnemyKind;
import java.util.Collections;
import java.util.List;

public sealed interface WaveNode {
    record EnemyGroup(EnemyKind kind, int count) implements WaveNode {
        public EnemyGroup {
            if (count < 1) {
                throw new IllegalArgumentException(
                        "Un grupo necesita al menos un enemigo");
            }
        }
    }

    record Wave(List<WaveNode> parts) implements WaveNode {
        public Wave {
            parts = List.copyOf(parts);
        }
    }

    default int size() {
        return switch (this) {
            case EnemyGroup group -> group.count();
            case Wave wave -> wave.parts().stream()
                    .mapToInt(WaveNode::size)
                    .sum();
        };
    }

    // los enemigos en orden de recorrido, aplanados
    default List<EnemyKind> enemies() {
        return switch (this) {
            case EnemyGroup group ->
                    Collections.nCopies(group.count(), group.kind());
            case Wave wave -> wave.parts().stream()
                    .flatMap(part -> part.enemies().stream())
                    .toList();
        };
    }
}
```

Main.java, completo. Cambian dos `import` nuevos (`EnemyGroup` y `Wave`) y la declaración de `deepKinds` en `play`, que ahora es una oleada con una subola.

app/Main.java (1/5)

```
package emberdeep.app;

import emberdeep.engine.achievement.AchievementTracker;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Skill.Heal;
import emberdeep.engine.character.Skill.Strike;
import emberdeep.engine.character.Skill.Stun;
import emberdeep.engine.character.Skill.Taunt;
import emberdeep.engine.character.Skill.Venom;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.event.EventBus;
import emberdeep.engine.flow.GameFlow;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.Dungeon;
import emberdeep.engine.world.DungeonBuilder;
import emberdeep.engine.world.Floor;
import emberdeep.engine.world.biome.BiomeFactory;
import emberdeep.engine.world.biome.CavernBiome;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.MapData;
import emberdeep.engine.world.map.MapParser;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.engine.world.wave.WaveNode.EnemyGroup;
import emberdeep.engine.world.wave.WaveNode.Wave;
import emberdeep.ui.ConsoleMenu;
import emberdeep.ui.input.ConsoleInput;
import emberdeep.ui.input.InputSource;
import emberdeep.ui.input.ScriptInput;
import emberdeep.ui.output.AnsiOutput;
import emberdeep.ui.output.Output;
import emberdeep.ui.output.PlainOutput;
import emberdeep.ui.view.HudView;
import emberdeep.ui.view.LogView;
import emberdeep.ui.view.MapView;
import java.io.IOException;
import java.nio.file.Path;
import java.util.Arrays;
import java.util.List;
import java.util.Random;
```

app/Main.java (2/5)

```
import java.util.Scanner;
public class Main {
    private static final Skill POWER_STRIKE =
            new Strike("Golpe poderoso", 12, DamageType.PHYSICAL);
    private static final Skill FLARE =
            new Strike("Llamarada", 10, DamageType.FIRE);
    private static final Skill HEAL = new Heal("Luz curativa", 8);
    private static final Skill TAUNT = new Taunt("Grito de guerra");
    private static final Skill VENOM = new Venom("Daga envenenada", 3, 2);
    private static final Skill STUN = new Stun("Golpe aturdidor", 1);

    private final GameConfig config;

    private Main(GameConfig config) {
        this.config = config;
    }

    public static void main(String[] args) throws IOException {
        GameConfig config = new GameConfig(42L, 3, new FlatFormula());
        new Main(config).play(args);
    }
    private void play(String[] args) throws IOException {
        String script = scriptOf(args);
        String[] options = optionsOf(args);
        int floorNumber = chooseFloor(options);
        InputSource input = script == null
                ? new ConsoleInput(new Scanner(System.in), System.out)
                : new ScriptInput(Path.of(script), System.out);
        Output output = chooseOutput(args, script != null);
        ConsoleMenu menu = new ConsoleMenu(input, System.out,
                new MapView(output));
        Random random = new Random(config.seed());

        BiomeFactory crypt = new CryptBiome(random);
        BiomeFactory cavern = new CavernBiome(random);
        List<EnemyKind> cryptKinds = List.of(
                EnemyKind.SKELETON, EnemyKind.ASH_RAT, EnemyKind.ZOMBIE);
        List<EnemyKind> cavernKinds = List.of(
                EnemyKind.GOBLIN, EnemyKind.BAT, EnemyKind.SPIDER);
        // cap. 13: la planta 3 es una oleada con una subola
        List<EnemyKind> deepKinds = new Wave(List.of(
                new EnemyGroup(EnemyKind.SKELETON, 1),
                new Wave(List.of(
                        new EnemyGroup(EnemyKind.NECROMANCER, 1)))))
                .enemies();
        Dungeon dungeon = new DungeonBuilder()
                .floor().biome(crypt).enemies(cryptKinds)
                        .map(MapData.floor(1)).done()
```

app/Main.java (3/5)

```
                .floor().biome(cavern).enemies(cavernKinds)
                        .map(MapData.floor(2)).done()
                .floor().biome(crypt).enemies(deepKinds)
                        .map(MapData.floor(3)).done()
                .build();
        if (options.length > 1 && options[1].equals("map")) {
            explore(menu, dungeon, floorNumber);
            return;
        }
        DamageFormula formula = chooseFormula(menu, options,
                script != null);
        Floor floor = dungeon.floors().get(floorNumber - 1);
        GameSession session = new GameSession(heroes(), floor.biome(),
                floor.enemies(), formula, new DamagePipeline(random, 0.0));
        GameMap entrance = new MapParser(new TileTypes())
                .parse(floor.map());
        session.setLocation(floorNumber,
                entrance.partyX(), entrance.partyY());
        // cap. 12: la partida de consola empieza en combate
        GameFlow flow = new GameFlow(session.heroes(), config.floors());
        flow.start(floorNumber);
        flow.enterCombat();
        EventBus events = session.events();
        events.subscribe(new LogView(output));
        events.subscribe(new HudView(output));
        events.subscribe(flow);
        // el tracker va el último
        events.subscribe(new AchievementTracker(events));
        menu.run(session);
    }

    private void explore(ConsoleMenu menu, Dungeon dungeon,
                         int firstFloor) {
        TileTypes types = new TileTypes();
        MapParser parser = new MapParser(types);
        int number = firstFloor;
        while (true) {
            Floor floor = dungeon.floors().get(number - 1);
            boolean descended =
                    menu.explore(parser.parse(floor.map()), types);
            if (!descended || number == dungeon.floors().size()) {
                return;
            }
            number++;
        }
    }
    // El guion es el argumento que acaba en .txt
    private static String scriptOf(String[] args) {
```

app/Main.java (4/5)

```
        return Arrays.stream(args)
                .filter(arg -> arg.endsWith(".txt"))
                .findFirst().orElse(null);
    }

    private static String[] optionsOf(String[] args) {
        return Arrays.stream(args)
                .filter(arg -> !arg.endsWith(".txt")
                        && !arg.startsWith("--"))
                .toArray(String[]::new);
    }

    private DamageFormula chooseFormula(ConsoleMenu menu, String[] options,
                                        boolean scripted) {
        boolean useDefault = scripted
                || (options.length > 1 && options[1].equals("default"));
        if (useDefault) {
            return config.defaultFormula();
        }
        return menu.askFormula() == 1
                ? new FlatFormula() : new ScaledFormula();
    }

    private int chooseFloor(String[] options) {
        int floor = options.length > 0 ? Integer.parseInt(options[0]) : 1;
        if (floor < 1 || floor > config.floors()) {
            throw new IllegalArgumentException(
                    "La planta debe estar entre 1 y " + config.floors());
        }
        return floor;
    }
    // --plain o --ansi mandan; si no, plano con guion y color a mano
    private static Output chooseOutput(String[] args, boolean scripted) {
        List<String> all = Arrays.asList(args);
        boolean plain = all.contains("--plain");
        boolean ansi = all.contains("--ansi");
        if (plain && ansi) {
            throw new IllegalArgumentException(
                    "Elige --plain o --ansi, no las dos");
        }
        if (ansi || (!plain && !scripted)) {
            return new AnsiOutput(System.out);
        }
        return new PlainOutput(System.out);
    }
```

app/Main.java (5/5)

```
    private List<Hero> heroes() {
        return List.of(
                new HeroBuilder().name("Aria").stats(new Stats(30, 8, 3))
                        .skill(POWER_STRIKE).skill(FLARE)
                        .skill(HEAL).build(),
                new HeroBuilder().name("Borin").stats(new Stats(40, 6, 5))
                        .skill(POWER_STRIKE).skill(FLARE)
                        .skill(TAUNT).skill(STUN).build(),
                new HeroBuilder().name("Cael").stats(new Stats(24, 9, 2))
                        .skill(POWER_STRIKE).skill(FLARE)
                        .skill(VENOM).build());
    }
}
```

Y el programa temporal de la parte 2:

app/Check14b.java

```java
package emberdeep.app;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.engine.world.wave.WaveNode.EnemyGroup;
import emberdeep.engine.world.wave.WaveNode.Wave;
import java.util.List;
import java.util.Random;
import java.util.stream.Collectors;

public class Check14b {
    public static void main(String[] args) {
        Wave wave = new Wave(List.of(
                new EnemyGroup(EnemyKind.SKELETON, 2),
                new Wave(List.of(
                        new EnemyGroup(EnemyKind.ASH_RAT, 1),
                        new EnemyGroup(EnemyKind.ZOMBIE, 1)))));
        System.out.println("Tamaño: " + wave.size());
        System.out.println("Enemigos: " + wave.enemies());

        Random random = new Random(42);
        List<Hero> heroes = List.of(new HeroBuilder().name("Aria")
                .stats(new Stats(30, 8, 3)).build());
        GameSession session = new GameSession(heroes,
                new CryptBiome(random), wave.enemies(),
                new FlatFormula(), new DamagePipeline(random, 0.0));
        System.out.println("En la sesión: " + session.enemies().stream()
                .map(Fighter::name)
                .collect(Collectors.joining(", ")));

        try {
            new EnemyGroup(EnemyKind.ZOMBIE, 0);
        } catch (IllegalArgumentException e) {
            System.out.println("Grupo vacío: " + e.getMessage());
        }
    }
}
```

Salida de Check14b

```
Tamaño: 4
Enemigos: [SKELETON, SKELETON, ASH_RAT, ZOMBIE]
En la sesión: Esqueleto, Esqueleto, Rata de ceniza, Zombi
Grupo vacío: Un grupo necesita al menos un enemigo
```

Por qué funciona (parte 2). `EnemyGroup` es la hoja: su tamaño es su cantidad y sus

enemigos, `count` copias de su `kind` (`Collections.nCopies`). `Wave` es el compuesto: suma los tamaños y *concatena* las listas de sus partes con `flatMap`, sin saber si cada parte es un grupo o una subola. $2+(1+1)=4$, y el orden de la lista es el del recorrido: los dos esqueletos, luego la rata y el zombi. La planta 3 de `Main` produce `[SKELETON, NECROMANCER]`, igual que antes de introducir los grupos, así que su partida no cambia; lo que cambia es que ahora se puede describir con subolas sin tocar `Floor`, que sigue recibiendo una `List<EnemyKind>`. El constructor compacto de `EnemyGroup` rechaza los grupos vacíos: un grupo de 0 enemigos no describe nada.

Parte 3: el Snapshot. Al ejecutar `./gradlew test` por primera vez, el compilador rechaza `SnapshotTest`: ya no existe `Inventory`. Esa es la señal de que el cambio de tipo del `Snapshot` (que ya hiciste en la parte 1 para que el proyecto compilara) llega también a las pruebas. La prueba actualizada y la nueva:

src/test/java/emberdeep/engine/save/SnapshotTest.java (1/2)

```
package emberdeep.engine.save;

import static org.junit.jupiter.api.Assertions.assertEquals;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.combat.status.Status.Poisoned;
import emberdeep.engine.combat.status.Status.Stunned;
import emberdeep.engine.item.InventoryNode.Bag;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.biome.CryptBiome;
import java.util.List;
import java.util.Random;
import org.junit.jupiter.api.Test;

class SnapshotTest {

    private static GameSession newSession() {
        Random random = new Random(42);
        List<Hero> heroes = List.of(
                new HeroBuilder().name("Aria")
                        .stats(new Stats(30, 8, 3)).build());
        return new GameSession(heroes, new CryptBiome(random),
                List.of(EnemyKind.SKELETON),
                new FlatFormula(), new DamagePipeline(random, 0.0));
    }
```

src/test/java/emberdeep/engine/save/SnapshotTest.java (2/2)

```
    @Test
    void restoreBringsBackSavedState() {
        GameSession session = newSession();
        Fighter aria = session.heroes().get(0);
        aria.addShield(5);
        Snapshot saved = session.createSnapshot();

        aria.takeDamage(10);
        aria.absorbShield(5);
        session.replaceInventory(new Bag("mochila", List.of()));
        session.restore(saved);

        assertEquals(30, aria.hp());
        assertEquals(5, aria.shield());
        assertEquals(3, session.inventory().countItems());
    }

    // cap. 12: el estado también forma parte del memento
    @Test
    void restoreBringsBackStatus() {
        GameSession session = newSession();
        Fighter aria = session.heroes().get(0);
        aria.setStatus(new Poisoned(3, 2));
        Snapshot saved = session.createSnapshot();

        aria.setStatus(new Stunned(1));
        session.restore(saved);

        assertEquals(new Poisoned(3, 2), aria.status());
    }

    // cap. 13: la mochila es inmutable, así que la instantánea no cambia
    @Test
    void savedBagIsNotChangedByLaterUse() {
        GameSession session = newSession();
        Snapshot saved = session.createSnapshot();

        session.replaceInventory(
                session.inventory().without("Poción de cripta"));

        assertEquals(2, session.inventory().countItems());
        assertEquals(3, saved.inventory().countItems());
    }
}
```

Por qué funciona (parte 3). La primera prueba vacía la mochila con una `Bag` nueva, restaura y comprueba que vuelven los 3 objetos: es lo que protege la línea `inventory = snapshot.inventory();` de `restore`, y el compilador no avisa de olvidarla. La tercera prueba fija lo que hace seguro guardar la bolsa por referencia: la instantánea tiene la `Bag` de antes de gastar la poción, y `without` devuelve una nueva sin tocarla, así que la instantánea sigue con 3 objetos mientras la sesión tiene 2. Con una lista mutable dentro del `Snapshot`, esa prueba fallaría con la misma copia superficial del capítulo 8. Al comentar la línea de `restore`, la primera prueba falla con `expected: <3> but was: <0>`.

Errores típicos:

- Dejar un `default` en el `switch` sobre `InventoryNode`: un nodo nuevo se trataría en silencio como el caso por omisión.
- Escribir en `Bag` un `add` que modifica la lista (con `ArrayList` dentro del record): la mochila guardada en el `Snapshot` cambiaría con ella.
- En `without`, seguir buscando después de quitar el primer objeto: se borrarían todas las pociones iguales y el `undo` repondría solo la mochila guardada.
- En `UseItem`, sustituir la mochila antes de guardar `bagBefore`: el `undo` repondría la mochila nueva y no la de antes.
- Comparar mochilas con `==` para saber si son «iguales»: `==` dice si es el mismo objeto; los records tienen `equals` por valor.
- Usar `get()` sobre el `Optional` de `find` sin comprobar: sin objeto, lanza `NoSuchElementException` en vez del mensaje de `UseItem`.
- Olvidar borrar `Inventory.java`: compilaría, pero quedaría una pieza que nadie usa en el árbol.

Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo 14: `engine/item/` con `Item`, `ItemKind` e `InventoryNode` (sin `Inventory`); `engine/world/wave/` con `WaveNode`; `GameSession` con una `Bag` raíz «mochila» (dos pociones y una bolsa de cinturón con la bomba); `UseItem` con `find` y `without`; `Snapshot` con una `Bag`; `SnapshotTest` con tres pruebas; y `Main` con la planta 3 descrita por una oleada. Borra `app/Check14a.java` y `app/Check14b.java`.

CAPÍTULO 14

Facade y el guardado en fichero: cerrar el juego

Objetivos del capítulo

- Reconocer el olor que pide Facade: un cliente que conoce, monta y ordena una docena de clases de un subsistema.
- Escribir una fachada (`GameEngine`) que construye y coordina el motor y deja que `Main` y la interfaz hablen solo con ella.
- Distinguir Facade de Mediator y de una capa de la arquitectura.
- Guardar y cargar una partida en un fichero de texto de formato fijo `clave=valor` con `java.nio.file`, reutilizando el Memento.
- Elegir y defender un patrón con tres preguntas: ¿cuál es el olor?, ¿cuál es la alternativa más simple?, ¿qué coste añade?
- Dar el hito H15b: el motor cerrado tras una fachada, con guardado y carga en `saves/`.

Emberdeep ya tiene todas sus piezas: estrategias, fábricas, decoradores, comandos, observadores, estados, composites. Cada una resolvió un problema concreto, y el resultado es un motor con muchas clases. Quien quiera usarlo (hoy `Main` y `ConsoleMenu`) tiene que saber cuáles hay, cuáles se crean primero y a quién se le avisa de qué. Este capítulo cierra el juego en tres pasos: una puerta única al motor (Facade), un guardado en fichero que reutiliza el Memento del capítulo 8 y, para terminar, una forma breve de decidir si el siguiente patrón que se te ocurra merece existir.

14.1 El olor que pide Facade: `Main` conoce doce clases del motor

Mira cómo arranca hoy una partida. Esto es, casi literal, lo que hace `Main.play` desde el capítulo 12 (las variables `formula`, `floor`, `random` y `output` salen de las líneas anteriores):

`Main.play` hasta el capítulo 13: montar y arrancar la partida

```
GameSession session = new GameSession(heroes(), floor.biome(),
        floor.enemies(), formula, new DamagePipeline(random, 0.0));
GameMap entrance = new MapParser(new TileTypes())
        .parse(floor.map());
session.setLocation(floorNumber,
        entrance.partyX(), entrance.partyY());
GameFlow flow = new GameFlow(session.heroes(), config.floors());
flow.start(floorNumber);
flow.enterCombat();
EventBus events = session.events();
events.subscribe(new LogView(output));
events.subscribe(new HudView(output));
```

```
events.subscribe(flow);
events.subscribe(new AchievementTracker(events));   // el último
menu.run(session);
```

Y `ConsoleMenu`, que solo debería preguntar al jugador qué quiere hacer, también conoce el motor por dentro:

`ConsoleMenu hasta el capítulo 13: crear y lanzar comandos`

```
Combat combat = session.combat();
CombatCommand command = switch (key) {
    case "1" -> new Attack(combat, hero,
            askTarget("Objetivo", session.enemies()));
    case "2" -> new Defend(combat, hero);
    case "3" -> chooseItem(session, hero);     // new UseItem(session, ...)
    case "4" -> chooseSkill(session, hero);    // new UseSkill(combat, ...)
    default -> null;
};
combat.perform(command);
```

Cuenta las clases del motor que esas dos piezas tienen que nombrar solo para jugar una partida: `GameSession`, `DamagePipeline`, `GameMap`, `GameFlow`, `EventBus`, `AchievementTracker`, `Combat`, `SnapshotStore`, `Attack`, `Defend`, `UseItem` y `UseSkill`. Son **doce**. Eso tiene cuatro problemas:

1. **El orden está en el cliente.** El `AchievementTracker` va el último para que el logro salga detrás del golpe que lo provoca; el `GameFlow` se crea después de la sesión y se suscribe antes de que haya combate. Nada de eso es asunto de quien juega, y nada lo recuerda si lo olvidas.
2. **Cada cliente repite el montaje.** Guardar y cargar necesita crear sesiones nuevas. Si ese código vive en `Main`, hay dos montajes que mantener iguales.
3. **Un cambio en el motor llega a la interfaz.** Si `GameSession` cambia su constructor o `UseSkill` recibe otro parámetro, hay que tocar `ConsoleMenu`, que no ha cambiado de comportamiento.
4. **La regla de capas es solo una regla.** Desde el capítulo 9 el motor no importa la interfaz, pero la interfaz importa medio motor. No hay ningún sitio donde se vea qué parte del motor es «la que se usa desde fuera».

Este es el olor: **un subsistema con muchas clases cuyo montaje, orden y detalles conocen sus clientes**. Facade lo cura poniendo *una clase* delante. Según el catálogo, su intención es dar una interfaz unificada a un conjunto de interfaces de un subsistema y definir una interfaz de más alto nivel que lo haga más fácil de usar [9, 24]. Head First lo cuenta junto a Adapter, con la imagen de un cine en casa: un solo botón en lugar de encender cada aparato en el orden correcto [7].

14.2 Piezas del patrón Facade

Facade es de los patrones con menos piezas: la fachada, el subsistema que oculta y los clientes que la usan. En Emberdeep, con el guardado como parte del subsistema:

Rol	Clase en el ejemplo	Propósito	Paquete
Fachada (clase)	`GameEngine`	Un único punto de entrada al motor	`engine`
Subsistema (clases)	`GameSession`, `Combat` (con `CombatArena` y `CommandHistory`), `EventBus`, `SnapshotStore`	Hacen el trabajo y no conocen la fachada	`engine.session`, `engine.combat`, `engine.event`, `engine.save`
Persistencia (clase)	`SaveFile`	Escribe y lee la partida en un fichero	`engine.save`
Cliente (clases)	`Main`, `ConsoleMenu` (la `ui`)	Piden cosas a la fachada, no al motor	`app`, `ui`

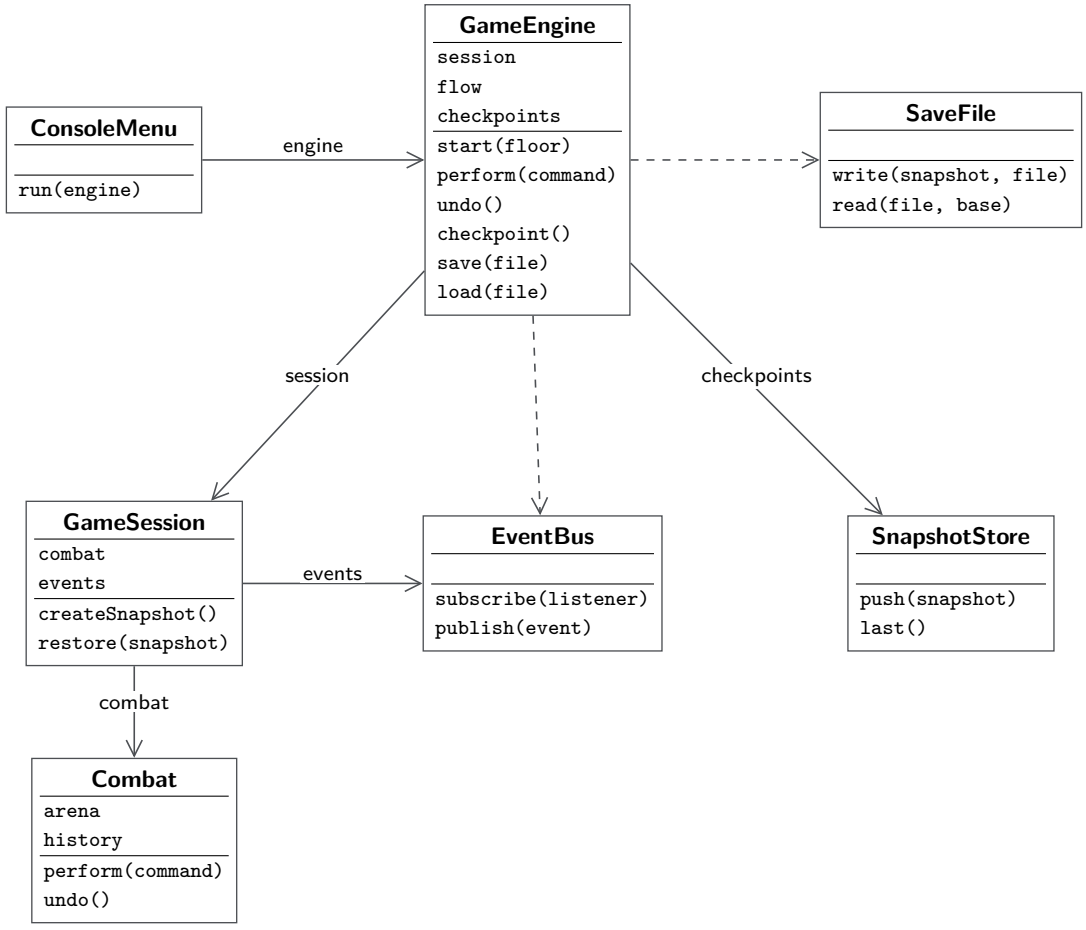


Diagrama UML del patrón Facade: `ConsoleMenu` solo apunta a `GameEngine`, y ninguna flecha del subsistema vuelve a la fachada. `CombatArena` y `CommandHistory` cuelgan de `Combat`.

Fíjate en el sentido de las flechas. Los clientes dependen de la fachada y la fachada de los subsistemas; *nunca al revés*. Esa asimetría es lo que distingue a Facade de otros patrones, y lo veremos con Mediator más abajo. Otra cosa que no dice el diagrama: la fachada no prohíbe nada. `GameSession` y `Combat` siguen siendo públicas, y las pruebas del capítulo 8 las usan sin pasar por `GameEngine`. La fachada es la puerta cómoda; no es una pared.

14.3 El ejemplo de Facade, pieza a pieza: `GameEngine`

Qué guarda. Una partida son tres cosas que nacen y mueren juntas: la sesión, la máquina de fases (`GameFlow`) y los puntos de control. Un record privado las junta, y la fachada guarda lo necesario para montar otra partida igual, que es lo que harán `load` y `descend`: la mazmorra, los héroes, la fórmula y el `Random`. Los datos son los que `Main` tenía a mano; `Main` los entrega una sola vez al construir el motor.

engine/GameEngine.java: datos y constructor

```
// La fachada del motor: la interfaz y Main hablan solo con esta clase.
public class GameEngine {
    // lo que hay que montar junto para una partida
    private record Game(GameSession session, GameFlow flow,
                        SnapshotStore checkpoints) {}

    private final Dungeon dungeon;
    private final List<Hero> heroes;
    private final DamageFormula formula;
    private final Random random;
    private final List<GameListener> listeners = new ArrayList<>();
    private Game game;

    public GameEngine(Dungeon dungeon, List<Hero> heroes,
                      DamageFormula formula, Random random) {
        this.dungeon = Objects.requireNonNull(dungeon);
        this.heroes = List.copyOf(heroes);
        this.formula = Objects.requireNonNull(formula);
        this.random = Objects.requireNonNull(random);
    }
```

Quién monta qué. El método privado `build` es el montaje de arriba, ahora en un solo sitio. Hace lo que `Main` hacía, y además lo que nadie recordaba: colocar el `AchievementTracker` al final de los observadores. Los observadores que trae la interfaz (`LogView`, `HudView`) los entrega `Main` con `subscribe` antes de empezar, y la fachada los apunta en cada partida nueva, delante de los del propio motor.

engine/GameEngine.java: subscribe, start y build

```
    // Los observadores de la interfaz se apuntan antes de empezar
    public void subscribe(GameListener listener) {
        if (game != null) {
            throw new IllegalStateException(
                    "Suscríbete antes de empezar la partida");
        }
        listeners.add(Objects.requireNonNull(listener));
    }

    // Una partida nueva en esa planta, en fase Exploring
    public void start(int floorNumber) {
        game = build(floorNumber);
    }

    private Game build(int floorNumber) {
        int last = dungeon.floors().size();
```

```
        if (floorNumber < 1 || floorNumber > last) {
            throw new IllegalArgumentException(
                    "La planta debe estar entre 1 y " + last);
        }
        Floor floor = dungeon.floors().get(floorNumber - 1);
        GameSession session = new GameSession(heroes, floor.biome(),
                floor.enemies(), formula, new DamagePipeline(random, 0.0));
        GameMap entrance = new MapParser(new TileTypes())
                .parse(floor.map());
        session.setLocation(floorNumber,
                entrance.partyX(), entrance.partyY());
        GameFlow flow = new GameFlow(session.heroes(), last);
        flow.start(floorNumber);
        EventBus events = session.events();
        listeners.forEach(events::subscribe);
        events.subscribe(flow);
        events.subscribe(new AchievementTracker(events));   // el último
        return new Game(session, flow, new SnapshotStore());
    }
```

Dos detalles de diseño. Primero, `build` no toca el campo `game`: devuelve una partida nueva y quien lo llama decide cuándo sustituir la anterior. Eso le dará a `load` una propiedad valiosa (si algo falla, la partida actual sigue intacta). Segundo, `start` deja la partida en la fase `Exploring`, la de «entre combates»; para pelear hay que pedir `enterCombat()`. Lo que antes hacía `Main` con `flow.start` y `flow.enterCombat` ahora son dos llamadas a la fachada.

Qué ve y qué pide la interfaz. Las consultas son delegaciones de una línea. Lo más útil está en los comandos. `ConsoleMenu` creaba `Attack`, `Defend`, `UseItem` y `UseSkill`, y para eso tenía que conocer `Combat`, el mediador y la sesión. Ahora los crea la fachada, que ya tiene todo eso a mano:

engine/GameEngine.java: los comandos

```
    // Lo que la interfaz puede pedir; los comandos se crean aquí para que
    // quien juega no necesite Combat, GameSession ni el mediador
    public CombatCommand attack(Fighter attacker, Fighter target) {
        return new Attack(session().combat(), attacker, target);
    }

    public CombatCommand defend(Fighter user) {
        return new Defend(session().combat(), user);
    }

    public CombatCommand useItem(Fighter user, Fighter target, Item item) {
        return new UseItem(session(), user, target, item);
    }

    public CombatCommand useSkill(Fighter user, Skill skill,
                                  Fighter target) {
        Combat combat = session().combat();
        return new UseSkill(combat, combat.mediator(), user, skill,
                target);
    }

    public void perform(CombatCommand command) {
        session().combat().perform(command);
```

```
24      }
```

Compara lo que cambia en `ConsoleMenu`. Antes, una línea del menú de acciones conocía tres clases; ahora no conoce ninguna del motor:

`ConsoleMenu: antes y después de la fachada`

```
// antes: el menú conoce Combat, Attack, Defend, UseItem y UseSkill
case "1" -> new Attack(combat, hero,
        askTarget("Objetivo", session.enemies()));
combat.perform(command);

// después: solo conoce GameEngine
case "1" -> engine.attack(hero,
        askTarget("Objetivo", engine.enemies()));
engine.perform(command);
```

La última pieza de esta sección son los puntos de control del capítulo 8. Antes los guardaba `ConsoleMenu` en su propio `SnapshotStore`; ahora los guarda la partida, detrás de dos métodos:

`engine/GameEngine.java: puntos de control`

```
    // Puntos de control en memoria (cap. 8); devuelve cuántos hay
    public int checkpoint() {
        current().checkpoints().push(session().createSnapshot());
        return current().checkpoints().size();
    }

    // false si todavía no hay ninguno
    public boolean restoreCheckpoint() {
        SnapshotStore checkpoints = current().checkpoints();
        if (checkpoints.isEmpty()) {
            return false;
        }
        session().restore(checkpoints.last());
        return true;
    }
```

`checkpoint()` devuelve cuántos hay para que el menú pueda escribir «Punto de control guardado (N)» sin conocer el almacén, y `restoreCheckpoint()` devuelve `false` si no hay ninguno. El texto que ve el jugador no cambia.

14.4 Si ya lo conoces: lo nuevo en Facade es frente a Mediator y a las capas

Si ya conocías Facade, lo nuevo aquí es:

- **No es una capa.** Una arquitectura por capas es una regla sobre paquetes («el motor no importa la interfaz») y no necesita ninguna clase para cumplirse. Facade es una clase concreta que hace barata esa regla desde fuera: es *la puerta* de la capa. Puedes tener capas sin fachada (más imports, más acoplamiento) y una fachada sin capas (un solo paquete con una clase de entrada).

- **Va en un solo sentido.** Los clientes llaman a la fachada y esta a los subsistemas; los subsistemas ni saben que existe. Mediator es lo contrario (sección 14.6).
- **Aquí la fachada coordina, no decide.** `GameEngine` monta el motor, vigila la fase («solo se guarda en `Exploring`») y sustituye partidas. No calcula daño ni decide turnos: eso sigue en el subsistema. Si una regla del juego aparece en la fachada, está en el sitio equivocado.
- **La fachada puede crear cosas por ti.** En los manuales, Facade solo delega. Aquí también fabrica los comandos de la interfaz (`engine.attack(...)`), porque es el único sitio que conoce el `Combat` y el mediador. Es una decisión de este diseño, no una obligación del patrón.
- **Esconde, no encierra.** El catálogo deja claro que los clientes pueden seguir usando las clases del subsistema si necesitan más control [9]. Las pruebas y los programas de comprobación de este curso lo hacen: usan `GameSession` directamente.

14.5 El Memento llega al disco: guardar y cargar con `clave=valor`

En el capítulo 8, el Memento guardaba el estado en memoria: un `Snapshot` con referencias a objetos vivos (la cadena de `StatSource`, el `Status`, la `Bag`). Eso no se puede escribir en un fichero tal cual: habría que convertir cada objeto en texto y volver a construirlo. Tienes dos caminos. El pesado es serializar todo el grafo; el ligero, decidir qué *poco* hace falta para reconstruir la partida y escribir solo eso. Emberdeep toma el ligero, con tres reglas de diseño:

(a) **Solo se guarda en la fase `Exploring`**, entre combates. Al salir de un combate, `endCombat()` (capítulo 12) ya ha limpiado estados, escudo y buffs, así que el fichero no necesita guardarlos.

(b) **El equipo permanente no se guarda.** Espada y armadura salen de `Main` y de `GameSession`. `load` crea una partida nueva con los mismos datos que `Main` entregó a la fachada, toma su `Snapshot` como *base* y `read` le sobrescribe lo que el fichero sí trae: planta, posición, vida de los héroes e inventario.

(c) **El inventario se aplana al guardar.** Un árbol de bolsas es difícil de escribir en líneas; la lista de objetos que da `items()` (capítulo 13) es fácil. Se pierde la organización en bolsas, y al cargar todo cae en la `Bag` raíz. Es un límite declarado, no un descuido.

El formato. Es texto UTF-8, una línea `clave=valor` por dato, y fijo: las claves y su orden no cambian. Así queda un fichero de ejemplo:

`saves/ejemplo.txt: formato de la versión 1`

```
version=1
floor=2
x=3
y=5
hero.0.hp=18
hero.1.hp=20
hero.2.hp=7
inventory.count=2
inventory.0=pocion|HEALING|20
inventory.1=bomba|BOMB|15
```

Clave	Ejemplo	Qué guarda
`version`	`version=1`	Versión del formato; `read` rechaza cualquier otra
`floor`	`floor=2`	Planta en la que estás
`x`, `y`	`x=3`, `y=5`	Casilla del grupo en el mapa de la planta
`hero.N.hp`	`hero.0.hp=18`	Vida del héroe N (0 es el primero)
`inventory.count`	`inventory.count=2`	Cuántos objetos hay
`inventory.N`	`inventory.0=pocion|HEALING|20`	Objeto N: nombre, tipo y poder, separados por `|`

Tres reglas del formato, para quien lo escribe y para quien lo lee: cada línea se divide en el **primer** `=` (lo que queda a la derecha es el valor, aunque tenga más signos `=`); las líneas en blanco se ignoran; y los nombres de objeto no pueden llevar `|` ni `=`, porque romperían el formato. Escribir un nombre así es un error, no algo que se arregla en silencio.

Lo que necesitas de `java.nio.file`

El capítulo 7 ya usó `Path` y `Files.readAllLines`. Aquí hacen falta tres cosas más, todas del JDK y sin nada que instalar; el comportamiento está comprobado en la documentación de `java.nio.file.Files` de Java 21:

- `Files.writeString(path, texto)`: escribe el texto en UTF-8. Si el fichero no existe lo crea, y si existe lo *sobrescribe* entero. Lanza `IOException` si falla.
- `Files.readAllLines(path)`: devuelve las líneas como `List<String>`, leyendo en UTF-8. Lanza `IOException` si no puede leer el fichero o si hay bytes que no son UTF-8 válido.
- `Files.createDirectories(dir)`: crea la carpeta y las que falten por encima, y *no* falla si ya existe. Sin ella, escribir en `saves/a.txt` falla la primera vez porque `saves/` no existe.
- `texto.split("=", 2)`: no es de `java.nio.file`, pero es el truco del formato. El segundo argumento es el máximo de trozos: con 2, divide solo en el primer `=` y deja el resto intacto. Sin él, `"a=b=c".split("=")` da tres trozos y pierde información.

Escribir. `SaveFile` es una clase de utilidad (constructor privado, métodos estáticos): no tiene estado. `write` recibe una instantánea y escribe sus combatientes como héroes; `GameEngine.save` le entrega una instantánea *recortada* a los héroes, para que el fichero no lleve a los enemigos de la planta.

`engine/save/SaveFile.java: escribir`

```
    // Escribe la instantánea: sus combatientes se guardan como héroes
    public static void write(Snapshot snapshot, Path path)
            throws IOException {
        StringBuilder text = new StringBuilder();
        put(text, "version", VERSION);
        put(text, "floor", snapshot.floor());
        put(text, "x", snapshot.x());
        put(text, "y", snapshot.y());
        List<FighterState> heroes = snapshot.fighters();
        for (int i = 0; i < heroes.size(); i++) {
```

```
            put(text, "hero." + i + ".hp", heroes.get(i).hp());
        }
        List<Item> items = snapshot.inventory().items();
        put(text, "inventory.count", items.size());
        for (int i = 0; i < items.size(); i++) {
            Item item = items.get(i);
            put(text, "inventory." + i, safe(item.name()) + "|"
                    + item.kind() + "|" + item.power());
        }
        Files.createDirectories(path.toAbsolutePath().getParent());
        Files.writeString(path, text.toString());
    }

    private static void put(StringBuilder text, String key, Object value) {
        text.append(key).append('=').append(value).append('\n');
    }

    // el formato separa campos con | y clave de valor con =
    private static String safe(String name) {
        if (name.contains("|") || name.contains("=")
                || name.contains("\n")) {
            throw new IllegalArgumentException(
                    "Nombre no válido para el fichero: " + name);
        }
        return name;
    }
```

Una línea concentra el formato: `put` escribe `clave=valor` y un salto de línea. La de los objetos añade el nombre, el tipo (`item.kind()` se convierte en `HEALING` o `BOMB`) y el poder, separados por `|`. `safe` comprueba el nombre antes de escribirlo.

Leer. La lectura parte de una *base*: una partida nueva de la planta que dice el fichero. `read` no construye la partida entera; toma la base y le cambia lo que el fichero trae. Lo que no está en el fichero (escudo, estado, equipo) se queda como lo dejó la base:

engine/save/SaveFile.java: leer

```
    // Parte de la base (una partida nueva de esa planta) y le
    // sobrescribe lo que trae el fichero
    public static Snapshot read(Path path, Snapshot base)
            throws IOException {
        Map<String, String> values = parse(path);
        List<FighterState> fighters = new ArrayList<>(base.fighters());
        for (int i = 0; values.containsKey("hero." + i + ".hp"); i++) {
            if (i >= fighters.size()) {
                throw new IllegalArgumentException(
                        "El fichero tiene más héroes que la partida");
            }
            FighterState old = fighters.get(i);
            int hp = number(values, "hero." + i + ".hp");
            if (hp < 0 || hp > old.stats().maxHp()) {
                throw new IllegalArgumentException(
                        "Vida fuera de rango en hero." + i + ".hp: " + hp);
            }
            fighters.set(i, new FighterState(hp, old.shield(),
                    old.stats(), old.status()));
```

```
        }
        List<InventoryNode> slots = new ArrayList<>();
        int count = number(values, "inventory.count");
        for (int i = 0; i < count; i++) {
            String text = required(values, "inventory." + i);
            slots.add(new ItemSlot(item(text)));
        }
        return new Snapshot(number(values, "floor"), number(values, "x"),
                number(values, "y"), base.round(), fighters,
                new Bag("mochila", slots));
    }
```

`read` valida: la vida de un héroe debe estar entre 0 y su máximo, no puede haber más héroes que combatientes, y faltar una clave o poner texto donde va un número lanza una excepción con un mensaje. Cuando el fichero es de otra partida, mejor fallar que cargar algo a medias. El troceado en claves y valores vive en un método aparte:

engine/save/SaveFile.java: dividir en claves y valores

```
    // clave=valor: se divide en el PRIMER '='; las líneas en blanco
    // se saltan
    private static Map<String, String> parse(Path path)
            throws IOException {
        Map<String, String> values = new LinkedHashMap<>();
        for (String line : Files.readAllLines(path)) {
            if (line.isBlank()) {
                continue;
            }
            String[] parts = line.split("=", 2);
            if (parts.length < 2) {
                throw new IllegalArgumentException(
                        "Línea sin '=': " + line);
            }
            values.put(parts[0], parts[1]);
        }
        if (!VERSION.equals(values.get("version"))) {
            throw new IllegalArgumentException(
                    "Versión no soportada: " + values.get("version"));
        }
        return values;
    }
```

Fíjate en `split("=", 2)`, en el `isBlank()` que salta las líneas vacías y en la comprobación de la versión: si un día cambia el formato, el número de versión evita que una versión vieja lea un fichero nuevo y obtenga datos erróneos sin avisar. El resto de `SaveFile` (los pequeños métodos `required`, `number` e `item`) está en la solución del hito.

Guardar y cargar desde la fachada. Aquí es donde Facade y Memento se encuentran. `save` comprueba la regla (a), recorta la instantánea a los héroes y llama a `SaveFile`. `load` necesita saber primero la *planta*, porque la base tiene que ser una partida de esa planta (con sus enemigos): por eso `SaveFile` ofrece `readFloor`. Después sigue el mismo patrón que `build`: monta una partida nueva, le aplica el fichero, y solo si todo ha ido bien la pone en lugar de la actual.

engine/GameEngine.java: save y load

```
    // Guardar en fichero: solo entre combates (fase Exploring)
    public void save(Path file) throws IOException {
        GamePhase phase = phase();
        if (!(phase instanceof Exploring)) {
            throw new IllegalStateException("Solo se guarda entre "
                    + "combates, no en la fase " + phase);
        }
        Snapshot all = session().createSnapshot();
        // el fichero lleva solo a los héroes, no a los enemigos
        Snapshot saved = new Snapshot(all.floor(), all.x(), all.y(),
                all.round(), all.fighters().subList(0, heroes.size()),
                all.inventory());
        SaveFile.write(saved, file);
    }

    // Una partida NUEVA en la planta guardada con lo guardado encima.
    // Si algo falla, la partida actual queda como estaba.
    public void load(Path file) throws IOException {
        Game loaded = build(SaveFile.readFloor(file));
        GameSession session = loaded.session();
        session.restore(SaveFile.read(file, session.createSnapshot()));
        game = loaded;
    }
```

Queda `descend()`, que no estaba en la lista inicial de la fachada pero sale gratis del mismo mecanismo: para bajar de planta, hay que montar una partida de la planta siguiente y *llevarle* la vida de los héroes y la mochila. Es exactamente lo que hace `load`, con los datos de la partida que acaba en vez de los del fichero. La utilidad de `descend` se ve enseguida: tras cargar una partida guardada al ganar la planta 1, hay que seguir por la planta 2.

engine/GameEngine.java: descend

```
    // Termina la planta y pasa a la siguiente con la vida y la mochila
    // de ahora; tras la última planta, la fase es GameOver(true)
    public void descend() {
        Snapshot finished = session().createSnapshot();
        current().flow().descend();
        if (phase() instanceof Exploring next) {
            Game following = build(next.floor());
            GameSession session = following.session();
            session.restore(carryOver(finished, session.createSnapshot()));
            game = following;
        }
    }

    // la base nueva, pero con la vida de los héroes y la mochila de antes
    private Snapshot carryOver(Snapshot finished, Snapshot fresh) {
        List<FighterState> states = new ArrayList<>(fresh.fighters());
        for (int i = 0; i < heroes.size(); i++) {
            FighterState old = states.get(i);
            states.set(i, new FighterState(finished.fighters().get(i).hp(),
                    old.shield(), old.stats(), old.status()));
        }
        return new Snapshot(fresh.floor(), fresh.x(), fresh.y(),
```

```
23                  fresh.round(), states, finished.inventory());
24      }
```

Límites que esta versión no resuelve (se declaran, no se ocultan):

1. **El inventario pierde sus bolsas.** Al guardar se aplana; al cargar todo cae en la `Bag` raíz «mochila». Guardar la estructura (con líneas como `bag.0.name`) es una ampliación opcional.
2. **No se guarda el equipo, el `Random` ni los puntos de control.** Tras cargar, el equipo vuelve a ser el de `GameSession` y los números aleatorios siguen otra secuencia.
3. **Solo se guarda entre combates.** Guardar a mitad de combate exigiría escribir escudo, estados, buffs y enemigos; la regla (a) es lo que evita esa complejidad.
4. **La escritura no es atómica.** Si el programa muere mientras escribe, el fichero queda cortado. Escribir en un fichero temporal y renombrarlo lo evita, y no se hace aquí.
5. **Una sola versión.** `version=1` es la única que se lee; no hay migración de formatos antiguos.

14.6 Facade: cuándo sí y cuándo no

Compensa cuando un subsistema tiene muchas piezas y varios clientes repiten su montaje; cuando quieres que una capa tenga una puerta clara (para leer, para probar y para cambiar por dentro sin avisar a nadie); y cuando una parte de los clientes solo necesita las operaciones comunes y no el control fino.

No compensa si el subsistema es pequeño: una fachada sobre dos o tres clases suele ser una capa de delegación que solo añade nombres. Tampoco si los clientes necesitan casi todo lo que el subsistema ofrece: acabas copiando su interfaz uno a uno.

Costes reales:

- **La tentación del «objeto dios».** Cada función nueva se queda en la fachada porque «ya está ahí». `GameEngine` tiene ya veintidós métodos públicos. Si empieza a contener reglas del juego, hay que sacarlas al subsistema.
- **Un paso más en cada cambio.** Añadir un tipo de comando exige un método en la fachada además de la clase del comando. Es el precio de que la interfaz no conozca el motor.
- **Esconde, pero no impide.** Mientras las clases del subsistema sean públicas, alguien puede saltarse la puerta. Es útil para pruebas, y peligroso si pasa en la interfaz sin que nadie lo vea; ayuda una regla de equipo («la `ui` solo importa `GameEngine`») que se revise en cada pull request.
- **Pruebas de integración, no unitarias.** Probar la fachada es probar el motor montado. Está bien, y conviene no sustituir con ello las pruebas de cada pieza.

Facade frente a Mediator (capítulo 11). Los dos tienen una clase en medio de muchas, y se confunden. El catálogo los separa por el sentido de la comunicación y por quién conoce a quién [9]:

	Facade	**Mediator**
Qué resuelve	Simplificar el acceso a un subsistema desde fuera	Que unos objetos se coordinen sin conocerse
Sentido	Un solo sentido: clientes → fachada → subsistema	Dos sentidos: los colegas llaman al mediador y este a los colegas
Quién la conoce	Los clientes; el subsistema no sabe que existe	Los colegas la reciben y la usan
Sin ella	El subsistema funciona igual, solo más incómodo	Los colegas no pueden interactuar
En Emberdeep	`GameEngine`: lo usan `Main` y `ConsoleMenu`	`CombatArena`: la usan `Fighter` y `UseSkill`

La prueba rápida: pregúntate si el subsistema *necesita* esa clase para funcionar. Si no, es una fachada. Si sí, y sus piezas la llaman para hablar entre ellas, es un mediador. `GameSession` funciona sin `GameEngine` (las pruebas lo demuestran); `Fighter` no puede provocar ni invocar sin `CombatArena`.

14.7 Elegir y defender el patrón

Con Facade ya conoces los diecisiete patrones del libro. El error más frecuente después de aprender patrones no es ignorarlos, sino usarlos donde nada duele. El catálogo lo anticipa: cada patrón viene con su *aplicabilidad* y sus *consecuencias*, es decir, cuándo usarlo y qué cuesta [9]. Y la forma más segura de llegar a uno es desde código que ya duele, refactorizando, no desde el patrón hacia el código [17]. Hay opiniones recientes que añaden que lambdas, records y tipos sellados han sustituido a varios patrones clásicos y que el valor de muchos es hoy sobre todo el vocabulario [16]. Todo eso cabe en tres preguntas que te haces *antes* de escribir la primera interfaz:

1. **¿Cuál es el olor?** Señala en el código de hoy (no en el de mañana) qué duele: un `if` repetido por tipo, una clase por cada combinación, un constructor de ocho parámetros, un cliente que conoce una docena de clases. Si no puedes poner el dedo en una línea, no hay olor y no hay patrón.
2. **¿Cuál es la alternativa más simple?** Escríbela de verdad: un `if`, un `switch`, una lambda, un record, un método. Si la alternativa duele menos que el patrón o lo mismo, gana la alternativa.
3. **¿Qué coste añade?** Clases nuevas, indirección, sitios que hay que tocar cuando crece, lo que tiene que entender el equipo, las pruebas que hacen falta. Y quién lo paga: no es lo mismo para un módulo de una persona que para una biblioteca pública.

Defender un patrón es contestar esas tres preguntas en voz alta, en una revisión de código, con tres líneas: «el olor es X (línea tal); la alternativa era Y y no basta porque Z; el coste es W y lo aceptamos porque V». Si una respuesta no sale, el patrón no está defendido. Así quedan cinco decisiones de este libro, incluida una en la que la respuesta fue *no usar el patrón*:

Decisión	Olor	Alternativa más simple	Coste que se acepta
Strategy para la IA y la fórmula (cap. 2)	Decisiones del combate escritas con `if` por nombre de enemigo	Seguir con los `if`	Una interfaz y varias clases (o lambdas)
`GameConfig` en lugar de Singleton (cap. 4)	Varias clases leen la misma configuración	Un Singleton global	Pasar el record por constructor
State para `Status` (cap. 12)	Banderas que permiten combinaciones imposibles	Banderas booleanas con cuidado	Un tipo sellado y un `switch` por operación
`switch` sobre sellada en lugar de Visitor (cap. 13)	Operaciones recursivas sobre el árbol del inventario	Un método en cada nodo	Cada tipo nuevo obliga a revisar cada `switch`
Facade (este capítulo)	`Main` y `ConsoleMenu` conocen doce clases del motor	Que cada cliente siga montando lo suyo	Una clase que puede crecer y un método por comando

Dos señales de sobrediseño que merece la pena aprenderse. La primera: **una sola implementación y ninguna a la vista**. Una interfaz con una sola clase y sin segunda prevista no abstrae nada; solo añade un fichero. La segunda: **una clase que solo reenvía**. Si cada método llama a otro sin añadir nada, la clase es ruido. Y una advertencia en sentido contrario: no usar un patrón porque «es mucho» también puede ser un error si el olor existe y cuesta cada semana. La checklist larga «del olor al patrón», con la revisión de código y la mirada atrás a dónde vive cada uno de los diecisiete patrones en Emberdeep, está en el apéndice A. Lo practicas ahora en los ejercicios 14.3 y 14.4.

14.8 En el proyecto: H15b con Facade

Este capítulo avanza el hito H15b, el último. Cambios del hito:

- **Nuevos archivos:** `engine/GameEngine.java` (la fachada) y `engine/save/SaveFile.java`. La carpeta `saves/` aparece sola la primera vez que se guarda; si usas git, añade `/saves/` a tu `.gitignore`.
- **Cambia `ConsoleMenu`:** `run` recibe un `GameEngine` en lugar de un `GameSession`, y ya no importa `GameSession`, `Combat`, `SnapshotStore`, `Attack`, `Defend`, `UseItem` ni `UseSkill`.
- **Cambia `Main`:** construye un `GameEngine`, le entrega las vistas y añade las opciones `--save=ruta` y `--load=ruta`. Ya no importa `GameSession`, `DamagePipeline`, `GameFlow`, `EventBus`, `AchievementTracker` ni `GameMap`.
- **No cambia** el resto del motor, ni los puntos de control (`c` y `r`), ni `SnapshotTest`.

Qué cambia en la partida. Sin las opciones nuevas, nada: las mismas rondas, las mismas frases y la misma salida del guion de demostración. Con `--save=saves/partida.txt`, al ganar el combate la partida se guarda en ese fichero (el combate deja la fase en `Exploring`) y se imprime «Partida guardada en saves/partida.txt.». Con `--load=saves/partida.txt` se carga ese fichero y se sigue por la *planta siguiente* con la vida y los objetos guardados. Una pequeña cosa más: el argumento que acaba en `.txt` es el guion de teclas, así que `scriptOf` ahora ignora los que empiezan por `--`; si no, `--save=saves/partida.txt` se tomaría por un guion.

Límites que esta versión no resuelve (se declaran, no se ocultan):

1. **Se guarda solo al ganar.** El menú de combate no ofrece guardar; la regla (a) de la sección 14.5 lo impide. Las opciones de línea de comandos son la forma más simple de usar el guardado sin construir un menú nuevo.
2. **Cada partida cargada sigue por la planta siguiente,** no por la misma. Es lo que tiene sentido si el fichero se escribió al ganar una planta.
3. **Los demás límites de la sección 14.5** (inventario aplanado, sin equipo ni `Random`, escritura no atómica, una sola versión).

Los tres pasos. Cada uno termina con un resultado que puedes comprobar:

- **Paso 1, guardar y la fachada.** `SaveFile` y `GameEngine`. Resultado: el proyecto compila con `./gradlew -q classes` y dos programas temporales (`Check15a`: guardar, abrir otro motor y cargar; `Check15b`: bajar de planta) imprimen lo esperado.
- **Paso 2, la interfaz y `Main` hablan con la fachada.** `ConsoleMenu` y `Main`. Resultado: la salida del guion de demostración es idéntica a la de antes, y guardar con `--save` y seguir con `--load` funciona.
- **Paso 3, limpieza.** Borrar los programas temporales y el fichero de prueba; comprobar que `./gradlew test` sigue pasando. Resultado: el árbol de abajo.

Hito 15b: la fachada del motor y el guardado en fichero

Tras este hito, el juego se maneja a través de `GameEngine`, y una partida ganada se puede guardar y continuar. La estructura final de Emberdeep:

Estructura final de Emberdeep (1/2)

```
emberdeep/
|-- settings.gradle
|-- build.gradle                   (con JUnit 6.1.3)
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
|-- scripts/
|   `-- demo.txt
|-- saves/                         (partidas; fuera de git)
`-- src/
    |-- main/java/emberdeep/
    |   |-- app/
    |   |   |-- Main.java              (composition root)
    |   |   `-- GameConfig.java
    |   |-- engine/
    |   |   |-- GameEngine.java        (la fachada)
    |   |   |-- character/             Stats, Hero, HeroBuilder, Enemy,
    |   |   |                          EnemyKind, Fighter, Skill,
    |   |   |                          EnemyTemplate, TemplateRegistry
    |   |   |-- stats/                 StatSource, StatKind, BaseStats,
    |   |   |                          StatDecorator, WeaponBonus,
    |   |   |                          ArmorBonus, TempBuff
    |   |   |-- combat/
    |   |   |   |-- Combat.java
    |   |   |   |-- ai/                EnemyAi, FirstAliveAi, LowestHpAi,
    |   |   |   |                      RandomTargetAi, SkillfulAi
    |   |   |   |-- arena/             TurnMediator, CombatArena
    |   |   |   |-- command/           CombatCommand, Attack, Defend,
    |   |   |   |                      UseItem, UseSkill, CommandHistory
    |   |   |   |-- damage/            DamageFormula, FlatFormula,
    |   |   |   |                      ScaledFormula, Damage, DamageType
    |   |   |   |-- defense/           DamageHandler, DamagePipeline,
    |   |   |   |                      DodgeHandler, ShieldHandler,
    |   |   |   |                      ArmorHandler, ResistanceHandler
    |   |   |   `-- status/            Status
```

Estructura final de Emberdeep (2/2)

```
    |   |   |-- item/                  Item, ItemKind, InventoryNode
    |   |   |-- event/                 GameEvent, GameListener, EventBus
    |   |   |-- flow/                  GamePhase, GameFlow
    |   |   |-- achievement/           AchievementTracker
    |   |   |-- save/                  Snapshot, FighterState,
    |   |   |                          SnapshotStore, SaveFile
    |   |   |-- session/               GameSession
    |   |   `-- world/
    |   |       |-- Dungeon.java, Floor.java,
    |   |       |   DungeonBuilder.java
    |   |       |-- biome/             BiomeFactory, CryptBiome,
    |   |       |                      CavernBiome
    |   |       |-- map/               TileType, TileTypes, GameMap,
    |   |       |                      MapParser, MapData
    |   |       `-- wave/              WaveNode
    |   `-- ui/
    |       |-- ConsoleMenu.java
    |       |-- input/                 InputSource, ConsoleInput,
    |       |                          ScriptInput
    |       |-- view/                  View, MapView, HudView, LogView
    |       `-- output/                Output, Style, AnsiOutput,
    |                                  PlainOutput, BufferOutput
    `-- test/java/emberdeep/
        `-- engine/save/               SnapshotTest
```

Lo practicas en el ejercicio 14.5. Antes, cuatro ejercicios más pequeños.

14.9 Ejercicios

Ejercicio 14.1: Una fachada para un reproductor de audio ★★★☆☆

No es de Emberdeep. Crea una carpeta vacía y, dentro, archivos sin paquete. Tres clases ya escritas simulan un reproductor de audio por dentro (imprimen lo que hacen); tú escribes la fachada. Guárdalas tal cual:

Decoder.java

```
public class Decoder {
    public int[] decode(String path) {
        System.out.println("Decoder: leyendo " + path);
        return new int[] {10, 20, 30};
    }
}
```

Mixer.java

```java
public class Mixer {
    public int[] mix(int[] samples, int volume) {
        System.out.println("Mixer: volumen " + volume);
        int[] mixed = new int[samples.length];
        for (int i = 0; i < samples.length; i++) {
            mixed[i] = samples[i] * volume / 100;
        }
        return mixed;
    }
}
```

AudioOutput.java

```java
public class AudioOutput {
    public void open() {
        System.out.println("Salida: abierta");
    }

    public void write(int[] samples) {
        System.out.println("Salida: " + samples.length
                + " muestras, primera " + samples[0]);
    }

    public void close() {
        System.out.println("Salida: cerrada");
    }
}
```

1. `Player`: la fachada. Crea ella misma un `Decoder`, un `Mixer` y un `AudioOutput`, y ofrece dos métodos públicos, `play(String path)` y `stop()`.
2. `play(path)`: si ya hay algo sonando, hace antes `stop()`. Después abre la salida, decodifica el archivo, mezcla con volumen 50 y escribe las muestras. Anota que algo suena.
3. `stop()`: si suena algo, cierra la salida y anota que ya no suena; si no, no hace nada.
4. `Main`: usa solo `Player`. Llama a `play("tema.mp3")`, a `play("otro.mp3")` y dos veces a `stop()`. Después, una línea que demuestra que la fachada no encierra los subsistemas: `new Decoder().decode("suelto.wav");`.

Compila con `javac *.java` y ejecuta con `java Main`. Está bien si la salida es exactamente:

Salida esperada

```
Salida: abierta
Decoder: leyendo tema.mp3
Mixer: volumen 50
Salida: 3 muestras, primera 5
Salida: cerrada
Salida: abierta
Decoder: leyendo otro.mp3
Mixer: volumen 50
Salida: 3 muestras, primera 5
Salida: cerrada
Decoder: leyendo suelto.wav
```

y si, salvo la última línea, `Main` no menciona `Decoder`, `Mixer` ni `AudioOutput`.

Pista: *Un campo* `boolean playing` *basta para saber si suena algo. El segundo* `stop()` *no imprime nada porque* `playing` *ya es* `false`*.*

Práctica ▪ solución en la pág. 669

Ejercicio 14.2: Un lector de clave=valor que se rompe ★★★☆☆

No es de Emberdeep. Este lector lee ficheros en el formato del capítulo (una línea `clave=valor`) y falla con dos entradas. Guarda los dos archivos sin paquete en una carpeta vacía:

ConfigReader.java

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.HashMap;
import java.util.Map;

public class ConfigReader {
    public static Map<String, String> read(Path file)
            throws IOException {
        Map<String, String> values = new HashMap<>();
        for (String line : Files.readAllLines(file)) {
            String[] parts = line.split("=");
            values.put(parts[0], parts[1]);
        }
        return values;
    }
}
```

Main.java

```java
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Map;

public class Main {
    public static void main(String[] args) throws IOException {
        Path file = Path.of("config.txt");
        Files.writeString(file, "name=Aria\nmotto=a=b\n\nlevel=3\n");
        Map<String, String> values = ConfigReader.read(file);
        System.out.println("name: " + values.get("name"));
        System.out.println("motto: " + values.get("motto"));
        System.out.println("level: " + values.get("level"));

        Files.writeString(file, "name=Aria\nsin igual\n");
        try {
            ConfigReader.read(file);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Haz esto, en orden:

1. Compila (`javac *.java`) y ejecuta (`java Main`). Apunta qué excepción sale, con qué mensaje, y qué línea del fichero la provoca.
2. Arregla *solo* ese fallo y ejecuta de nuevo. Ahora no hay ninguna excepción, pero una de las tres líneas impresas es incorrecta. Cuál es y por qué.
3. Arregla el segundo fallo.
4. Haz que una línea sin `=` lance una `IllegalArgumentException` con el mensaje `Línea sin '=': ` seguido de la línea.

Está bien si la salida final es exactamente:

Salida esperada

```
name: Aria
motto: a=b
level: 3
Error: Línea sin '=': sin igual
```

Pista: *Mira qué devuelven `"".split("=")` y `"motto=a=b".split("=")`, y cuántos elementos tiene cada array. `split` acepta un segundo argumento.*

Depuración ▪ solución en la pág. 670

Ejercicio 14.3: Revisar un pull request: cuatro cambios, cuatro veredictos ★★★★☆

No tienes que escribir código. Es una revisión de código de Emberdeep: alguien propone una tienda donde los héroes compran y venden objetos, en cuatro cambios independientes.

Lee los fragmentos (los nombres `Item`, `ItemKind`, `GameEvent` y `AchievementTracker` son los del proyecto):

Cambio A: crear los objetos de la tienda

```
public interface ShopFactory {
    Item potion();
    Item bomb();
}

public class DefaultShopFactory implements ShopFactory {
    public Item potion() {
        return new Item("Poción", ItemKind.HEALING, 10);
    }

    public Item bomb() {
        return new Item("Bomba", ItemKind.BOMB, 12);
    }
}
// Shop recibe una ShopFactory y la usa en stock().
// Es la única implementación; no hay otra prevista.
```

Cambio B: el precio depende de la hora

```
public interface PriceRule {
    int price(Item item);
}

public class Regular implements PriceRule {
    public int price(Item item) { return item.power() * 5; }
}

public class HappyHour implements PriceRule {
    public int price(Item item) { return item.power() * 4; }
}
// Antes, Shop repetía "if (night) {...} else {...}" en sell(),
// preview() y refund(). Ahora: shop.setRule(rule).
// Diseño ya tiene prevista una tercera regla, "Rebajas de gremio".
```

Cambio C: construir objetos

```
public class ItemBuilder {
    private String name;
    private ItemKind kind;
    private int power;

    public ItemBuilder name(String name) { this.name = name; return this; }
    public ItemBuilder kind(ItemKind k) { this.kind = k; return this; }
    public ItemBuilder power(int p) { this.power = p; return this; }
    public Item build() { return new Item(name, kind, power); }
}
// Uso: new ItemBuilder().name("Poción").kind(ItemKind.HEALING)
//          .power(10).build()
```

Cambio D: avisar de que se vendió algo

```
// dentro de GameEvent (interfaz sellada del capítulo 9)
record ItemSold(String item, int price) implements GameEvent {}

// Shop.sell
public int sell(Item item) {
    int price = rule.price(item);
    inventory = inventory.without(item.name());
    bus.publish(new ItemSold(item.name(), price));
    return price;
}
// AchievementTracker: a la tercera venta, logro "Cliente fiel".
// LogView, HudView y AchievementTracker ganan "case ItemSold e -> ..."
```

Para cada cambio, escribe un comentario de revisión con: qué patrón es, cuál es el olor (o que no hay ninguno), el veredicto («bien aplicado» o «sobrediseño»), tu argumento con las tres preguntas de la sección 14.7 y, si procede, la alternativa más simple o la mejora. Está bien si: (1) clasificas cada cambio con un veredicto y lo defiendes con las tres preguntas, no con una opinión; (2) en los sobrediseñados nombras la alternativa concreta (el código que escribirías); (3) en los bien aplicados nombras el coste que se acepta; (4) reconoces al menos un patrón del proyecto que el cambio duplica o reutiliza.

Lectura de código ▪ solución en la pág. 672

Ejercicio 14.4: Un jefe con fases, trampas y logros ★★★★★

No tienes que modificar Emberdeep; es un diseño, aunque conviene que sea implementable sobre el código del proyecto. El equipo quiere una mecánica nueva:

- **El Rey de ceniza,** un jefe con tres fases según su vida: por encima del 60 %, se comporta como un enemigo normal; entre el 30 % y el 60 %, apunta a los héroes más débiles; por debajo del 30 %, además, usa sus habilidades con la frecuencia de `SkillfulAi` (capítulo 7). Cambiar de fase debe verse en el registro de combate.
- **Trampas.** Las casillas `^` del mapa dañan con fuego al héroe que las pisa, una vez por casilla. El daño debe respetar escudos, resistencias y armadura como cualquier otro golpe, y debería poder deshacerse con `u` dentro de la ronda.
- **Dos logros:** «Matarreyes» (derrotar al jefe) y «Pies ligeros» (ganar la planta del jefe sin pisar ninguna trampa).

Entrega:

1. Una lista de decisiones de diseño. Para cada una: el olor, el patrón que eliges (o «ninguno»), la alternativa más simple que descartas y el coste que aceptas.
2. La relación con lo que Emberdeep ya tiene: qué reutilizas y qué clases o eventos nuevos aparecen.
3. El código completo de la pieza que consideres central (con sus `import`).
4. Al menos una decisión en la que *no* uses un patrón, y por qué.

Se valora, sin pasos fijos: (a) cada patrón elegido responde a un olor concreto de la mecánica y no a que «encaja»; (b) reutilizas piezas del proyecto (el bus de eventos, la tubería de daño, los comandos) en lugar de duplicarlas; (c) el diseño sobrevive a un cambio razonable (una cuarta fase, un segundo jefe); (d) hay al menos una decisión sin patrón y bien argumentada;

(e) los costes están dichos con honestidad. Hay varias soluciones válidas.

Diseño ▪ solución en la pág. 674

Ejercicio 14.5: Hito 15b: la fachada del motor y el guardado ★★★☆☆

Implementa el hito H15b partiendo del estado del capítulo anterior (solución del ejercicio 13.5), en tres partes. Antes de tocar nada, guarda la salida actual del guion de demostración para compararla al final:

```
$ ./gradlew -q run --args="scripts/demo.txt --plain" > antes.txt
```

Parte 1 (`SaveFile` y `GameEngine`).

1. `SaveFile`, en `engine/save/`: clase final con constructor privado y tres métodos estáticos. `write(Snapshot, Path)` escribe, con `Files.createDirectories` y `Files.writeString`, las líneas `version=1`, `floor`, `x`, `y`, un `hero.N.hp` por cada combatiente de la instantánea, `inventory.count` y un `inventory.N` con `nombre|TIPO|poder` por cada objeto de `items()`; rechaza con `IllegalArgumentException` un nombre que lleve `|`, `=` o un salto de línea. `readFloor(Path)` devuelve la planta. `read(Path, Snapshot base)` lee con `Files.readAllLines`, divide cada línea en el primer `=`, ignora las líneas en blanco, exige `version=1`, y devuelve la base con `floor`, `x`, `y`, la vida de los héroes (entre 0 y su máximo) y el inventario sustituidos; este último es una `Bag` llamada `"mochila"` con los objetos en orden.

2. `GameEngine`, en `engine/`: constructor `GameEngine(Dungeon, List<Hero>, DamageFormula, Random)`. `subscribe` (solo antes de empezar); `start(int floor)`, `phase()` y `enterCombat()`; las consultas `heroes()`, `enemies()`, `inventory()`, `round()`, `isOver()`, `startTurn(Fighter)` y `enemyPhase()`; los comandos `attack`, `defend`, `useItem` y `useSkill`; `perform`, `undo`, `checkpoint()` (devuelve cuántos hay) y `restoreCheckpoint()` (`false` si no hay ninguno); `save(Path)` (solo en la fase `Exploring`, si no `IllegalStateException`, y con una instantánea recortada a los héroes); `load(Path)` (monta una partida nueva de la planta del fichero, le aplica `read` y solo entonces sustituye la actual) y `descend()` (pasa a la planta siguiente llevando la vida de los héroes y la mochila; tras la última, la fase es `GameOver(true)`). Cada partida nueva suscribe los observadores de la interfaz, después el `GameFlow` y por último el `AchievementTracker`.

Está bien si `./gradlew -q classes` compila y estos dos programas temporales imprimen lo de abajo. `app/Check15a.java` monta una mazmorra de dos plantas (cripta con un esqueleto y caverna con un goblin), los héroes de `Main`, y un motor con ellos. En el primer motor: `start(2)`, quita 12 de vida a Aria y 17 a Cael, haz que Aria dé a Cael una «Poción de caverna» con un comando de `useItem` ejecutado a mano, y guarda en `saves/check.txt`; imprime el fichero. En un segundo motor idéntico: carga ese fichero e imprime la fase, los héroes («Nombre vida/máximo») y los objetos; guarda en `saves/check2.txt` e imprime si los dos ficheros son iguales; entra en combate y prueba a guardar de nuevo, imprimiendo el mensaje de la excepción.

Salida esperada de Check15a

```
version=1
floor=2
x=1
y=5
hero.0.hp=18
hero.1.hp=40
hero.2.hp=17
inventory.count=2
inventory.0=Poción de caverna|HEALING|10
inventory.1=Bomba de roca|BOMB|14
Fase: Exploring[floor=2]
Héroes: Aria 18/30, Borin 40/40, Cael 17/24
Objetos: Poción de caverna, Bomba de roca
Mismo fichero: true
Solo se guarda entre combates, no en la fase InCombat[floor=2]
```

`app/Check15b.java` usa el mismo motor de dos plantas: `start(1)`, quita 15 de vida a Borin, `descend()`; imprime la fase, los héroes, los objetos y cuántos enemigos hay; vuelve a llamar a `descend()` e imprime la fase.

Salida esperada de Check15b

```
Fase: Exploring[floor=2]
Héroes: Aria 30/30, Borin 25/40, Cael 24/24
Objetos: Poción de cripta, Poción de cripta, Bomba de ceniza
Enemigos: 1
Fase: GameOver[won=true]
```

Parte 2 (la interfaz y `Main` hablan con la fachada).

1. `ConsoleMenu`: `run` y todos los métodos privados que recibían un `GameSession` pasan a recibir un `GameEngine`. Los comandos salen de `engine.attack`, `engine.defend`, `engine.useItem` y `engine.useSkill`; los puntos de control, de `engine.checkpoint()` y `engine.restoreCheckpoint()` (los textos no cambian); el menú deja de tener su propio `SnapshotStore`.
2. `Main`: construye un `GameEngine` con la mazmorra, los héroes, la fórmula y el `Random`; le suscribe `LogView` y `HudView`; empieza con `start` (o, con `--load=ruta`, con `load` y `descend`); si la fase es `GameOver` imprime «Has recorrido toda la mazmorra.» y termina; si no, `enterCombat` y `menu.run(engine)`. Con `--save=ruta`, al terminar el combate en fase `Exploring`, guarda y imprime «Partida guardada en ruta.». `scriptOf` ignora los argumentos que empiezan por `--`.

Está bien si compila y si estas órdenes no imprimen nada (la salida del guion es idéntica a la de antes, y ninguna de las dos clases nombra ya las del motor que no necesita):

```
$ ./gradlew -q run --args="scripts/demo.txt --plain" > despues.txt
$ diff antes.txt despues.txt
$ cd src/main/java/emberdeep
$ for c in GameSession Combat SnapshotStore Attack \
>   Defend UseItem UseSkill; do grep -nw $c ui/ConsoleMenu.java; done
$ for c in GameSession DamagePipeline GameFlow EventBus \
>   AchievementTracker GameMap; do grep -nw $c app/Main.java; done
```

Parte 3 (guardar y continuar). Juega la cripta (planta 1) hasta ganar:

```
$ A="1 default --save=saves/partida.txt"
$ yes 1 | head -n 20 | ./gradlew -q run --args="$A"
```

Al ganar, aparece «Partida guardada en saves/partida.txt.». Abre el fichero: tiene `version=1`, `floor=1` y tres líneas `hero.N.hp` con la vida con la que terminaron los héroes. Luego sigue con ese fichero:

```
$ A="--load=saves/partida.txt"
$ yes 1 | head -n 20 | ./gradlew -q run --args="$A"
```

Debe empezar en la caverna (la planta 2): la primera línea «Héroes:» muestra esas mismas vidas, y el combate es contra el Goblin, el Murciélago y la Araña. Añade `/saves/` a tu `.gitignore` si usas git, borra `Check15a.java`, `Check15b.java`, `antes.txt`, `despues.txt` y la carpeta `saves/`, y comprueba que `./gradlew test` sigue con `BUILD SUCCESSFUL` (tres pruebas).

Pista: *Para que un `UseItem` se ejecute a mano: `engine.useItem(user, target, item).execute()`. Para el fichero igual de largo, `Files.readString(path)` lee todo el texto de una vez (UTF-8).*

Proyecto ▪ solución en la pág. 677

14.10 Resumen

Resumen

- El olor de Facade es un subsistema con muchas clases cuyo montaje, orden y detalles conocen sus clientes: `Main` y `ConsoleMenu` nombraban doce clases del motor. Facade pone una clase delante.
- Piezas: la fachada (`GameEngine`), el subsistema que oculta (`GameSession`, `Combat`, `EventBus`, `SnapshotStore`), la persistencia (`SaveFile`) y los clientes. Las flechas van de los clientes a la fachada y de la fachada al subsistema, nunca al revés.
- La fachada coordina (monta, ordena observadores, vigila la fase, sustituye partidas) y no decide reglas del juego. Esconde el subsistema pero no lo encierra: las pruebas siguen usándolo directamente.
- Facade no es una capa (una capa es una regla sobre paquetes, y la fachada es su puerta) ni es Mediator: Facade va en un solo sentido y el subsistema funciona sin ella; en Mediator los colegas la necesitan para hablarse.

- Cuesta una clase que puede crecer hasta ser un objeto dios, un método más por cada cosa nueva que la interfaz pida y pruebas de integración; no compensa sobre dos o tres clases.
- Guardar la partida es el Memento llevado al disco: se escribe lo mínimo para reconstruirla en un formato fijo `clave=valor` (UTF-8, se divide en el primer `=`, las líneas en blanco se ignoran, `version=1`), con `Files.writeString`, `Files.readAllLines` y `Files.createDirectories`.
- Reglas de diseño del guardado: solo en la fase `Exploring`; el equipo no se guarda y `load` parte de una partida nueva; el inventario se aplana. Los límites (bolsas, `Random`, escritura no atómica, una versión) están declarados.
- Elegir y defender un patrón son tres preguntas: ¿cuál es el olor?, ¿cuál es la alternativa más simple? y ¿qué coste añade? Si una no sale, el patrón no está defendido; una sola implementación sin segunda a la vista y una clase que solo reenvía son señales de sobrediseño.
- H15b: `GameEngine` (la fachada) con `start`, `perform`, `undo`, `checkpoint`, `save`, `load`, `subscribe` y `descend`; `SaveFile` en `engine/save/`; `Main` y `ConsoleMenu` solo hablan con la fachada; opciones `--save` y `--load`; carpeta `saves/` fuera de git.

Soluciones de los ejercicios

Solución del ejercicio 14.1

enunciado en la pág. 660

Player.java

```java
public class Player {
    private final Decoder decoder = new Decoder();
    private final Mixer mixer = new Mixer();
    private final AudioOutput output = new AudioOutput();
    private boolean playing;

    public void play(String path) {
        if (playing) {
            stop();
        }
        output.open();
        int[] samples = decoder.decode(path);
        output.write(mixer.mix(samples, 50));
        playing = true;
    }

    public void stop() {
        if (playing) {
            output.close();
            playing = false;
        }
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Player player = new Player();
        player.play("tema.mp3");
        player.play("otro.mp3");     // cierra la anterior antes
        player.stop();
        player.stop();               // no hace nada: ya no suena

        // la fachada no encierra los subsistemas
        new Decoder().decode("suelto.wav");
    }
}
```

Salida

```
Salida: abierta
Decoder: leyendo tema.mp3
Mixer: volumen 50
Salida: 3 muestras, primera 5
Salida: cerrada
Salida: abierta
Decoder: leyendo otro.mp3
Mixer: volumen 50
Salida: 3 muestras, primera 5
Salida: cerrada
Decoder: leyendo suelto.wav
```

Por qué funciona. `Player` es la fachada: construye los tres subsistemas, sabe en qué orden se usan (abrir la salida, decodificar, mezclar, escribir) y expone dos operaciones. `Main` ya no necesita saber que existen tres clases ni su orden. El segundo `play` llama a `stop` antes, y por eso aparece «cerrada» entre las dos reproducciones; el segundo `stop` no imprime nada porque `playing` es falso. Las cinco muestras de salida son $10 \cdot 50/100 = 5$. La última línea muestra que el subsistema sigue siendo público: la fachada es una comodidad, no un muro.
Errores típicos:

- Crear el `Decoder`, el `Mixer` y la salida en `Main` y pasárselos al `Player`: el cliente seguiría conociéndolos.
- Olvidar la bandera y cerrar siempre en `stop`: el segundo `stop` imprimiría «Salida: cerrada» de nuevo.
- Meter en `Player` la lógica de mezclar (el bucle de volumen): es trabajo del subsistema, no de la fachada.

Solución del ejercicio 14.2

enunciado en la pág. 662

1. Primer fallo. El programa lanza `ArrayIndexOutOfBoundsException: Index 1 out of bounds for length 1`. La línea que lo provoca es la **vacía** (la tercera del fichero): `"".split("=")` devuelve un array de un solo elemento, el texto vacío, y `parts[1]` no existe.
2. Segundo fallo. Si solo se salta la línea vacía, la salida es `name: Aria`, `motto: a` y `level: 3`: la incorrecta es la del lema. `"motto=a=b".split("=")` devuelve tres trozos (`motto`, `a` y `b`), y el código se queda con el segundo; el `=b` se pierde sin ninguna excepción. Es el peor tipo de fallo: el programa parece funcionar.

3 y 4. La corrección. Dividir solo en el primer `=` con `split("=", 2)`, saltar las líneas en blanco y rechazar las líneas sin `=`:

ConfigReader.java (corregido)

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.HashMap;
import java.util.Map;

public class ConfigReader {
    public static Map<String, String> read(Path file)
            throws IOException {
        Map<String, String> values = new HashMap<>();
        for (String line : Files.readAllLines(file)) {
            if (line.isBlank()) {
                continue;
            }
            String[] parts = line.split("=", 2);
            if (parts.length < 2) {
                throw new IllegalArgumentException(
                        "Línea sin '=': " + line);
            }
            values.put(parts[0], parts[1]);
        }
        return values;
    }
}
```

Main.java (sin cambios)

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Map;

public class Main {
    public static void main(String[] args) throws IOException {
        Path file = Path.of("config.txt");
        Files.writeString(file, "name=Aria\nmotto=a=b\n\nlevel=3\n");
        Map<String, String> values = ConfigReader.read(file);
        System.out.println("name: " + values.get("name"));
        System.out.println("motto: " + values.get("motto"));
        System.out.println("level: " + values.get("level"));

        Files.writeString(file, "name=Aria\nsin igual\n");
        try {
            ConfigReader.read(file);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Salida

```
name: Aria
motto: a=b
level: 3
Error: Línea sin '=': sin igual
```

Por qué funciona. El segundo argumento de `split` limita el número de trozos: con 2, el primer `=` separa la clave y todo lo que queda, con sus `=`, es el valor. La línea vacía se salta con `isBlank()` (también las que solo tienen espacios). Y una línea sin `=` da un solo trozo, que ahora se detecta y se convierte en un error con un mensaje claro en lugar de una excepción de índice.

Errores típicos:

- Arreglar solo el síntoma: capturar la `ArrayIndexOutOfBoundsException` en lugar de decidir qué es una línea válida.
- Usar `line.split("=")` y quedarse con `parts[1]`: pierde en silencio todo lo que haya tras el segundo `=`.
- Comprobar `line.isEmpty()` en lugar de `isBlank()`: una línea con solo un espacio no se saltaría y fallaría como la vacía.
- Cortar con `indexOf("=")` y `substring` sin mirar el caso `-1`.

Solución del ejercicio 14.3

enunciado en la pág. 663

Cambio A: `ShopFactory`. Veredicto: sobrediseño.

- **Olor:** ninguno. No hay un `new Item` repetido en muchos sitios ni una segunda familia de objetos que cambie. Hay una sola implementación y «no hay otra prevista».
- **Alternativa más simple:** dos opciones. Si la tienda necesita los objetos de un bioma, ya existe una fábrica (`BiomeFactory`, capítulo 3) con `item(kind)`; el cambio la duplica. Si son objetos fijos, basta el constructor del record.
- **Coste:** una interfaz, una clase, un parámetro en el constructor de `Shop` y un fichero más que mantener, para no ganar nada.

Alternativa para el cambio A

```
public class Shop {
    private final BiomeFactory biome;

    public Shop(BiomeFactory biome) {
        this.biome = biome;
    }

    public List<Item> stock() {
        return List.of(biome.item(ItemKind.HEALING),
                biome.item(ItemKind.BOMB));
    }
}
```

Cambio B: `PriceRule`. Veredicto: bien aplicado (Strategy, capítulo 2).

- **Olor:** el mismo `if (night)` repetido en tres métodos y una tercera regla ya prevista.

Eso es justo lo que cura Strategy.

- **Alternativa más simple:** dejar el `if`. Valdría con una regla y un solo sitio donde decidir; con tres métodos y una tercera regla en camino, cada regla nueva obligaría a tocar los tres.
- **Coste que se acepta:** una interfaz y una clase por regla. Mejora posible: `PriceRule` tiene un solo método, así que puede ser una interfaz funcional con lambdas y no hacen falta las clases `Regular` y `HappyHour`, igual que `DamageFormula`.

Mejora para el cambio B: interfaz funcional y lambdas

```
@FunctionalInterface
public interface PriceRule {
    int price(Item item);
}

// en Main o donde se monte la tienda
PriceRule regular = item -> item.power() * 5;
PriceRule happyHour = item -> item.power() * 4;
shop.setRule(night ? happyHour : regular);
```

Cambio C: `ItemBuilder`. Veredicto: sobrediseño.

- **Olor:** ninguno. `Item` es un record de tres campos que siempre se dan todos; no hay parámetros opcionales, ni pasos que ordenar, ni validación compleja.
- **Alternativa más simple:** el constructor del record, que además es legible con tres argumentos. El capítulo 4 reserva Builder para los casos con muchos campos, pasos u opcionales (el héroe, la mazmorra).
- **Coste:** una clase mutable de 15 líneas y un peligro nuevo: `build()` se puede llamar con campos sin rellenar (`name` a `null`), un error que el constructor del record no permite.

Alternativa para el cambio C

```
Item potion = new Item("Poción", ItemKind.HEALING, 10);
```

Cambio D: evento `ItemSold`. Veredicto: bien aplicado (Observer, capítulo 9).

- **Olor:** sin él, `Shop` tendría que llamar directamente al `AchievementTracker` (y mañana a otro oyente): la tienda conocería a quien reacciona a sus ventas.
- **Alternativa más simple:** una llamada directa, `tracker.itemSold()`. Funcionaría con un único oyente fijo; aquí ya hay tres vistas y un tracker que escuchan eventos.
- **Coste que se acepta:** un record en `GameEvent` y un `case` en cada `switch` sin `default` (`LogView`, `HudView`, `AchievementTracker`); el compilador avisa de los que falten. Reutiliza el bus del capítulo 9 en vez de inventar uno.

Qué se valora. Veredicto correcto en los cuatro (A y C, sobrediseño; B y D, bien aplicados), argumento con las tres preguntas y no con gustos, alternativa concreta en A y C, coste nombrado en B y D, y reconocer lo que ya había en el proyecto (`BiomeFactory` para A, el bus de eventos para D, el record `Item` para C, `DamageFormula` como modelo de interfaz funcional para B).

Errores típicos: dar el veredicto por el nombre del patrón («un Builder siempre es bueno»); aceptar B sin pedir la mejora de lambdas; olvidar que D obliga a tocar tres `switch`; no escribir la alternativa.

Solución del ejercicio 14.4

enunciado en la pág. 665

Solución de referencia. Seis decisiones, cada una con las tres preguntas:

Decisión	Olor	Elección	Alternativa más simple	Coste
1. Fases del jefe	Los umbrales de vida se consultarían en la IA, el HUD y el registro	State (`BossPhase` sellada) y Strategy (`ai()` por fase)	Un `if` dentro de `BossAi`	Un tipo sellado y un estado mutable en `BossAi`
2. Aviso del cambio de fase	`BossAi` no debe conocer el registro ni el HUD	Observer: evento `BossPhaseChanged` en el bus	Llamar a `LogView` directamente	Un record y un `case` en cada `switch`
3. Daño de la trampa	Debe respetar escudo, resistencia y armadura	Ninguno nuevo: reutilizar la tubería de daño (Chain, cap. 6)	Calcular el daño en la trampa	Cero clases; `deal` pide un atacante
4. Deshacer la trampa	`u` debe poder deshacerla	Command: `TriggerTrap` guarda vida y escudo, como `Attack`	Un método sin `undo`	Una clase de unas 30 líneas; el evento no se retira
5. Los dos logros	Hay que enterarse de cosas ajenas	Observer: el `AchievementTracker` ya suscrito	Contadores en `Combat`	Un booleano más en el tracker
6. Una vez por casilla	Recordar qué casillas ya se pisaron	**Ningún patrón:** un `Set` de posiciones	(es la más simple)	Nada que añadir

Lo que se reutiliza y lo que aparece. Se reutilizan el bus de eventos (capítulo 9), la tubería de daño y `combat.deal` (capítulo 6), la interfaz `CombatCommand` y `CommandHistory` (capítulo 7), `EnemyAi` y sus implementaciones (capítulo 2) y el `AchievementTracker`. Aparecen: `BossPhase` y `BossAi` (paquete `engine.combat.boss`), `TriggerTrap` (un comando), y dos records de `GameEvent`: `BossPhaseChanged(String boss, String phase)` y `TrapTriggered(String hero, int damage)`. Los tres `switch` sobre eventos (`LogView`, `HudView`, `AchievementTracker`) ganan un `case` por cada record; el tracker, además, desbloquea «Matarreyes» al ver caer al jefe (`FighterDefeated`), guarda si hubo un `TrapTriggered` y desbloquea «Pies ligeros» en un `CombatEnded` ganado sin trampas.

La pieza central: la fase y su estrategia.

engine/combat/boss/BossPhase.java

```java
package emberdeep.engine.combat.boss;

import emberdeep.engine.combat.ai.EnemyAi;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.SkillfulAi;

public sealed interface BossPhase {
    record Calm() implements BossPhase {}

    record Hunting() implements BossPhase {}

    record Desperate() implements BossPhase {}

    // la fase que toca según la vida: solo depende de dos números
    static BossPhase forHp(int hp, int maxHp) {
        if (hp * 100 > maxHp * 60) {
            return new Calm();
        }
        if (hp * 100 > maxHp * 30) {
            return new Hunting();
        }
        return new Desperate();
    }

    // Strategy: cada fase tiene su manera de elegir objetivo y actuar
    default EnemyAi ai() {
        return switch (this) {
            case Calm calm -> new FirstAliveAi();
            case Hunting hunting -> new LowestHpAi();
            case Desperate desperate ->
                    new SkillfulAi(new LowestHpAi());
        };
    }
}
```

engine/combat/boss/BossAi.java

```java
package emberdeep.engine.combat.boss;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.ai.EnemyAi;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.event.EventBus;
import emberdeep.engine.event.GameEvent.BossPhaseChanged;
import java.util.List;

public class BossAi implements EnemyAi {
    private final EventBus bus;
    private BossPhase phase = new BossPhase.Calm();

    public BossAi(EventBus bus) {
        this.bus = bus;
    }

    @Override
    public Fighter choose(Fighter self, List<Fighter> heroes) {
        return refresh(self).ai().choose(self, heroes);
    }

    @Override
    public CombatCommand act(Combat combat, Fighter self,
                             List<Fighter> heroes) {
        return refresh(self).ai().act(combat, self, heroes);
    }

    // State: la fase se recalcula con la vida de este momento y,
    // si cambia, se avisa por el bus (Observer)
    private BossPhase refresh(Fighter self) {
        BossPhase now = BossPhase.forHp(self.hp(), self.stats().maxHp());
        if (!now.equals(phase)) {
            phase = now;
            bus.publish(new BossPhaseChanged(self.name(),
                    phase.getClass().getSimpleName()));
        }
        return phase;
    }
}
```

Los dos records nuevos de `GameEvent` (que es una interfaz sellada, así que se declaran en su mismo archivo, junto a los demás):

engine/event/GameEvent.java: dos records nuevos

```java
record BossPhaseChanged(String boss, String phase)
        implements GameEvent {}

record TrapTriggered(String hero, int damage) implements GameEvent {}
```

Por qué funciona. `BossPhase.forHp` decide la fase con aritmética entera (`hp * 100 >`

`maxHp * 60` evita los decimales). La fase es un State sin datos propios; su método `ai()` es un Strategy: cada fase elige una estrategia que ya existía (`FirstAliveAi`, `LowestHpAi`, `SkillfulAi`). `BossAi` implementa `EnemyAi`, así que el resto del motor lo trata como a cualquier enemigo; el `Fighter` del jefe se crea con ella como IA. Cuando cambia la fase, `BossAi` publica un evento y no sabe quién lo oye: el registro puede escribir «El Rey de ceniza se enfurece» y el HUD ignorarlo.

Costes y límites que hay que decir. `BossAi` tiene estado mutable (la fase actual) y por eso no se puede compartir entre dos jefes; `combat.deal` pide un atacante, y una trampa no lo tiene (habría que añadir una sobrecarga o pasar al propio héroe); y deshacer la trampa no retira el evento ya publicado, igual que un logro desbloqueado no se retira con `u`.

Criterios para valorar otras soluciones. Una solución es buena si cada patrón se justifica con un olor concreto de esta mecánica, si no duplica el bus, la tubería de daño ni los comandos que ya existen, si el cambio de fase llega a registro y logros sin que la lógica del jefe los conozca, y si hay al menos una decisión sin patrón con su argumento. Es mejor decir un coste real que esconderlo.

Alternativa válida, más simple. Un jefe, tres fases y ningún segundo jefe previsto pueden resolverse sin State: un `enum Phase {CALM, HUNTING, DESPERATE}` dentro de `BossAi` con un `switch`, la trampa como un método de `GameSession` que llama a `combat.deal`, y los logros por Observer. Pierde el `undo` de la trampa y obliga a repetir el umbral si otra clase necesita conocer la fase. Es defendible si se dicen esas dos cosas y se acepta el cambio al primer síntoma: el segundo jefe o el `undo`.

Errores típicos: elegir State, Strategy, Observer, Chain y Command «porque están en el libro» sin decir qué olor cura cada uno; crear una cadena nueva para la trampa en lugar de reutilizar la tubería; hacer que `BossAi` llame al registro directamente; olvidar que añadir records a una interfaz sellada obliga a revisar los `switch`; no decir ningún coste.

Solución del ejercicio 14.5

enunciado en la pág. 666

Los archivos, completos. Primero la parte 1: el guardado y la fachada (en `engine/save/` y `engine/`).

engine/save/SaveFile.java (1/3)

```
package emberdeep.engine.save;

import emberdeep.engine.item.InventoryNode;
import emberdeep.engine.item.InventoryNode.Bag;
import emberdeep.engine.item.InventoryNode.ItemSlot;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public final class SaveFile {
    private static final String VERSION = "1";

    private SaveFile() {}

    // Escribe la instantánea: sus combatientes se guardan como héroes
    public static void write(Snapshot snapshot, Path path)
            throws IOException {
        StringBuilder text = new StringBuilder();
        put(text, "version", VERSION);
        put(text, "floor", snapshot.floor());
        put(text, "x", snapshot.x());
        put(text, "y", snapshot.y());
        List<FighterState> heroes = snapshot.fighters();
        for (int i = 0; i < heroes.size(); i++) {
            put(text, "hero." + i + ".hp", heroes.get(i).hp());
        }
        List<Item> items = snapshot.inventory().items();
        put(text, "inventory.count", items.size());
        for (int i = 0; i < items.size(); i++) {
            Item item = items.get(i);
            put(text, "inventory." + i, safe(item.name()) + "|"
                    + item.kind() + "|" + item.power());
        }
        Files.createDirectories(path.toAbsolutePath().getParent());
        Files.writeString(path, text.toString());
    }

    private static void put(StringBuilder text, String key, Object value) {
        text.append(key).append('=').append(value).append('\n');
    }
```

engine/save/SaveFile.java (2/3)

```
    // el formato separa campos con | y clave de valor con =
    private static String safe(String name) {
        if (name.contains("|") || name.contains("=")
                || name.contains("\n")) {
            throw new IllegalArgumentException(
                    "Nombre no válido para el fichero: " + name);
        }
        return name;
    }

    // Solo la planta: load la necesita para elegir la base de read
    public static int readFloor(Path path) throws IOException {
        return number(parse(path), "floor");
    }

    // Parte de la base (una partida nueva de esa planta) y le
    // sobrescribe lo que trae el fichero
    public static Snapshot read(Path path, Snapshot base)
            throws IOException {
        Map<String, String> values = parse(path);
        List<FighterState> fighters = new ArrayList<>(base.fighters());
        for (int i = 0; values.containsKey("hero." + i + ".hp"); i++) {
            if (i >= fighters.size()) {
                throw new IllegalArgumentException(
                        "El fichero tiene más héroes que la partida");
            }
            FighterState old = fighters.get(i);
            int hp = number(values, "hero." + i + ".hp");
            if (hp < 0 || hp > old.stats().maxHp()) {
                throw new IllegalArgumentException(
                        "Vida fuera de rango en hero." + i + ".hp: " + hp);
            }
            fighters.set(i, new FighterState(hp, old.shield(),
                    old.stats(), old.status()));
        }
        List<InventoryNode> slots = new ArrayList<>();
        int count = number(values, "inventory.count");
        for (int i = 0; i < count; i++) {
            String text = required(values, "inventory." + i);
            slots.add(new ItemSlot(item(text)));
        }
        return new Snapshot(number(values, "floor"), number(values, "x"),
                number(values, "y"), base.round(), fighters,
                new Bag("mochila", slots));
    }
```

engine/save/SaveFile.java (3/3)

```
    // clave=valor: se divide en el PRIMER '='; las líneas en blanco
    // se saltan
    private static Map<String, String> parse(Path path)
            throws IOException {
        Map<String, String> values = new LinkedHashMap<>();
        for (String line : Files.readAllLines(path)) {
            if (line.isBlank()) {
                continue;
            }
            String[] parts = line.split("=", 2);
            if (parts.length < 2) {
                throw new IllegalArgumentException(
                        "Línea sin '=': " + line);
            }
            values.put(parts[0], parts[1]);
        }
        if (!VERSION.equals(values.get("version"))) {
            throw new IllegalArgumentException(
                    "Versión no soportada: " + values.get("version"));
        }
        return values;
    }

    private static String required(Map<String, String> values,
                                   String key) {
        String value = values.get(key);
        if (value == null) {
            throw new IllegalArgumentException("Falta la clave " + key);
        }
        return value;
    }

    private static int number(Map<String, String> values, String key) {
        return Integer.parseInt(required(values, key));
    }

    // nombre|TIPO|poder
    private static Item item(String text) {
        String[] fields = text.split("\\|");
        if (fields.length != 3) {
            throw new IllegalArgumentException(
                    "Objeto mal formado: " + text);
        }
        return new Item(fields[0], ItemKind.valueOf(fields[1]),
                Integer.parseInt(fields[2]));
    }
}
```

engine/GameEngine.java (1/6)

```
package emberdeep.engine;

import emberdeep.engine.achievement.AchievementTracker;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.Skill;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.command.Attack;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.combat.command.Defend;
import emberdeep.engine.combat.command.UseItem;
import emberdeep.engine.combat.command.UseSkill;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.defense.DamagePipeline;
import emberdeep.engine.event.EventBus;
import emberdeep.engine.event.GameListener;
import emberdeep.engine.flow.GameFlow;
import emberdeep.engine.flow.GamePhase;
import emberdeep.engine.flow.GamePhase.Exploring;
import emberdeep.engine.item.InventoryNode.Bag;
import emberdeep.engine.item.Item;
import emberdeep.engine.save.FighterState;
import emberdeep.engine.save.SaveFile;
import emberdeep.engine.save.Snapshot;
import emberdeep.engine.save.SnapshotStore;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.Dungeon;
import emberdeep.engine.world.Floor;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.MapParser;
import emberdeep.engine.world.map.TileTypes;
import java.io.IOException;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
import java.util.Random;

// La fachada del motor: la interfaz y Main hablan solo con esta clase.
public class GameEngine {
    // lo que hay que montar junto para una partida
    private record Game(GameSession session, GameFlow flow,
                        SnapshotStore checkpoints) {}
```

engine/GameEngine.java (2/6)

```
    private final Dungeon dungeon;
    private final List<Hero> heroes;
    private final DamageFormula formula;
    private final Random random;
    private final List<GameListener> listeners = new ArrayList<>();
    private Game game;

    public GameEngine(Dungeon dungeon, List<Hero> heroes,
                      DamageFormula formula, Random random) {
        this.dungeon = Objects.requireNonNull(dungeon);
        this.heroes = List.copyOf(heroes);
        this.formula = Objects.requireNonNull(formula);
        this.random = Objects.requireNonNull(random);
    }

    // Los observadores de la interfaz se apuntan antes de empezar
    public void subscribe(GameListener listener) {
        if (game != null) {
            throw new IllegalStateException(
                    "Suscríbete antes de empezar la partida");
        }
        listeners.add(Objects.requireNonNull(listener));
    }

    // Una partida nueva en esa planta, en fase Exploring
    public void start(int floorNumber) {
        game = build(floorNumber);
    }
```

engine/GameEngine.java (3/6)

```
    private Game build(int floorNumber) {
        int last = dungeon.floors().size();
        if (floorNumber < 1 || floorNumber > last) {
            throw new IllegalArgumentException(
                    "La planta debe estar entre 1 y " + last);
        }
        Floor floor = dungeon.floors().get(floorNumber - 1);
        GameSession session = new GameSession(heroes, floor.biome(),
                floor.enemies(), formula, new DamagePipeline(random, 0.0));
        GameMap entrance = new MapParser(new TileTypes())
                .parse(floor.map());
        session.setLocation(floorNumber,
                entrance.partyX(), entrance.partyY());
        GameFlow flow = new GameFlow(session.heroes(), last);
        flow.start(floorNumber);
        EventBus events = session.events();
        listeners.forEach(events::subscribe);
        events.subscribe(flow);
        events.subscribe(new AchievementTracker(events));   // el último
        return new Game(session, flow, new SnapshotStore());
    }

    private Game current() {
        if (game == null) {
            throw new IllegalStateException("La partida no ha empezado");
        }
        return game;
    }

    private GameSession session() {
        return current().session();
    }

    public GamePhase phase() {
        return current().flow().phase();
    }

    public void enterCombat() {
        current().flow().enterCombat();
    }

    // Lo que la interfaz necesita ver de la partida
    public List<Fighter> heroes() {
        return session().heroes();
    }
```

engine/GameEngine.java (4/6)

```
    public List<Fighter> enemies() {
        return session().enemies();
    }

    public Bag inventory() {
        return session().inventory();
    }

    public int round() {
        return session().round();
    }

    public boolean isOver() {
        return session().combat().isOver();
    }

    public boolean startTurn(Fighter fighter) {
        return session().combat().startTurn(fighter);
    }

    public void enemyPhase() {
        session().combat().enemyPhase();
    }

    // Lo que la interfaz puede pedir; los comandos se crean aquí para que
    // quien juega no necesite Combat, GameSession ni el mediador
    public CombatCommand attack(Fighter attacker, Fighter target) {
        return new Attack(session().combat(), attacker, target);
    }

    public CombatCommand defend(Fighter user) {
        return new Defend(session().combat(), user);
    }

    public CombatCommand useItem(Fighter user, Fighter target, Item item) {
        return new UseItem(session(), user, target, item);
    }

    public CombatCommand useSkill(Fighter user, Skill skill,
                                  Fighter target) {
        Combat combat = session().combat();
        return new UseSkill(combat, combat.mediator(), user, skill,
                target);
    }

    public void perform(CombatCommand command) {
        session().combat().perform(command);
    }
```

engine/GameEngine.java (5/6)

```
    public boolean undo() {
        return session().combat().undo();
    }

    // Puntos de control en memoria (cap. 8); devuelve cuántos hay
    public int checkpoint() {
        current().checkpoints().push(session().createSnapshot());
        return current().checkpoints().size();
    }

    // false si todavía no hay ninguno
    public boolean restoreCheckpoint() {
        SnapshotStore checkpoints = current().checkpoints();
        if (checkpoints.isEmpty()) {
            return false;
        }
        session().restore(checkpoints.last());
        return true;
    }

    // Guardar en fichero: solo entre combates (fase Exploring)
    public void save(Path file) throws IOException {
        GamePhase phase = phase();
        if (!(phase instanceof Exploring)) {
            throw new IllegalStateException("Solo se guarda entre "
                    + "combates, no en la fase " + phase);
        }
        Snapshot all = session().createSnapshot();
        // el fichero lleva solo a los héroes, no a los enemigos
        Snapshot saved = new Snapshot(all.floor(), all.x(), all.y(),
                all.round(), all.fighters().subList(0, heroes.size()),
                all.inventory());
        SaveFile.write(saved, file);
    }

    // Una partida NUEVA en la planta guardada con lo guardado encima.
    // Si algo falla, la partida actual queda como estaba.
    public void load(Path file) throws IOException {
        Game loaded = build(SaveFile.readFloor(file));
        GameSession session = loaded.session();
        session.restore(SaveFile.read(file, session.createSnapshot()));
        game = loaded;
    }
```

engine/GameEngine.java (6/6)

```
    // Termina la planta y pasa a la siguiente con la vida y la mochila
    // de ahora; tras la última planta, la fase es GameOver(true)
    public void descend() {
        Snapshot finished = session().createSnapshot();
        current().flow().descend();
        if (phase() instanceof Exploring next) {
            Game following = build(next.floor());
            GameSession session = following.session();
            session.restore(carryOver(finished, session.createSnapshot()));
            game = following;
        }
    }

    // la base nueva, pero con la vida de los héroes y la mochila de antes
    private Snapshot carryOver(Snapshot finished, Snapshot fresh) {
        List<FighterState> states = new ArrayList<>(fresh.fighters());
        for (int i = 0; i < heroes.size(); i++) {
            FighterState old = states.get(i);
            states.set(i, new FighterState(finished.fighters().get(i).hp(),
                    old.shield(), old.stats(), old.status()));
        }
        return new Snapshot(fresh.floor(), fresh.x(), fresh.y(),
                fresh.round(), states, finished.inventory());
    }
}
```

Los dos programas temporales, en `app/`:

app/Check15a.java (1/2)

```java
package emberdeep.app;

import emberdeep.engine.GameEngine;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.item.Item;
import emberdeep.engine.world.Dungeon;
import emberdeep.engine.world.DungeonBuilder;
import emberdeep.engine.world.biome.CavernBiome;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.engine.world.map.MapData;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.Random;
import java.util.stream.Collectors;

public class Check15a {
    // Dos plantas (cripta y caverna) y los tres héroes de Main
    static GameEngine newEngine() {
        Random random = new Random(42);
        Dungeon dungeon = new DungeonBuilder()
                .floor().biome(new CryptBiome(random))
                        .enemies(List.of(EnemyKind.SKELETON))
                        .map(MapData.floor(1)).done()
                .floor().biome(new CavernBiome(random))
                        .enemies(List.of(EnemyKind.GOBLIN))
                        .map(MapData.floor(2)).done()
                .build();
        List<Hero> heroes = List.of(
                new HeroBuilder().name("Aria")
                        .stats(new Stats(30, 8, 3)).build(),
                new HeroBuilder().name("Borin")
                        .stats(new Stats(40, 6, 5)).build(),
                new HeroBuilder().name("Cael")
                        .stats(new Stats(24, 9, 2)).build());
        return new GameEngine(dungeon, heroes, new FlatFormula(), random);
    }
```

app/Check15a.java (2/2)

```
    static String describe(List<Fighter> group) {
        return group.stream()
                .map(f -> f.name() + " " + f.hp() + "/"
                        + f.stats().maxHp())
                .collect(Collectors.joining(", "));
    }

    public static void main(String[] args) throws IOException {
        Path file = Path.of("saves", "check.txt");
        Path again = Path.of("saves", "check2.txt");

        GameEngine first = newEngine();
        first.start(2);
        Fighter aria = first.heroes().get(0);
        Fighter cael = first.heroes().get(2);
        aria.takeDamage(12);                          // Aria 18/30
        cael.takeDamage(17);                          // Cael 7/24
        Item potion = first.inventory().find("Poción de caverna")
                .orElseThrow();
        first.useItem(aria, cael, potion).execute();    // Cael 17/24
        first.save(file);
        System.out.print(Files.readString(file));

        GameEngine second = newEngine();              // «abrir» otra vez
        second.load(file);
        System.out.println("Fase: " + second.phase());
        System.out.println("Héroes: " + describe(second.heroes()));
        System.out.println("Objetos: "
                + second.inventory().items().stream()
                        .map(Item::name)
                        .collect(Collectors.joining(", ")));
        second.save(again);
        System.out.println("Mismo fichero: "
                + Files.readString(file).equals(Files.readString(again)));
        second.enterCombat();
        try {
            second.save(again);
        } catch (IllegalStateException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

app/Check15b.java

```java
package emberdeep.app;

import emberdeep.engine.GameEngine;
import emberdeep.engine.item.Item;
import java.util.stream.Collectors;

public class Check15b {
    public static void main(String[] args) {
        GameEngine engine = Check15a.newEngine();
        engine.start(1);
        engine.heroes().get(1).takeDamage(15);      // Borin 25/40

        engine.descend();
        System.out.println("Fase: " + engine.phase());
        System.out.println("Héroes: "
                + Check15a.describe(engine.heroes()));
        System.out.println("Objetos: "
                + engine.inventory().items().stream()
                        .map(Item::name)
                        .collect(Collectors.joining(", ")));
        System.out.println("Enemigos: " + engine.enemies().size());

        engine.descend();
        System.out.println("Fase: " + engine.phase());
    }
}
```

Salida de Check15a

```
version=1
floor=2
x=1
y=5
hero.0.hp=18
hero.1.hp=40
hero.2.hp=17
inventory.count=2
inventory.0=Poción de caverna|HEALING|10
inventory.1=Bomba de roca|BOMB|14
Fase: Exploring[floor=2]
Héroes: Aria 18/30, Borin 40/40, Cael 17/24
Objetos: Poción de caverna, Bomba de roca
Mismo fichero: true
Solo se guarda entre combates, no en la fase InCombat[floor=2]
```

Salida de Check15b

```
Fase: Exploring[floor=2]
Héroes: Aria 30/30, Borin 25/40, Cael 24/24
Objetos: Poción de cripta, Poción de cripta, Bomba de ceniza
Enemigos: 1
Fase: GameOver[won=true]
```

Por qué funciona (parte 1). `SaveFile.write` convierte la instantánea en líneas `clave=valor` y las escribe de una vez con `Files.writeString`, después de crear la carpeta. `read` no reconstruye nada: parte de la base y cambia lo que el fichero trae, así que escudo, estado y equipo se quedan como los dejó la partida nueva. En `Check15a`, Aria baja a 18 de vida y Cael a 7; la poción de caverna cura 10 a Cael (17) y sale de la mochila (quedan la otra poción y la bomba). El fichero tiene `x=1` e `y=5` porque la entrada del mapa de la planta 2 es esa casilla. El segundo motor es otro objeto con otra sesión: lo único que comparten es el fichero, y por eso «Mismo fichero: true» prueba que guardar y cargar no pierde nada del formato. Entrar en combate cambia la fase a `InCombat[floor=2]` y `save` lo rechaza (regla a). En `Check15b`, `descend` monta la planta 2 y le pasa a Borin 25 de vida y la mochila de la cripta (con las pociones de cripta y la bomba de ceniza), y el segundo `descend`, en la última planta, deja la fase en `GameOver[won=true]`.

Parte 2: la interfaz y `Main`. `ConsoleMenu`, entero. Cambia el tipo de casi todo (de `GameSession` a `GameEngine`), no su comportamiento:

ui/ConsoleMenu.java (1/7)

```java
package emberdeep.ui;

import emberdeep.engine.GameEngine;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Skill.Heal;
import emberdeep.engine.character.Skill.Strike;
import emberdeep.engine.character.Skill.Stun;
import emberdeep.engine.character.Skill.Summon;
import emberdeep.engine.character.Skill.Taunt;
import emberdeep.engine.character.Skill.Venom;
import emberdeep.engine.combat.command.CombatCommand;
import emberdeep.engine.item.InventoryNode.Bag;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.TileType;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.input.InputSource;
import emberdeep.ui.view.MapView;
import java.io.PrintStream;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class ConsoleMenu {
    private enum Turn { ACTED, UNDONE, RESTORED }

    private final InputSource input;
    private final PrintStream out;
    private final MapView mapView;
    private int checkpointHero;     // a qué héroe le tocaba al guardar
    private List<Fighter> acting = List.of();       // quién actúa ahora
    private List<Fighter> checkpointActing = List.of();   // y al guardar

    public ConsoleMenu(InputSource input, PrintStream out,
                       MapView mapView) {
        this.input = input;
        this.out = out;
        this.mapView = mapView;
    }

    public int askFormula() {
        out.println("Fórmula de daño:");
        out.println("  [1] plana: ataque - defensa");
        out.println("  [2] escalada: ataque x 100 / (100 + defensa)");
        return ask(2);
    }
```

ui/ConsoleMenu.java (2/7)

```
    public void run(GameEngine engine) {
        out.println("=== Emberdeep ===");
        while (!engine.isOver()) {
            out.println("--- Ronda " + engine.round() + " ---");
            acting = startRound(engine);
            int i = 0;
            while (i < acting.size() && !engine.isOver()) {
                Turn turn = heroTurn(engine, acting.get(i), i);
                switch (turn) {
                    case ACTED -> i++;
                    case UNDONE -> i = Math.max(0, i - 1);
                    case RESTORED -> {
                        acting = checkpointActing;
                        i = checkpointHero;
                    }
                }
            }
            if (!engine.isOver()) {
                engine.enemyPhase();
            }
        }
    }

    private List<Fighter> aliveHeroes(GameEngine engine) {
        return engine.heroes().stream().filter(Fighter::isAlive).toList();
    }

    // Cap. 12: cada héroe consulta su estado al empezar la ronda. Actúan
    // los que siguen vivos y no pierden el turno.
    private List<Fighter> startRound(GameEngine engine) {
        List<Fighter> ready = new ArrayList<>();
        for (Fighter hero : aliveHeroes(engine)) {
            if (engine.startTurn(hero)) {
                ready.add(hero);
            }
        }
        return ready;
    }
```

ui/ConsoleMenu.java (3/7)

```
    // ACTED: el héroe actúa; UNDONE: se deshace la acción anterior;
    // RESTORED: se vuelve a un punto de control
    private Turn heroTurn(GameEngine engine, Fighter hero, int index) {
        while (true) {
            out.println("Héroes:   " + describe(engine.heroes()));
            out.println("Enemigos: " + describe(engine.enemies()));
            out.println("Objetos:  " + describeItems(engine.inventory()));
            out.println("Turno de " + hero.name()
                    + ". [1] atacar  [2] defender  [3] objeto"
                    + "  [4] habilidad  [u] deshacer"
                    + "  [c] guardar  [r] volver");
            String key = input.next();
            if (key.equals("u")) {
                if (engine.undo()) {
                    out.println("Deshaces la última acción.");
                    return Turn.UNDONE;
                }
                out.println("No hay nada que deshacer en esta ronda.");
                continue;
            }
            if (key.equals("c")) {
                saveCheckpoint(engine, index);
                continue;
            }
            if (key.equals("r")) {
                if (loadCheckpoint(engine)) {
                    return Turn.RESTORED;
                }
                continue;
            }
            CombatCommand command = switch (key) {
                case "1" -> engine.attack(hero,
                        askTarget("Objetivo", engine.enemies()));
                case "2" -> engine.defend(hero);
                case "3" -> chooseItem(engine, hero);
                case "4" -> chooseSkill(engine, hero);
                default -> {
                    out.println("Tecla no válida.");
                    yield null;
                }
            };
            if (command != null) {
                engine.perform(command);
                return Turn.ACTED;
            }
        }
    }
```

ui/ConsoleMenu.java (4/7)

```
    private void saveCheckpoint(GameEngine engine, int index) {
        int saved = engine.checkpoint();
        checkpointHero = index;
        checkpointActing = acting;
        out.println("Punto de control guardado (" + saved + ").");
    }

    // false si todavía no hay ninguno
    private boolean loadCheckpoint(GameEngine engine) {
        if (!engine.restoreCheckpoint()) {
            out.println("No hay puntos de control.");
            return false;
        }
        out.println("Vuelves al último punto de control (ronda "
                + engine.round() + ").");
        return true;
    }

    private CombatCommand chooseItem(GameEngine engine, Fighter hero) {
        List<Item> items = engine.inventory().items();
        if (items.isEmpty()) {
            out.println("No te quedan objetos.");
            return null;
        }
        Item item = items.get(choose("Objeto",
                items.stream().map(Item::name).toList()));
        Fighter target = item.kind() == ItemKind.HEALING
                ? askTarget("Aliado", engine.heroes())
                : askTarget("Objetivo", engine.enemies());
        return engine.useItem(hero, target, item);
    }
```

ui/ConsoleMenu.java (5/7)

```
    private CombatCommand chooseSkill(GameEngine engine, Fighter hero) {
        List<Skill> skills = hero.skills();
        if (skills.isEmpty()) {
            out.println("No tienes habilidades.");
            return null;
        }
        Skill skill = skills.get(choose("Habilidad",
                skills.stream().map(Skill::name).toList()));
        Fighter target = switch (skill) {
            case Strike strike -> askTarget("Objetivo", engine.enemies());
            case Heal heal -> askTarget("Aliado", engine.heroes());
            case Taunt taunt -> hero;       // se lanza sobre sí mismo
            case Venom venom -> askTarget("Objetivo", engine.enemies());
            case Stun stun -> askTarget("Objetivo", engine.enemies());
            case Summon summon -> hero;     // solo la usan los enemigos
        };
        return engine.useSkill(hero, skill, target);
    }

    private Fighter askTarget(String title, List<Fighter> group) {
        List<Fighter> alive = group.stream()
                .filter(Fighter::isAlive).toList();
        return alive.get(choose(title,
                alive.stream().map(Fighter::name).toList()));
    }

    // Muestra "Título:  [1] a  [2] b" y devuelve la posición (desde 0)
    private int choose(String title, List<String> labels) {
        StringBuilder menu = new StringBuilder(title + ":");
        for (int i = 0; i < labels.size(); i++) {
            menu.append("  [").append(i + 1).append("] ")
                    .append(labels.get(i));
        }
        out.println(menu);
        return ask(labels.size()) - 1;
    }
```

ui/ConsoleMenu.java (6/7)

```
    private int ask(int max) {
        while (true) {
            String line = input.next();
            try {
                int n = Integer.parseInt(line);
                if (n >= 1 && n <= max) {
                    return n;
                }
            } catch (NumberFormatException e) {
                // no es un número: se avisa más abajo
            }
            out.println("Escribe un número entre 1 y " + max + ".");
        }
    }

    private String describe(List<Fighter> group) {
        return group.stream()
                .map(f -> f.name() + " " + f.hp() + "/"
                        + f.stats().maxHp())
                .collect(Collectors.joining(", "));
    }

    private String describeItems(Bag inventory) {
        if (inventory.items().isEmpty()) {
            return "(ninguno)";
        }
        return inventory.items().stream()
                .map(Item::name)
                .collect(Collectors.joining(", "));
    }
```

ui/ConsoleMenu.java (7/7)

```
    public boolean explore(GameMap map, TileTypes types) {
        while (true) {
            mapView.show(map);
            out.println("[w a s d] mover, [t] casillas, [q] salir");
            String key = input.next();
            switch (key) {
                case "w" -> step(map, 0, -1);
                case "a" -> step(map, -1, 0);
                case "s" -> step(map, 0, 1);
                case "d" -> step(map, 1, 0);
                case "t" -> showTiles(map, types);
                case "q" -> { return false; }
                default -> out.println("Tecla no válida.");
            }
            if (map.onStairs()) {
                out.println("Bajas por la escalera.");
                return true;
            }
        }
    }

    private void step(GameMap map, int dx, int dy) {
        if (!map.move(dx, dy)) {
            out.println("No puedes pasar.");
        }
    }

    private void showTiles(GameMap map, TileTypes types) {
        TileType corner = map.tileAt(0, 0);
        TileType opposite = map.tileAt(map.width() - 1, map.height() - 1);
        out.println("Casillas: " + map.width() * map.height()
                + ", TileType creados: " + types.createdCount());
        out.println("Esquinas opuestas, mismo objeto: "
                + (corner == opposite));
    }
}
```

Y `Main`, entero, con la oleada de la planta 3 del capítulo 13, las vistas entregadas a la fachada y las dos opciones nuevas:

app/Main.java (1/6)

```
package emberdeep.app;

import emberdeep.engine.GameEngine;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Skill.Heal;
import emberdeep.engine.character.Skill.Strike;
import emberdeep.engine.character.Skill.Stun;
import emberdeep.engine.character.Skill.Taunt;
import emberdeep.engine.character.Skill.Venom;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.DamageType;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.flow.GamePhase.Exploring;
import emberdeep.engine.flow.GamePhase.GameOver;
import emberdeep.engine.world.Dungeon;
import emberdeep.engine.world.DungeonBuilder;
import emberdeep.engine.world.Floor;
import emberdeep.engine.world.biome.BiomeFactory;
import emberdeep.engine.world.biome.CavernBiome;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.engine.world.map.MapData;
import emberdeep.engine.world.map.MapParser;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.engine.world.wave.WaveNode.EnemyGroup;
import emberdeep.engine.world.wave.WaveNode.Wave;
import emberdeep.ui.ConsoleMenu;
import emberdeep.ui.input.ConsoleInput;
import emberdeep.ui.input.InputSource;
import emberdeep.ui.input.ScriptInput;
import emberdeep.ui.output.AnsiOutput;
import emberdeep.ui.output.Output;
import emberdeep.ui.output.PlainOutput;
import emberdeep.ui.view.HudView;
import emberdeep.ui.view.LogView;
import emberdeep.ui.view.MapView;
import java.io.IOException;
import java.nio.file.Path;
import java.util.Arrays;
import java.util.List;
import java.util.Random;
import java.util.Scanner;
```

app/Main.java (2/6)

```
public class Main {
    private static final Skill POWER_STRIKE =
            new Strike("Golpe poderoso", 12, DamageType.PHYSICAL);
    private static final Skill FLARE =
            new Strike("Llamarada", 10, DamageType.FIRE);
    private static final Skill HEAL = new Heal("Luz curativa", 8);
    private static final Skill TAUNT = new Taunt("Grito de guerra");
    private static final Skill VENOM = new Venom("Daga envenenada", 3, 2);
    private static final Skill STUN = new Stun("Golpe aturdidor", 1);

    private final GameConfig config;

    private Main(GameConfig config) {
        this.config = config;
    }

    public static void main(String[] args) throws IOException {
        GameConfig config = new GameConfig(42L, 3, new FlatFormula());
        new Main(config).play(args);
    }
```

app/Main.java (3/6)

```
    private void play(String[] args) throws IOException {
        String script = scriptOf(args);
        String[] options = optionsOf(args);
        int floorNumber = chooseFloor(options);
        InputSource input = script == null
                ? new ConsoleInput(new Scanner(System.in), System.out)
                : new ScriptInput(Path.of(script), System.out);
        Output output = chooseOutput(args, script != null);
        ConsoleMenu menu = new ConsoleMenu(input, System.out,
                new MapView(output));
        Random random = new Random(config.seed());

        BiomeFactory crypt = new CryptBiome(random);
        BiomeFactory cavern = new CavernBiome(random);
        List<EnemyKind> cryptKinds = List.of(
                EnemyKind.SKELETON, EnemyKind.ASH_RAT, EnemyKind.ZOMBIE);
        List<EnemyKind> cavernKinds = List.of(
                EnemyKind.GOBLIN, EnemyKind.BAT, EnemyKind.SPIDER);
        // la planta 3 es una oleada con una subola (cap. 13)
        List<EnemyKind> deepKinds = new Wave(List.of(
                new EnemyGroup(EnemyKind.SKELETON, 1),
                new Wave(List.of(
                        new EnemyGroup(EnemyKind.NECROMANCER, 1)))))
                .enemies();
        Dungeon dungeon = new DungeonBuilder()
                .floor().biome(crypt).enemies(cryptKinds)
                        .map(MapData.floor(1)).done()
                .floor().biome(cavern).enemies(cavernKinds)
                        .map(MapData.floor(2)).done()
                .floor().biome(crypt).enemies(deepKinds)
                        .map(MapData.floor(3)).done()
                .build();
```

app/Main.java (4/6)

```
        if (options.length > 1 && options[1].equals("map")) {
            explore(menu, dungeon, floorNumber);
            return;
        }
        DamageFormula formula = chooseFormula(menu, options,
                script != null);
        GameEngine engine = new GameEngine(dungeon, heroes(), formula,
                random);
        engine.subscribe(new LogView(output));
        engine.subscribe(new HudView(output));
        Path loadFile = pathOf(args, "--load=");
        if (loadFile == null) {
            engine.start(floorNumber);
        } else {
            engine.load(loadFile);
            engine.descend();       // se sigue por la planta siguiente
        }
        if (engine.phase() instanceof GameOver) {
            System.out.println("Has recorrido toda la mazmorra.");
            return;
        }
        engine.enterCombat();
        menu.run(engine);
        Path saveFile = pathOf(args, "--save=");
        if (saveFile != null && engine.phase() instanceof Exploring) {
            engine.save(saveFile);
            System.out.println("Partida guardada en " + saveFile + ".");
        }
    }

    private void explore(ConsoleMenu menu, Dungeon dungeon,
                         int firstFloor) {
        TileTypes types = new TileTypes();
        MapParser parser = new MapParser(types);
        int number = firstFloor;
        while (true) {
            Floor floor = dungeon.floors().get(number - 1);
            boolean descended =
                    menu.explore(parser.parse(floor.map()), types);
            if (!descended || number == dungeon.floors().size()) {
                return;
            }
            number++;
        }
    }
```

app/Main.java (5/6)

```
    // El guion es el argumento que acaba en .txt
    private static String scriptOf(String[] args) {
        return Arrays.stream(args)
                .filter(arg -> arg.endsWith(".txt")
                        && !arg.startsWith("--"))
                .findFirst().orElse(null);
    }

    // --save=ruta o --load=ruta: el fichero de partida (null si no está)
    private static Path pathOf(String[] args, String prefix) {
        return Arrays.stream(args)
                .filter(arg -> arg.startsWith(prefix))
                .map(arg -> Path.of(arg.substring(prefix.length())))
                .findFirst().orElse(null);
    }

    private static String[] optionsOf(String[] args) {
        return Arrays.stream(args)
                .filter(arg -> !arg.endsWith(".txt")
                        && !arg.startsWith("--"))
                .toArray(String[]::new);
    }

    private DamageFormula chooseFormula(ConsoleMenu menu, String[] options,
                                        boolean scripted) {
        boolean useDefault = scripted
                || (options.length > 1 && options[1].equals("default"));
        if (useDefault) {
            return config.defaultFormula();
        }
        return menu.askFormula() == 1
                ? new FlatFormula() : new ScaledFormula();
    }

    private int chooseFloor(String[] options) {
        int floor = options.length > 0 ? Integer.parseInt(options[0]) : 1;
        if (floor < 1 || floor > config.floors()) {
            throw new IllegalArgumentException(
                    "La planta debe estar entre 1 y " + config.floors());
        }
        return floor;
    }
```

app/Main.java (6/6)

```
    // --plain o --ansi mandan; si no, plano con guion y color a mano
    private static Output chooseOutput(String[] args, boolean scripted) {
        List<String> all = Arrays.asList(args);
        boolean plain = all.contains("--plain");
        boolean ansi = all.contains("--ansi");
        if (plain && ansi) {
            throw new IllegalArgumentException(
                    "Elige --plain o --ansi, no las dos");
        }
        if (ansi || (!plain && !scripted)) {
            return new AnsiOutput(System.out);
        }
        return new PlainOutput(System.out);
    }

    private List<Hero> heroes() {
        return List.of(
                new HeroBuilder().name("Aria").stats(new Stats(30, 8, 3))
                        .skill(POWER_STRIKE).skill(FLARE)
                        .skill(HEAL).build(),
                new HeroBuilder().name("Borin").stats(new Stats(40, 6, 5))
                        .skill(POWER_STRIKE).skill(FLARE)
                        .skill(TAUNT).skill(STUN).build(),
                new HeroBuilder().name("Cael").stats(new Stats(24, 9, 2))
                        .skill(POWER_STRIKE).skill(FLARE)
                        .skill(VENOM).build());
    }
}
```

Por qué funciona (parte 2). `ConsoleMenu` ya no importa nada de la sesión ni de los comandos concretos: pide todo a `engine`. Los textos son los mismos, así que la salida del guion de demostración no cambia (`diff` vacío). `Main` monta los datos (mazmorra, héroes, fórmula, `Random`) y se los da al motor una vez; con `--load`, `load` deja la partida en `Exploring` en la planta guardada y `descend` la lleva a la siguiente con la vida y la mochila del fichero. La comprobación `phase() instanceof GameOver` cubre cargar una partida guardada en la última planta. Con `--save`, el guardado se hace después del combate y solo si la fase es `Exploring`: tras una derrota la fase es `GameOver(false)` y no se guarda nada. `scriptOf` ignora lo que empieza por `--` para que `--save=saves/partida.txt` no se confunda con un guion.

Parte 3: guardar y continuar. Con `--save=saves/partida.txt` y una cripta ganada, el fichero queda con `floor=1`, la posición de entrada de la planta 1 (`x=1`, `y=1`), tres vidas y tres objetos. Con `--load=saves/partida.txt` empieza la caverna, y la línea «Héroes:» del primer turno repite las vidas guardadas. Después borra los programas temporales y los ficheros de prueba y añade `/saves/` a `.gitignore`. `./gradlew test` sigue con las tres pruebas de `SnapshotTest`: la fachada no tocó el Memento.

Errores típicos:

- Dividir con `split("=")` sin límite en `parse`: un valor con `=` se corta en silencio.
- No recortar la instantánea a los héroes en `save`: el fichero llevaría también los enemigos como si fueran héroes y, al cargar, `read` les pondría vida de héroe.
- Reemplazar la partida antes de terminar de leer en `load`: un fichero roto dejaría el

motor a medias. `build` devuelve una partida nueva y solo después se asigna.

- Suscribir el `AchievementTracker` antes que `LogView`: el logro saldría antes que el golpe que lo provoca.
- Olvidar `Files.createDirectories`: la primera vez, `saves/` no existe y `writeString` lanza `NoSuchFileException`.
- Guardar fuera de `Exploring`: el fichero no recoge escudo, estados ni enemigos y la partida cargada no sería la misma.
- Meter reglas del juego en `GameEngine` (por ejemplo, calcular daño): la fachada coordina, no decide.
- Dejar `Check15a`, `Check15b` o la carpeta `saves/` de pruebas.

Estado final de Emberdeep. Es el de la estructura de la sección «En el proyecto»: `GameEngine` en `engine/`, `SaveFile` en `engine/save/`, `ConsoleMenu` y `Main` hablando solo con la fachada, y la carpeta `saves/` fuera de git. Con esto termina el proyecto: diecisiete patrones, cada uno nacido de un dolor concreto.

APÉNDICE A

Guía visual de consulta

Este apéndice no enseña nada nuevo: reúne en pocas páginas lo que ya has visto, para consultarlo cuando tengas un código delante y una duda. Está pensado para abrirlo por la tabla que necesites: qué hace cada uno de los diecisiete patrones, cómo se confunden entre sí, cómo pasar de un olor de código a un patrón, qué mirar en una revisión de código y dónde vive cada patrón en Emberdeep. Si quieres un catálogo con más patrones y diagramas para completar la consulta, el de Refactoring.Guru es de acceso libre [24]; la referencia original es el libro de Gamma y colaboradores [9]. Y para repasar el lenguaje (interfaces, lambdas, clases selladas, *pattern matching*), un manual de referencia al día con Java 21 es [14].

A.1 Los diecisiete patrones de un vistazo

El libro ordena los patrones por el problema del juego (crear, ampliar, comunicar, cambiar de estado), no por familia. El catálogo original los agrupa en tres familias según lo que resuelven [9]: los de *creación* se ocupan de cómo se construyen los objetos, los de *estructura* de cómo se componen clases y objetos en piezas mayores, y los de *comportamiento* de cómo se reparten las responsabilidades y se comunican los objetos. La tabla siguiente sigue el orden del libro y añade la familia para que puedas reconciliar ambas formas de ordenar. Los nombres son los de las tablas de piezas de cada capítulo.

Patrón	**Familia GoF**	**Propósito en una línea**	**Cap.**
Strategy	Comportamiento	Elegir entre varias versiones de un algoritmo sin condicionales en quien lo usa	2
Factory Method	Creación	Dejar que una subclase decida qué clase concreta se crea	3
Abstract Factory	Creación	Crear familias de objetos que deben ir juntos sin nombrar clases concretas	3
Builder	Creación	Construir un objeto complejo por pasos y validarlo al final	4
Singleton	Creación	Garantizar una sola instancia con un punto de acceso (y saber cuándo no usarlo)	4
Prototype	Creación	Obtener un objeto nuevo copiando uno ya montado	5
Flyweight	Estructura	Compartir un único objeto inmutable en lugar de miles de copias iguales	5

Patrón	Familia GoF	Propósito en una línea	Cap.
Decorator	Estructura	Añadir comportamiento envolviendo un objeto, sin subclases por combinación	6
Chain of Responsibility	Comportamiento	Pasar una petición por una fila de manejadores, cada uno puede resolverla o cortarla	6
Command	Comportamiento	Convertir una acción en un objeto para ponerla en cola, deshacerla o repetirla	7
Memento	Comportamiento	Guardar y restaurar el estado de un objeto sin exponer su interior	8
Observer	Comportamiento	Avisar a quien escuche de lo que ocurre, sin que el emisor lo conozca	9
Bridge	Estructura	Separar dos ejes que cambian por separado para no multiplicar clases	10
Mediator	Comportamiento	Centralizar en un objeto cómo se coordinan varios objetos que no se conocen	11
State	Comportamiento	Cambiar el comportamiento de un objeto según su estado, sin banderas sueltas	12
Composite	Estructura	Tratar igual un objeto y un grupo de objetos mediante un árbol	13
Facade	Estructura	Ofrecer una entrada simple a un subsistema con muchas piezas	14

La segunda tabla dice cómo se usa cada patrón y con cuál se confunde. «Cómo se usa» es el esquema de una línea; la forma completa está en la tabla de piezas de su capítulo.

Patrón	Piezas centrales	Cómo se usa	Se confunde con
Strategy	`EnemyAi`, `DamageFormula`	El contexto guarda la estrategia y le delega la decisión; el cliente la elige al montar	Bridge, State, Decorator
Factory Method	`EnemySpawner`, `create`	El creador llama a un método abstracto `create`; cada subclase devuelve su producto	Abstract Factory, Prototype
Abstract Factory	`BiomeFactory`	El cliente recibe la fábrica y pide enemigos y objetos sin saber de qué bioma son	Factory Method
Builder	`HeroBuilder`, `build()`	Se encadenan pasos que devuelven `this` y `build()` valida y crea	Constructor de `record`
Singleton	`GlobalSettings`, `getInstance()`	Un acceso estático a la única instancia; en Emberdeep se evita pasando `GameConfig`	Variable global
Prototype	`EnemyTemplate`, `copy()`	Se copia la plantilla (copia profunda de lo mutable) y se retoca la copia	Factory, Flyweight
Flyweight	`TileType`, `TileTypes`	Una fábrica con `computeIfAbsent` devuelve siempre el mismo objeto inmutable	Prototype, Factory
Decorator	`StatSource`, `StatDecorator`	Cada capa guarda lo envuelto, lo llama y suma lo suyo	Chain, Strategy
Chain of Responsibility	`DamageHandler`, `DamagePipeline`	Cada eslabón decide si resuelve o llama a `next`; la tubería monta la cadena	Decorator, Strategy
Command	`CombatCommand`, `CommandHistory`	Cada acción es un objeto con `execute` y `undo`; el historial los apila	Memento
Memento	`Snapshot`, `SnapshotStore`	El originador crea un `record` inmutable con su estado y el cuidador lo guarda	Command
Observer	`GameListener`, `EventBus`	Los oyentes se suscriben al bus; el emisor publica eventos y no sabe quién escucha	Mediator
Bridge	`View`, `Output`	La abstracción guarda una implementación; cada eje tiene su jerarquía	Strategy
Mediator	`TurnMediator`, `CombatArena`	Los colegas piden al mediador, que decide y actúa sobre los demás	Observer, Facade
State	`Status`, `next()`	El estado es un tipo sellado; un `switch` decide la transición	Strategy
Composite	`InventoryNode`, `Bag`	Hoja y compuesto comparten tipo; las operaciones recorren el árbol con `switch`	Visitor
Facade	`GameEngine`	Los clientes hablan solo con la fachada; el subsistema sigue accesible	Mediator, capas

Para saber cuánta confianza darle al catálogo completo en el código de hoy conviene recordar dos cosas. La primera: con lambdas, records y tipos sellados, varios patrones (Strategy, Command, partes de Factory Method) se escriben hoy con mucho menos código, y el valor del catálogo es sobre todo el vocabulario y el modo de pensar los compromisos [16]. La segunda: existe literatura empírica sobre si los patrones ayudan de verdad, por ejemplo una revisión sistemática de la evidencia [27] y un estudio sobre su efecto en la estabilidad de las clases de programas Java [1]; este libro no se apoya en sus resultados, solo te las deja como lecturas para ir más allá.

A.2 Cuatro contrastes para no confundirlos

Estas son las cuatro parejas (o tríos) que más se confunden. Cada tabla resume la sección de contraste de su capítulo; los diagramas y el criterio completo están allí.

A.2.1 Decorator, Chain of Responsibility y Strategy (cap. 6)

	Decorator	**Chain of Responsibility**	**Strategy**
Idea	Envolver para añadir	Pasar y poder cortar	Elegir una de varias
¿Llama al siguiente?	Siempre	A veces: puede cortar	No hay siguiente
Pregunta clave	¿Qué cifra sale tras sumar todo?	¿Quién se hace cargo?	¿Cuál de las versiones uso?
En Emberdeep	`WeaponBonus` sobre `BaseStats`	`DodgeHandler` antes que `ShieldHandler`	`FlatFormula` o `ScaledFormula`

A.2.2 Prototype, Factory y Flyweight (cap. 5)

	Prototype	**Factory**	**Flyweight**
Pregunta	¿Cómo obtengo otro como este?	¿Cómo construyo uno de este tipo desde cero?	¿Cómo evito mil copias de lo mismo?
Objeto nuevo	Sí, una copia por llamada	Sí, uno por llamada	Casi nunca: el mismo
`==` entre dos resultados	`false`	`false`	`true` (mismo tipo)
En Emberdeep	`EnemyTemplate.copy()`	`CryptBiome.enemy(kind)`	`TileTypes.get(glyph)`

A.2.3 Bridge y Strategy (cap. 10)

	Bridge	**Strategy**
Problema	Dos ejes que cambian por separado y se multiplicarían por herencia	Un algoritmo con varias versiones entre las que se elige
Jerarquías	Dos, y crecen las dos	Una; el contexto no tiene subclases
Cómo se nota	Sin puente serían $N \times M$ clases	Hay un `if` o `switch` sobre «cuál usar»
En Emberdeep	`HudView` y `AnsiOutput`	`Fighter` y `LowestHpAi`

A.2.4 Mediator y Observer (cap. 11)

	Observer	Mediator
Dirección	Uno anuncia y muchos escuchan	Muchos piden y uno decide
Respuesta	Ninguna: el emisor no espera nada	Puede devolver algo (`healAlly` devuelve lo curado)
Altas y bajas	Dinámicas: `subscribe` y `unsubscribe`	Fijas: la arena une a los colegas
En Emberdeep	`EventBus` y `LogView`	`TurnMediator` y `Fighter`

A.3 Del olor al patrón: la checklist

Los patrones se llegan desde un olor, no al revés [17]. Esta tabla va del síntoma que ves en el código al patrón candidato y a la alternativa más simple que debes haber descartado antes (las tres preguntas del capítulo 14).

Olor	Patrón candidato	Alternativa más simple	Cap.
`if` o `switch` por tipo para elegir un algoritmo	Strategy	Una lambda o un método	2
`new` de clases concretas repartido por el código	Factory Method	Un método estático que cree	3
Objetos de una familia que deben ir juntos	Abstract Factory	Dos métodos en una sola clase	3
Constructor con muchos parámetros	Builder	`record` con constructor compacto	4
Una configuración que todos leen	Singleton, o mejor pasarla	Un `record` pasado por constructor	4
Montar la misma plantilla muchas veces	Prototype	Un método que construya	5
Miles de objetos idénticos e inmutables	Flyweight	Un `enum` o una constante	5
Una subclase por cada combinación de extras	Decorator	Una lista de modificadores	6
Un método con una serie de pasos que pueden cortar	Chain of Responsibility	Los mismos pasos en un solo método legible	6
Acciones que hay que deshacer, repetir o poner en cola	Command	Una lambda (si no hay deshacer)	7
Deshacer cuando hay cien cambios posibles	Memento	Command, si cada cambio es invertible	8
Una parte avisa a otras que no debería conocer	Observer	Una llamada directa si es una sola	9
Una clase por cada combinación de dos ejes	Bridge	Un parámetro o un `enum`	10
Objetos que se referencian entre sí	Mediator	Reducir las dependencias	11
Banderas que permiten combinaciones imposibles	State	Banderas booleanas con cuidado	12
Partes que contienen partes del mismo tipo	Composite	Una lista plana	13
Un cliente que conoce una docena de clases	Facade	Que cada cliente monte lo suyo	14

A.3.1 Señales de sobrediseño

- Una interfaz con una sola implementación y ninguna segunda a la vista.
- Una clase cuyos métodos solo reenvían a otra sin añadir nada.
- Un patrón nombrado en el código (`Factory`, `Manager`, `Helper`) sin que se pueda señalar el olor que resuelve.
- Un Singleton que se lee desde todas partes y obliga a las pruebas a reiniciarlo.
- Un mediador o una fachada que acumula lógica de negocio y se convierte en un «dios».
- Una jerarquía de dos niveles que podría ser una lambda o un `switch` sobre una interfaz sellada.

A.3.2 Checklist de revisión de código

Cuando revises un cambio que introduce o toca un patrón, comprueba estos puntos. Las tres primeras preguntas son las de la sección «Elegir y defender el patrón».

- ❑ ¿Se puede señalar la línea del código de hoy que duele (el olor)?
- ❑ ¿Se ha escrito de verdad la alternativa más simple, y duele más que el patrón?
- ❑ ¿Se sabe qué coste añade, quién lo paga y cuántos sitios hay que tocar cuando crezca?
- ❑ ¿Los nombres dicen el rol (`Handler`, `Command`, `Listener`) y coinciden con la tabla de piezas del patrón?
- ❑ ¿Cada pieza tiene una sola responsabilidad? ¿Hay alguna clase que solo reenvía?
- ❑ ¿Las partes mutables compartidas se copian (copia profunda en Prototype y Memento)?
- ❑ ¿Hay un `switch` exhaustivo sobre tipos sellados, sin `default` que oculte casos nuevos?
- ❑ ¿Se puede probar la pieza sin montar todo el sistema (mediador, fachada, observador)?
- ❑ ¿Los observadores se dan de baja y el bus tolera que se den de baja mientras se notifica?
- ❑ ¿Hay una prueba que demuestre el comportamiento del patrón (por ejemplo, `==` en Flyweight o `undo` tras `execute`)?

A.4 Mirada atrás: dónde vive cada patrón en Emberdeep

El proyecto terminó con diecisiete patrones y todos tienen un hogar. La estructura final de carpetas (completa en el capítulo 14) con el patrón que vive en cada una:

Dónde vive cada patrón

```
emberdeep/src/main/java/emberdeep/
|-- app/                 Singleton (alternativa: GameConfig), composition root
|-- engine/
|   |-- GameEngine       Facade
|   |-- character/       Builder (HeroBuilder), Prototype (EnemyTemplate)
|   |-- stats/           Decorator (StatSource, StatDecorator)
|   |-- combat/
|   |   |-- ai/          Strategy (EnemyAi)
|   |   |-- damage/      Strategy (DamageFormula)
|   |   |-- defense/     Chain of Responsibility (DamageHandler)
|   |   |-- command/     Command (CombatCommand, CommandHistory)
|   |   |-- arena/       Mediator (TurnMediator, CombatArena)
|   |   `-- status/      State (Status)
|   |-- item/            Composite (InventoryNode, Bag)
|   |-- event/           Observer (EventBus, GameListener)
|   |-- flow/            State (GamePhase, GameFlow)
|   |-- save/            Memento (Snapshot, SnapshotStore)
|   |-- session/         GameSession (originador del Memento)
|   `-- world/
|       |-- DungeonBuilder.java   Builder
|       |-- biome/       Abstract Factory (BiomeFactory)
|       |-- map/         Flyweight (TileTypes)
|       `-- wave/        Composite (WaveNode)
`-- ui/
    |-- view/            Bridge (View, abstracción)
    `-- output/          Bridge (Output, implementador)
```

Y una tabla para ir de cada patrón a las clases concretas del proyecto:

Patrón	Clases en Emberdeep	Cap.
Strategy	`EnemyAi` con `FirstAliveAi`, `LowestHpAi` y `RandomTargetAi`; `DamageFormula` con `FlatFormula` y `ScaledFormula`	2
Factory Method	`EnemySpawner`, con `CryptSpawner` y `CavernSpawner`; los retira el capítulo 3 cuando llega `BiomeFactory`	3
Abstract Factory	`BiomeFactory`, `CryptBiome` y `CavernBiome`, que usa `GameSession`	3
Builder	`HeroBuilder`, `DungeonBuilder` y `FloorBuilder`	4
Singleton	No vive en el proyecto: `GlobalSettings` es solo el ejemplo; `GameConfig` es la alternativa creada en `Main`	4
Prototype	`EnemyTemplate.copy()` y `TemplateRegistry`, usados por `CryptBiome` y `CavernBiome`	5
Flyweight	`TileType` y `TileTypes`, usados por `GameMap` y `MapParser`	5
Decorator	`StatSource`, `BaseStats`, `WeaponBonus`, `ArmorBonus` y `TempBuff`	6
Chain of Responsibility	`DamageHandler` y `DamagePipeline`, con `DodgeHandler`, `ShieldHandler`, `ArmorHandler` y `ResistanceHandler`	6
Command	`CombatCommand`, `Attack`, `Defend`, `UseItem`, `UseSkill` y `CommandHistory`	7
Memento	`Snapshot`, `FighterState` y `SnapshotStore`; `GameSession.createSnapshot` y `restore`	8
Observer	`EventBus`, `GameListener` y `GameEvent`, con `LogView`, `HudView` y `AchievementTracker` como oyentes	9
Bridge	`View` con `HudView`, `MapView` y `LogView`; `Output` con `AnsiOutput`, `PlainOutput` y `BufferOutput`	10
Mediator	`TurnMediator` y `CombatArena`, con `Fighter` como colega	11
State	`Status` (`Normal`, `Poisoned`, `Stunned`); `GamePhase` y `GameFlow`	12
Composite	`InventoryNode`, `ItemSlot` y `Bag`; `WaveNode`, `EnemyGroup` y `Wave`	13
Facade	`GameEngine`, con `SaveFile` para el guardado	14

Un último comentario sobre lo que muestra la tabla. Ningún patrón apareció porque «tocaba»: cada uno entró cuando un olor concreto de Emberdeep lo pidió, y uno de ellos (Singleton) terminó sustituido por una alternativa más simple. Esa es la habilidad que queda: reconocer el olor, escribir la alternativa más simple y decidir con el coste a la vista.

Referencias

[1] Apostolos Ampatzoglou et al. «The Effect of GoF Design Patterns on Stability: A Case Study». En: *IEEE Transactions on Software Engineering* 41.8 (2015), págs. 781-802. DOI: 10.1109/TSE.2015.2414917.

[2] Kent Beck y Ward Cunningham. «Using Pattern Languages for Object-Oriented Programs». En: *OOPSLA-87 Workshop on Specification and Design for Object-Oriented Programming*. Technical Report CR-87-43. Sep. de 1987. URL: https://c2.com/doc/oopsla87.html (visitado 24-10-2026).

[3] Gavin Bierman. *JEP 440: Record Patterns*. Final en Java 21. OpenJDK. 2023. URL: https://openjdk.org/jeps/440 (visitado 24-10-2026).

[4] Gavin Bierman. *JEP 441: Pattern Matching for switch*. Final en Java 21. OpenJDK. 2023. URL: https://openjdk.org/jeps/441 (visitado 24-10-2026).

[5] Joshua Bloch. *Effective Java*. 3.ª ed. Addison-Wesley, 2018. ISBN: 978-0-13-468599-1.

[6] Martin Fowler. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3.ª ed. Copyright 2004 según InformIT. Addison-Wesley, 2003. ISBN: 978-0-321-19368-1.

[7] Eric Freeman y Elisabeth Robson. *Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software*. 2.ª ed. O'Reilly Media, 2020. ISBN: 978-1-4920-7800-5.

[8] Erich Gamma et al. «Design Patterns: Abstraction and Reuse of Object-Oriented Design». En: *ECOOP'93 — Object-Oriented Programming*. Lecture Notes in Computer Science. Springer, 1993, págs. 406-431. DOI: 10.1007/3-540-47910-4_21.

[9] Erich Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Publicado el 31-10-1994; copyright 1995 según InformIT. Addison-Wesley, 1994. ISBN: 978-0-201-63361-0.

[10] Brian Goetz. *Data Oriented Programming in Java*. InfoQ. Jun. de 2022. URL: https://www.infoq.com/articles/data-oriented-programming-java/ (visitado 24-10-2026).

[11] Gradle. *Building Java & JVM projects*. 2026. URL: https://docs.gradle.org/current/userguide/building_java_projects.html (visitado 24-10-2026).

[12] Gradle. *Gradle User Manual, versión 9.8.2*. 2026. URL: https://docs.gradle.org/current/userguide/userguide.html (visitado 24-10-2026).

[13] Jan Hannemann y Gregor Kiczales. «Design Pattern Implementation in Java and AspectJ». En: *Proceedings of the 17th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '02)*. Seattle, WA, 2002, págs. 161-173. DOI: 10.1145/582419.582436.

[14] Cay S. Horstmann. *Core Java, Volume I: Fundamentals*. 13.ª ed. Oracle Press, 2024. ISBN: 978-0-13-532837-8.

[15] JUnit. *JUnit User Guide, versión 6.1.3*. 2026. URL: https://docs.junit.org/current/user-guide/ (visitado 24-10-2026).

[16] Andreas Jürgensen. *Are Classic Design Patterns Still Valid With Modern Java?* JAVAPRO International. Jul. de 2026. URL: https://javapro.io/2026/07/23/are-classic-design-patterns-still-valid-with-modern-java/ (visitado 24-10-2026).

[17] Joshua Kerievsky. *Refactoring to Patterns*. Copyright 2005 según InformIT. Addison-Wesley, 2004. ISBN: 978-0-321-21335-8.

[18] Peter Norvig. *Design Patterns in Dynamic Languages*. Presentado el 5-5-1996; publicado en la web el 17-3-1998. 1996. URL: https://norvig.com/design-patterns/ (visitado 24-10-2026).

[19] Robert Nystrom. *Game Programming Patterns*. Versión web gratuita en gameprogrammingpatterns.com. Edición del autor, 2014. ISBN: 978-0-9905829-0-8.

[20] Object Management Group. *Unified Modeling Language (UML), versión 2.5.1*. formal/17-12-05. Dic. de 2017. URL: https://www.omg.org/spec/UML/2.5.1 (visitado 24-10-2026).

[21] Oracle. *Java Platform, Standard Edition: Java Language Updates, Release 21*. Documento F79543-03. Ene. de 2025. URL: https://docs.oracle.com/en/java/javase/21/language/ (visitado 24-10-2026).

[22] Oracle. *Package java.util.function (Java SE 21 & JDK 21 API)*. 2026. URL: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/function/package-summary.html (visitado 24-10-2026).

[23] Nicolai Parlog. *Java 21 Pattern Matching Tutorial*. nipafx.dev. Sep. de 2023. URL: https://nipafx.dev/java-21-pattern-matching/ (visitado 24-10-2026).

[24] Refactoring.Guru. *The Catalog of Design Patterns*. 2026. URL: https://refactoring.guru/design-patterns/catalog (visitado 24-10-2026).

[25] Till Tantau y The PGF/TikZ Team. *PGF and TikZ, versión 3.1.12*. CTAN. 2026. URL: https://ctan.org/pkg/pgf (visitado 24-10-2026).

[26] Raoul-Gabriel Urma, Mario Fusco y Alan Mycroft. *Modern Java in Action: Lambdas, Streams, Functional and Reactive Programming*. Manning, 2018. ISBN: 978-1-61729-356-6.

[27] Cheng Zhang y David Budgen. «What Do We Know about the Effectiveness of Software Design Patterns?» En: *IEEE Transactions on Software Engineering* 38.5 (2012), págs. 1213-1231. DOI: 10.1109/TSE.2011.79.